EegFun.jl

EEG/ERP analysis in Julia

Raw data readers

Biosemi, BrainVision Analyser, EDF, BIDS, EEGLAB, Fieldtrip ...

Data Processing

Filtering, referencing, artifact detection/ correction, ICA, epoching ...

Analysis

Epochs, time-frequency analysis, ERP measurements, statistical testing

Visualization

Interactive plots, topographic maps, and publication-quality figures with Makie.jl

Contents

I Home 1

0.1 Quick Start 1

0.2 Documentation 2

0.3 Getting Help 2

II Getting Started 3

1 Getting Started with EegFun.jl 4

1.1 Installing Julia 4

1.2 The Julia REPL 4

1.3 IDE Workflows 4

1.4 Installing EegFun 4

1.5 First Steps 5

1.6 EegFun Philosophy 5

1.7 Supported Data Formats 6

1.8 Next Steps and Resources 6

2 IDE Workflows 8

2.1 IDE Workflows for Julia 8

3 Why Julia? 10

3.1 Why Julia for EEG Research? 10

4 Julia Basics 12

4.1 Julia Basics 12

III Tutorials 21

5 Overview 22

5.1 Tutorials 22

6 Data Structures 23

6.1 EEG Data Structures 23

7 Manual Preprocessing 28

7.1 Manual Preprocessing 28

8 Epoch Selection 36

8.1 Epoch Selection 36

9 Artifact Handling 43

9.1 Automated Artifact Handling 43

10 Layouts and Neighbors 49

10.1 Layouts and Neighbors 49

11 Import 53

11.1 Data Import 53

11.2 BioSemi Import 55
11.3 BrainVision Import 58
11.4 EDF Import 61
11.5 FIF Import 62
11.6 XDF Import 63
11.7 EEGLAB Import 64
11.8 FieldTrip Import 66
11.9 Data 67
11.10 Data Persistence (JLD2) 70
11.11 BIDS Export 75
12 Data 80
12.1 Data 80
12.2 Data Access 82
12.3 Selection Helpers 85
13 Preprocessing 89
13.1 Filter 89
13.2 Rereference 92
13.3 Resample 95
13.4 Baseline 97
13.5 Mirror 100
13.6 Triggers 103
13.7 Epoch Extraction 106
13.8 Channel Operations 108
13.9 Channel Metrics 111
13.10 Channel Repair 114
13.11 Channel Summary 117
13.12 ICA 118
13.13 Layouts & Neighbors 127
13.14 Analysis Settings 130
14 Artifacts 133
14.1 Artifacts 133
14.2 Artifact Detection 139
15 ERP Analysis 146
15.1 ERP Measurements 146
15.2 Global Field Power 150
15.3 Grand Average 152
15.4 Jackknife Average 155
15.5 Lateralised Readiness Potential 157

15.6 Realignment 159
15.7 Condition Operations 162
16 Time-Frequency 165
16.1 TF Analysis 165
16.2 TF Morlet 169
16.3 TF Multitaper 173
16.4 TF STFT 176
16.5 TF Operations 179
17 Statistics 184
17.1 Statistics 184
17.2 TF Statistics 188
17.3 Decoding 192
17.4 RSA 196
18 Plotting 199
18.1 Plot Artifacts 199
18.2 Plot Channel Spectrum 202
18.3 Plot Channel Summary 205
18.4 Plot Correlation Heatmap 208
18.5 Plot Epochs 212
18.6 Plot ERP 215
18.7 Plot ERP Image 222
18.8 Plot ERP Measurements 225
18.9 Plot Filter 227
18.10 Plot GFP 230
18.11 Plot ICA 232
18.12 Plot Joint Probability 235
18.13 Plot Layout 238
18.14 Plot Topography 242
18.15 Plot Triggers 247
18.16 Saving Figures 250
19 Interactive / GUI 254
19.1 Plot Databrowser 254
19.2 Plot ERP Filter GUI 259
19.3 Plot ERP Measurement GUI 260
20 Specialized Plotting 263
20.1 Plot Decoding 263
20.2 Plot RSA 264
20.3 Plot Statistics 267

20.4 Plot Time-Frequency 272

IV How-to Guides **275**

21 Filter Data 276

21.1 How to Filter EEG Data 276

22 Selection Patterns 278

22.1 Selection Patterns 278

23 TOML Format 283

23.1 TOML Format 283

24 Batch Processing 286

24.1 Automated Batch Processing 286

25 Pipeline Templates 291

25.1 Pipeline Templates 291

26 Preprocessing Walkthrough 294

26.1 Preprocessing Workflow 294

27 Plot GUI 308

27.1 Plot GUI 308

28 Benchmark 314

28.1 Benchmark 314

29 Worked Examples 317

29.1 Visual Attention (Posner Cueing) 317

29.2 N170 (Face/Body) 353

V Teaching Demos **376**

30 Signal Processing 377

30.1 Sampling 377

30.2 Composition 379

30.3 Dot Product 381

30.4 Convolution 383

30.5 Time-Frequency 385

31 ICA 390

31.1 CLT 390

31.2 Mixture 391

31.3 Separation 392

31.4 Geometry 394

31.5 Sphering 396

31.6 Optimization 397

32 Machine Learning 399

32.1 Decoding 399

33 ERP 402

33.1 Simulate ERP 402
34 Psychophysiology (EOG / EMG / ECG) 405
34.1 EOG Dipole & Derivation 405
34.2 Surface EMG Cascade 405
34.3 ECG Einthoven's Triangle 406
34.4 ECG Heart Rate Variability 406
VI Cheatsheet **408**
35 Cheatsheet 409
35.1 Data Import 409
35.2 Data Persistence (JLD2) 409
35.3 Preprocessing 409
35.4 ICA 410
35.5 Epoching 411
35.6 ERP Operations 411
35.7 ERP Measurements & Export 412
35.8 Time-Frequency Analysis 412
35.9 Statistics 413
35.10 Plotting 413
35.11 Selection Helpers 414
VII Reference **415**
36 Overview 416
36.1 API Reference 416
37 Analysis 417
37.1 Analysis Functions 417
38 Plotting 484
38.1 Plotting & Interactive Demos 484
39 Types 552
39.1 Core Types 552

Part I

Home

0.1 Quick Start

Install EegFun.jl from the Julia REPL:

```
using Pkg
Pkg.add("EegFun")
```

Note for plotting: To display plots or use the interactive GUIs, you must also install a Makie backend of your choice (e.g., `Pkg.add("GLMakie")` for interactive plots, or `Pkg.add("CairoMakie")` for static plots).

Read and preprocess EEG data:

```
using EegFun
using GLMakie # or CairoMakie for static plots

# Load EEG data and layout
dat = EegFun.read_raw_data("your_data.bdf")
layout = EegFun.read_layout("biosemi64.csv")

# Create Eegfun datatype
dat = EegFun.create_eegfun_data(dat, layout)

# Basic preprocessing
EegFun.highpass_filter!(dat, 1)      # High-pass filter at 1 Hz
EegFun.rereference!(dat, :avg)       # Average reference
EegFun.is_extreme_value!(dat, 100)   # Mark extreme values

# Continuous data Browser
EegFun.plot_databrowser(dat)

# Create epochs and compute ERPs
epoch_cfg = [
  EegFun.EpochCondition(name = "Cond1", trigger_sequences = [[1]]),
  EegFun.EpochCondition(name = "Cond2", trigger_sequences = [[2]])
  ]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 0.8))

# Epoch data browser
EegFun.plot_databrowser(epochs[1]) # Cond1

# Plot epochs
```

```
EegFun.plot_epochs(epochs, layout = :grid)
EegFun.plot_epochs(epochs, layout = :topo)
EegFun.plot_epochs(epochs, layout = :single, channel_selection =
EegFun.channels(:PO7))
EegFun.plot_epochs(epochs, layout = :single, channel_selection =
EegFun.channels([:PO7, :PO8]))

# ERPs
erps = EegFun.average_epochs(epochs)

EegFun.plot_erp(erps, layout = :grid)
EegFun.plot_erp(erps, layout = :topo)
EegFun.plot_topography(erps, interval_selection = (0.1, 0.2)) # between 100 and
200 ms
```

0.2 Documentation

Getting Started Tutorial

Section	Description
Tutorials	Step-by-step guides
How-to Guides	Code examples and demonstrations
Background	Core concepts and architecture
API Reference	Complete function and type documentation

0.3 Getting Help

- Report bugs on GitHub Issues

Part II

Getting Started

Chapter 1

Getting Started with EegFun.jl

1.1 Installing Julia

EegFun.jl requires Julia 1.13.

The recommended way to install and manage Julia versions is with **juliaup**. Alternatively, download an installer directly from the Julia Downloads page.

1.2 The Julia REPL

Julia is an interactive language built around a Read-Eval-Print Loop (REPL). The REPL provides different modes accessed by special keys:

Key	Mode	Purpose
(default)	Julia mode	Execute Julia code
]	Package mode	Install and manage packages
?	Help mode	Access inline documentation
;	Shell mode	Run shell commands

Press `Backspace` to return to Julia mode from any other mode.

1.3 IDE Workflows

Most users pair the REPL with an editor that adds syntax highlighting and — most importantly — lets you send code directly into the live Julia REPL session. See **IDE Workflows** for a comparison of VS Code/VSCodium, Positron, JetBrains, and the Neovim + Iron.nvim terminal workflow.

1.4 Installing EegFun

You can install EegFun.jl through the standard Julia package manager.

Standard Installation

From the Julia REPL, enter Pkg mode by pressing `]` and run:

```
add EegFun
```

Or using `Pkg` in the code:

```
using Pkg
Pkg.add("EegFun")
```

Note for plotting: `EegFun.jl` uses Makie.jl for visualizations. To display plots or use the interactive GUIs, you must also install a Makie backend of your choice:

- `add GLMakie` (Recommended for interactive GUIs and the Data Browser)
- `add CairoMakie` (Recommended for static, publication-quality plots or headless servers)

Development Version (via GitHub)

To install the latest development version directly from GitHub:

```
using Pkg
Pkg.add(url="https://github.com/igmmgi/EegFun.jl")
```

Then load the package in any Julia REPL session:

```
using EegFun
```

1.5 First Steps

All EegFun functions are called with the `EegFun.` prefix. The fastest way to get started is to use our built-in tutorial datasets:

```
using EegFun
using GLMakie # Required for interactive plotting

# 1. Automatically download the tutorial datasets
data_dir = EegFun.download_eegfun_datasets()
file_path = joinpath(data_dir, "example1.bdf")

# 2. Load the raw recording
dat = EegFun.read_raw_data(file_path)

# 3. Attach an electrode layout
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

# 4. Browse the raw data interactively
EegFun.plot_databrowser(dat)
```

> [!NOTE] When you are ready to load your own data, simply replace `file_path` with the string path to your own file on disk (e.g., `EegFun.read_raw_data("C:/EEG/participant1.bdf")`).

> [!TIP] Functions ending with `!` (e.g. `filter!`) mutate their input in-place. Functions without `!` return a new copy. Functions starting with _ are internal helpers and not part of the public API.

1.6 EegFun Philosophy

EegFun.jl is designed with ease-of-use as a core principle, making it accessible even for those without extensive programming experience. However, whilst EegFun provides many interactive GUIs for data visualization and exploration, it is not a full GUI application — the package emphasises a code-based workflow that is intended to be simple and intuitive.

The package offers a mix of high-level and lower-level functions, including complete analysis pipelines that take you from raw data through to ERP analyses, while still allowing fine-grained control when needed. In practice, a complete EEG analysis can be accomplished with little to zero traditional "coding" — simply typing commands in the Julia REPL and/or combining them into small, readable/runnable scripts.

1.7 Supported Data Formats

Format	Extension(s)	Notes
BioSemi	`.bdf`	via BiosemiDataFormat.jl
BrainVision	`.vhdr` / `.eeg` / `.vmrk`	via BrainVisionDataFormat.jl
European Data Format	`.edf`	EDF / EDF+ via `EuropeanDataFormat.jl`
Neuromag / MNE	`.fif`	Raw, Epochs, and Evoked via `FunctionalImageFormat.jl`
Lab Streaming Layer	`.xdf` / `.xdf.gz`	via `ExtensibleDataFormat.jl`
EEGLAB	`.set` / `.fdt`	Continuous, epoched, and ICA metadata
FieldTrip	`.mat`	Continuous, epoched, and time locked ERPs
BIDS	directory	`EegFun.read_bids` and `EegFun.export_bids`

> [!NOTE] Additional file format support is planned for future releases. The format-specific packages are automatically installed as dependencies of EegFun.jl.

1.8 Next Steps and Resources

Resource	Link
EegFun.jl GitHub	github.com/igmmgi/EegFun.jl
Manual Preprocessing tutorial	Manual Preprocessing
All how-to guides	How-to Guides
Julia learning resources	julialang.org/learning

Julia cheat sheet	cheatsheet.juliadocs.org
MATLAB–Python–Julia cheat sheet	cheatsheets.quantecon.org
Makie.jl (plotting)	docs.makie.org
DataFrames.jl	dataframes.juliadata.org

Chapter 2

IDE Workflows

2.1 IDE Workflows for Julia

> [!TIP] Working in a good editor significantly improves the Julia experience. Choose whichever option fits your existing workflow best.

While the Julia REPL is powerful on its own, most users pair it with an IDE that provides syntax highlighting, inline documentation, and — most importantly — tight REPL integration so that code can be sent from a script to a live Julia REPL session with a single keypress combination.

VS Code/VSCodium

Visual Studio Code is probably the most widely used editor for Julia development. VSCodium is a fully open-source build of VS Code without Microsoft telemetry, and is compatible with the same Julia extensions.

Install the **Julia extension** (`julialang.language-julia`) to unlock:

- Integrated REPL — send lines or selections with `Shift+Enter`
- Inline evaluation results and variable display
- Debugger, profiler, and plot pane
- Workspace variable explorer and data viewer

The extension can be installed from within the IDE via the **Extensions panel** (`Ctrl+Shift+X`) by searching for `Julia` .

If Julia's shell mode (`;` in the REPL) doesn't work inside VS Code, installing Git Bash and adding the following to your VS Code `settings.json` (open via `Ctrl+Shift+P` → *Preferences: Open User Settings (JSON)*) has been reported to fix the issue. This was found via trial and error and may not be the only/best solution:

```
{
    "terminal.integrated.env.windows": {
        "SHELL": "C:\\Program Files\\Git\\bin\\bash.exe",
        "Path": "${env:Path};C:\\Program Files\\Git\\usr\\bin"
    },
    "terminal.integrated.profiles.windows": {
        "Git Bash": {
            "path": "C:\\Program Files\\Git\\bin\\bash.exe"
        }
    },
    "terminal.integrated.defaultProfile.windows": "Git Bash"
}
```

Positron is an open-source fork of VS Code by Posit (makers of RStudio) with first-class support for both R and Python. It ships a dedicated *Connections*, *Variables*, and *Plots* pane out of the box, making it feel closer to RStudio for researchers coming from an R background.

There is also a dedicated **positron-julia extension** available on Open VSX, installable via the Extensions panel. The `positron-julia` extension is in very early development and may be unstable.

JetBrains IDEs

JetBrains offer Julia support via the **Flexible Julia plugin**. It should work in any JetBrains IDE (e.g., IntelliJ IDEA, PyCharm) and provides:

- Full language server support (completion, go-to-definition, refactoring)
- Integrated Julia REPL within the IDE
- Package manager UI

> [!NOTE] JetBrains IDEs are commercial software, though free community editions and academic licences are available.

Neovim + Iron.nvim

For more terminal-focused workflows, Neovim combined with **Iron.nvim** provides a lightweight but highly capable environment:

- `iron.nvim` opens a Julia REPL split and lets you send lines, visual selections, or entire files with configurable keymaps
- Pair with **nvim-lspconfig** + the Julia language server (`julials`) for completion and diagnostics

Chapter 3

Why Julia?

3.1 Why Julia for EEG Research?

Most EEG analysis today happens in MATLAB (EEGLAB, FieldTrip, ERPLAB) or Python (MNE-Python). These are well-established packages with many years of development, large user communities, and broad feature sets — they will typically offer more functionality than EegFun.jl. Each ecosystem has clear strengths, and researchers should choose the tool that best fits their needs. This page explains why EegFun.jl is built on Julia and where the trade-offs lie.

Strengths

Fully open source — language and libraries. Julia itself and its entire package ecosystem are open source. MATLAB-based toolboxes such as EEGLAB, FieldTrip, and ERPLAB are open source, but they still require a proprietary MATLAB licence to run. GNU Octave provides a free alternative, but compatibility with these toolboxes is not 100%, and performance and graphics support can differ significantly.

One language from prototype to production. Scientific computing often hits a "two-language problem": researchers prototype in a high-level language and then rewrite performance-critical code in C or Fortran. In practice this means R code calling Rcpp/C++, Python relying on C extensions (NumPy, SciPy, MNE's compiled backends), and MATLAB using MEX files. Julia is designed to avoid this split — user-level code compiles to efficient native code, so there is no need to drop into a second language for speed. In EegFun.jl, there is no hidden C or Fortran layer; the analysis code you read is the code that runs.

Performance without boilerplate. Julia's JIT compiler generates machine code specialised to the types you actually use. For numerically intensive EEG workflows — filtering, time- frequency decomposition, permutation statistics — this often matches C/Fortran performance.

Multiple dispatch for clean extensibility. Julia's type system and multiple dispatch make it straightforward to extend existing functions to new data types. In EegFun.jl this means, for example, the same `lowpass_filter!` function works on continuous data, epoched data, and ERP averages.

Built-in package manager and reproducibility. Every Julia project records its exact dependency versions in `Manifest.toml` . Collaborators can reproduce your environment with a single `Pkg.instantiate()` call, without conda environments or Docker containers.

Trade-offs

Time to first execution (TTFX). Julia compiles code the first time it runs in a session, which means the first call to a function can take noticeably longer than subsequent calls. Python users familiar with Numba will recognise this pattern — both compile code to fast machine code on first use — but in Julia, the compilation applies to the entire language rather than individual decorated functions. This "compilation latency" has improved substantially in recent Julia versions (1.9+ introduced package

images, and 1.12 brings further improvements), but it remains more noticeable than starting a Python or R session. In practice, most users start a Julia session once and keep it running.

Smaller ecosystem. The Julia ecosystem for EEG is young compared to MATLAB and Python. Packages like EEGLAB (2004), FieldTrip (2011), and MNE-Python (2014) have over a decade of development, thousands of users, and extensive functionality that EegFun.jl does not yet match.

Tooling is still maturing. IDE support, debugging, and profiling tools exist (VS Code with the Julia extension is quite capable), but they are not as mature or polished as the equivalent MATLAB IDE or established Python tool-chains (PyCharm, Jupyter ecosystem). The debugger, in particular, can be slow on large codebases.

Package loading time. Large packages with many dependencies take time to load (`using EegFun` is not instant). The very first `using EegFun` after installation is particularly slow because Julia precompiles (partly) the package and all of its dependencies — this is a one-off cost. Subsequent `using EegFun` calls in new sessions are considerably faster, but still slower than `import mne` in Python.

> [!TIP] If you are coming from MATLAB or Python, the MATLAB-Python-Julia cheat sheet is a helpful reference for translating familiar idioms.

Chapter 4

Julia Basics

4.1 Julia Basics

EegFun.jl is designed so that you do not need to be a Julia programmer to use it. In practice, most EEG analysis with EegFun involves calling functions and interacting with plots — there is very little traditional "programming" required, and in that sense the choice of language is almost secondary. That said, a basic understanding of Julia always helps, especially when you want to write small scripts, customise a pipeline, or understand an error message. This page covers the essentials.

The REPL as a Calculator

The simplest way to start with Julia is to type expressions directly into the REPL. Julia evaluates them immediately and prints the result:

```
julia> 2 + 3
5

julia> 7 * 8.5
59.5

julia> 2^10
1024

julia> sqrt(144)
12.0
```

Standard mathematical operators work as expected: `+` , `-` , `*` , `/` , `^` (exponentiation), and `%` (remainder). Parentheses control evaluation order:

```
julia> (3 + 4) * 2
14
```

Variables

Assign values to names with `=` . Julia is dynamically typed, so you do not need to declare a type:

```
julia> sample_rate = 512
512

julia> duration = 2.5
2.5
```

```
julia> n_samples = sample_rate * duration
1280.0
```

Variable names can include Unicode characters, which is useful for writing code that reads like textbook notation:

```
julia> μ = 0.0
0.0

julia> σ = 1.5
1.5

julia> Δt = 1 / 512
0.001953125
```

> [!TIP] In the REPL or VS Code, type the LaTeX name and press `Tab` to insert Unicode symbols: `\mu` ⟶ μ , `\sigma` ⟶ σ , `\Delta` ⟶ Δ .

To see all variables defined in your current session (similar to MATLAB's `whos`), use `varinfo()` :

```
julia> varinfo()
  name                    size summary
  ––––––––––––––––– ––––––––––– ———————
  Δt                    8 bytes Float64
  μ                     8 bytes Float64
  σ                     8 bytes Float64
  duration              8 bytes Float64
  n_samples             8 bytes Float64
  sample_rate           8 bytes Int64
```

Types

Every value in Julia has a type. You can inspect it with `typeof` :

```
julia> typeof(42)
Int64

julia> typeof(3.14)
Float64

julia> typeof("hello")
String

julia> typeof(true)
Bool
```

Vectors and Arrays

Square brackets create vectors (1D arrays):

```
julia> voltages = [1.2, -0.5, 3.1, 0.8]
4-element Vector{Float64}:
 1.2
 -0.5
  3.1
  0.8

julia> voltages[1]       # Julia uses 1-based indexing
1.2

julia> voltages[end]     # 'end' refers to the last element
0.8

julia> voltages[2:3]     # slicing
2-element Vector{Float64}:
 -0.5
  3.1
```

Create ranges and regular sequences:

```
julia> 1:5               # a range from 1 to 5
1:5

julia> collect(1:5)      # materialise into a vector
5-element Vector{Int64}:
 1
 2
 3
 4
 5

julia> 0:0.5:2           # start:step:stop
0.0:0.5:2.0
```

Matrices (2D arrays, channels × time points in EEG) use semicolons or spaces:

```
julia> data = [1.0 2.0 3.0; 4.0 5.0 6.0]
2×3 Matrix{Float64}:
 1.0  2.0  3.0
 4.0  5.0  6.0

julia> size(data)
(2, 3)
```

```
julia> data[1, :]        # first row (e.g. first channel)
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

Control Flow

if-else

```
if p_value < 0.05
    println("Significant")
else
    println("Not significant")
end
```

for loops

```
channels = ["Fz", "Cz", "Pz"]
for ch in channels
    println("Processing channel: $ch")
end
```

Loops in Julia are fast — unlike MATLAB or Python, there is no performance penalty for writing explicit loops instead of vectorised code.

Comprehensions

Concise syntax for building arrays from loops:

```
julia> [i^2 for i in 1:5]
5-element Vector{Int64}:
  1
  4
  9
 16
 25
```

Strings

Strings use double quotes. String interpolation uses `$` :

```
julia> participant = "P01"
"P01"
```

```
julia> condition = "congruent"
"congruent"

julia> filename = "data/$(participant)_$(condition).bdf"
"data/P01_congruent.bdf"
```

Single characters use single quotes (`'a'`). Use `string()` or * to concatenate strings:

```
julia> "hello" * " " * "world"
"hello world"
```

Dictionaries

Key–value storage, useful for mapping trigger codes to condition labels:

```
julia> triggers = Dict(1 => "standard", 2 => "deviant", 3 => "novel")
Dict{Int64, String} with 3 entries:
  2 => "deviant"
  3 => "novel"
  1 => "standard"

julia> triggers[2]
"deviant"
```

Tuples

Tuples are fixed-size, immutable collections. You have already seen them — `size()` returns a tuple:

```
julia> dims = (64, 512)
(64, 512)

julia> dims[1]
64

julia> typeof(dims)
Tuple{Int64, Int64}
```

Unlike vectors, tuples cannot be modified after creation. They are useful for grouping a small number of related values, such as a time window:

```
julia> baseline_window = (-0.2, 0.0)
(-0.2, 0.0)
```

Named Tuples

Named tuples add field names to each element, giving you lightweight, self-documenting containers without defining a custom type:

```
julia> participant = (id = "P01", age = 25, group = "control")
(id = "P01", age = 25, group = "control")

julia> participant.age
25

julia> participant[:group]
"control"
```

Named tuples are immutable like regular tuples. They are commonly used for passing groups of options or returning multiple values from a function.

Functions

Standard Form

Functions are defined with the `function ... end` block:

```
julia> function add(a, b)
           return a + b
       end
add (generic function with 1 method)

julia> add(3, 5)
8
```

Short Form

For simple one-liners, the same function can be written more concisely:

```
julia> add(a, b) = a + b
add (generic function with 1 method)

julia> add(3, 5)
8
```

Anonymous Functions

Short throwaway functions use the `->` syntax. These are common with `map` and `filter`:

```
julia> map(x -> x^2, [1, 2, 3])
3-element Vector{Int64}:
 1
 4
 9
```

Mutating Functions

By convention, functions that modify their input end with `!`:

```
EegFun.lowpass_filter!(data, 30.0)   # modifies data in-place
EegFun.lowpass_filter(data, 30.0)    # returns a new copy, data is unchanged
```

This is a naming convention, not enforced by the language, but Julia packages (including EegFun.jl) follow it consistently.

Broadcasting (The Dot Syntax)

Adding a dot (`.`) before an operator or after a function name applies it element-wise to each value in an array:

```
julia> a = [1, 2, 3, 4]

julia> sin.(a)            # apply sin to each element
4-element Vector{Float64}:
 0.8414709848078965
 0.9092974268256817
 0.1411200080598672
 -0.7568024953079282
```

Using Packages

Julia packages are loaded with `using`. For example, the `Statistics` standard library provides `mean`:

```
julia> using Statistics

julia> data = [1.2, -0.5, 3.1, 0.8, 2.4]

julia> mean(data)
1.4

julia> std(data)
1.3266499161421599
```

The first `using` in a session triggers compilation (see Why Julia? — Trade-offs). Subsequent calls in the same session are instant.

Multiple Dispatch

In Julia, the same function name can have different **methods** depending on the types of its arguments. Julia automatically picks the right method:

```
julia> describe(x::Int) = println("$x is an integer")
julia> describe(x::Float64) = println("$x is a floating-point number")
julia> describe(x::String) = println("$x is a string")
```

```
julia> describe(42)
42 is an integer

julia> describe(3.14)
3.14 is a floating-point number

julia> describe("hello")
hello is a string
```

You can check how many methods a function has with `methods` :

```
julia> methods(describe)
# 3 methods for generic function "describe"
```

Types in Julia are organised in a hierarchy that you can explore with `supertype` and `subtypes` . Both `Int64` and `Float64` are subtypes of `Number` , so you can write a single method that accepts any number:

```
julia> double(x::Number) = x * 2

julia> double(3)
6

julia> double(3.14)
6.28
```

This is how EegFun.jl works internally — many functions have different methods for different data types (continuous, epoched, averaged), so you use the same function name regardless of what you pass in. See **Data Structures** for EegFun.jl's own type hierarchy.

Getting Help

The REPL help mode (press `?`) gives you inline documentation:

```
help?> sqrt
search: sqrt isqrt

  sqrt(x)

  Return √x.
```

For EegFun functions:

```
help?> EegFun.lowpass_filter!
```

Next Steps

Resource	Link

Julia Manual	docs.julialang.org
Julia learning resources	julialang.org/learning
MATLAB–Python–Julia cheat sheet	cheatsheets.quantecon.org
Think Julia (free book)	benlauwens.github.io/ThinkJulia.jl

Part III

Tutorials

Chapter 5

Overview

5.1 Tutorials

Step-by-step guides for learning EegFun.jl, from installation through to advanced analysis techniques.

Getting Started

- Getting Started - Installation, Julia basics, and your first EEG analysis

Core Workflows

- Manual Preprocessing - Step-by-step preprocessing with full control over each stage
- Epoch Selection - Extracting and selecting epochs using trigger sequences
- Artifact Handling - Detecting and managing artifacts in your data

Data & Configuration

- TOML Format - Quick reference for the TOML configuration format
- Layouts and Neighbors - Working with electrode layouts and neighbor structures
- Selection Patterns - Channel, time, and epoch selection patterns

Scaling Up

- Batch Processing - Processing multiple participants efficiently

Visualization

- Plot GUI - Exploring data interactively with the plot GUI

Chapter 6

Data Structures

6.1 EEG Data Structures

Understanding EegFun.jl's core data structures.

Julia Types

If you're coming from Python or MATLAB, Julia's type system works a little differently. The full details are in the Julia documentation, but the key idea for everyday EegFun.jl use is **multiple dispatch**.

This means a function can have the same name but do different things depending on the *type* of data passed to it.

In Julia function signatures, the `::` notation means *"this argument must be of this type"*:

```
EegFun.plot_erp(erp::EegFun.ErpData)          # plots a single ERP
EegFun.plot_erp(erp::Vector{EegFun.ErpData}) # plots multiple ERPs overlaid
```

Here `erp::EegFun.ErpData` means "an argument called `erp` of type `EegFun.ErpData` ". Julia automatically selects the right function version based on what you pass in — you never need to call a different function name.

More examples from EegFun.jl:

```
# Same function, different input types → different behaviour
EegFun.plot_topography(erp::EegFun.ErpData, ...)          # plots ERP topography
EegFun.plot_topography(tf::EegFun.TimeFreqData, ...)      # plots time-frequency
topography


EegFun.average_epochs(dat::EegFun.EpochData)              # averages a single
condition
EegFun.average_epochs(dat::Vector{EegFun.EpochData})      # averages multiple
conditions


EegFun.extract_epochs(dat::EegFun.ContinuousData, ...)    # extracts from
continuous recording
```

Both are called `plot_erp` — Julia picks the right function version automatically based on what you pass in. You never need to call a different function name.

If you're ever unsure what type a variable is, use `typeof()` :

```
typeof(my_data)   # e.g. EegFun.ContinuousData
```

This is useful when reading error messages — if Julia says *"no method matching `plot_erp(::ContinuousData)`* "it means you passed the wrong type, and `typeof()` can help you check what you actually have.

Abstract vs Concrete Types

Julia also distinguishes between **concrete** and **abstract** types:

- **Concrete types** are the actual data structures you create and work with — for example `ContinuousData` , `ErpData` , or `EpochData` . These hold real data.
- **Abstract types** are categories or groupings — you can never create one directly, but they let functions accept a *family* of related types. For example, `EegData` is abstract; it simply says "any EEG data type".

This matters in practice because a function written for `EegData` will work on *any* concrete EEG type — `ContinuousData` , `ErpData` , etc. — without needing a separate version for each one.

Type Hierarchy

```
EegFunData (abstract)
└── EegData (abstract) - All EEG data types
    ├── SingleDataFrameEeg (abstract) - Data in a single DataFrame
    │   ├── ContinuousData - Raw continuous recordings
    │   ├── ErpData - Averaged event-related potentials
    │   ├── TimeFreqData - Time-frequency decomposition
    │   └── SpectrumData - Power spectrum
    └── MultiDataFrameEeg (abstract) - Data across multiple DataFrames
        ├── EpochData - Trial-segmented data
        └── TimeFreqEpochData - TF with individual trials
```

See the Types Reference for full field-by-field documentation of each type.

ContinuousData

Raw, unsegmented EEG recordings.

Field	Type	Description
`file`	`String`	Source filename
`data`	`DataFrame`	Continuous data (time × channels)
`layout`	`Layout`	Electrode positions and neighbors
`sample_rate`	`Int64`	Sampling frequency in Hz
`analysis_info`	`AnalysisInfo`	Preprocessing metadata

EpochData

Trial-segmented data for event-related analysis.

Field	Type	Description
`file`	`String`	Source filename
`condition`	`Int64`	Condition number
`condition_name`	`String`	Human-readable condition name
`data`	`Vector{DataFrame}`	One DataFrame per epoch
`layout`	`Layout`	Electrode positions
`sample_rate`	`Int64`	Sampling frequency in Hz
`analysis_info`	`AnalysisInfo`	Preprocessing metadata

ErpData

Averaged event-related potentials.

Field	Type	Description
`file`	`String`	Source filename
`condition`	`Int64`	Condition number
`condition_name`	`String`	Human-readable condition name
`data`	`DataFrame`	Averaged ERP (time × channels)
`layout`	`Layout`	Electrode positions
`sample_rate`	`Int64`	Sampling frequency in Hz
`analysis_info`	`AnalysisInfo`	Preprocessing metadata
`n_epochs`	`Int64`	Number of epochs averaged

TimeFreqData

Time-frequency decomposition results.

Field	Type	Description
`file`	`String`	Source filename
`condition`	`Int64`	Condition number
`condition_name`	`String`	Human-readable condition name

`data_power`	`DataFrame`	Power values (time × freq × channels)
`data_phase`	`DataFrame`	Phase values in radians
`layout`	`Layout`	Electrode positions
`sample_rate`	`Int64`	Original sampling frequency
`method`	`Symbol`	Analysis method (`:wavelet` , `:multitaper` , etc.)
`baseline`	`BaselineInfo`	Baseline correction info (if applied)
`analysis_info`	`AnalysisInfo`	Preprocessing metadata

SpectrumData

Power spectral density (frequency domain only).

Field	Type	Description
`file`	`String`	Source filename
`condition`	`Int64`	Condition number
`condition_name`	`String`	Human-readable condition name
`data`	`DataFrame`	PSD values (freq × channels)
`layout`	`Layout`	Electrode positions
`sample_rate`	`Int64`	Original sampling frequency
`method`	`Symbol`	Analysis method (`:welch` , etc.)
`analysis_info`	`AnalysisInfo`	Preprocessing metadata

Layout

Electrode spatial information and neighbor relationships.

Field	Type	Description
`data`	`DataFrame`	Positions with columns: `label` , `x` , `y` , `z`
`neighbors`	`Union{Nothing, OrderedDict{Symbol,Neighbors}}`	Neighbor info per electrode

`criterion`	`Union{Nothing, Float64}`	Distance criterion used to compute neighbors (mm)
`criterion_type`	`Union{Nothing, Symbol}`	Type of criterion (e.g. `:xy` , `:xyz`)

Data Flow

```
Raw EEG File (.bdf, .vhdr, .set, .mat)
    ↓ read_raw_data()
ContinuousData
    ↓ extract_epochs()
EpochData
    ↓ average_epochs()
ErpData
```

See Also

- Types Reference
- Getting started tutorial

Chapter 7

Manual Preprocessing

7.1 Manual Preprocessing

While `EegFun.jl` provides the automated `preprocess()` pipeline for batch processing, you may want to manually control each step of preprocessing for exploration, customization, or teaching purposes. This tutorial explains the **why** and **when** behind each preprocessing step.

> [!TIP] See the preprocessing_workflow demo for a complete working example. The automated equivalent of this workflow is documented in Batch Processing.

The Preprocessing Philosophy

EEG preprocessing is a balancing act between **removing noise** and **preserving signal**. Every step involves trade-offs:

- Too lenient artifact rejection ⟶ contaminate your ERPs
- Over-reliance on ICA ⟶ remove brain activity that correlates with artifacts

The goal is to make data **cleaner** without making it **artificial**.

Step-by-Step Rationale

1. Load Data and Configure Layout

```
dat = EegFun.create_eegfun_data(raw_data, layout)
EegFun.polar_to_cartesian_xy!(layout)
EegFun.polar_to_cartesian_xyz!(layout)
EegFun.get_neighbors_xy!(layout, 0.4)
```

Why?

- **2D coordinates**: Required for topographic plots
- **3D coordinates**: Required for advanced interpolation (spherical spline)
- **Neighbors**: Pre-calculated for efficient channel repair

When to Customize?

- Adjust neighbor distance based on electrode density — the distance is in normalized units where 1.0 = the scalp equator. Think of it as a fraction of the head radius: 0.4 means “neighbors within 40% of the head radius”
- Higher density arrays (e.g., 128 channels) may need smaller radius (0.3)

---

2. Mark Epoch Windows

```
EegFun.mark_epoch_intervals!(dat, epoch_cfg, [-0.2, 1.0])
```

Why?

Creates a boolean column `:epoch_interval` in your data, allowing you to:

- Exclude inter-trial intervals from artifact detection
- Focus EOG correlation analysis on epochs

When to Skip?

If you want to analyze the **entire recording** (e.g., resting-state data), skip this step.

---

3. Rereference BEFORE Filtering

```
EegFun.rereference!(dat, :avg)
```

Why This Order?

Rereferencing **before** filtering prevents reference channel artifacts from spreading during filtering. This is the `pipeline_v1` standard.

Common References:

- `:avg` - Average reference (recommended for dense arrays)
- `:Cz` - Central reference
- `[:M1, :M2]` - Linked mastoids (for auditory/language studies)

When to Customize?

- **Sparse arrays** (<32 channels): Use a specific channel like `:Cz`
- **Auditory ERPs**: Consider mastoid reference
- **High-density arrays** (≥64 channels): Average reference is ideal

---

4. Apply Initial Filters

```
EegFun.highpass_filter!(dat, 0.1)  # Remove DC drift (Biosemi Data Format)
EegFun.lowpass_filter!(dat, 30.0)  # Remove higher-frequency noise
```

Why Two Filters?

High-pass (0.1 Hz):

- Removes slow voltage drifts
- Removes DC offsets
- **Critical** for stable baseline

Low-pass (30 Hz):

- Removes muscle artifacts (>30 Hz)
- Removes line noise (50/60 Hz) + harmonics

When to Customize?

Study Type	High-pass	Low-pass	Rationale
Standard ERPs	0.1 Hz	30 Hz	Preserve slow components (P300, etc.)
ICA preprocessing	1.0 Hz	40 Hz	ICA performs better with <1 Hz removed
Gamma analysis	0.5 Hz	100 Hz	Need high-frequency content
Infants/Children	0.3 Hz	20 Hz	More conservative filtering

> [!WARNING] **Avoid** high-pass filtering above 0.5 Hz for standard ERP analyses — this can distort slow components like the P300.

---

5. Calculate EOG Channels

```
# vEOG = mean(Fp1, Fp2) - mean(IO1, IO2)
EegFun.channel_difference!(dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG
)
```

Why?

- **vEOG**: Detects vertical eye movements (blinks)
- **hEOG**: Detects horizontal eye movements (saccades)

These channels help identify:
1. Bad channels that correlate with eye movements
2. ICA components that capture eye artifacts

When to Customize?a

If your montage doesn't have dedicated EOG electrodes, use:

- **vEOG**: `Fp1 - IO1` or just `Fp1` alone
- **hEOG**: `F7 - F8` or `F9 - F10`

---

6. Extract Initial "Original" Epochs

```
epochs_uncorrected = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))
```

Why?

Saving "original" epochs allows you to:

- Compare cleaned vs. uncleaned data
- Quantify artifact rejection effectiveness
- Verify that cleaning didn't distort the signal

When to Skip?

If disk space is limited or you're confident in your pipeline.

---

7. Detect Artifacts in Continuous Data

```
# Extreme artifacts (exclude from ICA)
EegFun.is_extreme_value!(dat, 250.0, channel_out = :is_extreme_value_100)

# Moderate artifacts (for epoch rejection)
EegFun.is_extreme_value!(dat, 75.0, channel_out = :is_artifact_value_75)
```

Why Two Thresholds?

250 µV (strict): Excludes extreme sections from ICA training

- Prevents ICA from wasting components modeling saturated samples
- ICA works best on relatively clean data

75 µV (lenient): Used later for epoch-level rejection

- More permissive during continuous processing
- Final judgment happens after ICA cleaning

---

8. Identify Bad Channels

```
summary = EegFun.channel_summary(dat, sample_selection =
EegFun.samples(:epoch_interval))
cjp = EegFun.channel_joint_probability(dat, sample_selection =
EegFun.samples(:epoch_interval))
bad_channels = EegFun.identify_bad_channels(summary, cjp)
```

Why?

Bad channels have:

- **High variance** (noisy)

- **Low variance** (dead/broken channels)
- **Extreme kurtosis** (spiky)
- **Low joint probability** (statistically deviant)

Why Repair Early?

Repairing bad channels **before** ICA prevents them from:

- Dominating ICA components
- Reducing effective rank of the data
- Creating spurious brain-artifact correlations

---

9. Run ICA

```
dat_ica = EegFun.subset(dat, channel_selection =
EegFun.channels_not(bad_channels))
EegFun.highpass_filter!(dat_ica, 1.0)  # Stricter for ICA
ica_result = EegFun.run_ica(dat_ica, sample_selection =
EegFun.samples_not(:is_extreme_value_100))
```

Why 1 Hz High-pass for ICA?

ICA assumes **stationarity**. Slow drifts (<1 Hz) violate this assumption and reduce ICA quality.

Why Remove Bad Channels First?

Bad channels:

- Reduce effective data rank
- Waste ICA components
- Create misleading component topographies

When to Skip ICA?

- Very clean data (no eye movements)
- Low channel count (<32 channels)

---

10. Repair Bad Channels

```
EegFun.repair_channels!(dat, bad_channels, method = :neighbor_interpolation)
```

Why After ICA?

Prevents bad channel noise from:

- Contaminating ICA decomposition
- Creating spurious component correlations

Methods:

- `:neighbor_interpolation` - Fast, works in 2D
- `:spherical_spline` - Higher quality, requires 3D coordinates

---

11. Recalculate EOG After ICA

```
EegFun.channel_difference!(dat, ...)  # Recalculate vEOG and hEOG
```

Why?

ICA component removal **changes** the underlying channel data. EOG channels must be recalculated to reflect the cleaned data.

---

12. Extract Epochs from Cleaned Data

```
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))
```

Now extracting from **ICA-cleaned**, **channel-repaired** continuous data.

---

13. Baseline Correction

```
EegFun.baseline!(epochs)  # Defaults to entire epoch
```

Why Before Rejection?

Baseline correction removes DC offsets that could bias artifact detection statistics.

When to Customize?

```
EegFun.baseline!(epochs, (-0.2, 0.0))  # Only pre-stimulus interval
```

Use pre-stimulus baseline for:

- Standard ERP analyses
- Comparing conditions with different baselines

14-17. Two-Stage Epoch Rejection

```
# Stage 1: Detect bad epochs
rejection_info_step1 = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion = 75.0)

# Stage 2: Repair bad channels within epochs
EegFun.repair_artifacts!(epochs, rejection_info_step1)

# Stage 3: Re-detect after repair
```

```
rejection_info_step2 = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion =
75.0)

# Stage 4: Reject remaining bad epochs
epochs_clean = EegFun.reject_epochs(epochs, rejection_info_step2)
```

Why Two Stages?

Many "bad" epochs have only 1-2 bad channels. Repairing those channels **rescues** trials that would otherwise be rejected.

Typical outcomes:

- 20% of epochs initially flagged
- 50% of those rescued via repair
- Only 10% actually rejected

Common Pitfalls

1. Over-Filtering

Problem: High-pass filtering at 1 Hz for ERP analysis distorts slow components. **Solution**: Use 0.1 Hz for ERPs, 1 Hz only for ICA preprocessing.

2. Skipping ICA

Problem: Eye blinks contaminate frontal electrodes. **Solution**: Always run ICA unless you have very clean data.

3. Not Recalculating EOG

Problem: EOG channels contain removed ICA components. **Solution**: Always recalculate EOG after ICA.

4. Rejecting Too Many Trials

Problem: Less than X (component dependent) trials per condition → unstable ERPs. **Solution**: Lower artifact thresholds or improve recording quality.

Decision Flow

```
graph TD
    A[Load Data] --> B{High-density array?}
    B -->|Yes >64 ch| C[Average Reference]
    B -->|No <32 ch| D[Cz Reference]
    C --> E[Filter: 0.1 Hz highpass, 30 Hz lowpass]
    D --> E
    E --> F{Clean data?}
```

```
    F -->|Yes| G[Skip ICA]
    F -->|No| H[Run ICA with 1 Hz highpass]
    H --> I[Remove artifact components]
    I --> J[Repair bad channels]
    G --> K[Extract epochs]
    J --> K
    K --> L[Baseline correct]
    L --> M[Two-stage rejection with repair]
    M --> N[Average to ERPs]
```

Summary

Step	When to Customize	Key Parameter
Rereference	Sparse arrays, auditory studies	`:avg` vs `:Cz` vs `[:M1,:M2]`
High-pass	ICA vs ERP analysis	0.1 Hz (ERP), 1.0 Hz (ICA)
Low-pass	Gamma vs ERP analysis	30 Hz (ERP), 100 Hz (gamma)
Artifact threshold	Clean vs noisy recordings	75 µV (typical), 50-150 µV range
ICA	Very clean data or <32 ch	Skip if no eye artifacts
Baseline	Stimulation timing	Entire epoch vs pre-stimulus

Further Reading

- **Artifact detection** — Artifact Handling for detecting and repairing bad channels and epochs
- **Automating this workflow** — Batch Processing for running `preprocess` across a full cohort
- **Selection syntax** — Selection Patterns for channel, sample, and epoch filters used throughout preprocessing
- **Working example** — Preprocessing Workflow demo

Chapter 8

Epoch Selection

8.1 Epoch Selection

`EegFun.jl` provides a system for selecting epochs from continuous data. This includes experimental paradigms that use simple single triggers or complex sequences of events with timing constraints.

This tutorial covers the various ways you can define epochs, from the simplest single-trigger approach to advanced pattern matching.

Quick Reference

Feature	Field(s)	Example
Single trigger	`trigger_sequences = [[1]]`	Epoch around trigger 1
Multiple triggers (OR)	`trigger_sequences = [[1], [2]]`	Either trigger 1 or 2
Trigger sequence	`trigger_sequences = [[1, 10]]`	Trigger 1 followed by 10
Wildcard	`trigger_sequences = [[1, :any, 3]]`	1, then anything, then 3
Range	`trigger_sequences = [[1:5, 10]]`	Any of 1–5, then 10
t=0 reference	`reference_index = 2`	Align to 2nd trigger in sequence
Timing constraint	`timing_pairs` , `min_interval` , `max_interval`	Only if triggers 200–800ms apart
Position: after trigger	`mask_before_trigger = 99`	Only after marker trigger 99
Position: before trigger	`mask_after_trigger = 88`	Only before marker trigger 88
Hide stray triggers	`mask_triggers = [50]`	Ignore trigger 50 when matching sequences
Hide a practice block	`mask_between_triggers = [(15, 16)]`	Ignore all triggers between 15 and 16

The `EpochCondition` Structure

At the heart of epoch selection is the `EpochCondition` structure. It allows you to define exactly what constitutes an epoch in your study.

```
@kwdef struct EpochCondition
    name::String
    trigger_sequences::Vector{Vector{Union{Int,Symbol,UnitRange{Int}}}}
    reference_index::Int = 1
    timing_pairs::Union{Nothing,Vector{Tuple{Int,Int}}} = nothing
    min_interval::Union{Nothing,Float64} = nothing
    max_interval::Union{Nothing,Float64} = nothing
    mask_before_trigger::Union{Nothing,Int} = nothing   # only keep epochs AFTER
    this trigger
    mask_after_trigger::Union{Nothing,Int} = nothing    # only keep epochs BEFORE
    this trigger
    mask_triggers::Union{Nothing,Vector{Int}} = nothing # hide stray triggers
    before matching
    mask_between_triggers::Union{Nothing,Vector{Tuple{Int,Int}}} = nothing # hide
    whole blocks
end
```

Simple Epoching: Single Values

The most common case is extracting an epoch around a single trigger value.

```
# Define a condition for trigger 1
condition = EegFun.EpochCondition(
    name = "Condition1",
    trigger_sequences = [[1]]
)

# Extract epoch (-200ms to 1000ms around the trigger)
epochs = EegFun.extract_epochs(dat, condition, (-0.2, 1.0))
```

To match any of multiple trigger values for a single condition (OR logic), provide multiple sequences:

```
# Match trigger 1 OR trigger 2
condition = EegFun.EpochCondition(
    name = "Condition1",
    trigger_sequences = [[1], [2]] # matches either trigger 1 or trigger 2
)

# Extract epoch (-200ms to 1000ms around the trigger)
epochs = EegFun.extract_epochs(dat, condition, (-0.2, 1.0))
```

Multiple Conditions

Most experiments have more than one condition. Pass a vector of `EpochCondition` objects to `extract_epochs` to extract epochs for each condition separately:

```
# Define two separate conditions
```

```
conditions = [
    EegFun.EpochCondition(name = "Condition1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Condition2", trigger_sequences = [[2]]),
]

# Extract epochs for both conditions
epochs = EegFun.extract_epochs(dat, conditions, (-0.2, 1.0))
```

Each condition is matched independently, and the resulting `EpochData` retains the condition labels for later analysis (e.g., averaging, comparison).

Sequence Matching

Sometimes an "event" is actually a sequence of triggers. For example, a target stimulus (1) followed by a response (10).

```
# Match the sequence [1, 2]
condition = EegFun.EpochCondition(
    name = "TargetResponse",
    trigger_sequences = [[1, 2]] # a 1 followed by a 2 with t=0 being the 1
)
```

Wildcards and Ranges

You can use wildcards and ranges within your sequences:

- `:any` : Matches any trigger value.
- `UnitRange` (e.g., `1:10`): Matches any value within the range.

```
# Match trigger 1, then any trigger, then trigger 3
condition = EegFun.EpochCondition(
    name = "Wildcard",
    trigger_sequences = [[1, :any, 3]]
)

# Match any trigger from 1 to 5, then trigger 10
condition = EegFun.EpochCondition(
    name = "Range",
    trigger_sequences = [[1:5, 10]]
)
```

Onset (t=0) Reference

By default, the first trigger in a sequence is considered $t = 0$. You can change this using `reference_index` .

```
# Sequence: [Warning (1), Stimulus (2), Response (10)]
# We want t=0 to be the Stimulus (index 2)
```

```
condition = EegFun.EpochCondition(
    name = "StimulusOnset",
    trigger_sequences = [[1, 2, 10]], # 1 followed by 2 followed by 10 with t=0
    being the 2
    reference_index = 2
)
```

`reference_index` refers to the position within the trigger sequence, not the trigger value itself. The sequence matcher internally tracks the *actual sample position* for each element in the matched sequence, so `reference_index = 2` correctly resolves to the sample where trigger 2 occurred — even when zero-valued samples are skipped between triggers.

You can restrict matches to sequences where triggers occur within specific time intervals. This is useful for filtering out trials with late responses or accidental double-triggers.

```
# Match [1, 10] only if they occur between 200ms and 800ms apart
condition = EegFun.EpochCondition(
    name = "ValidResponse",
    trigger_sequences = [[1, 10]],
    timing_pairs = [(1, 2)], # Calculate interval between 1st and 2nd trigger
    min_interval = 0.2,
    max_interval = 0.8
)
```

Position Constraints

You can also filter sequences based on whether they occur before or after certain "marker" triggers.

```
# Only find sequences that occur AFTER trigger 99 (e.g., start of an experimental
block)
condition = EegFun.EpochCondition(
    name = "Block2",
    trigger_sequences = [[1, 2]],
    mask_before_trigger = 99   # discard any sequence that comes before trigger 99
)

# Only find sequences that occur BEFORE trigger 88 (e.g., end of first half)
condition = EegFun.EpochCondition(
    name = "Phase1",
    trigger_sequences = [[1, 2]],
    mask_after_trigger = 88    # discard any sequence that comes after trigger 88
)
```

> [!NOTE] You cannot specify both `mask_before_trigger` and `mask_after_trigger` on the same condition — use separate conditions if needed.

Hiding Stray or Practice Triggers

Sometimes your trigger stream contains extra triggers that would break sequence matching — stray button presses, practice blocks, or inter-block markers. Two masking options cover these cases.

`mask_triggers` — hide individual trigger values

Adds a pre-processing step that temporarily removes specific trigger values **before** sequence matching runs. The surrounding sequence is unchanged; the masked trigger simply disappears.

```
# The stream is: 101, 50, 111, 127
# We want [101, 111, 127] — but trigger 50 is a stray button press that gets in
the way.
condition = EegFun.EpochCondition(
    name = "target_response",
    trigger_sequences = [[101, 111, 127]],
    reference_index = 1,
    mask_triggers = [50]   # hide trigger 50 before matching
)
```

`mask_between_triggers` — hide entire blocks

Marks all triggers that fall **between** a pair of boundary markers as invisible before matching. This is ideal for excluding complete practice blocks.

```
# Practice block is bounded by triggers 15 (start) and 16 (end).
# Any sequence inside that range should be ignored.
condition = EegFun.EpochCondition(
    name = "experimental_trials",
    trigger_sequences = [[101, 111, 127]],
    reference_index = 1,
    mask_between_triggers = [(15, 16)]  # hide everything between 15 and 16
)
```

Both options can be combined with each other and with all other constraints.

External TOML Configuration

For complex studies, defining your epoch conditions in an external TOML file is often cleaner and is the recommended approach for the pipeline procedure.

The TOML Format

Create a file (e.g., `epochs.toml`):

```
[epochs]

[[epochs.conditions]]
name = "condition_1"
trigger_sequences = [[1]]
```

```
[[epochs.conditions]]
name = "condition_2"
trigger_sequences = [[2]]

[[epochs.conditions]]
name = "condition_3"
trigger_sequences = [[3]]

[[epochs.conditions]]
name = "condition_4"
trigger_sequences = [[4]]
```

Here is a more advanced example using sequence matching, timing constraints, and position constraints:

```
[epochs]

# Stimulus followed by a response, time-locked to the stimulus
[[epochs.conditions]]
name = "stimulus_response1"
trigger_sequences = [[1, 10]]
reference_index = 1
timing_pairs = [[1, 2]]
min_interval = 0.1
max_interval = 0.8

[[epochs.conditions]]
name = "stimulus_response2"
trigger_sequences = [[2, 10]]
reference_index = 1
timing_pairs = [[1, 2]]
min_interval = 0.1
max_interval = 0.8
```

Loading and Using the TOML

```
using TOML

# Load the configuration
config = TOML.parsefile("epochs.toml")

# Parse into EpochCondition objects
conditions = EegFun.condition_parse_epoch(config)

# Extract epochs
```

```
epochs = EegFun.extract_epochs(dat, conditions, (-0.2, 1.0))
```

Next Steps

- **Artifact handling** — Artifact Handling for detecting and rejecting bad epochs after extraction
- **Batch pipelines** — Batch Processing uses epoch condition TOML files as part of its automated workflow
- **Selection predicates** — Selection Patterns for filtering channels, samples, and time intervals in downstream analysis

Chapter 9

Artifact Handling

9.1 Automated Artifact Handling

Artifacts are an unavoidable aspect of EEG research. `EegFun.jl` provides a structured workflow for identifying artifacts, repairing what can be salvaged, and rejecting what cannot.

Continuous Data: Sample-Level Detection

Before epoching, you can flag artifact-contaminated samples in continuous data. This is useful for marking segments to exclude from downstream analysis (e.g., ICA training) or for data quality reporting.

Extreme Values

`is_extreme_value!` flags samples where the absolute voltage exceeds a threshold. This catches amplifier saturation, large movement artifacts, and disconnected electrodes.

```
# Flag samples exceeding ±100 µV (adds column :is_extreme_value_100)
EegFun.is_extreme_value!(dat, 100)

# Check how many samples were flagged
EegFun.n_values(dat, :is_extreme_value_100)

# Only check specific channels
EegFun.is_extreme_value!(dat, 100, channel_selection =
EegFun.channels([:Fp1, :Fp2]))
```

Step Values

`is_step_value!` flags samples where the voltage jump between consecutive samples exceeds a threshold. This detects sudden discontinuities (cable pulls, electrode pops).

```
# Flag jumps > 50 µV between consecutive samples (adds
column :is_step_value_50.0)
EegFun.is_step_value!(dat, 50.0)

# Check how many step artifacts were detected
EegFun.n_values(dat, :is_step_value_50.0)
```

> [!TIP] Both functions also have non-mutating versions (`is_extreme_value` , `is_step_value`) that return a boolean mask without modifying the data.

Once flagged, these artifact columns can be visualised in `EegFun.plot_databrowser` — select the relevant column from the **Extra Channels** dropdown menu to highlight flagged samples on the EEG traces.

Continuous Data: Bad Channel Detection

Before epoching, it is worth checking whether any channels are consistently noisy or broken across the entire recording. In particular, removing or repairing excessively bad channels **before ICA** is strongly recommended.

Channel Summary Statistics

`channel_summary` computes per-channel descriptive statistics — minimum, maximum, standard deviation, variance, range, and critically the **z-scored variance** (`zvar`). A channel whose variance is far from the group mean is likely problematic.

```
summary_df = EegFun.channel_summary(dat)
```

You can restrict the summary to samples within marked epoch intervals (or any other defined interval), which often gives a better picture of data quality. Periods outside the task — such as block breaks where participants may stretch or the experimenter may adjust electrodes — can inflate channel summary values and thus, influence channel quality judgements:

```
summary_epoch = EegFun.channel_summary(dat, sample_selection =
EegFun.samples(:epoch_interval))
```

Channel Joint Probability

`channel_joint_probability` offers a complementary metric. It computes how likely each channel's data is under a probability model — channels with improbable distributions are flagged.

```
cjp = EegFun.channel_joint_probability(dat)
```

> [!TIP] Blink artefacts can inflate variance and joint probability for frontal channels (e.g. Fp1, Fp2), causing them to be falsely identified as bad. If you have already detected EOG onsets with `detect_eog_onsets!` , you can mark an interval around each onset and exclude those samples when computing channel metrics:
>
> ```
> # Mark a −100 to +300 ms interval around each vEOG onset
> EegFun.mark_epoch_intervals!(dat, :is_vEOG, [-0.1, 0.3])
>
> # Compute channel metrics excluding blink intervals and extreme values
> clean = EegFun.samples_or_not([:is_vEOG_interval, :is_extreme_value_100])
> summary_df = EegFun.channel_summary(dat, sample_selection = clean)
> cjp = EegFun.channel_joint_probability(dat, sample_selection = clean)
> ```

Identifying Bad Channels

`identify_bad_channels` combines both measures. Channels exceeding the z-variance criterion **or** flagged by joint probability are labelled as bad:

```
bad = EegFun.identify_bad_channels(summary_df, cjp)

# More conservative criterion
bad = EegFun.identify_bad_channels(summary_df, cjp, zvar_criterion = 2.5)
```

Partitioning by EOG Correlation

Some channels may look "bad" simply because they pick up a lot of eye-movement activity. These are better handled later by ICA rather than labeled as "bad" or interpolated. `partition_channels_by_eog_correlation` separates the two groups:

```
eog_cm = EegFun.correlation_matrix_eog(dat, eog_cfg)
EegFun.add_zscore_columns!(eog_cm)

non_eog_bad, eog_bad = EegFun.partition_channels_by_eog_correlation(
    bad, eog_cm;
    eog_channels = [:hEOG, :vEOG],
    threshold = 0.3,
)
```

Repairing Bad Channels

Channels that are genuinely broken (not EOG-related) can be repaired by interpolation. Two methods are available:

Neighbor interpolation — weighted average of nearby channels. Requires pre-computed neighbor information:

```
# Compute neighbors first (only needed once)
EegFun.get_neighbors_xyz!(dat.layout, 0.5)

repair_info = EegFun.create_continuous_repair_info(:neighbor_interpolation)
EegFun.channel_repairable!(repair_info, non_eog_bad, dat.layout)
EegFun.repair_channels!(dat, repair_info; method = :neighbor_interpolation)
```

Spherical spline interpolation — uses all remaining good channels weighted by their spatial relationship on the scalp (Perrin et al., 1989). Does not require neighbors, but does require 3D coordinates in the layout:

```
EegFun.repair_channels!(dat, non_eog_bad; method = :spherical_spline)
```

Epoch-Level: Artifact Detection

After epoching, each trial needs to be checked for artifacts. EegFun provides automatic detection, interactive visual review, and a combined workflow.

Automatic Detection

`detect_bad_epochs_automatic` flags epochs based on two complementary criteria:

- **Z-score criterion** — channels whose variance, kurtosis, or range is far from the group mean across epochs
- **Absolute criterion** — channels where the voltage exceeds a hard threshold (e.g. ±100 μV)

An epoch is flagged if **any** channel exceeds **either** criterion.

```
# Default: z_criterion = 3.0, abs_criterion = 100 μV
artifacts = EegFun.detect_bad_epochs_automatic(epochs)

# More aggressive z-score threshold
artifacts = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 2.0)

# Absolute threshold only (disable z-score)
artifacts = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 0,
abs_criterion = 80.0)
```

The return value is an `EpochRejectionInfo` object that records which channels and epochs were flagged, and why.

Interactive Review

`detect_bad_epochs_interactive` opens a Makie grid view where you can visually inspect each epoch and toggle it as good or bad:

```
state = EegFun.detect_bad_epochs_interactive(epochs)
```

Controls:

- **Toggles** — check to reject, uncheck to keep
- **◂ Prev / Next ▸** — navigate pages
- **Show bad channels only** — filter to channels flagged by automatic detection (requires `artifact_info`)

Combined Workflow

The most effective approach runs automatic detection first, then passes the results into the interactive viewer so flagged epochs are pre-marked and you can filter to only the problematic channels:

```
# Step 1: automatic detection
artifacts = EegFun.detect_bad_epochs_automatic(epochs)

# Step 2: visual review with pre-marked epochs
state = EegFun.detect_bad_epochs_interactive(epochs, artifact_info = artifacts)
```

Epoch-Level: Channel Repair

If only one or two channels are noisy in an otherwise clean trial, you can repair them rather than discarding the whole epoch.

Analyse Repairability

Before repairing with neighbor interpolation, use `channel_repairable!` to check if bad channels have enough clean neighbors:

```
EegFun.channel_repairable!(artifacts, epochs.layout)
```

Apply Repair

```
# Method 1: Weighted neighbor interpolation (fast, requires neighbors)
EegFun.repair_artifacts!(epochs, artifacts, method = :neighbor_interpolation)

# Method 2: Spherical spline interpolation (higher quality, requires 3D
coordinates)
EegFun.repair_artifacts!(epochs, artifacts, method = :spherical_spline)
```

Epoch-Level: Rejection

After repairing what can be salvaged, remove the remaining bad epochs:

```
EegFun.reject_epochs!(epochs, artifacts)
```

Participant-Level Subsetting

If a participant has lost too much data across a condition, it may be better to exclude them entirely:

```
# Exclude participants with < 70% data retention
EegFun.subset_bad_data("preprocessed_files", 70.0)
```

Summary Workflow

A typical epoch-level artifact handling workflow:

```
# 1. Detect
artifacts = EegFun.detect_bad_epochs_automatic(epochs)

# 2. Review (optional)
state = EegFun.detect_bad_epochs_interactive(epochs, artifact_info = artifacts)

# 3. Repair (salvage single bad channels)
EegFun.channel_repairable!(artifacts, epochs.layout)
EegFun.repair_artifacts!(epochs, artifacts, method = :neighbor_interpolation)

# 4. Reject (remove non-repairable trials)
```

```
EegFun.reject_epochs!(epochs, artifacts)
```

Next Steps

- **Before epoching** — Manual Preprocessing covers continuous-data artifact detection and ICA in full context
- **Defining epochs** — Epoch Selection for trigger-based epoch extraction which feeds into artifact handling
- **Automating rejection** — Batch Processing runs detect → repair → reject automatically across a cohort
- **Interactive exploration** — Plot GUI for visually reviewing flagged epochs and channel data

Chapter 10

Layouts and Neighbors

10.1 Layouts and Neighbors

Spatial analysis, topographic plotting, and artifact repair in `EegFun.jl` all depend on knowing where electrodes are physically located. A **layout** stores electrode positions; **neighbors** are the set of nearby electrodes computed from those positions.

Reading a Layout

A layout CSV contains electrode labels and their polar coordinates (incidence and azimuth angles, in degrees). EegFun ships with layouts for BioSemi and EasyCap caps:

```
layout = EegFun.read_layout("resources/layouts/biosemi/biosemi64.csv")

# Check the electrode labels
EegFun.channel_labels(layout)
```

> [!TIP] When you load data with `create_eegfun_data` , the layout is automatically attached to your data object. You can access it via `dat.layout` .

Coordinate Systems

Layout files store positions in **polar coordinates** (incidence from the vertex, azimuth from the nose). Different operations require different Cartesian representations.

2D Coordinates (for Topographic Plots)

Topographic maps need flat x/y positions. The conversion projects electrodes onto a 2D plane, with the vertex (Cz) at the centre:

```
EegFun.polar_to_cartesian_xy!(layout)

# Check positions
EegFun.positions_2D(layout)
```

By default, radial distance is preserved — electrodes beyond the scalp edge (inc > 90°, e.g. eye channels from EEGLAB imports) will appear outside the head circle, matching EEGLAB's topoplot rendering. To normalise all electrodes to fit within a fixed radius instead:

```
EegFun.polar_to_cartesian_xy!(layout, preserve_radial_distance = false)
```

3D Coordinates (for Spherical Spline Repair)

Spherical spline interpolation and source localisation require 3D positions on the unit sphere:

```
EegFun.polar_to_cartesian_xyz!(layout)

# Check positions
EegFun.positions_3D(layout)
```

> [!NOTE] Both coordinate conversions modify the layout in-place and clear any previously calculated neighbor information, since the distance metric changes.

Visualising the Layout

Use `plot_layout_2d` or `plot_layout_3d` to verify that electrode positions look correct before proceeding with analysis:

```
# 2D view with head outline
EegFun.plot_layout_2d(layout)

# 3D view
EegFun.plot_layout_3d(layout)
```

Computing Neighbors

Many algorithms — neighbor interpolation for channel repair, spatial clustering, correlation-based diagnostics — require knowing which electrodes are near each other. EegFun finds neighbors based on Euclidean distance within a specified criterion.

2D Neighbors

Computed from the 2D projected coordinates. Suitable for topographic analysis:

```
EegFun.get_neighbors_xy!(layout, 0.5)
```

3D Neighbors

Computed from the 3D spherical coordinates. Preferred for channel repair since distances on the sphere are more accurate than projected distances:

```
EegFun.get_neighbors_xyz!(layout, 0.5)
```

> [!IMPORTANT] The **distance criterion** is in normalised coordinate units where 1.0 = the scalp equator (ear level). Think of it as a fraction of the head radius: 0.5 means "neighbors within 50% of the head radius." After computing neighbors, check the average count to ensure the value makes sense for your cap density:
>
> ```
> avg = EegFun.average_number_of_neighbors(layout.neighbors)
> ```

> A good target is typically 4–6 neighbors per electrode, although this can vary depending on the cap density. If the average is much lower, increase the criterion; if much higher, decrease it. But visual inspection is always recommended.

Inspecting Neighbors

For transparency and reproducibility, you can export the full neighbor structure — including distances and interpolation weights — to a TOML file:

```
EegFun.print_layout_neighbors(layout, "neighbors.toml")
```

Visualising Neighbors

Pass `neighbors = true` to the layout plot to interactively inspect connections. Hovering over an electrode highlights its neighbors:

```
EegFun.plot_layout_2d(layout, neighbors = true)
EegFun.plot_layout_3d(layout, neighbors = true)
```

Checking and Querying

```
# Does the layout have computed neighbors?
EegFun.has_neighbors(layout)

# What distance criterion was used?
EegFun.neighbor_criterion(layout)

# Does the layout have valid (non-zero) coordinates?
EegFun.has_valid_coordinates(layout)
```

> [!NOTE] Auto-generated layouts (created when loading data without a layout file) have all-zero coordinates. These work for basic data browsing but cannot be used for topographic plots or spatial operations. `has_valid_coordinates` returns `false` for such layouts.

Summary

A typical layout setup for analysis:

```
# Read the layout
layout = EegFun.read_layout("resources/layouts/biosemi/biosemi64.csv")

# Convert coordinates
EegFun.polar_to_cartesian_xy!(layout)   # for topoplots
EegFun.polar_to_cartesian_xyz!(layout)  # for spherical spline repair
```

```
# Compute neighbors
EegFun.get_neighbors_xyz!(layout, 0.5)

# Verify visually
EegFun.plot_layout_2d(layout, neighbors = true)
```

Next Steps

- **Channel repair** — Artifact Handling uses the neighbor structure computed here for `repair_channels!` and `repair_artifacts!`
- **Manual preprocessing** — Manual Preprocessing shows how layout setup fits into the full preprocessing workflow
- **Batch pipelines** — Batch Processing configures the layout and neighbor criterion via TOML

Chapter 11

Import

11.1 Data Import

Comprehensive Data Import

Overview

Demonstrates Comprehensive Data Import functionality.

Code Examples

```
# Demo: Importing and Preprocessing Various Data Formats
# Shows how to load different file formats, create EegFun data structures,
# apply basic preprocessing (filter, rereference), and visualize.

using EegFun

# ================================================================================
# 1. Biosemi (.bdf)
# ================================================================================
# Load raw BDF data
raw_bdf = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))

# Load a standard Biosemi 72-channel layout for 3D/2D visualization
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)

# Create EegFun data structure
dat_bdf = EegFun.create_eegfun_data(raw_bdf, layout)

# Basic preprocessing
EegFun.highpass_filter!(dat_bdf, 1.0) # 1 Hz high-pass
EegFun.rereference!(dat_bdf, :avg)    # average reference

# Check triggers and display
println("BDF Triggers: ", EegFun.trigger_count(dat_bdf))
# EegFun.plot_databrowser(dat_bdf)

# ================================================================================
# 2. European Data Format (.edf)
```

```julia
# ==============================================================================
raw_edf = EegFun.read_raw_data(EegFun.example_path("data/edf/test.edf"))

# Using the fallback auto-generated layout (if you don't have a specific .csv
layout)
dat_edf = EegFun.create_eegfun_data(raw_edf)

EegFun.highpass_filter!(dat_edf, 1.0)
EegFun.rereference!(dat_edf, :avg)

println("EDF Triggers: ", EegFun.trigger_count(dat_edf))
# EegFun.plot_databrowser(dat_edf)

# ==============================================================================
# 3. BrainVision (.vhdr)
# ==============================================================================
raw_bv = EegFun.read_raw_data(EegFun.example_path("data/brainvision/
example1.vhdr"))
dat_bv = EegFun.create_eegfun_data(raw_bv)

EegFun.highpass_filter!(dat_bv, 1.0)
EegFun.rereference!(dat_bv, :avg)

println("BrainVision Triggers: ", EegFun.trigger_count(dat_bv))
# EegFun.plot_databrowser(dat_bv)

# ==============================================================================
# 4. Functional Image Format (.fif)
# ==============================================================================
raw_fif = EegFun.read_raw_data(EegFun.example_path("data/fif/test_raw.fif"))
dat_fif = EegFun.create_eegfun_data(raw_fif)

EegFun.highpass_filter!(dat_fif, 1.0)
EegFun.rereference!(dat_fif, :avg)

println("FIF Triggers: ", EegFun.trigger_count(dat_fif))
# EegFun.plot_databrowser(dat_fif)

# ==============================================================================
# 5. Extensible Data Format (.xdf)
# ==============================================================================
raw_xdf = EegFun.read_raw_data(EegFun.example_path("data/xdf/test1.xdf"))
dat_xdf = EegFun.create_eegfun_data(raw_xdf)

EegFun.highpass_filter!(dat_xdf, 1.0)
```

```
EegFun.rereference!(dat_xdf, :avg)

println("XDF Triggers: ", EegFun.trigger_count(dat_xdf))
# EegFun.plot_databrowser(dat_xdf)

# ================================================================================
# 6. EEGLAB (.set) & FieldTrip (.mat)
# ================================================================================
# EEGLAB and FieldTrip data is read directly into the EegFunData structure
# (bypassing the `create_eegfun_data` step) because they are typically
# preprocessed data saved natively in MATLAB.
# Therefore, we can often skip the basic preprocessing steps.

dat_eeglab = EegFun.read_raw_data(EegFun.example_path("data/eeglab/
eeglab_data.set"))
println("EEGLAB Triggers: ", EegFun.trigger_count(dat_eeglab))
# EegFun.plot_databrowser(dat_eeglab)

dat_fieldtrip = EegFun.read_raw_data(EegFun.example_path("data/fieldtrip/
continuous.mat"))
println("FieldTrip Triggers: ", EegFun.trigger_count(dat_fieldtrip))
# EegFun.plot_databrowser(dat_fieldtrip)
```

- API Reference

11.2 BioSemi Import

BioSemi Import

This demo demonstrates importing BioSemi BDF files into EegFun.jl.

What is BioSemi BDF Format?

BioSemi is a popular manufacturer of active electrode EEG systems. The BDF (BioSemi Data Format) is a 24-bit variant of the European Data Format (EDF) designed specifically for BioSemi's ActiveTwo and ActiveOne systems.

Key features:

- 24-bit resolution for high-precision recordings
- Continuous raw data storage
- Trigger channel for event markers (last channel)
- Standard format widely supported across analysis tools

Import Capabilities

Data loading:

- Raw continuous EEG time series (24-bit precision)

- Sampling rate and channel metadata
- Trigger/event markers (encoded in Status channel)

What you need:

- `.bdf` file (contains all data)
- Electrode layout file matching your cap configuration
- BioSemi systems typically use standard cap layouts (16, 32, 64, 72, or 128 channels)

Data Mapping

EegFun.read*raw*data loads BioSemi data, then **create*eegfun*data** converts to native EegFun types:

- BioSemi BDF ⟶ `BiosemiData` (intermediate) ⟶ `ContinuousData` (EegFun)
- Triggers extracted from Status channel
- Layout coordinates added separately

Important Notes

Layout Files

BioSemi doesn't embed electrode positions in the data file, so you must provide a matching layout. EegFun includes standard BioSemi layouts in `resources/layouts/biosemi/` :

- `biosemi16.csv`
- `biosemi32.csv`
- `biosemi64.csv`
- `biosemi72.csv` (64 + 8 external channels)
- `biosemi128.csv`

Choose the layout matching your recording setup.

Trigger Encoding

BioSemi encodes triggers in a dedicated Status channel using binary encoding. EegFun automatically extracts these during import, making them available via `trigger_count()` and epoch extraction functions.

Workflow Summary

This demo shows the complete BioSemi import workflow:

Load Raw BDF Data

- Use `read_raw_data()` with path to `.bdf` file
- Automatic format detection based on file extension

Prepare Layout

- Load appropriate BioSemi layout file
- Convert polar coordinates to cartesian for visualization

Create EegFun Data Structure

- Combine raw data with layout using `create_eegfun_data()`
- Results in `ContinuousData` ready for analysis

Code Examples

```
# Demo: Loading and Processing BioSemi BDF Files
# Shows how to load BioSemi .bdf files, create EegFun data structures
# with layouts, and get the triggers/events.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# Load raw BDF data
# read_raw_data automatically detects the .bdf extension
raw_data = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))

# EegFun uses the Julia package: BiosemiDataFormat.jl
# https://github.com/igmmgi/BiosemiDataFormat.jl

raw_data.header.channel_labels  # electrode labels
raw_data.header.sample_rate     # sampling rate
raw_data.header.num_channels    # number of channels
raw_data.data
raw_data.triggers

# Load and prepare electrode layout
# BioSemi systems typically use their standard cap configurations
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)

layout.data # DataFrame with labels and positions

# Create EegFun data structure
dat = EegFun.create_eegfun_data(raw_data, layout)

EegFun.all_data(dat)
EegFun.meta_data(dat)
EegFun.all_labels(dat)
EegFun.channel_labels(dat)
EegFun.meta_labels(dat)
```

```
EegFun.extra_labels(dat) # empty

# Check trigger information
println("\nTrigger summary:")
EegFun.trigger_count(dat) # BioSemi data uses 8 bit trigger lines

# Basic preprocessing
EegFun.highpass_filter!(dat, 1.0) # 1 Hz high-pass
EegFun.rereference!(dat, :avg)    # Average reference

# Open interactive databrowser
EegFun.plot_databrowser(dat);
```

- API Reference

11.3 BrainVision Import

BrainVision Import

This demo demonstrates importing BrainVision format files into EegFun.jl.

What is BrainVision Format?

BrainVision is the data format used by Brain Products GmbH EEG systems (e.g., BrainAmp, actiCHamp, LiveAmp). It's a flexible, text-header based format widely used in research.

The format consists of three files:

- `.vhdr` - Header file (ASCII text) containing metadata, channel information, and data format specifications
- `.eeg` - Binary data file containing the actual EEG samples
- `.vmrk` - Marker file (ASCII text) with trigger/event information

Key features:

- Human-readable header and marker files
- Flexible data encoding (16-bit, 32-bit integer or float)
- Separate marker file for triggers and annotations
- Wide support across analysis platforms

Import Capabilities

Data loading:

- Raw continuous EEG time series
- Sampling rate and channel metadata
- Trigger/event markers with descriptions

What you need:

- BrainVision triplet (`.vhdr` , `.eeg` , `.vmrk` files in same directory)

- Electrode layout file matching your cap configuration
- Brain Products systems often use actiCap or EasyCap layouts

Data Mapping

EegFun.read*raw*data loads BrainVision data, then **create*eegfun*data** converts to native EegFun types:

- BrainVision → `BrainVisionData` (intermediate) → `ContinuousData` (EegFun)
- Markers extracted from `.vmrk` file
- Layout coordinates added separately

Important Notes

File Organization

All three files must be in the same directory with matching base names:

```
my_recording.vhdr
my_recording.eeg
my_recording.vmrk
```

You can specify any of the three files to `read_raw_data()` - it will automatically find the others.

Layout Files

BrainVision doesn't embed 3D electrode positions, so you must provide a matching layout. Depending on your cap manufacturer:

Brain Products actiCap: Check `resources/layouts/acticap/` **EasyCap**: Check `resources/layouts/easycap/`

Choose the layout matching your recording configuration.

Marker Types

BrainVision supports rich marker information including:

- Stimulus markers (trigger codes)
- Response markers (button presses)
- Comments and annotations
- Segment boundaries

EegFun imports all markers and makes them available for epoch extraction and analysis.

Workflow Summary

This demo shows the complete BrainVision import workflow:

Load Raw BrainVision Data

- Use `read_raw_data()` with path to any file in the triplet (`.vhdr` , `.eeg` , or `.vmrk`)

- Automatic format detection based on file extension
- All three files loaded automatically

Prepare Layout

- Load appropriate layout file for your cap configuration
- Convert polar coordinates to cartesian for visualization

Create EegFun Data Structure

- Combine raw data with layout using `create_eegfun_data()`
- Results in `ContinuousData` ready for analysis

Code Examples

```
# Demo: Loading and Processing BrainVision Files
# Shows how to load BrainVision format files (.vhdr, .eeg, .vmrk),
# create EegFun data structures, apply basic preprocessing,
# and get the triggers/events.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.vhdr")

using EegFun
using GLMakie

# Load raw BrainVision data
# read_raw_data automatically detects BrainVision files (.vhdr, .eeg, or .vmrk)
# You can specify any of the three files; it will find the others, assuming
# they are in the same directory
dat = EegFun.read_raw_data(EegFun.example_path("data/brainvision/example1.vhdr"))

# EegFun uses the Julia package: BrainVisionDataFormat.jl
# https://github.com/igmmgi/BrainVisionDataFormat.jl
dat.data
dat.header

# Create EegFun data structure
eeg = EegFun.create_eegfun_data(dat)

EegFun.all_data(eeg)
EegFun.meta_data(eeg)
EegFun.all_labels(eeg)
EegFun.channel_labels(eeg)
EegFun.meta_labels(eeg)
```

```
EegFun.extra_labels(eeg)

# Check trigger information
# EegFun creates a hash of the marker names for the numeric trigger column
# triggers_info contains the mapping of trigger names to numeric values
println("\nTrigger summary:")
EegFun.trigger_count(eeg)

# Basic preprocessing
EegFun.highpass_filter!(eeg, 1.0) # 1 Hz high-pass
EegFun.rereference!(eeg, :avg)    # Average reference

# Open interactive databrowser
EegFun.plot_databrowser(eeg);
```

- API Reference

11.4 EDF Import

EDF Import

Overview

Demonstrates EDF Import functionality.

Code Examples

```
# Demo: Loading and Processing EDF Files
# Shows how to load European Data Format (.edf) files,
# create EegFun data structures, apply basic preprocessing,
# and visualize the data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.edf")

using EegFun
using GLMakie

# Load raw EDF data
raw_edf = EegFun.read_raw_data(EegFun.example_path("data/edf/test.edf"))

# Using the fallback auto-generated layout (if you don't have a specific .csv
layout)
dat = EegFun.create_eegfun_data(raw_edf)
```

```
EegFun.all_data(dat)
EegFun.meta_data(dat)
EegFun.all_labels(dat)
EegFun.channel_labels(dat)
EegFun.meta_labels(dat)
EegFun.extra_labels(dat) # empty


# Basic preprocessing
EegFun.highpass_filter!(dat, 1.0) # 1 Hz high-pass filter
EegFun.rereference!(dat, :avg)    # Average reference

println("\nTrigger summary:")
EegFun.trigger_count(dat)

# Open interactive databrowser
EegFun.plot_databrowser(dat);
```

- API Reference

11.5 FIF Import

FIF Import

Overview

Demonstrates FIF Import functionality.

Code Examples

```
# Demo: Loading and Processing FIF Files
# Shows how to load Functional Image Format (.fif) files,
# create EegFun data structures, apply basic preprocessing,
# and visualize the data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.fif")

using EegFun
using GLMakie

# Load raw FIF data
raw_fif = EegFun.read_raw_data(EegFun.example_path("data/fif/test_raw.fif"))
```

```
# Using the fallback auto-generated layout (if you don't have a specific .csv
layout)
dat = EegFun.create_eegfun_data(raw_fif)

EegFun.all_data(dat)
EegFun.meta_data(dat)
EegFun.all_labels(dat)
EegFun.channel_labels(dat)
EegFun.meta_labels(dat)
EegFun.extra_labels(dat) # empty



# Basic preprocessing
EegFun.highpass_filter!(dat, 1.0) # 1 Hz high-pass filter
EegFun.rereference!(dat, :avg)    # Average reference

println("\nTrigger summary:")
EegFun.trigger_count(dat)

# Open interactive databrowser
EegFun.plot_databrowser(dat);
```

- API Reference

11.6 XDF Import

XDF Import

Overview

Demonstrates XDF Import functionality.

Code Examples

```
# Demo: Loading and Processing XDF Files
# Shows how to load Extensible Data Format (.xdf) files,
# create EegFun data structures, apply basic preprocessing,
# and visualize the data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.xdf")

using EegFun
using GLMakie
```

```
# Load raw XDF data
raw_xdf = EegFun.read_raw_data(EegFun.example_path("data/xdf/test1.xdf"))

# Using the fallback auto-generated layout
dat = EegFun.create_eegfun_data(raw_xdf)

EegFun.all_data(dat)
EegFun.meta_data(dat)
EegFun.all_labels(dat)
EegFun.channel_labels(dat)
EegFun.meta_labels(dat)
EegFun.extra_labels(dat) # empty

# Basic preprocessing
EegFun.highpass_filter!(dat, 1.0) # 1 Hz high-pass filter
EegFun.rereference!(dat, :avg)    # Average reference

println("\nTrigger summary:")
EegFun.trigger_count(dat)

# Open interactive databrowser
EegFun.plot_databrowser(dat);
```

- API Reference

11.7 EEGLAB Import

EEGLAB Import

This demo demonstrates importing EEGLAB `.set` files into EegFun.jl. EEGLAB is a widely used open-source toolboxes for EEG processing (in MATLAB).

About EEGLAB .set Format

The `.set` format is a MATLAB-based file that contains a header structure with all recording parameters, trigger information, and often precomputed info like ICA weights or epoch definitions. The actual EEG data may be stored within the `.set` file or in a separate `.fdt` file.

Key features:

- Supports both continuous and epoched data
- Often includes ICA components and weights
- Comprehensive metadata storage

Import Capabilities

Data loading:

- Automatic detection of continuous vs. epoched data

- Import of ICA weights and sphere matrices, if available
- Mapping of event/trigger labels (hashed to triggers)
- Support for external data files (.fdt)

Data Mapping

EegFun.read_eeglab handles mapping of EEGLAB structures to native EegFun types:

- EEGLAB Dataset ⟶ `ContinuousData` , `EpochData` , or `ErpData`
- ICA info ⟶ `ICA` structure
- Event labels ⟶ Available in `:trigger_info` column

Workflow Summary

1. **Load Data**: Depending on the file content, `read_eeglab()` can return just data or data plus ICA information.
2. **Check Triggers**: EEGLAB often uses string labels for events. EegFun hashes these for its trigger system while preserving the original labels.
3. **Verification**: Use `plot_databrowser()` to verify the imported time series or epochs.
4. **ICA Visualization**: If ICA information was imported, you can immediately plot components and activations.

Code Examples

```
# Demo: Loading and Processing EEGLAB .set Files
# Note: trigger/event strings are hashed for :triggers, but available
in :trigger_info.
# Once loaded, all EegFun functions work as expected.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_eeglab("/path/to/your/data.set")

using EegFun
using GLMakie

# this seems to be a raw continuous data file without any ICA info
dat = EegFun.read_eeglab(EegFun.example_path("data/eeglab/eeglab_data.set"))
EegFun.plot_databrowser(dat)
EegFun.trigger_count(dat)

# this seems to be a epoched data file with ica info
dat, ica = EegFun.read_eeglab(EegFun.example_path("data/eeglab/epochs.set"))
EegFun.plot_databrowser(dat)

# We can plot the ICA activations
EegFun.plot_topography(ica, component_selection = EegFun.components([1]))
```

- API Reference

11.8 FieldTrip Import

FieldTrip Import

This demo demonstrates importing FieldTrip `.mat` files into EegFun.jl. FieldTrip is a MATLAB toolbox for EEG/MEG analysis, and its structures are a common way to exchange processed data.

About FieldTrip .mat Format

FieldTrip stores data in MATLAB structures (generic `.mat` files). EegFun.jl uses `MAT.jl` to parse them and map them onto EegFun types.

Key features:

- Supports raw, epoched, and averaged (timelock) data

Import Capabilities

Data loading:

- Import of continuous `raw` data
- Import of segmented `epochs` data
- Import of averaged `timelock` (ERP) data

Data Mapping

EegFun.read_fieldtrip maps MATLAB structures to EegFun:

- FieldTrip `raw` / `epochs` ⟶ `ContinuousData` / `EpochData`
- FieldTrip `timelock` ⟶ `ErpData`
- Sample rates and time axes are automatically reconstructed

Workflow Summary

1. **Load Layout**: Load a layout file and prepare it for plotting.
2. **Import Data**: Use `read_fieldtrip()` providing the path and the layout.
3. **Validation**: Visualize the imported structures using `plot_databrowser()` for continuous/epoched data or `plot_erp()` for averaged data.

Code Examples

```
# FieldTrip Import Demo
#
# Demonstrates loading FieldTrip .mat files

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_fieldtrip("/path/to/your/data.mat", layout)
```

```
using EegFun
using GLMakie

# Load layout (FieldTrip doesn't store layout with data)
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)

# Load continuous data
println("Loading continuous data...")
continuous_data = EegFun.read_fieldtrip(EegFun.example_path("data/fieldtrip/
continuous.mat"), layout)
EegFun.trigger_count(continuous_data)
EegFun.plot_databrowser(continuous_data)

# Load epoched data
println("\nLoading epoched data...")
epoch_data = EegFun.read_fieldtrip(EegFun.example_path("data/fieldtrip/
epochs.mat"), layout)
EegFun.plot_databrowser(epoch_data)

# Load ERP data
println("\nLoading ERP data...")
erp_data = EegFun.read_fieldtrip(EegFun.example_path("data/fieldtrip/erp.mat"),
layout)

# Biologische Psychologie Labor Tübingen Custom mat files
# Essentially slightly stripped down FieldTrip structures
println("\nLoading ERP data...")
epoch_data = EegFun.read_fieldtrip(EegFun.example_path("data/fieldtrip/
custom_epochs.mat"), layout)

# Load ERP data
println("\nLoading ERP data...")
erp_data = EegFun.read_fieldtrip(EegFun.example_path("data/fieldtrip/
custom_erp.mat"), layout)
EegFun.plot_erp(erp_data, layout = :grid)
```

- API Reference

11.9 Data

Data

This demo demonstrates understanding the core data structures in EegFun.jl and how to access and subset data.

Core Data Structures

ContinuousData:

- Raw EEG time series from continuous recording
- Contains electrode data, triggers, sampling rate, metadata
- Used for preprocessing, filtering, artifact detection

EpochData:

- Segmented trials around experimental events
- Organized by experimental conditions
- Vector of DataFrames (one per trial)

ErpData:

- Averaged event-related potentials
- One waveform per condition
- Used for statistical analysis and plotting

Data Access Functions

all_data: Complete data including channels and metadata

channel_data: Only EEG channel columns

meta_data: Only metadata columns (time, sample, triggers, etc.)

extra_data: Derived/computed channels (EOG, artifacts, boolean flags, etc.)

Workflow Summary

This demo shows basic data operations:

Load and Structure Data

- Load raw BioSemi data
- Load electrode layout
- Create ContinuousData structure

Access Data Components

- Extract all data, channel data, and metadata
- Understand different data access patterns

Create Epochs

- Define epoch conditions around triggers
- Extract segmented trials
- Access epoch data with selection functions

Subset Data

- Subset by channel selection (specific electrodes)
- Subset by sample selection (time intervals)

- Create smaller datasets for analysis

Code Examples

```
# Demo: Data Manipulation
# Shows basic data access, subsetting, and manipulation functions.

using EegFun

# read raw data
dat = EegFun.read_raw_data("./resources/data/bdf/example1.bdf");

# read and preprate layout file
layout_file = EegFun.read_layout("./resources/layouts/biosemi/biosemi72.csv");
EegFun.polar_to_cartesian_xy!(layout_file)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout_file);

# data/channel access functions
EegFun.all_data(dat)
EegFun.meta_data(dat)
EegFun.channel_data(dat)
EegFun.extra_data(dat) # empty

# some epoched data
epoch_cfg = [EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))

EegFun.all_data(epochs)
EegFun.all_data(epochs, epoch_selection = EegFun.epochs(1:2))
EegFun.meta_data(epochs)
EegFun.meta_data(epochs, epoch_selection = EegFun.epochs(1:2))
EegFun.channel_data(epochs)
EegFun.channel_data(epochs, epoch_selection = EegFun.epochs(1:2))

# We can subset out EegFun datatypes
dat_subset = EegFun.subset(dat, channel_selection = EegFun.channels([:Fp1, :Fp2]))
# only Fp1 and Fp2
EegFun.all_data(dat_subset)

dat_subset = EegFun.subset(dat, sample_selection = x -> x.sample .<= 10_000)
# first 10000 samples
EegFun.all_data(dat_subset)
```

```
dat_subset = EegFun.subset(dat, sample_selection = x -> x.time .<= 10)
# first 10 seconds
EegFun.all_data(dat_subset)
```

- API Reference

11.10 Data Persistence (JLD2)

Data Persistence (JLD2)

This demo shows how to save and load processed EEG data using the JLD2 format and EegFun's built-in loading utilities.

Why JLD2?

JLD2 is the standard approach for persisting Julia objects. When used with EegFun.jl, it preserves all data types and metadata without conversion, so you can save and reload continuous data, epochs, ERPs, and ICA results exactly as they are.

Key Functions

Function	Purpose
`jldsave`	Save one or more objects to a JLD2 file
`load`	Load specific keys or all data from a JLD2 file
`read_data`	Smart EegFun loader — auto-detects data types from JLD2
`read_all_data`	Batch load all matching files from a directory
`group_by_condition`	Organise loaded data into an OrderedDict by condition

Key Patterns

Saving data — use Julia's `jldsave` to store one or more objects:

```
jldsave("epochs.jld2"; data = epochs)
jldsave("results.jld2"; epochs = epochs, erps = erps, ica = ica_result)
```

Loading data — use `load` to read specific keys:

```
epochs = load("epochs.jld2", "data")
results = load("results.jld2")    # loads everything as a Dict
```

EegFun utilities — smart loading and batch operations:

```
loaded = EegFun.read_data("erps.jld2")                    # auto-detect types
```

```
all_erps = EegFun.read_all_data("./derivatives/erps/erps_final")  # load all
matching
grouped = EegFun.group_by_condition(all_erps)              # organise by
condition
```

Direct file paths — many EegFun plot functions accept JLD2 file paths directly:

```
EegFun.plot_databrowser("continuous_data.jld2")
EegFun.plot_databrowser("continuous_data.jld2", "ica.jld2")
```

Workflow Summary

Basic Saving and Loading

- Save and reload continuous data, epochs, ERPs, and ICA results
- Verify round-trip consistency

Multiple Items per File

- Store related objects (epochs, ERPs, ICA) in a single file
- Load individual keys by name

EegFun Loading Utilities

- `read_data` for smart single-file loading with auto type detection
- `read_all_data` for batch loading a cohort with participant selection
- `group_by_condition` for organising loaded data by condition number

Batch Loading

- Use file patterns to load a cohort of participants
- Feed into group-level analysis (e.g., grand average)

File Organization

- Recommended project directory structure
- Naming conventions for easy batch processing
- Memory considerations for large datasets

Code Examples

```
# Demo: Data Persistence with JLD2
# Shows how to save and load processed EEG data using JLD2 format,
# including EegFun's built-in loading utilities and file organization.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")
```

```julia
using EegFun
using GLMakie
using JLD2

const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)


############################################################################
# BASIC SAVING AND LOADING
############################################################################

# Read and prepare some data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

# Some preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# Save continuous data
jldsave(joinpath(DEMO_OUTPUT, "continuous_data.jld2"); data = dat)

# Load it back
loaded_dat = load(joinpath(DEMO_OUTPUT, "continuous_data.jld2"), "data")

# Verify it's the same
EegFun.all_data(loaded_dat) == EegFun.all_data(dat)

EegFun.plot_databrowser(loaded_dat);


############################################################################
# SAVING EPOCHED DATA
############################################################################

# Extract epochs
epoch_cfg = [
    EegFun.EpochCondition(name = "Condition1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Condition2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))
```

```
# Save all conditions
jldsave(joinpath(DEMO_OUTPUT, "epochs_all.jld2"); data = epochs)

# Save individual conditions
jldsave(joinpath(DEMO_OUTPUT, "epochs_condition1.jld2"); data = epochs[1])
jldsave(joinpath(DEMO_OUTPUT, "epochs_condition2.jld2"); data = epochs[2])

# Load epochs back
loaded_epochs = load(joinpath(DEMO_OUTPUT, "epochs_all.jld2"), "data")


#########################################################################
# SAVING ERPs
#########################################################################

# Create ERPs
erps = EegFun.average_epochs(epochs)

# Save ERPs
jldsave(joinpath(DEMO_OUTPUT, "erps.jld2"); data = erps)

# Load ERPs
loaded_erps = load(joinpath(DEMO_OUTPUT, "erps.jld2"), "data")


#########################################################################
# SAVING ICA RESULTS
#########################################################################

# Run ICA
EegFun.is_extreme_value!(dat, 100)
ica_result = EegFun.run_ica(dat, sample_selection =
EegFun.samples_not(:is_extreme_value_100))

# Save ICA decomposition
jldsave(joinpath(DEMO_OUTPUT, "ica_decomposition.jld2"); data = ica_result)

# Load ICA
loaded_ica = load(joinpath(DEMO_OUTPUT, "ica_decomposition.jld2"), "data")

# Can use loaded ICA with databrowser
EegFun.plot_databrowser(dat, loaded_ica)
```

```
#########################################################################
# SAVING MULTIPLE ITEMS IN ONE FILE
#########################################################################

# Save multiple related objects together
jldsave(joinpath(DEMO_OUTPUT, "analysis_results.jld2"); epochs = epochs, erps =
erps, ica = ica_result)

# Load specific items
my_epochs = load(joinpath(DEMO_OUTPUT, "analysis_results.jld2"), "epochs")
my_erps = load(joinpath(DEMO_OUTPUT, "analysis_results.jld2"), "erps")

# Or load everything
results = load(joinpath(DEMO_OUTPUT, "analysis_results.jld2"))
results["epochs"]
results["erps"]
results["ica"]



#########################################################################
# EEGFUN DATA LOADING UTILITIES
#########################################################################

# read_data — smart loader that auto-detects EegFun data types from JLD2 files
# Returns EegFunData, Vector{EegFunData}, or nothing
loaded = EegFun.read_data(joinpath(DEMO_OUTPUT, "erps.jld2"))

# read_all_data — batch load all files matching a pattern from a directory
# Useful for loading an entire cohort for group analysis
# all_erps = EegFun.read_all_data("derivatives/erps/erps")

# With participant selection
# all_erps = EegFun.read_all_data("derivatives/erps/erps", EegFun.participants([1,
2, 3]))

#########################################################################
# BATCH LOADING FROM DIRECTORY
#########################################################################

# Suppose you have multiple participant files in a directory:
# - sub-01_epochs.jld2
# - sub-02_epochs.jld2
# - sub-03_epochs.jld2

# Load all participants at once using EegFun's built-in batch loader
```

```
# all_participant_epochs = EegFun.read_all_data("my_output_directory/epochs")

# Or select specific participants
# selected_epochs = EegFun.read_all_data("my_output_directory/epochs",
EegFun.participants([1, 2, 3]))



#########################################################################
# READING DATA IN PLOT FUNCTIONS
#########################################################################

# Many EegFun plot functions accept file paths directly
EegFun.plot_databrowser(joinpath(DEMO_OUTPUT, "continuous_data.jld2"))
EegFun.plot_databrowser(joinpath(DEMO_OUTPUT, "epochs_all.jld2"))
EegFun.plot_databrowser(joinpath(DEMO_OUTPUT, "continuous_data.jld2"),
joinpath(DEMO_OUTPUT, "ica_decomposition.jld2"))
```

- API Reference

11.11 BIDS Export

BIDS Export

Export your EegFun data (raw files and/or preprocessed outputs) to a BIDS-compliant directory structure.

The generated dataset can be validated using the BIDS Validator.

Minimal Example (Raw Data Only)

```
using EegFun

EegFun.export_bids(
    task = "posner",
    raw_dir = ".",
    output_dir = "./bids_dataset",
)
```

This discovers all `.bdf` files in `raw_dir` , auto-extracts subject IDs from the filenames (`example1.bdf` → `sub-01`), and creates the BIDS structure with all required sidecar files.

Full Example

```
EegFun.export_bids(
    # Required
    task = "posner",
    raw_dir = ".",
```

```
    output_dir = "./bids_dataset",
    layout_file = "biosemi72.csv",

    # Dataset metadata
    dataset_name = "Visual Attention (Posner Cueing)",
    dataset_authors = ["Author A", "Author B"],
    dataset_license = "CC0",

    # Equipment
    manufacturer = "BioSemi",
    manufacturer_model = "ActiveTwo",
    cap_manufacturer = "BioSemi",
    cap_model = "headcap with 72 active electrodes",

    # Task
    task_description = "Posner cueing task with valid and invalid spatial cues",
    instructions = "Fixate on the central cross. Press the button as quickly as
    possible when the target appears.",

    # Recording context
    institution_name = "Your University",
    institution_address = "123 University Street, City, Country",
    institution_department = "Department of Psychology",
    eeg_ground = "CMS/DRL",
    eeg_placement_scheme = "10-10 subset with 72 electrodes",
    power_line_frequency = 50,

    overwrite = true,
)
```

Including Preprocessed Data (Derivatives)

If you have already run the EegFun preprocessing pipeline, pass the `derivatives_dir` argument to include the preprocessed JLD2 files:

```
EegFun.export_bids(
    task = "posner",
    raw_dir = ".",
    derivatives_dir = "./preprocessed_files",
    output_dir = "./bids_dataset",
    layout_file = "biosemi72.csv",
    dataset_name = "Visual Attention (Posner Cueing)",
    overwrite = true,
)
```

Pipeline output files are automatically renamed to BIDS naming conventions:

Pipeline Suffix	BIDS Label
`_continuous_raw`	`desc-continuousOriginal`
`_continuous_corrected`	`desc-continuousCleaned`
`_epochs_uncorrected`	`desc-epochsUncorrected`
`_epochs_unrejected`	`desc-epochsUnrejected`
`_epochs_final`	`desc-epochsFinal`
`_erps_uncorrected`	`desc-erpsOriginal`
`_erps_unrejected`	`desc-erpsCleaned`
`_erps_final`	`desc-erpsGood`
`_ica`	`desc-ica`
`_artifact_info`	`desc-artifactInfo`

Custom Subject ID Mapping

By default, subject IDs are auto-extracted from filenames using the first number found (`example12.bdf` → `sub-12`). For custom mappings, use the `participant_map` argument:

```
EegFun.export_bids(
    task = "posner",
    raw_dir = ".",
    participant_map = Dict(
        "Pract6" => "sub-01",
        "ExpA2"  => "sub-02",
        "test_file_99" => "sub-03",
    ),
)
```

Generated Directory Structure

```
bids_dataset/
├── dataset_description.json
├── participants.tsv
├── participants.json
├── sub-01/
│   └── eeg/
│       ├── sub-01_task-posner_eeg.bdf
│       ├── sub-01_task-posner_eeg.json
│       ├── sub-01_task-posner_channels.tsv
│       ├── sub-01_task-posner_events.tsv
│       ├── sub-01_task-posner_events.json
```

```
│       ├── sub-01_task-posner_electrodes.tsv
│       └── sub-01_task-posner_coordsystem.json
├── sub-02/ ...
└── derivatives/
    └── EegFun/
        ├── dataset_description.json
        └── sub-01/
            └── eeg/
                ├── sub-01_task-posner_desc-epochsFinal_eeg.jld2
                ├── sub-01_task-posner_desc-erpsGood_eeg.jld2
                └── ...
```

Auto-Computed Fields

The following fields are extracted automatically from your data — you don't need to provide them:

Field	Source
`SamplingFrequency`	From raw data
`EEGReference`	From `AnalysisInfo`
`EEGChannelCount` / `EOGChannelCount` / etc.	Auto-classified from channel names
`RecordingDuration`	Computed from data length
`RecordingType`	Always `"continuous"` for raw data
Channel names, types, units	From layout and channel labels
Event onsets and trigger values	From trigger column
Electrode positions (x, y, z)	From layout file

Validation

After exporting, validate your dataset using the BIDS Validator. Providing more of the optional metadata arguments will reduce the number of validator warnings.

API Reference

`EegFun.export_bids` – Function.

```
export_bids(; task, raw_dir = ".", raw_pattern = "\\.bdf", derivatives_dir =
"",
              output_dir = "./bids_dataset", layout_file = "", participant_map
              = nothing,
              dataset_name = "", dataset_authors = String[], dataset_license =
              "CC0",
              power_line_frequency = 50, overwrite = false, ...)
```

Export an EegFun project (raw data and optional preprocessed derivatives) to a BIDS-compliant directory structure.

Keyword Arguments

- `task::String` : **Required.** BIDS task label (e.g., `"posner"`).
- `raw_dir::String` : Directory containing raw data files (default: `"."`).
- `raw_pattern::String` : Regex pattern for raw files (default: `"\\.bdf"`).
- `derivatives_dir::String` : Pipeline output directory containing JLD2 files (default: `""`).
- `output_dir::String` : Output BIDS directory (default: `"./bids_dataset"`).
- `layout_file::String` : Electrode layout CSV for 3D electrode positions (default: `""`).
- `participant_map::Union{Nothing,Dict{String,String}}` : Optional mapping from raw basenames (without extension) to BIDS subject IDs (e.g., `"sub-01"`).
- `dataset_name::String` : Name for `dataset_description.json` (default: `""`).
- `dataset_authors::Vector{String}` : Authors for `dataset_description.json` (default: `String[]`).
- `dataset_license::String` : Dataset license (default: `"CC0"`).
- `power_line_frequency::Int` : Power line frequency in Hz (default: `50`).
- `overwrite::Bool` : Overwrite existing output directory (default: `false`).
- Recommended sidecar metadata keywords: `task_description` , `instructions` , `manufacturer` , `manufacturer_model` , `cap_manufacturer` , `cap_model` , `institution_name` , `institution_address` , `institution_department` , `eeg_ground` , `eeg_placement_scheme` .

source

Chapter 12

Data

12.1 Data

Data

This demo demonstrates the core data structures in EegFun.jl and how to access and subset data.

Core Data Structures

ContinuousData:
- Raw EEG time series from continuous recording
- Contains electrode data, triggers, sampling rate, metadata
- Single DataFrame

EpochData:
- Segmented trials around experimental events
- Organized by experimental conditions
- Vector of DataFrames (one per trial)

ErpData:
- Averaged event-related potentials
- Single DataFrame

Data Access Functions

all_data: Complete data including channels and metadata

channel_data: Only EEG channel columns

meta_data: Only metadata columns (time, sample, triggers, etc.)

extra_data: Derived/computed channels (EOG, artifacts, boolean flags, etc.)

Workflow Summary

This demo shows basic data operations:

Load and Structure Data

- Load raw BioSemi data
- Load electrode layout
- Create ContinuousData structure

Access Data Components

- Extract all data, channel data, and metadata
- Understand different data access patterns

Create Epochs

- Define epoch conditions around triggers
- Extract segmented trials
- Access epoch data with selection functions

Subset Data

- Subset by channel selection (specific electrodes)
- Subset by sample selection (time intervals)
- Create smaller datasets for analysis

Code Examples

```
# Demo: Data Manipulation
# Shows basic data access, subsetting, and manipulation functions.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# data/channel access functions
EegFun.all_data(dat)
EegFun.meta_data(dat)
EegFun.channel_data(dat)
EegFun.extra_data(dat) # empty

# some epoched data
epoch_cfg = [EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])]
```

```
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))

EegFun.all_data(epochs)
EegFun.all_data(epochs, epoch_selection = EegFun.epochs(1:2))
EegFun.meta_data(epochs)
EegFun.meta_data(epochs, epoch_selection = EegFun.epochs(1:2))
EegFun.channel_data(epochs)
EegFun.channel_data(epochs, epoch_selection = EegFun.epochs(1:2))

# We can subset out EegFun datatypes
dat_subset = EegFun.subset(dat, channel_selection = EegFun.channels([:Fp1, :Fp2]))
# only Fp1 and Fp2
EegFun.all_data(dat_subset)

dat_subset = EegFun.subset(dat, sample_selection = x -> x.sample .<= 10_000)
# first 10000 samples
EegFun.all_data(dat_subset)

dat_subset = EegFun.subset(dat, sample_selection = x -> x.time .<= 10)
# first 10 seconds
EegFun.all_data(dat_subset)
```

- API Reference

12.2 Data Access

Data Access

This demo shows how to inspect and navigate EegFun data structures using common access utilities.

Data Access Functions

EegFun provides several functions to access different parts of a data structure:

- **`all_data`** : Returns the complete DataFrame (metadata + channels + extra columns)
- **`meta_data`** : Returns only metadata columns (time, sample, triggers)
- **`channel_data`** : Returns only EEG channel columns
- **`extra_data`** : Returns derived/extra columns (EOG flags, artifact markers)
- **`channel_labels`** : Returns the channel names as a vector of Symbols

Quick Preview

For quick data inspection without viewing the entire dataset:

- **`head(dat)`** : Shows the first N rows (default: 5)
- **`tail(dat)`** : Shows the last N rows (default: 5)
- **`viewer(dat)`** : Opens data in VS Code's table viewer (falls back to console display)

Works Across Data Types

All access functions work consistently across ContinuousData, EpochData, and ErpData. For EpochData, you can add an `epoch_selection` parameter to access specific epochs.

Workflow Summary

This demo covers:

- Accessing all data, metadata, channel data, and extra columns
- Using head and tail for quick data preview
- Using viewer for VS Code integration
- Accessing data with epoch selection
- Querying data dimensions (n*epochs*, *n*values)

Code Examples

```
# Demo: Data Access and Inspection
# Shows how to inspect and navigate EegFun data structures using
# head, tail, viewer, and other access utilities.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# LOAD SOME DATA
############################################################################
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)



############################################################################
# DATA ACCESS FUNCTIONS
############################################################################

# Get all data as a DataFrame
EegFun.all_data(dat)

# Get only metadata columns (time, sample, triggers, etc.)
EegFun.meta_data(dat)
```

```
# Get only EEG channel columns
EegFun.channel_data(dat)

# Get extra/derived columns (EOG flags, etc.) — empty until calculated
EegFun.extra_data(dat)

# Get specific channel labels
EegFun.channel_labels(dat)

# Get the filename
EegFun.file_name(dat)



##########################################################################
# HEAD AND TAIL — Quick Data Preview
##########################################################################

# View the first rows (default: 5)
EegFun.head(dat)

# View the first 10 rows
EegFun.head(dat, n = 10)

# View the last rows (default: 5)
EegFun.tail(dat)

# View the last 20 rows
EegFun.tail(dat, n = 20)



##########################################################################
# VIEWER — VS Code Integration
##########################################################################

# If in VS Code, opens data in the table viewer; otherwise prints to console
EegFun.viewer(EegFun.all_data(dat))



##########################################################################
# DATA ACCESS WITH EPOCHS
##########################################################################

# Create some epochs
epoch_cfg = [EegFun.EpochCondition(name = "Cond1", trigger_sequences = [[1]])]
```

```
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))

# Access with epoch selection
EegFun.all_data(epochs, epoch_selection = EegFun.epochs(1:3))
EegFun.channel_data(epochs, epoch_selection = EegFun.epochs(1))

# Head and tail work on all data types
EegFun.head(epochs[1])
EegFun.tail(epochs[1])

# Get number of epochs, channels, etc.
EegFun.n_epochs(epochs[1])
EegFun.n_channels(epochs[1])
```

- API Reference

12.3 Selection Helpers

Selection Helpers

This demo shows how to use EegFun's selection helper functions for filtering, subsetting, and targeting specific parts of your data.

Selection Helpers

EegFun uses predicate-generator functions that create selection criteria:

- **`channels()`** : Select channels by name, vector, or regex pattern
- **`times()`** : Select time windows by start/end in seconds
- **`epochs()`** : Select epochs by index or range
- **`samples()`** : Select samples using custom predicates on metadata columns

How They Work

Selection helpers return functions (predicates) that are passed to `subset()` , and other functions via keyword arguments:

```
# EegFun.channels(:Cz, :Pz) returns a function that selects those channels
EegFun.subset(erp, channel_selection = EegFun.channels(:Cz, :Pz))
```

Composing Selections

Multiple selections can be combined in a single call:

```
EegFun.subset(epochs[1],
    channel_selection = EegFun.channels(:Cz),
    epoch_selection = EegFun.epochs(1:3),
    interval_selection = EegFun.times(0.0, 0.5),
```

```
)
```

This pattern is consistent across subsetting, plotting, and analysis functions.

Workflow Summary

This demo covers:

- Channel selection by name, vector, and regex pattern
- Time window selection with `times(start, end)`
- Epoch selection with `epochs(range)`
- Sample-level predicates with `samples()`
- Combining multiple selections in subset and plotting calls

Code Examples

```
# Demo: Selection Helpers
# Shows how to use EegFun's selection helper functions (channels, conditions,
# participants, epochs, times, samples) for filtering and subsetting data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

################################################################################
# SETUP
################################################################################
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

# Create epochs and ERPs
epoch_cfg =
    [EegFun.EpochCondition(name = "Cond1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Cond2", trigger_sequences = [[2]])]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))
erps = EegFun.average_epochs(epochs)



################################################################################
# CHANNELS() — Channel Selection
################################################################################
```

```
# Select all channels (default)
EegFun.subset(erps[1], channel_selection = EegFun.channels())

# Select specific channels by name
EegFun.subset(erps[1], channel_selection = EegFun.channels(:Cz, :Pz, :Fz))

# Select channels by vector
EegFun.subset(erps[1], channel_selection = EegFun.channels([:Fp1, :Fp2]))



##############################################################################
# TIMES() — Time Window Selection
##############################################################################

# Select all time points (default)
EegFun.subset(erps[1], interval_selection = EegFun.times())

# Select a time window (in seconds)
EegFun.subset(erps[1], interval_selection = EegFun.times(0.0, 0.5))

# Combine channel and time selection
sub = EegFun.subset(erps[1], channel_selection = EegFun.channels(:Cz, :Pz),
interval_selection = EegFun.times(0.1, 0.4))



##############################################################################
# EPOCHS() — Epoch Selection
##############################################################################

# Select all epochs (default)
EegFun.all_data(epochs, epoch_selection = EegFun.epochs())

# Select specific epoch indices
EegFun.all_data(epochs, epoch_selection = EegFun.epochs(1:5))

# Subset epochs from an EpochData object
EegFun.subset(epochs[1], epoch_selection = EegFun.epochs(1:3))



##############################################################################
# SAMPLES() — Sample-level Predicates
##############################################################################
```

```
# Custom predicate on metadata columns
EegFun.subset(dat, sample_selection = x -> x.sample .<= 5000)

# Time-based predicate
EegFun.subset(dat, sample_selection = x -> x.time .<= 5.0)



##########################################################################
# COMBINING SELECTIONS
##########################################################################

# Multiple selections at once
sub = EegFun.subset(
    epochs[1],
    channel_selection = EegFun.channels(:Cz),
    epoch_selection = EegFun.epochs(1:3),
    interval_selection = EegFun.times(0.0, 0.5),
)

# Use with plotting
EegFun.plot_erp(erps, channel_selection = EegFun.channels(:Cz, :Pz),
interval_selection = EegFun.times(-0.1, 0.8))
```

- API Reference

Chapter 13

Preprocessing

13.1 Filter

Filter

This demo shows how to apply temporal filters to EEG data to remove unwanted frequency components.

Why Filter?

- **High-pass filter** removes slow drifts and DC offsets that obscure the signal of interest
- **Low-pass filter** removes high-frequency noise (muscle activity, line noise harmonics)
- Together they define the frequency band you want to analyse

Key Functions

Function	Purpose	Typical Use
`highpass_filter!`	Remove frequencies below cutoff	0.1 Hz for ERPs, 1 Hz for ICA
`lowpass_filter!`	Remove frequencies above cutoff	30–40 Hz for ERPs
`create_highpass_filter`	Create a filter object for inspection	Visualize filter response
`create_lowpass_filter`	Create a filter object for inspection	Compare filter settings
`plot_filter`	Plot filter frequency response	Verify filter characteristics

IIR vs FIR

- **IIR** (default): Butterworth filter — fast, minimal parameters, good for most uses
- **FIR**: Windowed-sinc filter — linear phase, better when precise timing matters

Workflow Summary

High-Pass Filtering

- Standard 0.1 Hz for ERP analysis
- Stronger 1 Hz for ICA preprocessing

Low-Pass Filtering

- 30 Hz for typical ERP analysis
- Anti-aliasing before resampling

Channel-Specific Filtering

- Apply filters to selected channels only

Filter Inspection

- Create filter objects and visualise frequency response
- Compare IIR vs FIR characteristics

Code Examples

```
# Demo: Filtering
# Shows how to apply high-pass and low-pass filters to EEG data,
# compare IIR and FIR methods, and visualize filter characteristics.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# LOAD DATA
############################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)



############################################################################
# HIGH-PASS FILTER
############################################################################

# Remove slow drifts with a 0.1 Hz high-pass (standard for ERPs)
EegFun.highpass_filter!(dat, 0.1)
# EegFun.highpass_filter!(dat, 0.1, filter_method="fir")

# Stronger high-pass for ICA preprocessing (1 Hz)
dat_ica = copy(dat)
```

```
EegFun.highpass_filter!(dat_ica, 1.0)


#########################################################################
# LOW-PASS FILTER
#########################################################################

# Remove high-frequency noise (30 Hz low-pass)
EegFun.lowpass_filter!(dat, 30.0)

# Anti-aliasing filter before downsampling (40 Hz)
EegFun.lowpass_filter!(dat, 40.0, order = 3)


#########################################################################
# FILTER SPECIFIC CHANNELS
#########################################################################

# Apply filter to only certain channels
EegFun.lowpass_filter!(dat, 30.0, channel_selection =
EegFun.channels([:Cz, :Pz, :Fz]))


#########################################################################
# IIR vs FIR FILTERS
#########################################################################

# Default: IIR (Butterworth) — fast, good for most uses
EegFun.highpass_filter!(dat, 0.1, filter_method = "iir", order = 1)

# FIR (windowed sinc) — linear phase, better for time-domain analysis
EegFun.highpass_filter!(dat, 0.1, filter_method = "fir")


#########################################################################
# VISUALIZE FILTER RESPONSE
#########################################################################

# Create a filter object for inspection
filt = EegFun.create_highpass_filter(0.1, 2048.0)
EegFun.plot_filter_response(filt)

filt_lp = EegFun.create_lowpass_filter(30.0, 2048.0)
EegFun.plot_filter_response(filt_lp)
```

```
# Compare IIR vs FIR
filt_iir = EegFun.create_highpass_filter(0.1, 2048.0, filter_method = "iir")
filt_fir = EegFun.create_highpass_filter(0.1, 2048.0, filter_method = "fir")
EegFun.plot_filter_response(filt_iir)
EegFun.plot_filter_response(filt_fir)
```

- API Reference

13.2 Rereference

Rereference

This demo demonstrates applying different re-referencing schemes to EEG data.

What is Re-referencing?

Re-referencing changes the voltage baseline by subtracting a reference signal from all electrodes. Since EEG measures potential differences, the choice of reference affects:

- Signal amplitudes and topographies
- Component interpretations
- Statistical comparisons

Why Re-reference?

Recording reference: The reference used during data acquisition (e.g., Cz, linked mastoids, or manufacturer default)

Analysis reference: The reference appropriate for your research question and field conventions

Re-referencing allows you to change from the recording reference to a more appropriate analysis reference.

Common Reference Schemes

Average Reference (`:avg`):

- Subtracts mean of all electrodes from each channel
- Assumes zero average scalp potential
- Most common for high-density EEG (64+ channels)
- Reference-free topographies

Mastoid Reference:

- References to average of left/right mastoid electrodes (e.g., `:M1` , `:M2`)
- Common in ERP research
- Reduces contamination from reference site activity

Single Electrode Reference:

- References to a specific electrode (e.g., `:Cz` , `:Fz`)
- Useful for specific research questions

- Activity at reference site becomes invisible (single reference electrode = 0)

Best Practices

- Apply the same reference across all conditions and participants
- Re-reference **before** interpolating bad channels (bad channels affect average reference)
- Document your reference choice in publications
- Use average reference for high-density recordings

Re-referencing is Reversible

You can change references multiple times. EegFun tracks the current reference and updates accordingly.

Workflow Summary

This demo shows re-referencing workflows:

Continuous Data

- Load and preprocess data
- Apply different reference schemes (`:Fp1` , `:AF7`)
- Visualize in databrowser to see effects

Epoched Data

- Create epochs from continuous data
- Apply references to segmented trials
- Data structure automatically updated

ERP Data

- Average epochs into ERPs
- Apply references to averaged data
- Reference changes preserved across data types

Code Examples

```
# Demo: Rereferencing
# Shows different rereferencing methods (average, linked mastoids, specific
channel reference).

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie
```

```
# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.highpass_filter!(dat, 1)

# Rereference to :Fp1
EegFun.rereference!(dat, :Fp1)
dat.data

# Rereference to :AF7
EegFun.rereference!(dat, :AF7)
dat.data

# Rereference to :AF7
EegFun.rereference!(dat, [:Fp1, :AF7])
dat.data


# We can also see the influence of reference using the databrowser
EegFun.plot_databrowser(dat)


##############################
# Epoching
##############################

# Create some epoched data
epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))  # -200 to 1000 ms

EegFun.rereference!(epochs, :Fp1)
EegFun.all_data(epochs[1]) # first condition
EegFun.all_data(epochs[2]) # second condition
```

```
##############################
# ERPs
##############################
# ERPs
erps = EegFun.average_epochs(epochs)

EegFun.rereference!(erps, :Fp1)
EegFun.all_data(erps[1])
EegFun.all_data(erps[2])

EegFun.rereference!(erps, :F1)
EegFun.all_data(erps[1])
EegFun.all_data(erps[2])
```

- API Reference

13.3 Resample

Resample

This demo shows how to change the sampling rate of EEG data through polyphase resampling.

What is Resampling?

Resampling changes the number of samples per second (Hz) in your recording. `EegFun.resample` and `EegFun.resample!` resample continuous, epoched, or ERP data to a positive integer target sampling rate (e.g., 2048 Hz → 512 Hz or 256 Hz) using `DSP.resample` polyphase anti-aliasing filtering with mirror padding.

Why Resample?

Downsampling benefits:

- **Reduce memory & disk footprint**: Fewer samples per channel
- **Speed up downstream steps**: Faster ICA, filtering, time-frequency decomposition, and MVPA decoding
- **Harmonize multi-site cohorts**: Bring recordings with different acquisition rates onto a common sampling grid

The Nyquist Theorem

The target sampling rate must be **strictly greater than 2× the highest frequency** of interest:

- **256 Hz sampling** → Nyquist = 128 Hz (well-suited for standard ERP analyses < 40 Hz)
- **512 Hz sampling** → Nyquist = 256 Hz (well-suited for time-frequency and gamma oscillations up to 100 Hz)
- **1000 Hz sampling** → Nyquist = 500 Hz

Anti-Aliasing & Mirror Padding

`EegFun.resample` automatically constructs a polyphase anti-aliasing filter (`DSP.resample_filter`) and applies mirror padding at segment boundaries to suppress edge transients:

```
dat_512 = EegFun.resample(dat, 512) # Resample to 512 Hz
dat_256 = EegFun.resample(dat, 256) # Resample to 256 Hz
```

Trigger & Metadata Preservation

Event triggers and extra channels (such as EOG or boolean artifact flags) are mapped to the new sample grid automatically:

```
EegFun.trigger_count(dat)      # Original trigger counts
EegFun.trigger_count(dat_256)  # Preserved after resampling
```

Workflow Summary

1. **Load continuous data** at the acquisition sampling rate
2. **Inspect current rate** with `EegFun.sample_rate(dat)`
3. **Resample to target Hz** (`EegFun.resample(dat, 512)` or `EegFun.resample!(dat, 256)`)
4. **Verify triggers preserved** with `EegFun.trigger_count(dat_256)`

Code Examples

```
# Demo: Resampling
# Shows how to resample EEG data to a target sampling rate (Hz).

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

EegFun.sample_rate(dat)      # current sample rate (Hz)
EegFun.trigger_count(dat)    # current triggers in file
EegFun.plot_databrowser(dat) # view current data
```

```
dat_512 = EegFun.resample(dat, 512) # resample to 512 Hz (polyphase anti-aliasing)
EegFun.sample_rate(dat_512)         # 512 Hz
EegFun.trigger_count(dat_512)       # triggers are preserved
EegFun.plot_databrowser(dat_512)    # view resampled data

dat_256 = EegFun.resample(dat, 256) # resample to 256 Hz
EegFun.sample_rate(dat_256)         # 256 Hz
EegFun.trigger_count(dat_256)       # triggers are preserved
EegFun.plot_databrowser(dat_256)    # view resampled data
```

- API Reference

13.4 Baseline

Baseline

This demo demonstrates baseline correction methods for ERP data, including different baseline intervals and correction approaches for continuous, epoch, and ERP data.

What is Baseline Correction?

Baseline correction removes the pre-stimulus mean from each trial, ensuring that activity is measured relative to a neutral reference period. This is essential for ERP analysis because:

- **Removes slow drifts**: Eliminates DC offsets that vary across trials
- **Standardizes pre-stimulus activity**: Ensures comparable starting points across conditions
- **Enables amplitude interpretation**: Makes measurements interpretable relative to baseline activity

Common Baseline Windows

Pre-stimulus period (most common):

- Standard: −200 to 0 ms before stimulus onset
- Captures typical pre-stimulus activity
- Assumes stable activity before stimulus

Baseline Methods

Mean correction (default):

- Subtract the mean voltage during baseline period
- Standard approach in ERP research
- Appropriate when baseline is stable

Best Practices

Baseline interval selection:

- Choose based on experimental design and paradigm timing
- Standard: −200 to 0 ms for most ERP paradigms
- Document chosen interval in methods section
- Keep consistent across conditions and participants

Timing considerations:

- Ensure baseline period is free from stimulus overlap
- Avoid including activity from previous trials
- Verify baseline period is stable (visually inspect)

When to baseline:

- Apply after filtering and before averaging
- Can apply to epoched or average ERPs

IMPORTANT

Baseline correction assumes the pre-stimulus period represents neutral brain activity. This assumption may be violated in designs with:

- Very short inter-trial intervals
- Anticipatory activity (e.g., motor preparation)
- Sustained activity from previous trials

Workflow Summary

This demo demonstrates baseline correction for different data types:

Baseline Continuous Data

- Load raw BioSemi data
- Apply baseline to entire continuous recording
- Visualize DC offset removal in databrowser
- Baseline to specific timepoint (t=0)

Baseline Epoch Data

- Extract epochs around experimental events
- Apply baseline at different timepoints (t=0, t=0.5)
- Visualize effect on individual trials
- Compare baseline choices

Baseline ERP Data

- Average epochs into ERPs
- Baseline to single timepoint (t=0, t=0.5)
- Baseline to time interval (-200 to 0 ms)
- Compare effects across baseline choices

Code Examples

```
# Demo: Baseline Correction
# Shows baseline correction on continuous, epoched, and ERP data using
# different baseline intervals and visualizes the effects.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Plot
EegFun.plot_databrowser(dat) # DC shift visible

# Baseline stuff and replot
EegFun.baseline!(dat)
EegFun.plot_databrowser(dat) # now zero mean over all samples (reduced DC shift)

# NB. such DC shift is removed when applying a high-pass filter (e.g., 0.1 Hz)

# baseline to timepoint
EegFun.baseline!(dat, (0, 0)); # timepoint = 0
EegFun.plot_databrowser(dat)

# EpochData
# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# Create some epoched data
epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
```

```
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))  # -200 to 1000 ms

EegFun.plot_epochs(epochs, channel_selection = EegFun.channels([:Fp1]))

EegFun.baseline!(epochs, (0.0, 0.0)) # baseline to t=0
EegFun.plot_epochs(epochs, channel_selection = EegFun.channels([:Fp1]))

EegFun.baseline!(epochs, (0.5, 0.5)) # baseline to t=0.5 (just for demo purposes!)
EegFun.plot_epochs(epochs, channel_selection = EegFun.channels([:Fp1]))

EegFun.baseline!(epochs, (-0.2, 0.0)) # baseline from -200 to 0 ms t=0 (common
pre-event baseline)
EegFun.plot_epochs(epochs, channel_selection = EegFun.channels([:Fp1]))



# Create some ERP data
erps = EegFun.average_epochs(epochs)

EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Fp1]))

EegFun.baseline!(erps, (0.0, 0.0)) # baseline to t=0
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Fp1]))

EegFun.baseline!(erps, (0.5, 0.5)) # baseline to t=0.5 (just for demo purposes!)
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Fp1]))

EegFun.baseline!(erps, (-0.2, 0)) # baseline to t=-0.2 to 0.0
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Fp1]))

# We can see the influence of baseline interval interactively using the
plot_erp_measurement_gui
EegFun.plot_erp_measurement_gui(erps)
```

- API Reference

13.5 Mirror

Mirror

This demo demonstrates mirror padding, a technique for extending time-series data by reflecting it at the edges.

What is Mirror Padding?

Mirror padding extends your data at the beginning and/or end by creating a reflected copy:

```
Original:     [A B C D E]
Pre-padding:  [E D C B A | A B C D E]
Post-padding: [A B C D E | E D C B A]
Both:         [E D C B A | A B C D E | E D C B A]
```

This creates smooth, continuous edges that avoid discontinuities.

Why Use Mirror Padding?

Edge Artifact Reduction:

Filtering operations can introduce artifacts at the edges of your data because:

- Filters "see" discontinuities at boundaries
- Edge effects propagate inward
- Lost information at boundaries

Mirror padding solves this by:

- Creating smooth, continuous edges
- Allowing filters to process edges naturally
- Removing padding after filtering to discard corrupted edge regions

Typical Workflow:

1. **Mirror** your data (add padding)
2. **Filter** (artifacts now occur in padding region)
3. **Remove padding** (discard corrupted edges, keep clean data)

Padding Options

The `mirror()` function accepts three padding modes:

Mode	Description
:pre	Mirror before the start
:post	Mirror after the end
:both	Mirror at both ends

Use Cases

High-pass filtering:

High-pass filters are particularly sensitive to edge effects. Mirror padding helps preserve data near epoch boundaries.

Time-frequency analysis:

Wavelet transforms and spectral analysis benefit from smooth edges.

Concatenated epochs:

When processing multiple epochs sequentially, padding prevents edge artifacts between epochs.

Workflow Summary

This demo shows:
1. **Load and preprocess** continuous data
2. **Extract epochs** (-200 to 1000 ms)
3. **Apply mirror padding** with different modes (:pre, :post, :both)
4. **Visualize padded epochs** to see the effect
5. **Average to ERPs** and apply padding
6. **Visualize padded ERPs**

The visual comparison clearly shows how padding extends the time interval while maintaining smooth edges.

Code Examples

```
# Demo: Mirror Padding
# Shows how to pad epoch and ERP data with mirrored edges (:pre, :post, :both)
# to reduce edge artifacts when filtering.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# Create some epoched data
epoch_cfg = [EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))  # -200 to 1000 ms

EegFun.plot_epochs(epochs, channel_selection = EegFun.channels([:Fp1]))
```

```
epochs_new = EegFun.mirror(epochs, :pre)
EegFun.plot_epochs(epochs_new, channel_selection = EegFun.channels([:Fp1]))

epochs_new = EegFun.mirror(epochs, :post)
EegFun.plot_epochs(epochs_new, channel_selection = EegFun.channels([:Fp1]))

epochs_new = EegFun.mirror(epochs, :both)
EegFun.plot_epochs(epochs_new, channel_selection = EegFun.channels([:Fp1]))

# ERPs
erps = EegFun.average_epochs(epochs)

EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Fp1]))

erps_new = EegFun.mirror(erps, :pre)
EegFun.plot_erp(erps_new, channel_selection = EegFun.channels([:Fp1]))

erps_new = EegFun.mirror(erps, :post)
EegFun.plot_erp(erps_new, channel_selection = EegFun.channels([:Fp1]))

erps_new = EegFun.mirror(erps, :both)
EegFun.plot_erp(erps_new, channel_selection = EegFun.channels([:Fp1]))
```

- API Reference

13.6 Triggers

Triggers

This demo shows how to inspect trigger (event marker) data and search for trigger sequences in EEG recordings.

Why Triggers?

Triggers mark experimental events (stimulus onsets, responses, conditions) in the continuous EEG recording. Before epoching, you need to verify that triggers are present, correctly coded, and occur at the expected frequencies.

Key Functions

Function	Purpose
`trigger_count`	Count occurrences of each trigger value
`search_sequence`	Find indices where trigger patterns occur
`plot_trigger_overview`	Visualise trigger distribution

`plot_trigger_timing`	Visualise trigger timing

Sequence Search Features

- **Single values**: find all onsets of trigger `1`
- **Multi-trigger sequences**: find `[1, 2]` (cue followed by target)
- **Wildcards**: `:any` matches any trigger between specific values
- **Ranges**: `1:5` matches triggers 1 through 5
- **Multiple sequences (OR logic)**: `[[1, 2], [3, 4]]` finds either sequence

Workflow Summary

Trigger Counting

- View raw and cleaned trigger counts in a formatted table

Sequence Searching

- Find single triggers, multi-trigger sequences, and patterns with wildcards

Visualisation

- Plot trigger timing and distribution

Code Examples

```
# Demo: Triggers
# Shows how to inspect, count, and search for trigger sequences in EEG data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

################################################################################
# LOAD DATA
################################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)
```

```
#########################################################################
# TRIGGER OVERVIEW
#########################################################################

# Count how often each trigger occurs (raw and cleaned)
trigger_info = EegFun.trigger_count(dat)

# View the trigger table
trigger_info

# Access the underlying DataFrame
trigger_info.data


#########################################################################
# SEARCH FOR TRIGGER SEQUENCES
#########################################################################

# Find all onsets of trigger value 1
idx = EegFun.search_sequence(dat.data.trigger, 1)

# Find a two-trigger sequence (e.g., warning cue followed by target)
idx = EegFun.search_sequence(dat.data.trigger, [1, 2])

# Find a sequence while ignoring zeros between triggers
idx = EegFun.search_sequence(dat.data.trigger, [1, 2], ignore_values = [0])


#########################################################################
# WILDCARDS AND RANGES
#########################################################################

# Use :any wildcard to match any trigger between two specific triggers
idx = EegFun.search_sequence(dat.data.trigger, [1, :any, 2])

# Use ranges to match a set of trigger values
idx = EegFun.search_sequence(dat.data.trigger, [1:5])     # triggers 1 through 5
idx = EegFun.search_sequence(dat.data.trigger, [1:3, 10:12])  # multiple ranges


#########################################################################
# MULTIPLE SEQUENCES (OR LOGIC)
#########################################################################

# Find onsets where any of several sequences occurs
```

```
idx = EegFun.search_sequence(dat.data.trigger, [[1, 2], [3, 4]])



######################################################################
# VISUALIZE TRIGGERS
######################################################################

# Plot trigger timing overview
EegFun.plot_trigger_overview(dat)
EegFun.plot_trigger_timing(dat)
```

- API Reference

13.7 Epoch Extraction

Epoch Extraction

This demo explores the core functionality of segmenting continuous data into epochs (or trials) based on event markers, and basic manipulation of these segments.

What are Epochs?

Epoching is the process of extracting specific time intervals around events (e.g., stimuli or responses) from a continuous recording. This allows for epoch-based analysis and averaging to reveal Event-Related Potentials (ERPs).

Key features of EegFun's epoching:

- Time-relative segmentation
- Multi-condition definition via trigger sequences

Capabilities

- **Flexible Extraction**: Define intervals relative to trigger onset (e.g., −200ms to +1000ms).
- **Condition Matching**: Match triggers or sequences of triggers to specific condition names.

Workflow Summary

1. **Data Preparation**: Load continuous data, apply layout, and perform basic preprocessing (filtering, re-referencing).
2. **Define Conditions**: Use `EpochCondition` to specify which triggers belong to which experimental condition.
3. **Extraction**: Use `extract_epochs()` to create segments. This returns a collection of `EpochData` objects.

Code Examples

```
# Demo: Epoch Extraction
# Shows how to extract epochs from continuous data using trigger sequences.
```

```
# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# Read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# Read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# Create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Minimal preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# Define simple epoch condition
epoch_cfg = EegFun.EpochCondition(name = "Trigger1", trigger_sequences = [[1]])
# Extract epochs (-200ms to 1000ms around triggers)
epoch = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0));

# Or multiple epoch conditions
epoch_cfg = [
    EegFun.EpochCondition(name = "Trigger1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Trigger2", trigger_sequences = [[2]]),
]
# Extract epochs (-200ms to 1000ms around triggers)
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0));

# Plot individual epochs for first condition for three channels
EegFun.plot_epochs(epochs[1], channel_selection =
EegFun.channels([:Fp1, :Cz, :Oz]))

# Plot different channel selections
EegFun.plot_epochs(epochs[1], channel_selection = EegFun.channels([:Fp1]))
EegFun.plot_epochs(epochs[2], channel_selection = EegFun.channels([:Cz]))

# Baseline correction to stimulus onset (t=0)
EegFun.baseline!(epochs, (-0.2, 0.0))
EegFun.plot_epochs(epochs[1], channel_selection = EegFun.channels([:Cz]))
```

```
# Compare both conditions on same plot
erps = EegFun.average_epochs(epochs)
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Cz]))
```

- API Reference

13.8 Channel Operations

Channel Operations

This demo shows how to manipulate channels by averaging groups together, computing differences between them, and deleting unwanted channels.

Key Functions

Function	Purpose	Example
`channel_difference!`	Subtract one channel group from another	EOG calculation
`calculate_eog_channels!`	Calculate both vEOG and hEOG from config	Convenience wrapper
`channel_average!`	Average channel groups into new columns	ROI means
`channel_delete!`	Remove channels from data and layout	Drop non-EEG channels

Common Use Cases

- **EOG channels**: `channel_difference!` calculates vEOG (Fp1/Fp2 minus IO1/IO2) and hEOG (F9 minus F10)
- **ROI averages**: `channel_average!` creates region-of-interest means (e.g., frontal, parietal)
- **Cleanup**: `channel_delete!` removes non-EEG channels (e.g., photodiode, reference)

Workflow Summary

Channel Difference

- Calculate EOG channels from electrode pairs
- Create arbitrary difference channels

EOG Configuration

- Use `EogConfig` to calculate both EOG channels in one call

Channel Averaging

- Average channel groups with custom labels
- Reduce dataset to only averaged channels

Channel Deletion

- Remove single or multiple channels (mutating and non-mutating versions)

Code Examples

```
# Demo: Channel Operations
# Shows how to average, delete, and compute differences between channels.
# Covers channel_average!, channel_delete!, and channel_difference!.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# LOAD DATA
############################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)

dat = EegFun.create_eegfun_data(dat, layout)

# minimal preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)



############################################################################
# CHANNEL DIFFERENCE (EOG CALCULATION)
############################################################################

# Vertical EOG: average of upper channels minus average of lower channels
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
)
```

```
# Horizontal EOG: left minus right
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
)



#########################################################################
# EOG WITH CONFIGURATION
#########################################################################

# Calculate both EOG channels from a configuration
eog_cfg = EegFun.EogConfig(
    vEOG_criterion = 50.0,
    hEOG_criterion = 30.0,
    vEOG_channels = [["Fp1", "Fp2"], ["IO1", "IO2"], ["vEOG"]],
    hEOG_channels = [["F9"], ["F10"], ["hEOG"]],
)
EegFun.calculate_eog_channels!(dat, eog_cfg)



#########################################################################
# CHANNEL AVERAGING
#########################################################################

# Average all channels into a single column
EegFun.channel_average!(dat)

dat.data[!, :avg] # average of all channels

# Average specific channel groups (average of Fp1 and Fp2, average of O1 and O2
with
# default output labels of :Fp1_Fp2 and :O1_O2)
EegFun.channel_average!(dat, channel_selections = [EegFun.channels([:Fp1, :Fp2]),
EegFun.channels([:O1, :O2])])

# Average with custom output labels
EegFun.channel_average!(
    dat,
    channel_selections = [EegFun.channels([:Fp1, :Fp2]),
    EegFun.channels([:O1, :O2])],
    output_labels = [:avg1, :avg2],
```

```
)

# Reduce to just the averaged channels (drops originals)
dat_reduced = EegFun.channel_average(dat, channel_selections =
[EegFun.channels([:Fp1, :Fp2])], reduce = true)



#########################################################################
# CHANNEL DELETION
#########################################################################

# Delete a single channel
EegFun.channel_delete!(dat, :avg)
EegFun.channel_delete!(dat, [:Fp1_Fp2, :O1_O2])

# Delete multiple channels
EegFun.channel_delete!(dat, [:vEOG, :hEOG])

# Non-mutating version (returns a copy)
dat_subset = EegFun.channel_delete(dat, [:Fp1, :Fp2])
```

- API Reference

13.9 Channel Metrics

Channel Metrics

This demo demonstrates how to calculate and visualize channel quality metrics for identifying bad channels and detecting artifacts.

What are Channel Metrics?

Channel quality metrics quantify various characteristics of the EEG signal at each electrode. These metrics help identify problematic channels that may need repair or rejection before analysis.

Variance:

- Measures overall signal power/amplitude variability at each channel
- High variance may indicate artifacts (muscle, movement)
- Very low variance suggests poor contact or disconnected electrode

Kurtosis:

- Measures the distribution shape (tailedness) of the signal
- High kurtosis indicates spiky signals with extreme values (artifacts)

Correlation:

- Average correlation between a channel and its spatial neighbors
- Low correlation suggests the channel is behaving differently from nearby electrodes

- Detects bad contacts, bridging, or electrode-specific noise

Joint Probability:

- Statistical outlier detection across channels simultaneously
- Identifies channels that deviate from the typical multi-channel pattern
- Combines information from variance and correlation metrics

Use Cases

Quality control:

- Identify bad channels before epoching or averaging
- Screen data quality during or after acquisition
- Track bad channels across participants - if a specific electrode is consistently bad across multiple participants, this may indicate a broken electrode or faulty electrode set that needs replacement

Workflow Summary

This demo shows channel quality assessment workflows:

Basic Channel Metrics

- Load and preprocess raw data (average reference, high-pass filter)
- Calculate channel joint probability metrics
- Identify channels with extreme values

EOG Correlation Analysis

- Compute vertical and horizontal EOG channels
- Detect EOG onsets automatically
- Calculate correlation between EEG channels and EOG
- Partition bad channels into EOG-related vs. non-EOG artifacts

Metric Interpretation

- Add z-score columns for standardized thresholds
- Identify channels that are artifact-related vs. bad contacts
- Guide decisions on repair (interpolation) vs. rejection

Code Examples

```
# Demo: Channel Metrics
# Shows how to compute and visualize channel-wise metrics (variance, kurtosis,
etc.).


# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")
```

```
using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

dat = EegFun.create_eegfun_data(dat, layout)

# minimal preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# extreme value bool column
EegFun.is_extreme_value!(dat, 100)

# channel joint probability
channel_joint_probability = EegFun.channel_joint_probability(dat)
channel_joint_probability = EegFun.channel_joint_probability(dat, sample_selection
= EegFun.samples_not(:is_extreme_value_100))

# Calculate EOG signals
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
); # vertical EOG = mean(Fp1, Fp2) - mean(IO1, I02)
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
); # horizontal EOG = F9 - F10

EegFun.detect_eog_onsets!(dat, 50, :vEOG, :is_vEOG)
EegFun.detect_eog_onsets!(dat, 30, :hEOG, :is_hEOG)


# Calculate correlations between all channels and EOG channels
cm = EegFun.correlation_matrix_dual_selection(
```

```
    dat,
    sample_selection = EegFun.samples(),  # All samples
    channel_selection1 = EegFun.channels(),  # All EEG channels
    channel_selection2 = EegFun.channels([:vEOG, :hEOG]),  # EOG channels
)
EegFun.add_zscore_columns!(cm)

bad_channels = [:Fp1, :AF3]
non_eog_related, eog_related =
EegFun.partition_channels_by_eog_correlation(bad_channels, cm)
```

- API Reference

13.10 Channel Repair

Channel Repair

This demo demonstrates channel interpolation methods for repairing bad electrodes using spatial interpolation techniques.

What is Channel Repair?

Channel repair (interpolation) estimates the signal at bad electrodes using data from neighboring good channels. This preserves data quality while maintaining electrode count for spatial analyses.

When to Repair Channels

Channel repair is appropriate when:

- Individual electrodes have poor contact/noisy signal
- Isolated channels show excessive noise or artifacts
- A small number of channels are affected while most data is clean
- You need to maintain electrode count for spatial analyses (e.g., source localization)

Do not repair when:

- Too many channels are bad
- The entire dataset is noisy
- Bad channels cluster together spatially

Interpolation Methods

Neighbor Interpolation:

- Weighted average of spatially nearby electrodes
- Fast and computationally efficient
- Good for isolated bad channels
- Requires neighbor calculation based on distance threshold

Spherical Spline:

- Uses spherical spline functions to model scalp potential distribution

Best Practices

Timing:
- Identify bad channels using quality metrics first
- Repair before re-referencing (reference calculation needs all channels)
- Repair before averaging or statistical analysis

Limits:
- Avoid interpolating too many channels
- Consider rejecting datasets with an excessive number of bad channels

Validation:
- Visually verify repair quality (before/after comparison)
- Check that interpolated channels match neighbors

Workflow Summary

This demo shows channel repair workflows:

Identify Channels to Repair

- Load and preprocess data
- Select specific channels for demonstration
- Verify channels exist in the dataset

Calculate Neighbor Relationships

- Compute spatial neighbors based on 3D electrode positions
- Use distance threshold to define neighborhood
- Required for neighbor interpolation method

Apply Interpolation

- Test neighbor interpolation method
- Test spherical spline method
- Store original data for comparison

Validate Repair Quality

- Compare interpolated vs. original data
- Verify that interpolation changed the data as expected
- Use visual inspection to confirm successful repair

Code Examples

```
# Demo: Channel Repair
```

```
# Shows neighbor interpolation and spherical spline for repairing bad channels.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

dat = EegFun.create_eegfun_data(dat, layout)

# minimal preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)

EegFun.plot_databrowser(dat)

# Select a channel to repair and make this channel noisy!
channel_to_repair = :Cz
dat.data[!, channel_to_repair] .+= randn(size(dat.data[:, channel_to_repair])) *
200 # v. noisy!

# We can now see this noisy channel in the databrowser
# NB. we can actually press "R" and select Cz and apply the repair in the browser
EegFun.plot_databrowser(dat)

# Try neighbor interpolation
EegFun.repair_channels!(dat, [channel_to_repair], method
= :neighbor_interpolation)

# Cz is now repaired
EegFun.plot_databrowser(dat)

# Try neighbor interpolation
EegFun.repair_channels!(dat, [channel_to_repair], method = :spherical_spline)

# Cz is now repaired
EegFun.plot_databrowser(dat)
```

- API Reference

13.11 Channel Summary

Channel Summary

This demo demonstrates how to generate summary statistics across all channels for data quality assessment.

What is Channel Summary?

Channel summary provides aggregate statistics across all EEG channels, offering a quick overview of data characteristics. This complements channel-specific metrics by revealing patterns across the entire montage.

Statistical measures:

- Mean amplitude per channel
- Variance/standard deviation
- Minimum and maximum values
- Sample counts

Use Cases

Data quality overview:

- Quick assessment of recording quality across all channels
- Identify systematic patterns or issues across the montage
- Compare data quality before and after preprocessing

Channel comparison:

- Identify channels with systematically different properties
- Detect asymmetries or systematic biases in the recording
- Verify that preprocessing affected channels as expected

Workflow Summary

This demo shows channel summary analysis:

Generate Initial Summary

- Load and preprocess data (average reference, high-pass filter)
- Calculate summary statistics across all channels
- Display results using formatted table

Summary with Sample Selection

- Mark extreme values for exclusion
- Recalculate summary excluding artifacts

- Compare clean vs. raw summary statistics

Code Examples

```
# Demo: Channel Summary
# Shows how to create summary statistics for channels across epochs.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

dat = EegFun.create_eegfun_data(dat, layout)

# minimal preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# summary statistics across all channels
summary = EegFun.channel_summary(dat)
EegFun.log_pretty_table(summary; title = "Initial Channel Summary")

# summary statistics across all channels excluding v. extreme values
EegFun.is_extreme_value!(dat, 200);
summary = EegFun.channel_summary(dat, sample_selection =
EegFun.samples_not(:is_extreme_value_200))
EegFun.log_pretty_table(summary; title = "Channel Summary (excluding extreme
values)")

# summary statistics across all Midline channels via predicate selection
summary = EegFun.channel_summary(dat, channel_selection = EegFun.channels(x ->
endswith.(string.(x), "z")))
EegFun.log_pretty_table(summary; title = "Channel Summary (Midline)")
```

- API Reference

13.12 ICA

ICA

This demo demonstrates the complete ICA workflow for decomposing EEG into independent components, identifying artifacts, and removing them.

What is ICA?

Independent Component Analysis (ICA) is a blind source separation technique that decomposes multi-channel EEG into maximally independent sources:

- **Decomposes mixed signals**: Separates brain activity from artifacts based on statistical independence
- **Identifies artifact sources**: Eye movements, blinks, heartbeat, muscle activity, line noise

How ICA Works (Conceptually)

EEG is a mixture of sources:

Recorded EEG = Brain + Eye movements + Muscle + Heartbeat + Noise

ICA finds the unmixing matrix that separates these:

Components = Unmixing × Recorded EEG

Each component represents one independent source (e.g., one component might be pure eye blinks, another pure alpha rhythm).

ICA Workflow

1. Preprocessing

:

- Apply high-pass filter (≥1 Hz recommended)
- Detect and reject extreme artifacts or extreme sections of data
- Optionally remove bad channels
- Create EOG channels for identification

2. Run ICA

:

- `run_ica()` on continuous or epoched data
- Choose algorithm: `:infomax` (default) or `:infomax_extended` (for sub-gaussian sources)
- Can use subset of data to speed up (20-50% often sufficient)
- Exclude extreme artifacts from decomposition

3. Identify Artifact Components

:

- `identify_components()` - all artifact types automatically
- Or identify individually:
  - `identify_eog_components()` - eye movements and blinks
  - `identify_ecg_components()` - heartbeat

  - `identify_line_noise_components()` - 50/60 Hz interference
  - `identify_spatial_kurtosis_components()` - channel noise

4. Visualize Components

:

- `plot_topography()` - spatial patterns (component topographies)
- `plot_ica_component_activation()` - time courses
- `plot_ica_component_spectrum()` - frequency content
- `plot_databrowser()` - interactive exploration

5. Remove Artifacts

:

- `EegFun.subtract_ica_components!` - subtract artifact components
- `EegFun.add_ica_components!` - add back if needed (for validation)

Component Identification

EOG (Eye) Components:

- **Spatial pattern**: Strong frontal (blinks) or side (saccades) loading
- **Time course**: Correlates with EOG channels
- **Frequency**: Low frequency (<5 Hz)

ECG (Heartbeat) Components:

- **Spatial pattern**: Localized or bilateral temporal
- **Time course**: Regular rhythm (60-100 bpm)
- **Frequency**: Spectral peak at heart rate

Line Noise Components:

- **Spatial pattern**: Diffuse or specific to noisy channels
- **Frequency**: Strong peak at 50 or 60 Hz and harmonics

Muscle Components:

- **Spatial pattern**: Temporal or frontal regions
- **Frequency**: High frequency (>20 Hz)
- **Time course**: Brief bursts

Brain Components:

- **Alpha rhythm**: Posterior, 10 Hz, eyes-closed modulation
- **Mu rhythm**: Central, 10 Hz, motor-related
- **Task-related**: Modulated by experimental conditions

Algorithms

Infomax (default):

- Fast and robust
- Optimized for super-gaussian sources (most artifacts)

- Good general-purpose choice

Infomax Extended:

- Handles both super- and sub-gaussian sources
- Better for mixed artifact types
- Slightly slower

Best Practices

Data requirements:

- **Channels**: ≥30 recommended (more data = better separation

)

- **Duration**: Several minutes minimum
- **Preprocessing**: High-pass filter ≥1 Hz to remove slow drifts, remove extreme artifacts

Component selection:

- Use automated identification as starting point
- Visually inspect topographies and activations
- When in doubt, err on side of caution (don't remove)
- Document which components were removed

Validation:

- Compare before/after data quality
- Check that brain components preserved
- Verify artifact reduction in problematic channels/trials

When to Use ICA

Recommended:

- Eye movement artifacts in visual tasks
- Muscle artifacts in speech/motor tasks
- Heartbeat artifacts in source localization
- Isolating specific brain rhythms

Not recommended:

- Very small datasets (<30 channels, <1 minute)
- Electrode bridging or poor contact (interpolate instead)
- When simple rejection suffices (e.g., few contaminated trials)

Continuous vs. Epoched ICA

Continuous data:

- Use full dataset for decomposition

Epoched data:

- Can concatenate epochs for ICA

Both approaches work - choose based on your workflow and data characteristics.

Workflow Summary

This demo shows the complete ICA pipeline:

Prepare Data

- Load and preprocess continuous data
- Create EOG channels
- Detect extreme values
- Apply high-pass filter

Run ICA

- Standard infomax on full dataset
- Extended infomax on subset (20%)
- Compare algorithms

Visualize Components

- Topographic maps of components
- Component activations over time
- Frequency spectra
- Interactive databrowser

Identify Artifacts

- Automated identification (all types)
- Individual identification methods
- Plot component features

Remove and Validate

- Remove identified artifact components
- Reconstruct to verify correctness
- Compare original vs. cleaned data

Apply to Epochs

- Run ICA on epoched data
- Same identification and removal workflow
- Validate removal quality

Code Examples

```
# Demo: Independent Component Analysis (ICA)
# Shows ICA decomposition on continuous and epoched data, component identification
```

```
# (EOG, ECG, line noise, channel noise), component removal/restoration, and
visualization.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference, highpass filter, and detect
extreme values)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)
EegFun.is_extreme_value!(dat, 250);

# Calculate EOG signals
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
); # vertical EOG = mean(Fp1, Fp2) - mean(IO1, I02)
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
); # horizontal EOG = F9 - F10
EegFun.detect_eog_onsets!(dat, 50, :vEOG, :is_vEOG)
EegFun.detect_eog_onsets!(dat, 30, :hEOG, :is_hEOG)

# NOTE: the use_gpu versions will require loading the appropiate GPU package:
# using CUDA (for NVIDIA GPUs)
# using AMDGPU (for AMD GPUs)
# using Metal (for Apple Silicon GPUs)
```

```
# # ICA on continuous data
# ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :infomax)
# @time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :infomax, use_gpu = true)
#
# @time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :infomax_extended)
# @time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :infomax_extended, use_gpu
= true)
#
@time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :picard)
# @time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :picard, use_gpu = true)
#
# @time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :picard_extended)
# @time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :picard_extended, use_gpu =
true)
#
# @time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :amica)
# @time ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_250), algorithm = :amica, use_gpu = true)



# Databrowser (here we can turn on/off component removal's)
EegFun.plot_databrowser(dat, ica_result)

# ICA type plots
# Basic Topoplots
EegFun.plot_topography(ica_result, component_selection = EegFun.components(1:20));

# # Component spectra
# EegFun.plot_ica_component_spectrum(dat, ica_result, component_selection =
EegFun.components(1:70))
#
# # Component data/activation
EegFun.plot_ica_component_activation(dat, ica_result)
#
# # Databrowser (here we can turn on/off component removal's)
# EegFun.plot_databrowser(dat)
#
#
```

```
# # Databrowser (here we can turn on/off component removal's)
# EegFun.plot_databrowser(dat, ica_result)
#
#
# # Extended ICA
# ica_result = EegFun.run_ica(
#     dat;
#     sample_selection = EegFun.samples_not(:is_extreme_value_200),
#     percentage_of_data = 20,
#     algorithm = :infomax_extended,
# )
#
# # ICA type plots
# # Basic Topoplots
# EegFun.plot_topography(ica_result, component_selection =
EegFun.components(1:20));
#
# # Component spectra
# EegFun.plot_ica_component_spectrum(dat, ica_result, component_selection =
EegFun.components(1:70))
#
# # Component data/activation
# EegFun.plot_ica_component_activation(dat, ica_result)
#
# # Databrowser (here we can turn on/off component removal's)
# EegFun.plot_databrowser(dat, ica_result)
#
# # identify components (default correlation method)
# @time component_artifacts, component_metrics =
#     EegFun.identify_components(dat, ica_result, sample_selection =
EegFun.samples_not(:is_extreme_value_200));
#
# # identify components (Combined method: union of correlation and ICLabel)
# component_artifacts_comb, component_metrics_comb =
#     EegFun.identify_components(dat, ica_result, method = :combined,
sample_selection = EegFun.samples_not(:is_extreme_value_200));
#
# # or individually
# eog_comps, eog_comps_metrics_df =
#     EegFun.identify_eog_components(dat, ica_result, sample_selection =
EegFun.samples_not(:is_extreme_value_200));
#
# ecg_comps, ecg_comps_metrics_df =
#     EegFun.identify_ecg_components(dat, ica_result, sample_selection =
EegFun.samples_not(:is_extreme_value_200));
#
```

```
# line_noise_comps, line_noise_comps_metrics_df =
EegFun.identify_line_noise_components(dat, ica_result);
#
# channel_noise_comps, channel_noise_comps_metrics_df =
EegFun.identify_spatial_kurtosis_components(ica_result);
#
#
# # Get all identified component artifacts
# all_comps = EegFun.get_all_ica_components(component_artifacts)
# dat_ica_removed, ica_result_updated =
#     EegFun.subtract_ica_components(dat, ica_result, component_selection =
EegFun.components(all_comps))
#
# # Reconstruct for sanity check (ie., add components back to data)
# dat_ica_reconstructed, ica_result_restored =
#     EegFun.add_ica_components(dat_ica_removed, ica_result_updated,
component_selection = EegFun.components(all_comps))
#
# # Original should = reconstructed
# EegFun.channel_data(dat) ≈ EegFun.channel_data(dat_ica_reconstructed)
#
# # Plot component features
# EegFun.plot_eog_component_features(eog_comps, eog_comps_metrics_df)
# EegFun.plot_ecg_component_features(ecg_comps, ecg_comps_metrics_df)
# EegFun.plot_line_noise_components(line_noise_comps, line_noise_comps_metrics_df)
# EegFun.plot_spatial_kurtosis_components(channel_noise_comps,
channel_noise_comps_metrics_df)
#
#
# ################################
# # Epoched DataFrameEeg
# ################################
# # Create some epoched data
# epoch_cfg = [
#     EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
#     EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
# ]
# epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))  # -200 to 1000 ms
#
# # ICA on epoched data
# ica_result = EegFun.run_ica(epochs; sample_selection =
EegFun.samples_not(:is_extreme_value_200))
#
# # ICA type plots
# EegFun.plot_topography(ica_result, component_selection =
EegFun.components(1:4));
```

```
# EegFun.plot_ica_component_activation(dat, ica_result)
# EegFun.plot_ica_component_spectrum(dat, ica_result, component_selection =
EegFun.components(1:70))
# EegFun.plot_databrowser(dat, ica_result)
```

- API Reference

13.13 Layouts & Neighbors

Layouts & Neighbors

This demo demonstrates how to load, inspect, modify, and manage electrode layouts and their spatial neighbor relationships.

What are Layouts?

Layouts define the spatial positions of EEG electrodes on the scalp. They are essential for:

- Topographic plotting (scalp maps)
- Spherical spline interpolation (channel repair)
- Cluster-based permutation tests (spatial adjacency)
- Neighbor-based artifact repair

Coordinate Systems

EegFun supports multiple coordinate representations:

Polar coordinates: The raw format stored in layout CSV files (theta/radius pairs)

2D Cartesian (x, y): Used for topographic plots and 2D neighbor calculations (`polar_to_cartesian_xy!`)

3D Cartesian (x, y, z): Used for spherical spline interpolation and 3D neighbor calculations (`polar_to_cartesian_xyz!`)

Neighbors

Spatial neighbors define which electrodes are "close" to each other. This adjacency information is used by:

- Cluster-based permutation tests (defining spatial clusters)
- Neighbor-based artifact repair (interpolating bad channels from nearby good channels)

Neighbors can be calculated in 2D (`get_neighbors_xy!`) or 3D (`get_neighbors_xyz!`) space with a configurable distance threshold.

Workflow Summary

This demo covers:

- Loading layout files and converting coordinates
- Calculating and inspecting neighbor relationships

- Subsetting layouts to specific channels
- Renaming channels in a layout
- Creating custom layouts from scratch
- Validating layout structure

Code Examples

```
# Demo: Layout and Neighbor Management
# Shows how to load, inspect, modify, and manage electrode layouts
# and their spatial neighbor relationships.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

##############################################################################
# LOADING AND INSPECTING LAYOUTS
##############################################################################

# Load a layout file (CSV with polar coordinates)
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
layout  # show summary

# Convert polar to Cartesian coordinates for 2D plotting
EegFun.polar_to_cartesian_xy!(layout)

# Convert to 3D Cartesian coordinates (for spherical spline interpolation)
EegFun.polar_to_cartesian_xyz!(layout)

# Check coordinate validity
EegFun.has_valid_coordinates(layout)



##############################################################################
# NEIGHBOR CALCULATION
##############################################################################

# Calculate 2D neighbors using a distance criterion (in arbitrary units)
EegFun.get_neighbors_xy!(layout, 0.5)
layout  # now shows neighbors

# Check how many neighbors each channel has
```

```
EegFun.average_number_of_neighbors(layout.neighbors)

# Print neighbors to screen
EegFun.print_layout_neighbors(layout)

# Save neighbors to a TOML file
# EegFun.print_layout_neighbors(layout, "my_neighbors.toml")

# Check if a specific channel has enough neighbors
EegFun.check_channel_neighbors([:Cz], layout)

# Clear and recalculate with a different criterion
EegFun.clear_neighbors!(layout)
EegFun.get_neighbors_xyz!(layout, 0.5)  # 3D neighbors



##############################################################################
# LAYOUT SUBSETTING
##############################################################################

# Subset layout to only include specific channels
layout_subset = EegFun.subset_layout(layout, channel_selection =
EegFun.channels([:Fp1, :Fp2, :F3, :F4, :Fz, :Cz, :Pz]))
layout_subset  # 7 channels

# In-place subsetting (modifies the original)
layout_copy = copy(layout)
EegFun.subset_layout!(layout_copy, channel_selection =
EegFun.channels([:Cz, :Pz, :Oz]))
layout_copy  # 3 channels



##############################################################################
# RENAMING CHANNELS
##############################################################################

# Rename a single channel (non-mutating)
layout_renamed = EegFun.rename_channel(layout, Dict(:Fp1 => :FP1_new))

# Rename in-place
layout_copy2 = copy(layout)
EegFun.rename_channel!(layout_copy2, Dict(:Fp2 => :FP2_new))



##############################################################################
```

```
# CUSTOM LAYOUTS
#####################################################################

# Create a custom layout from scratch (e.g., for non-standard montages)
custom_layout = EegFun.create_custom_layout([(0.0, 1.0), (1.0, 0.0), (0.0, -1.0),
(-1.0, 0.0)], [:Ch1, :Ch2, :Ch3, :Ch4])
custom_layout

# Validate a layout has required structure
EegFun.validate_layout(layout)
```

- API Reference

13.14 Analysis Settings

Analysis Settings

This demo shows how to save and replay preprocessing settings using `AnalysisSettings` .

What are Analysis Settings?

`AnalysisSettings` stores a preprocessing recipe that can be applied to data in one step. This is useful for:

- Reproducing the same preprocessing across multiple datasets
- Storing settings from interactive exploration (e.g., databrowser GUI)
- Standardising processing across participants in a study

Tracked Settings

An `AnalysisSettings` object can contain:

- **`hp_filter`** : High-pass filter cutoff frequency
- **`lp_filter`** : Low-pass filter cutoff frequency
- **`reference`** : Re-referencing scheme (e.g., `:avg` , `:Cz`)
- **`repaired_channels`** : Channels to interpolate
- **`repair_method`** : Interpolation method
- **`removed_ica_components`** : ICA components to remove
- **`selected_regions`** : ROI definitions

Applying Settings

- **`apply_analysis_settings!(dat, settings)`** : Apply in-place (mutating)
- **`apply_analysis_settings(dat, settings)`** : Return a processed copy (non-mutating)
- **`apply_analysis_settings!(dat, ica, settings)`** : Apply with ICA component removal

Analysis Info Tracking

Every EegFun data object automatically tracks its preprocessing history via `analysis_info` . This records what filters, references, and other operations have been applied.

Workflow Summary

This demo covers:

- Inspecting analysis info on data objects
- Creating AnalysisSettings with filter and reference parameters
- Applying settings to data (both mutating and non-mutating)

Code Examples

```
# Demo: Analysis Settings
# Shows how to save and replay preprocessing settings
# (filter, rereference, ICA) using AnalysisSettings.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

################################################################################
# SETUP
################################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)


################################################################################
# INSPECTING ANALYSIS INFO
################################################################################

# Every EegFun data object tracks its preprocessing state
dat.analysis_info  # shows reference, filter info

# Apply some preprocessing
EegFun.highpass_filter!(dat, 0.1)
EegFun.rereference!(dat, :avg)

# Check that analysis info is updated
```

```julia
dat.analysis_info  # now shows hp_filter = 0.1, reference = :avg



#########################################################################
# USING ANALYSIS SETTINGS
#########################################################################

# AnalysisSettings stores a recipe for preprocessing
settings = EegFun.AnalysisSettings(1.0, 30.0, :avg, Symbol[], :none,
Tuple{Float64,Float64}[], Int[])

# Apply settings to fresh data (non-mutating — returns a processed copy)
dat_fresh = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
dat_fresh = EegFun.create_eegfun_data(dat_fresh, layout)

dat_processed = EegFun.apply_analysis_settings(dat_fresh, settings)
dat_processed.analysis_info

# Or apply in-place (mutating)
# EegFun.apply_analysis_settings!(dat_fresh, settings)
```

- API Reference

Chapter 14

Artifacts

14.1 Artifacts

Artifacts

This demo demonstrates artifact detection functionality, including identification, repair, and rejection strategies.

What are EEG Artifacts?

EEG artifacts are unwanted signals in the recording that originate from sources other than brain activity. They can severely compromise data quality. Common artifact sources include:

Physiological Artifacts:

- **Eye movements (EOG)** - Horizontal and vertical eye movements create large amplitude deflections, particularly in frontal electrodes
- **Eye blinks** - Produce characteristic sharp, high-amplitude signals in frontal channels
- **Muscle activity (EMG)** - Jaw clenching, facial movements, and neck tension create high-frequency noise
- **Cardiac signals (ECG)** - Heart electrical activity can contaminate EEG

External Artifacts:

- **Electrode issues** - Poor contact, bridging, or movement can create extreme values
- **Movement artifacts** - Head or body movements produce low-frequency drifts and sharp transients
- **Environmental interference** - Line noise (50/60 Hz), electromagnetic interference from equipment

It is common in EEG experiments to lose trials due to artifacts, and the percentage of data remaining after artifact rejection is often reported in method sections. In some cases, entire datasets may be considered too noisy, resulting in too few trials remaining for a participant to be included in the final analysis. Again, such details are often (should be) reported in the methods section of published papers.

Detection Strategies

Available approaches include:

1. **Threshold-based detection** - Identify samples/epochs exceeding absolute amplitude criteria (e.g., ±100 μV)
2. **Statistical detection** - Use z-scores to flag outliers based on variance, range, kurtosis, and other metrics
3. **EOG-specific detection** - Compute differential channels and detect characteristic eye movement patterns
4. **Visual inspection** - Interactive tools for manual review and refinement of automatic detection

5. **Epoch intervals** - Mark time periods around triggers for focused artifact assessment

Repair vs. Rejection

After detecting artifacts, there are two main options:

Artifact Repair (Interpolation):

- **When to use**: Bad channels in otherwise good epochs, isolated channel artifacts
- **Methods**:
  - `:neighbor_interpolation`
  - `:spherical_spline`
- **Advantage**: Retains maximum data, maintains epoch count
- **Limitation**: Cannot fix all artifacts (e.g., extreme eye movements affecting multiple channels)

Artifact Rejection:

- **When to use**: Entire epochs, or a large number of individual channels contaminated, movement artifacts, or when interpolation is inadequate
- **Advantage**: Removes contribution of noisy epoch to analysis
- **Limitation**: Reduces trial count, may bias remaining data if rejection is systematic

Best/Recommended Practice: Combine both approaches - repair isolated channel issues, reject severely contaminated epochs.

IMPORTANT

While artifact detection and correction techniques (including ICA, interpolation, and rejection) are valuable tools, no computational method can rescue extremely poor data quality. **Garbage in, garbage out** applies to EEG analysis. The most effective approach is careful data collection: proper electrode preparation, clear participant instructions, comfortable seating, and monitoring during recording. Time invested in quality data acquisition will far exceed any benefit from post-hoc correction of noisy data.

Workflow Summary

This demo demonstrates some features available in EegFun for artifact detection and correction:

1. Data Preparation

- Load raw BioSemi data and electrode layout
- Apply average reference and high-pass filtering (0.5 Hz)
- Visualize data in interactive databrowser

2. EOG Detection (Continuous Data)

- Compute vertical EOG channel (Fp1/Fp2 - IO1/IO2)
- Compute horizontal EOG channel (F9 - F10)
- Automatically detect eye movement onsets with threshold criteria

3. Extreme Value Detection (Continuous Data)

- Detect samples exceeding absolute voltage thresholds
- Apply detection globally or to specific channel/sample subsets
- Use predicate functions for flexible channel selection

4. Epoch Window Marking

- Mark time intervals around experimental triggers
- Apply artifact detection specifically within epoch intervals
- Visualize epoch boundaries and detected artifacts in databrowser

5. Bad Epoch Detection (Epoched Data)

- Extract epochs around experimental events
- **Automatic detection**: Combine absolute and z-score criteria across multiple metrics
  - Default metrics: variance, max, min, abs, range, kurtosis
  - Customizable thresholds and metric selection
- **Visual inspection**: Interactive grid view for manual verification
- **Hybrid approach**: Start with automatic detection, refine manually

6. Artifact Information

- Query unique rejection reasons, affected channels, and bad epochs
- Retrieve comprehensive artifact summaries per condition

7. Artifact Repair

- Interpolate bad channels using neighbor interpolation or spherical spline
- Visualize before/after comparison with repair plots
- Compare repair quality across different methods

8. Artifact Rejection

- Remove contaminated epochs entirely
- Visualize rejection impact on epoch count and data distribution

Code Examples

```
# Demo: Comprehensive Artifact Detection and Repair
# Shows EOG detection, extreme value detection, step artifact detection,
# automatic and interactive epoch rejection, artifact repair, and visualization.

using EegFun

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));
```

```
# read and prepare layout file
layout_file = EegFun.read_layout(EegFun.example_path("layouts/biosemi/
biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout_file)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout_file);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.5)

# Plot data in data browser
EegFun.plot_databrowser(dat)
# NB. The databrowser is interactive, with mouse and keyboard combinations.
# Press "i" for information about the interactions.

# Available functionality
# Region selection -> plot topography, plot spectrum
# Channel selectoin -> repair
# Rereference
# Filtering (NB. if high-pass already applied as above, not available again in
browser)
# Additiional channels
# Channel selection
# Highlight extreme activity channel wise
# View trigger onsets



# Calculate a vertical and a horizontal EOG channel
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
); # vertical EOG = mean(Fp1, Fp2) - mean(IO1, I02)
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
); # horizontal EOG = F9 - F10

# Automatically detect EOG onsets
EegFun.detect_eog_onsets!(dat, 50, :vEOG, :is_vEOG)
```

```
EegFun.detect_eog_onsets!(dat, 50, :hEOG, :is_hEOG)

EegFun.mark_epoch_intervals!(dat, :is_vEOG, [-0.05, 0.4])

EegFun.plot_databrowser(dat)



# How many vEOG/hEOG onsets were detected?
EegFun.n_values(dat, :is_vEOG)
EegFun.n_values(dat, :is_hEOG)

# We should now be able to see newly calculated EOG signals in
# the databrowser under "Extra Channels"
EegFun.plot_databrowser(dat)

# Look for extreme values (adds Bool column to dataframe :is_extreme_value_100)
EegFun.is_extreme_value!(dat, 100); # 100 mV criterion
EegFun.is_step_value!(dat, 50, channel_selection = EegFun.channels([:Fp1])); # 50
mV criterion

EegFun.plot_databrowser(dat)

# How many extreme values
EegFun.n_extreme_value(dat, 100)                    # across all electrodes
EegFun.n_extreme_value(dat, 100, mode = :separate)  # separately for each
electrode

# Additional examples with channel selection predicate
EegFun.is_extreme_value!(dat, 100; channel_selection =
EegFun.channels_not([:Fp1, :Fp2]));
EegFun.is_extreme_value!(dat, 100; channel_selection = x -> endswith.(string.(x),
"z"));
EegFun.is_extreme_value!(dat, 100; channel_selection = x -> .!(endswith.(string.
(x), "z")));
EegFun.is_extreme_value!(dat, 100; channel_selection = x -> .!(endswith.(string.
(x), "z")), sample_selection = x -> x.sample .> 1000);

# Step artifact detection - detect sudden voltage jumps between consecutive
samples
# This complements extreme value detection by catching artifacts like cable
disconnections
# or sharp movements (e.g., [1, 2, 3, 50, 2, ...] where extreme value < 100
wouldn't catch it)
EegFun.is_step_value!(dat, 50.0); # 50 µV jump criterion
EegFun.n_step_value(dat, 50.0)                    # how many step artifacts
(combined)
EegFun.n_step_value(dat, 50.0, mode = :separate)  # separately for each electrode
```

```
# Step artifact detection with channel selection
EegFun.is_step_value!(dat, 50.0; channel_selection =
EegFun.channels([:Fp1, :Fp2]), channel_out = :is_step_frontal);



# Artifact Detection in Epoched Data
EegFun.trigger_count(dat) # our markers or triggers
EegFun.mark_epoch_intervals!(dat, [1, 2, 3, 5, 6], [-0.2, 1.0]) # mark 200 ms and
1000 ms around trigger values

# Can use marked epoch interval as a basis for automatic artifact detection
EegFun.is_extreme_value!(dat, 50; sample_selection =
EegFun.samples(:epoch_interval), channel_out = :is_extreme_value_epoch)

# We can now see the epoch intervals and artifacts within them in the databrowser
under "Epoch Intervals"
EegFun.plot_databrowser(dat)



# Create some epoched data
epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))  # -200 to 1000 ms

# Detect bad epochs
# Default is combination of absolute and z-score based criteria with z-score based
on all epochs using the
# following metrics: [:variance, :max, :min, :abs, :range, :kurtosis]
bad_epochs = EegFun.detect_bad_epochs_automatic(epochs)
bad_epochs[1]

# But we can change the metrics (here only an absolute criterion is used)
bad_epochs = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 0,
abs_criterion = 100)

# Or here, using only z-score based criteria for variance
bad_epochs = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 2,
z_measures = [:variance])

# We can inspect the automatic detection visually
EegFun.plot_artifact_detection(epochs[1], bad_epochs[1]) # first epoch
EegFun.plot_artifact_detection(epochs[2], bad_epochs[2]) # second epoch
```

```
# We can also perform visual artifact detection
bad_epochs_manual = EegFun.detect_bad_epochs_interactive(epochs[1], dims = (4, 4))



# Our a combination of both
bad_epochs = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 0,
abs_criterion = 100)
bad_epochs_manual_visual = EegFun.detect_bad_epochs_interactive(epochs[1],
artifact_info = bad_epochs[1], dims = (4, 4))



# We can get information about the bad epochs that were automatically or visually
detected
# Artifact Info
# First condition
EegFun.unique_rejections(bad_epochs[1])
EegFun.unique_channels(bad_epochs[1])
EegFun.unique_epochs(bad_epochs[1])
EegFun.get_rejected(bad_epochs[1])

# repair
bad_epochs = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 0,
abs_criterion = 100)

epochs_repaired = EegFun.repair_artifacts(epochs, bad_epochs, method
= :neighbor_interpolation)
# epochs_repaired = EegFun.repair_artifacts(epochs, bad_epochs, method
= :spherical_spline)
# epochs_repaired = EegFun.repair_artifacts(epochs, bad_epochs, method = :reject)

EegFun.plot_artifact_repair(epochs[1], epochs_repaired[1], bad_epochs[1])
EegFun.plot_artifact_repair(epochs[1], epochs_repaired[1], bad_epochs[1], ylim =
(-100, 100))

# reject
epochs_rejected = EegFun.reject_epochs(epochs, bad_epochs)
EegFun.plot_artifact_rejection(epochs[1], epochs_rejected[1], bad_epochs[1])
```

- API Reference

14.2 Artifact Detection

Artifact Detection

This demo offers a walkthrough of artifact handling in EegFun: continuous-data artifact detection, EOG channels, epoched artifact detection, interactive review, artifact repair, and epoch rejection.

Key Functions

Function	Purpose
`channel_difference!`	Create derived channels (e.g., vEOG, hEOG)
`detect_eog_onsets!`	Detect EOG events in continuous data
`is_extreme_value!`	Flag samples exceeding a voltage threshold
`is_step_value!`	Flag sudden voltage jumps between consecutive samples
`detect_bad_epochs_automatic`	Flag bad epochs using z-score and/or absolute criteria
`detect_bad_epochs_interactive`	Visual grid review with optional pre-filled detections
`repair_artifacts`	Interpolate bad channels within flagged epochs
`reject_epochs`	Remove flagged epochs entirely
`plot_artifact_detection`	Visualise detection results per condition
`plot_artifact_repair`	Compare original vs repaired epochs
`plot_artifact_rejection`	Compare original vs rejected epochs

Workflow Summary

EOG Channel Creation

- Derive vertical and horizontal EOG from frontal (e.g., IO1/IO2, Fp1/Fp2) and lateral (e.g., F9/F10) channels using `channel_difference!`
- Detect blink/saccade onsets with `detect_eog_onsets!` and mark affected intervals with `mark_epoch_intervals!`

Continuous-Data Artifact Detection

- `is_extreme_value!` — marks samples exceeding an absolute threshold (e.g., ±100 μV)
- `is_step_value!` — marks sudden voltage jumps (e.g., cable disconnections)
- Both support `channel_selection` and `sample_selection` for targeted detection
- Use `n_extreme_value` / `n_step_value` to count detections (combined or per-channel with `mode = :separate`)

Epoch Artifact Detection

- **`z_criterion`** (default: 3.0) — epochs with z-scores above this are rejected. Set to 0 to disable.
- **`abs_criterion`** (default: 100 μV) — epochs exceeding this absolute voltage are rejected. Set to 0 to disable.

- **`z_measures`** — which metrics to use for z-score:
  `[:variance, :max, :min, :abs, :range, :kurtosis]`
- Use `channel_selection` for channel-specific detection (e.g., frontal channels only)

Inspect and Review

- Use `unique_epochs` , `unique_channels` , `unique_rejections` , and `get_rejected` to assess rejection rates
- Visualise with `plot_artifact_detection`
- Optionally use `detect_bad_epochs_interactive` for manual verification

Repair and Rejection

- `repair_artifacts` interpolates bad channels within flagged epochs (`:neighbor_interpolation` , `:spherical_spline`)
- `reject_epochs` removes flagged epochs entirely
- Visualise results with `plot_artifact_repair` and `plot_artifact_rejection`

Code Examples

```
# Demo: Artifact Detection, Repair, and Rejection
# Comprehensive walkthrough of artifact handling in EegFun:
# continuous-data artifact detection, EOG channels, epoched artifact detection,
# interactive review, artifact repair, and epoch rejection.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

##########################################################################
# LOAD AND PREPROCESS DATA
##########################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# Browse raw data (press "i" for interaction help)
```

```
EegFun.plot_databrowser(dat)



##########################################################################
# EOG CHANNEL CREATION AND ONSET DETECTION
##########################################################################

# Calculate vertical and horizontal EOG channels
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
)
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
)

# Automatically detect EOG onsets and mark intervals
EegFun.detect_eog_onsets!(dat, 50, :vEOG, :is_vEOG)
EegFun.detect_eog_onsets!(dat, 50, :hEOG, :is_hEOG)
EegFun.mark_epoch_intervals!(dat, :is_vEOG, [-0.05, 0.4])

# How many EOG onsets were detected?
EegFun.n_values(dat, :is_vEOG)
EegFun.n_values(dat, :is_hEOG)



##########################################################################
# CONTINUOUS-DATA ARTIFACT DETECTION
##########################################################################

# Extreme value detection (adds Bool column :is_extreme_value_100)
EegFun.is_extreme_value!(dat, 100)
EegFun.n_extreme_value(dat, 100)                    # count across all channels
EegFun.n_extreme_value(dat, 100, mode = :separate)  # per channel

# Step artifact detection — catches sudden voltage jumps (e.g., cable
disconnections)
EegFun.is_step_value!(dat, 50.0)
EegFun.n_step_value(dat, 50.0)                      # count across all channels
EegFun.n_step_value(dat, 50.0, mode = :separate)    # per channel
```

```
# Channel-specific detection examples
EegFun.is_extreme_value!(dat, 100; channel_selection =
EegFun.channels_not([:Fp1, :Fp2]))
EegFun.is_step_value!(dat, 50.0; channel_selection =
EegFun.channels([:Fp1, :Fp2]), channel_out = :is_step_frontal)

# Epoch interval marking for continuous data
EegFun.mark_epoch_intervals!(dat, [1, 2, 3, 5, 6], [-0.2, 1.0])
EegFun.is_extreme_value!(dat, 50; sample_selection =
EegFun.samples(:epoch_interval), channel_out = :is_extreme_value_epoch)

# Browse data with artifact annotations visible
# EegFun.plot_databrowser(dat)



##########################################################################
# EXTRACT AND BASELINE EPOCHS
##########################################################################

epoch_cfg = [
    EegFun.EpochCondition(name = "Trigger1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Trigger2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))
EegFun.baseline!(epochs)



##########################################################################
# AUTOMATIC EPOCH ARTIFACT DETECTION
##########################################################################

# Default: combined z-score (z > 3) and absolute threshold (> 100 μV)
# Uses all z-score measures: [:variance, :max, :min, :abs, :range, :kurtosis]
bad_epochs = EegFun.detect_bad_epochs_automatic(epochs)

# Print rejection summary
bad_epochs[1]  # first condition
bad_epochs[2]  # second condition

# Only absolute threshold (no z-score)
bad_abs = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 0,
abs_criterion = 100)

# Only z-score (no absolute threshold)
```

```
bad_z = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 2, abs_criterion
= 0)

# Only variance-based z-score (common for ERP studies)
bad_var = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 2.5, z_measures
= [:variance])

# Stricter thresholds
bad_strict = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 2,
abs_criterion = 50)

# Channel-specific detection
bad_frontal = EegFun.detect_bad_epochs_automatic(epochs, channel_selection =
EegFun.channels([:Fp1, :Fp2, :Fz]))



##########################################################################
# INSPECT REJECTION RESULTS
##########################################################################

# Get summary information
EegFun.unique_rejections(bad_epochs[1])  # all unique (channel, epoch) pairs
EegFun.unique_channels(bad_epochs[1])    # which channels had artifacts
EegFun.unique_epochs(bad_epochs[1])      # which epoch indices were flagged
EegFun.get_rejected(bad_epochs[1])       # indices of rejected epochs



##########################################################################
# VISUALISE DETECTION RESULTS
##########################################################################

# Plot detection summary for each condition
EegFun.plot_artifact_detection(epochs[1], bad_epochs[1])
EegFun.plot_artifact_detection(epochs[2], bad_epochs[2])



##########################################################################
# INTERACTIVE REVIEW (MANUAL + AUTOMATIC)
##########################################################################

# Interactive grid review — start with automatic detections
# bad_manual = EegFun.detect_bad_epochs_interactive(epochs[1], artifact_info =
bad_epochs[1], dims = (4, 4))

# Purely manual review (no pre-filled detections)
# bad_manual = EegFun.detect_bad_epochs_interactive(epochs[1], dims = (4, 4))
```

```
#########################################################################
# ARTIFACT REPAIR
#########################################################################

bad_epochs = EegFun.detect_bad_epochs_automatic(epochs, z_criterion = 0,
abs_criterion = 100)

# Repair via neighbor interpolation
epochs_repaired = EegFun.repair_artifacts(epochs, bad_epochs, method
= :neighbor_interpolation)
# epochs_repaired = EegFun.repair_artifacts(epochs, bad_epochs, method
= :spherical_spline)
# epochs_repaired = EegFun.repair_artifacts(epochs, bad_epochs, method = :reject)

# Visualise repair results
EegFun.plot_artifact_repair(epochs[1], epochs_repaired[1], bad_epochs[1])
EegFun.plot_artifact_repair(epochs[1], epochs_repaired[1], bad_epochs[1], ylim =
(-100, 100))



#########################################################################
# EPOCH REJECTION
#########################################################################

epochs_rejected = EegFun.reject_epochs(epochs, bad_epochs)
EegFun.plot_artifact_rejection(epochs[1], epochs_rejected[1], bad_epochs[1])
```

- API Reference

Chapter 15

ERP Analysis

15.1 ERP Measurements

ERP Measurements

This demo shows how to extract quantitative measurements from ERPs and analyse them with traditional statistics using AnovaFun.

Key Functions

Function	Purpose
`erp_measurements`	Batch extract amplitude/latency/area from all participant ERPs → CSV
`plot_erp_measurement_gui`	Interactive GUI to explore measurements before batch extraction
`paired_ttest`	AnovaFun: paired t-test between two conditions
`independent_ttest`	AnovaFun: independent samples t-test
`anova`	AnovaFun: repeated-measures or between-subject ANOVA
`emmeans` / `pairwise`	AnovaFun: estimated marginal means and pairwise contrasts

Available Measurement Types

Category	Types
Amplitude	`mean_amplitude` , `max_peak_amplitude` , `min_peak_amplitude` , `peak_to_peak_amplitude`
Latency	`max_peak_latency` , `min_peak_latency` , `peak_to_peak_latency` , `fractional_area_latency` , `fractional_peak_latency`
Area	`rectified_area` , `integral` , `positive_area` , `negative_area`

Workflow Summary

Extract Measurements

- Use `plot_erp_measurement_gui` to explore and choose parameters interactively
- Run `erp_measurements` to batch-extract values across all participants
- Output is a DataFrame and a saved CSV with columns: participant, condition, channel, measurement

Traditional Statistics with AnovaFun

- Load the CSV or use the in-memory DataFrame
- Use `paired_ttest` for condition comparisons at specific channels
- Use `anova` for multi-factor within/between-subject designs
- Follow up with `emmeans` and `pairwise` for post-hoc contrasts

Code Examples

```
# Demo: ERP Measurements and Traditional Statistics
# Shows how to extract ERP measurements (amplitudes, latencies) and
# analyse them using AnovaFun for traditional t-tests and ANOVA.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie
using AnovaFun

##############################################################################
# EXTRACT ERP MEASUREMENTS
##############################################################################

# This batch function loads all participant ERP files, extracts
# measurements, and saves results to a CSV file.

input_dir = EegFun.example_path("data/julia/erps")
file_pattern = "erps_final"

# Mean amplitude in the P300 window
mean_amp = EegFun.erp_measurements(
    file_pattern,
    "mean_amplitude",
    input_dir = input_dir,
    condition_selection = EegFun.conditions([1, 2]),
    channel_selection = EegFun.channels([:Pz, :Cz, :Fz]),
    analysis_interval = (0.3, 0.5),
```

```
    baseline_interval = (-0.2, 0.0),
)

# Result is an ErpMeasurementsResult containing a DataFrame
# It's also saved as a CSV to the output directory
mean_amp.data  # DataFrame with: participant, condition, channel, measurement


#####################################################################
# AVAILABLE MEASUREMENT TYPES
#####################################################################

# Amplitude measurements
# "mean_amplitude"          – mean voltage in interval
# "max_peak_amplitude"      – maximum peak value (robust detection)
# "min_peak_amplitude"      – minimum peak value (robust detection)
# "peak_to_peak_amplitude"  – difference between max and min peaks

# Latency measurements
# "max_peak_latency"        – time of maximum peak
# "min_peak_latency"        – time of minimum peak
# "peak_to_peak_latency"    – time difference between max and min peaks
# "fractional_area_latency" – time point dividing area into fraction
# "fractional_peak_latency" – time where amplitude is fraction of peak

# Area measurements
# "rectified_area"          – sum of absolute voltage values
# "integral"                – signed area (positive minus negative)
# "positive_area"           – area of positive deflections only
# "negative_area"           – area of negative deflections only


#####################################################################
# EXPLORE MEASUREMENTS INTERACTIVELY
#####################################################################

# GUI for exploring measurements interactively before batch extraction
dat = EegFun.read_data(EegFun.example_path("data/julia/erps/
example1_erps_final.jld2"))
EegFun.plot_erp_measurement_gui(dat)     # all conditions
# EegFun.plot_erp_measurement_gui(dat[1]) # first condition only


#####################################################################
# PEAK AMPLITUDE AND LATENCY
```

```
#########################################################################

# Max peak with robust detection (requires peak to be larger than neighbors)
max_peak = EegFun.erp_measurements(
    file_pattern,
    "max_peak_amplitude",
    input_dir = input_dir,
    condition_selection = EegFun.conditions([1, 2]),
    channel_selection = EegFun.channels([:Pz]),
    analysis_interval = (0.3, 0.6),
    baseline_interval = (-0.2, 0.0),
    local_interval = 3,  # peak must be larger than 3 neighbors on each side
)

# Corresponding latency
max_latency = EegFun.erp_measurements(
    file_pattern,
    "max_peak_latency",
    input_dir = input_dir,
    condition_selection = EegFun.conditions([1, 2]),
    channel_selection = EegFun.channels([:Pz]),
    analysis_interval = (0.3, 0.6),
    baseline_interval = (-0.2, 0.0),
)



#########################################################################
# TRADITIONAL STATISTICS WITH ANOVAFUN
#########################################################################

# The CSV output from erp_measurements can be used directly with AnovaFun.
# Here we demonstrate the workflow in-memory using the DataFrame.

# --- Paired t-test: condition 1 vs condition 2 at Pz ---
df = mean_amp.data

# Channels are in wide format (each channel is a column, e.g. :Pz, :Cz, :Fz)
cond1_pz = df[df.condition .== 1, :Pz]
cond2_pz = df[df.condition .== 2, :Pz]
result = paired_ttest(cond1_pz, cond2_pz)
result.t   # t-statistic
result.p   # p-value

# --- Independent t-test ---
# result = independent_ttest(group1_data, group2_data)
```

```
# --- Repeated-measures ANOVA ---
# For a 2-factor within-subject ANOVA (condition × channel), reshape your
# data into the format AnovaFun expects and call:
# result = anova(data, within = [:condition, :channel])
# anova_table(result)
# emmeans(result, :condition)
# pairwise(result, :condition)
```

- API Reference

15.2 Global Field Power

Global Field Power

This demo shows how to compute Global Field Power (GFP) and Global Dissimilarity from ERP data.

What are GFP and Global Dissimilarity?

- **Global Field Power (GFP)** is the standard deviation across all channels at each time point. It provides a reference-independent measure of overall response strength — high GFP means strong, synchronised activity across the scalp.
- **Global Dissimilarity** measures the rate of topographic change over time. Peaks in dissimilarity indicate moments when the scalp distribution is changing rapidly, which may mark transitions between ERP components or brain states.

Key Functions

Function	Returns
`gfp`	DataFrame with `:time` and `:gfp` columns
`global_dissimilarity`	DataFrame with `:time` and `:dissimilarity` columns
`gfp_and_dissimilarity`	DataFrame with both `:gfp` and `:dissimilarity`

Options

- `channel_selection` — restrict to specific channels
- `normalize` — scale to 0–100% for cross-dataset comparison
- `condition_selection` — compute for specific conditions

Workflow Summary

GFP Calculation

- All channels or a subset; raw or normalised

Global Dissimilarity

- Identify topographic transition points

Combined Calculation

- Compute both metrics efficiently in one call

Code Examples

```
# Demo: Global Field Power (GFP) and Global Dissimilarity
# Shows how to compute reference-independent measures of response strength
# and topographic change from ERP data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

################################################################################
# LOAD ERP DATA
################################################################################
dat = EegFun.read_data(EegFun.example_path("data/julia/erps/
example1_erps_final.jld2"))

################################################################################
# GLOBAL FIELD POWER (GFP)
################################################################################

# GFP = standard deviation across all channels at each time point
# High GFP = strong, synchronized activity

# All channels
gfp_result = EegFun.gfp(dat)

# Specific channels
gfp_result = EegFun.gfp(dat, channel_selection = EegFun.channels([:Cz, :Pz, :Fz]))

# Normalized to 0-100% (for comparing across datasets)
gfp_result = EegFun.gfp(dat, normalize = true)



################################################################################
```

```
# GLOBAL DISSIMILARITY
#########################################################################

# Global dissimilarity = rate of topographic change over time
# Peaks indicate transitions between brain states or ERP components

gd_result = EegFun.global_dissimilarity(dat)

# With normalization
gd_result = EegFun.global_dissimilarity(dat, normalize = true)



#########################################################################
# COMBINED CALCULATION
#########################################################################

# Calculate both GFP and global dissimilarity in one call
result = EegFun.gfp_and_dissimilarity(dat)

# Access the values (one result per condition)
result[1].time
result[1].gfp
result[1].dissimilarity



#########################################################################
# MULTIPLE CONDITIONS
#########################################################################

# GFP for all conditions
gfp_all = EegFun.gfp(dat)

# GFP for a specific condition
gfp_cond1 = EegFun.gfp(dat, condition_selection = EegFun.conditions([1]))
```

- API Reference

15.3 Grand Average

Grand Average

Grand averaging is the process of averaging ERP data across multiple participants. `EegFun.jl` provides two primary ways to calculate grand averages: using in-memory data or processing files directly from disk.

In-Memory Grand Averaging

If you already have a collection of `ErpData` objects loaded in your session, you can average them directly.

```
using EegFun

# Create grand averages for all conditions found in the list
# results = [erp_p1, erp_p2, erp_p3]
grand_avgs = EegFun.grand_average(results)
```

Batch Averaging (from Disk)

For large studies, it is often more efficient to process files directly from their storage directory. This approach automatically handles file discovery and condition grouping.

```
# Averages all JLD2 files in the folder that match the "erps_unrejected" pattern
EegFun.grand_average("erps_unrejected", input_dir = "derivatives/erp_analysis/")
```

How it Works

When `grand_average` is called:
1. **Grouping**: Data is automatically grouped by condition number across all participants.
2. **Intersection**: The system finds the intersection of channels available across all participants.
3. **Averaging**: Channel voltages are averaged point-by-point across the cohort.
4. **Metadata**: The resulting `ErpData` object tracks the total `n_epochs` .

Visualizing the Result

Once calculated, grand averages can be visualized using the same tools as individual ERPs.

```
# Plot with standard error shading (if n > 1)
EegFun.plot_erp(ga_results, legend_labels = ["Standard", "Target"])
```

Code Examples

```
# Demo: Grand Average
# Shows how to compute grand averages across participants, both from
# in-memory ERP data and from saved JLD2 files on disk.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)
```

```
##########################################################################
# LOAD MULTI-PARTICIPANT ERP DATA
##########################################################################

# Each file is one participant's averaged ERPs (saved by batch_erp or manually)
erps_p1 = EegFun.read_data(EegFun.example_path("data/julia/erps/
example1_erps_final.jld2"))
erps_p2 = EegFun.read_data(EegFun.example_path("data/julia/erps/
example2_erps_final.jld2"))
erps_p3 = EegFun.read_data(EegFun.example_path("data/julia/erps/
example3_erps_final.jld2"))

# Flatten into a single list — one ErpData per participant per condition
all_erps = vcat(erps_p1, erps_p2, erps_p3)



##########################################################################
# GRAND AVERAGE (IN-MEMORY)
##########################################################################

# Average across participants for every condition found in the list
grand_avgs = EegFun.grand_average(all_erps)

# Only average specific conditions
grand_avgs_12 = EegFun.grand_average(all_erps, condition_selection =
EegFun.conditions([1, 2]))



##########################################################################
# VISUALIZE
##########################################################################

# Plot grand average ERPs (SE shading is automatic when n > 1)
EegFun.plot_erp(grand_avgs, channel_selection = EegFun.channels([:Cz, :Pz]))

# Single condition
EegFun.plot_erp(grand_avgs, condition_selection = EegFun.conditions([1]),
channel_selection = EegFun.channels(:Pz))



##########################################################################
# BATCH GRAND AVERAGE (FROM DISK)
##########################################################################

# Averages all JLD2 files whose name contains "erps_final" in the given directory.
# Saves the result to tutorials/output/grand_average_erps_final.jld2
```

```
EegFun.grand_average("erps_final", input_dir = EegFun.example_path("data/julia/
erps/"), output_dir = DEMO_OUTPUT)

# With participant and condition filtering
EegFun.grand_average(
    "erps_final",
    input_dir = EegFun.example_path("data/julia/erps/"),
    participant_selection = EegFun.participants(1:10),
    condition_selection = EegFun.conditions([1, 2]),
    output_dir = DEMO_OUTPUT,
)



######################################################################
# LOAD AND PLOT BATCH RESULT
######################################################################

ga = EegFun.read_data(joinpath(DEMO_OUTPUT, "grand_average_erps_final.jld2"))
EegFun.plot_erp(ga, channel_selection = EegFun.channels([:Cz, :Pz]))
```

- API Reference

15.4 Jackknife Average

Jackknife Average

This demo shows how to create jackknife (leave-one-out) averages for robust statistical testing of ERP latency measures.

What is Jackknife Averaging?

The jackknife technique creates N grand averages from N participants, where each average excludes one participant. This produces smoother waveforms with clearly defined peaks, enabling reliable measurement of onset or peak latencies that would be noisy in individual-participant data.

Why Use It?

Standard ERP peak latency measures are unreliable in noisy single-participant waveforms. The jackknife method:

- Produces cleaner waveforms by averaging many participants
- Enables latency measurement on smooth grand-average-like waveforms
- Requires a simple correction to the t-statistic: t_corrected = t / (n − 1)

Key Functions

Function	Purpose

`jackknife_average(erps)`	Single-condition jackknife from a vector of ERPs
`jackknife_average(file_pattern)`	Batch jackknife across participant files

Reference

Miller, Patterson, & Ulrich (1998). Jackknife-based method for measuring LRP onset latency differences. *Psychophysiology*, 35, 99–115.

Workflow Summary

Single-Condition Jackknife

- Create leave-one-out averages from a vector of participant ERPs

Batch Processing

- Process all participant files and save jackknife averages

Typical Pipeline

- Calculate LRP → Jackknife average → Measure onset latency → Apply correction to t-test

Code Examples

```
# Demo: Jackknife Averaging
# Shows how to create leave-one-out (jackknife) averages for statistical
# testing of ERP latency measures.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)

##########################################################################
# LOAD MULTI-PARTICIPANT ERP DATA
##########################################################################

# We need multiple participants for jackknifing
erps_p1 = EegFun.read_data(EegFun.example_path("data/julia/erps/
example1_erps_final.jld2"))
```

```
erps_p2 = EegFun.read_data(EegFun.example_path("data/julia/erps/
example2_erps_final.jld2"))
erps_p3 = EegFun.read_data(EegFun.example_path("data/julia/erps/
example3_erps_final.jld2"))

# Create a list of one condition across participants (e.g., Condition 1)
cond_1_erps = [erps_p1[1], erps_p2[1], erps_p3[1]]

#########################################################################
# JACKKNIFE AVERAGING (IN-MEMORY)
#########################################################################

# Calculate jackknife averages: For N participants, this creates N averages
# where the i-th average includes all participants EXCEPT participant i.
jackknife_erps = EegFun.jackknife_average(cond_1_erps)

# Plot the jackknife averages
EegFun.plot_erp(jackknife_erps, channel_selection = EegFun.channels(:Pz))



#########################################################################
# BATCH JACKKNIFE AVERAGING (FROM DISK)
#########################################################################

# Averages all JLD2 files whose name contains "erps_final" in the given directory.
# This automatically handles multiple conditions and saves the leave-one-out
# datasets for each participant.
EegFun.jackknife_average("erps_final", input_dir = EegFun.example_path("data/
julia/erps/"), output_dir = DEMO_OUTPUT)
```

- API Reference

15.5 Lateralised Readiness Potential

Lateralised Readiness Potential

This demo shows how to calculate the Lateralised Readiness Potential (LRP) from ERP data.

What is the LRP?

The LRP is an ERP measure of lateralised motor preparation. It is computed from the difference in activity between hemispheres contralateral and ipsilateral to the responding hand, and is widely used in cognitive and motor neuroscience to study response preparation timing.

How is it Calculated?

For each channel pair (e.g., C3/C4), the LRP is:

```
LRP_C3 = 0.5 × ((C3_right − C4_right) + (C4_left − C3_left))
```

Where “left” and “right” refer to the hand used for the response.

Key Functions

Function	Purpose
`lrp(erp_left, erp_right)`	Single-participant LRP
`lrp(file_pattern, pairs)`	Batch LRP across participants

Channel Pairing

- By default, all odd/even channel pairs are detected automatically (e.g., C3/C4, CP1/CP2)
- Use `channel_selection` to restrict to specific pairs

Workflow Summary

Single-Participant LRP

- Calculate LRP from left and right response conditions

Batch Processing

- Process all participants with specified condition pairs

Visualisation

- Plot LRP waveforms with `plot_erp`

Code Examples

```
# Demo: Lateralised Readiness Potential (LRP)
# Shows how to calculate the LRP from left-hand and right-hand ERP conditions
# and perform batch processing across participants.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)

##########################################################################
```

```
# LOAD ERP DATA
#########################################################################

# Load a participant's ERPs
erps = EegFun.read_data(EegFun.example_path("data/julia/erps/
example1_erps_final.jld2"))

#########################################################################
# LATERALISED READINESS POTENTIAL (IN-MEMORY)
#########################################################################

# Calculate LRP from two conditions (e.g., Left response vs Right response)
# Assuming condition 1 is Left Hand and condition 2 is Right Hand
# The default channel_selection = EegFun.channels() auto-detects all
# odd/even lateral pairs (e.g., C3/C4, C1/C2).
lrp_data = EegFun.lrp(erps[1], erps[2])

# Plot the LRP (e.g., at C3/C4)
EegFun.plot_erp(lrp_data, channel_selection = EegFun.channels([:C3, :C4]))

# Calculate LRP for multiple condition pairs simultaneously from an array of ERPs
# e.g., Left vs Right (1 vs 2) and Left vs Right under a different condition (3 vs
4)
pairs = [(1, 2), (3, 4)]
lrp_results = EegFun.lrp(erps, pairs)



#########################################################################
# BATCH LRP (FROM DISK)
#########################################################################

# Batch calculate LRPs for a list of condition pairs across all participant files
EegFun.lrp("erps_final", [(1, 2)], input_dir = EegFun.example_path("data/julia/
erps/"), output_dir = DEMO_OUTPUT)
```

- API Reference

15.6 Realignment

Realignment

This demo shows how to realign stimulus-locked epochs to a different time point, such as response time, for response-locked ERP analysis.

When to Use Realignment

Stimulus-locked epochs are aligned to stimulus onset (t=0). Realignment shifts t=0 to a different event — typically the participant's response — so that you can study activity time-locked to that event instead.

Common use cases:

- **Response-locked ERPs** — study motor preparation relative to button press
- **Saccade-locked ERPs** — study activity relative to eye movement onset
- **Any event-locked analysis** — realign to any column in the epoch data

How it Works

1. Each epoch's time vector is shifted so that the realignment value becomes t=0
2. All epochs are cropped to the common time interval that is valid across all trials
3. A uniform time vector is regenerated to ensure consistency

Key Functions

Function	Purpose
`realign!(epochs, :rt)`	Realign in place (mutating)
`realign(epochs, :rt)`	Return a realigned copy
`realign(file_pattern, :rt)`	Batch realign across participants

Workflow Summary

Single-Participant Realignment

- Realign epochs to response time column

Batch Realignment

- Process all participant files in a directory

Typical Pipeline

- Extract stimulus-locked epochs ⟶ realign to RT ⟶ average ⟶ LRP ⟶ jackknife

Code Examples

```
# Demo: Response-Locked Realignment
# Shows how to realign stimulus-locked epochs to a different time point
# (e.g., response time) for response-locked ERP analysis.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie
```

```
const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)

##############################################################################
# LOAD EPOCHED DATA
##############################################################################

# Load stimulus-locked epochs for a single participant
epochs = EegFun.read_data(EegFun.example_path("data/julia/epochs/
example1_epochs.jld2"))

##############################################################################
# REALIGN EPOCHS (IN-MEMORY)
##############################################################################

# Realign epochs to response triggers (e.g., 201, 202).
# This shifts the time axis of each epoch so that the response trigger becomes
t=0,
# and drops epochs that do not contain any of these triggers.
EegFun.realign!(epochs, [201, 202])

# The time axis is now response-locked! Let's compute the ERP.
# We will use condition 1.
# Note that we use grand_average since we are taking the mean over epochs
response_locked_erp = EegFun.grand_average([epochs[1]])

# Plot the response-locked ERP
EegFun.plot_erp(response_locked_erp)

##############################################################################
# TRIGGER INTERVALS (REACTION TIMES)
##############################################################################

# We can also calculate the time interval between the stimulus and the response
# (i.e. the reaction time) and append it as a column to the epoch data.
epochs_rt = EegFun.read_data(EegFun.example_path("data/julia/epochs/
example1_epochs.jld2"))

# Calculate the interval between stimulus (e.g., 101) and response (e.g., 201)
EegFun.calculate_trigger_interval!(epochs_rt, [101], [201], column_name
= :reaction_time)



##############################################################################
```

```
# BATCH REALIGNMENT (FROM DISK)
#####################################################################

# Realign all epoch files matching the pattern to response triggers
EegFun.realign("epochs", [201, 202], input_dir = EegFun.example_path("data/julia/
epochs/"), output_dir = DEMO_OUTPUT)
```

- API Reference

15.7 Condition Operations

Condition Operations

This demo shows how to combine epoch conditions, compute ERP difference waves, and create condition average waves.

Key Functions

Function	Purpose	Input
`condition_combine`	Merge epoch conditions into new groups	Epoch data
`condition_difference`	Subtract one condition's ERP from another	ERP data
`condition_average`	Average multiple conditions into one ERP	ERP data

When to Use

- **`condition_combine`** : When you have many conditions and want to pool some together before averaging (e.g., merge "Go left" and "Go right" into a single "Go" condition)
- **`condition_difference`** : When you need difference waves for analysis (e.g., target minus standard, congruent minus incongruent)
- **`condition_average`** : When you want to average across conditions after computing ERPs (e.g., collapsing across irrelevant factors)

Important Notes

- `condition_combine` works on epoch data (before averaging); `condition_difference` and `condition_average` work on ERP data (after averaging)
- All three also have batch versions that process all matching JLD2 files in a directory

Workflow Summary

Condition Combining

- Merge multiple epoch conditions into new groups (operates on epochs, before averaging)

Condition Differences

- Create difference waves from ERP condition pairs (e.g., `[(1, 2), (3, 4)]`)

Condition Averaging

- Average ERP conditions into combined waves (e.g., `[[1, 2], [3, 4]]` or `[[1, 2, 3, 4]]`)

Typical Pipeline

- Preprocess → Combine conditions → Average → Difference/Average waves → Grand average

Code Examples

```
# Demo: Condition Operations
# Shows how to combine epoch conditions, compute ERP difference waves,
# and create condition average waves.
# Covers condition_combine, condition_difference, and condition_average.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# CREATE EXAMPLE DATA
############################################################################
# Load raw data and do minimal preprocessing
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)

dat = EegFun.create_eegfun_data(dat, layout)

# minimal preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# Extract epochs for four conditions
epoch_cfg = [
    EegFun.EpochCondition(name = "Trigger1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Trigger2", trigger_sequences = [[2]]),
    EegFun.EpochCondition(name = "Trigger3", trigger_sequences = [[3]]),
    EegFun.EpochCondition(name = "Trigger4", trigger_sequences = [[5]]),
]
```

```
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))
EegFun.baseline!(epochs, (-0.2, 0.0))


##############################################################################
# CONDITION COMBINE (EPOCHS)
##############################################################################

# Combine conditions 1+2 into one group and 3+4 into another
epochs_combined = EegFun.condition_combine(epochs, [[1, 2], [3, 4]])


##############################################################################
# CONDITION DIFFERENCE WAVES (ERPS)
##############################################################################

# Create ERPs from the original four conditions
erps = EegFun.average_epochs(epochs)

# Difference wave: condition 1 minus condition 2
diff_waves = EegFun.condition_difference(erps, [(1, 2)])
EegFun.plot_erp(diff_waves, channel_selection = EegFun.channels([:Cz]))

# Multiple difference pairs
diff_waves = EegFun.condition_difference(erps, [(1, 2), (3, 4)])


##############################################################################
# CONDITION AVERAGE WAVES (ERPS)
##############################################################################

# Average conditions 1 and 2 together, and 3 and 4 together
avg_waves = EegFun.condition_average(erps, [[1, 2], [3, 4]])

# Average all four conditions into one
avg_all = EegFun.condition_average(erps, [[1, 2, 3, 4]])
```

- API Reference

Chapter 16

Time-Frequency

16.1 TF Analysis

TF Analysis Workflow

This demo shows how to work with time-frequency data after decomposition — including condition operations, channel operations, subsetting, and batch processing.

Prerequisites

Time-frequency data (`TimeFreqData`) is produced by one of the decomposition methods:

- `tf_morlet` — Morlet wavelet decomposition
- `tf_multitaper` — Multitaper spectral estimation
- `tf_stft` — Short-Time Fourier Transform

Each `TimeFreqData` object contains two DataFrames: `data_power` (spectral power) and `data_phase` (phase angles). All operations below act on **both** DataFrames consistently.

Key Functions

Function	Purpose	Mutating?
`condition_average`	Average power/phase across c onditions	No
`condition_difference`	Subtract one condition's TF from another	No
`channel_average!` / `channel_ average`	Average across channels	Yes / No
`channel_difference!` / `chann el_difference`	Compute channel differences	Yes / No
`subset`	Select channels, time points, or conditions	No

Condition Operations

Condition averaging and differencing work identically to their ERP counterparts — they auto-detect `TimeFreqData` and dispatch appropriately:

- **Averaging**: Combine conditions into a single mean TF representation
- **Differencing**: Compute condition contrasts (e.g., target minus standard)

Channel Operations

- **`channel_average`** : Create region-of-interest (ROI) averages (e.g., mean frontal power)
- **`channel_difference`** : Compute laterality indices (e.g., left − right)
- Both update `data_power` and `data_phase` consistently

Batch Processing

The batch versions of `condition_average` and `condition_difference` automatically detect whether JLD2 files contain `ErpData` or `TimeFreqData` and dispatch accordingly. Use the same API for both:

```
# Works for both ERP and TF files
EegFun.condition_average("tf_morlet", [[1, 2], [3, 4]], input_dir = "./output")
EegFun.condition_difference("tf_morlet", [(1, 2), (3, 4)], input_dir = "./output")
```

Subsetting

Use `subset` to select specific channels, time intervals, or conditions. For `TimeFreqData` , both `data_power` and `data_phase` are subset consistently:

```
sub = EegFun.subset(tf, channel_selection = EegFun.channels([:Fz, :Cz]))
sub = EegFun.subset(tfs, condition_selection = EegFun.conditions(1, 2))
```

Code Examples

```
# Demo: TF Analysis Workflow
# Shows how to perform condition and channel operations on time-frequency data.

using EegFun

############################################################################
@info EegFun.section("1. Generate TF data from epochs")
############################################################################

# Load and preprocess data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# Extract epochs for four conditions
epoch_cfg = [
    EegFun.EpochCondition(name = "Cond1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Cond2", trigger_sequences = [[2]]),
    EegFun.EpochCondition(name = "Cond3", trigger_sequences = [[3]]),
```

```
    EegFun.EpochCondition(name = "Cond4", trigger_sequences = [[5]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.5, 1.5))

# Morlet decomposition per condition
tfs = [EegFun.tf_morlet(epochs[i], frequencies = logrange(2, 40, length = 30),
cycles = (3, 7)) for i in eachindex(epochs)]

# Plot single condition
EegFun.plot_tf(
    tfs[1];
    baseline_interval = (-0.4, -0.1),
    baseline_method = :db,
    xlim = (-0.2, 1.0),
    colorrange = (-3, 3),
)

##############################################################################
@info EegFun.section("2. Condition Average")
##############################################################################

# Average conditions 1+2 and 3+4
avg_tfs = EegFun.condition_average(tfs, [[1, 2], [3, 4]])
@info "Created $(length(avg_tfs)) average TF objects"

EegFun.plot_tf(
    avg_tfs[1];
    baseline_interval = (-0.4, -0.1),
    baseline_method = :db,
    xlim = (-0.2, 1.0),
    colorrange = (-3, 3),
)

##############################################################################
@info EegFun.section("3. Condition Difference")
##############################################################################

# Difference: condition 1 minus condition 2
diff_tfs = EegFun.condition_difference(tfs, [(1, 2), (3, 4)])
@info "Created $(length(diff_tfs)) difference TF objects"

EegFun.plot_tf(
    diff_tfs[1];
    baseline_interval = (-0.4, -0.1),
    baseline_method = :db,
```

```
    xlim = (-0.2, 1.0),
    colorrange = (-3, 3),
)

##########################################################################
@info EegFun.section("4. Channel Operations")
##########################################################################

# Average across frontal channels
tf_frontal = EegFun.channel_average(
    tfs[1],
    channel_selections = [EegFun.channels([:Fz, :F3, :F4])],
    output_labels = [:Frontal],
    reduce = true,
)

EegFun.plot_tf(
    tf_frontal;
    baseline_interval = (-0.4, -0.1),
    baseline_method = :db,
    xlim = (-0.2, 1.0),
)

# Channel difference (laterality)
tf_lat = EegFun.channel_difference(
    tfs[1],
    channel_selection1 = EegFun.channels([:C3]),
    channel_selection2 = EegFun.channels([:C4]),
    channel_out = :Laterality,
)

##########################################################################
@info EegFun.section("5. Subsetting")
##########################################################################

# Subset by channels
tf_sub = EegFun.subset(tfs[1], channel_selection =
EegFun.channels([:Fz, :Cz, :Pz]))
@info "Subset has $(length(EegFun.channel_labels(tf_sub))) channels"

# Subset vector by condition
tf_sub_vec = EegFun.subset(tfs, condition_selection = EegFun.conditions(1, 2))
@info "Subset has $(length(tf_sub_vec)) conditions"
```

- TF Morlet — Morlet wavelet decomposition
- TF Multitaper — Multitaper spectral estimation

- TF STFT — Short-Time Fourier Transform
- Condition Operations — ERP condition operations
- API Reference

16.2 TF Morlet

TF Morlet

This demo demonstrates time-frequency analysis using Morlet wavelets to decompose EEG signals into time-varying frequency content.

What are Morlet Wavelets?

Morlet wavelets are oscillating waves with Gaussian envelopes that provide localized time-frequency analysis:

- **Wavelet**: Small wave that is both time-limited and frequency-specific
- **Gaussian envelope**: Smooth windowing that minimizes spectral leakage
- **Time-frequency trade-off**: Cannot have perfect precision in both simultaneously

Key Parameter: Number of Cycles

The `cycles` parameter controls the time-frequency resolution trade-off:

Fixed cycles (e.g., `cycles = 5`):

- Same spectral bandwidth at all frequencies
- **Low frequencies**: Broad temporal windows (poor time resolution)
- **High frequencies**: Narrow temporal windows (good time resolution)

Variable cycles (e.g., `cycles = (3, 10)`):

- Adaptive resolution across frequencies
- **Low frequencies**: Fewer cycles = better temporal precision
- **High frequencies**: More cycles = better spectral precision
- Balances resolution optimally across the spectrum

Frequency Spacing

Linear spacing (`frequencies = 1:1:40`):

- Equal Hz steps
- Good for narrow frequency ranges
- Easier interpretation

Logarithmic spacing (`frequencies = logrange(1, 40, length = 30)`):

- Proportional frequency steps
- Better for wide ranges (e.g., 2-80 Hz)
- Use with `yscale = :log10` in plots

Baseline Correction

Apply baseline correction to isolate event-related changes:

```
baseline_interval = (-0.5, -0.2)  # Pre-stimulus baseline
baseline_method = :db           # Decibel conversion (10*log10(activity/baseline))
```

Baseline methods:

- **:db**: Decibel scale (relative power change)
- **:percent**: Percent change from baseline
- **:zscore**: Z-score normalization

Edge Effects

The `filter_edges` and `pad` parameters control edge artifacts:

```
EegFun.tf_morlet(epochs, pad = :both)         # Mirror padding
EegFun.tf_morlet(epochs, filter_edges = true) # Remove edge samples
```

Workflow Summary

This demo shows:
1. **Synthetic validation**: Known frequencies (5, 25, 35 Hz) for sanity checking
2. **Fixed cycles**: Constant resolution across frequencies
3. **Variable cycles**: Adaptive resolution with `cycles = (3, 10)`
4. **Linear vs log spacing**: Different frequency sampling strategies
5. **Baseline correction**: Isolating event-related power changes

Further Reading

Cohen, M. X. (2014). *Analyzing Neural Time Series Data: Theory and Practice*. Chapter 12: Morlet Wavelets and Wavelet Convolution

Code Examples

```
# Demo: Time-Frequency Analysis - Morlet Wavelets
# Shows wavelet-based time-frequency decomposition with synthetic signals
# and real data, demonstrating different cycle counts and frequency resolution.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

##########################################################################
@info EegFun.section("TEST 1: Synthetic Signal with Known Frequencies")
```

```
#########################################################################

# Generate synthetic signal
sample_rate = 1000.0
times, signal = EegFun.generate_signal(
    400,                                    # n_trials
    [-1.0, 3.0],                            # time_interval
    sample_rate,                            # sample_rate
    [5.0, 25, 35.0],                        # frequencies
    [5.0, 5.0, 5.0],                        # amplitudes
    [[0.1, 0.5], [0.6, 1.0], [1.1, 1.5]],   # time intervals for each freq
    0.0,                                    # noise amplitude
);
epochs_synthetic = EegFun.signal_to_data(times, signal, :Channel1, sample_rate)
# EegFun.plot_epochs(epochs_synthetic, channel_selection =
EegFun.channels([:Channel1]))

spectrum = EegFun.freq_spectrum(epochs_synthetic, max_freq = 80.0)
# EegFun.plot_channel_spectrum(spectrum, channel_selection =
EegFun.channels([:Channel1]))

# tf_morlet
@time tf_data = EegFun.tf_morlet(epochs_synthetic, frequencies = 1:1:40, cycles =
3, pad = :both, use_gpu = true)
EegFun.plot_tf(tf_data, yscale = :linear)

tf_data = EegFun.tf_morlet(epochs_synthetic, frequencies = 1:0.5:40, cycles = 10)
EegFun.plot_tf(tf_data, yscale = :linear)

tf_data = EegFun.tf_morlet(epochs_synthetic, frequencies = logrange(1, 40, length
= 30), cycles = 3)
EegFun.plot_tf(tf_data, yscale = :log10)

tf_data = EegFun.tf_morlet(epochs_synthetic, frequencies = logrange(1, 40, length
= 30), cycles = 10)
EegFun.plot_tf(tf_data, yscale = :log10)



#########################################################################
@info EegFun.section("TEST 2: Cohen Data Chapter 13")
#########################################################################
# This is some data that was presented in Cohen: Analyzin Neural Time Series Data
data_cohen = EegFun.read_data(EegFun.example_path("data/julia/tf/
tf_test_epochs.jld2"));

# Figure 13.11 A)
```

```
tf_data = EegFun.tf_morlet(data_cohen, frequencies = logrange(2, 80, length = 80),
cycles = 3, filter_edges = false)
EegFun.plot_tf(
    tf_data;
    plot_title = "3-cycle wavelets",
    colorbar_kwargs = (; label = "dB change from baseline"),
    baseline_interval = (-0.5, -0.2),
    baseline_method = :db,
    xlim = (-0.2, 1.0),
    colorrange = (-3, 3),
    yscale = :log10,
    colormap = :jet,
    interpolate = true,
    yticks = [2, 4, 10, 20, 40, 80],
    xticks = [0, 0.2, 0.4, 0.6, 0.8],
    time_unit = :ms,
)

# Figure 13.11 B)
tf_data = EegFun.tf_morlet(data_cohen, frequencies = 2:1:80, cycles = (3, 10))
EegFun.plot_tf(
    tf_data;
    baseline_interval = (-0.5, -0.2),
    baseline_method = :db,
    xlim = (-0.2, 1.0),
    colorrange = (-3, 3),
    yscale = :linear,
    colormap = :jet,
)

# Figure 13.14 A)
tf_data = EegFun.tf_morlet(data_cohen, frequencies = logrange(2, 80, length = 40),
cycles = 3)
EegFun.plot_tf(
    tf_data;
    baseline_interval = (-0.5, -0.2),
    baseline_method = :db,
    xlim = (-0.2, 1.0),
    colorrange = (-3, 3),
    yscale = :log10,
    colormap = :jet,
)

# Figure 13.14 B)
```

```
tf_data = EegFun.tf_morlet(data_cohen, frequencies = logrange(2, 80, length = 40),
cycles = 10, filter_edges = false)
EegFun.plot_tf(
    tf_data;
    baseline_interval = (-0.5, -0.2),
    baseline_method = :db,
    xlim = (-0.2, 1.0),
    colorrange = (-3, 3),
    yscale = :log10,
    colormap = :jet,
)

# Figure 13.14 C)
tf_data = EegFun.tf_morlet(data_cohen, frequencies = logrange(2, 80, length = 40),
cycles = (3, 10))
EegFun.plot_tf(
    tf_data;
    baseline_interval = (-0.5, -0.2),
    baseline_method = :db,
    xlim = (-0.2, 1.0),
    colorrange = (-3, 3),
    yscale = :log10,
    colormap = :jet,
    interpolate = true,
)
```

- API Reference

16.3 TF Multitaper

TF Multitaper

This demo demonstrates time-frequency analysis using the multitaper method for high-quality spectral estimation.

What is the Multitaper Method?

The multitaper method improves spectral estimates by averaging across multiple orthogonal tapers (windows):

- **Slepian tapers**: Mathematically optimal windows that minimize spectral leakage
- **Multiple estimates**: Each taper provides an independent spectral estimate
- **Averaging**: Reduces variance while preserving spectral features
- **Time-bandwidth product (NW)**: Controls frequency smoothing

Key Parameters

Cycles:

Similar to Morlet wavelets, controls the time window length:

```
EegFun.tf_multitaper(epochs, cycles = 5)
```

Frequency Spacing

Linear spacing (`frequencies = 1:1:40`):

- Equal Hz steps across range
- Good for narrow frequency bands

Logarithmic spacing (`frequencies = logrange(1, 40, length = 30)`):

- Proportional steps (constant ratios)
- Better for wide ranges (2-80 Hz)
- Use with `yscale = :log10` for visualization

Baseline Correction

```
EegFun.plot_tf(
    tf_data,
    baseline_interval = (-0.5, -0.2),
    baseline_method = :db
)
```

Isolates event-related changes relative to pre-stimulus baseline.

Workflow Summary

This demo shows:
1. **Synthetic validation**: Known frequencies (5, 25, 35 Hz)
2. **Fixed cycles**: Constant resolution analysis
3. **Linear vs log spacing**: Different frequency sampling strategies
4. **Baseline correction**: Event-related power changes
5. **Edge filtering**: Removing edge artifacts

Practical Tips

Choose cycles based on needs:

- **3-5 cycles**: Better temporal resolution
- **7-10 cycles**: Better frequency resolution
- **5 cycles**: Good balance for most EEG work

Frequency sampling:

- Dense sampling (every 0.5 Hz) for detailed spectra
- Coarse sampling (every 2-5 Hz) for faster computation

Time steps:

- Control temporal smoothness of the output

- Smaller steps = smoother time course but slower computation

Further Reading

Cohen, M. X. (2014). *Analyzing Neural Time Series Data: Theory and Practice*. Chapter 16: Multitapers

Code Examples

```
# Demo: Time-Frequency Analysis - Multitaper
# Shows multitaper method for time-frequency analysis with reduced variance.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

######################################################################
@info EegFun.section("TEST 1: Synthetic Signal with Known Frequencies")
######################################################################

# Generate synthetic signal
sample_rate = 1000.0
times, signal = EegFun.generate_signal(
    400,                                    # n_trials
    [-1.0, 3.0],                            # time_interval
    sample_rate,                            # sample_rate
    [5.0, 25, 35.0],                        # frequencies
    [5.0, 5.0, 5.0],                        # amplitudes
    [[0.1, 0.5], [0.6, 1.0], [1.1, 1.5]],   # time intervals for each freq
    0.0,                                    # noise amplitude
);
epochs_synthetic = EegFun.signal_to_data(times, signal, :Channel1, sample_rate)
EegFun.plot_epochs(epochs_synthetic, channel_selection =
EegFun.channels([:Channel1]))

spectrum = EegFun.freq_spectrum(epochs_synthetic, max_freq = 80.0)
EegFun.plot_channel_spectrum(spectrum, channel_selection =
EegFun.channels([:Channel1]))

# tf_multitaper
@time tf_data = EegFun.tf_multitaper(epochs_synthetic, frequencies = 1:1:40,
cycles = 5, pad = :both, use_gpu = true)
EegFun.plot_tf(tf_data, yscale = :linear)
```

```
tf_data = EegFun.tf_multitaper(epochs_synthetic, frequencies = logrange(1, 40,
length = 30), cycles = 5)
EegFun.plot_tf(tf_data, yscale = :log10)


#########################################################################
@info EegFun.section("TEST 2: Cohen Data Chapter 13")
#########################################################################
# This is some data that was presented in Cohen: Analyzin Neural Time Series Data
data_cohen = EegFun.read_data(EegFun.example_path("data/julia/tf/
tf_test_epochs.jld2"));

# Figure 13.11 A)
tf_data = EegFun.tf_multitaper(data_cohen, frequencies = logrange(2, 80, length =
30), cycles = 5, time_steps = 0.05, filter_edges = true)
EegFun.plot_tf(tf_data; baseline_interval = (-0.5, -0.2), baseline_method = :db,
colorrange = (-3, 3), yscale = :log10, colormap = :jet)
```

- API Reference

16.4 TF STFT

TF STFT

This demo demonstrates time-frequency analysis using the Short-Time Fourier Transform (STFT), a classic approach to spectrograms.

What is STFT?

STFT applies the Fast Fourier Transform (FFT) to windowed segments of the signal:
1. **Window the signal**: Apply time-limited window (e.g., Hann, Tukey)
2. **Compute FFT**: Calculate frequency spectrum for that segment
3. **Slide window**: Move forward in time and repeat
4. **Build spectrogram**: Stack spectra to create time-frequency representation

Key Parameters

Window length (`window_length`):

Controls time-frequency resolution:

```
EegFun.tf_stft(epochs, window_length = 0.5)  # 500 ms windows
```

- **Longer windows** (e.g., 500 ms): Better frequency resolution, worse time resolution
- **Shorter windows** (e.g., 200 ms): Better time resolution, worse frequency resolution

Cycles (alternative):

Instead of specifying window length, specify cycles at each frequency:

```
EegFun.tf_stft(epochs, cycles = 7)
```

Creates frequency-adaptive windows similar to Morlet wavelets.

Time steps:

Controls temporal resolution of output:

```
EegFun.tf_stft(epochs, time_steps = 0.005)  # 5 ms steps
```

Smaller = smoother time course, larger = faster computation.

Frequency Spacing

Linear (`frequencies = 2:1:80`):

- Equal Hz spacing
- Natural for narrow bands

Logarithmic (`frequencies = logrange(2, 80, length = 100)`):

- Proportional spacing
- Better for wide ranges
- Use `yscale = :log10` in plots

Baseline Correction

```
EegFun.plot_tf(
    tf_data,
    baseline_interval = (-0.5, -0.2),
    baseline_method = :db,
    colorrange = (-3, 3)
)
```

Shows relative power changes from pre-stimulus baseline.

Edge Effects

Filter edges (`filter_edges = true`):

- Removes edge samples where windowing causes artifacts
- Recommended for cleaner results

Workflow Summary

This demo shows:
1. **Synthetic validation**: Known frequencies (5, 25, 35 Hz)
2. **Fixed window length**: Constant resolution analysis
3. **Cycles mode**: Adaptive resolution
4. **Linear vs log spacing**: Different frequency sampling
5. **Baseline correction**: Event-related power changes
6. **Edge filtering**: Removing edge artifacts

Further Reading

Cohen, M. X. (2014). *Analyzing Neural Time Series Data: Theory and Practice*. Chapter 15: Short-Time FFT

Code Examples

```
# Demo: Time-Frequency Analysis - STFT
# Shows Short-Time Fourier Transform for time-frequency analysis.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

######################################################################
@info EegFun.section("TEST 1: Synthetic Signal with Known Frequencies")
######################################################################

# Generate synthetic signal
sample_rate = 1000.0
times, signal = EegFun.generate_signal(
    400,                                    # n_trials
    [-1.0, 3.0],                            # time_interval
    sample_rate,                            # sample_rate
    [5.0, 25, 35.0],                        # frequencies
    [5.0, 5.0, 5.0],                        # amplitudes
    [[0.1, 0.5], [0.6, 1.0], [1.1, 1.5]],   # time intervals for each freq
    0.0,                                    # noise amplitude
);
epochs_synthetic = EegFun.signal_to_data(times, signal, :Channel1, sample_rate)
# EegFun.plot_epochs(epochs_synthetic, channel_selection =
EegFun.channels([:Channel1]))

spectrum = EegFun.freq_spectrum(epochs_synthetic, max_freq = 80.0)
EegFun.plot_channel_spectrum(spectrum, channel_selection =
EegFun.channels([:Channel1]))

# tf_stft
@time tf_data = EegFun.tf_stft(epochs_synthetic, frequencies = 1:1:40,
window_length = 0.5, pad = :both, use_gpu = true)
EegFun.plot_tf(tf_data, yscale = :linear)

tf_data = EegFun.tf_stft(epochs_synthetic, frequencies = logrange(1, 40, length =
30), window_length = 0.5)
EegFun.plot_tf(tf_data, yscale = :log10)
```

```
#########################################################################
@info EegFun.section("TEST 2: Cohen Data Chapter 13")
#########################################################################
# This is some data that was presented in Cohen: Analyzin Neural Time Series Data
data_cohen = EegFun.read_data(EegFun.example_path("data/julia/tf/
tf_test_epochs.jld2"));

# Figure 13.11 A)
tf_data =
    EegFun.tf_stft(data_cohen, frequencies = logrange(2, 80, length = 100),
    window_length = 0.5, time_steps = 0.001, filter_edges = true)
EegFun.plot_tf(tf_data; baseline_interval = (-0.5, -0.2), baseline_method = :db,
colorrange = (-3, 3), yscale = :log10, colormap = :jet)

# Figure 13.11 A)
tf_data = EegFun.tf_stft(data_cohen, frequencies = 2:1:80, cycles = 7, time_steps
= 0.005, filter_edges = false)
EegFun.plot_tf(tf_data; baseline_interval = (-0.5, -0.2), baseline_method = :db,
colorrange = (-3, 3), yscale = :linear, colormap = :jet)

# Figure 13.14 A)
tf_data =
    EegFun.tf_stft(data_cohen, frequencies = logrange(2, 80, length = 30),
    window_length = 0.5, time_steps = 0.005, filter_edges = true)
EegFun.plot_tf(tf_data; baseline_interval = (-0.5, -0.2), baseline_method = :db,
colorrange = (-3, 3), yscale = :log10, colormap = :jet)

# Figure 13.14 B)
tf_data =
    EegFun.tf_stft(data_cohen, frequencies = logrange(2, 80, length = 30),
    window_length = 0.5, time_steps = 0.005, filter_edges = true)
EegFun.plot_tf(tf_data; baseline_interval = (-0.5, -0.2), baseline_method = :db,
colorrange = (-3, 3), yscale = :log10, colormap = :jet)

# Figure 13.14 C)
tf_data = EegFun.tf_stft(data_cohen, frequencies = logrange(2, 80, length = 30),
cycles = 5, time_steps = 0.005, filter_edges = true)
EegFun.plot_tf(tf_data; baseline_interval = (-0.5, -0.2), baseline_method = :db,
colorrange = (-3, 3), yscale = :log10, colormap = :jet)
```

- API Reference

16.5 TF Operations

TF Operations

This demo shows how to perform common operations on time-frequency data: baseline correction, channel averaging and differencing, condition averaging and differencing, and grand averaging.

Key Functions

Function	Purpose
`tf_baseline!` / `tf_baseline`	Apply baseline correction to TF data
`channel_average!` / `channel_average`	Average channel groups (TF-specific)
`channel_difference!` / `channel_difference`	Subtract channel groups (TF-specific)
`condition_average`	Average conditions together (TF)
`condition_difference`	Subtract conditions (TF)
`grand_average`	Average across participants (TF)

Important Notes

- All TF operations update **both** `data_power` and `data_phase` DataFrames consistently
- The API mirrors the ERP operations but is dispatched on `TimeFreqData`
- `tf_baseline!` supports multiple methods: `:db` , `:absolute` , `:relative` , `:relchange` , `:normchange` , `:zscore` , `:percent`

Workflow Summary

Baseline Correction

- Apply after TF decomposition and before statistical analysis
- Decibel (`:db`) is the most common method for EEG
- Cannot be applied twice — re-compute TF data to change baseline

Channel Operations

- `channel_average` creates ROI averages (e.g., midline, frontal cluster)
- `channel_difference` computes laterality or other contrasts
- Use `reduce = true` to drop original channels

Condition Operations

- `condition_difference` subtracts one condition's power from another
- `condition_average` pools conditions together

Grand Average

- Load multi-participant data with `read_all_data`
- `grand_average` groups by condition and averages power across participants

Code Examples

```
# Demo: Time-Frequency Operations
# Shows baseline correction, channel operations, condition operations,
# and grand averaging for time-frequency data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# CREATE EXAMPLE TF DATA
############################################################################

# Load and preprocess
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# Extract epochs
epoch_cfg = [
    EegFun.EpochCondition(name = "Trigger1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Trigger2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))

# Compute time-frequency decomposition (Morlet wavelets)
tf_data = EegFun.tf_morlet(epochs, frequencies = 4:1:30, cycles = 3)


############################################################################
# TF BASELINE CORRECTION
############################################################################

# Decibel baseline (default, most common)
tf_bl = EegFun.tf_baseline(tf_data[1], (-0.2, 0.0); method = :db)

# Other methods: :absolute, :relative, :relchange, :normchange, :zscore, :percent
tf_pct = EegFun.tf_baseline(tf_data[1], (-0.2, 0.0); method = :percent)
```

```
# In-place version
# EegFun.tf_baseline!(tf_data[1], (-0.2, 0.0); method = :db)

# Plot the result
EegFun.plot_tf(tf_bl, channel_selection = EegFun.channels([:Cz]))



######################################################################
# TF CHANNEL AVERAGE
######################################################################

# Average specific channel groups (creates new column, preserves originals)
tf_avg = EegFun.channel_average(tf_data[1], channel_selections =
[EegFun.channels([:Fz, :Cz, :Pz])], output_labels = [:midline])

# Reduce to only the averaged channel (drops originals)
tf_reduced =
    EegFun.channel_average(tf_data[1], channel_selections =
    [EegFun.channels([:Fz, :Cz, :Pz])], output_labels = [:midline], reduce = true)



######################################################################
# TF CHANNEL DIFFERENCE
######################################################################

# Laterality: left minus right hemisphere
tf_lat = EegFun.channel_difference(
    tf_data[1],
    channel_selection1 = EegFun.channels([:C3]),
    channel_selection2 = EegFun.channels([:C4]),
    channel_out = :laterality,
)



######################################################################
# TF CONDITION DIFFERENCE
######################################################################

# Subtract condition 2 from condition 1
tf_diff = EegFun.condition_difference(tf_data, [(1, 2)])

# Multiple differences
# tf_diff = EegFun.condition_difference(tf_data, [(1, 2), (3, 4)])
```

```
##########################################################################
# TF CONDITION AVERAGE
##########################################################################

# Average conditions into one
tf_avg_cond = EegFun.condition_average(tf_data, [[1, 2]])



##########################################################################
# TF GRAND AVERAGE
##########################################################################

# Grand average across participants (requires multi-participant data)
# all_tf = EegFun.read_all_data(EegFun.TimeFreqData, "derivatives/tf/tf_morlet")
# grand_avg = EegFun.grand_average(all_tf)

# With condition selection
# grand_avg = EegFun.grand_average(all_tf, condition_selection =
EegFun.conditions([1, 2]))
```

- API Reference

Chapter 17

Statistics

17.1 Statistics

Statistics

This demo shows permutation-based statistical testing for ERP data, with multiple thresholding and cluster correction approaches.

Key Functions

Function	Purpose
`prepare_stats`	Load multi-participant ERPs and prepare for statistical comparison
`analytic_test`	Parametric t-test (uncorrected or Bonferroni)
`permutation_test`	Cluster-based permutation test with multiple thresholding options
`plot_erp_stats`	Visualise results with significance shading and critical t-values

Thresholding Methods

Method	Description
`:parametric`	T-distribution threshold (fastest, default)
`:nonparametric_common`	Single threshold from pooled permutation distribution
`:nonparametric_individual`	Point-specific thresholds from permutation distributions

Cluster-Based Permutation Testing

Addresses the multiple comparisons problem in ERP analysis:
1. Compute test statistic at each time point × channel
2. Identify clusters of contiguous spatiotemporal significant effects
3. Permute condition labels and repeat (e.g., 1000 times)
4. Compare observed cluster mass to permutation distribution

Key advantages:

- Controls family-wise error rate without being overly conservative
- Sensitive to spatially and temporally distributed effects

Workflow Summary

Prepare Data

- Load group ERP data with `prepare_stats` specifying conditions, channels, and time intervals
- Supports `:paired` (within-subject) and `:independent` (between-subject) designs

Analytic Tests

- Quick uncorrected or Bonferroni-corrected t-tests for exploration

Permutation Tests

- Choose thresholding method based on assumptions and computational budget
- Set `cluster_type = :spatiotemporal` and `min_num_neighbors` for spatial constraints
- Visualise results with `plot_erp_stats`

Code Examples

```
# Demo: Permutation-Based Statistical Testing for ERPs
# Shows how to run permutation tests on ERP data using different
# thresholding and cluster correction approaches.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

input_dir = EegFun.example_path("data/julia/erps")
file_pattern = "erps"

println("Preparing data...")
stat_data = EegFun.prepare_stats(
    file_pattern,  # Pattern to match ERP files
    :paired;       # :paired for within-subject, :independent for between-subject
    input_dir = input_dir,
    condition_selection = EegFun.conditions([1, 2]), # Conditions to compare
    channel_selection = EegFun.channels(1:72),       # Select all 72 channels
    interval_selection = EegFun.times(),  # Full time interval
    baseline_interval = EegFun.times((-0.2, 0.0)),   # Baseline: -200 to 0 ms
    analysis_interval = EegFun.times((0.0, 1.0)),    # Analysis interval: 100-1000
    ms
```

```
)

# ------------------------------------------------------------------------
# Option: Analytic t-test with NO correction
# ------------------------------------------------------------------------
result_analytic_no = EegFun.analytic_test(stat_data)

EegFun.plot_erp_stats(
    result_analytic_no,
    channel_selection = EegFun.channels(:PO8),
    plot_erp = true,
    plot_difference = false,
    plot_significance = true,
    plot_critical_t = true,
)

# ------------------------------------------------------------------------
# Option: Analytic t-test with BONFERRONI correction (v. conservative; really only
useful for demonstrating)
# ------------------------------------------------------------------------
result_analytic_bonf = EegFun.analytic_test(stat_data, correction_method
= :bonferroni)

EegFun.plot_erp_stats(
    result_analytic_bonf,
    channel_selection = EegFun.channels(:PO8),
    plot_erp = true,
    plot_difference = false,
    plot_significance = true,
    plot_critical_t = true,
)

# ------------------------------------------------------------------------
# Option: Parametric Thresholding (Default, Fastest)
# ------------------------------------------------------------------------
@time result_permutation_parametric = EegFun.permutation_test(
    stat_data,
    n_permutations = 50000,           # Number of permutations (more = more
    accurate)
    threshold_method = :parametric,  # Use t-distribution for thresholding
    cluster_type = :spatiotemporal,  # Cluster in space AND time
    min_num_neighbors = 3,           # Pre-filter: need 3+ neighbors
    show_progress = true,            # Show progress bar
    use_gpu = false,
)
```

```
EegFun.plot_erp_stats(
    result_permutation_parametric,
    channel_selection = EegFun.channels(:PO8),
    plot_erp = true,
    plot_difference = false,
    plot_significance = true,
    plot_critical_t = true,
)



# ------------------------------------------------------------------------
# Option: Non-Parametric Common Thresholding
# ------------------------------------------------------------------------
# What actually happens:
# Run ALL permutations first (collect t-matrices)
# Pool all t-values from all permutations
# Compute (1-α) percentile threshold from pooled distribution
# Use this single threshold for all points
# Reuse stored t-matrices for cluster-level inference
# Equivalent to FieldTrip: method='montecarlo', corrMethod='cluster',
clusterThreshold='nonparametric_common'

@time result_permutation_nonparametric_common = EegFun.permutation_test(
    stat_data,
    n_permutations = 10000,
    threshold_method = :nonparametric_common,  # Non-parametric common threshold
    cluster_type = :spatiotemporal,
    min_num_neighbors = 3,
    show_progress = true,
    use_gpu = true,
)

# ------------------------------------------------------------------------
# Option: Non-Parametric Individual Thresholding
# ------------------------------------------------------------------------
# What actually happens:
# Run ALL permutations first (collect t-matrices)
# For each electrode × time point:
#    - Extract t-values from all permutations at that point
#    - Compute (1-α) percentile threshold from point-specific distribution
# Threshold observed data using point-specific thresholds
# Reuse stored t-matrices for cluster-level inference
# Equivalent to FieldTrip: method='montecarlo', corrMethod='cluster',
clusterThreshold='nonparametric_individual'
```

```
@time result_permutation_nonparametric_individual = EegFun.permutation_test(
    stat_data,
    n_permutations = 10000,
    threshold_method = :nonparametric_individual,  # Non-parametric individual
    thresholds
    cluster_type = :spatiotemporal,
    min_num_neighbors = 3,
    show_progress = true,
    use_gpu = true,
)
```

- API Reference

17.2 TF Statistics

TF Statistics

This demo shows statistical testing and topography visualization for time-frequency data, including analytic tests and cluster-based permutation tests.

Key Functions

Function	Purpose
`tf_morlet`	Compute TF decomposition (batch, from epoch files)
`prepare_stats`	Prepare multi-participant TF data for statistical comparison
`analytic_test`	Parametric test (fast, no permutations)
`permutation_test`	Cluster-based permutation test (robust)
`plot_tf`	Plot TF power for a single channel
`plot_tf_stats`	Per-channel TF heatmap with significance
`plot_topography`	TF topography for a frequency/time window
`plot_topography_stats`	Topography with statistical significance overlay

Workflow Summary

TF Decomposition

- Batch compute Morlet wavelets from epoch data with `tf_morlet`

Single-Participant Visualization

- `plot_tf` for per-channel time-frequency plots with baseline correction
- `plot_topography` for scalp maps at specific frequency bands and time windows

Group Statistics

- `prepare_stats` loads all participants and prepares for comparison (paired or independent)
- Supports frequency selection, time intervals, and baseline correction
- `analytic_test` for quick exploration
- `permutation_test` with spatiotemporal clustering for robust inference

Statistical Visualization

- `plot_tf_stats` shows per-channel significance as contours
- `plot_topography_stats` shows scalp topography with significance highlighting for specific frequency bands

Code Examples

```
# Demo: Time-Frequency Statistics & Topography Plots
# Shows TF topography plotting, statistical tests, and
# visualization using plot_topography_stats for TF data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# ================================================================================
# Regenerate TF morlet data from epoch data
# ================================================================================
epoch_dir = joinpath(homedir(), "Documents", "Julia", "TestDataSets",
"AttentionExp", "output_data")  # ← Update to your data path
println("Regenerating TF morlet data from epochs...")

EegFun.tf_morlet(
    "epochs_final";
    input_dir = epoch_dir,
    condition_selection = EegFun.conditions([1, 2]),
    frequencies = 1:2:40,
    cycles = 7,
    pad = :both,
)
```

```
# ================================================================================
# Setup
# ================================================================================
input_dir = joinpath(epoch_dir, "tf_morlet_epochs_final")
file_pattern = "epochs_final"

# ================================================================================
# Load a single participant to test plot_topography for TF data
# ================================================================================
println("Loading single participant TF data...")
single_data = EegFun.read_data(joinpath(input_dir, "example1_epochs_final.jld2"))

# Inspect conditions
println("Number of datasets: ", length(single_data))
for d in single_data
    println("  Condition $(d.condition): $(d.condition_name), file=$(d.file)")
end

# Pick the first condition for topo testing
tf_cond1 = single_data[1]
println("\nFrequencies: ", sort(unique(tf_cond1.data_power.freq)))
println("Time range: ", extrema(tf_cond1.data_power.time))
println("Channels: ", EegFun.channel_labels(tf_cond1))

# ================================================================================
# Test plot_topography for single TimeFreqData
# ================================================================================
println("\n--- Testing plot_topography (single TF) ---")

EegFun.plot_tf(tf_cond1, baseline_interval = (-0.5, -0.3), baseline_method = :db,
channel_selection = EegFun.channels(:Fp1))

# Alpha band (8-12 Hz) in the 200-400 ms window, with dB baseline
EegFun.plot_topography(
    tf_cond1,
    freq_range = (8.0, 20.0),
    interval_selection = EegFun.times(0.2, 0.4),
    baseline_interval = (-0.5, -0.3),
    baseline_method = :percent,
)

# ================================================================================
# Test plot_topography for Vector{TimeFreqData} (multiple conditions)
# ================================================================================
println("\n--- Testing plot_topography (multi-condition) ---")
```

```
EegFun.plot_topography(single_data, freq_range = (8.0, 12.0), interval_selection =
EegFun.times(0.2, 0.4), baseline_interval = (-0.3, 0.0))

# ============================================================================
# Prepare TF statistics (all participants, 2 conditions)
# ============================================================================
println("\n--- Preparing TF statistics ---")

tf_stat_data = EegFun.prepare_stats(
    file_pattern,
    :paired;
    input_dir = input_dir,
    condition_selection = EegFun.conditions([1, 2]),
    frequency_selection = (2.0, 40.0),          # 2-40 Hz
    interval_selection = (0.0, 1.0),            # 0-1000 ms
    baseline_interval = (-0.5, -0.3),           # Baseline: -500 to -300 ms
    baseline_method = :db,
)
println(tf_stat_data)

# ============================================================================
# Analytic test (fast, no permutations)
# ============================================================================
println("\n--- Running analytic test ---")
result_analytic = EegFun.analytic_test(tf_stat_data)
println(result_analytic)

# ============================================================================
# Plot TF stats (existing per-channel heatmap)
# ============================================================================
println("\n--- Plotting TF stats (heatmap) ---")

EegFun.plot_tf_stats(result_analytic, channel_selection = EegFun.channels(:Cz),
significance = :contour)

# ============================================================================
# Plot TF topo stats (NEW — topography with significance)
# ============================================================================
println("\n--- Plotting TF topo stats (alpha band) ---")

EegFun.plot_topography_stats(result_analytic, freq_range = (8.0, 12.0), n_topos =
10, highlight_threshold = 0.5)

# Theta band
```

```
println("\n--- Plotting TF topo stats (theta band) ---")

EegFun.plot_topography_stats(result_analytic, freq_range = (4.0, 7.0), n_topos =
8, highlight_threshold = 0.3, topo_data = :difference)

# ============================================================================
# Cluster permutation test (slower but more robust)
# ============================================================================
println("\n--- Running cluster permutation test ---")

result_perm = EegFun.permutation_test(
    tf_stat_data,
    n_permutations = 500,
    threshold_method = :parametric,
    cluster_type = :spatiotemporal,
    min_num_neighbors = 3,
    show_progress = true,
)
println(result_perm)

# ============================================================================
# Plot cluster permutation TF topo stats
# ============================================================================
println("\n--- Plotting permutation TF topo stats ---")

EegFun.plot_topography_stats(result_perm, freq_range = (8.0, 12.0), n_topos = 10,
highlight_threshold = 0.5)

println("\n✓ All TF stats tests completed!")
```

- API Reference

17.3 Decoding

Decoding

This demo demonstrates time-resolved multivariate pattern analysis (MVPA) for decoding experimental conditions from EEG data.

What is MVPA Decoding?

MVPA uses machine learning classifiers to decode experimental conditions from spatial patterns of brain activity:

- **Multivariate**: Uses all channels simultaneously (not one channel at a time)
- **Pattern analysis**: Finds distributed spatial patterns that discriminate conditions
- **Time-resolved**: Decodes at each time point separately to track when information is available

This reveals **when** and **how well** neural patterns can distinguish between experimental conditions.

Why Use Decoding?

Information content:

Decoding tells you **when information is present** in neural patterns, even if traditional ERPs don't show clear differences.

Spatial patterns:

Uses distributed activity across channels, potentially more sensitive than univariate approaches.

Temporal dynamics:

Track when discriminative information emerges, peaks, and decays across the trial.

Workflow

1. Prepare data

:

```
participant_epochs = EegFun.prepare_decoding(
    "epochs_final",
    condition_selection = EegFun.conditions([1, 2]),
    sample_selection = EegFun.samples((-0.2, 1.5))
)
```

2. Run decoding

:

```
all_decoded = EegFun.decode_libsvm(
    participant_epochs,
    n_iterations = 5,
    n_folds = 3
)
```

Uses cross-validated support vector machine (SVM) classification.

3. Grand average

:

```
grand_avg = EegFun.grand_average(all_decoded)
```

Average decoding accuracy across participants.

4. Statistical testing

:

```
stats = EegFun.test_against_chance(all_decoded, alpha = 0.05)
stats_cluster = EegFun.test_against_chance_cluster(all_decoded, alpha = 0.05)
```

5. Visualization

:

```
EegFun.plot_decoding(grand_avg, stats)
```

Cross-Validation

Decoding uses **k-fold cross-validation**:
1. Split data into k folds
2. Train on k-1 folds
3. Test on held-out fold
4. Repeat for all folds
5. Average accuracy across folds

This prevents overfitting and gives unbiased accuracy estimates.

Statistical Testing

Multiple Comparison Correction:

Method	Description
:none	No correction (liberal)
:bonferroni	Divide alpha by number of time points (conservative)
:cluster	Cluster-based permutation testing (recommended)

Cluster-based testing:

Identifies contiguous time intervals where decoding is above chance while controlling family-wise error rate.

Interpreting Results

Decoding accuracy:

- **50%** = Chance level (for 2-class problems)
- **60-70%** = Moderate decoding (information present)
- **>80%** = Strong decoding (highly discriminative patterns)

Temporal profile:

- **Early peaks** (< 200 ms): Sensory processing
- **Mid-latency** (200-400 ms): Perceptual/cognitive processing
- **Late sustained** (> 400 ms): Decision-making, motor preparation

Significance:

Only interpret time points that survive statistical testing with appropriate correction.

Demo Structure

Synthetic data:

Creates artificial data with controllable signal-to-noise ratio to validate the pipeline.

Real data:

Applies decoding to actual experimental data, comparing two conditions across participants.

Multiple corrections:

Demonstrates different statistical correction methods for comparison.

Best Practices

Data requirements:

- **Balanced classes**: Equal number of trials per condition (use `equalize_trials = true`)
- **Sufficient trials**: At least 30-50 trials per condition
- **Clean data**: Artifact rejection before decoding

Cross-validation settings:

- **n_folds**: 3-10 folds (fewer for small trial counts)
- **n_iterations**: 5-20 iterations (more = more stable, but slower)

Statistical testing:

- Use cluster-based correction as default
- Bonferroni is very conservative for time-series data
- Report corrected p-values and time intervals

Code Examples

```
# Demo: Multivariate Pattern Analysis (MVPA) / Decoding
# Shows time-resolved classification using LIBSVM, with synthetic data creation,
# grand averaging, and statistical testing (uncorrected, Bonferroni, cluster-
based).

using EegFun
using GLMakie
using DataFrames
using Random

@info EegFun.section("MVPA/DECODING MANUAL TEST")

# Create synthetic data for 10 participants
# Adjustable difficulty via signal_strength and noise_level
difficulty = "hard"

if difficulty == "easy" # maybe a bit extreme! :-)
    signal_strength, noise_level = 2.0, 0.1
elseif difficulty == "medium"
    signal_strength, noise_level = 1.0, 0.3
else  # hard
```

```
    signal_strength, noise_level = 0.25, 0.75
end

println("  Using difficulty: $difficulty (signal=$signal_strength,
noise=$noise_level)")
all_synthetic = [
    [
        EegFun.create_synthetic_epochs(p, 1, "Cond1", 100; signal_strength =
        signal_strength, noise_level = noise_level),
        EegFun.create_synthetic_epochs(p, 2, "Cond2", 100; signal_strength =
        signal_strength, noise_level = noise_level),
    ] for p = 1:10
]
EegFun.plot_epochs(all_synthetic[1]) # VP1
EegFun.plot_epochs(all_synthetic[2]) # VP2
EegFun.plot_epochs(all_synthetic[3]) # VP3
# and so on

# Decode synthetic data (batch method)
decoded_synthetic = EegFun.decode_libsvm(all_synthetic; n_iterations = 20, n_folds
= 3)
grand_avg_synthetic = EegFun.grand_average(decoded_synthetic)

EegFun.plot_decoding(decoded_synthetic)    # every "VP"
EegFun.plot_decoding(grand_avg_synthetic)  # grand average

# Test and plot with different methods
stats_none = EegFun.test_against_chance(decoded_synthetic, alpha = 0.05,
correction_method = :none)
EegFun.plot_decoding(grand_avg_synthetic, stats_none, plot_title = "Synthetic
Data: No Correction")

stats_bonf = EegFun.test_against_chance(decoded_synthetic, alpha = 0.05,
correction_method = :bonferroni)
EegFun.plot_decoding(grand_avg_synthetic, stats_bonf, plot_title = "Synthetic
Data: Bonferroni")

stats_cluster = EegFun.test_against_chance_cluster(decoded_synthetic, alpha =
0.05)
EegFun.plot_decoding(grand_avg_synthetic, stats_cluster, plot_title = "Synthetic
Data: Cluster-based")
```

- API Reference

17.4 RSA

RSA

Overview

This demo demonstrates Representational Similarity Analysis (RSA) for analyzing neural representational geometries.

What is RSA?

RSA quantifies how neural patterns represent different stimuli/conditions:
1. **Compute dissimilarity**: Calculate distance between conditions (e.g., correlation distance)
2. **Create RDM**: Representational Dissimilarity Matrix showing all pairwise distances
3. **Compare geometries**: Relate neural RDMs to behavior, models, or other brain regions

RSA Workflow

1. **Extract patterns**: Get spatial or spatiotemporal patterns for each condition
2. **Compute RDM**: Calculate dissimilarity between all condition pairs
3. **Statistical testing**: Assess significance of representational structure
4. **Model comparison**: Compare neural RDMs to theoretical predictions

Applications

- Compare representational geometry across time
- Test computational models of cognition
- Relate neural to behavioral similarity
- Cross-modality comparisons

Advantages

- Multivariate (uses all channels)
- Model-agnostic representational space
- Can compare across different measurement modalities

Code Examples

```
# Demo: Representational Similarity Analysis (RSA)
# Shows RSA analysis with multiple participants, dissimilarity measures,
# grand averaging with noise ceiling, RDM heatmaps, and timecourse visualization.

using EegFun
using GLMakie

@info EegFun.section("Representational Similarity Analysis (RSA)")
println("Basic RSA analysis...")

# Create test data for multiple participants
all_participant_epochs = [EegFun.create_test_epoch_data_vector(
    conditions = 1:3,
    n_epochs = 100,
```

```
    n_channels = 3,
    fs = 250,
    n = 250,  # 1 second of data at 250 Hz
) for _ = 1:5]

# Perform RSA for each participant using batch method
all_rsa_results = EegFun.rsa(all_participant_epochs; dissimilarity_measure
= :correlation, average_trials = true)
all_rsa_results =
    EegFun.rsa(all_participant_epochs; dissimilarity_measure = :correlation,
    average_trials = true, normalize_method = :zscore)

# Compute grand average with noise ceiling
grand_avg_rsa = EegFun.grand_average(all_rsa_results)

# RSA Plots
EegFun.plot_rdm_heatmap(grand_avg_rsa, time_point = 0.0, plot_title = "Grand
Average RDM")
EegFun.plot_rdm_heatmap(grand_avg_rsa, plot_title = "Grand Average RDM (time-
averaged)")
EegFun.plot_rdm_timecourse(grand_avg_rsa, plot_title = "Dissimilarity Over Time")
```

- API Reference

Chapter 18

Plotting

18.1 Plot Artifacts

Plot Artifacts

This demo demonstrates methods for detecting and visualizing artifacts in EEG data.

What are EEG Artifacts?

Artifacts are unwanted signals from non-neural sources:

Physiological:

- **Eye movements/blinks**: Large amplitude deflections in frontal channels
- **Muscle activity (EMG)**: High-frequency noise from jaw, face, or neck
- **Cardiac signals (ECG)**: Heart electrical activity contamination

External:

- **Electrode issues**: Poor contact, bridging, movement
- **Line noise**: 50/60 Hz environmental interference
- **Movement**: Low-frequency drifts and transients

Detection Methods

Threshold-based:

- Absolute amplitude criteria (e.g., ±100 μV)
- Simple and interpretable
- Quick identification of extreme values

Statistical:

- Z-score outlier detection
- Variance, range, and kurtosis metrics
- Flags unusual distributions

Visual inspection:

- Grid-based epoch review
- Manual validation of automatic detection
- Refinement of rejection criteria

Visualization Approaches

This demo shows multiple complementary views:

Channel-wise:

- Which electrodes are most affected
- Spatial patterns of artifact contamination
- EOG correlation for eye artifact identification

Epoch-wise:

- Trial-by-trial rejection statistics
- Temporal distribution of artifacts
- Condition-specific artifact rates

Rejection summaries:

- Percentage of data flagged per condition
- Channel quality metrics
- Decision support for repair vs. rejection

Artifact Management Strategy

Repair (Interpolation):

- Use for isolated bad channels in good epochs
- Maintains trial count
- Cannot fix widespread contamination

Rejection:

- Use for severely contaminated epochs
- Removes entire trials
- Reduces but cleans the dataset

Hybrid approach (recommended):

1. Repair isolated channel issues
2. Reject epochs with widespread artifacts

Workflow Summary

This demo shows:

1. **Loading data**: Continuous or epoched EEG
2. **Detection**: Applying threshold and statistical criteria
3. **Visualization**: Plotting detected artifacts across channels and epochs
4. **Assessment**: Evaluating artifact patterns to guide cleaning strategy
5. **Reporting**: Quantifying data attrition for methods documentation

Critical Principle

> **Garbage in, garbage out**: No computational method can rescue poor quality data. The most effective strategy is careful data collection with proper electrode preparation and participant instruction. Artifact management is a secondary cleaning strategy, not a substitute for quality acquisition.

Code Examples

```
# Demo: Artifact Detection Visualization
# Shows plots for visualizing artifact detection and rejection.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));
dat = EegFun.create_eegfun_data(dat)

# minimal preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)

# extract epochs
epoch_cfg = EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))

# Artifacts
artifacts = EegFun.detect_bad_epochs_automatic(epochs)

# Plot artifacts
EegFun.plot_artifact_detection(epochs, artifacts)

# Or with custom criteria
artifacts = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion = 100,
z_criterion = 0)
EegFun.plot_artifact_detection(epochs, artifacts)

artifacts = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion = 0,
z_criterion = 3)
EegFun.plot_artifact_detection(epochs, artifacts)

# extract multiple epochs
epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
]
```

```
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))

# Artifacts
artifacts = EegFun.detect_bad_epochs_automatic(epochs)

# Plot artifacts
EegFun.plot_artifact_detection(epochs, artifacts)
EegFun.plot_artifact_detection(epochs[1], artifacts[1])



# Interactive artifact detection
EegFun.detect_bad_epochs_interactive(epochs, dims = (4, 4))

# Or combine with automatic artifact detection
artifacts = EegFun.detect_bad_epochs_automatic(epochs)
EegFun.detect_bad_epochs_interactive(epochs, artifact_info = artifacts, dims = (4,
4))

# Or combine with automatic artifact detection
EegFun.detect_bad_epochs_interactive(epochs, artifact_info = artifacts, dims = (4,
4), colormap = :GnBu)
```

- API Reference

18.2 Plot Channel Spectrum

Plot Channel Spectrum

This demo shows how to plot power spectra from `SpectrumData` objects.

`plot_channel_spectrum` Multiple Dispatch

Function	Input	Best For
`plot_channel_spectrum(dat)`	Raw/preprocessed data	Quick interactive inspection with Welch's method
`plot_channel_spectrum(spectrum)`	`SpectrumData`	Plotting pre-computed spectra with full control

Key Parameters

Parameter	Default	Description
`channel_selection`	`EegFun.channels()`	Which channels to plot
`xscale`	`:linear`	`:linear` or `:log10`

`yscale`	`:linear`	`:linear` or `:log10`
`unit`	`:linear`	`:linear` (μV²/Hz) or `:dB`
`max_freq`	`nothing`	Maximum frequency to display
`legend`	`true`	Show channel name legend

What You'll Learn

1. Plotting spectra for all or selected channels
2. Switching between linear and log scales
3. Using decibel units
4. Limiting the frequency range

Code Examples

```
# Demo: Channel Power Spectrum Plotting
# Shows how to visualise power spectra from both raw continuous data
# and pre-computed SpectrumData, featuring interactive controls for
# axis scaling, unit conversions, and frequency band overlays.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

################################################################################
# LOAD AND PREPARE DATA
################################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

EegFun.highpass_filter!(dat, 0.5)

################################################################################
# WORKFLOW 1: COMPUTING ON THE FLY FROM RAW DATA
################################################################################
# plot_channel_spectrum can compute the spectrum dynamically from raw data

# Plot all channels
```

```
EegFun.plot_channel_spectrum(dat)

# Plot a single channel with a specific title
EegFun.plot_channel_spectrum(dat, channel_selection = EegFun.channels([:Fp1]),
plot_title = "Fp1 Power Spectrum")

# Plot multiple frontal channels
EegFun.plot_channel_spectrum(
    dat,
    channel_selection = EegFun.channels([:Fp1, :Fp2, :F3, :F4]),
    plot_title = "Frontal Channels Power Spectrum",
)



##########################################################################
# WORKFLOW 2: PLOTTING PRE-COMPUTED SPECTRUM DATA
##########################################################################
# If you have already run freq_spectrum, you can pass the result directly

# Compute spectrum first
spectrum = EegFun.freq_spectrum(dat)

# Plot all channels from pre-computed data
EegFun.plot_channel_spectrum(spectrum)

# Single channel from pre-computed data
EegFun.plot_channel_spectrum(spectrum, channel_selection = EegFun.channels(:Cz))

##########################################################################
# INTERACTIVE/INITIAL PARAMETER OVERRIDES
##########################################################################
# Note: You can toggle these visually in the GUI, but you can also
# set them as starting defaults via keyword arguments.

# Start with log-scale axes
EegFun.plot_channel_spectrum(spectrum, channel_selection = EegFun.channels(:Cz),
xscale = :log10, yscale = :log10)

# Start with Decibel (dB) units instead of linear power
EegFun.plot_channel_spectrum(spectrum, channel_selection =
EegFun.channels([:Cz, :Pz]), unit = :dB, max_freq = 100.0)

# Custom styling for lines
EegFun.plot_channel_spectrum(
    spectrum,
```

```
    channel_selection = EegFun.channels([:Cz, :Oz]),
    linewidth = 3,
    line_alpha = 0.6,
    plot_title = "Custom Styled Power Spectrum",
)
```

- API Reference

18.3 Plot Channel Summary

Plot Channel Summary

This demo shows how to compute and visualize channel summary statistics for quality control and data characterization.

What is Channel Summary?

Channel summary computes aggregate statistics across time for each channel:

- **Descriptive statistics**: Mean, median, min, max, standard deviation
- **Distribution metrics**: Variance, kurtosis, skewness
- **Range metrics**: Absolute range, interquartile range

These statistics help identify problematic channels, assess data quality, and characterize recording conditions.

Available Metrics

The `channel_summary()` function computes:

Metric	Description
mean	Average amplitude
median	Median amplitude
std	Standard deviation
var	Variance
zvar	Z-scored variance
min	Minimum value
max	Maximum value
range	Max - Min
kurtosis	Distribution tail weight

Visualization Options

Single Metric:

```
EegFun.plot_channel_summary(cs, :range)
```

Shows one statistic across all channels as a bar plot

Multiple Metrics:

```
EegFun.plot_channel_summary(cs, [:min, :max, :std, :range])
```

Creates multiple subplots for comparison

Epoched Data:

```
EegFun.plot_channel_summary(cs, :range, average_over = :epoch)
```

Averages statistics across epochs before plotting

Common Use Cases

1. Quality Control

:

- **High variance/range**: Noisy or artifactual channels
- **Extreme min/max**: Electrode saturation or poor contact
- **High z-variance**: Outlier channels requiring interpolation

2. Preprocessing Validation

:

- **After filtering**: Verify reduced variance in target frequencies
- **After baseline correction**: Check mean values near zero
- **After rereferencing**: Assess reference choice effectiveness

3. Hardware Issues

:

- **Systematic patterns**: Poor grounding, bridging between neighboring channels
- **Single channel anomalies**: Loose connection, bad electrode

Interpretation

Healthy channels typically show:

- Moderate, similar variance across scalp
- No extreme outliers in min/max
- Range appropriate for recording gain settings

Problematic patterns:

Very high variance (>2-3× median):

- Likely artifact (muscle, movement, poor contact)
- Consider interpolation or exclusion

Very low variance (<0.5× median):

- May indicate flat/dead channel
- Check hardware connections

Extreme values (saturated at amplifier limits):

- Indicates clipping/saturation
- Cannot be recovered, must exclude

Workflow Summary

This demo demonstrates:
1. **Load and preprocess** continuous data
2. **Compute summaries** with flexible channel/sample selection
3. **Plot single or multiple metrics** for visualization
4. **Apply to epoched data** with epoch averaging

The channel selection examples show how to:

- Select specific channels by name
- Use pattern matching (e.g., midline channels ending in "z")
- Apply custom sample selections

Code Examples

```
# Demo: Channel Summary Plots
# Shows visualization of channel-wise summary statistics.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));
dat = EegFun.create_eegfun_data(dat);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)

# basic channel summary statistics
cs = EegFun.channel_summary(dat)
cs = EegFun.channel_summary(dat, channel_selection =
EegFun.channels([:Fp1, :Fp2]))
cs = EegFun.channel_summary(dat, channel_selection =
EegFun.channels([:Fp1, :Fp2]), sample_selection = x -> x.sample .< 2000)
```

```
cs = EegFun.channel_summary(dat, channel_selection = x -> endswith.(string.(x),
"z")) # all midline channels
cs = EegFun.channel_summary(dat, channel_selection = x -> .!(endswith.(string.(x),
"z"))) # all non-midline channels

# Plotting Channel Summaries
EegFun.plot_channel_summary(cs, :range)
EegFun.plot_channel_summary(cs, :min)
EegFun.plot_channel_summary(cs, :min, bar_color = :red)
EegFun.plot_channel_summary(cs, [:min, :max, :std, :range, :var, :zvar])
EegFun.plot_channel_summary(cs, [:range, :var])

################################
# Epoched DataFrameEeg
################################
# some epoched data
epoch_cfg = EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))
cs = EegFun.channel_summary(epochs)

EegFun.plot_channel_summary(cs, :range, average_over = :epoch)
EegFun.plot_channel_summary(cs, [:min, :max, :std, :range, :var, :zvar],
average_over = :epoch)

# some epoched data
epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))
cs = EegFun.channel_summary(epochs)

EegFun.plot_channel_summary(cs, :range, average_over = :epoch)
EegFun.plot_channel_summary(cs, [:min, :max, :std, :range, :var, :zvar],
average_over = :epoch)
```

- API Reference

18.4 Plot Correlation Heatmap

Plot Correlation Heatmap

This demo demonstrates how to compute and visualize channel correlation matrices for quality control and artifact detection.

What is a Correlation Matrix?

A correlation matrix shows the Pearson correlation coefficient between all channel pairs:

- **Values range from −1 to +1**
- **High positive correlation** (near +1): Channels move together
- **Low correlation** (near 0): Channels are independent
- **Negative correlation** (near −1): Channels move in opposite directions

Why Use Correlation Matrices?

Quality Control:

- **Identify bad channels**: Low correlation with all neighbors
- **Detect bridging**: Unexpectedly high correlation between channels
- **Assess reference choice**: Impact on spatial correlation structure

Visualization Types

Full Correlation Heatmap:

Shows all channel-to-channel correlations as a matrix where:

- **Diagonal**: Always 1.0 (channel correlated with itself)
- **Off-diagonal**: Correlation between channel pairs
- **Color scale**: Red (high correlation) to blue (low correlation)

Dual Selection Heatmap:

Correlates two different channel sets:

- **Rows**: One set of channels (e.g., all EEG channels)
- **Columns**: Another set (e.g., EOG channels)

Layout Overlay:

Displays channel-to-channel correlations directly on the 2D sensor layout:

- Select a reference channel
- See its correlation with all other channels as colored dots
- **Use case**: Quick visual inspection of spatial correlation patterns

Expected Patterns

Healthy Data:

- **High correlation with neighbors** (0.7-0.9): Nearby channels capture similar signals
- **Decreasing correlation with distance**: Far channels less correlated
- **Symmetric matrix**: Correlation(A,B) = Correlation(B,A)

Problematic Patterns:

Single bad channel:

- Low correlation (<0.3) with all neighbors
- Stands out as a blue row/column in heatmap
- **Solution**: Interpolate or exclude

Bridged channels:

- Abnormally v. high correlation (>0.99) between adjacent channels
- Indicates electrical connection between electrodes
- **Solution**: Usually, more careful setup is required to avoid this

EOG contamination:

- High correlation between frontal channels and EOG
- Expected for Fp1/Fp2 (near eyes)
- **Solution**: Use ICA for removal if excessive

Common Use Cases

1. Post-Acquisition Quality Check

:

```
cm = EegFun.correlation_matrix(dat)
EegFun.plot_correlation_heatmap(cm)
```

Quick overview to identify problematic channels

2. Targeted Channel Inspection

:

```
cm = EegFun.correlation_matrix(dat, channel_selection =
EegFun.channels([:Fp1, :Fp2, :C3, :C4]))
```

Focus on specific regions or suspected bad channels

3. EOG Artifact Assessment

:

```
cm = EegFun.correlation_matrix_dual_selection(
    dat,
    channel_selection1 = EegFun.channels(),  # All EEG
    channel_selection2 = EegFun.channels([:vEOG, :hEOG])  # EOG
)
```

Identify which channels are affected by eye movements

Workflow Summary

This demo shows:

1. **Compute full correlation matrix** for all channels
2. **Visualize as heatmap** for overview
3. **Compute subset** targeting specific channels
4. **Create EOG channels** for artifact assessment
5. **Dual selection correlation** to identify EOG-contaminated channels
6. **Layout overlay** for spatial visualization

The correlation matrix is a powerful diagnostic tool for both quality control and understanding spatial relationships in your data!

Code Examples

```
# Demo: Correlation Heatmap
# Shows correlation matrices between channels or trials.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)

# Correlation Matrix
cm = EegFun.correlation_matrix(dat)
EegFun.plot_correlation_heatmap(cm, plot_title = "Full Correlation Matrix")

cm = EegFun.correlation_matrix(dat, channel_selection =
EegFun.channels([:Fp1, :Fp2, :C3, :C4]))
EegFun.plot_correlation_heatmap(cm, plot_title = "Selected Correlations")

# Calculate EOG signals
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
); # vertical EOG = mean(Fp1, Fp2) - mean(IO1, I02)
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
```

```
    channel_out = :hEOG,
); # horizontal EOG = F9 - F10

# Calculate correlations between all channels and EOG channels
cm_eog = EegFun.correlation_matrix_dual_selection(
    dat,
    sample_selection = EegFun.samples(),  # All samples
    channel_selection1 = EegFun.channels(),  # All EEG channels
    channel_selection2 = EegFun.channels([:vEOG, :hEOG]),  # EOG channels
)

# Visualize which EEG channels are correlated with EOG
EegFun.plot_correlation_heatmap(cm_eog, plot_title = "EEG-EOG Correlations",
xlabel = "EOG Channels", ylabel = "EEG Channels")

# Display channel correlations in layout
cm = EegFun.correlation_matrix(dat)
EegFun.plot_layout_2d(layout, correlation_matrix = cm)
```

- API Reference

18.5 Plot Epochs

Plot Epochs

This demo demonstrates visualizing individual epoch waveforms with flexible layout options.

What is Epoch Plotting?

Epoch plots display individual trial timecourses, showing:

- **Single-trial waveforms**: Raw epoch data (not averaged)
- **Time-locked activity**: Aligned to stimulus or response events
- **Trial variability**: Individual differences across repetitions
- **ERP overlay**: Optional averaged waveform on top

Layout Options

`:single` (default):

- All channels overlaid on one plot
- Best for comparing few channels
- Clear view of channel differences

`:grid` :

- Each channel in separate subplot
- Auto-calculated grid dimensions
- Best for many channels

- Synchronized axes

`:topo` :

- Channels arranged by scalp position
- Preserves spatial relationships
- Intuitive interpretation
- Optional scale axis

Use Cases

Trial-level inspection:

- View variability across trials
- Identify outlier epochs
- Assess signal quality

Verify epoching:

- Check time interval selection
- Confirm event alignment
- Validate baseline correction

Artifact detection:

- Spot trial-specific artifacts
- Find contaminated epochs
- Guide rejection decisions

Condition comparison:

- Overlay multiple conditions
- Compare trial-level activity
- Assess consistency

Customization Options

Channel selection:

- Select specific channels or regions
- Focus on electrodes of interest

Visual styling:

- Individual trial transparency
- Average line width
- Color schemes
- Axis limits

Layout parameters:

- Grid dimensions
- Spacing and gaps
- Scale axis (topo layout)

Interactive Features

When `interactive=true` (default):

- **Control panel** (press 'c'): Toggle conditions, adjust baseline
- **Keyboard shortcuts**: Scaling, help
- **Linked axes**: Synchronized zoom/pan across subplots

Workflow Summary

This demo shows epoch visualization workflows:

Load and Prepare Data

- Read raw data
- Apply preprocessing
- Extract epochs

Basic Plotting

- Plot single condition
- Select specific channels
- Compare conditions with overlay

Layout Options

- Single plot: All channels together
- Grid layout: Individual subplots
- Topo layout: Spatial arrangement

Channel Selection

- Focus on specific channels
- Combine with different layouts
- Maintain consistent scaling

Code Examples

```
# Demo: Epoch Plotting
# Shows epoch visualization with channel selection and overlay options.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie
```

```
# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)

# EPOCHS
epoch_cfg = EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.5, 1.0))

# Basic plots for Epochs
EegFun.plot_epochs(epochs) # not that useful as crowded!
EegFun.plot_epochs(epochs, channel_selection = EegFun.channels([:PO7, :PO8]))

EegFun.plot_epochs(epochs, layout = :grid)
EegFun.plot_epochs(epochs, layout = :grid, channel_selection =
EegFun.channels([:Fp1, :Fp2, :PO7, :PO8]))

EegFun.plot_epochs(epochs, layout = :topo)
EegFun.plot_epochs(epochs, layout = :topo, add_xy_origin = false, theme_fontsize =
10, layout_kwargs = (; topo_show_scale = true))

EegFun.plot_epochs(epochs, channel_selection = EegFun.channels([:Fp1, :Fp2, :Cz]))
EegFun.plot_epochs(epochs, channel_selection = EegFun.channels([:Fp1, :Fp2, :Cz]);
layout = :topo)
```

- API Reference

18.6 Plot ERP

Plot ERP

This demo demonstrates plotting event-related potentials (ERPs) with comprehensive customization options.

What is an ERP Plot?

ERP plots visualize averaged brain responses time-locked to events:

- **Time course**: Amplitude changes over time
- **Waveforms**: Characteristic positive and negative deflections
- **Condition comparison**: Overlay multiple experimental conditions
- **Channel-specific**: View individual electrodes or averages

Layout Options

The demo shows three layout modes:

Layout	Description
:single	One plot with overlaid conditions
:grid	Multiple subplots (one per channel)
:topo	Channels arranged by scalp location

Single Layout

Average across channels:

```
EegFun.plot_erp(erps, average_channels = true)
```

Shows grand average waveform across all selected channels.

Individual channels:

```
EegFun.plot_erp(erps, average_channels = false, colormap = :viridis)
```

Overlays all channels with color-coding.

Selected channels:

```
EegFun.plot_erp(erps,
    channel_selection = EegFun.channels([:Cz, :PO7, :PO8]),
    average_channels = false
)
```

Shows only specified channels.

Grid Layout

Displays multiple channels as subplots:

```
EegFun.plot_erp(erps, layout = :grid)
```

Custom grid dimensions:

```
EegFun.plot_erp(erps,
    channel_selection = EegFun.channels([:F3, :Cz, :PO7, :PO8, :Fp1, :Fp2]),
    layout = :grid,
    layout_kwargs = (; grid_dims = (3, 2))  # 3 rows × 2 columns
```

```
)
```

Skip positions:

```
EegFun.plot_erp(erps,
    layout_kwargs = (; grid_dims = (3, 4), grid_skip_positions = [(2, 1), (2, 3)])
    # Leave empty
)
```

Creates custom layouts with empty spaces.

Adjust spacing:

```
EegFun.plot_erp(erps,
    layout = :grid,
    layout_kwargs = (; grid_rowgap = 0, grid_colgap = 0)
)
```

Topographic Layout

Arranges channels by scalp position:

```
EegFun.plot_erp(erps, layout = :topo)
```

Each channel plotted at its actual spatial location for intuitive interpretation.

Customization Options

Y-axis orientation:

```
EegFun.plot_erp(erps, yreversed = true)  # Negative up (common convention)
```

Legend placement:

```
EegFun.plot_erp(erps,
    legend_channel = [:Fp1, :M2],  # Channels for legend
    legend_nbanks = 3              # Number of legend columns
)
```

Figure padding:

```
EegFun.plot_erp(erps,
    figure_padding = (150, 150, 150, 150)  # left, right, bottom, top
)
```

Combining with Topography

Create publication-quality figures with embedded topographies:

```
using GLMakie
fig = Figure(size = (800, 800))
```

```
ax1 = Axis(fig[1, 1])
ax2 = Axis(fig[1, 1], width = Relative(0.2), height = Relative(0.2))

EegFun.plot_erp!(fig, ax1, erps, average_channels = true)
EegFun.plot_topography!(fig, ax2, erps[1],
    point_plot = false,
    label_plot = false,
    colorbar_plot = true
)
```

Shows ERP waveform with scalp distribution at a specific time point.

Common Use Cases

Condition comparison:

- Overlay multiple experimental conditions
- Identify differences in amplitude or latency
- Statistical intervals highlighted

Component identification:

- Classic ERP components (N1, P1, N170, P3, etc.)
- Measure peak amplitudes and latencies
- Compare across channels

Publication figures:

- High-quality vector graphics
- Customizable colors and styles
- Grid layouts for multiple channels

Interpretation

Positive/negative deflections:

- **P1, P2, P3**: Positive peaks (sometimes plotted downward with yreversed = true)
- **N1, N2, N4**: Negative peaks (plotted upward)

Typical components:

- **P1/N1** (100-200 ms): Early sensory processing
- **N170** (170 ms): Face perception (occipito-temporal)
- **P3** (300-600 ms): Attention, memory updating
- **N400** (400 ms): Semantic processing

Workflow Summary

This demo shows:
1. **Basic plotting**: All three layouts
2. **Channel averaging**: Grand average vs individual channels
3. **Custom grids**: Flexible subplot arrangements

4. **Grid customization**: Gaps, skip positions, padding
5. **Legend control**: Placement and formatting
6. **Combined plots**: ERP + topography insets

ERP plots are the foundation of event-related brain potential analysis!

Code Examples

```
# Demo: ERP Plotting
# Shows ERP waveform visualization with various customization options.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie
using JLD2

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

# EPOCHS -> ERPs
epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))
erps = EegFun.average_epochs(epochs)

EegFun.plot_erp(erps, layout = :single)
EegFun.plot_erp(erps, layout = :grid)
EegFun.plot_erp(erps[1], layout = :topo)
```

```
with_theme(Theme(colormap = :coolwarm)) do
    EegFun.plot_erp(
        erps,
        layout = :grid,
        channel_selection = EegFun.channels([:Fp1, :M2]),
        legend_channel = [:Fp1, :M2],
        yreversed = true,
        theme_fontsize = 30,
    )
end

with_theme(theme_dark()) do
    EegFun.plot_erp(erps, layout = :grid, legend_channel = [:Fp1, :M2], yreversed
    = true)
end

EegFun.plot_erp(
    erps,
    channel_selection = EegFun.channels([:F3, :Cz, :PO7, :PO8, :Fp1, :Fp2]),
    layout = :grid,
    layout_kwargs = (; grid_dims = (3, 2), grid_skip_positions = [(2, 1)]),
)

EegFun.plot_erp(
    erps,
    channel_selection = EegFun.channels([:Cz, :PO7, :PO8, :Fp1, :Fp2, :F3]),
    layout = :grid,
    layout_kwargs = (; grid_dims = (2, 3)),
    axis_type = :origin,
    scale_x_value = 0.5,
    scale_y_value = 5.0,
    xlabel = "s",
    ylabel = "µV",
)



EegFun.plot_erp(
    erps,
    channel_selection =
    EegFun.channels([:Cz, :PO7, :PO8, :Fp1, :Fp2, :F3, :T8, :F4]),
    layout = :grid,
    layout_kwargs = (; grid_dims = (3, 4), grid_skip_positions = [(2, 1), (2,
    3)]),
)
```

```
EegFun.plot_erp(
    erps,
    channel_selection = EegFun.channels([:Cz, :PO7, :PO8, :Fp1, :Fp2, :F3]),
    layout = :grid,
    layout_kwargs = (; grid_dims = (2, 4), grid_skip_positions = [(2, 1), (2, 3)],
    grid_rowgap = 0, grid_colgap = 0),
    figure_padding = (150, 150, 150, 150),
)

# ERP Plots (:single)
EegFun.plot_erp(erps, average_channels = false, colormap = :viridis, legend_nbanks
= 12)
EegFun.plot_erp(erps[1], average_channels = false, colormap = :viridis,
legend_nbanks = 12)

EegFun.plot_erp(erps, average_channels = true)
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Cz, :PO7, :PO8]),
average_channels = true)
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Cz, :PO7, :PO8]),
average_channels = false)
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:Cz, :PO7, :PO8]),
average_channels = false, legend_nbanks = 3)
EegFun.plot_erp([erps[1], erps[1]], channel_selection = EegFun.channels([:PO8]))

# ERP Plots (:grid)
EegFun.plot_erp(erps, layout = :grid)
EegFun.plot_erp(erps, channel_selection =
EegFun.channels([:Cz, :PO7, :PO8, :Fp1, :Fp2, :F3]), layout = :grid)

# EPR Plots (:topo)
EegFun.plot_erp(erps, layout = :topo)
EegFun.plot_erp([erps[1], erps[1]], layout = :topo)
EegFun.plot_erp(erps, layout = :topo, channel_selection =
EegFun.channels([:Fp1, :Fp2, :PO8]))

# Combined plots
using GLMakie
fig = Figure(size = (800, 800))
ax1 = Axis(fig[1, 1])
ax2 = Axis(fig[1, 1], width = Relative(0.2), height = Relative(0.2), halign = 0,
valign = 0)
EegFun.plot_erp!(fig, ax1, erps, average_channels = true)
EegFun.plot_topography!(
    fig,
    ax2,
```

```
    erps[1];
    point_plot = false,
    label_plot = false,
    colorbar_plot = true,
    colorbar_kwargs = (;
        width = Relative(0.03),
        height = Relative(0.2),
        tellheight = false,
        tellwidth = false,
        position = (1, 1),
        halign = 0.25,
        valign = 0,
        flipaxis = true,
    ),
)
fig

GLMakie.closeall()
```

- API Reference

18.7 Plot ERP Image

Plot ERP Image

This demo shows how to create ERP image plots for visualizing single-trial EEG activity.

What is an ERP Image?

An ERP image displays single-trial data as a 2D heatmap:

- **Y-axis**: Individual trials/epochs
- **X-axis**: Time points
- **Color**: Amplitude at each time-trial coordinate

This reveals trial-to-trial variability that is hidden when viewing only averaged ERPs.

Why Use ERP Images?

See variability beyond the average:

Traditional ERPs show the average across trials, but ERP images show:

- Trial-to-trial amplitude fluctuations
- Temporal jitter in component latency
- Phase consistency vs. phase resetting

Quality control:

Quickly spot:

- Artifact-contaminated trials (bright/dark streaks)
- Drift across the experiment
- Individual outlier trials

Understand components:

Distinguish between:

- **Time-locked** activity: Consistent latency across trials (vertical bands)
- **Phase-locked** activity: Consistent phase but variable amplitude
- **Induced** activity: Not phase-locked to stimulus

Layout Options

The demo shows three layout modes:

Layout	Description
:single	All selected channels in one column
:grid	Channels arranged in a grid
:topo	Channels positioned by scalp location

Visualization Features

Boxcar averaging:

Smooth the image by averaging across neighboring trials:

```
EegFun.plot_erp_image(epochs, boxcar_average = 20)
```

Reduces noise while preserving overall patterns.

Colorbar control:

```
EegFun.plot_erp_image(epochs, colorbar_plot = false)  # Hide colorbar
EegFun.plot_erp_image(epochs, colorrange = (-50, 50))  # Custom range
```

Optional ERP overlay:

```
EegFun.plot_erp_image(epochs, plot_erp = false)  # Hide the averaged ERP
```

By default, the averaged ERP is plotted above the image for reference.

Interpretation

Vertical bands:

Strong vertical alignment indicates time-locked activity with consistent latency across trials.

Gradual color changes:

Smooth transitions suggest sustained activity or slow baseline drift.

Random speckles:

High-frequency noise or lack of consistent time-locked activity.

Outlier rows:

Trials with extreme values may indicate artifacts (blinks, movements, poor contact).

Workflow Summary

This demo demonstrates:
1. **Load and preprocess** data (rereferencing, filtering)
2. **Extract epochs** (-2 to 4 seconds)
3. **Plot ERP images** with different layouts
4. **Apply boxcar averaging** to smooth trial-to-trial noise
5. **Customize visualization** with colorbars and color ranges

ERP images are a powerful complement to traditional ERP plots, revealing the underlying trial structure that averages can obscure.

Code Examples

```
# Demo: ERP Image Plots
# Shows trial-by-trial ERP activity as image/heatmap plots.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

dat = EegFun.create_eegfun_data(dat, layout)

# minimal preprocessing
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)

# EPOCHS
epoch_cfg = EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))
```

```
EegFun.plot_erp_image(epochs, layout = :single)  # average of all channels
EegFun.plot_erp_image(epochs, layout = :single, channel_selection =
EegFun.channels([:PO7, :PO8]))
EegFun.plot_erp_image(epochs, layout = :single, channel_selection =
EegFun.channels([:PO7, :PO8]), interpolate = true)

EegFun.plot_erp_image(epochs, layout = :grid, colorbar_plot = false)
EegFun.plot_erp_image(epochs, layout = :grid, colorbar_plot = true)

EegFun.plot_erp_image(epochs, layout = :topo)
EegFun.plot_erp_image(epochs, channel_selection = EegFun.channels([:PO7]))
EegFun.plot_erp_image(epochs, channel_selection = EegFun.channels([:Fp1]),
plot_erp = false)



EegFun.plot_erp_image(epochs, layout = :single)

EegFun.plot_erp_image(
    epochs,
    channel_selection = EegFun.channels([:PO7, :PO8]),
    layout = :single,
    boxcar_average = 2,
    colorrange = (-50, 50),
)

EegFun.plot_erp_image(epochs, layout = :topo, boxcar_average = 20, colorrange =
(-50, 50))
```

- API Reference

18.8 Plot ERP Measurements

Plot ERP Measurements

This demo shows how to plot ERP waveforms with measurement overlays using `EegFun.plot_erp_measurements` .

What Does It Do?

`plot_erp_measurements` combines ERP waveform visualization with quantified measurements in a single plot. It highlights the analysis interval and displays the computed measurement directly on the waveform, making it easy to see exactly what is being measured and where.

Supported Measurements

- **Mean amplitude** — average voltage within the analysis interval
- **Peak amplitude** — maximum or minimum voltage within the interval

- **Peak latency** — time point of the peak within the interval

Key Options

Parameter	Purpose
`analysis_interval`	Time window for measurement (seconds)
`baseline_interval`	Baseline correction window
`channel_selection`	Which channels to plot
`condition_selection`	Which conditions to include
`layout`	`:overlay` (default) or `:grid`

Workflow Summary

This demo covers:

Mean Amplitude Overlay

- Plot ERP with shaded analysis window and computed mean

Peak Latency

- Show peak markers on specific channels

Grid Layout

- Display each channel in its own panel with measurement annotations

File Path Input

- Load data directly from JLD2 file path without pre-loading

Code Examples

```
# Demo: Plotting ERPs with Measurement Overlays
# Computes and visualizes ERP measurements in a single call.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

dat = EegFun.read_data(EegFun.example_path("data/julia/erps/
example1_erps_final.jld2"))
```

```
# Mean amplitude
# EegFun.plot_erp_measurements(dat, "mean_amplitude", analysis_interval = (0.3,
0.5), baseline_interval = (-0.2, 0.0)) # not really useful as too crowded!

# Peak latency with specific channels
EegFun.plot_erp_measurements(
    dat,
    "max_peak_latency",
    analysis_interval = (0.0, 1.0),
    baseline_interval = (-0.2, 0.0),
    channel_selection = EegFun.channels([:Cz, :Pz]),
)

# Grid layout
EegFun.plot_erp_measurements(dat, "max_peak_amplitude", analysis_interval = (0.0,
1.0), baseline_interval = (-0.2, 0.0), layout = :grid)

# Load from file path
EegFun.plot_erp_measurements(
    EegFun.example_path("data/julia/erps/example1_erps_final.jld2"),
    "max_peak_amplitude",
    analysis_interval = (0.0, 1.0),
    baseline_interval = (-0.2, 0.0),
    condition_selection = EegFun.conditions([1]),
    channel_selection = EegFun.channels([:Cz, :Pz]),
    average_channels = true,
)
```

- API Reference

18.9 Plot Filter

Plot Filter

This demo demonstrates visualizing filter frequency and phase responses to verify filter characteristics before applying them to data.

What is Filter Visualization?

Filter response plots show how a filter affects different frequencies:

- **Magnitude response**: Attenuation (in dB) at each frequency
- **Phase response**: Time delays introduced across frequencies
- **Cutoff characteristics**: Transition band steepness and rolloff

Why Visualize Filters?

Verify design parameters:

- Confirm cutoff frequencies are correct
- Check passband and stopband behavior
- Ensure appropriate attenuation levels

Identify potential issues:

- Excessive ripple in passband
- Slow transition bands
- Phase distortion effects

Documentation:

- Include in methods sections
- Show filter characteristics clearly
- Support reproducibility

Filter Types Supported

High-pass filters:

- Remove slow drifts and DC offset
- Typical: 0.1-1 Hz cutoff
- Preserve task-related activity

Low-pass filters:

- Remove high-frequency noise
- Typical: 30-40 Hz cutoff
- Anti-aliasing before downsampling

Band-pass filters:

- Isolate specific frequency ranges
- Combine high-pass and low-pass
- Focus on frequency bands of interest

Filter Methods

IIR (Infinite Impulse Response):

- Butterworth filters
- Efficient computation
- Steeper rolloff with fewer coefficients
- Can introduce phase distortion

FIR (Finite Impulse Response):

- Linear phase (no distortion)
- Requires more coefficients
- Computationally more expensive
- Symmetric impulse response

Visualization Features

Reference lines:

- Common attenuation levels (-3dB, −6dB, −12dB)
- Highlight filter characteristics
- Customizable positions and styling

Customization:

- Line colors and widths
- Title and labels
- Frequency resolution (n_points)
- Reference line positions

Workflow Summary

This demo shows filter response visualization:

Create Filters

- Lowpass IIR filter (30 Hz)
- Highpass IIR filter (1 Hz)
- Lowpass FIR filter (40 Hz)
- Various cutoff frequencies

Plot Responses

- Basic magnitude response plots
- Compare IIR vs FIR characteristics
- Visualize different cutoffs

Customize Appearance

- Custom colors and line widths
- Reference lines at specific dB levels
- Titles and styling options

Verify Characteristics

- Check cutoff frequency accuracy
- Assess transition band steepness
- Evaluate filter suitability

Code Examples

```
# Demo: Filter Visualization
# Shows filter frequency response and impulse response visualization.

```

```
using EegFun
using GLMakie

# Create lowpass IIR filter using create_filter
filter_info = EegFun.create_lowpass_filter(30.0, 256.0; filter_method = "iir")

# Plot filter response
EegFun.plot_filter_response(filter_info)

# Test with custom parameters
EegFun.plot_filter_response(
    filter_info,
    plot_title = "Custom Lowpass Filter Plot",
    color = :blue,
    linewidth = 3,
    reference_lines = [-3, -12, -24],
    reference_color = :red,
    n_points = 1000,
)

# Create highpass IIR filter using create_filter
filter_info = EegFun.create_highpass_filter(1.0, 256.0; filter_method = "iir")
EegFun.plot_filter_response(filter_info, plot_title = "High-pass Filter", color
= :green)

# Create FIR filter using create_filter
filter_info = EegFun.create_lowpass_filter(40.0, 256.0; filter_method = "fir")
EegFun.plot_filter_response(filter_info, plot_title = "FIR Lowpass Filter", color
= :purple)

# Test additional filter with separate plotting
filter_info = EegFun.create_highpass_filter(0.5, 256.0; filter_method = "iir")
EegFun.plot_filter_response(filter_info, plot_title = "High-pass Filter with
Plot")
```

- API Reference

18.10 Plot GFP

Plot GFP

This demo shows how to visualise Global Field Power (GFP) and Global Dissimilarity for ERP data.

What is GFP?

- **GFP** is the spatial standard deviation across all electrodes at each time point
- It summarises the overall strength of the scalp field, independent of polarity

- **Global Dissimilarity** measures how much the scalp topography changes between time points

Key Functions

Function	Purpose	Typical Use
`plot_gfp(erp)`	Plot GFP for one condition	Quick inspection
`plot_gfp([erp1, erp2])`	Compare GFP across conditions	Condition effects
`plot_gfp(gfp_dataframe)`	Plot pre-computed GFP	Custom workflows

Key Parameters

Parameter	Default	Description
`normalize`	`true`	Percentage (true) or raw µV (false)
`show_erp_traces`	`false`	Add overlaid ERP waveforms
`show_dissimilarity`	`false`	Add Global Dissimilarity panel
`channel_selection`	`channels()`	Subset of channels for GFP

What You'll Learn

1. Plotting GFP for single and multiple conditions
2. Showing ERP traces and dissimilarity in a multi-panel layout
3. Using pre-computed GFP results
4. Selecting specific channels for GFP calculation

Code Examples

```
# Demo: Plotting Global Field Power
# Shows how to visualise GFP (Global Field Power) and Global Dissimilarity
# across conditions, with optional ERP trace overlay.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

######################################################################
# LOAD DATA AND CREATE ERPS
```

```
#########################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

# preprocessing
EegFun.highpass_filter!(dat, 0.1)
EegFun.lowpass_filter!(dat, 30.0)

# extract epochs
epoch_cfg = EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))

EegFun.baseline!(epochs, (-0.2, 0.0))
erps = EegFun.average_epochs(epochs)

# plot GFP for a single condition
EegFun.plot_gfp(erps)
```

- API Reference

18.11 Plot ICA

Plot ICA

This demo shows how to visualise ICA component topographies as scalp maps.

Why Plot ICA Topographies?

- **Identify artefact components** — blink and eye movement components have characteristic frontal patterns
- **Verify decomposition** — well-separated components should have distinct, interpretable scalp maps
- **Select components** — focus on a subset of components for inspection or removal

Key Functions

Function	Purpose	Typical Use
`plot_topography(ica)`	Plot all components	Full overview
`plot_topography(ica, component_selection=...)`	Plot selected components	Focused inspection

Key Parameters

Parameter	Default	Description
`component_selection`	`components()`	Which components to display
`dims`	auto	Grid size as `(rows, cols)`
`use_global_scale`	`false`	Share colour scale across all maps
`colormap`	`:RdBu`	Colour map for the topography
`colorbar_plot`	`true`	Show colorbars

What You'll Learn

1. Displaying all ICA component topographies in a grid
2. Selecting specific components by index or range
3. Adjusting grid layout and colour scaling
4. Using keyboard shortcuts to scale the colour range

Code Examples

```
# Demo: ICA Visualisation
# Shows how to visualise ICA component topographies, browse continuous data
# with ICA overlays in the databrowser, and inspect component activations
# (time courses) with the interactive component activation viewer.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# LOAD DATA AND RUN ICA
############################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

# highpass at 1 Hz for ICA (critical for good decomposition)
EegFun.highpass_filter!(dat, 1.0)
```

```
# average reference
EegFun.rereference!(dat, :avg)

# create additional Bool colum for v. extreme values
EegFun.is_extreme_value!(dat, 200)

# run ICA (excludinng the v. extreme samples)
ica = EegFun.run_ica(dat, sample_selection =
EegFun.samples_not(:is_extreme_value_200))

#########################################################################
# BASIC TOPOGRAPHY GRID
#########################################################################

# plot all components in a grid
EegFun.plot_topography(ica, label_plot = false, point_plot = false)

#########################################################################
# SELECT SPECIFIC COMPONENTS
#########################################################################

# plot only the first 10 components
EegFun.plot_topography(ica, component_selection = EegFun.components(1:10))

#########################################################################
# CUSTOM GRID SIZE
#########################################################################

# control rows × columns layout
EegFun.plot_topography(ica, component_selection = EegFun.components(1:12), dims =
(4, 3))

#########################################################################
# DISPLAY OPTIONS
#########################################################################

# shared colour scale across all components
EegFun.plot_topography(ica, component_selection = EegFun.components(1:10),
use_global_scale = true)

# adjust colour map
EegFun.plot_topography(ica, component_selection = EegFun.components(1:6), colormap
= :RdBu)
```

```
#########################################################################
# DATABROWSER WITH ICA
#########################################################################

# browse continuous data with ICA components overlaid
# (toggle individual components on/off for interactive artifact removal)
EegFun.plot_databrowser(dat, ica)

#########################################################################
# ICA COMPONENT ACTIVATION (TIME COURSES)
#########################################################################

# interactive viewer: topography + time-series for every component
EegFun.plot_ica_component_activation(dat, ica)

# show only the first 10 components
EegFun.plot_ica_component_activation(dat, ica, component_selection =
EegFun.components(1:10))

# show specific components of interest
EegFun.plot_ica_component_activation(dat, ica, component_selection =
EegFun.components([1, 2, 3, 4]))

#########################################################################
# ICA COMPONENT ACTIVATION WITH ARTIFACT LABELS
#########################################################################

# identify artifact components automatically
component_artifacts, _ = EegFun.identify_components(dat, ica, sample_selection =
EegFun.samples_not(:is_extreme_value_200))

# activation viewer with artifact category checkboxes
# (components are labelled by type: EOG, ECG, line noise, channel noise)
EegFun.plot_ica_component_activation(dat, ica, artifacts = component_artifacts)
```

- API Reference

18.12 Plot Joint Probability

Plot Joint Probability

This demo demonstrates using joint probability analysis to detect bad channels based on multi-dimensional statistical outliers.

What is Joint Probability?

Joint probability analysis detects outlier channels by examining how each channel's distribution relates to all other channels simultaneously:

- **Multi-dimensional comparison**: Unlike single metrics (variance, kurtosis), joint probability considers the full distribution
- **Statistical outlier detection**: Channels that don't fit the overall pattern are flagged
- **More sensitive**: Catches unusual channels that may pass single-metric tests

How It Works

1. **Calculate channel statistics**: Extract features from each channel (amplitude, variance, etc.)
2. **Build joint distribution**: Model the typical pattern across all channels
3. **Compute probabilities**: For each channel, calculate how likely its pattern is
4. **Identify outliers**: Channels with very low probability are suspicious

Interpreting Results

Low probability values (close to 0):

- Channel doesn't fit typical pattern
- Potential bad channel
- May have unusual artifacts or noise

Consistent probability values (similar across channels):

- Data quality is uniform
- No obvious outliers

Spatial clusters of low probability:

- Systematic recording issues
- Poor contact in a region
- Regional artifacts

Use Cases

Pre-processing quality control:

- Automated bad channel detection
- Objective identification criteria
- Reduces manual inspection time

Complement other metrics:

- Use with variance and kurtosis
- Catches different types of artifacts
- More comprehensive quality assessment

Research validation:

- Document channel exclusion decisions
- Consistent criteria across datasets
- Reproducible preprocessing

Threshold Selection

The default threshold identifies statistical outliers, but you may need to adjust based on:

- **Data quality**: Noisy data may need more lenient thresholds
- **Channel count**: More channels → more statistical power
- **Analysis goals**: Conservative vs aggressive cleaning

Advantages Over Single Metrics

Variance alone: Misses channels with unusual distributions but normal variance

Kurtosis alone: Misses channels with shifted means or different scales

Joint probability: Combines multiple dimensions to catch subtle issues

Workflow Summary

This demo shows joint probability analysis:

Load and Preprocess

- Read continuous data
- Apply basic preprocessing (reference, filter)

Compute Joint Probability

- Calculate multi-dimensional statistics
- Build probability distribution
- Returns metrics for each channel

Visualize Results

- Plot probability values per channel
- Identify low-probability outliers
- Assess spatial patterns

Use Results for QC

- Flag suspicious channels
- Combine with other metrics
- Make informed exclusion decisions

Code Examples

```
# Demo: Joint Probability Plot
# Shows joint probability distributions for artifact detection.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
```

```
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)

# test plot_channel_summary
jp = EegFun.channel_joint_probability(dat)
EegFun.plot_joint_probability(jp)
```

- API Reference

18.13 Plot Layout

Plot Layout

This demo demonstrates visualizing electrode layout configurations in 2D and 3D space.

What is Electrode Layout?

Electrode layout defines the spatial positions of EEG sensors on the scalp using:

- **2D coordinates** (x, y): Projected positions for topographic maps
- **3D coordinates** (x, y, z): Actual positions on spherical head model
- **Spherical coordinates** (incidence, azimuth): Angular positions

Layout Visualization Functions

plot*layout*2d:

- Standard topographic view from above
- Shows electrode positions and labels
- Customizable head outline, markers, and labels
- Supports region-of-interest highlighting

plot*layout*3d:

- Full 3D spherical visualization
- Interactive rotation and zoom
- Shows true spatial arrangement

Neighbor visualization:
- Display spatial relationships between electrodes
- Helps verify interpolation neighborhoods
- Interactive hover to show connections

Use Cases

Verify electrode montage:
- Confirm correct channel positions
- Check for mislabeled electrodes
- Validate custom layouts

Publication figures:
- Document electrode positions
- Highlight regions of interest
- Create publication-ready diagrams

Analysis planning:
- Understand spatial sampling density
- Plan interpolation strategies
- Define ROIs for analysis

Customization Options

Head outline:
- Color, line width, radius

Electrode markers:
- Marker style, size, color
- Show/hide markers

Labels:
- Font size, color
- Position offsets
- Show/hide labels

Regions of interest (ROIs):
- Highlight electrode groups
- Customizable borders and fills
- Multiple ROIs with different styles

Neighbor Detection

Calculate spatial neighbors based on distance thresholds:

- **2D neighbors**: Using projected coordinates
- **3D neighbors**: Using spherical coordinates
- Export neighbor definitions for channel repair

Workflow Summary

This demo shows electrode layout visualization:

Basic 2D Visualization

- Load electrode layout
- Convert to 2D coordinates
- Plot with default settings

Customize Appearance

- Adjust head outline (color, width, radius)
- Modify markers (style, size, color)
- Customize labels (size, color, offsets)

Add Regions of Interest

- Highlight electrode groups
- Customize ROI borders and fills
- Create multiple ROIs with different styles

Neighbor Visualization

- Calculate neighbor relationships
- Plot with neighbor connections
- Export neighbor definitions

Save Figures

- Export publication-ready figures
- Use CairoMakie for vector graphics

Code Examples

```
# Demo: Layout Visualization
# Shows electrode layout plotting and manipulation.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")
```

```
using EegFun
using GLMakie

const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)

layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout, preserve_radial_distance = true);

# basic plot
EegFun.plot_layout_2d(layout)

# some customisations
EegFun.plot_layout_2d(layout; head_color = :grey, head_linewidth = 5, head_radius
= 1.5)
EegFun.plot_layout_2d(layout; point_plot = false, label_plot = false)
EegFun.plot_layout_2d(layout; point_marker = :x, point_markersize = 20)
EegFun.plot_layout_2d(layout; point_color = :red)

EegFun.plot_layout_2d(layout; label_fontsize = 20, label_color = :grey)

# Annotating plots with ROIs
result = EegFun.plot_layout_2d(layout)
EegFun.add_topo_rois!(result.axes[1], layout, [[:PO7, :PO3, :P1],
[:PO8, :PO4, :P2]], roi_border_size = 0.05)
EegFun.add_topo_rois!(result.axes[1], layout, [[:PO7, :PO3, :P1],
[:PO8, :PO4, :P2]], roi_border_size = 0.1)
EegFun.add_topo_rois!(
    result.axes[1],
    layout,
    [[:Fp1]],
    roi_border_size = 0.10,
    roi_fill = [true],
    roi_linecolor = [:black],
    roi_fillcolor = [:red],
    roi_fillalpha = [0.2],
)
EegFun.add_topo_rois!(
    result.axes[1],
    layout,
    [[:CPz, :C2, :FCz, :C1]],
    roi_border_size = 0.15,
    roi_linewidth = [5],
    roi_fill = [true],
    roi_linecolor = [:blue],
```

```
    roi_fillcolor = [:green],
    roi_fillalpha = [0.5],
)

# Plots with neighbors (hover mouse over channel to show neighbor connections)
EegFun.get_neighbors_xy!(layout, 0.5);
EegFun.plot_layout_2d(layout, neighbors = true)

# Print neighbors while 2D neighbors are still computed
# (the 3D coordinate conversion below clears cached neighbors)
EegFun.print_layout_neighbors(layout, joinpath(DEMO_OUTPUT,
"electrode_neighbors_1.toml"))
EegFun.print_layout_neighbors(layout.neighbors, joinpath(DEMO_OUTPUT,
"electrode_neighbors_2.toml"))

# 3D layout with neighbors
EegFun.get_neighbors_xyz!(layout, 0.5);
EegFun.plot_layout_3d(layout, neighbors = true)

# save a basic figure
# NB. for vector graphics, use CairoMakie
# save("topo_roi.png", result.fig)
```

- API Reference

18.14 Plot Topography

Plot Topography

This demo demonstrates creating topographic scalp maps to visualize the spatial distribution of EEG activity at specific time points.

What are Topographic Maps?

Topographic maps (topoplots) show the spatial distribution of electrical activity across the scalp:

- **Color-coded amplitudes**: Activity levels represented by color
- **Interpolated surfaces**: Smooth maps between discrete electrode positions
- **Time-specific snapshots**: Activity at particular latencies or time intervals

Interpolation Methods

`:thin_plate` : Thin-plate spline (default; smooth, natural-looking)

`:nearest` : Nearest neighbor (no interpolation, fastest)

`:shepard` : Inverse distance weighting

`:multiquadratic` : Radial basis function

`:spherical_spline` : Spherical spline (accounts for head curvature)

> **Note**: All interpolation methods except `:spherical_spline` are implemented using the ScatteredInterpolation.jl package. The spherical spline method uses a custom implementation based on Perrin et al. (1989).

Interpretation

Focal activity:

- Localized color patches suggest specific neural sources
- Sharp gradients indicate nearby sources

Widespread activity:

- Diffuse patterns suggest distributed processing
- Gradual transitions indicate distant or multiple sources

Polarity conventions:

- Warm colors (red/yellow): Positive voltage
- Cool colors (blue): Negative voltage
- Convention may vary by field

Use Cases

Visualize ERP components:

- Show spatial distribution of P1, N170, P300, etc.
- Identify component topographies

Compare conditions:

- Side-by-side condition comparisons
- Difference topographies (condition A - B)

Publication figures:

- High-quality scalp maps
- Customizable appearance
- Multiple time points or conditions

Customization Options

Interpolation:

- Method selection
- Grid resolution (gridscale)

Appearance:

- Colormap selection
- Color limits (`colorrange`)
- Head outline radius

Labels and markers:

- Electrode positions
- Channel labels
- Font sizes and colors

Colorbar:

- Position and orientation
- Size and tick labels
- Show/hide per plot

Working with Different Data Types

Continuous data: Average over time interval

Epoched data: Specify epoch number and time interval

ERP data: Average directly (already averaged)

Workflow Summary

This demo shows topographic visualization workflows:

Basic Continuous Data

- Load and preprocess data
- Create topographic maps with different methods
- Customize time intervals and appearance

Epoched Data

- Extract epochs from continuous data
- Plot topographies for specific epochs
- Customize interpolation and display

ERP Data

- Average epochs into ERPs
- Create condition-specific topographies
- Control colorbar placement

Multi-Panel Figures

- Create custom figure layouts
- Combine multiple topographies
- Control colorbar positions for each plot

Code Examples

```
# Demo: Topographic Maps
```

```
# Shows scalp topography visualization for ERP data and components.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 1)

# visually selected blink like artifact via interval selection times
with_theme(Theme(colormap = :coolwarm)) do
    EegFun.plot_topography(dat, interval_selection = EegFun.times(5.973, 6.02),
    colorrange = (-200, 200))
end

EegFun.plot_topography(dat, interval_selection = EegFun.times(6), colorrange =
(-200, 200))

# blink like artifact via sample selection predicate
EegFun.plot_topography(dat, sample_selection = x -> x.time .>= 5.973 .&&
x.time .<= 6.02, colorrange = (-200, 200))

# blink like artifact across multiple time points
EegFun.plot_topography(dat, sample_selection = x -> x.time .>= 5.5 .&& x.time .<=
6.5, colorrange = (-75, 75), n_topo = 5, dims = (1, 5))
EegFun.plot_topography(
    dat,
    sample_selection = x -> x.time .>= 5.5 .&& x.time .<= 6.5,
    colorrange = (-75, 75),
    n_topo = 5,
    dims = (1, 5),
```

```
    colorbar_plot_numbers = 5,
)

# Various combinations
EegFun.plot_topography(dat, colorbar_plot = false, head_radius = 1.25)
EegFun.plot_topography(dat, gridscale = 250)
EegFun.plot_topography(dat, colormap = :inferno)
EegFun.plot_topography(dat, colormap = :redblue)
EegFun.plot_topography(dat, plot_title = "Custom Title", plot_title_fontsize = 30)
EegFun.plot_topography(dat, sample_selection = x -> x.time .>= 0.4 .&& x.time .<=
0.6)
EegFun.plot_topography(dat, sample_selection = x -> x.time .>= 0.4 .&& x.time .<=
0.6, method = :spherical_spline)
EegFun.plot_topography(dat, channel_selection =
EegFun.channels([:Fp1, :Fp2, :Cz]))
EegFun.plot_topography(dat, colorbar_kwargs = (; size = 20, position = (2, 1),
vertical = false))

################################
# Epoched DataFrameEeg
################################
# some epoched data
epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))

# Separate plots
EegFun.plot_topography(epochs[1], 1) # Condition 1, Epoch 1
EegFun.plot_topography(epochs[2], 1) # Condition 2, Epoch 1
EegFun.plot_topography(epochs, colorrange = (-0.1, 0.1)) # NOTE: Consider adding
aspect ratio and global scale controls in the future
EegFun.plot_topography(epochs, sample_selection = x -> x.time .>= 0.4 .&&
x.time .<= 0.6)




################################
# ERP like data
################################
erps = EegFun.average_epochs(epochs)

EegFun.plot_topography(erps, sample_selection = x -> x.time .>= 0.4 .&& x.time .<=
0.6, colorrange = (-2, 2), time_unit = :ms)
```

```
EegFun.plot_topography(erps, interval_selection = (0.4, 0.6), colorrange = (-2,
2))

EegFun.plot_topography(erps, interval_selection = (0.4, 0.6), colorrange = (-2,
2), colorbar_plot_numbers = [1, 2])

EegFun.plot_topography(
    erps,
    interval_selection = (0.4, 0.6),
    colorrange = (-2, 2),
    colorbar_plot = true,
    colorbar_kwargs = (; position = (2, 1), vertical = false),
)

EegFun.plot_topography(erps)
EegFun.plot_topography(erps, colorrange = (-2, 2))
EegFun.plot_topography(erps[2])
EegFun.plot_topography(erps[1], gridscale = 50)
EegFun.plot_topography(erps[2], gridscale = 1000)
EegFun.plot_topography(erps[1], colormap = :inferno)
EegFun.plot_topography(erps[2], plot_title = "Custom Title", plot_title_fontsize =
30)
```

- API Reference

18.15 Plot Triggers

Plot Triggers

This demo demonstrates visualizing event markers and triggers in continuous EEG data to verify timing and event sequences.

What are Triggers?

Triggers (also called event markers or stimulus codes) are time-stamped codes that mark when experimental events occurred during recording:

- Stimulus presentations
- Participant responses
- Experimental conditions
- Trial boundaries
- Hardware events
- etc.

Trigger Cleaning

EegFun automatically cleans triggers by default removing consecutive duplicates. For example, the raw sequence `0 0 1 1 0 0 2 2 2 0` becomes `0 0 1 0 0 0 2 0 0 0` . This ensures each trigger represents a single event rather than a sustained hardware signal.

Trigger Visualization Functions

trigger_count:

- Summary statistics of all trigger codes
- Counts of each trigger type
- Identifies missing or unexpected triggers

plot*trigger*overview:

- Visual representation of trigger occurrences
- Color-coded by trigger type
- Shows distribution across recording

plot*trigger*timing:

- Inter-trigger intervals (ITIs)
- Timing precision verification

Use Cases

Quality control:

- Verify triggers were recorded correctly
- Confirm expected trigger counts
- Identify missing or duplicate triggers

Timing analysis:

- Check inter-stimulus intervals
- Verify experimental timing

Troubleshooting:

- Identify spurious triggers
- Find timing drift or jitter

Filtering Triggers

Use `ignore_triggers` to exclude specific codes:

- Filter out hardware markers
- Remove boundary codes
- Focus on experimental events only

Workflow Summary

This demo shows trigger visualization workflows:

Count Triggers

- Load raw data
- Count triggers before processing
- Verify expected trigger codes exist

Create Data Structure

- Load layout and create EegFun structure
- Count triggers again to verify preservation

Visualize Overview

- Plot trigger distribution
- Optionally ignore certain trigger codes
- Assess trigger patterns

Analyze Timing

- Plot inter-trigger intervals
- Verify timing consistency
- Identify timing issues

Code Examples

```
# Demo: Trigger Visualization
# Shows trigger/event marker visualization in continuous data.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# basic trigger count from raw file
count = EegFun.trigger_count(dat)

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# basic trigger count from EegFun data structure
```

```
count = EegFun.trigger_count(dat)

# trigger overview
EegFun.plot_trigger_overview(dat)

# trigger overview with ignored triggers
EegFun.plot_trigger_overview(dat; ignore_triggers = [3, 253])

# trigger timing i.e, when did each trigger occur and interval between triggers
EegFun.plot_trigger_timing(dat)

# trigger timing with ignored triggers (timing interval is updated)
EegFun.plot_trigger_timing(dat; ignore_triggers = [3])
```

- API Reference

18.16 Saving Figures

Saving Figures

This demo shows how to save Makie figures in different file formats with resolution and quality settings.

Backends

Backend	Purpose
GLMakie	Interactive work (windows, zooming, panning)
CairoMakie	High-quality export (raster and vector)

You only need `using CairoMakie` — you do **not** need to activate it. Pass `backend = CairoMakie` in the `save()` call instead.

File Formats

Format	Type	Key Option
PNG	Raster	`px_per_unit` — scale resolution (e.g. 4 for print quality)
SVG	Vector	`pt_per_unit` — scale canvas size in points
PDF	Vector	`pt_per_unit` — scale canvas size in points

PNG (Raster)

```
save("figure.png", fig)                    # default resolution
save("figure.png", fig; px_per_unit = 4)   # high resolution
save("figure.png", fig; update = false)    # save current interactive view as-is
```

SVG & PDF (Vector)

Always pass `backend = CairoMakie` when saving vector formats. Without it, `save()` uses the active backend (GLMakie), which embeds a rasterized bitmap — not true vector graphics.

```
save("figure.svg", fig; backend = CairoMakie)
save("figure.pdf", fig; backend = CairoMakie)
save("figure.pdf", fig; backend = CairoMakie, pt_per_unit = 1.5)  # larger
```

Quick Reference

```
using GLMakie      # interactive work
using CairoMakie   # loaded but not activated

# ... create your plot ...
save("raster.png", fig; px_per_unit = 4)                          # high-res PNG
save("vector.svg", fig; backend = CairoMakie)                     # true vector
SVG
save("vector.pdf", fig; backend = CairoMakie, pt_per_unit = 1.5)  # scaled PDF
```

Code Examples

```
# Demo: Saving Figures
# Shows how to save Makie figures in different file formats (PNG, SVG, PDF)
# with resolution and quality settings.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie
using CairoMakie  # loaded for backend = CairoMakie in save(), not activated

const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)

# ── Prepare some data ──────────────────────────────────────────────────
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)
```

```
dat = EegFun.create_eegfun_data(dat, layout);
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.1)

epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-2, 4))
erps = EegFun.average_epochs(epochs)

# Create a figure (GLMakie is the active backend → interactive window)
GLMakie.activate!()
result = EegFun.plot_erp(erps, channel_selection =
EegFun.channels([:Cz, :PO7, :PO8]), average_channels = true)
fig = result.fig



# ╔═══════════════════════════════════════════════════════════════════════╗
# ║  PNG (raster)                                                         ║
# ╚═══════════════════════════════════════════════════════════════════════╝

# Default PNG — figure size (in units) determines pixel dimensions
save(joinpath(DEMO_OUTPUT, "erp_default.png"), fig)

# Higher resolution — px_per_unit scales every unit by this factor.
# px_per_unit = 4 gives ~4× the pixel count → great for posters / print.
save(joinpath(DEMO_OUTPUT, "erp_hires.png"), fig; px_per_unit = 4)

# Save the current interactive view exactly as-is (skip layout recalculation).
# Useful after you have interactively zoomed / panned in the GLMakie window.
save(joinpath(DEMO_OUTPUT, "erp_current_view.png"), fig; update = false)



# ╔═══════════════════════════════════════════════════════════════════════╗
# ║  SVG & PDF (vector)                                                   ║
# ╚═══════════════════════════════════════════════════════════════════════╝
# IMPORTANT: always pass `backend = CairoMakie` when saving vector formats.
# Without it, save() uses the currently active backend (GLMakie), which
# embeds a rasterized bitmap inside the SVG/PDF — not true vector graphics!

# --- SVG ---
# Ideal for web or documents where you need infinitely scalable graphics.
save(joinpath(DEMO_OUTPUT, "erp.svg"), fig; backend = CairoMakie)
```

```
# pt_per_unit controls the overall size of the SVG/PDF canvas in points.
# Larger values → bigger document; smaller → more compact.
save(joinpath(DEMO_OUTPUT, "erp_large.svg"), fig; backend = CairoMakie,
pt_per_unit = 1.5)

# --- PDF ---
# The standard for journal submissions and LaTeX documents.
save(joinpath(DEMO_OUTPUT, "erp.pdf"), fig; backend = CairoMakie)

# Scaled-up PDF — useful when the default appears too small in a document.
save(joinpath(DEMO_OUTPUT, "erp_large.pdf"), fig; backend = CairoMakie,
pt_per_unit = 1.5)

GLMakie.closeall()
```

- API Reference

Chapter 19

Interactive / GUI

19.1 Plot Databrowser

Plot Databrowser

This demo demonstrates the interactive databrowser for exploring and analyzing EEG data in real-time.

What is the Databrowser?

The databrowser is an interactive visualization tool for navigating through EEG data:

- **Continuous data**: Scroll through raw recordings dynamically
- **Epoch data**: Navigate individual trials interactively
- **Analysis settings**: Select regions, channels, and configurations visually

Key Features

Interactive Navigation:

- **Scroll through time**: Mouse wheel or drag to pan
- **Zoom in/out**: Adjust time scale for detailed inspection
- **Channel selection**: Click channels to select for analysis
- **Region selection**: Drag to select time regions
- **Epoch browsing**: Navigate through individual trials

Visual Overlays:

- **Trigger markers**: See event timing
- **Epoch intervals**: Visualize extraction boundaries
- **Extreme values**: Highlight artifact-contaminated samples
- **Channel repair**: Mark and repair problematic channels

ICA Integration:

```
EegFun.plot_databrowser(dat, ica_result)
```

Analysis Settings

The databrowser can also be used to return analysis settings for further processing:

```
res = EegFun.plot_databrowser(dat)
dat_new = EegFun.apply_analysis_settings(dat, res.analysis_settings)
```

Extract user-selected:

- Time regions
- Channel subsets
- Sample masks

Keyboard Shortcuts

Key	Action
I	Show help/info
R	Open channel repair menu
C	Clear current selections
Arrow keys	Navigate time intervals

Custom Styling

Customize visual appearance:

```
EegFun.plot_databrowser(dat,
    channel_line_width = 2,
    selection_color = (:red, 0.5)
)
```

Working with Files

Load data directly from saved files:

```
EegFun.plot_databrowser("dat.jld2")                    # saved continuous data
EegFun.plot_databrowser("epochs.jld2", "ica.jld2")     # saved epochs and ica
result
```

Common Workflows

1. Initial Quality Control

:

- Load raw data
- Scroll through recording
- Identify noisy channels
- Mark extreme values
- Plan preprocessing strategy

2. Artifact Identification

:

- Add `is_extreme_value!()` column
- Visualize artifact-contaminated regions

- Select clean segments for ICA

3. Epoch Validation

:

- Mark epoch intervals with `mark_epoch_intervals!()`
- Verify trigger timing
- Check epoch boundaries

Use Cases

Interactive channel repair:

- Select bad channels visually
- Choose repair method (spherical spline, neighbors)
- Apply and verify results immediately

Spectral analysis region selection:

- Identify interesting time intervals
- Select for further time-frequency analysis
- Export settings for batch processing

Preprocessing validation:

- Verify filtering effects
- Check rereferencing results
- Validate artifact rejection

Workflow Summary

This demo shows:
1. **Basic databrowser**: Simple visualization
2. **Analysis settings**: Capture and apply user selections
3. **Additional overlays**: Extreme values, epoch markers
4. **Custom styling**: Visual customization
5. **ICA integration**: Component inspection
6. **Epoch browsing**: Individual trial navigation
7. **File loading**: Direct file inspection

The databrowser is essential for interactive EEG exploration and quality control!

Code Examples

```
# Demo: Interactive Databrowser
# Shows the interactive databrowser for continuous and epoched data with ICA,
# analysis settings, custom channels, and file-based loading.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")
```

```
using EegFun
using GLMakie
using JLD2

const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)

# Quick demo with bundled raw dataset and full 72-channel layout:
# EegFun.plot_databrowser()

# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));
dat = EegFun.create_eegfun_data(dat); # works but no layout info available, thus,
no topographic plots

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.5)
EegFun.plot_databrowser(dat);

##############################################################
# read raw data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));

# read and prepare layout file
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"));
EegFun.polar_to_cartesian_xy!(layout)

# create EegFun data structure (EegFun.ContinuousData)
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
# EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.5)
EegFun.plot_databrowser(dat);

# Basic databrowser
EegFun.plot_databrowser(dat);

# return some analysis settings
# check highpass filter, lowpass filter, rereference and mark some regions
# these are returned in analysis_settings
_, _, analysis_settings = EegFun.plot_databrowser(dat)
```

```
# we can now apply these settings to the data
dat_new = EegFun.apply_analysis_settings(dat, analysis_settings)

# settings applied
EegFun.plot_databrowser(dat_new)

#########################################################################
# We can add new columns to our data
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"));
dat = EegFun.create_eegfun_data(dat, layout);

# Some minimal preprocessing (average reference and highpass filter)
EegFun.rereference!(dat, :avg)
EegFun.highpass_filter!(dat, 0.5)

# E.g. mark data sections (some extreme values and intervals around triggers;
Boolean)
EegFun.is_extreme_value!(dat, 100);
EegFun.mark_epoch_intervals!(dat, [1, 2], [-0.2, 1.0]) # simple epoch marking with
trigger 1 and 2

# Add new channel with a 10 Hz sine wave (50 µV amplitude)
dat.data[:, :sinewave] = 50.0 .* sin.(2π .* 10.0 .* dat.data[:, :time])

# these channels can now be viewed in the databrowser under the Extra channels tab
EegFun.plot_databrowser(dat)

EegFun.all_labels(dat)
EegFun.meta_labels(dat)
EegFun.channel_labels(dat)
EegFun.extra_labels(dat)

# try some custom styling
EegFun.plot_databrowser(dat; channel_line_width = 2, selection_color = (:green,
0.1))



#########################################################################
# run ica
ica_result = EegFun.run_ica(dat; sample_selection =
EegFun.samples_not(:is_extreme_value_100), percentage_of_data = 25)
EegFun.plot_databrowser(dat, ica_result)



#########################################################################
```

```
# EPOCHS
epoch_cfg = [EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.5, 1.0))

# databrowser for epochs
EegFun.plot_databrowser(epochs[1])  # can now browse epochs
EegFun.plot_databrowser(epochs[1], ica_result)



# We can save our EegFun data to a file and pass a file name to plot_databrowser
# plot_databrowser epochs from file
jldsave(joinpath(DEMO_OUTPUT, "dat.jld2"), data = dat)
jldsave(joinpath(DEMO_OUTPUT, "epochs.jld2"), data = epochs)
jldsave(joinpath(DEMO_OUTPUT, "ica.jld2"), data = ica_result)

# Plot by loading from file
EegFun.plot_databrowser(joinpath(DEMO_OUTPUT, "dat.jld2"))
EegFun.plot_databrowser(joinpath(DEMO_OUTPUT, "epochs.jld2"))
EegFun.plot_databrowser(joinpath(DEMO_OUTPUT, "dat.jld2"), joinpath(DEMO_OUTPUT,
"ica.jld2"))
EegFun.plot_databrowser(joinpath(DEMO_OUTPUT, "epochs.jld2"),
joinpath(DEMO_OUTPUT, "ica.jld2"))
```

- API Reference

19.2 Plot ERP Filter GUI

Plot ERP Filter GUI

This demo shows how to use the interactive filter GUI to explore the effect of different filter settings on ERP waveforms.

Why an Interactive Filter Tool?

- **Teaching** — show students how filters affect ERP signals in real time
- **Parameter selection** — experiment with cutoff frequency and filter order before committing
- **Method comparison** — toggle between IIR (Butterworth) and FIR filters, `filtfilt` vs `filt`

Key Features

Feature	Description
Lowpass / Highpass toggles	Enable each filter independently
Cutoff sliders	Adjust frequency in real time
Order sliders	Change filter steepness

Method menu	Switch between Butterworth and FIR
Mirror toggle	Enable mirror padding to reduce edge artifacts
Multi-condition	Compare filter effects across conditions side-by-side

What You'll Learn

1. Launching the GUI for a single ERP
2. Focusing on a specific channel
3. Comparing filter effects across multiple conditions

Code Examples

```
# Demo: ERP Filter GUI
# Shows how to launch the interactive filter exploration tool
# to compare the effect of different filter settings on ERP waveforms.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# Launch with default demo data (bundled example ERPs, channel :Cz):
gui = EegFun.plot_erp_filter_gui()

# Alternatively, specify a channel or custom data / file:
# gui = EegFun.plot_erp_filter_gui(channel = :Pz)
# dat = EegFun.read_data(EegFun.example_path("data/julia/erps/
example1_erps.jld2"))
# gui = EegFun.plot_erp_filter_gui(dat, channel = :Cz)

# Keep window open when run from terminal
if !isinteractive() && !isnothing(gui.screen)
    wait(gui.screen)
end
```

- API Reference

19.3 Plot ERP Measurement GUI

Plot ERP Measurement GUI

This demo shows how to use the interactive ERP measurement GUI for visual exploration and parameter tuning.

What Does It Do?

`plot_erp_measurement_gui` opens an interactive window that lets you explore ERP measurements in real time. You can adjust the analysis interval, baseline window, measurement type, and channel — all with live visual feedback. This is useful for:

- **Teaching** — demonstrating how measurement choices affect results
- **Parameter exploration** — finding the right analysis window before batch processing
- **Visual validation** — confirming that automated measurements are sensible

Relationship to `erp_measurements`

Use the GUI to explore and decide on your measurement parameters, then apply those parameters programmatically with `EegFun.erp_measurements()` for your final analysis.

Workflow Summary

This demo covers:

Basic Usage

- Launch the GUI with all conditions overlaid

Single Condition

- Explore one condition at a time

Pre-configured Settings

- Open the GUI with specific initial channel, measurement type, and intervals

Code Examples

```
# Demo: Interactive ERP Measurement GUI
# Explore ERP measurements interactively with live visual feedback.
# Useful for teaching, parameter exploration, and visual validation
# before batch processing with erp_measurements().

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

# Launch with default demo data (dynamically 30 Hz low-pass filtered, :Cz, 300-500
ms):
```

```
gui = EegFun.plot_erp_measurement_gui()

# Alternatively, pass custom data or specific initial settings:
# dat = EegFun.read_data(EegFun.example_path("data/julia/erps/
example1_erps.jld2"))
# gui = EegFun.plot_erp_measurement_gui(
#     dat,
#     channel = :Cz,
#     analysis_type = "mean_amplitude",
#     analysis_interval = (0.3, 0.5),
#     baseline_interval = (-0.2, 0.0),
# )

# Keep window open when run from terminal
if !isinteractive() && !isnothing(gui.screen)
    wait(gui.screen)
end
```

- API Reference

Chapter 20

Specialized Plotting

20.1 Plot Decoding

Plot Decoding

This demo shows how to visualise MVPA (multivariate pattern analysis) decoding results.

Why Plot Decoding Results?

- **Temporal dynamics** — see when brain signals become discriminative
- **Error shading** — visualise variability across cross-validation folds or subjects
- **Significance markers** — overlay statistical test results on the accuracy curve

Key Functions

Function	Purpose	Typical Use
`plot_decoding(decoded)`	Plot single-subject accuracy	Quick inspection
`plot_decoding(decoded_list)`	Multi-subject subplot grid	Individual differences
`plot_decoding(decoded, stats)`	Accuracy + significance	Publication figure

What You'll Learn

1. Plotting decoding accuracy over time with error shading
2. Customising colours, line width and titles
3. Comparing subjects in a subplot grid
4. Overlaying significance markers from `test_against_chance`

Code Examples

```
# Demo: Plotting Decoding Results
# Shows how to visualise MVPA decoding accuracy over time,
# compare individual subjects, and overlay statistical significance.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
```

```
using GLMakie

# LOAD DATA AND PREPARE FOR DECODING
dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

# basic preprocessing
EegFun.highpass_filter!(dat, 0.1)
EegFun.lowpass_filter!(dat, 30.0)

# extract epochs
epoch_cfg = [
    EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "ExampleEpoch2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-1, 2))
EegFun.baseline!(epochs, (-0.2, 0.0))

# decode condition from EEG patterns
decoded = EegFun.decode_libsvm(epochs)

# plot accuracy over time with error shading
EegFun.plot_decoding(decoded)
```

- API Reference

20.2 Plot RSA

Plot RSA

This demo shows how to visualise Representational Similarity Analysis (RSA) results.

What is RSA?

- **RDM** (Representational Dissimilarity Matrix) captures pairwise differences between neural representations
- **RDM Timecourse** shows how representational structure evolves over time
- **Model Correlations** compare observed RDMs against theoretical models

Key Functions

Function	Purpose
`plot_rdm_heatmap(rsa_result)`	Display an RDM as a heatmap

`plot_rdm_timecourse(rsa_result)`	Dissimilarity over time
`plot_model_correlations(rsa_result)`	Model comparison over time

Key Parameters

Parameter	Function	Description
`time_point`	`plot_rdm_heatmap`	Specific time (seconds or index); `nothing` = average
`condition_pairs`	`plot_rdm_timecourse`	Which pairs to plot; `:all` = all
`colorbar_plot`	All	Show/hide color bar
`colormap`	All	Colormap for the heatmap

What You'll Learn

1. Plotting RDM heatmaps at specific time points or averaged
2. Visualising dissimilarity timecourses for selected condition pairs
3. Comparing neural RDMs against theoretical model predictions

Code Examples

```
# Demo: Plotting RSA Results
# Shows how to visualise Representational Similarity Analysis results:
# RDM heatmaps, dissimilarity timecourses, and model correlations.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# LOAD DATA AND COMPUTE RSA
############################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

EegFun.highpass_filter!(dat, 0.1)
```

```
EegFun.lowpass_filter!(dat, 30.0)

epoch_cfg = [
    EegFun.EpochCondition(name = "Condition1", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Condition2", trigger_sequences = [[2]]),
]
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 0.8))
EegFun.baseline!(epochs, (-0.2, 0.0))

# compute RSA
rsa_result = EegFun.rsa(epochs)

##########################################################################
# RDM HEATMAP — AVERAGE ACROSS TIME
##########################################################################

# visualise overall representational structure
EegFun.plot_rdm_heatmap(rsa_result)

##########################################################################
# RDM HEATMAP — SPECIFIC TIME POINT
##########################################################################

# RDM at 300 ms post-stimulus
EegFun.plot_rdm_heatmap(rsa_result, time_point = 0.3)

# RDM at time index 50
EegFun.plot_rdm_heatmap(rsa_result, time_point = 50)

##########################################################################
# DISSIMILARITY TIMECOURSE
##########################################################################

# plot dissimilarity over time for all condition pairs
EegFun.plot_rdm_timecourse(rsa_result)

# only specific pairs
EegFun.plot_rdm_timecourse(rsa_result, condition_pairs = [(1, 2)])

##########################################################################
# MODEL CORRELATIONS
##########################################################################

# compare RSA results against theoretical models
# (requires having run compare_models beforehand)
```

```
# EegFun.plot_model_correlations(rsa_result)
```

- API Reference

20.3 Plot Statistics

Plot Statistics

This demo shows how to visualise ERP statistical test results, including condition comparisons, significance markers, and t-values.

Why Plot Statistics?

- **Identify significant differences** — see where conditions diverge with corrected significance bars
- **Inspect effect size** — overlay difference waves and t-statistics
- **Compare methods** — works with both analytic (t-test) and permutation test results

Key Functions

Function	Purpose
`EegFun.plot_erp_stats(result, channel_selection=EegFun.channels(:Cz))`	Plot analytic test results
`EegFun.plot_erp_stats(perm_result, channel_selection=EegFun.channels(:Cz))`	Plot permutation test results

Key Parameters

Parameter	Default	Description
`layout`	`:single`	Layout: `:single` , `:grid` , or `:topo`
`channel_selection`	`EegFun.channels()`	Channel predicate (e.g., `EegFun.channels(:Cz)`)
`plot_erp`	`true`	Show condition ERP waveforms
`plot_difference`	`false`	Show difference wave (A − B)
`plot_tvalues`	`false`	Overlay t-statistic curve
`plot_significance`	`false`	Add significance bars
`plot_critical_t`	`false`	Show critical t-value lines
`difference_offset`	`0.0`	Vertical offset for difference wave (μV)

`significance_position`	`:auto`	`:auto` , `:bottom` , `:zero` , or a `Float64`
`significance_color`	`(:gray, 0.6)`	Colour for significance bars

What You'll Learn

1. Plotting ERP waveforms with significance markers
2. Adding difference waves and t-value overlays
3. Controlling significance bar position and colour
4. Working with both analytic and permutation test results
5. Using layout options for multi-channel views

Code Examples

```
# Demo: Plotting Statistical Test Results
# Shows how to visualise analytic and permutation test results with
# ERP waveforms, difference waves, t-values, and significance markers.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# PREPARE DATA (requires pre-saved per-participant ERPs)
############################################################################

input_dir = EegFun.example_path("data/julia/erps")
file_pattern = "erps_final"

prepared = EegFun.prepare_stats(
    file_pattern,
    :paired;
    input_dir = input_dir,
    condition_selection = EegFun.conditions([1, 2]),
    baseline_interval = EegFun.times((-0.2, 0.0)),
    analysis_interval = EegFun.times((0.0, 2.0)),
)

############################################################################
# RUN ANALYTIC TEST
############################################################################
```

```
result = EegFun.analytic_test(prepared, correction_method = :no)

#########################################################################
# 1. BASIC – ERP waveforms (single channel)
#########################################################################

EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:PO7))

#########################################################################
# 2. ADD DIFFERENCE WAVE
#########################################################################

EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:Cz),
plot_difference = true)

#########################################################################
# 3. ADD SIGNIFICANCE MARKERS
#########################################################################

EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:PO7),
plot_difference = true, plot_significance = true)

#########################################################################
# 4. T-VALUES ONLY (no ERPs)
#########################################################################

EegFun.plot_erp_stats(
    result,
    channel_selection = EegFun.channels(:PO8),
    plot_erp = false,
    plot_tvalues = true,
    plot_critical_t = true,
    plot_significance = true,
)

#########################################################################
# 5. T-VALUES + ERPs TOGETHER
#########################################################################

EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:Pz),
plot_tvalues = true, plot_critical_t = true)

#########################################################################
# 6. SHIFT DIFFERENCE WAVE (for visibility)
#########################################################################
```

```
EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:Cz),
plot_difference = true, difference_offset = 3.0)

##########################################################################
# 7. SIGNIFICANCE BAR POSITIONING
##########################################################################

# :bottom
EegFun.plot_erp_stats(
    result,
    channel_selection = EegFun.channels(:Cz),
    plot_significance = true,
    significance_position = :bottom,
    significance_color = (:red, 0.5),
)

# :zero
EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:Cz),
plot_significance = true, significance_position = :zero)

# custom y-position
EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:Cz),
plot_significance = true, significance_position = -5.0)

##########################################################################
# 8. CUSTOM FIGURE TITLE
##########################################################################

EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:Cz),
plot_significance = true, figure_title = "N2pc Analytic Test")

##########################################################################
# 9. GRID LAYOUT — multiple channels
##########################################################################

EegFun.plot_erp_stats(result, channel_selection =
EegFun.channels([:Cz, :Pz, :Oz, :Fz]), layout = :grid, plot_significance = true)

##########################################################################
# 10. TOPO LAYOUT — channels at electrode positions
##########################################################################

EegFun.plot_erp_stats(result, layout = :topo, plot_significance = true)

##########################################################################
```

```
# 11. CAPTURE RETURN VALUE for further customisation
##########################################################################

(; fig, axes) = EegFun.plot_erp_stats(
    result,
    channel_selection = EegFun.channels(:Cz),
    plot_difference = true,
    plot_significance = true,
    display_plot = false,
)
# axes[1] is a regular Makie Axis — add annotations, save, etc.

##########################################################################
# 12. SEM BANDS (±1 SEM around waveforms)
##########################################################################

# ±1 SEM
EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:PO7), plot_se =
true)

# SEM + difference wave
EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:PO7), plot_se =
true, plot_difference = true)

# SEM + significance markers + t-values
EegFun.plot_erp_stats(result, channel_selection = EegFun.channels(:PO7), plot_se =
true, plot_significance = true, plot_tvalues = true)

##########################################################################
# RUN PERMUTATION TEST
##########################################################################

result_perm = EegFun.permutation_test(prepared, n_permutations = 1000)

##########################################################################
# 12. PERMUTATION RESULTS — ERPs + significance
##########################################################################

EegFun.plot_erp_stats(result_perm, channel_selection = EegFun.channels(:Cz),
plot_significance = true)

##########################################################################
# 13. PERMUTATION RESULTS — t-values + critical t
##########################################################################
```

```
EegFun.plot_erp_stats(result_perm, channel_selection = EegFun.channels(:Cz),
plot_erp = false, plot_tvalues = true, plot_critical_t = true)
```

- API Reference

20.4 Plot Time-Frequency

Plot Time-Frequency

This demo shows how to plot time-frequency decomposition results as heatmaps.

When to Use

Use `plot_tf` to visualise the output of `tf_morlet` , `tf_multitaper` , or `tf_stft` — the three time-frequency decomposition methods in EegFun.

Key Parameters

Parameter	Default	Description
`channel`	First channel	Which channel to plot
`baseline_interval`	`nothing`	Pre-stimulus interval for on-the-fly baseline correction
`baseline_method`	`:db` , `:percent` , `:dbchange` , `:absolute` , `:relative` , or `:zscore`	
`colormap`	`:viridis`	Colour map for the heatmap
`colorrange`	auto	Explicit `(min, max)` colour range
`yscale`	`:linear`	Frequency axis scale (`:linear` or `:log10`)
`colorbar_plot`	`true`	Show colour bar

What You'll Learn

1. Plotting time-frequency data for a specific channel
2. Applying baseline correction on the fly (dB, percent, relative change)
3. Using log-scaled frequency axes
4. Customising colour maps and colour range for publication figures

Code Examples

```
# Demo: Plotting Time-Frequency Data
# Shows how to plot time-frequency decomposition results as heatmaps,
# with options for baseline correction, colour maps, and log-scaled axes.
```

```julia
# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# LOAD DATA AND RUN TF DECOMPOSITION
############################################################################

dat = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
EegFun.polar_to_cartesian_xy!(layout)
dat = EegFun.create_eegfun_data(dat, layout)

EegFun.highpass_filter!(dat, 0.5)
EegFun.lowpass_filter!(dat, 100.0)

epoch_cfg = EegFun.EpochCondition(name = "ExampleEpoch1", trigger_sequences =
[[1]])
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.5, 1.0))  # -200 to 1000 ms

EegFun.baseline!(epochs, (-0.2, 0.0))

# Morlet wavelet decomposition
tf_data = EegFun.tf_morlet(epochs, frequencies = 2:2:60)

############################################################################
# BASIC TIME-FREQUENCY PLOT
############################################################################

# plot first available channel
EegFun.plot_tf(tf_data)

############################################################################
# SPECIFIC CHANNEL
############################################################################

EegFun.plot_tf(tf_data, channel_selection = EegFun.channels(:Cz))

############################################################################
# WITH BASELINE CORRECTION
############################################################################
```

```
# apply dB baseline
EegFun.plot_tf(tf_data, channel_selection = EegFun.channels(:Cz),
baseline_interval = (-0.3, 0.0), baseline_method = :db)

# percentage change baseline
EegFun.plot_tf(tf_data, channel_selection = EegFun.channels(:Cz),
baseline_interval = (-0.3, 0.0), baseline_method = :percent)

##########################################################################
# LOG-SCALED FREQUENCY AXIS
##########################################################################

EegFun.plot_tf(tf_data, channel_selection = EegFun.channels(:Cz),
baseline_interval = (-0.3, 0.0), yscale = :log10)

##########################################################################
# CUSTOM COLOUR MAP AND RANGE
##########################################################################

EegFun.plot_tf(
    tf_data,
    channel_selection = EegFun.channels(:Cz),
    baseline_interval = (-0.3, 0.0),
    colormap = :RdBu,
    colorrange = (-15.0, 15.0),
)
```

- API Reference

Part IV

How-to Guides

Chapter 21

Filter Data

21.1 How to Filter EEG Data

This guide shows you how to apply different types of filters to your EEG data.

High-Pass Filtering

Remove slow drifts and DC offsets:

```
using EegFun

# Apply 1 Hz high-pass filter (removes frequencies below 1 Hz)
EegFun.highpass_filter!(dat, 1.0)

# Standard 0.1 Hz filter (removes very slow drifts only)
EegFun.highpass_filter!(dat, 0.1)
```

When to use: Always apply high-pass filtering (0.1-1 Hz) to remove slow drifts.

Low-Pass Filtering

Remove high-frequency noise:

```
# Apply 40 Hz low-pass filter (removes frequencies above 40 Hz)
EegFun.lowpass_filter!(dat, 40.0)

# For very clean data or sleep studies
EegFun.lowpass_filter!(dat, 30.0)
```

When to use: Typically 30-50 Hz to remove muscle artifacts and line noise.

Band-Pass Filtering

Isolate a specific frequency range:

```
# Apply high-pass + low-pass to isolate a band
EegFun.highpass_filter!(dat, 8.0)
EegFun.lowpass_filter!(dat, 12.0)
```

> [!NOTE] There is no dedicated `bandpass_filter!` — combine a high-pass and low-pass filter instead.

Choosing Filter Parameters

Cutoff frequency — For ERP work, 0.1 Hz high-pass is standard. Use 1 Hz only for ICA preprocessing. Low-pass at 30-40 Hz removes most muscle noise.

Filter order — EegFun defaults to order 1 for high-pass and order 3 for low-pass (Butterworth IIR). Higher orders give sharper rolloff but increase ringing and phase distortion. The defaults are safe for most EEG work.

IIR vs FIR — IIR (default) is fast and suitable for most use cases. FIR filters have linear phase but require many more taps; use `filter_method="fir"` if you need guaranteed zero phase distortion beyond what `filtfilt` provides.

Zero-phase filtering — `filtfilt` (default) applies the filter forwards and backwards, eliminating phase distortion but effectively doubling the filter order.

```
# Customise filter parameters
EegFun.highpass_filter!(dat, 0.1; order=2, filter_method="iir")
EegFun.lowpass_filter!(dat, 30.0; order=4, filter_method="fir")
```

Verifying Filter Response

```
# Create a filter and inspect it
filter_info = EegFun.create_highpass_filter(0.1, 256.0)
EegFun.print_filter_characteristics(filter_info)
EegFun.plot_filter_response(filter_info)
```

See Also

- API Reference - Complete function documentation

Chapter 22

Selection Patterns

22.1 Selection Patterns

EegFun.jl uses **predicate generators** — functions that return selection rules — to filter channels, samples, epochs, components, and time intervals. This approach keeps analysis code readable, composable, and independent of the data at hand.

Why Predicates?

You might wonder why we use `channels([:Cz, :Pz])` instead of just `[:Cz, :Pz]`.

1. **Lazy evaluation** — the selection rule is defined once but resolved when the data is actually accessed, so it works regardless of which channels a file happens to contain
2. **Order preservation** — `channels([:Pz, :Cz])` returns data in the order you specified, not file order
3. **Composability** — you can combine rules and pass them to any function that accepts a selection keyword

File Selection

In TOML configuration files, file selectors use strings or regex patterns:

```
### Basic patterns ###

# Single file
raw_data_files = "subject_01.bdf"

# All BDF files in the directory
raw_data_files = "\\.bdf$"

# All BrainVision files
raw_data_files = "\\.vhdr$"

# Files starting with "sub-" followed by digits
raw_data_files = "^sub-\\d+\\.bdf$"

### Filtering by participant range ###

# Only participants 1–9 (single digit)
raw_data_files = "sub-0[1-9]\\.bdf$"

# Two-digit participant IDs (01–99)
```

```
raw_data_files = "sub-\\d{2}\\.bdf$"

### Advanced patterns ###

# Exclude specific participants (03, 07, 12)
raw_data_files = "sub-(?!03|07|12)\\d+\\.bdf$"

# Match multiple naming conventions with alternation
raw_data_files = "(sub|pp)[-_]\\d+\\.bdf$"

# Files containing a specific task name anywhere
raw_data_files = "task-flanker.*\\.bdf$"
```

In Julia scripts, use `EegFun.get_files` to resolve a pattern against a directory:

```
# Pattern matching — same regex syntax as the TOML config
files = EegFun.get_files("./raw_data", "\\.bdf$")

# Only participants with two-digit IDs
files = EegFun.get_files("./raw_data", "sub-\\d{2}\\.bdf\$")

# Explicit list of files
files = EegFun.get_files("./raw_data", ["sub-01.bdf", "sub-05.bdf"])
```

Files are returned in natural sort order (e.g. `sub-2` before `sub-10`).

Channel Selection

Used in functions like `subset`, `rereference!`, `repair_channels!`, and `detect_bad_epochs_automatic`.

Predicate	Selects
`channels()`	All channels
`channels(:Cz)`	A single channel by name
`channels([:Fp1, :Fp2])`	A list of channels
`channels(1:10)`	Channels by index
`channels_not(:hEOG)`	Everything except the named channel(s)
`channels_not([:vEOG, :hEOG])`	Everything except a list of channels

```
# Rereference only scalp channels (exclude EOG)
EegFun.rereference!(dat, :avg, channel_selection =
EegFun.channels_not([:vEOG, :hEOG]))

# Run artifact detection on a subset of channels
```

```
EegFun.detect_bad_epochs_automatic(epochs, channel_selection =
EegFun.channels([:Cz, :Pz, :Fz]))
```

Custom Channel Selection

Pass a function that receives a `Vector{Symbol}` of all available channel labels:

```
# Select all frontal channels (labels containing "F")
frontal = EegFun.channels(x -> contains.(string.(x), "F"))
EegFun.subset(dat, channel_selection = frontal)
```

Sample Selection

Used for filtering rows of continuous data by metadata columns (e.g. artifact flags). Common in `run_ica` , `correlation_matrix` , and `channel_summary` .

Predicate	Selects
`samples()`	All samples
`samples(:is_epoch_interval)`	Samples where the flag column is `true`
`samples_not(:is_extreme)`	Samples where the flag is `false`
`samples_and([:is_epoch, :is_clean])`	Samples where **all** flags are `true`
`samples_or([:is_extreme, :is_muscle])`	Samples where **any** flag is `true`
`samples_and_not([:is_extreme, :is_step])`	Samples where **at least one** flag is `false`
`samples_or_not([:is_extreme, :is_step])`	Samples where **none** of the flags are `true`

```
# Run ICA only on clean, non-extreme data
ica = EegFun.run_ica(dat, sample_selection = EegFun.samples_not(:is_extreme))

# Compute channel summary for the epoch interval only
EegFun.channel_summary(dat, sample_selection = EegFun.samples(:epoch_interval))
```

Custom Sample Selection

You can also pass a custom function where `x` refers to the data:

```
# Select samples between 100 and 1000
EegFun.subset(dat, sample_selection = EegFun.samples(x -> 100 .<= x.sample .<=
1000))
```

Time Selection

Used to define an interval in seconds for functions like `subset` , `erp_measurements` , and `plot_erp` .

Predicate	Selects
`EegFun.times()`	All time points
`EegFun.times(0.3, 0.5)`	300–500 ms
`EegFun.times(-0.2, 0.0)`	Baseline period

```
# Measure mean amplitude in the N400 interval
EegFun.erp_measurements(erps, "mean_amplitude", analysis_interval = (0.3, 0.5))

# Subset data to the baseline period
EegFun.subset(dat, interval_selection = EegFun.times(-0.2, 0.0))
```

Epoch Selection

Used in `subset` and `reject_epochs` to filter by epoch index or by epoch content.

Predicate	Selects
`EegFun.epochs()`	All epochs
`EegFun.epochs(1:20)`	Epochs 1–20
`EegFun.epochs(5)`	A single epoch
`EegFun.epochs_not(1:5)`	All epochs except 1–5

```
# Keep only the first 50 epochs
EegFun.subset(epochs_data, epoch_selection = EegFun.epochs(1:50))
```

Custom Epoch Selection

Pass a function that receives a `DataFrame` (the epoch's data) and returns a `Bool` . The predicate is mapped over all epochs, so each epoch is evaluated independently.

```
# Keep only epochs that have more than 100 samples
EegFun.subset(epochs_data, epoch_selection = EegFun.epochs(df -> nrow(df) > 100))

# Drop RT outlier trials (requires calculate_trigger_interval! to have added
an :interval column)
EegFun.subset(epochs_data, epoch_selection = EegFun.epochs(df -> 0.2 <
df.interval[1] < 1.0))
```

Component Selection

Used with ICA functions to select or exclude specific independent components.

Predicate	Selects

`EegFun.components()`	All components
`EegFun.components(1:3)`	Components 1–3
`EegFun.components(5)`	A single component
`EegFun.components_not(1:3)`	All components except 1–3

```
# Remove identified artifact components
EegFun.subtract_ica_components!(dat, ica, component_selection =
EegFun.components([1, 3, 7]))
```

Summary

Type	Select	Exclude	Combine
Channels	`channels(...)`	`channels_not(...)`	—
Samples	`samples(...)`	`samples_not(...)`	`samples_and(...)`, `samples_or(...)`
Time	`times(...)`	—	—
Epochs	`epochs(...)`	`epochs_not(...)`	—
Components	`components(...)`	`components_not(...)`	—
Conditions	`conditions(...)`	`conditions_not(...)`	—
Participants	`participants(...)`	`participants_not(...)`	—
Files	regex / list	—	—

Next Steps

- **Using selections in preprocessing** — Manual Preprocessing shows channel and sample selectors applied throughout the pipeline
- **Using selections with artifacts** — Artifact Handling uses `channel_selection` and `sample_selection` in detection functions
- **File patterns in batch pipelines** — Batch Processing uses the regex file selector in TOML configuration

Chapter 23

TOML Format

23.1 TOML Format

EegFun.jl uses TOML (Tom's Obvious Minimal Language) for its pipeline configuration files and epoch condition files. TOML is designed to be easy to read and write — if you can read an INI file, you already know most of what you need.

Key–Value Pairs

The most basic building block. Values can be strings, numbers, or booleans:

```
directory = "./preprocessed_files"
epoch_start = -0.2
apply = true
```

Strings use double quotes. Numbers and booleans (`true` / `false`) are unquoted.

Comments

Lines starting with **#** are comments and are ignored:

```
# High-pass filter settings
freq = 0.1   # cutoff in Hz
```

Tables (Sections)

Square brackets define a **table** (a group of related settings). Everything below a table header belongs to that table, until the next header:

```
[files.input]
directory = "."
layout_file = "biosemi72.csv"

[files.output]
directory = "./preprocessed_files"
save_ica_data = true
```

Dots create **nested** tables. The example above is equivalent to:

```
[files]

  [files.input]
  directory = "."
```

```
[files.output]
directory = "./preprocessed_files"
```

Arrays

Square brackets around values create arrays (lists):

```
raw_data_files = ["participant_01.bdf", "participant_02.bdf"]
```

Nested arrays are arrays of arrays:

```
# Each inner array lists alternative channel names (first match wins)
vEOG_channels = [["Fp1", "Fp2"], ["IO1", "IO2"], ["vEOG"]]
```

Arrays of Tables

Double square brackets `[[...]]` define an **array of tables** — a list of identically structured sections. This is how epoch conditions are specified:

```
[epochs]

[[epochs.conditions]]
name = "standard"
trigger_sequences = [[1]]
reference_index = 1

[[epochs.conditions]]
name = "deviant"
trigger_sequences = [[2]]
reference_index = 1

[[epochs.conditions]]
name = "novel"
trigger_sequences = [[3, 4]]
reference_index = 1
timing_pairs = [[1, 2]]
min_interval = 0.1
max_interval = 2.0
```

Each `[[epochs.conditions]]` block adds one entry to the `conditions` list inside the `epochs` table.

Quick Reference

Feature	Syntax	Example
String	`"..."`	`"avg"`

Integer	bare number	`1`
Float	decimal number	`0.1`
Boolean	`true` / `false`	`true`
Array	`[...]`	`[1, 2, 3]`
Table	`[name]`	`[preprocess]`
Nested table	`[a.b]`	`[preprocess.eeg]`
Array of tables	`[[name]]`	`[[epochs.conditions]]`
Comment	`# ...`	`# cutoff in Hz`

Further Reading

- TOML homepage — introduction and full language reference
- Pipeline configuration — using TOML with EegFun.jl's preprocessing pipeline

Chapter 24

Batch Processing

24.1 Automated Batch Processing

EegFun.jl provides automated pipelines that process an entire cohort of EEG recordings from raw data to cleaned epochs. You provide a TOML configuration file; the pipeline handles file discovery, preprocessing, artifact management, and cohort-level reporting.

Running a Pipeline

```
using EegFun

# Run the pipeline
EegFun.preprocess("config.toml")
```

Relative paths in the TOML file are resolved relative to the config file's directory, so you can keep the config alongside your analysis script.

TOML Configuration

A pipeline config must specify **input files**, **output settings**, and **preprocessing parameters**. You only need to include values that differ from the defaults — EegFun merges your config with `src/config/default.toml`.

Minimal Example

```
[files.input]
directory = "./raw_data"
raw_data_files = "\\.bdf"          # regex pattern matching raw files
layout_file = "biosemi64.csv"
epoch_condition_file = "epochs.toml"

[files.output]
directory = "./preprocessed_files"

[preprocess]
epoch_start = -0.2
epoch_end = 0.8
reference_channel = "avg"
```

Key Configuration Sections

Section	Controls
`[files.input]`	Raw data directory, file pattern, layout file, epoch condition file
`[files.output]`	Output directory, which intermediate files to save (epochs, ERPs, ICA)
`[preprocess]`	Epoch interval, reference channel
`[preprocess.eeg]`	Artifact thresholds (absolute µV and z-score)
`[preprocess.eog]`	EOG channel definitions and detection criteria
`[preprocess.filter.highpass]`	High-pass filter (cutoff, order, method)
`[preprocess.filter.lowpass]`	Low-pass filter (cutoff, order, method)
`[preprocess.filter.ica_highpass]`	Separate high-pass for the ICA copy (typically 1 Hz)
`[preprocess.filter.ica_lowpass]`	Optional low-pass for the ICA copy
`[preprocess.ica]`	Whether to apply ICA, percentage of data to use
`[preprocess.layout]`	Neighbor distance criterion for channel repair

> [!TIP] See `src/config/default.toml` for all available options and their defaults. Copy it as a starting point and modify only the values you need to change.

Epoch Condition File

The epoch condition file is a separate TOML that defines which trigger sequences form each experimental condition:

```toml
[epochs]

[[epochs.conditions]]
name = "standard"
trigger_sequences = [[1]]
reference_index = 1

[[epochs.conditions]]
name = "deviant"
trigger_sequences = [[2]]
reference_index = 1

# Multi-trigger sequence with timing constraint
[[epochs.conditions]]
```

```toml
name = "response_locked"
trigger_sequences = [[1, 100], [2, 100]]
reference_index = 2
timing_pairs = [[1, 2]]
min_interval = 0.1
max_interval = 1.5
```

Processing Lifecycle

Phase 1 — Setup

- Loads and validates the TOML configuration
- Reads the electrode layout and computes spatial neighbors
- Discovers raw data files matching the file pattern
- Parses epoch condition definitions

Phase 2 — Continuous Processing (per file)

- **Rereferencing** — standardises electrode voltages (e.g. average reference)
- **Filtering** — high-pass to remove drift, optional low-pass
- **Artifact detection** — marks extreme values and identifies bad channels via channel joint probability and z-score variance
- **ICA** — optionally runs ICA on a separately filtered copy of the data, identifies artifact components (eye blinks, muscle, line noise), and removes them from the original
- **Channel repair** — interpolates bad scalp channels using neighbor weights

Phase 3 — Epoching and Cleanup

- **Epoch extraction** — cuts the continuous data around triggers
- **Baseline correction** — subtracts the pre-stimulus mean
- **Per-epoch artifact detection** — flags epochs exceeding amplitude thresholds
- **Per-epoch repair** — interpolates channels that are only bad in specific epochs
- **Epoch rejection** — removes epochs that cannot be repaired

Phase 4 — Cohort Consolidation

After all files are processed:

- **Epoch count summary** — how many trials survived per participant and condition
- **Channel reliability** — which electrodes were frequently repaired
- **ICA statistics** — average components removed per participant

Logging and Error Isolation

Comprehensive logging ensures that all information regarding the preprocessing steps can be easily retrieved.

If a single file fails (corrupt data, missing triggers, etc.), the pipeline logs the error and continues with the remaining files. As part of the output, you receive:

- A **study-level log** summarising the entire cohort run, including aggregate statistics.
- **Per-file logs** detailing every warning, decision, and step taken for an individual participant. This includes critical metrics such as:
  - The number and percentage of trials removed/remaining per condition.
  - The number of ICA components identified and removed.
  - Channels that were repaired or interpolated.
- **Traceability** — output files record the exact pipeline version and configuration used to ensure reproducibility.

Output Files

The pipeline saves intermediate and final outputs as JLD2 files (controlled by `[files.output]` flags):

File suffix	Contents
`_continuous_raw`	Raw continuous data after import
`_continuous_corrected`	Continuous data after filtering, ICA, and repair
`_epochs_uncorrected`	Epochs before artifact handling
`_epochs_unrejected`	Epochs after channel repair
`_epochs_final`	Epochs after rejection (final clean data)
`_erps_uncorrected` / `_erps_unrejected` / `_erps_final`	Averaged ERPs at each stage
`_ica`	ICA decomposition results
`_artifact_info`	Artifact tracking (repaired channels, rejected epochs, ICA components)

Custom Pipelines

We provide a dedicated function that creates a skeleton template so researchers can piece together their own tailored preprocessing pipelines. If the built-in pipelines don't perfectly match your workflow, this command generates a template with the standard boilerplate (config loading, logging, error handling) already wired up:

```
EegFun.generate_pipeline_template("my_pipeline.jl", "my_preprocess")
```

This creates a Julia file with a complete pipeline skeleton. You can then edit the specific processing steps to suit your experiment, and run it just like the default pipeline:

```
include("my_pipeline.jl")
my_preprocess("config.toml")
```

Post-Processing

After batch preprocessing, use `subset_bad_data` to exclude participants with too much data loss:

```
EegFun.subset_bad_data("preprocessed_files", 70.0)
```

This moves participants with less than 70% data retention to an "excluded" subdirectory.

Philosophy

The pipeline follows a **pragmatic, minimal-intervention** approach to preprocessing. While guided by the principle that excessive cleaning can distort the signal (Delorme, 2023), our approach sits slightly above "EEG is better left alone." We actively remove clear artifacts and by default use ICA to remove standard artifact components. Rather than applying an exhaustive sequence of transformations, the pipeline focuses on:

- **Filtering conservatively** — a gentle high-pass to remove drift, with optional low-pass
- **Using ICA primarily for clear EOG artifacts** — eye blinks and saccades are the main targets for component removal, rather than attempting to classify and remove every possible source of noise
- **Repairing rather than rejecting** where possible — interpolating isolated bad channels preserves trial counts
- **Rejecting only clearly contaminated epochs** — using statistical thresholds rather than overly aggressive criteria

Several processing steps draw on the statistical thresholding approach described in FASTER (Nolan, Whelan, & Reilly, 2010), and the practical workflow has been influenced by Dudschig, Mackenzie, Strozyk, Kaup, & Leuthold (2016).

References:

- Delorme, A. (2023). EEG is better left alone. *Scientific Reports*, 13(1), 2372.
- Nolan, H., Whelan, R., & Reilly, R. B. (2010). FASTER: Fully automated statistical thresholding for EEG artifact rejection. *Journal of Neuroscience Methods*, 192(1), 152–162.
- Dudschig, C., Mackenzie, I. G., Strozyk, J., Kaup, B., & Leuthold, H. (2016). The sounds of sentences: Differentiating the influence of physical sound, sound imagery, and linguistically implied sounds on physical sound processing. *Cognitive, Affective, & Behavioral Neuroscience*, 16(5), 940–961.

Next Steps

- **Understanding each step** — Manual Preprocessing explains the rationale behind every pipeline stage
- **Artifact detection options** — Artifact Handling for interactive review and fine-grained control
- **Epoch condition syntax** — Epoch Selection for trigger patterns and TOML condition files
- **Selection predicates** — Selection Patterns for channel, sample, and time filters used in config and scripts

Chapter 25

Pipeline Templates

25.1 Pipeline Templates

This demo shows how to generate scaffold files for custom preprocessing pipelines and configuration files.

Pipeline Templates

`generate_pipeline_template` creates a ready-to-use Julia file with:

- Structured preprocessing loop over participants
- Built-in logging setup and error handling
- Numbered processing steps with placeholder comments
- Configuration loading from TOML files

This saves time when starting a new preprocessing project by providing a well-organised starting point.

Configuration Templates

`generate_config_template` creates a TOML configuration file with all available preprocessing parameters:

- **Files**: Input/output directory paths
- **Filter**: High-pass, low-pass, and ICA-specific filter settings
- **EOG**: EOG channel names and correlation thresholds
- **EEG**: Artifact and extreme value detection thresholds
- **ICA**: Whether to apply ICA and data percentage settings
- **Layout**: Neighbor distance criterion
- **Epoch**: Timing parameters (start/end times)

Using Templates with Pipelines

The generated pipeline and config files are designed to work together:
1. Generate both template files
2. Edit the config TOML with your study-specific parameters
3. Customise the pipeline Julia file with your processing steps
4. Run the pipeline

Workflow Summary

This demo covers:

- Generating a pipeline template file
- Customising template options (number of steps, subsections)
- Generating a configuration template
- Understanding the generated config sections

Code Examples

```
# Demo: Pipeline Templates
# Shows how to generate scaffold files for custom preprocessing pipelines
# and configuration templates.

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie

############################################################################
# GENERATE A PIPELINE TEMPLATE
############################################################################

# Generate a basic pipeline template with default settings
# This creates a .jl file with all the boilerplate: logging, config loading,
# error handling, and a structured loop for custom processing steps
EegFun.generate_pipeline_template("my_custom_pipeline.jl")

# Generate with a custom function name and more steps
EegFun.generate_pipeline_template(
    "my_pipeline.jl",
    "my_preprocess";
    options = EegFun.PipelineTemplateOptions(num_steps = 8, subsections_per_step =
    3),
)



############################################################################
# GENERATE A CONFIG TEMPLATE
############################################################################

# Generate a TOML configuration template with all available settings
EegFun.generate_config_template(filename = "my_config.toml")

# The generated config includes sections for:
# - files (input/output paths)
```

```
# - filter (highpass, lowpass, ICA-specific filters)
# - eog (EOG channel configuration and thresholds)
# - eeg (artifact and extreme value thresholds)
# - ica (whether to apply, percentage of data)
# - layout (neighbor criterion)
# - epoch timing (start/end times)

# You can read and inspect a config file
# cfg = EegFun.read_config("my_config.toml")
# EegFun.print_config(cfg)



##########################################################################
# CLEANUP
##########################################################################

# Remove generated template files (for demo purposes)
isfile("my_custom_pipeline.jl") && rm("my_custom_pipeline.jl")
isfile("my_pipeline.jl") && rm("my_pipeline.jl")
isfile("my_config.toml") && rm("my_config.toml")
```

- API Reference

Chapter 26

Preprocessing Walkthrough

26.1 Preprocessing Workflow

This demo walks through a complete manual preprocessing pipeline from raw BioSemi data to cleaned ERPs. It mirrors the steps and order used in `EegFun.preprocess` , making it a useful reference for understanding what the automated pipeline does or for building a custom workflow.

When to Use This

- **Learning** — understand each preprocessing step and why it is applied in a specific order
- **Custom workflows** — adapt the pipeline to your specific experimental design
- **Debugging** — step through the pipeline interactively to diagnose issues with your data

Processing Phases

The workflow follows three phases:

Phase 1: Setup and Initial Preprocessing

- Load raw data and configure electrode layout with coordinates and neighbors
- Mark epoch intervals for targeted analysis
- Rereference to average
- Apply initial filters (0.1 Hz high-pass for ERPs)
- Calculate and detect EOG (vEOG, hEOG)
- Extract “original” epochs for later comparison

Phase 2: Artifact Detection and Cleaning (Continuous Level)

- Compute channel summary statistics and joint probability
- Detect extreme values at two thresholds (250 μV for ICA exclusion, 75 μV for artifact rejection)
- Identify and partition bad channels (EOG-related vs. non-EOG-related)
- Run ICA on a separate 1 Hz high-pass filtered copy, identify and remove artifact components
- Repair bad channels via neighbor interpolation
- Recalculate EOG channels after ICA and repair

Phase 3: Epoch Extraction and Epoch-Level Processing

- Extract epochs from cleaned continuous data
- Baseline correction
- Two rounds of automatic epoch detection with channel repair between rounds

- Reject remaining bad epochs
- Average to ERPs and compare against originals

Key Takeaway

The demo produces both "original" (lightly preprocessed) and "good" (fully cleaned) ERPs so you can visually assess the effect of the entire pipeline.

Code Examples

```
# Demo: End-to-End Preprocessing Workflow
# Shows a complete manual preprocessing pipeline from raw data to ERPs.
# This mirrors the steps and order used in pipeline_v1.jl

# Note: EegFun.example_path() resolves bundled example data paths.
# When using your own data, simply pass the file path directly, e.g.:
# dat = EegFun.read_raw_data("/path/to/your/data.bdf")

using EegFun
using GLMakie
using JLD2

const DEMO_OUTPUT = "./tutorials/output/"
mkpath(DEMO_OUTPUT)

##########################################################################
#
# PHASE 1: BASIC SETUP AND INITIAL PREPROCESSING
#
# This phase prepares the data with basic preprocessing before
# artifact detection and ICA
#
##########################################################################

##########################################################################
# LOAD RAW DATA
##########################################################################

@info EegFun.section("Raw Data")

# Read raw data file
raw_data = EegFun.read_raw_data(EegFun.example_path("data/bdf/example1.bdf"))

# Load electrode layout and configure all coordinate systems
layout = EegFun.read_layout(EegFun.example_path("layouts/biosemi/biosemi72.csv"))
```

```
EegFun.polar_to_cartesian_xy!(layout)   # For 2D topography plots
EegFun.polar_to_cartesian_xyz!(layout)  # For 3D interpolation (spherical spline)
EegFun.get_neighbors_xy!(layout, 0.35) # Calculate neighbors (normalized units for
this layout)

# Optional: Visualize layout and neighbors
# EegFun.plot_layout_2d(layout, neighbors = true)

# Create EegFun data structure
dat = EegFun.create_eegfun_data(raw_data, layout)

# Optional: save original continuous data
# jldsave(joinpath(DEMO_OUTPUT, "continuous_raw.jld2"); data = dat)



##############################################################################
# MARK EPOCH WINDOWS
##############################################################################

@info EegFun.section("Marking epoch intervals")

# Define epoch conditions
epoch_cfg =
    [EegFun.EpochCondition(name = "Standard", trigger_sequences = [[1]]),
    EegFun.EpochCondition(name = "Target", trigger_sequences = [[2]])]

# Mark which samples belong to epoch intervals
# This creates a :epoch_interval column for targeted analysis
EegFun.mark_epoch_intervals!(dat, epoch_cfg, [-0.2, 1.5]) # -200 to 1.5 seconds

# Optional: visualize triggers
# EegFun.plot_trigger_overview(dat)
# EegFun.plot_trigger_timing(dat)



##############################################################################
# REREFERENCE
##############################################################################

@info EegFun.section("Rereference")

# Rereference to average
EegFun.rereference!(dat, :avg)
```

```
#########################################################################
# INITIAL FILTERS
#########################################################################

@info EegFun.section("Initial Filters")

# Apply high-pass filter (0.1 Hz for ERPs)
EegFun.highpass_filter!(dat, 0.1) # NB a 1Hz filter is applied to the data for ICA
later

#########################################################################
# EOG CALCULATION AND DETECTION
#########################################################################

@info EegFun.section("EOG")

# Calculate vertical EOG (vEOG)
@info EegFun.subsection("Calculating EOG (vEOG/hEOG) channels")
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
)

# Calculate horizontal EOG (hEOG)
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
)

# Detect EOG onsets (blinks and saccades)
@info EegFun.subsection("Detecting EOG (vEOG/hEOG) onsets")
EegFun.detect_eog_onsets!(dat, 50.0, :vEOG, :is_veog)
EegFun.detect_eog_onsets!(dat, 30.0, :hEOG, :is_heog)

# Calculate channel x EOG correlation matrices
@info EegFun.subsection("Channel x vEOG/hEOG Correlation Matrix")

# Create EOG configuration to match pipeline structure
eog_cfg = EegFun.EogConfig(
    vEOG_criterion = 50.0,
    hEOG_criterion = 30.0,
```

```
    vEOG_channels = [["Fp1", "Fp2"], ["IO1", "IO2"], ["vEOG"]],
    hEOG_channels = [["F9"], ["F10"], ["hEOG"]],
)

# Calculate correlations across whole dataset
hEOG_vEOG_cm = EegFun.correlation_matrix_eog(dat, eog_cfg)
# Optional: view correlations
# println(hEOG_vEOG_cm)

# Calculate correlations within epoch intervals only
@info EegFun.subsection("Channel x vEOG/hEOG Correlation Matrix (epoch interval)")
hEOG_vEOG_cm_epoch = EegFun.correlation_matrix_eog(dat, eog_cfg, sample_selection
= EegFun.samples(:epoch_interval))
# This is used later to identify channels with high EOG correlation

################################################################################
# INITIAL EPOCH EXTRACTION (FOR COMPARISON)
################################################################################

@info EegFun.section("Initial Epoch Extraction")

# Extract epochs from "lightly preprocessed" continuous data
# (filtered and rereferenced, but NO ICA or artifact removal yet)
epochs_uncorrected = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))
erps_uncorrected = EegFun.average_epochs(epochs_uncorrected)

# Optional: save for comparison
# jldsave(joinpath(DEMO_OUTPUT, "epochs_uncorrected.jld2"); data =
epochs_uncorrected)
# jldsave(joinpath(DEMO_OUTPUT, "erps_uncorrected.jld2"); data = erps_uncorrected)


################################################################################
#
# PHASE 2: ARTIFACT DETECTION AND CLEANING (CONTINUOUS LEVEL)
#
# Now we identify and remove artifacts from the continuous data
# before final epoch extraction
#
################################################################################

################################################################################
# CHANNEL SUMMARY AND BAD CHANNEL DETECTION
################################################################################
```

```
@info EegFun.section("Artifact Detection: Continuous Data")
@info EegFun.subsection("Channel Summary")

# Calculate channel statistics on whole dataset
summary_whole_dataset = EegFun.channel_summary(dat)

# Calculate channel statistics on epoch interval only
summary_epoch_interval = EegFun.channel_summary(dat, sample_selection =
EegFun.samples(:epoch_interval))



######################################################################
# EXTREME VALUE DETECTION
######################################################################

@info EegFun.subsection("Artifact Detection (extreme values)")

# Detect extreme values (250 μV) - will exclude from ICA
@info "Detecting extreme values: 250.0 μV"
EegFun.is_extreme_value!(dat, 250.0, channel_out = :is_extreme_value_250)

# Detect artifact values (75 μV) - for epoch rejection
@info EegFun.subsection("Artifact Detection (criterion values)")
@info "Detecting artifact values: 75.0 μV"
EegFun.is_extreme_value!(dat, 75.0, channel_out = :is_artifact_value_75)



######################################################################
# CHANNEL JOINT PROBABILITY
######################################################################

@info EegFun.subsection("Bad Channel Detection using Channel Joint Probability +
Z-Score Variance")

# Calculate joint probability on whole dataset
cjp_whole_dataset = EegFun.channel_joint_probability(dat)

# Calculate joint probability on epoch interval
cjp_epoch_interval = EegFun.channel_joint_probability(dat, sample_selection =
EegFun.samples(:epoch_interval))

# Identify bad channels
@info EegFun.subsubsection("Bad Channels")
bad_channels_whole_dataset = EegFun.identify_bad_channels(summary_whole_dataset,
cjp_whole_dataset)
```

```
bad_channels_epoch_interval = EegFun.identify_bad_channels(summary_epoch_interval,
cjp_epoch_interval)

# Take intersection (more conservative)
bad_channels = intersect(bad_channels_whole_dataset, bad_channels_epoch_interval)

# Partition into EOG-related vs non-EOG-related
# EOG-related bad channels are better handled by ICA
bad_channels_non_eog_related, bad_channels_eog_related =
EegFun.partition_channels_by_eog_correlation(
    bad_channels,
    hEOG_vEOG_cm_epoch;
    eog_channels = [:hEOG, :vEOG],
    threshold = 0.3,
    use_z = false,
)

@info "Bad channels (non-EOG related): $(length(bad_channels_non_eog_related)) -
$bad_channels_non_eog_related"

# Create repair info for continuous-level repair
continuous_repair_info = nothing
if !isempty(bad_channels_non_eog_related)
    continuous_repair_info =
    EegFun.create_continuous_repair_info(:neighbor_interpolation)
    EegFun.channel_repairable!(continuous_repair_info,
    bad_channels_non_eog_related, layout)
end



#########################################################################
# ICA
#########################################################################

@info EegFun.section("ICA")

# Create a copy for ICA preprocessing
dat_ica = copy(dat)

# Remove bad channels before ICA (if any were identified as repairable)
if !isnothing(continuous_repair_info) && !isempty(continuous_repair_info.repaired)
    @info EegFun.subsection("Removing repairable bad channels for ICA")
    dat_ica = EegFun.subset(dat_ica, channel_selection =
    EegFun.channels_not(continuous_repair_info.repaired), include_extra = true)
    @info "Removed $(length(continuous_repair_info.repaired)) channels:
    $(continuous_repair_info.repaired)"
```

```
end

# Apply ICA-specific filters (1 Hz high-pass)
@info EegFun.subsection("ICA filters")
EegFun.highpass_filter!(dat_ica, 1.0)  # Stronger high-pass for ICA
EegFun.lowpass_filter!(dat_ica, 40.0)  # Optional: wider bandwidth

# Run ICA on clean sections only
@info EegFun.subsection("Running ICA")
ica_result = EegFun.run_ica(
    dat_ica,
    sample_selection = EegFun.samples_not(:is_extreme_value_250),
    percentage_of_data = 100,  # Use all clean data (or 20-50 for speed)
)

# Identify artifact components
@info EegFun.subsection("Component Identification")
component_artifacts, component_metrics = EegFun.identify_components(
    dat,  # Use original data for identification (important!)
    ica_result,
    sample_selection = EegFun.samples_not(:is_extreme_value_250),
)

# plot examples to visually inspect automatic component selections
EegFun.plot_topography(ica_result, component_selection =
EegFun.components(component_artifacts.eog[:vEOG]))
EegFun.plot_topography(ica_result, component_selection =
EegFun.components(component_artifacts.eog[:hEOG]))
EegFun.plot_topography(ica_result, component_selection = EegFun.components([1, 2,
3, 4]))
EegFun.plot_topography(ica_result, component_selection =
EegFun.components(component_artifacts.line_noise))
EegFun.plot_ica_component_spectrum(
    dat,
    ica_result,
    component_selection = EegFun.components(component_artifacts.line_noise),
    sample_selection = EegFun.samples_not(:is_extreme_value_250),
)
EegFun.plot_topography(ica_result, component_selection =
EegFun.components(component_artifacts.channel_noise))
EegFun.plot_ica_component_spectrum(dat_ica, ica_result, component_selection =
EegFun.components(component_artifacts.channel_noise))
EegFun.plot_ica_component_activation(dat_ica, ica_result, component_selection =
EegFun.components([1, 2, 3, 4]))
EegFun.plot_ica_component_activation(dat, ica_result)
```

```
EegFun.plot_ica_component_activation(dat, ica_result, artifacts =
component_artifacts)

# Optional: view component metrics
component_metrics[:eog_metrics]
component_metrics[:line_noise_metrics]

# Remove artifact components
@info EegFun.subsection("Removing ICA components")
all_removed_components = EegFun.get_all_ica_components(component_artifacts)
@info "Removed $(length(all_removed_components)) ICA components:
$component_artifacts"
EegFun.subtract_ica_components!(dat, ica_result, component_selection =
EegFun.components(all_removed_components))

# Optional: save ICA decomposition
# jldsave(joinpath(DEMO_OUTPUT, "ica.jld2"); data = ica_result)



##############################################################################
# REPAIR BAD CHANNELS (CONTINUOUS LEVEL)
##############################################################################

if !isnothing(continuous_repair_info)
    @info EegFun.section("Channel Repair")
    EegFun.repair_channels!(dat, continuous_repair_info; method
    = :neighbor_interpolation)
    # EegFun.repair_channels!(dat, continuous_repair_info; method
    = :spline_interpolation)
    @info continuous_repair_info
end



##############################################################################
# RECALCULATE EOG AFTER ICA AND REPAIR
##############################################################################

@info EegFun.section("EOG Recalculation")
@info EegFun.subsection("Recalculating EOG (vEOG/hEOG) channels after ICA and
repair")
# EOG channels must be recalculated because underlying data changed
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
```

```
)
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
)


########################################################################
# FINAL ARTIFACT DETECTION FOR EPOCHING
########################################################################

@info EegFun.section("Detecting artifact values in continuous data")

# Re-detect artifacts after ICA and repair
EegFun.is_extreme_value!(dat, 75.0, channel_out = :is_artifact_value_75_final)

# Optional: save cleaned continuous data
# jldsave(joinpath(DEMO_OUTPUT, "continuous_corrected.jld2"); data = dat)


########################################################################
#
# PHASE 3: EPOCH EXTRACTION AND EPOCH-LEVEL PROCESSING
#
# Extract final epochs from cleaned data and perform epoch-level
# artifact rejection with repair
#
########################################################################

########################################################################
# EXTRACT EPOCHS (CLEANED)
########################################################################

@info EegFun.section("Extracting cleaned epoched data")
# Extract epochs from fully cleaned continuous data
epochs = EegFun.extract_epochs(dat, epoch_cfg, (-0.2, 1.0))


########################################################################
# BASELINE WHOLE EPOCHS
########################################################################

@info EegFun.section("Baseline whole epochs")
```

```
# Apply baseline correction (using entire epoch by default)
EegFun.baseline!(epochs)


################################################################################
# DETECT BAD EPOCHS (ROUND 1)
################################################################################

@info EegFun.section("Automatic epoch detection")
# First round of bad epoch detection
rejection_info_step1 = EegFun.detect_bad_epochs_automatic(
    epochs;
    z_criterion = 0.0,    # No z-score (pipeline_v1 uses 0.0)
    abs_criterion = 75.0, # Absolute voltage threshold
    name = "rejection_step1",
)
# Identify which channels in bad epochs are repairable
EegFun.channel_repairable!(rejection_info_step1, epochs[1].layout)
@info rejection_info_step1


################################################################################
# CHANNEL REPAIR PER EPOCH
################################################################################

@info EegFun.section("Channel Repair per Epoch")
# Repair bad channels within individual epochs
# This rescues trials with isolated bad channels
EegFun.repair_artifacts!(epochs, rejection_info_step1)

# Optional: save epochs with repairs
# jldsave(joinpath(DEMO_OUTPUT, "epochs_unrejected.jld2"); data = epochs)


################################################################################
# RE-DETECT ARTIFACTS AFTER REPAIR (ROUND 2)
################################################################################

@info EegFun.subsection("Re-detecting artifacts after repair")
# Second round of detection to check repair effectiveness
rejection_info_step2 = EegFun.detect_bad_epochs_automatic(epochs; z_criterion =
0.0, abs_criterion = 75.0, name = "rejection_step2")
EegFun.channel_repairable!(rejection_info_step2, epochs[1].layout)
@info rejection_info_step2
```

```
# Optional: compare before and after repair
# rejection_comparison = EegFun.compare_rejections(rejection_info_step1,
rejection_info_step2)



################################################################################
# SAVE ARTIFACT INFO
################################################################################

# Collect all artifact information
artifact_info = EegFun.ArtifactInfo(
    !isnothing(continuous_repair_info) ? [continuous_repair_info] :
    EegFun.ContinuousRepairInfo[],
    vcat(rejection_info_step1, rejection_info_step2),
    component_artifacts,
)
# jldsave(joinpath(DEMO_OUTPUT, "artifact_info.jld2"); data = artifact_info)



################################################################################
# LOG EPOCH COUNTS
################################################################################

# Compare original vs. final epoch counts
epoch_count_comparison =
    EegFun.log_epochs_table(epochs_uncorrected, epochs, title = "Epoch counts per
    condition (after repair and rejection):")



################################################################################
# REJECT BAD EPOCHS
################################################################################

@info EegFun.subsection("Rejecting bad epochs")
# Remove remaining bad epochs
epochs_final = EegFun.reject_epochs(epochs, rejection_info_step2)



################################################################################
# SAVE GOOD EPOCH DATA
################################################################################

# jldsave(joinpath(DEMO_OUTPUT, "epochs_final.jld2"); data = epochs_final)
```

```
#########################################################################
# AVERAGE TO ERPs
#########################################################################

erps_final = EegFun.average_epochs(epochs_final)
# jldsave(joinpath(DEMO_OUTPUT, "erps_final.jld2"); data = erps_final)


#########################################################################
# VISUALIZE COMPARISON
#########################################################################

@info EegFun.section("End of Processing")

# Compare original vs. cleaned ERPs
EegFun.plot_erp(erps_uncorrected, channel_selection = EegFun.channels([:Cz, :Pz]))
EegFun.plot_erp(erps_final, channel_selection = EegFun.channels([:Cz, :Pz]))


#########################################################################
# SUMMARY
#########################################################################

# This workflow demonstrates the processing steps from pipeline_v1:
#
# PHASE 1: BASIC SETUP AND INITIAL PREPROCESSING
#   - Load raw data and configure layout
#   - Mark epoch intervals
#   - Rereference (before filtering!)
#   - Apply initial filters (0.1 Hz high, 30 Hz low)
#   - Calculate EOG and detect onsets
#   - Extract "original" epochs for comparison
#
# PHASE 2: ARTIFACT DETECTION AND CLEANING (CONTINUOUS LEVEL)
#   - Channel summary statistics
#   - Extreme value detection (two thresholds: 250 µV and 75 µV)
#   - Channel joint probability and bad channel identification
#   - Run ICA (1 Hz high-pass, exclude bad channels and extreme values)
#   - Repair bad channels (continuous level)
#   - Recalculate EOG after ICA and repair
#   - Final artifact detection
#
# PHASE 3: EPOCH EXTRACTION AND EPOCH-LEVEL PROCESSING
#   - Extract epochs from cleaned continuous data
#   - Baseline correction
```

```
#   - Detect bad epochs (round 1)
#   - Repair channels per epoch
#   - Re-detect bad epochs (round 2)
#   - Save artifact info
#   - Log epoch count comparison
#   - Reject remaining bad epochs
#   - Save good epochs
#   - Average to ERPs
#   - Visualize comparison
#
# For automated batch processing of multiple participants,
# see the preprocess() function and the batch-processing tutorial.
```

- API Reference

Chapter 27

Plot GUI

27.1 Plot GUI

The Plot GUI is a quick-access point for interactively exploring your EEG data. Launch it with a single command and use it to inspect any raw EEG file or saved JLD2 data file — or an entire batch of files at once using filename patterns.

Launching the GUI

```
using EegFun
EegFun.plot_gui()
```

A window opens with three columns allowing file selection, plot selection, and plot setting type controls.

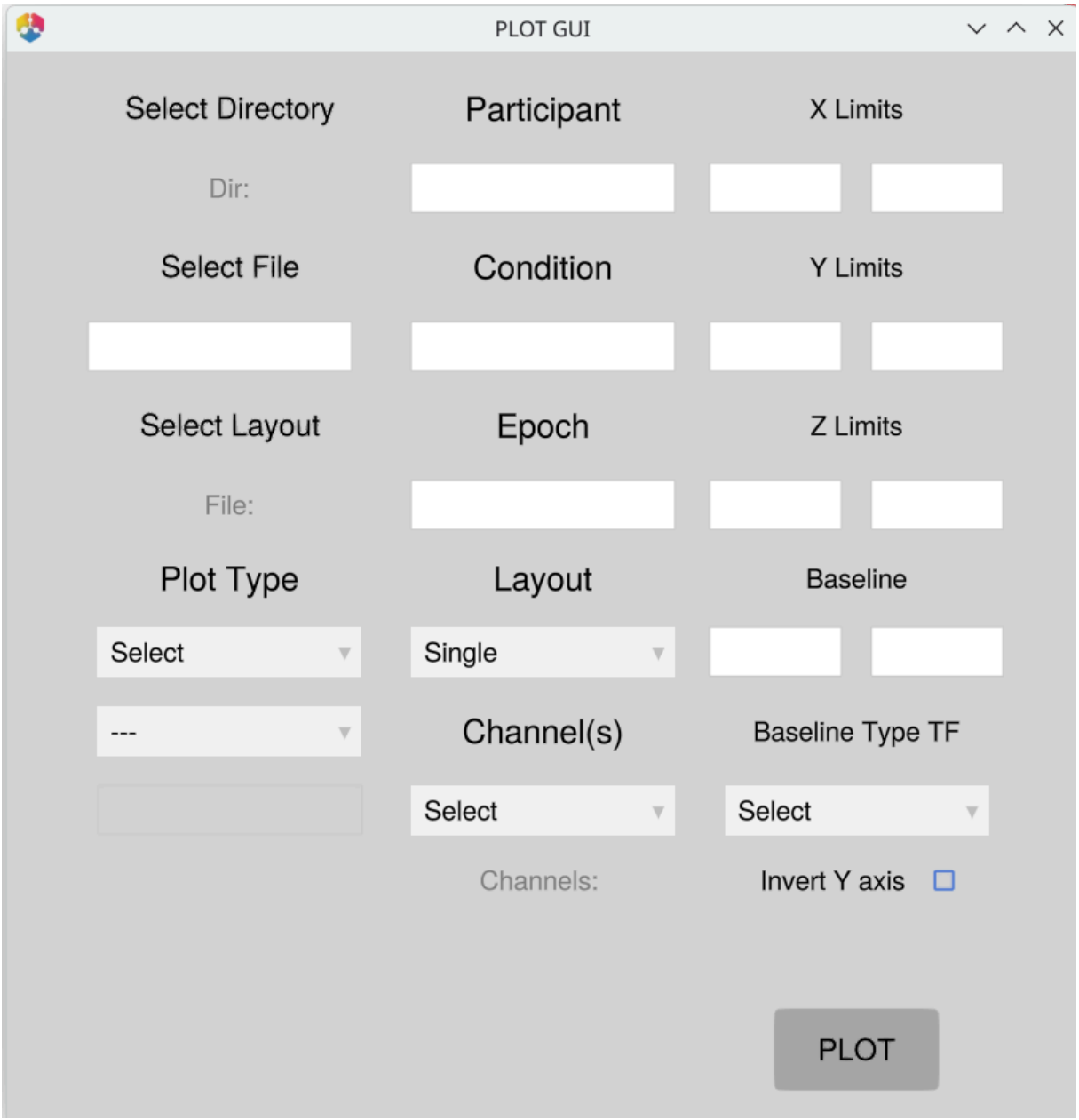

Figure 27.1: Plot GUI overview

GUI Layout

The window is divided into three columns.

Column 1 — File & Plot Selection

Control	Description
Select Directory	Sets the working directory used for batch file discovery
(directory label)	Shows the currently selected directory path

Select File	Opens a file picker to choose a `.jld2` or raw EEG file
(file / pattern box)	Type a filename, full path, or batch pattern directly
Select Layout	Opens a file picker to load an electrode layout (`.csv`)
(layout label)	Shows the currently loaded layout filename
Plot Type	Dropdown — direct plots or category sub-menus (*ICA >*, *ERP Analysis >*, *Diagnostic Plots >*)
(submenu)	Appears when a category is selected; choose the specific plot type
(components box)	Active only for ICA plots — enter component numbers to plot

Column 2 — Data Filters

Control	Description
Participant	Space/comma-separated integers or ranges (e.g. `1 2 3` or `1:10`)
Condition	Condition numbers to include; blank = all
Epoch	Epoch numbers to include; blank = all
Layout	Plot layout: *Single*, *Single Avg*, *Grid*, *Topo*
Channel(s)	Multi-select menu (populated when a layout is loaded)
(selected label)	Shows the currently selected channels

Column 3 — Axis Settings

Control	Description
X Limits	Min / max for the time axis
Y Limits	Min / max for the amplitude axis
Z Limits	Min / max for the colour axis (TF and ERP Image)
Baseline	Start / end times in seconds for baseline correct ion

Baseline Type TF	Baseline normalisation method for Time-Frequency plots
Invert Y axis	Checkbox — flips the Y axis (ERP, Epochs, ERP Image)
PLOT	Triggers the selected plot

Step-by-Step: Plotting a Single File

1. Pick a directory

Click **Select Directory** to set the working directory.

For **batch mode** (pattern input) this is essential — it tells the GUI where to search for matching files. If left unset, the GUI falls back to `pwd()`, which is the directory Julia was launched from and is rarely where your data lives.

For **single-file mode** (Select File picks a full path) the directory is not required.

2. Select a file

Either:

- Click **Select File** and navigate to a `.jld2` or raw EEG file, or
- Type the filename directly into the text box.

The text box accepts full paths, relative paths, or just a filename if the directory is already set.

3. Choose a plot type

Click the **Plot Type** dropdown and select one of the direct plot types:

Selection	Produces
Data Browser	Interactive scrollable signal browser
Epochs	Epoch-level waveform viewer
ERP	ERP waveforms
ERP Image	Heatmap of individual epochs
Topography	Scalp topographic map
Time-Frequency	Time-frequency power plot

For more specialised plots, select a **category** to reveal a submenu:

Category	Submenu options
ICA >	Components (topographies), Activation (time-course)
ERP Analysis >	ERP GUI

Diagnostic Plots >	Channel Summary, Layout View

4. Configure controls (optional)

All controls are optional — leave them blank to use sensible defaults.

X / Y / Z limits — Enter a min and max value in the paired text boxes to constrain the plotted time range (X), amplitude scale (Y), or colour scale (Z). Both boxes must be filled for the limit to apply.

Baseline — Enter start and end times in seconds (e.g. `-0.2` and `0.0`) and choose a correction method from the dropdown.

Channels — If a layout file is loaded, pick one or more channels from the Channels menu. Click **Select** to clear the selection.

Layout — Choose how channels are arranged:

- *Single* — all selected channels overlaid on one axis
- *Grid* — one axis per channel in a grid
- *Topo* — channels positioned according to their electrode locations
- *Single Avg* — channels averaged together before plotting

Invert Y axis — Tick this checkbox to plot positive deflections downward, matching the neurophysiology convention used in some ERP labs. Applies to ERP, Epochs, and ERP Image plots.

5. Click Plot

Click the **Plot** button. The plot opens in a new window.

Batch Mode (Pattern Input)

Instead of a single filename, type a **filename pattern** into the file box. EegFun will discover all matching `.jld2` files in the selected directory and plot them one after another.

```
# Example: type this into the file box
erps_final
```

This matches any file containing `erps_final` in the directory (e.g. `01_erps_final.jld2` , `02_erps_final.jld2` , …).

Filtering by participant

Enter participant numbers in the **Participant** box (space- or comma-separated integers, or ranges):

```
1 2 3         # participants 1, 2, and 3
1:10          # participants 1 through 10
1 3 5:8       # mixed
```

If the box is empty, all matching participants are plotted.

Filtering by condition

Enter condition numbers in the **Condition** box using the same syntax. Leave blank to plot all conditions.

> [!TIP] If you enter a participant ID that does not exist in the discovered files, the GUI will warn you and list the available IDs before plotting.

ICA Plots

Select **ICA** > from the Plot Type dropdown to see two options.

Components — Plots the ICA component topographies. The file must be an `InfoIca` `.jld2` produced by `run_ica()` .

Activation — Plots the ICA component activation time-courses. The GUI automatically locates the original EEG source file from the path stored inside the `InfoIca` object. If the source file is a raw format (`.bdf` , `.set` , `.vhdr`), a 0.1 Hz high-pass filter and average rereference are applied automatically before plotting.

Use the **Components** text box to restrict which components are shown:

```
1:10          # first ten components
1 3 5         # specific components
              # (blank = all components)
```

Layout File

Some plot types (Data Browser with a raw `.bdf` file, Layout View) require an electrode layout. Click **Select Layout** to load a `.csv` layout file. The channel list in the Channels menu is then populated from the layout.

Tips

- **Data Browser** with a `.bdf` file requires a layout to be loaded.
- **Z limits** apply only to Time-Frequency and ERP Image plots (colour axis).
- **Baseline** fields apply to ERP and Time-Frequency plots.
- The GUI state is preserved between plots — change one setting at a time to compare results.
- If a plot fails, a brief warning appears in the Julia REPL. The GUI remains open and usable.

Next Steps

- **Preprocessing your data** — Manual Preprocessing or Batch Processing to produce clean epochs and ERPs to explore in the GUI
- **Layouts** — Layouts and Neighbors for loading the layout file required by topographic and data browser plots
- **Artifact review** — Artifact Handling for the `detect_bad_epochs_interactive` GUI workflow

Chapter 28

Benchmark

28.1 Benchmark

`EegFun.jl` includes a benchmark suite to compare its execution time against other standard EEG toolboxes like **MNE-Python** and **EEGLAB** (MATLAB).

> [!NOTE] The goal of this pipeline is **not** to demonstrate the ideal EEG/ERP processing methodology (it is certainly not!). Rather, it provides a standardized, basic workflow—read data, re-reference, filter, run ICA, extract epochs, and average—to measure realistic performance characteristics across different software platforms.

Downloading the Dataset

The benchmark uses the same raw Visual Attention (Posner Cueing) dataset featured in our other tutorials.

For Interactive Tutorials: You do not need to download the dataset manually. You can seamlessly load any tutorial dataset directly within Julia:

```
using EegFun
data_dir = EegFun.download_eegfun_datasets() # Returns the local path to the downloaded directory
file_path = joinpath(data_dir, "example1.bdf")
dat = EegFun.read_raw_data(file_path)
```

For the Automated Benchmark Scripts: If you are running the cross-platform benchmark scripts (Python, MATLAB, Julia CLI), you will need the physical files on disk. You can download the complete dataset from Zenodo:

Once downloaded, you can extract the files into any folder on your computer (e.g., `~/Downloads/AttentionExp`). The benchmark scripts will accept this directory as a command line argument and save all output files directly into that same folder.

Prerequisites & Installation

To run the full suite, you need to ensure the respective toolboxes are set up on your machine.

Julia (EegFun.jl) All dependencies (Makie, BenchmarkTools, DelimitedFiles) are automatically resolved when you run the scripts using the `--project=.` flag from the repository root.

Python (MNE-Python) We recommend using the lightning-fast `uv` package manager to create an isolated Python virtual environment for the benchmark:

```
# 1. Create a virtual environment
uv venv
```

```
# 2. Activate it (Linux/macOS)
source .venv/bin/activate
# (On Windows: .venv\Scripts\activate)

# 3. Install MNE-Python and Matplotlib
uv pip install mne matplotlib
```

MATLAB (EEGLAB) Ensure you have downloaded EEGLAB and added it to your MATLAB path. The benchmark script relies on standard EEGLAB core functions and requires the `biosig` plugin to load the raw `.bdf` files.

Running the Benchmark Suite

The official benchmark scripts are located in the repository under the `benchmark/` folder.

This script demonstrates a completely realistic, full-scale pipeline from raw data to a Grand Average plot:

1. Loads raw BDF data and custom electrode layouts.
2. Re-references to the average and applies high-pass filtering (automatically distributed across CPU threads).
3. Identifies and flags extreme artifacts.
4. Executes Extended Infomax Independent Component Analysis (ICA).
   - *Note on ICA:* In this automated script, we simply remove the single largest variance component (IC1) without manual review. While IC1 is highly likely to reflect an eye blink, no care is taken to verify this. This pipeline is strictly designed as a **timing benchmark** for computational throughput, and to provide a rough comparison of the resulting visual attention effect across parietal-occipital electrodes.
5. Extracts specific epoch conditions and applies baseline correction.
6. Averages the trials into Event-Related Potentials (ERPs).
7. Computes a Grand Average across multiple subjects.
8. Generates final comparison figures.

Running the Benchmarks

All benchmark scripts (Julia, Python, and MATLAB) require you to pass the absolute path to the downloaded dataset. They also accept optional arguments to control the execution length and whether to run Independent Component Analysis (ICA).

Arguments:
1. `data_dir` *(Required)*: Absolute path to the folder containing the downloaded Zenodo `.bdf` files.
2. `n_files` *(Optional, Default: `0`)*: Number of files to process. `0` means process all files in the directory.
3. `run_ica` *(Optional, Default: `true`)*: Boolean flag to enable or disable the ICA step. Disabling ICA is highly recommended for isolating raw I/O and mathematical filtering throughput.

Julia Execution: We highly recommend launching Julia with multi-threading enabled (`--threads=auto`) to accurately measure the pipeline's concurrent performance characteristics.

```
# Example 1: Process 3 files and disable ICA
julia --threads=auto --project=. benchmark/julia/eegfun_benchmark.jl /path/to/
downloaded/AttentionExp 3 false

# Example 2: Process ALL files (0) and disable ICA
julia --threads=auto --project=. benchmark/julia/eegfun_benchmark.jl /path/to/
downloaded/AttentionExp 0 false
```

Python (MNE) Execution:

```
python benchmark/python/mne_benchmark.py /path/to/downloaded/AttentionExp 3 false
```

MATLAB (EEGLAB) Execution: Because MATLAB handles command-line arguments poorly, you must run the benchmark function directly from your MATLAB command window. Make sure you first add EEGLAB and the benchmark script folder to your MATLAB path!

```
% 1. Add EEGLAB to path and initialize it
addpath('/path/to/eeglab');
eeglab;

% 2. Add the benchmark script folder to path
addpath('/path/to/EegFun.jl/benchmark/matlab');

% 3. Run the benchmark function
eeglab_benchmark('/path/to/downloaded/AttentionExp', 3, false)
```

Plotting the Results

Once you have run the benchmarks for the different toolboxes, they will automatically save their execution times (`.txt`) and exported ERP arrays (`.csv`) directly into your dataset directory.

You can then run the Julia comparison script to generate a single, unified plot that overlays the resulting ERP waveforms and automatically embeds the total execution times directly into the legend!

```
# Generate the cross-pipeline comparison plot
julia --project=. benchmark/plot_benchmark_comparison.jl /path/to/downloaded/
AttentionExp
```

Chapter 29

Worked Examples

29.1 Visual Attention (Posner Cueing)

Example Experiment: Visual Attention (Posner Cueing)

This tutorial uses a complete example dataset to walk through the EegFun.jl analysis pipeline from raw data exploration through to publication-ready plots and statistical analyses.

The Experiment

The task is a variation of a standard **Posner cueing paradigm** investigating the neural correlates of endogenous attention allocation. On each trial, participants fixate a central cross, receive a directional cue indicating the likely location (80% valid) of an upcoming target, then respond (simple target detection) to the target.

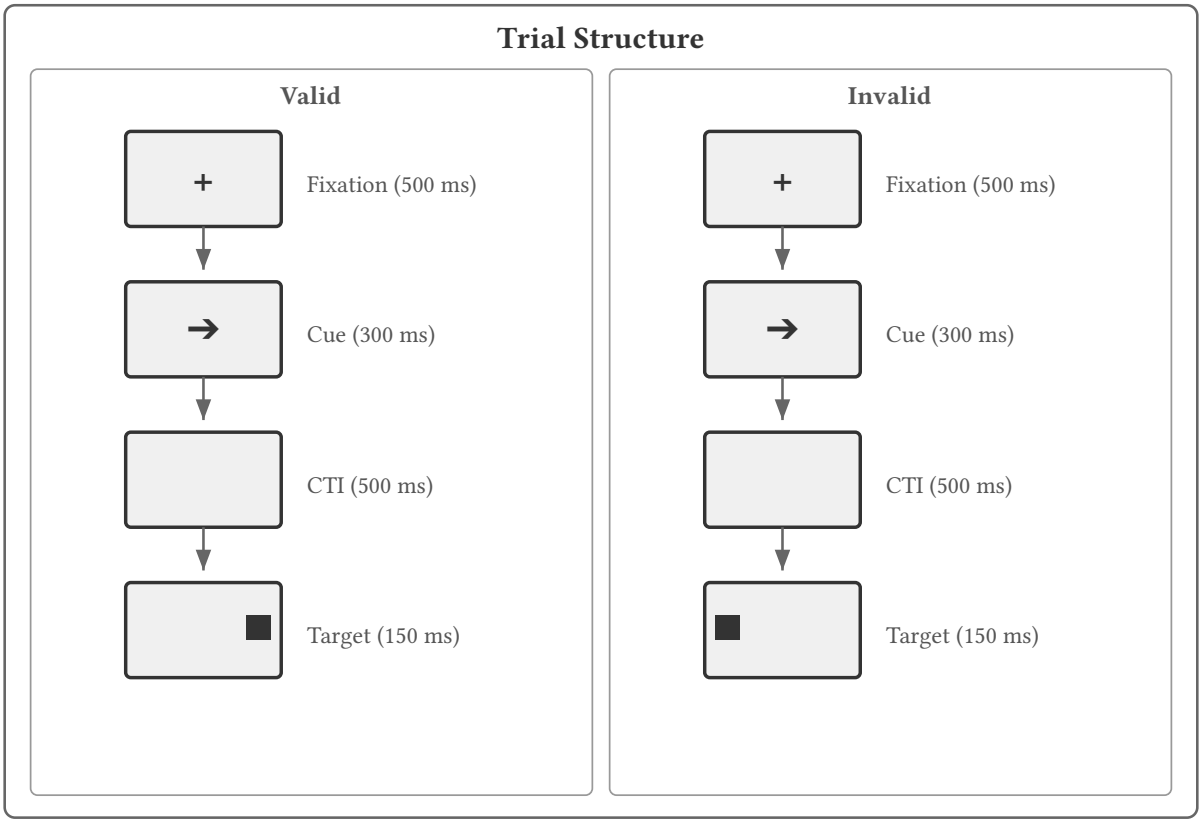

Figure 29.1: Trial Structure

- **Valid trials**: The target appears at the cued location
- **Invalid trials**: The target appears at the uncued location

Experimental Design

The design is a **2 × 2 within-subjects factorial**:

Factor	Levels
Validity	Valid, Invalid
Target Side	Left, Right

Trigger	Condition	Description
1	`valid_left`	Cued left ⟶ Target left
2	`invalid_left`	Cued right ⟶ Target left
3	`valid_right`	Cued right ⟶ Target right
4	`invalid_right`	Cued left ⟶ Target right

Data

Property	Value
Participants	12
EEG System	BioSemi ActiveTwo
Channels	66 Cap EEG + 4 EOG + 2 Mastoids (72 in total)
File Format	BDF (BioSemi Data Format)
Sample Rate	256 Hz
Trials	400 per participant

The data files are available on Zenodo:

Part 1: Single Participant Exploration

We start by working with a single file to explore the data, understand the preprocessing steps, and build intuition before scaling to the full dataset.

1.1 Load Raw Data

```
using EegFun

# Read the raw BDF file and channel layout
dat = EegFun.read_raw_data("example1.bdf")
layout = EegFun.read_layout("biosemi72.csv")

# Create the EegFun data structure
```

```
dat = EegFun.create_eegfun_data(dat, layout)
```

1.2 Channel Layout and Neighbors

Plot the channel layout to check that electrode positions look correct:

```
EegFun.plot_layout_2d(dat)
```

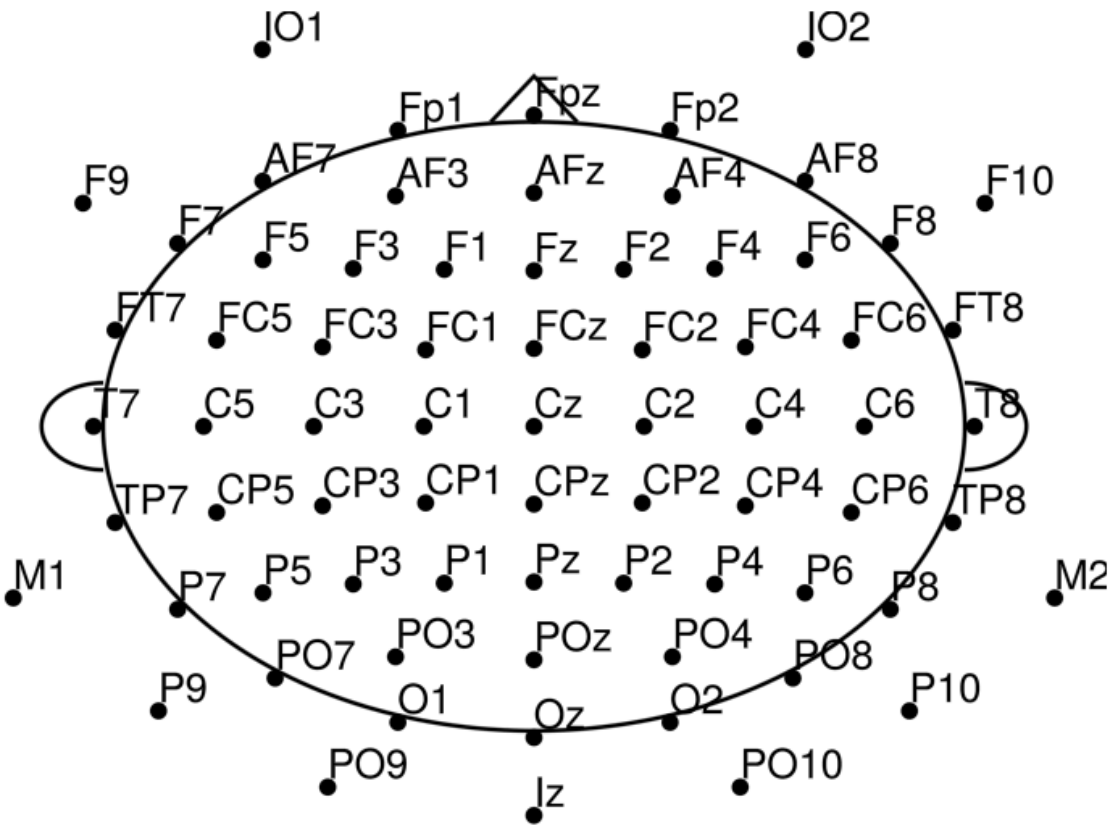

Figure 29.2: Biosemi 72-channel layout

Define channel neighbors (used later potential electrode repair via nearest neighbors, or for cluster based statistics):

```
EegFun.get_neighbors_xy!(dat, 0.35)
```

Plot again with neighbor connections (interactive with mouse hover):

```
EegFun.plot_layout_2d(dat, neighbors = true)
```

1.3 Inspect Triggers

Check which trigger codes are present and how many of each:

```
EegFun.trigger_count(dat)
```

Trigger	Count	Description

1	160	Valid left (cue)
2	40	Invalid left (cue)
3	160	Valid right (cue)
4	40	Invalid right (cue)
5	400	Target onset

Get a visual overview of the trigger distribution:

```
# Trigger overview
EegFun.plot_trigger_overview(dat)
```

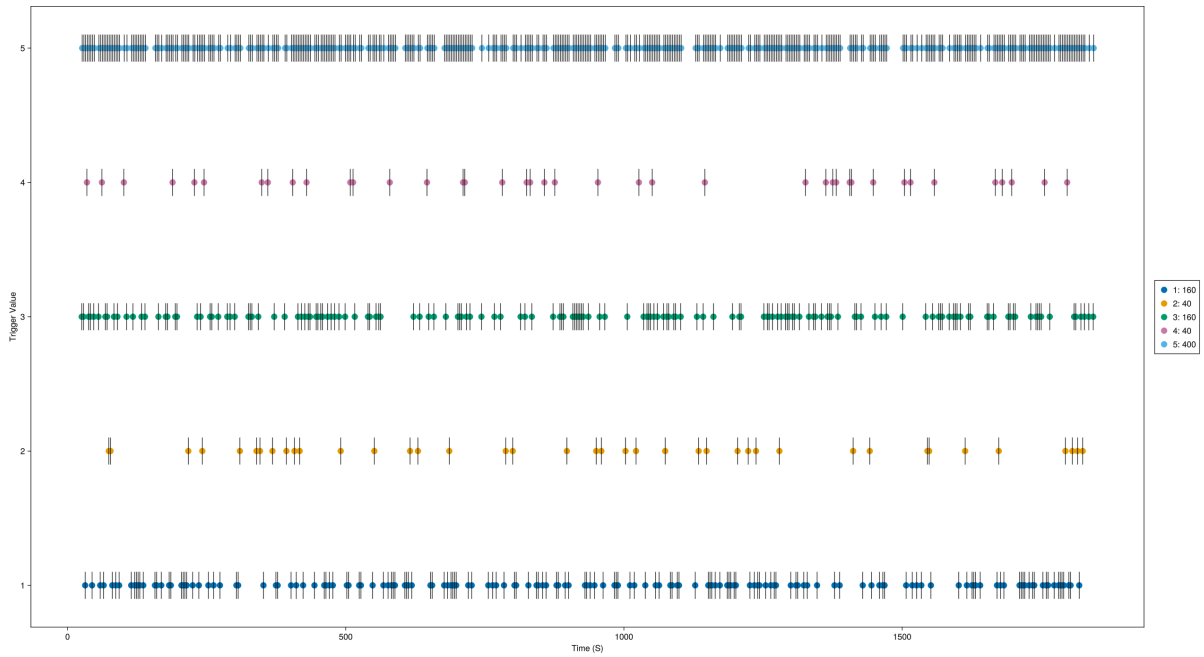

Figure 29.3: Trigger overview

Inspect trigger timing — when each trigger occurred and the interval between triggers:

```
# Trigger timing
EegFun.plot_trigger_timing(dat)
```

You should see triggers 1–4 corresponding to the four cueing conditions, with trigger 5 appearing at 800 ms after each cue trigger.

1.4 Browse the Raw Data

The databrowser lets you scroll through the continuous recording and visually inspect data quality:

```
EegFun.plot_databrowser(dat)
```

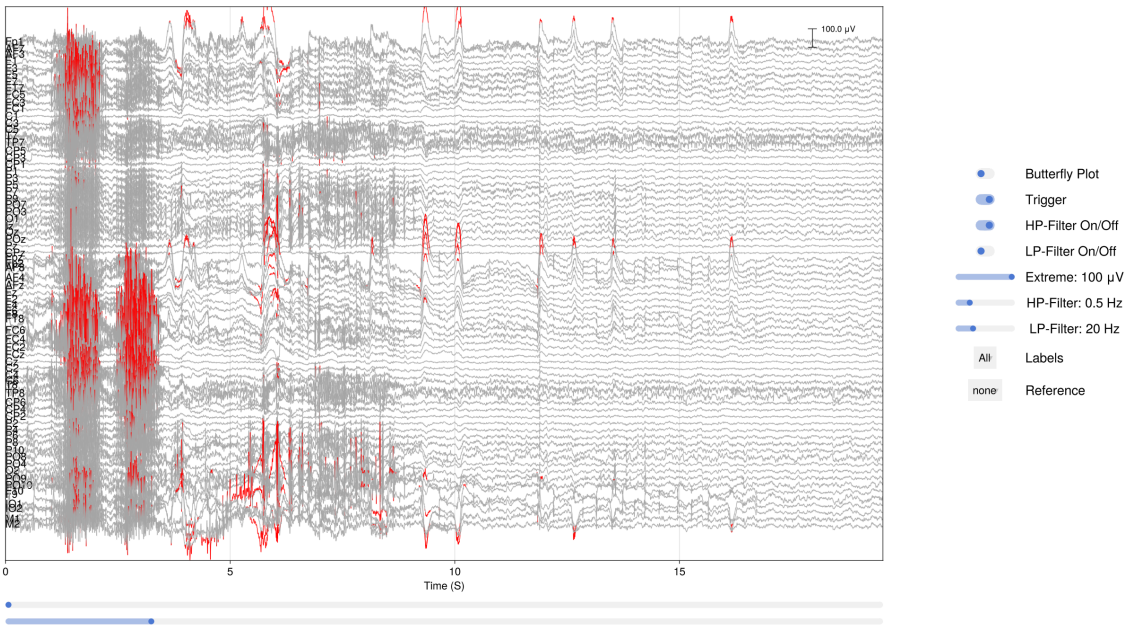

Figure 29.4: Data browser

Before going further, remove the DC offset with a high-pass filter and rereference to the average — this makes the traces much easier to read:

```
# Remove DC offset
EegFun.highpass_filter!(dat, 0.1)

# Rereference to the average of all channels
EegFun.rereference!(dat, :avg)

# Plot again to see the effect
EegFun.plot_databrowser(dat)
```

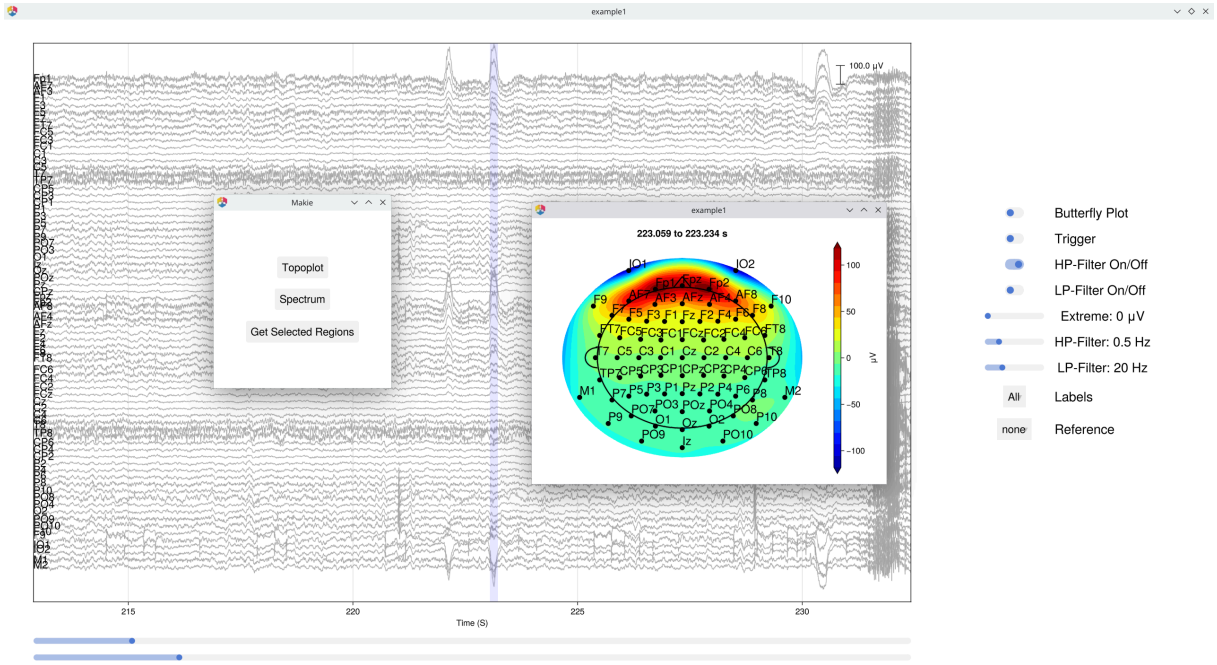

Figure 29.5: Data browser after filtering

> [!TIP] Look for obvious artifacts: flat channels, excessive drift, large muscle movements. This gives you a feel for data quality before automated processing. [!IMPORTANT] Preprocessing is not magic. While some artifacts can be repaired, there is no substitute for collecting clean data. Careful data collection is one of the most important steps in any EEG study.

The databrowser also supports interactive topographic maps and channel spectra — click on any time point to see the scalp topography, or select a channel to view its power spectrum.

1.5 Derive EOG Channels

The raw recording includes four EOG electrodes (IO1, IO2, F9, F10) placed around the eyes. By computing the difference between electrode pairs we get clean vertical and horizontal EOG signals that make blinks and saccades easy to spot:

```
# Vertical EOG: average of upper channels minus average of lower channels
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
)
# Horizontal EOG: left minus right
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
```

```
)
```

Next, automatically detect EOG onsets and mark the surrounding interval as contaminated. This creates Boolean columns (`:is_vEOG` , `:is_hEOG`) that the databrowser draws as vertical markers/ shaded regions:

```
EegFun.detect_eog_onsets!(dat, 50, :vEOG, :is_vEOG)
EegFun.detect_eog_onsets!(dat, 50, :hEOG, :is_hEOG)
```

The new `:vEOG` and `:hEOG` channels appear under the **Extra Channels** tab in the databrowser, so you can scroll through the recording and verify that blinks and saccades are captured correctly.

1.6 Visualise Epoch Regions

Before extracting epochs, it is useful to see where they would land on the continuous recording. `mark_epoch_intervals!` creates a Boolean channel that marks every sample falling within a time window around each trigger:

```
# Mark the epoch window (−1000 ms to 1000 ms) around every target trigger (5)
EegFun.mark_epoch_intervals!(dat, [5], [-0.5, 1.0])

# Browse again — the epoch regions now appear as shaded overlays
EegFun.plot_databrowser(dat)
```

The highlighted regions let you quickly check whether epoch boundaries overlap, whether any epochs contain obvious artifacts, and whether the trigger timing looks correct.

1.7 Channel Quality and Extreme Values

Use `channel_summary` to get a quick overview of each channel's range, variance, and standard deviation — this helps spot noisy or flat channels:

```
# Summary across the entire recording
cs = EegFun.channel_summary(dat)

# Summary restricted to only the epoch regions we just marked
cs_epoch = EegFun.channel_summary(dat,
    sample_selection = EegFun.samples(:epoch_interval)
)

# Visualise with a bar chart
EegFun.plot_channel_summary(cs, :range)
EegFun.plot_channel_summary(cs, [:range, :min, :max, :var])
EegFun.plot_channel_summary(cs_epoch, :range)
```

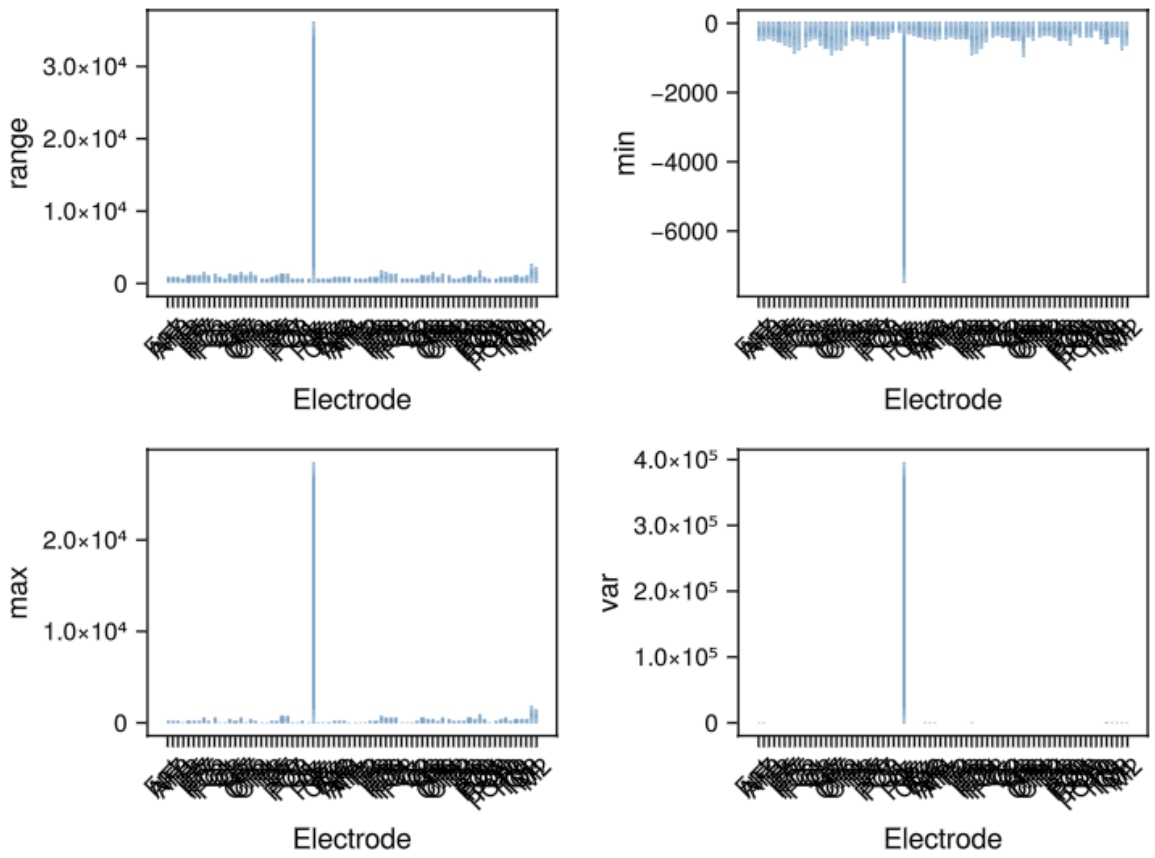

Figure 29.6: Channel summary (range)

Next, flag every sample where **any** channel exceeds ±200 µV. This creates a Boolean column (`:is_extreme_value_200`) that the databrowser renders as shaded overlays:

```
# Mark extreme samples
EegFun.is_extreme_value!(dat, 200)

# Browse — extreme-value regions now appear as red highlights
EegFun.plot_databrowser(dat)
```

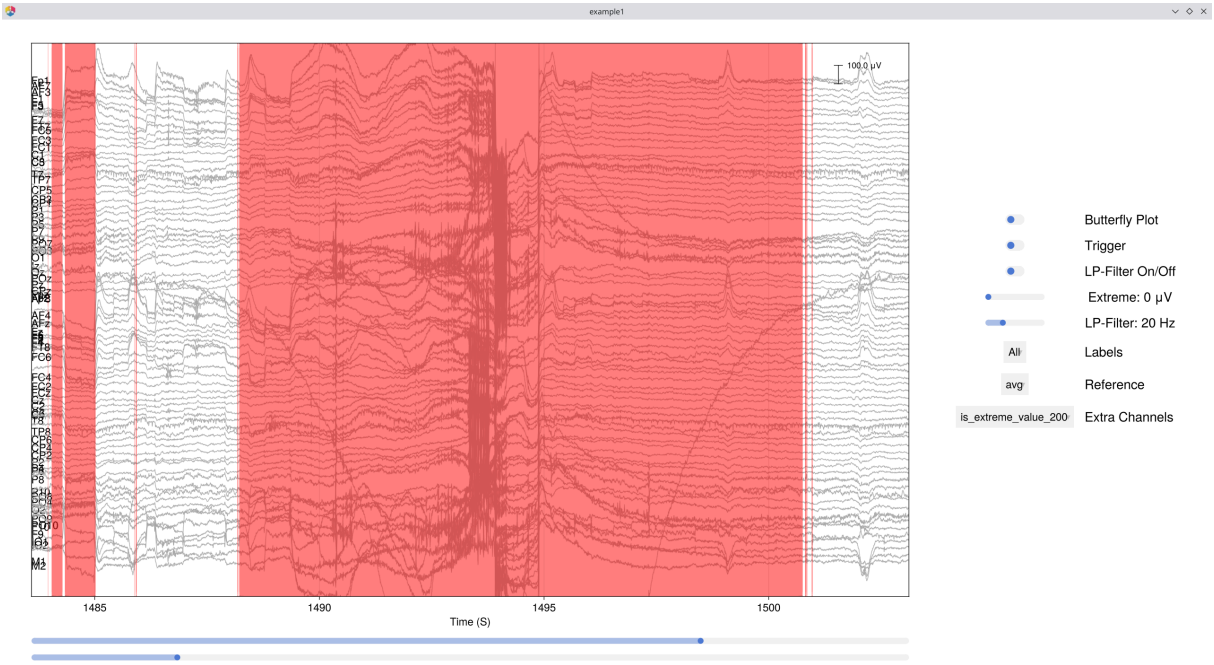

Figure 29.7: Extreme values in the databrowser

1.8 Detect and Repair Bad Channels

Combine the channel summary and joint probability metrics to automatically flag potential bad channels:

```
# Channel summary and joint probability across the whole recording
cs = EegFun.channel_summary(dat)
jp = EegFun.channel_joint_probability(dat)

# Or restrict to just the epoch regions — often more relevant
cs = EegFun.channel_summary(dat,
    sample_selection = EegFun.samples(:epoch_interval)
)
jp = EegFun.channel_joint_probability(dat,
    sample_selection = EegFun.samples(:epoch_interval)
)

# Identify bad channels (high z-variance or high joint probability)
bad_channels = EegFun.identify_bad_channels(cs, jp)
```

If any channels are flagged, you can repair them interactively in the databrowser. Press **R** to open the channel repair menu, which lets you select the repair method (neighbor interpolation or spherical spline) and apply it immediately:

```
EegFun.plot_databrowser(dat)
# → Press R to open the repair menu
```

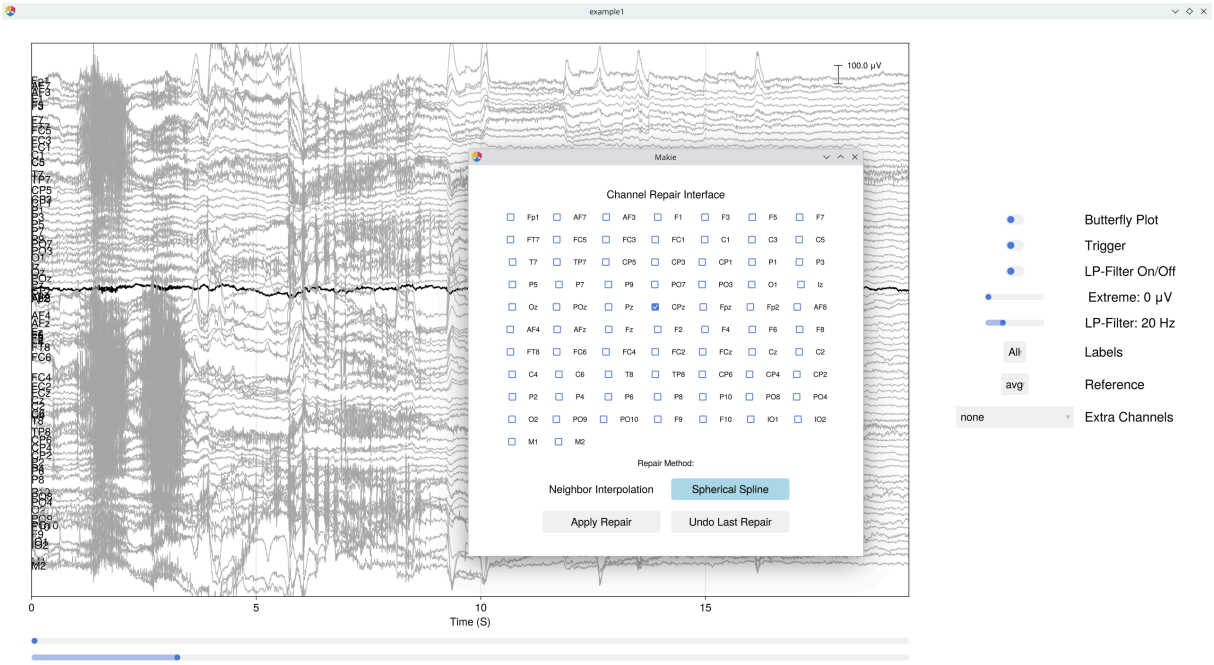

Figure 29.8: Channel repair interface

1.9 ICA

First, mark extreme values on the main data — these will be excluded from ICA training and component identification:

```
# Mark extreme values on the original data (used for ICA exclusion)
EegFun.is_extreme_value!(dat, 250)
```

ICA works best on data that has been high-pass filtered (e.g., ≥ 1 Hz). We copy the data, apply a stricter filter, and exclude extreme samples so the decomposition focuses on genuine brain and artifact sources:

```
# Copy data and apply stricter filter for ICA
dat_ica = copy(dat)
EegFun.highpass_filter!(dat_ica, 1.0)

# Run ICA, excluding extreme samples
ica_result = EegFun.run_ica(dat_ica,
    sample_selection = EegFun.samples_not(:is_extreme_value_250)
)

# For long recordings, subsample can be used to speed up ICA:
# ica_result = EegFun.run_ica(dat_ica,
#     sample_selection = EegFun.samples_not(:is_extreme_value_250),
#     percentage_of_data = 25.0
# )
```

Inspect the component topographies and time-course activations:

```
# Component topographies
```

```
EegFun.plot_topography(ica_result, component_selection = EegFun.components(1:10));
EegFun.plot_ica_component_activation(dat, ica_result)
```

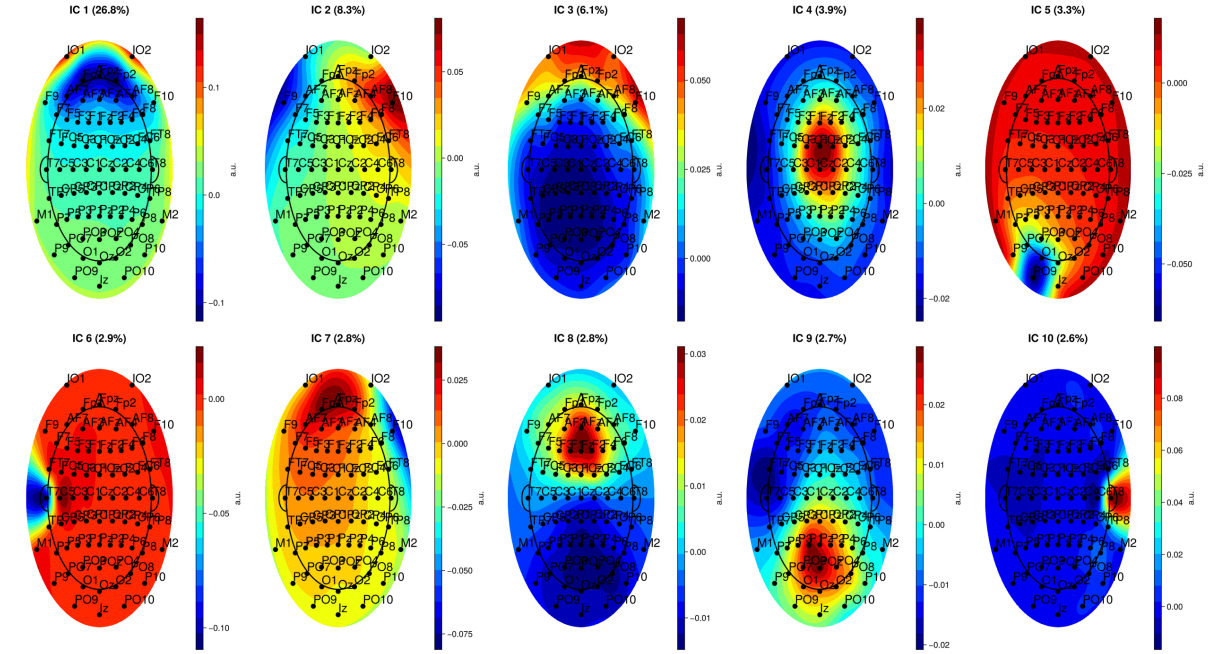

Figure 29.9: ICA component topographies

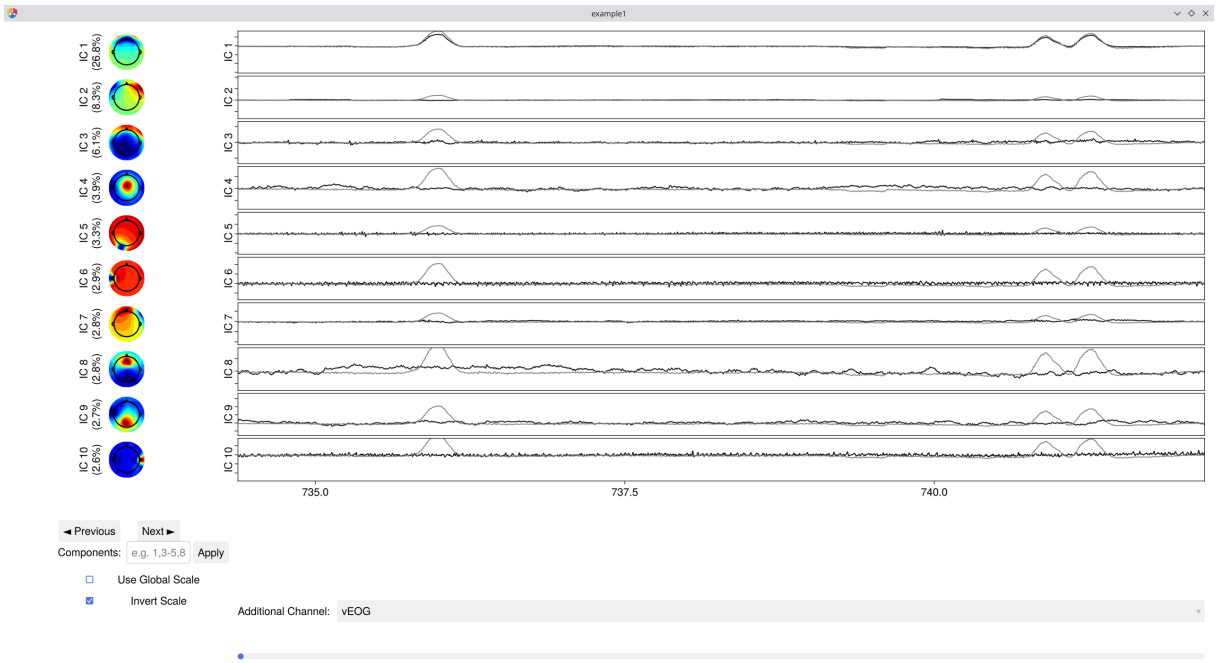

Figure 29.10: ICA component activation

Components are ordered by the percentage of total variance they explain — the first few typically capture the most prominent signals or artifacts. In most EEG recordings, the largest components are **vertical EOG activity** (eye blinks, showing a frontal topography) and **horizontal eye movements** (lateral frontal pattern). Standard ERP analyses use ICA primarily to remove these ocular components so that the cleaned data better reflects neural activity.

You can immediately browse the data with ICA components overlaid — this lets you see each component's contribution to the signal and toggle removal interactively:

```
EegFun.plot_databrowser(dat, ica_result)
```

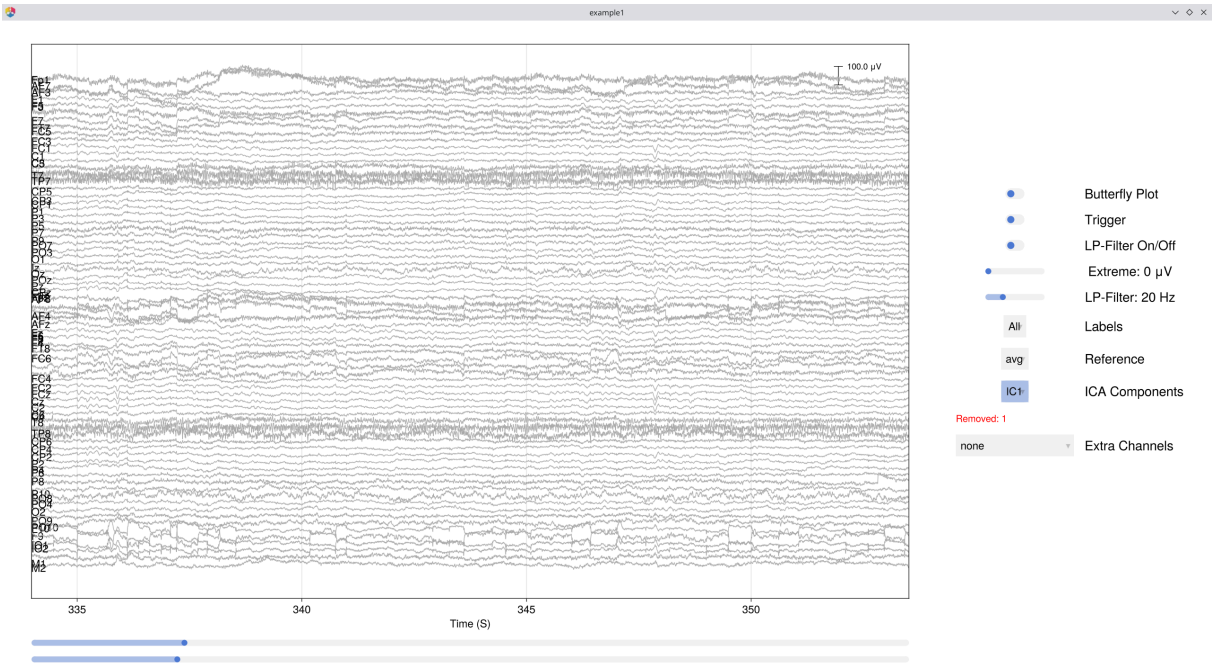

Figure 29.11: Databrowser with ICA components

Automatically identify artifact components via EOG correlation and spatial kurtosis:

```
# Identify artifact components (excluding extreme samples)
component_artifacts, component_metrics = EegFun.identify_components(dat,
ica_result,
    sample_selection = EegFun.samples_not(:is_extreme_value_250)
)

# Inspect with artifact labels overlaid (can be used to easily select
automatically identified components)
EegFun.plot_ica_component_activation(dat, ica_result, artifacts =
component_artifacts)
```

Remove the identified components and verify the cleaned data in the databrowser:

```
# Remove artifact components from the original (0.1 Hz filtered) data
all_removed = EegFun.get_all_ica_components(component_artifacts)
EegFun.subtract_ica_components!(dat, ica_result,
    component_selection = EegFun.components(all_removed)
)
```

1.10 Define Epoch Conditions

Define the epoch conditions directly in Julia. Each condition maps a cue trigger to a descriptive label — we will epoch around trigger 5 (target onset = 0), with the cue identity telling us which condition each trial belongs to:

```
epoch_conditions = [
    EegFun.EpochCondition(name = "valid_left",    trigger_sequences = [[1, 5]],
    reference_index = 2),
    EegFun.EpochCondition(name = "invalid_left",  trigger_sequences = [[2, 5]],
    reference_index = 2),
    EegFun.EpochCondition(name = "valid_right",   trigger_sequences = [[3, 5]],
    reference_index = 2),
    EegFun.EpochCondition(name = "invalid_right", trigger_sequences = [[4, 5]],
    reference_index = 2),
]
```

Each `EpochCondition` specifies:

- **`name`** — a descriptive label for the condition
- **`trigger_sequences`** — the trigger pattern to match (here cue → target)
- **`reference_index`** — which trigger in the sequence is t = 0 (2 = the target onset)

> [!TIP] For batch processing, you can define these same conditions in a TOML file and load them with `EegFun.condition_parse_epoch("epochs.toml")` — we'll cover this in Part 2.

1.11 Extract Epochs

```
# Extract epochs: 500 ms pre-target to 1000 ms post-target
epochs = EegFun.extract_epochs(dat, epoch_conditions, (-0.5, 1.0))

# Baseline correction (200 ms pre-target)
EegFun.baseline!(epochs, (-0.2, 0.0))
```

Before running artifact rejection, inspect the single-trial data. `plot_epochs` offers several layouts — a single channel overlay, a grid of all channels, or a topographic arrangement:

```
# Example plots with epoched data
# Single-trial waveforms at one channel
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(1),
channel_selection = EegFun.channels(:PO7))

# All channels in a grid
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(1), layout
= :grid)

# Topographic layout
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(2), layout
= :topo)
```

1.12 Reject Artifacts

Two-stage artifact rejection — first repair channels that can be repaired (interpolated), then reject remaining bad epochs:

```
rejection_info = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion =
100.0, z_criterion = 0.0)

# Inspect which epochs and channels were flagged
EegFun.plot_artifact_detection(epochs[1], rejection_info[1]) # Condition 1
EegFun.plot_artifact_detection(epochs[2], rejection_info[2]) # Condition 2
```

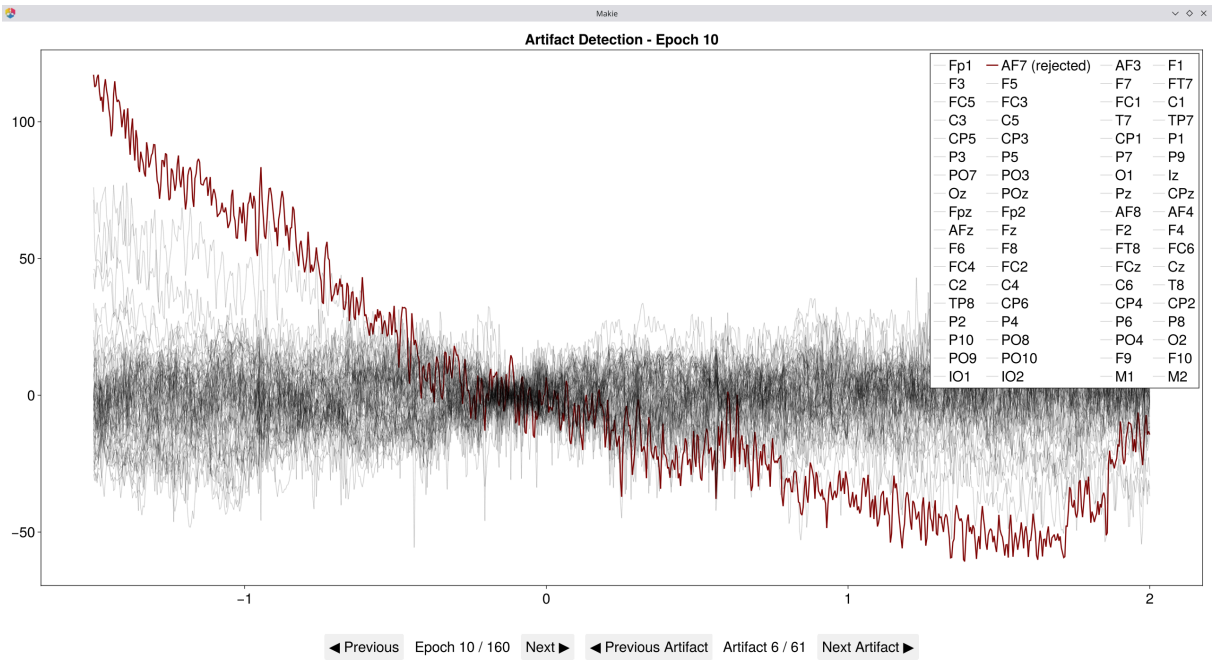

Figure 29.12: Artifact detection

```
# Repair channels that can be interpolated
EegFun.channel_repairable!(rejection_info, epochs[1].layout)
EegFun.repair_artifacts!(epochs, rejection_info)

# Second pass on repaired data
rejection_info2 = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion =
100.0, z_criterion = 0.0)

# Visually review epochs in a grid — automatically detected artifacts are pre-
checked
EegFun.detect_bad_epochs_interactive(epochs[1], artifact_info =
rejection_info2[1], dims = (3, 3))
```

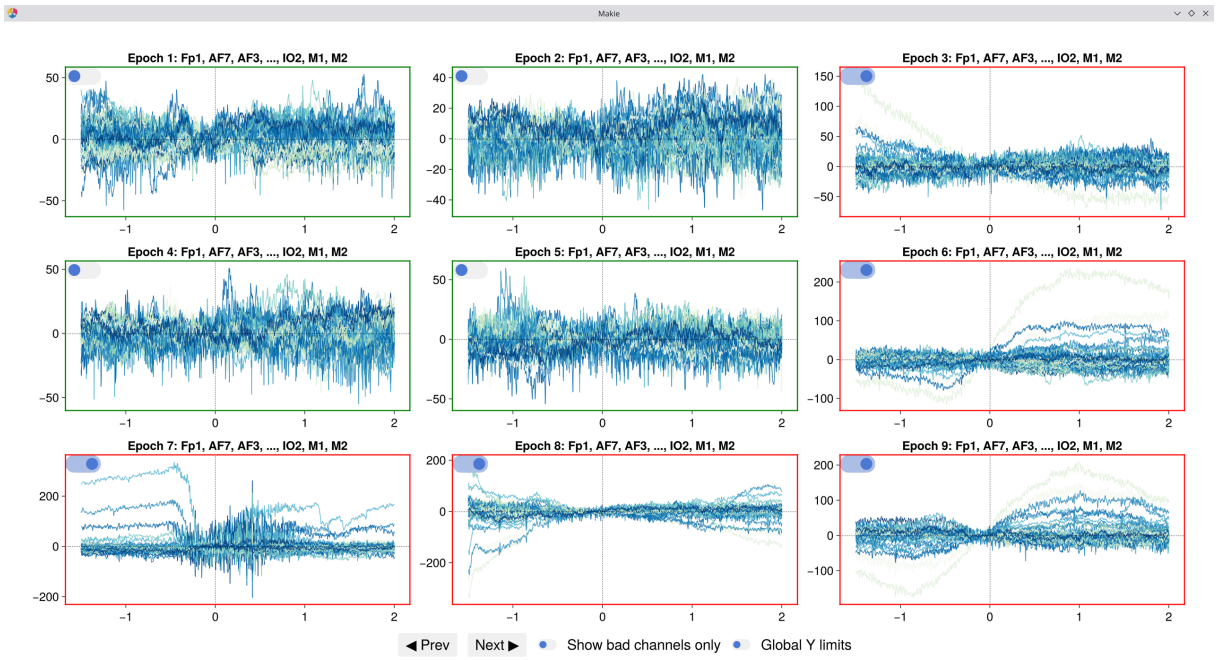

Figure 29.13: Interactive epoch rejection grid

```
# Accept the rejection and keep only good epochs
epochs_final = EegFun.reject_epochs(epochs, rejection_info2)
```

1.13 Average and Plot Single-Participant ERPs

```
erps = EegFun.average_epochs(epochs_final)

# Plot all conditions at posterior sites
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:PO7, :PO8, :O1, :O2]))

# Topography at the P1 time window (peak posterior negativity)
EegFun.plot_topography(erps, interval_selection = EegFun.times(0.10))
```

Part 2: Batch Processing and Group Analysis

File Organisation

```
AttentionExp/
├── example1.bdf ... example16.bdf    # Raw BDF files (one per participant)
├── biosemi72.csv                     # Channel layout (label, inclination,
azimuth)
├── epochs.toml                       # Epoch condition definitions
├── pipeline.toml                     # Preprocessing pipeline configuration
└── output_data/                      # Preprocessed output (after batch
processing)
```

The walk-through in Part 1 gave you the general idea of a single-participant analysis pipeline — loading data, filtering, ICA, epoch extraction, artifact rejection, and ERP averaging. In practice, we want to run these steps automatically across all participants so that every file receives the same analysis in a consistent, reproducible way. Visual inspection of the raw data and of the individual-level outputs remains important, but the automated pipeline ensures a uniform setup that is easy to document and log.

> [!NOTE] The pipeline below uses `preprocess`, which provides sensible defaults for a standard ERP analysis. If you need a different set of steps or parameters, you can generate a fully customisable pipeline scaffold with `EegFun.generate_pipeline_template("my_pipeline.jl", "my_preprocess")` and a matching configuration file with `EegFun.generate_config_template(filename = "my_config.toml")`. The template provides all the boilerplate — logging, configuration loading, file iteration, and error handling — so you can focus on combining the lower-level functions from Part 1 (filtering, ICA, epoching, artifact rejection, etc.) into your own processing sequence.

2.1 Batch Preprocessing

The `preprocess` pipeline automates all single-participant steps across every BDF file:

```
EegFun.preprocess("pipeline.toml")
```

For each participant, the pipeline runs through the following stages:

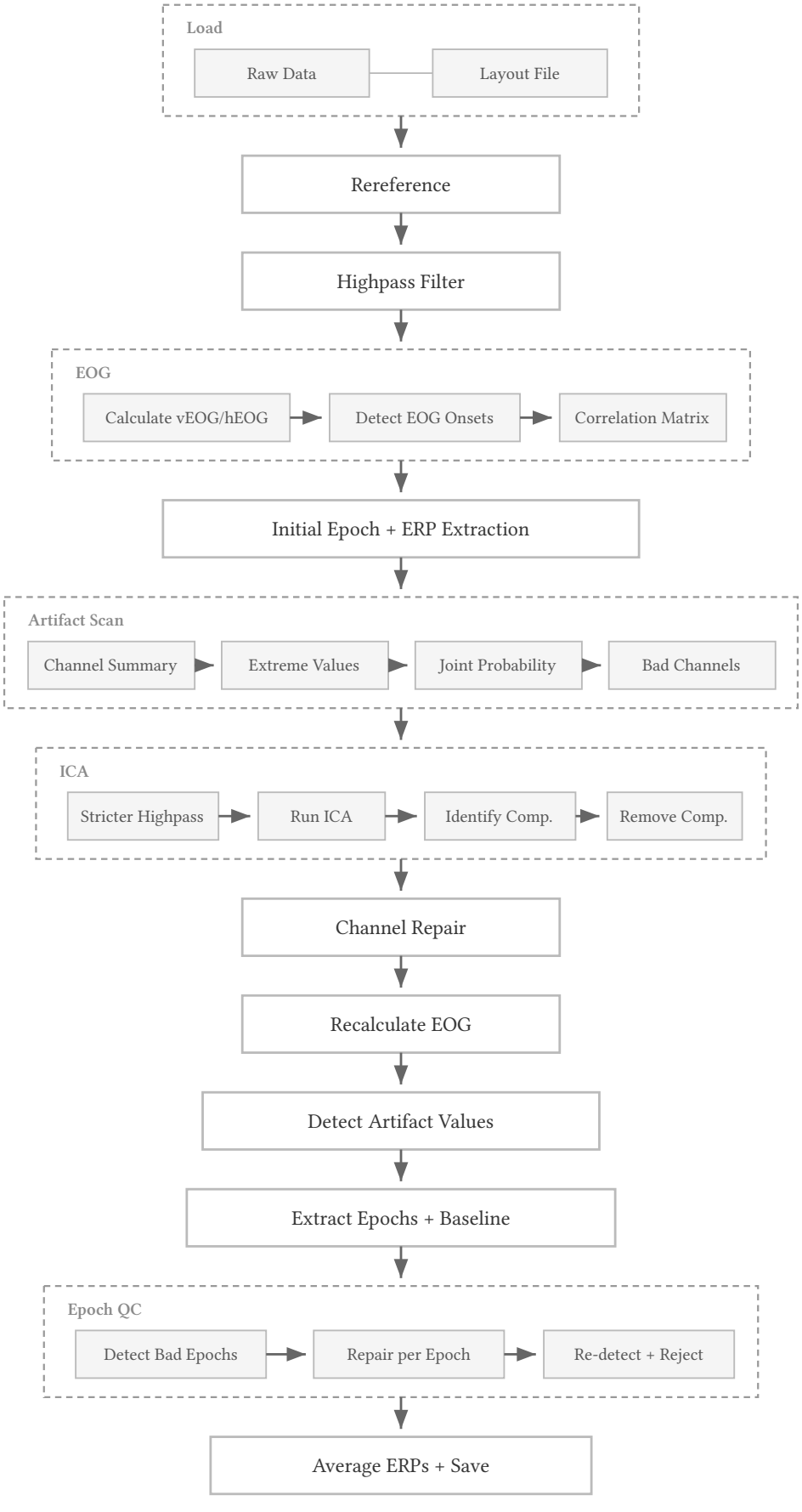

Figure 29.14: Pipeline flowchart

↻ *Repeated for each participant file*

#	Stage	Description
1	**Load**	Read raw BDF file and apply electrode layout
2	**Rereference**	Apply the chosen reference (e.g., average, mastoid)
3	**Filter**	High-pass filter the continuou s data (e.g., 0.1 Hz)
4	**EOG**	Calculate vEOG / hEOG, de- tect onsets, compute chan- nel–EOG correlations
5	**Initial Epochs**	Extract preliminary epochs and ERPs (before cleaning)
6	**Artifact Scan**	Channel summary, extreme- value detection, joint probabili ty, bad-channel identification
7	**ICA**	Apply stricter high-pass filter (on copy of data), decompose, auto-identify artifact compone nts (e.g., EOG, ECG), remove them
8	**Channel Repair**	Interpolate bad channels (pre viously removed) from their neighbors (or spline interpolat ion)
9	**Recalculate EOG**	Recompute vEOG / hEOG af- ter ICA and channel repair
10	**Artifact Values**	Re-detect extreme values on cleaned continuous data (used for epoch exclusion)
11	**Epoch + Baseline**	Cut cleaned continuous data into trial segments and base- line-correct
12	**Epoch QC**	Detect bad epochs, attempt per-epoch channel repair, re- detect and reject remaining artifacts

13	**Average & Save**	Compute ERPs per condition and save all outputs to `output_data/`

Key Default Parameters

The most commonly adjusted parameters are shown below. The full configuration (with all available options) can be generated with `EegFun.generate_config_template(filename = "my_config.toml")`.

Section	Parameter	Default	Description
Epochs	`epoch_start`	−1 s	Epoch start relative to trigger
	`epoch_end`	1 s	Epoch end relative to trigger
Reference	`reference_channel`	`avg`	Reference type (avg, mastoid, etc.)
Highpass	`filter.highpass.freq`	0.1 Hz	Main highpass cutoff
Lowpass	`filter.lowpass.apply`	`false`	Lowpass off by default
	`filter.lowpass.freq`	30 Hz	Lowpass cutoff (if enabled)
ICA	`ica.apply`	`true`	ICA on by default
	`filter.ica_highpass.freq`	1.0 Hz	Stricter highpass for ICA
	`ica.percentage_of_data`	100%	Proportion of data used for ICA
Artifacts	`eeg.extreme_value_abs_criterion`	500 µV	Extreme value threshold (continuous)
	`eeg.artifact_value_abs_criterion`	100 µV	Artifact threshold (epoch rejection)
EOG	`eog.vEOG_channels`	Fp1/Fp2 – IO1/IO2	Vertical EOG electrode pairs
	`eog.hEOG_channels`	F9 – F10	Horizontal EOG electrode pairs
Layout	`layout.neighbor_criterion`	0.35	Distance for neighbor definition

```toml
# ——— Files ———

[files.input]
directory = "."
epoch_condition_file = ""
layout_file = "biosemi72.csv"
raw_data_files = "\\.bdf"

[files.output]
directory = "./preprocessed_files"
save_continuous_data_raw = true
save_continuous_data_corrected = true
save_epoch_data_uncorrected = true
save_epoch_data_unrejected = true
save_epoch_data_final = true
save_erp_data_uncorrected = true
save_erp_data_unrejected = true
save_erp_data_final = true
save_ica_data = true

# ——— Preprocessing ———

[preprocess]
epoch_start = -1
epoch_end = 1
reference_channel = "avg"

[preprocess.eeg]
extreme_value_abs_criterion = 500    # µV — flags extreme samples
artifact_value_abs_criterion = 100   # µV — epoch rejection threshold
artifact_value_z_criterion = 0       # z-score (0 = off)

[preprocess.eog]
vEOG_channels = [["Fp1", "Fp2"], ["IO1", "IO2"], ["vEOG"]]
vEOG_criterion = 50
hEOG_channels = [["F9"], ["F10"], ["hEOG"]]
hEOG_criterion = 30

# ——— Filters ———

[preprocess.filter.highpass]
apply = true
freq = 0.1
order = 1
method = "iir"
```

```
func = "filtfilt"
type = "hp"

[preprocess.filter.lowpass]
apply = false
freq = 30.0
order = 3
method = "iir"
func = "filtfilt"
type = "lp"

[preprocess.filter.ica_highpass]
apply = true
freq = 1.0
order = 1
method = "iir"
func = "filtfilt"
type = "hp"

[preprocess.filter.ica_lowpass]
apply = false
freq = 30.0
order = 3
method = "iir"
func = "filtfilt"
type = "lp"

# ——— ICA ———

[preprocess.ica]
apply = true
percentage_of_data = 100.0

# ——— Layout ———

[preprocess.layout]
neighbor_criterion = 0.35
```

Apply a low-pass filter to the saved ERP files (e.g., 30 Hz for clean plotting):

```
EegFun.lowpass_filter("erps_final", 30)
```

The interactive filter GUI lets you preview the effect of different filter settings before applying them in batch:

```
EegFun.plot_erp_filter_gui("example1_erps_final.jld2")
```

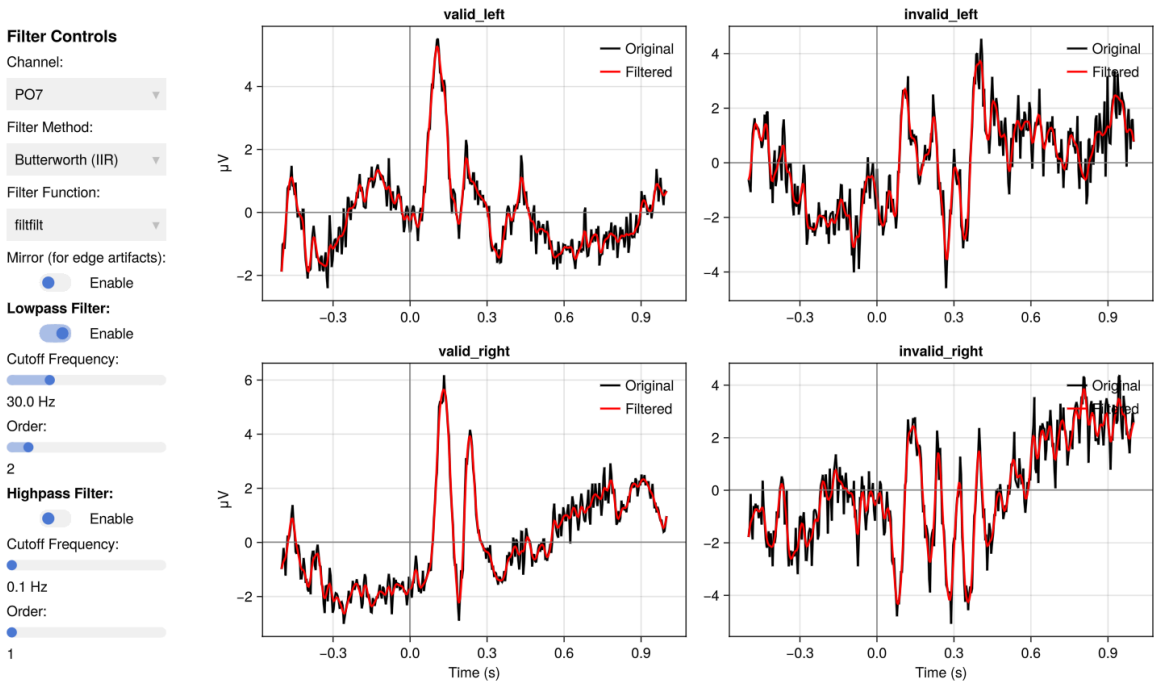

Figure 29.15: Filter GUI

This creates a new directory `filtered_erps_final_lp_30hz/` with filtered copies of each participant's ERPs.

> [!TIP] Batch functions like `lowpass_filter` , `condition_average` , `condition_difference` , and `grand_average` automatically create an output directory named after the operation and its parameters (e.g., `filtered_erps_final_lp_30hz/` , `averages_erps_final_1-3_2-4/`). To save to a different location/folder name, pass the `output_dir` keyword argument.

2.3 Grand Average

Compute the grand average across all participants:

```
EegFun.grand_average("erps_final")


# Plotting the grandaverage across parietal/occipital electrodes
EegFun.plot_erp("grand_average_erps_final.jld2", channel_selection =
EegFun.channels([:PO7, :PO8]))

# but we want to baseline the data
EegFun.plot_erp("grand_average_erps_final.jld2", channel_selection =
EegFun.channels([:PO7, :PO8]), baseline_interval = (-0.2, 0))

# and lets make separate plot for each channel
EegFun.plot_erp("grand_average_erps_final.jld2", channel_selection =
EegFun.channels([:PO7, :PO8]), baseline_interval = (-0.2, 0), layout = :grid,
layout_kwargs = (; grid_dims = (1,2)))
```

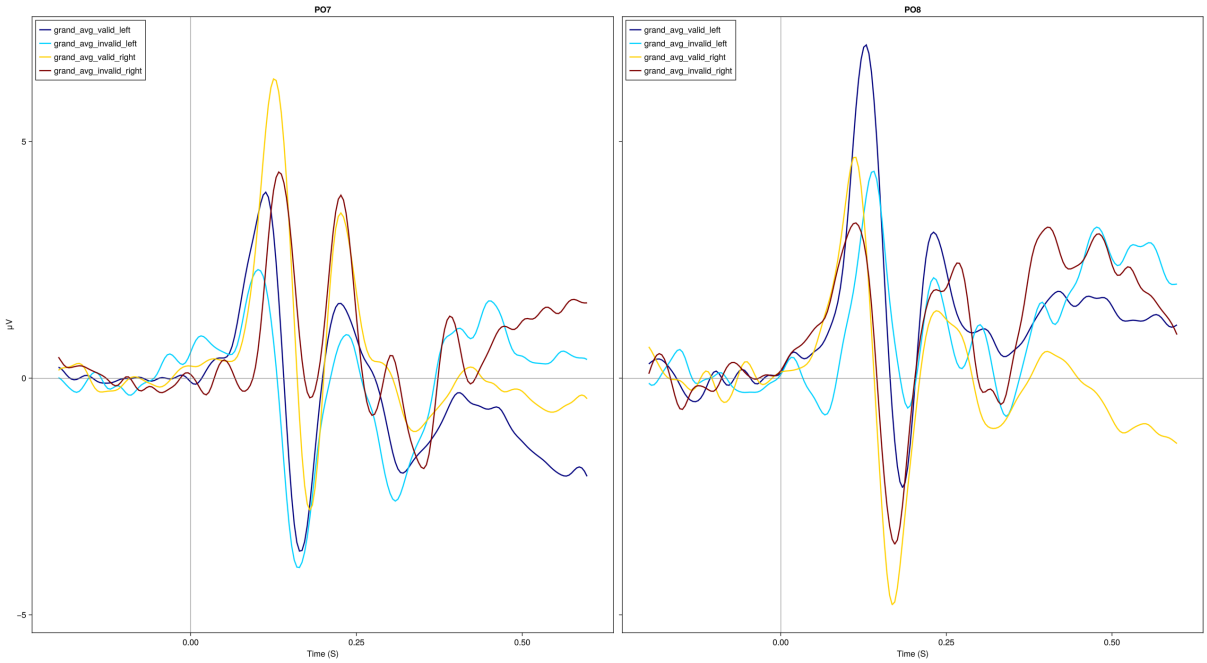

Figure 29.16: Grand average ERP waveforms at PO7 and PO8

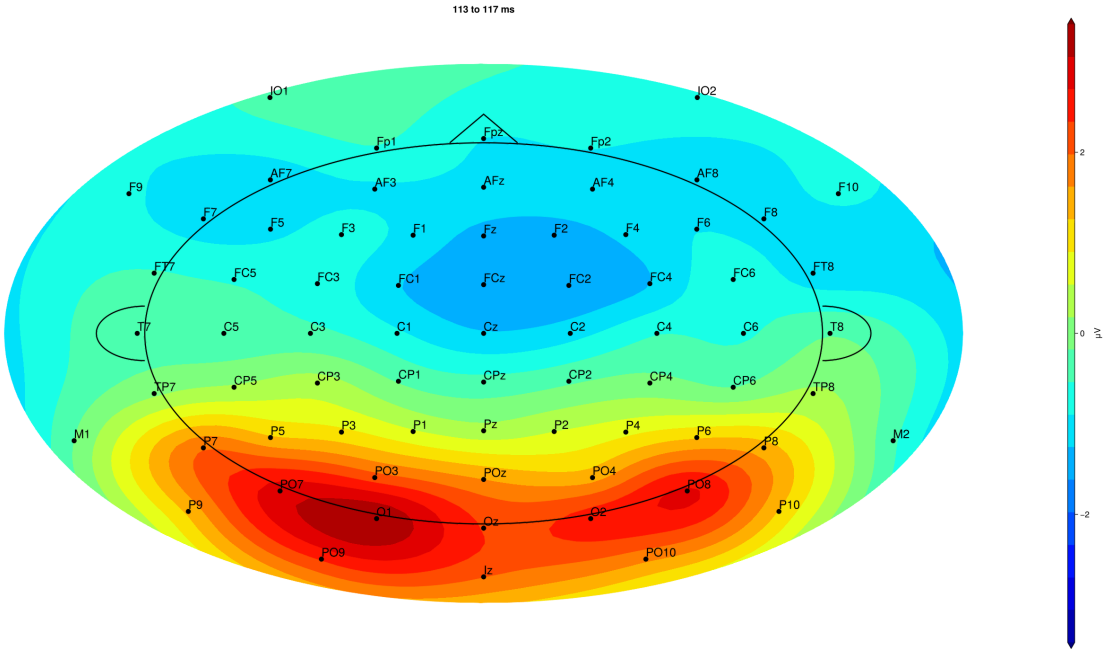

Figure 29.17: P1 topography (113–117 ms)

2.4 Combining Conditions

Sometimes we want to average ERP comditions. For example, we might just want to look at a basic valid vs. invalid comparison.

```
# Average conditions: [1,3] = Valid (left+right), [2,4] = Invalid (left+right)
# We will do this here on the individual participant ERP files, then create a
grand average.
EegFun.condition_average("erps_final", [[1, 3], [2, 4]])
```

```
# Plotting the grandaverage across parietal/occipital electrodes
EegFun.grand_average("erps_final")

# and lets make plot averaging across channels
EegFun.plot_erp("grand_average_erps_final.jld2", channel_selection =
EegFun.channels([:PO7, :PO8]), baseline_interval = (-0.2, 0), average_channels =
true)

# topography plot for P1
EegFun.plot_topography("grand_average_erps_final.jld2", baseline_interval = (-0.2,
0), interval_selection = (0.1, 0.1), dims = (1,2), colorrange = (-3, 3),
label_plot = false)
```

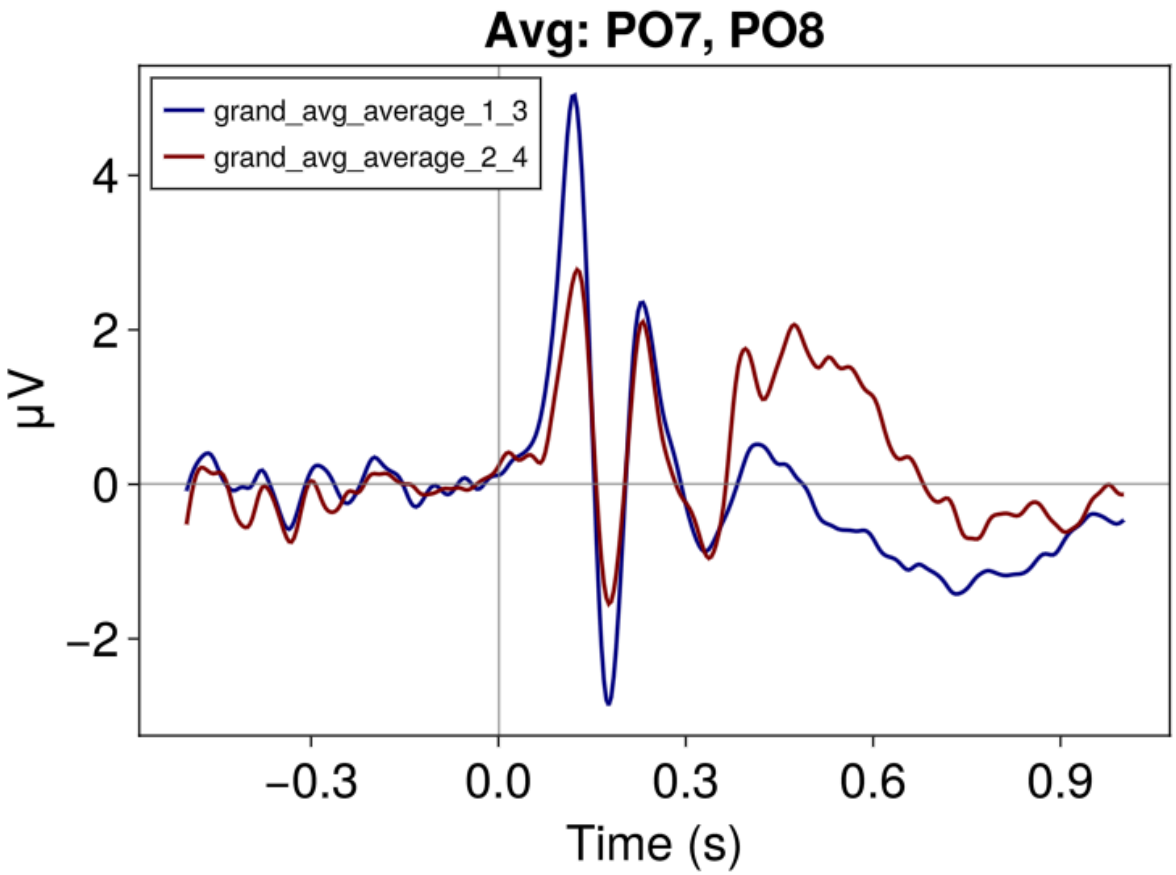

Figure 29.18: Condition-averaged ERP (Valid vs Invalid)

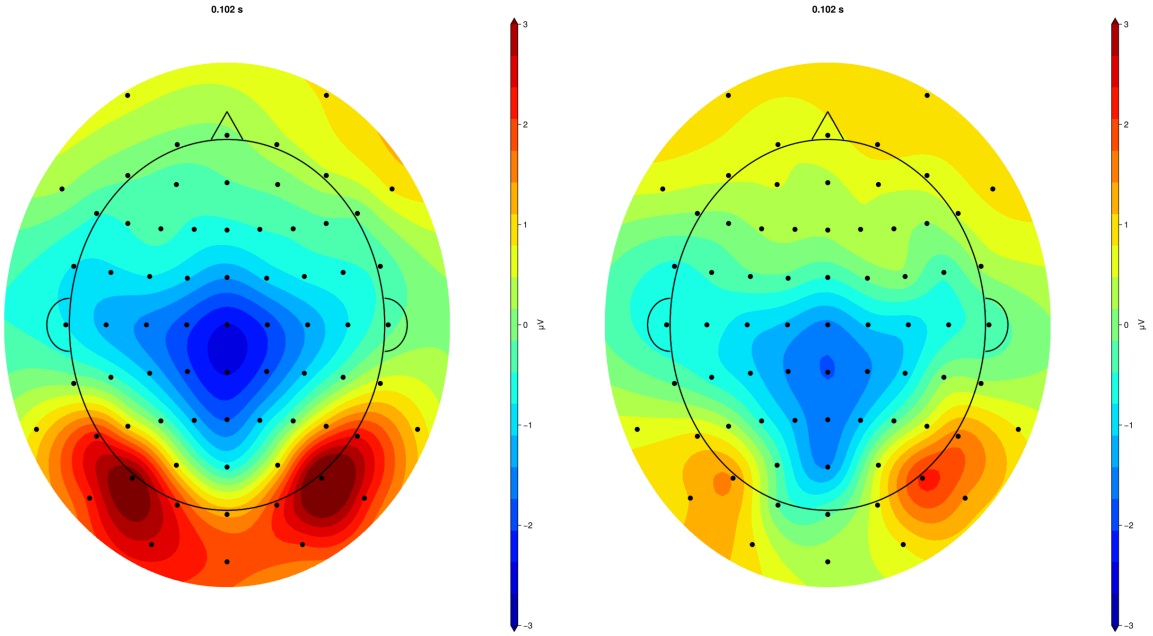

Figure 29.19: Condition-averaged Topography P1 (Valid vs Invalid)

2.5 Create Difference Waves

Sometimes we want to plot the diifference between ERP comditions. For example, we might just want to look at the validity effect by computing the **Invalid – Valid** difference wave:

```
# For a basic plot, we can just do this on the averaged data; condition 1 =
Valid, condition 2 = Invalid
EegFun.condition_difference("erps_final", [(1, 2)])


# And plot (We now have a single difference wavee)
fig = EegFun.plot_erp("grand_average_erps_final.jld2", channel_selection =
EegFun.channels([:PO7, :PO8]), baseline_interval = (-0.2, 0), average_channels =
true)
```

2.6 Plot Customisation

Several options are available for plot customisation. As an example, we can change line colour, style, legend labels, and highlight components with the following:

```
fig = EegFun.plot_erp("grand_average_erps_final.jld2",
    channel_selection = EegFun.channels([:PO7, :PO8]),
    baseline_interval = (-0.2, 0),
    average_channels = true,
    color = [:black, :grey],
    linestyle = [:solid, :dash],
    legend_labels = ["Valid", "Invalid"],
    highlight_regions = [
        (x1 = 0.09, x2 = 0.14, color = :lightgrey, alpha = 0.25),    # P1 window
        (x1 = 0.15, x2 = 0.20, color = :lightgrey, alpha = 0.25),    # N1 window
```

```
    ],
    ylim = (-4, 6),
    xlim = (-0.25, 0.65),
    xticks = -0.2:0.1:0.6,
    time_unit = :ms,
    plot_title = "Validity Effect (Avg: PO7/PO8)",
)

# and save the figure
save("erp_plot1_custom.png", fig.fig)

# or apply further customisation via Makie
fig.axes[1].titlecolor = :red
fig.axes[1].yticklabelsize = 30
# and so on ...
```

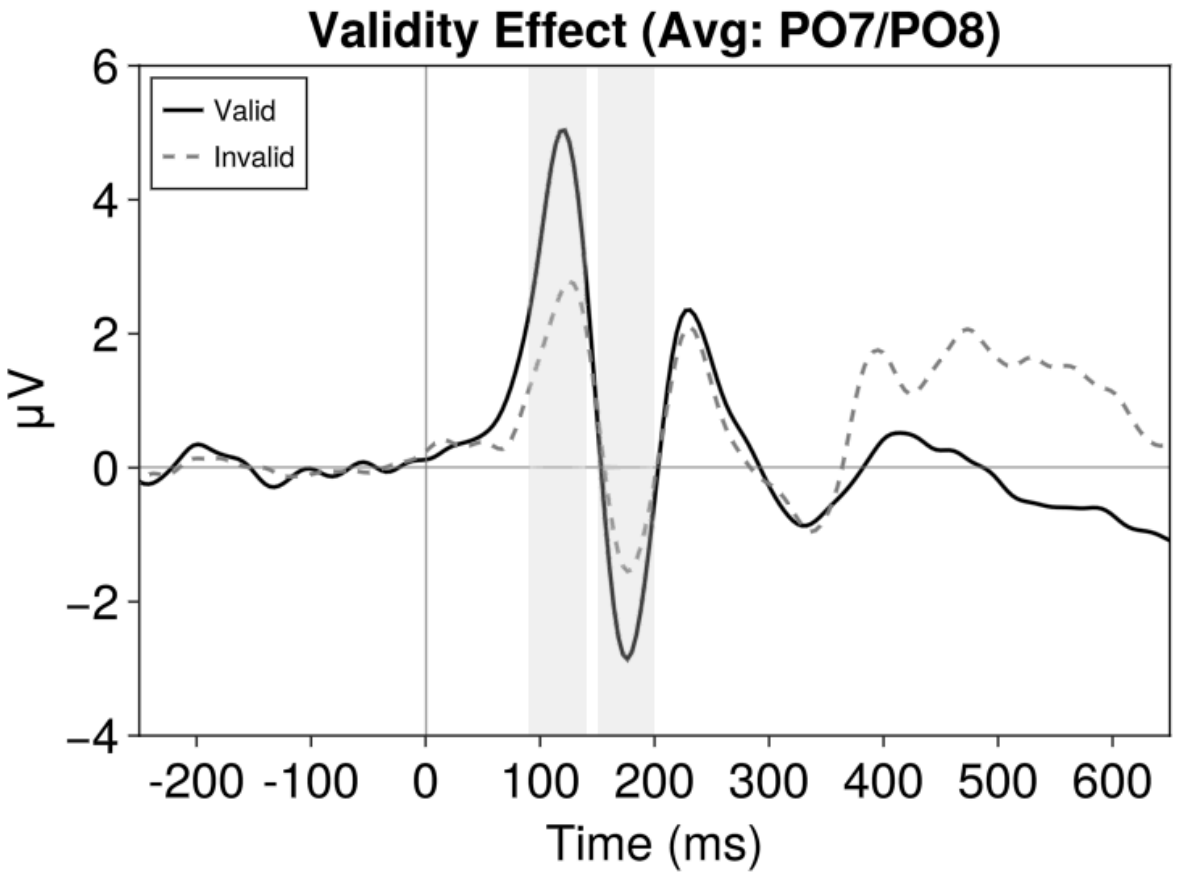

Figure 29.20: Publication-quality ERP with highlighted components

---

Part 3: Statistical Analysis

To test our hypotheses statistically, we need to extract numerical summaries from each participant's ERP waveforms. Standard ERP measurements fall into two families: **amplitude** measures (e.g., mean

amplitude, peak amplitude, area under the curve) that quantify *how large* a component is, and **latency** measures (e.g., peak latency, fractional area latency) that quantify *when* a component occurs.

3.1 ERP Measurement GUI

Before extracting measurements in batch, use the interactive GUI to explore where and when to measure:

```
# Load a single participant's ERPs
erps = EegFun.read_data("example1_erps_final.jld2")
# Open the measurement GUI
EegFun.plot_erp_measurement_gui(erps)

# or directly with a file name
EegFun.plot_erp_measurement_gui("grand_average_erps_final.jld2")
```

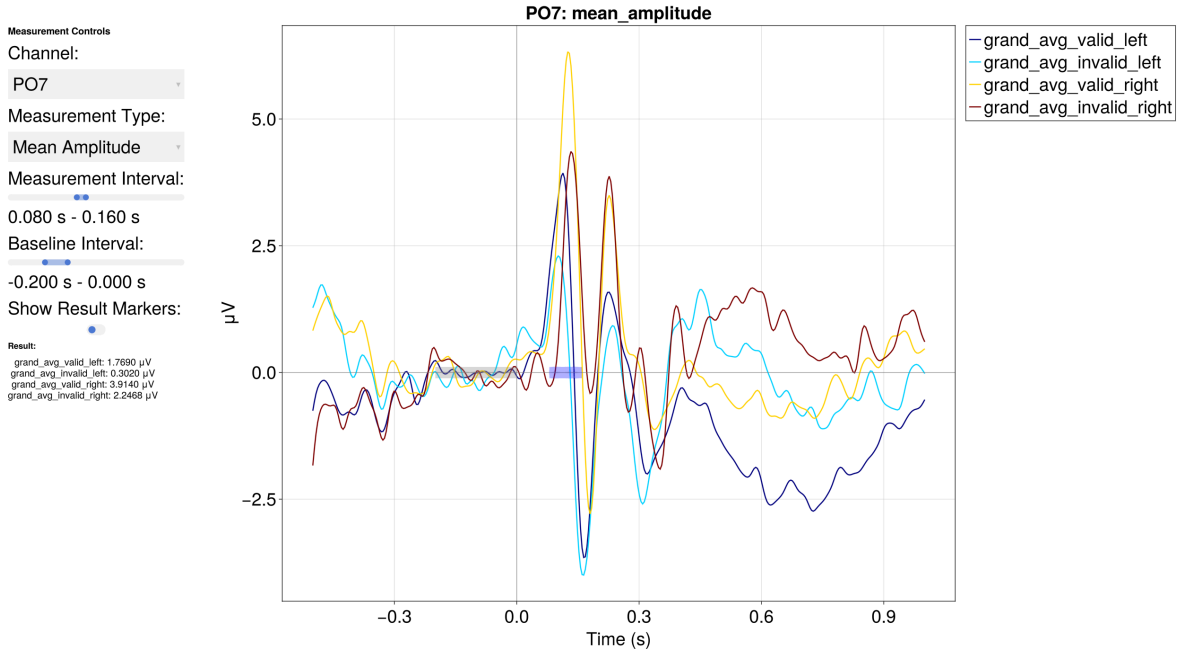

Figure 29.21: ERP measurement GUI

The GUI lets you select channels, time windows, and measurement types interactively to determine optimal parameters before running batch extraction.

> [!WARNING] The purpose of this step is to **verify** that your measurement window and type capture the component you intend to measure — not to search for channels or intervals where a difference happens to be significant. Choosing parameters based on observed effects inflates false-positive rates. Define your measurement strategy based on prior literature or orthogonal data, then use the GUI to confirm the settings look sensible.

3.2 Extract ERP Measurements

Extract mean amplitude in the P1 window at posterior-occipital sites across all participants:

```
mean_amp = EegFun.erp_measurements(
    "erps_final",
    "mean_amplitude",
    channel_selection = EegFun.channels([:PO7, :PO8]),
    analysis_interval = (0.08, 0.12),
    baseline_interval = (-0.2, 0.0),
    average_channels = true
)

# Result is an ErpMeasurementsResult containing a DataFrame
# Columns: participant, condition, PO7, PO8, O1, O2
mean_amp.data
```

3.3 Traditional Statistics with AnovaFun

```
using AnovaFun

df = mean_amp.data # our DataFrame

# --- Simple comparison (Valid vs. Invalid) via paired t-test/anova at dv_channels
(avg of PO7/PO8)
t_result = paired_ttest(df, :dv_channels, by = :condition)
anova_result = anova(df, :dv_channels, :participant, within = [:condition])

plot_anova(anova_result, x_grouping = :condition)
# or with more customisation
plot_anova(anova_result, x_grouping = :condition, individual_data
= :connected_points, theme = Theme(palette = (color = [:black],)),
axis_xlabel = "Validity", axis_ylabel = "Mean P1 Amplitude [µV]", axis_xticklabels
= ["Valid", "Invalid"])
```

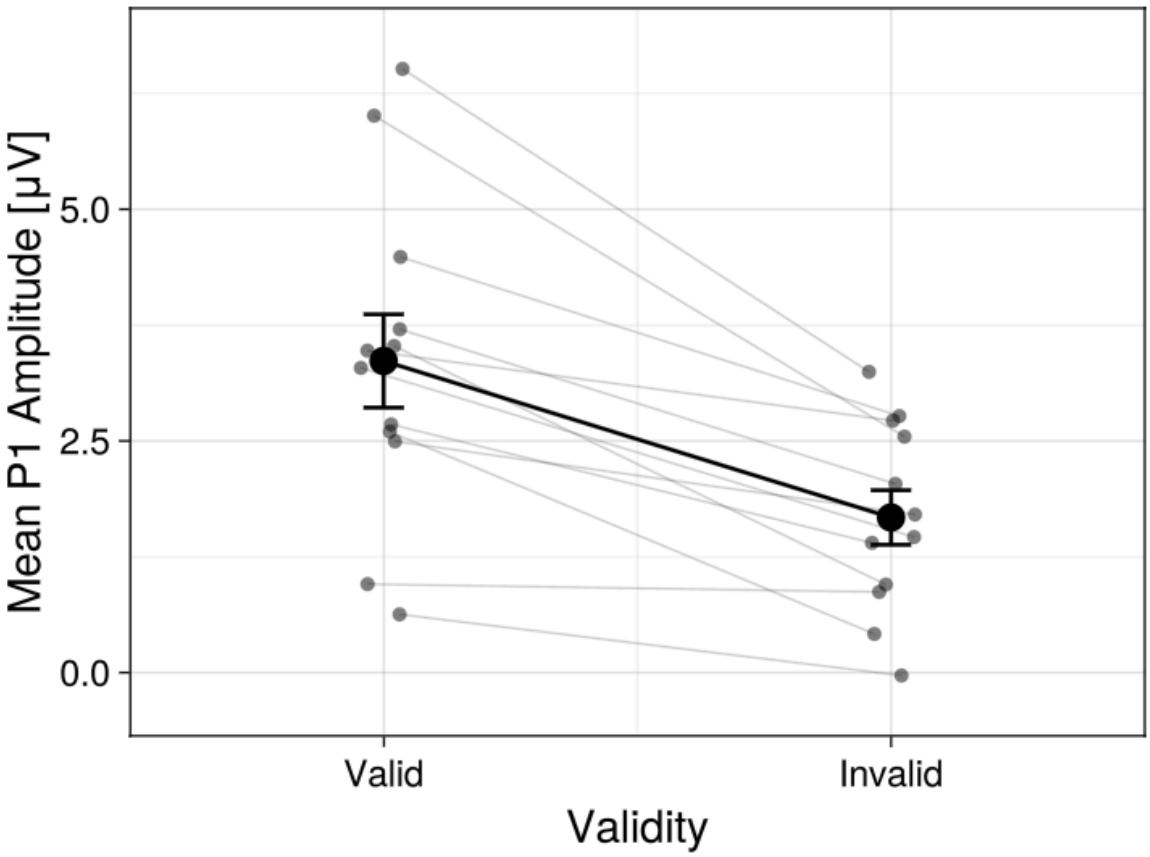

Figure 29.22: P1 mean amplitude by validity condition

3.4 Permutation-Based Statistics

Cluster-based permutation tests address the multiple comparisons problem across channels and time points:

```
# Prepare data for statistical testing
stat_data = EegFun.prepare_stats(
    "erps_final",
    :paired;
    condition_selection = EegFun.conditions([1, 2]),   # Valid left vs. Invalid
    left
    channel_selection = EegFun.channels(1:72),
    baseline_interval = EegFun.times(-0.2, 0.0),
    analysis_interval = EegFun.times(0.0, 0.3),
)

# Run cluster-based permutation test
result_perm = EegFun.permutation_test(
    stat_data,
    n_permutations = 1000,
    cluster_type = :spatiotemporal,
```

```
    show_progress = true,
)

# ERP waveforms with significance
EegFun.plot_erp_stats(
    result_perm,
    channel_selection = EegFun.channels([:PO7, :PO8, :O1, :O2]),
    plot_erp = true,
    plot_significance = true,
    plot_se = true,
    layout = :grid,
    channel_plot_order = [:O1, :O2, :PO7, :PO8],
    xlim = (-0.2, 0.5),
    ylim = (-6, 6),
    legend_labels = ["Valid", "Invalid"],
    xticks = -0.2:0.2:0.6,
    time_unit = :ms,
    legend_kwargs = (; framevisible = false)
)

# Topographies with significance
EegFun.plot_topography_stats(result_perm, interval_selection = EegFun.times(0.05,
0.2), n_topos = 10, dims = (2, 5), highlight_threshold = 1)
```

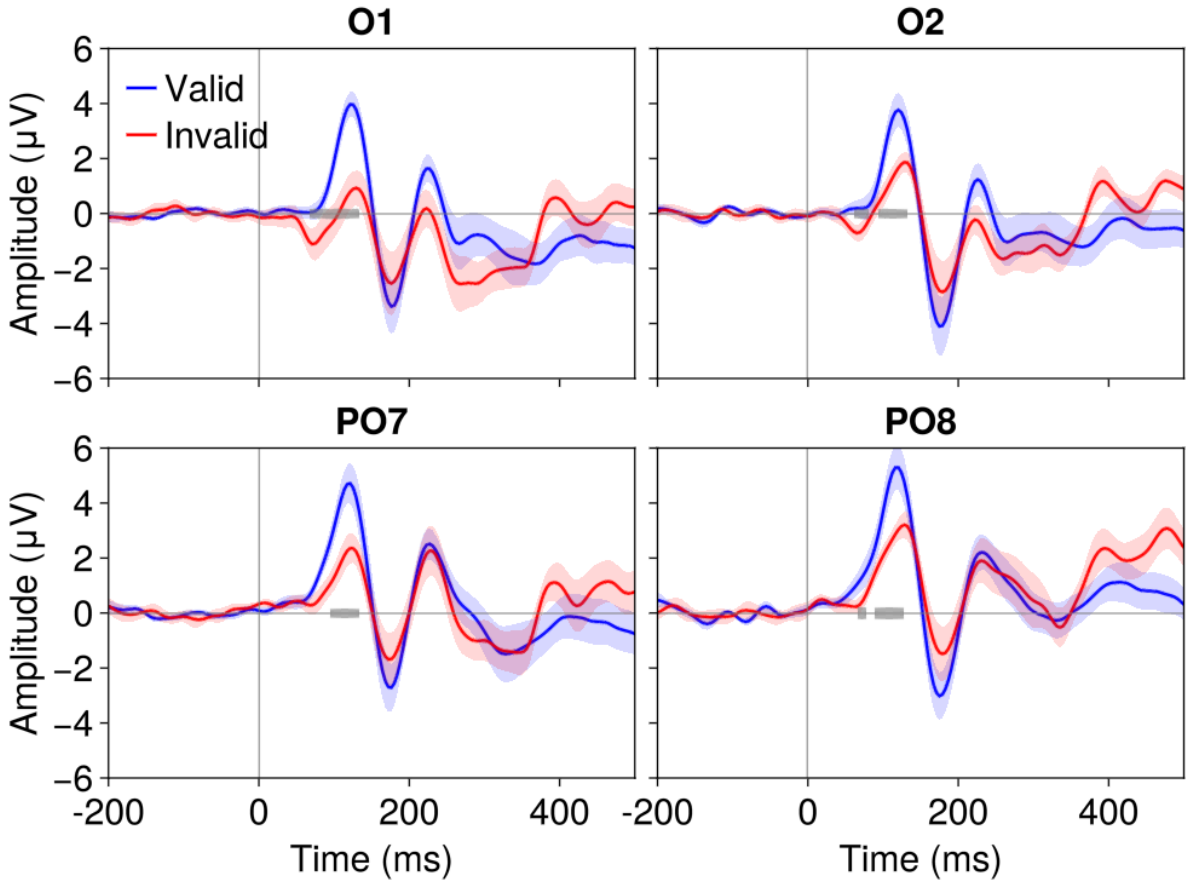

Figure 29.23: ERP cluster-based permutation test results

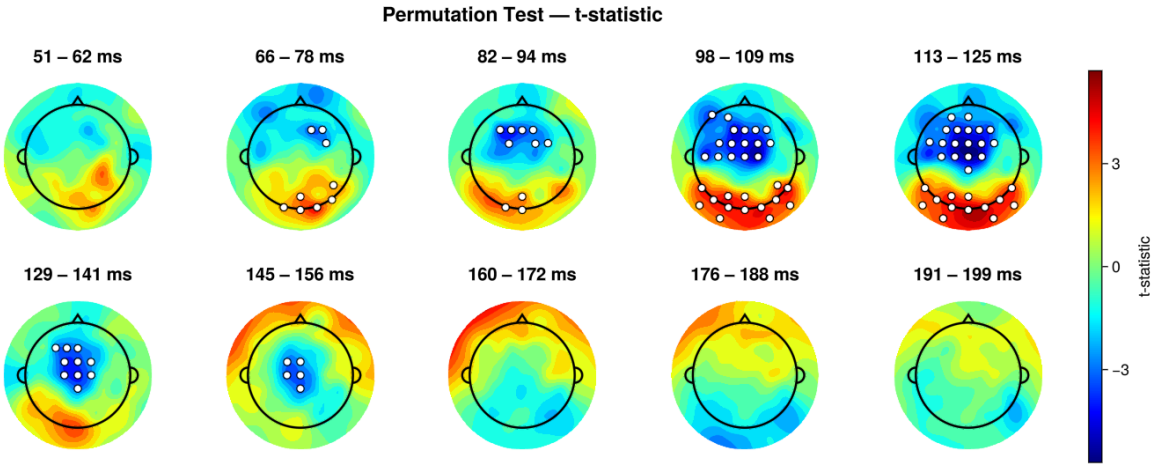

Figure 29.24: ERP cluster-based permutation test results

Further Reading

- Manual Preprocessing — Rationale behind each preprocessing step
- Batch Processing — `preprocess()` configuration
- Selection Patterns — The selector API
- Artifact Handling — Strategies for trial rescue

References

- Mangun, G. R., & Hillyard, S. A. (1991). Modulations of sensory-evoked brain potentials indicate changes in perceptual processing during visual-spatial priming. *Journal of Experimental Psychology: Human Perception and Performance*, 17(4), 1057–1074. doi:10.1037/0096-1523.17.4.1057
- Posner, M. I. (1980). Orienting of attention. *Quarterly Journal of Experimental Psychology*, 32(1), 3–25. doi:10.1080/00335558008248231

The Whole Code

All code from this tutorial combined for easy copy-paste.

```
using EegFun

# ── 1.1 Load Raw Data ──
dat = EegFun.read_raw_data("example1.bdf")
layout = EegFun.read_layout("biosemi72.csv")
dat = EegFun.create_eegfun_data(dat, layout)

# ── 1.2 Channel Layout and Neighbors ──
EegFun.plot_layout_2d(dat)
EegFun.get_neighbors_xy!(dat, 0.35)
EegFun.plot_layout_2d(dat, neighbors = true)

# ── 1.3 Inspect Triggers ──
EegFun.trigger_count(dat)
EegFun.plot_trigger_overview(dat)
EegFun.plot_trigger_timing(dat)

# ── 1.4 Browse the Raw Data ──
EegFun.plot_databrowser(dat)
EegFun.highpass_filter!(dat, 0.1)
EegFun.rereference!(dat, :avg)
EegFun.plot_databrowser(dat)

# ── 1.5 Derive EOG Channels ──
EegFun.channel_difference!(dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
)
EegFun.channel_difference!(dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
```

```
)
EegFun.detect_eog_onsets!(dat, 50, :vEOG, :is_vEOG)
EegFun.detect_eog_onsets!(dat, 50, :hEOG, :is_hEOG)

# ── 1.6 Visualise Epoch Regions ──
EegFun.mark_epoch_intervals!(dat, [5], [-0.5, 1.0])
EegFun.plot_databrowser(dat)

# ── 1.7 Channel Quality and Extreme Values ──
cs = EegFun.channel_summary(dat)
cs_epoch = EegFun.channel_summary(dat,
    sample_selection = EegFun.samples(:epoch_interval)
)
EegFun.plot_channel_summary(cs, :range)
EegFun.plot_channel_summary(cs, [:range, :min, :max, :var])
EegFun.plot_channel_summary(cs_epoch, :range)
EegFun.is_extreme_value!(dat, 200)
EegFun.plot_databrowser(dat)

# ── 1.8 Detect and Repair Bad Channels ──
cs = EegFun.channel_summary(dat,
    sample_selection = EegFun.samples(:epoch_interval)
)
jp = EegFun.channel_joint_probability(dat,
    sample_selection = EegFun.samples(:epoch_interval)
)
bad_channels = EegFun.identify_bad_channels(cs, jp)
EegFun.plot_databrowser(dat)  # Press R to open repair menu

# ── 1.9 ICA ──
EegFun.is_extreme_value!(dat, 250)
dat_ica = copy(dat)
EegFun.highpass_filter!(dat_ica, 1.0)
ica_result = EegFun.run_ica(dat_ica,
    sample_selection = EegFun.samples_not(:is_extreme_value_250)
)
EegFun.plot_topography(ica_result, component_selection = EegFun.components(1:10))
EegFun.plot_ica_component_activation(dat, ica_result)
EegFun.plot_databrowser(dat, ica_result)
component_artifacts, component_metrics = EegFun.identify_components(dat,
ica_result,
    sample_selection = EegFun.samples_not(:is_extreme_value_250)
)
EegFun.plot_ica_component_activation(dat, ica_result, artifacts =
component_artifacts)
```

```
all_removed = EegFun.get_all_ica_components(component_artifacts)
EegFun.subtract_ica_components!(dat, ica_result,
    component_selection = EegFun.components(all_removed)
)

# ── 1.10 Define Epoch Conditions ──
epoch_conditions = [
    EegFun.EpochCondition(name = "valid_left",   trigger_sequences = [[1, 5]],
    reference_index = 2),
    EegFun.EpochCondition(name = "invalid_left",  trigger_sequences = [[2, 5]],
    reference_index = 2),
    EegFun.EpochCondition(name = "valid_right",  trigger_sequences = [[3, 5]],
    reference_index = 2),
    EegFun.EpochCondition(name = "invalid_right", trigger_sequences = [[4, 5]],
    reference_index = 2),
]

# ── 1.11 Extract Epochs ──
epochs = EegFun.extract_epochs(dat, epoch_conditions, (-1.0, 1.0))
EegFun.baseline!(epochs, (-0.2, 0.0))
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(1),
channel_selection = EegFun.channels(:PO7))
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(1), layout
= :grid)
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(2), layout
= :topo)

# ── 1.12 Reject Artifacts ──
rejection_info = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion = 100.0,
z_criterion = 0.0)
EegFun.plot_artifact_detection(epochs[1], rejection_info[1])
EegFun.plot_artifact_detection(epochs[2], rejection_info[2])
EegFun.channel_repairable!(rejection_info, epochs[1].layout)
EegFun.repair_artifacts!(epochs, rejection_info)
rejection_info2 = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion =
100.0)
EegFun.detect_bad_epochs_interactive(epochs[1], artifact_info =
rejection_info2[1], dims = (3, 3))
epochs_final = EegFun.reject_epochs(epochs, rejection_info2)

# ── 1.13 Average and Plot Single-Participant ERPs ──
erps = EegFun.average_epochs(epochs_final)
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:PO7, :PO8, :O1, :O2]))
EegFun.plot_topography(erps, interval_selection = EegFun.times(0.17))
```

```
using EegFun
```

```
# ── 2.1 Batch Preprocessing ──
EegFun.preprocess("pipeline.toml")

# ── 2.2 Batch Filter ERPs ──
EegFun.lowpass_filter("erps_final", 30.0)

# ── 2.3 Grand Average ──
EegFun.grand_average("erps_final")

# ── 2.4 Create Condition Averages ──
EegFun.condition_average("erps_final", [[1, 3], [2, 4]],
    input_dir = "filtered_erps_final_lp_30.0hz"
)

# ── 2.5 Create Difference Waves ──
EegFun.condition_difference("erps_final", [(2, 1)],
    input_dir = "filtered_erps_final_lp_30.0hz/averages_erps_final_1-3_2-4"
)

# ── 2.6 Grand Average (Condition-Averaged) ──
EegFun.grand_average("erps_final",
    input_dir = "filtered_erps_final_lp_30.0hz/averages_erps_final_1-3_2-4"
)

# ── 2.7 Publication-Quality Plots ──
ga = EegFun.read_data(
    "filtered_erps_final_lp_30.0hz/averages_erps_final_1-3_2-4/
    grand_average_erps_final/grand_average_erps_final.jld2"
)
EegFun.plot_erp(ga, channel_selection = EegFun.channels([:PO7, :PO8, :O1, :O2]))
EegFun.plot_topography(ga, interval_selection = EegFun.times(0.1))    # P1
EegFun.plot_topography(ga, interval_selection = EegFun.times(0.17))   # N1

ga_diff = EegFun.read_data(
    "filtered_erps_final_lp_30.0hz/averages_erps_final_1-3_2-4/
    differences_erps_final_2-1/grand_average_erps_final/
    grand_average_erps_final.jld2"
)
EegFun.plot_erp(ga_diff, channel_selection =
EegFun.channels([:PO7, :PO8, :O1, :O2]))
```

```
using EegFun, AnovaFun

# ── 3.1 ERP Measurement GUI ──
erps = EegFun.read_data("example1_erps_final.jld2")
EegFun.plot_erp_measurement_gui(erps)
```

```
# ── 3.2 Extract ERP Measurements ──
mean_amp = EegFun.erp_measurements(
    "erps_final",
    "mean_amplitude",
    input_dir = "output_data",
    condition_selection = EegFun.conditions([1, 2, 3, 4]),
    channel_selection = EegFun.channels([:PO7, :PO8, :O1, :O2]),
    analysis_interval = (0.15, 0.2),
    baseline_interval = (-0.2, 0.0),
)

# ── 3.3 Traditional Statistics ──
df = mean_amp.data
valid_po7   = (df[df.condition .== 1, :PO7] .+ df[df.condition .== 3, :PO7]) ./ 2
invalid_po7 = (df[df.condition .== 2, :PO7] .+ df[df.condition .== 4, :PO7]) ./ 2

result = paired_ttest(valid_po7, invalid_po7)
result.t   # t-statistic
result.p   # p-value

# ── 3.4 Permutation-Based Statistics ──
stat_data = EegFun.prepare_stats(
    "erps_final",
    :paired;
    condition_selection = EegFun.conditions([1, 2]),
    channel_selection = EegFun.channels(1:72),
    baseline_interval = EegFun.times(-0.2, 0.0),
    analysis_interval = EegFun.times(0.0, 0.3),
)

result_perm = EegFun.permutation_test(
    stat_data,
    n_permutations = 1000,
    cluster_type = :spatiotemporal,
    show_progress = true,
)

# ERP waveforms with significance
EegFun.plot_erp_stats(
    result_perm,
    channel_selection = EegFun.channels([:PO7, :PO8, :O1, :O2]),
    plot_erp = true,
    plot_significance = true,
    plot_se = true,
```

```
    layout = :grid,
    channel_plot_order = [:O1, :O2, :PO7, :PO8],
    xlim = (-0.2, 0.5),
    ylim = (-6, 6),
    legend_labels = ["Valid", "Invalid"],
    xticks = -0.2:0.2:0.6,
    time_unit = :ms,
    legend_kwargs = (; framevisible = false)
)

# Topographies with significance
EegFun.plot_topography_stats(result_perm, interval_selection = EegFun.times(0.05,
0.2), n_topos = 10, dims = (2, 5), highlight_threshold = 1)
```

29.2 N170 (Face/Body)

Example Experiment: N170 (Face/Body)

This tutorial uses a complete example dataset to walk through the EegFun.jl analysis pipeline, focusing on the **N170 component** — a face-sensitive ERP that peaks around 170 ms at posterior electrodes.

The Experiment

Participants passively viewed images of **faces** and **body parts** while counting images depicting an injury (task-relevant targets to maintain attention). On each trial, a fixation cross appeared for 800 ms, followed by the image for 200 ms, then a blank inter-trial interval of 1000 ms.

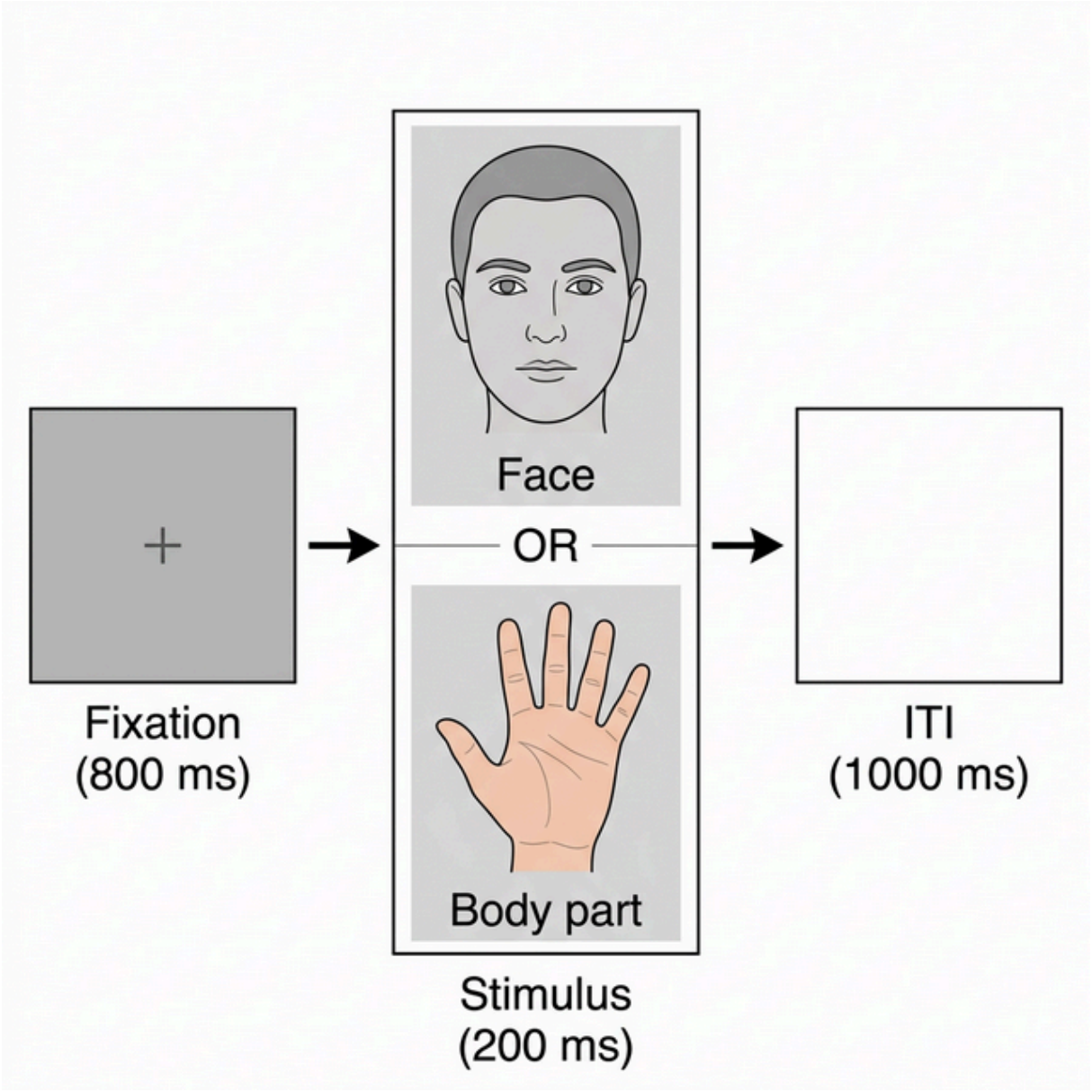

Figure 29.25: Paradigm figure showing trial sequence: fixation cross (800 ms), then face or body part stimulus (200 ms), then blank ITI (1000 ms)

Experimental Design

The design is a **single-factor within-subjects** comparison:

Factor	Levels
Stimulus Type	Body, Face

Trigger	Category
1	Body
2	Body

3	Body
5	Face
6	Face

Data

Property	Value
Participants	10
EEG System	BioSemi ActiveTwo
Channels	66 Cap EEG + 4 EOG + 2 Mastoids (72 in total)
File Format	BDF (BioSemi Data Format)
Sample Rate	512 Hz

The N170 Component

The **N170** is a negative-going ERP component peaking around 170 ms post-stimulus at posterior lateral electrodes (P7, P8, PO7, PO8). It is reliably larger for **faces** compared to other visual categories, making it one of the most studied markers of face-specific processing (Bentin et al., 1996).

Part 1: Single Participant Exploration

We start by working with a single file to explore the data, understand the preprocessing steps, and build intuition before scaling to the full dataset.

1.1 Load Raw Data

```
using EegFun

# Read the raw BDF file and channel layout
dat = EegFun.read_raw_data("example1.bdf")
layout = EegFun.read_layout("biosemi72.csv")

# Create the EegFun data structure
dat = EegFun.create_eegfun_data(dat, layout)
```

1.2 Channel Layout and Neighbors

Plot the channel layout to check that electrode positions look correct:

```
EegFun.plot_layout_2d(dat)
```

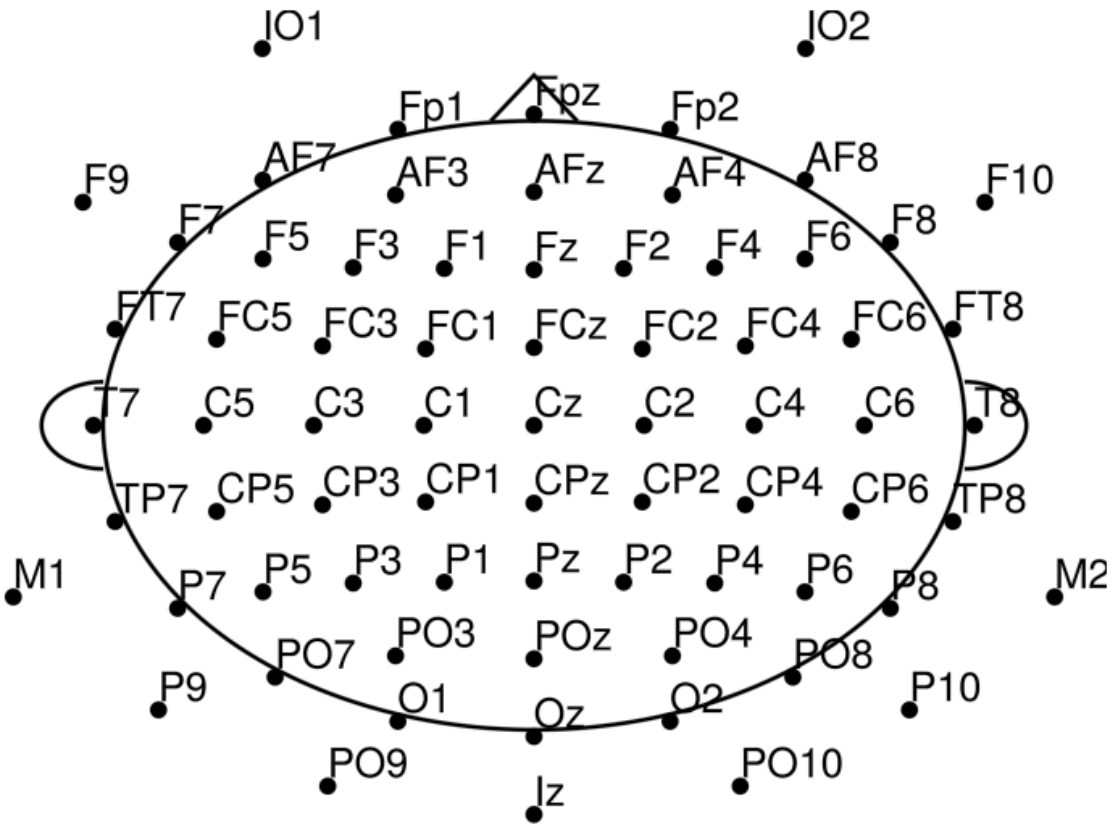

Figure 29.26: Biosemi 72-channel layout

Define channel neighbors (used later for electrode repair and cluster-based statistics):

```
EegFun.get_neighbors_xy!(dat, 0.4)
```

Plot again with neighbor connections (interactive with mouse hover):

```
EegFun.plot_layout_2d(dat, neighbors = true)
```

You can also inspect the layout interactively and export the neighbor definitions:

```
EegFun.viewer(dat.layout)
dat.layout.neighbors
EegFun.print_layout_neighbors(layout, "./neighbors.toml")
```

1.3 Inspect Triggers

Check which trigger codes are present and how many of each:

```
EegFun.trigger_count(dat)
```

```
File: example7
Type: EegFun.ContinuousData
Labels: Fp1, AF7, AF3, F1, F3, ..., F10, IO1, IO2, M1, M2
Duration: 464.998046875 S
```

```
Sample Rate: 512

Trigger Count Summary
┌─────────┬─────────┐
│ trigger │ count   │
│   Int64 │ Int64   │
├─────────┼─────────┤
│       1 │    36   │
│       2 │    26   │
│       3 │    26   │
│       5 │    55   │
│       6 │    55   │
│     253 │     1   │
└─────────┴─────────┘
```

Triggers 1–3 are **body** stimuli and triggers 5–6 are **face** stimuli. Trigger 253 is a recording artefact (BioSemi status byte) and can be ignored.

Get a visual overview of the trigger distribution:

```
EegFun.plot_trigger_overview(dat)
```

Inspect trigger timing — when each trigger occurred and the interval between triggers:

```
EegFun.plot_trigger_timing(dat)
```

You should see triggers 1–3 corresponding to body images and triggers 5–6 for face images, with a regular 2000 ms inter-trial interval (800 ms fixation + 200 ms image + 1000 ms ITI).

1.4 Browse the Raw Data

You can inspect the raw data structure directly:

```
EegFun.viewer(dat)
```

The databrowser lets you scroll through the continuous recording and visually inspect data quality:

```
EegFun.plot_databrowser(dat)
```

Before going further, remove the DC offset with a high-pass filter and rereference to the average — this makes the traces much easier to read:

```
# Remove DC offset
EegFun.highpass_filter!(dat, 0.1)

# Rereference to the average of all channels
EegFun.rereference!(dat, :avg)

# Plot again to see the effect
EegFun.plot_databrowser(dat)
```

> [!TIP] Look for obvious artifacts: flat channels, excessive drift, large muscle movements. This gives you a feel for data quality before automated processing.

1.5 Derive EOG Channels

The raw recording includes four EOG electrodes (IO1, IO2, F9, F10) placed around the eyes. By computing the difference between electrode pairs we get clean vertical and horizontal EOG signals that make blinks and saccades easy to spot:

```
# Vertical EOG: average of upper channels minus average of lower channels
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
)
# Horizontal EOG: left minus right
EegFun.channel_difference!(
    dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
)
```

Next, automatically detect EOG onsets and mark the surrounding interval as contaminated. This creates Boolean columns (`:is_vEOG` , `:is_hEOG`) that the databrowser renders as shaded overlays:

```
EegFun.detect_eog_onsets!(dat, 50, :vEOG, :is_vEOG)
EegFun.detect_eog_onsets!(dat, 50, :hEOG, :is_hEOG)
```

The new `:vEOG` and `:hEOG` channels appear under the **Extra Channels** tab in the databrowser, so you can scroll through the recording and verify that blinks and saccades are captured correctly.

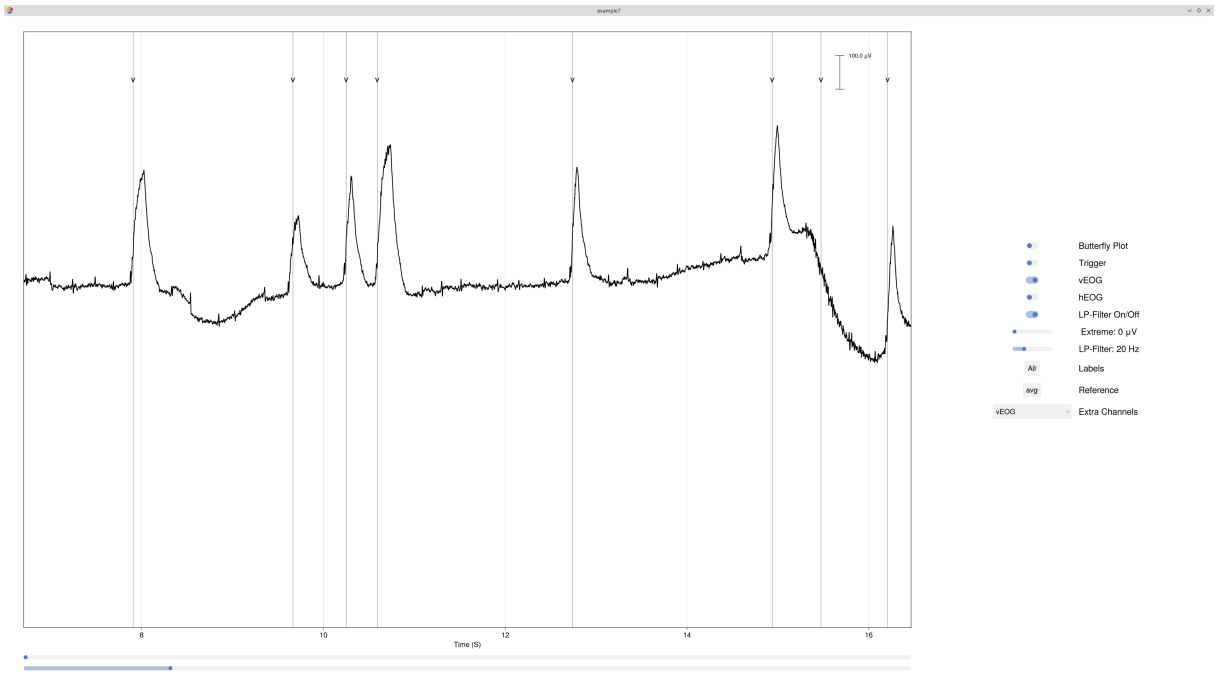

Figure 29.27: Databrowser showing vEOG channel with detected blink onsets highlighted

1.6 Visualise Epoch Regions

Before extracting epochs, it is useful to see where they would land on the continuous recording. `mark_epoch_intervals!` creates a Boolean channel that marks every sample falling within a time window around each trigger:

```
# Mark the epoch window (−200 ms to 1200 ms) around every stimulus trigger
EegFun.mark_epoch_intervals!(dat, [1, 2, 3, 5, 6], [-0.2, 1.2])

# Browse again — the epoch regions now appear as shaded overlays, and we can also
see the EOG detection
EegFun.plot_databrowser(dat)
```

The highlighted regions let you quickly check whether epoch boundaries overlap, whether any epochs contain obvious artifacts, and whether the trigger timing looks correct.

1.7 Channel Quality and Extreme Values

Use `channel_summary` to get a quick overview of each channel's range, variance, and standard deviation — this helps spot noisy or flat channels:

```
# Summary across the entire recording
cs = EegFun.channel_summary(dat)
EegFun.viewer(cs)

# Summary restricted to only the epoch regions we just marked
cs_epoch = EegFun.channel_summary(dat,
    sample_selection = EegFun.samples(:epoch_interval)
)
```

```
EegFun.viewer(cs)

# Visualise with a bar chart
EegFun.plot_channel_summary(cs, :range)
EegFun.plot_channel_summary(cs, [:range, :min, :max, :var])
EegFun.plot_channel_summary(cs_epoch, :range)
```

Next, flag every sample where **any** channel exceeds ±500 µV. This creates a Boolean column (`:is_extreme_value_500`) that the databrowser renders as shaded overlays:

```
# Mark extreme samples
EegFun.is_extreme_value!(dat, 500)

# Browse — extreme-value regions now appear as red highlights
EegFun.plot_databrowser(dat)
```

1.8 ICA

ICA works best on data that has been high-pass filtered (e.g., ≥ 1 Hz). We copy the data, apply a stricter filter, and exclude extreme samples so the decomposition focuses on genuine brain and artifact sources:

```
# Copy data and apply stricter filter for ICA
dat_ica = copy(dat)
EegFun.highpass_filter!(dat_ica, 1.0)

# Run ICA, excluding bad channels
# ica_result = EegFun.run_ica(dat_ica)
# ica_result = EegFun.run_ica(dat_ica, channel_selection =
EegFun.channels_not(bad_channels), sample_selection =
EegFun.samples_not(:is_extreme_value_500))
ica_result = EegFun.run_ica(dat_ica, sample_selection =
EegFun.samples_not(:is_extreme_value_500))
```

Inspect the component topographies and time-course activations:

```
# Component topographies
EegFun.plot_topography(ica_result, component_selection = EegFun.components(1:10));
EegFun.plot_ica_component_activation(dat, ica_result)
EegFun.plot_databrowser(dat, ica_result)
```

> [!TIP] For long recordings, subsample (percentage*of*data) can be used to speed up ICA:
>
> ```
> ica_result = EegFun.run_ica(dat_ica,
>     sample_selection = EegFun.samples_not(:is_extreme_value_500),
>     percentage_of_data = 25.0
> )
> ```

Automatically identify artifact components via EOG correlation and spatial kurtosis:

```
# Identify artifact components (excluding extreme samples)
component_artifacts, component_metrics = EegFun.identify_components(dat,
ica_result,
    sample_selection = EegFun.samples_not(:is_extreme_value_500)
)

# Inspect with artifact labels overlaid
EegFun.plot_ica_component_activation(dat, ica_result, artifacts =
component_artifacts)
```

Remove the identified components:

```
# Remove artifact components from the original (0.1 Hz filtered) data
all_removed = EegFun.get_all_ica_components(component_artifacts)
EegFun.subtract_ica_components!(dat, ica_result,
    component_selection = EegFun.components(all_removed)
)
```

View the cleaned data with ICA components available for selection:

```
# View data with ICA components cleaned dataset
EegFun.plot_databrowser(dat)
```

1.9 Define Epoch Conditions

Define the epoch conditions directly in Julia. Each condition maps stimulus triggers to a descriptive label:

```
epoch_conditions = [
    EegFun.EpochCondition(name = "body", trigger_sequences = [[1], [2], [3]]),
    EegFun.EpochCondition(name = "face", trigger_sequences = [[5], [6]]),
]
```

Each `EpochCondition` specifies:

- **`name`** — a descriptive label for the condition
- **`trigger_sequences`** — the trigger pattern(s) to match

> [!TIP] For batch processing, you can define these same conditions in a TOML file and load them with `EegFun.condition_parse_epoch("epochs.toml")` — we'll cover this in Part 2.

1.10 Extract Epochs

```
# Extract epochs: 200 ms pre-stimulus to 1200 ms post-stimulus
epochs = EegFun.extract_epochs(dat, epoch_conditions, (-0.2, 1.2))

# Baseline correction (200 ms pre-stimulus)
EegFun.baseline!(epochs, (-0.2, 0.0))
```

Before running artifact rejection, inspect the single-trial data:

```
# Single-trial waveforms at one channel
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(1, 2),
channel_selection = EegFun.channels(:P7))

# All channels in a grid
EegFun.plot_epochs(epochs, layout = :grid)

# Topographic layout
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(2), layout
= :topo)
```

1.11 Reject Artifacts

Artifact rejection — flag bad epochs and reject them:

```
rejection_info = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion =
100.0, z_criterion = 0.0)

# Inspect which epochs and channels were flagged
EegFun.plot_artifact_detection(epochs[1], rejection_info[1]) # Body
EegFun.plot_artifact_detection(epochs[2], rejection_info[2]) # Face
```

```
# Repair channels that can be interpolated
EegFun.channel_repairable!(rejection_info, epochs[1].layout)
EegFun.repair_artifacts!(epochs, rejection_info)

# Second pass on repaired data
rejection_info2 = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion =
100.0)

# Accept the rejection and keep only good epochs
epochs_final = EegFun.reject_epochs(epochs, rejection_info2)
```

1.13 Average and Plot Single-Participant ERPs

```
erps = EegFun.average_epochs(epochs_final)

# Plot all conditions at posterior sites (N170 ROI)
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]))

# Topography at the N170 time window (~170 ms)
EegFun.plot_topography(erps, interval_selection = EegFun.times(0.17, 0.17))
```

You should already see a larger N170 (negative peak at 170 ms) for faces compared to body parts at posterior lateral sites.

Part 2: Batch Processing and Group Analysis

File Organisation

```
FaceBodyExp/
├── example1.bdf ... example10.bdf     # Raw BDF files (one per participant)
├── biosemi72.csv                      # Channel layout (label, inclination,
azimuth)
├── epochs.toml                        # Epoch condition definitions
├── pipeline.toml                      # Preprocessing pipeline configuration
└── output_data/                       # Preprocessed output (after batch
processing)
```

The walk-through in Part 1 gave you the general idea of a single-participant analysis pipeline. In practice, we want to run these steps automatically across all participants so that every file receives the same analysis in a consistent, reproducible way.

> [!NOTE] The pipeline below uses `preprocess` , which provides sensible defaults for a standard ERP analysis. See the Visual Attention tutorial for full details on customising the pipeline.

2.1 Batch Preprocessing

The `preprocess` pipeline automates all single-participant steps across every BDF file:

```
EegFun.preprocess("pipeline.toml")
```

Key Parameters

Section	Parameter	Value	Description
Epochs	`epoch_start`	−0.2 s	Epoch start relative to trigger
	`epoch_end`	1.2 s	Epoch end relative to trigger
Reference	`reference_channel`	`avg`	Average reference
Artifacts	`extreme_value_abs_criterion`	200 µV	Extreme value threshold (continuous)
	`artifact_value_abs_criterion`	100 µV	Artifact threshold (epoch rejection)
ICA	`ica.apply`	`true`	ICA enabled
	`ica.percentage_of_data`	100%	Use all data for ICA

Layout	`neighbor_criterion`	0.4	Distance for neighbor definition

2.2 Batch Filter ERPs

Apply a low-pass filter to the saved ERP files (e.g., 30 Hz for clean plotting):

```
EegFun.lowpass_filter("erps_final", 30, input_dir = "./output_data")
```

2.3 Grand Average

Compute the grand average across all participants:

```
# Grand average of the ERPs
EegFun.grand_average("erps_final",
    input_dir = "./output_data/filtered_erps_final_lp_30hz"
)
```

2.4 Publication-Quality Plots

```
# Load the grand average
ga = EegFun.read_data(
    "./output_data/filtered_erps_final_lp_30hz/grand_average_erps_final/
    grand_average_erps_final.jld2"
)

# ERP waveforms at the N170 ROI
EegFun.plot_erp(ga, baseline_interval = EegFun.times(-0.2, 0),
    channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]), layout = :grid,
    ylim = (-5, 15))
```

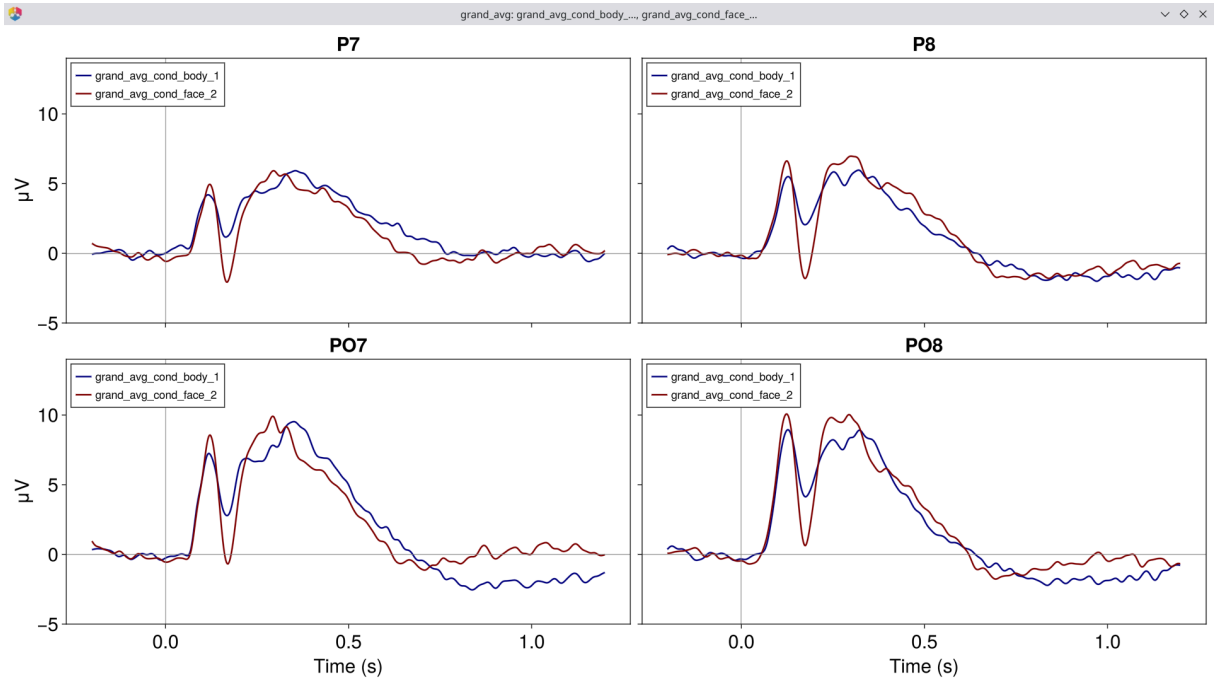

Figure 29.28: Grand average ERP waveforms at the N170 ROI (P7, P8, PO7, PO8)

```
# Topography at the N170 peak (~170 ms)
EegFun.plot_topography(ga, interval_selection = EegFun.times(0.17))
```

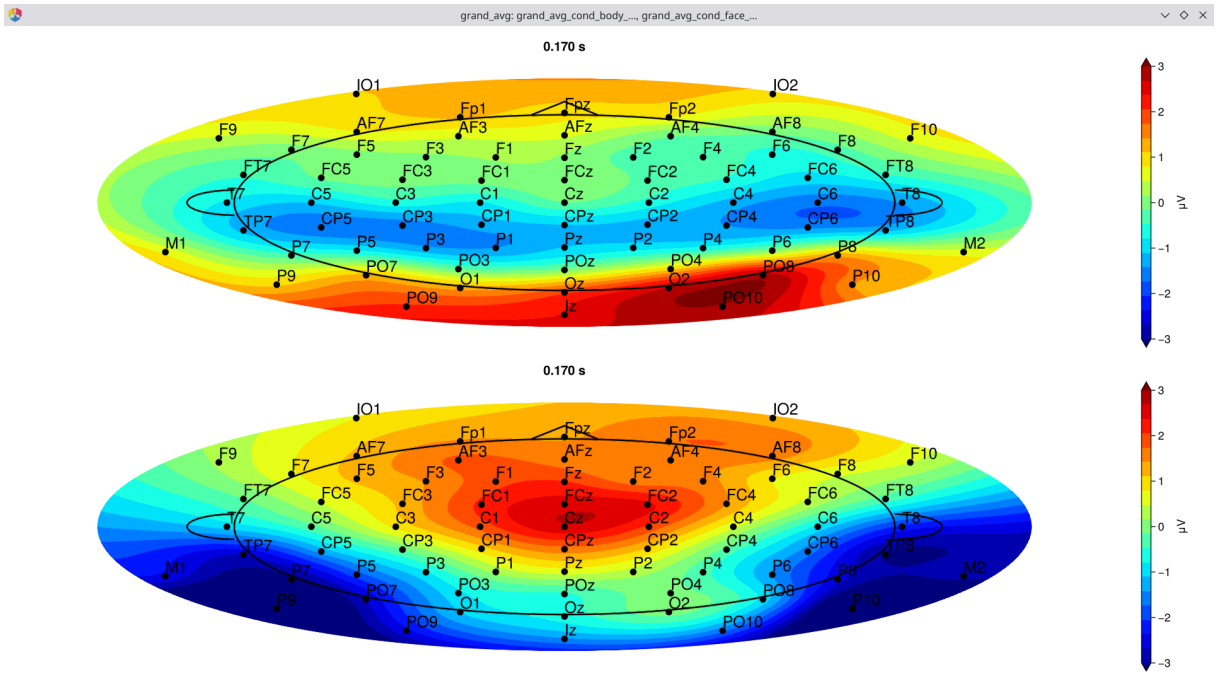

Figure 29.29: Topographic map at the N170 time window (170 ms)

2.5 Difference Wave

Compute the difference wave (Face minus Body) to isolate the N170 face effect:

```
diff = EegFun.condition_difference(ga, [[2, 1]])
```

Plot the difference wave ERP at the N170 ROI:

```
EegFun.plot_erp(diff, baseline_interval = EegFun.times(-0.2, 0),
    channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]), layout = :grid)
```

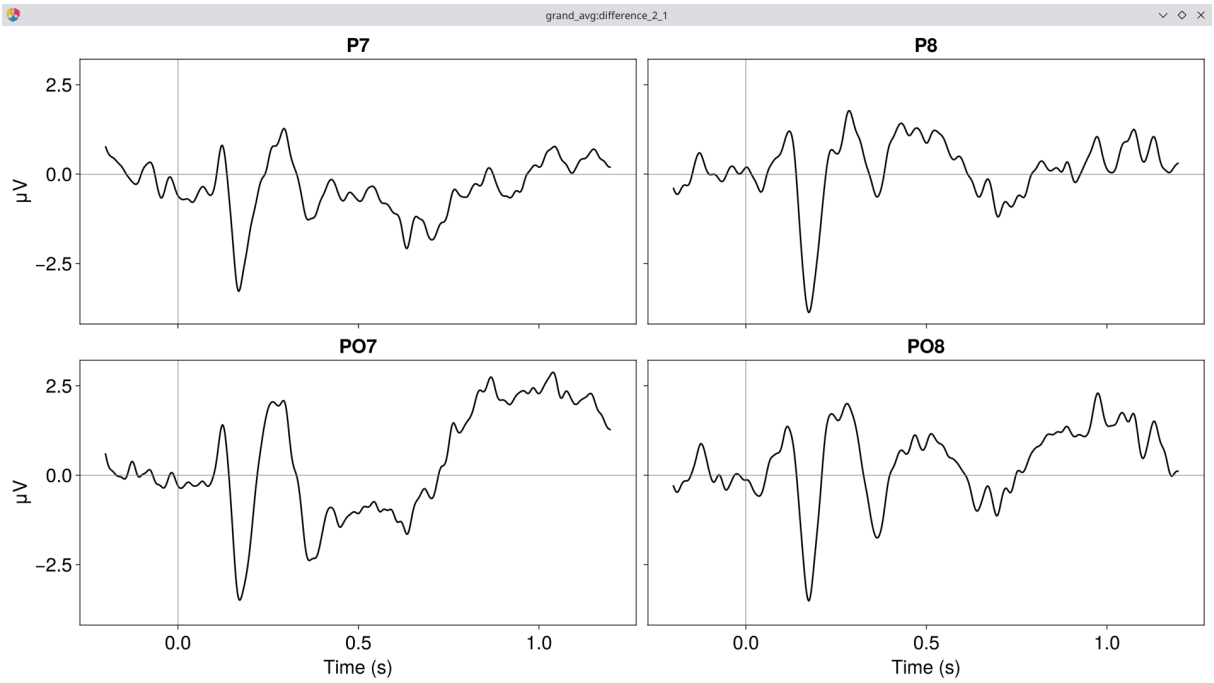

Figure 29.30: Difference wave (Face – Body) at the N170 ROI

Topography of the difference wave at the N170 peak:

```
EegFun.plot_topography(diff, interval_selection = EegFun.times(0.17), colorrange
= (-3, 3))
```

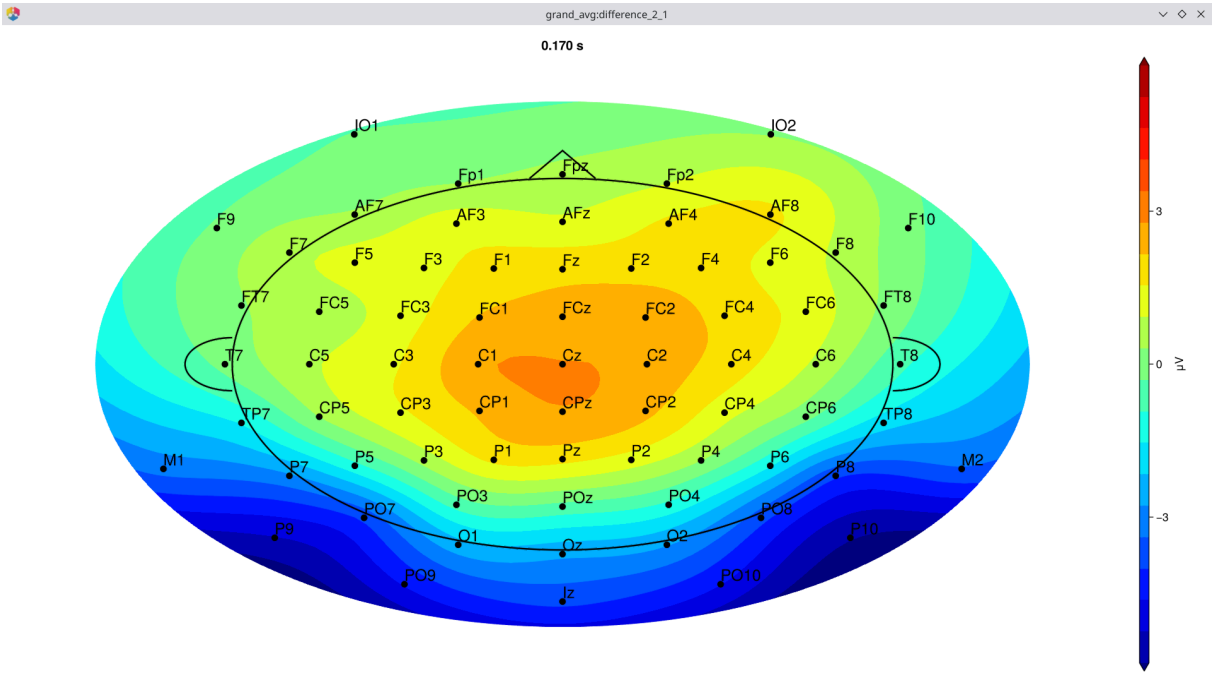

Figure 29.31: Difference wave topography at 170 ms

Part 3: Statistical Analysis

3.1 ERP Measurement GUI

Before extracting measurements in batch, use the interactive GUI to explore where and when to measure:

```
# Open the measurement GUI
EegFun.plot_erp_measurement_gui("./output_data/filtered_erps_final_lp_30hz/
example1_erps_final.jld2")
```

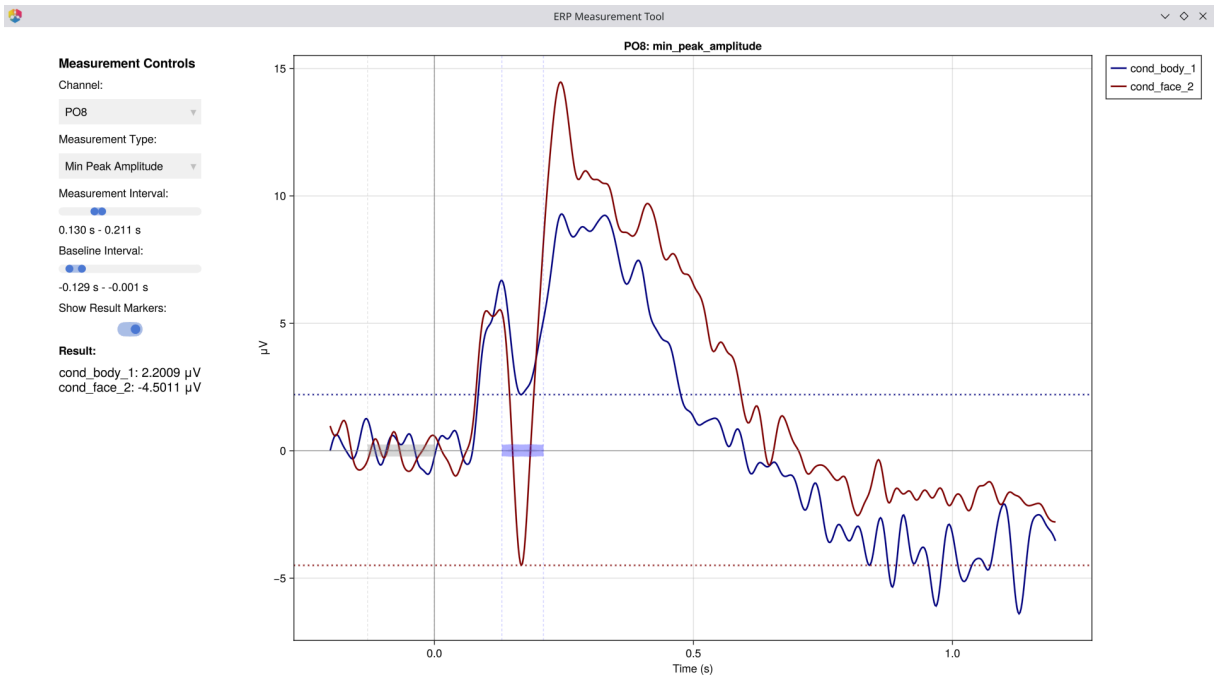

Figure 29.32: ERP Measurement GUI

The GUI lets you select channels, time windows, and measurement types interactively to determine optimal parameters before running batch extraction.

> [!WARNING] The purpose of this step is to **verify** that your measurement window and type capture the component you intend to measure — not to search for channels or intervals where a difference happens to be significant. Choosing parameters based on observed effects inflates false-positive rates. Define your measurement strategy based on prior literature or orthogonal data, then use the GUI to confirm the settings look sensible.

3.2 Extract ERP Measurements

Extract mean amplitude in the N170 window at posterior sites across all participants:

```
mean_amp = EegFun.erp_measurements(
    "erps_final",
    "mean_amplitude",
    input_dir = "./output_data/filtered_erps_final_lp_30hz/",
    condition_selection = EegFun.conditions([1, 2]),
```

```
    channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]),
    analysis_interval = (0.15, 0.2),
    baseline_interval = (-0.2, 0.0),
)

# Result is an ErpMeasurementsResult containing a DataFrame
# Columns: participant, condition, P7, P8, PO7, PO8
mean_amp.data
```

3.3 Traditional Statistics

```
using AnovaFun
using Statistics

# Average across the 4 N170 ROI channels
mean_amp.data.roi .= mean(Matrix(mean_amp.data[:, [:P7, :P8, :PO7, :PO8]]),
dims=2)

# --- Paired t-test: Body vs Face ---
result = paired_ttest(mean_amp.data, :roi, by = :condition)
result.t   # t-statistic
result.p   # p-value
```

3.4 Permutation-Based Statistics

Cluster-based permutation tests address the multiple comparisons problem across channels and time points:

```
# Prepare data for statistical testing
stat_data = EegFun.prepare_stats(
    "erps_final",
    :paired;
    input_dir = "./output_data/filtered_erps_final_lp_30hz/",
    condition_selection = EegFun.conditions([1, 2]),   # Body vs Face
    channel_selection = EegFun.channels(1:72),
    baseline_interval = EegFun.times(-0.2, 0.0),
    analysis_interval = EegFun.times(0.0, 0.3),
)

# Run cluster-based permutation test
result_perm = EegFun.permutation_test(
    stat_data,
    n_permutations = 1000,
    cluster_type = :spatiotemporal,
    show_progress = true,
)
```

```
# ERP waveforms with significance
fig = EegFun.plot_erp_stats(
    result_perm,
    channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]),
    plot_erp = true,
    plot_significance = true,
    plot_se = true,
    layout = :grid,
    channel_plot_order = [:P7, :P8, :PO7, :PO8],
    xlim = (-0.2, 0.5),
    ylim = (-4, 12),
    legend_labels = ["Body", "Face"],
    xticks = -0.2:0.2:0.6,
    time_unit = :ms,
    legend_kwargs = (; framevisible = false)
)
```

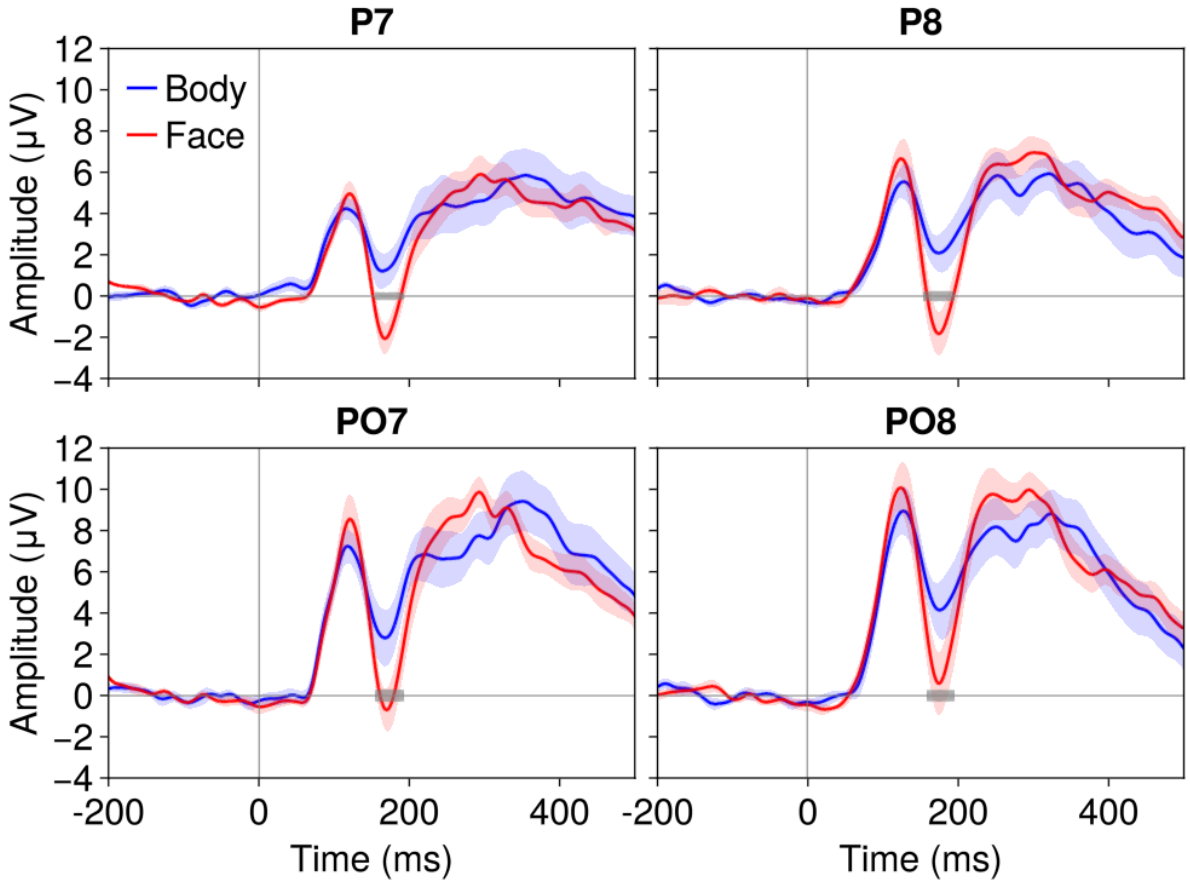

Figure 29.33: ERP cluster-based permutation test results

What to Look For

The N170 Face Effect

At **posterior lateral channels** (P7, P8, PO7, PO8):

- **N170 component** (170 ms): Larger (more negative) for faces compared to body parts — reflects face-specific processing in the fusiform gyrus
- **P1 component** (100 ms): May show a category difference, but typically smaller than the N170 effect

Lateralisation

The N170 is often **right-lateralised** for faces — compare P8/PO8 (right hemisphere) with P7/PO7 (left hemisphere) to check whether the face effect is stronger over the right hemisphere.

Further Reading

- Visual Attention (Posner Cueing) — Another worked example with cued attention
- Manual Preprocessing — Rationale behind each preprocessing step
- Batch Processing — `preprocess()` configuration
- Selection Patterns — The selector API
- Artifact Handling — Strategies for trial rescue

References

- Bentin, S., Allison, T., Puce, A., Perez, E., & McCarthy, G. (1996). Electrophysiological studies of face perception in humans. *Journal of Cognitive Neuroscience*, 8(6), 551–565. doi:10.1162/jocn.1996.8.6.551

The Whole Code

All code from this tutorial combined for easy copy-paste.

```
using EegFun

# ── 1.1 Load Raw Data ──
dat = EegFun.read_raw_data("example1.bdf")
layout = EegFun.read_layout("biosemi72.csv")
dat = EegFun.create_eegfun_data(dat, layout)

# ── 1.2 Channel Layout and Neighbors ──
EegFun.plot_layout_2d(dat)
EegFun.get_neighbors_xy!(dat, 0.4)
EegFun.plot_layout_2d(dat, neighbors = true)
EegFun.viewer(dat.layout)
dat.layout.neighbors
EegFun.print_layout_neighbors(layout, "./neighbors.toml")
```

```
# ── 1.3 Inspect Triggers ──
EegFun.trigger_count(dat)
EegFun.plot_trigger_overview(dat)
EegFun.plot_trigger_timing(dat)

# ── 1.4 Browse the Raw Data ──
EegFun.viewer(dat)
EegFun.plot_databrowser(dat)
EegFun.highpass_filter!(dat, 0.1)
EegFun.rereference!(dat, :avg)
EegFun.plot_databrowser(dat)

# ── 1.5 Derive EOG Channels ──
EegFun.channel_difference!(dat,
    channel_selection1 = EegFun.channels([:Fp1, :Fp2]),
    channel_selection2 = EegFun.channels([:IO1, :IO2]),
    channel_out = :vEOG,
)
EegFun.channel_difference!(dat,
    channel_selection1 = EegFun.channels([:F9]),
    channel_selection2 = EegFun.channels([:F10]),
    channel_out = :hEOG,
)
EegFun.detect_eog_onsets!(dat, 50, :vEOG, :is_vEOG)
EegFun.detect_eog_onsets!(dat, 50, :hEOG, :is_hEOG)

# ── 1.6 Visualise Epoch Regions ──
EegFun.mark_epoch_intervals!(dat, [1, 2, 3, 5, 6], [-0.2, 1.2])
EegFun.plot_databrowser(dat)

# ── 1.7 Channel Quality and Extreme Values ──
cs = EegFun.channel_summary(dat)
EegFun.viewer(cs)
cs_epoch = EegFun.channel_summary(dat,
    sample_selection = EegFun.samples(:epoch_interval)
)
EegFun.viewer(cs)
EegFun.plot_channel_summary(cs, :range)
EegFun.plot_channel_summary(cs, [:range, :min, :max, :var])
EegFun.plot_channel_summary(cs_epoch, :range)
EegFun.is_extreme_value!(dat, 500)
EegFun.plot_databrowser(dat)

# ── 1.8 ICA ──
```

```
dat_ica = copy(dat)
EegFun.highpass_filter!(dat_ica, 1.0)
ica_result = EegFun.run_ica(dat_ica, sample_selection =
EegFun.samples_not(:is_extreme_value_500))
EegFun.plot_topography(ica_result, component_selection = EegFun.components(1:10))
EegFun.plot_ica_component_activation(dat, ica_result)
EegFun.plot_databrowser(dat, ica_result)
component_artifacts, component_metrics = EegFun.identify_components(dat,
ica_result,
    sample_selection = EegFun.samples_not(:is_extreme_value_500)
)
EegFun.plot_ica_component_activation(dat, ica_result, artifacts =
component_artifacts)
all_removed = EegFun.get_all_ica_components(component_artifacts)
EegFun.subtract_ica_components!(dat, ica_result,
    component_selection = EegFun.components(all_removed)
)
EegFun.plot_databrowser(dat)

# ── 1.9 Define Epoch Conditions ──
epoch_conditions = [
    EegFun.EpochCondition(name = "body", trigger_sequences = [[1], [2], [3]]),
    EegFun.EpochCondition(name = "face", trigger_sequences = [[5], [6]]),
]

# ── 1.10 Extract Epochs ──
epochs = EegFun.extract_epochs(dat, epoch_conditions, (-0.2, 1.2))
EegFun.baseline!(epochs, (-0.2, 0.0))
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(1, 2),
channel_selection = EegFun.channels(:P7))
EegFun.plot_epochs(epochs, layout = :grid)
EegFun.plot_epochs(epochs, condition_selection = EegFun.conditions(2), layout
= :topo)

# ── 1.11 Reject Artifacts ──
rejection_info = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion = 100.0,
z_criterion = 0.0)
EegFun.plot_artifact_detection(epochs[1], rejection_info[1])
EegFun.plot_artifact_detection(epochs[2], rejection_info[2])
EegFun.channel_repairable!(rejection_info, epochs[1].layout)
EegFun.repair_artifacts!(epochs, rejection_info)
rejection_info2 = EegFun.detect_bad_epochs_automatic(epochs, abs_criterion =
100.0)
epochs_final = EegFun.reject_epochs(epochs, rejection_info2)

# ── 1.12 Average and Plot Single-Participant ERPs ──
```

```
erps = EegFun.average_epochs(epochs_final)
EegFun.plot_erp(erps, channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]))
EegFun.plot_topography(erps, interval_selection = EegFun.times(0.17, 0.17))
```

```
using EegFun

# ── 2.1 Batch Preprocessing ──
EegFun.preprocess("pipeline.toml")

# ── 2.2 Batch Filter ERPs ──
EegFun.lowpass_filter("erps_final", 30, input_dir = "./output_data")

# ── 2.3 Grand Average ──
EegFun.grand_average("erps_final",
    input_dir = "./output_data/filtered_erps_final_lp_30hz"
)

# ── 2.4 Publication-Quality Plots ──
ga = EegFun.read_data(
    "./output_data/filtered_erps_final_lp_30hz/grand_average_erps_final/
    grand_average_erps_final.jld2"
)
EegFun.plot_erp(ga, baseline_interval = EegFun.times(-0.2, 0),
    channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]), layout = :grid,
    ylim = (-5, 15))
EegFun.plot_topography(ga, interval_selection = EegFun.times(0.17))

# ── 2.5 Difference Wave ──
diff = EegFun.condition_difference(ga, [[2, 1]])
EegFun.plot_erp(diff, baseline_interval = EegFun.times(-0.2, 0),
    channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]), layout = :grid)
EegFun.plot_topography(diff, interval_selection = EegFun.times(0.17), colorrange =
(-3, 3))
```

```
using EegFun, AnovaFun, Statistics

# ── 3.1 ERP Measurement GUI ──
EegFun.plot_erp_measurement_gui("./output_data/filtered_erps_final_lp_30hz/
example1_erps_final.jld2")

# ── 3.2 Extract ERP Measurements ──
mean_amp = EegFun.erp_measurements(
    "erps_final",
    "mean_amplitude",
    input_dir = "./output_data/filtered_erps_final_lp_30hz/",
    condition_selection = EegFun.conditions([1, 2]),
```

```
    channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]),
    analysis_interval = (0.15, 0.2),
    baseline_interval = (-0.2, 0.0),
)

# ── 3.3 Traditional Statistics ──
mean_amp.data.roi .= mean(Matrix(mean_amp.data[:, [:P7, :P8, :PO7, :PO8]]),
dims=2)
result = paired_ttest(mean_amp.data, :roi, by = :condition)
result.t   # t-statistic
result.p   # p-value

# ── 3.4 Permutation-Based Statistics ──
stat_data = EegFun.prepare_stats(
    "erps_final",
    :paired;
    input_dir = "./output_data/filtered_erps_final_lp_30hz/",
    condition_selection = EegFun.conditions([1, 2]),
    channel_selection = EegFun.channels(1:72),
    baseline_interval = EegFun.times(-0.2, 0.0),
    analysis_interval = EegFun.times(0.0, 0.3),
)

result_perm = EegFun.permutation_test(
    stat_data,
    n_permutations = 1000,
    cluster_type = :spatiotemporal,
    show_progress = true,
)

# ERP waveforms with significance
fig = EegFun.plot_erp_stats(
    result_perm,
    channel_selection = EegFun.channels([:P7, :P8, :PO7, :PO8]),
    plot_erp = true,
    plot_significance = true,
    plot_se = true,
    layout = :grid,
    channel_plot_order = [:P7, :P8, :PO7, :PO8],
    xlim = (-0.2, 0.5),
    ylim = (-4, 12),
    legend_labels = ["Body", "Face"],
    xticks = -0.2:0.2:0.6,
    time_unit = :ms,
    legend_kwargs = (; framevisible = false)
```

```
)
```

Part V

Teaching Demos

Chapter 30

Signal Processing

30.1 Sampling

Signal Example — Sampling & Reconstruction

Interactive demonstration of the **Nyquist–Shannon sampling theorem**.

This demo shows how sampling rate affects the reconstruction of a sine wave.

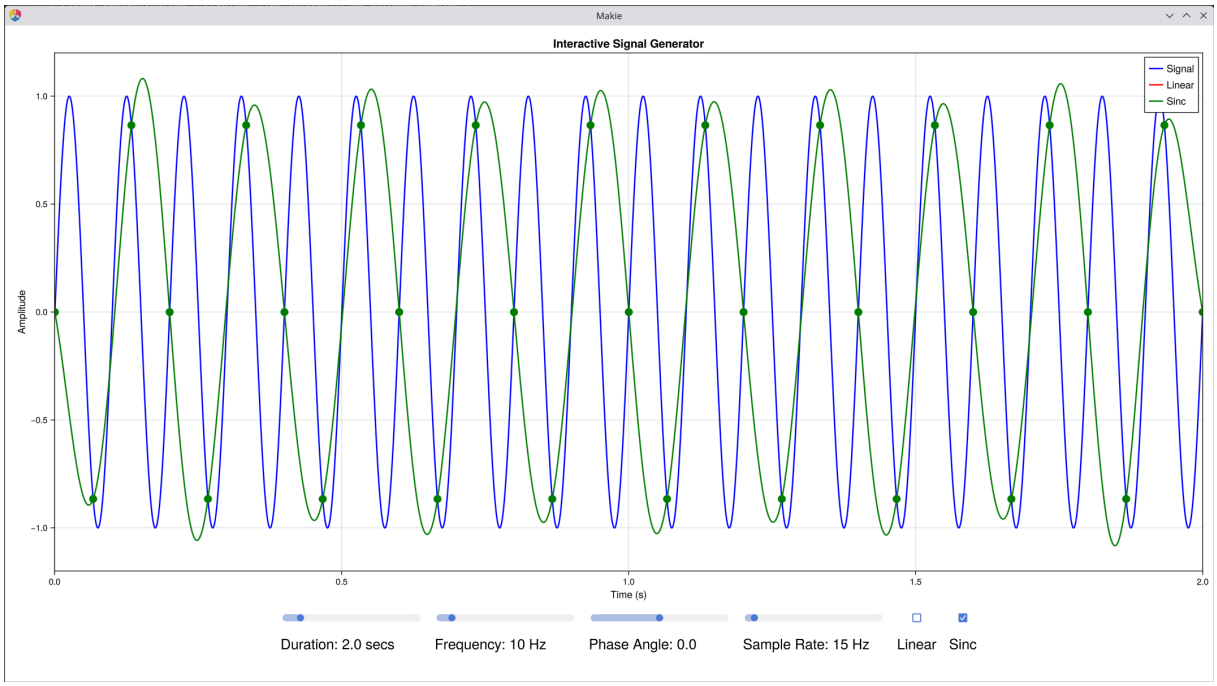

Figure 30.1: Signal Example 1

What it shows

Feature	Description
Original signal	A “continuous” sine wave at the chosen frequency
Sampled points	The discrete samples taken at the chosen sampling rate
Linear reconstruction	Straight-line interpolation between samples
Sinc reconstruction	Whittaker–Shannon (ideal) interpolation

Linear reconstruction connects adjacent samples with straight lines. It is simple but introduces distortion — particularly at low sampling rates relative to the signal frequency.

Sinc reconstruction uses the Whittaker–Shannon interpolation formula to reconstruct the signal exactly (within the Nyquist limit).

When the sampling rate falls **below** twice the signal frequency, the signal cannot be recovered: the sampled waveform appears at the wrong frequency — a phenomenon known as **aliasing**. Modern EEG systems typically sample at 256–2048 Hz, which is far above the Nyquist limit for brain signals of interest (usually < 100 Hz), ensuring faithful digitisation.

Things to Try

- Start with a sampling rate well above Nyquist (e.g. 10× the signal frequency) and observe that both methods agree with the original signal.
- Gradually reduce the sampling rate toward and then below the Nyquist limit (2× the signal frequency) to show reconstruction breakdown and **aliasing**.
- Enable both reconstructions simultaneously to highlight the superiority of sinc interpolation near the Nyquist limit.

Controls

Control	Range	Description
Duration	1–10 s	Length of the displayed signal
Signal Frequency	1–100 Hz	Frequency of the underlying sine wave
Phase Angle	$-\pi$ to π	Phase offset of the sine wave
Sampling Rate	1–300 Hz	Number of samples per second
☐ Linear	—	Toggle linear interpolation overlay
☐ Sinc	—	Toggle sinc reconstruction overlay

See Also

- Luck, S. J. (2014). *An Introduction to the Event-Related Potential Technique* (2nd ed.). MIT Press. — Chapter 5 (Section: Amplifying, Filtering, and Digitizing the Signal, Figure 5.7)
- Sinc Interpolation for Signal Reconstruction (Wolfram Demonstrations)
- Signal Example — Composition — multi-signal composition and filtering
- Signal Example — Dot Product — how the DFT detects frequencies

Code

```
using EegFun
EegFun.signal_example_sampling()
```

30.2 Composition

Signal Example — Signal Composition & Filtering

Interactive multi-signal composer demonstrating how complex waveforms are built from simple sine waves, how noise affects a signal, and how filters shape the frequency content.

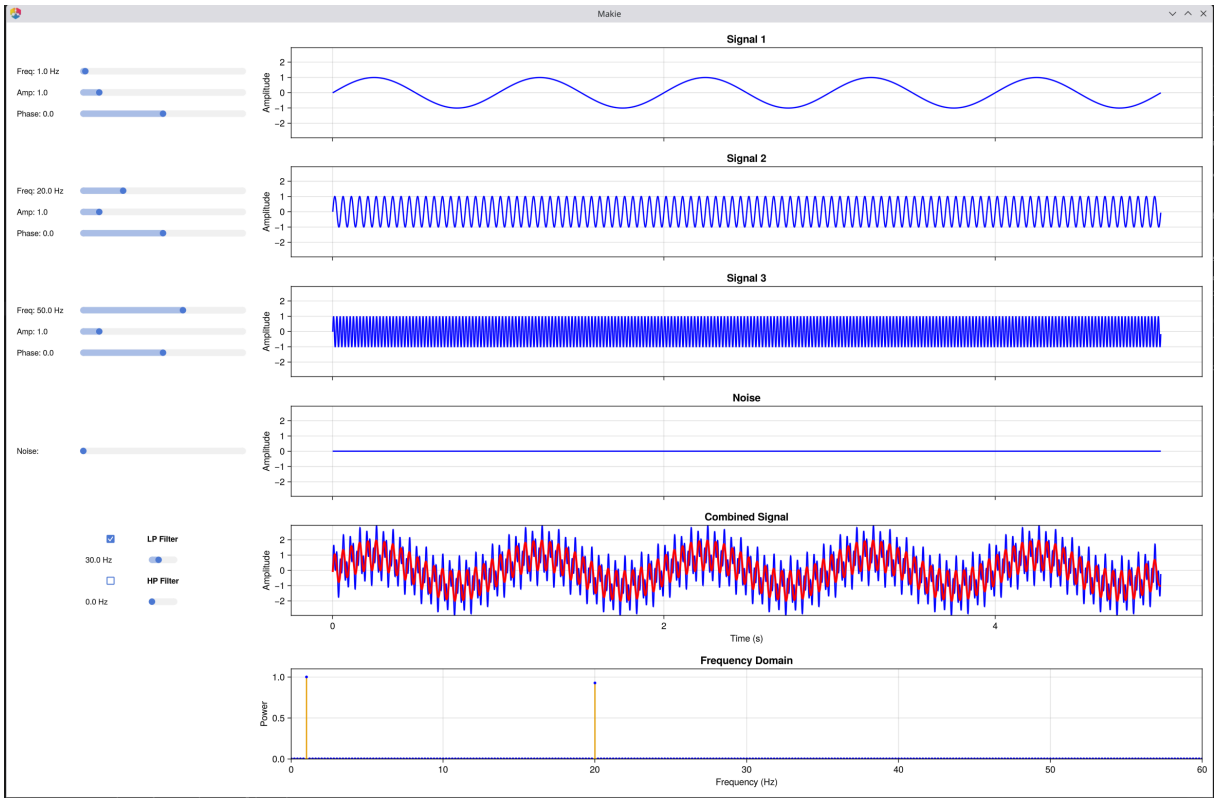

Figure 30.2: Signal Example 2

What it shows

Row	Plot	Description
1–3	Signal 1–3	Individual sine waves (y-axes linked for direct amplitude comparison)
4	Noise	Additive Gaussian noise
5	Combined Signal	Sum of all signals plus noise; a red overlay shows the filtered version when a filter is active
6	Frequency Domain	Power spectrum (amplitude²) of the combined signal (or the filtered signal when a filter is active)

The x-axes of all five time-domain plots are linked — panning or zooming one panel updates all of them simultaneously. The y-axes of Signal 1–3 are also linked, so amplitude changes remain visually comparable across all three.

Things to Try

- Start with a single signal (Amp > 0 for Signal 1 only) and observe its clean spectral peak.
- Add a second frequency to show how signals superpose in the time domain and produce two peaks in the spectrum.
- Increase noise and watch the spectral floor rise until the signal peaks are buried — a direct illustration of the signal-to-noise problem in EEG.
- Enable the LP filter and sweep its cutoff to demonstrate how filtering removes high-frequency content while preserving low-frequency structure.
- Enable the HP filter and set a low cutoff (e.g. 0.1–1 Hz) to show how slow drifts are removed — a common preprocessing step in EEG.

Controls

Each signal row has its own frequency, amplitude, and phase controls in the left panel.

Control	Range	Description
Freq (×3)	0–80 Hz	Frequency of each sine wave
Amp (×3)	0–10	Amplitude of each sine wave
Phase (×3)	$-\pi$ to π	Phase offset of each sine wave
Noise	0–2	Standard deviation of additive Gaussian noise
☐ LP Filter	0–100 Hz	Low-pass filter cutoff
☐ HP Filter	0–2 Hz	High-pass filter cutoff (typical EEG values: 0.1–1 Hz)

See Also

- Luck, S. J. (2014). *An Introduction to the Event-Related Potential Technique* (2nd ed.). MIT Press. — Chapter 1 (Figure 1.6)
- Signal Example — Sampling — Nyquist theorem and signal reconstruction
- Signal Example — Dot Product — how the DFT detects individual frequencies

Code

```
using EegFun
EegFun.signal_example_composition()
```

30.3 Dot Product

Signal Example — Dot Product & Frequency Detection

Interactive demo showing the single core idea behind the **Discrete Fourier Transform (DFT)**: multiplying a signal by a test sinusoid and summing the result.

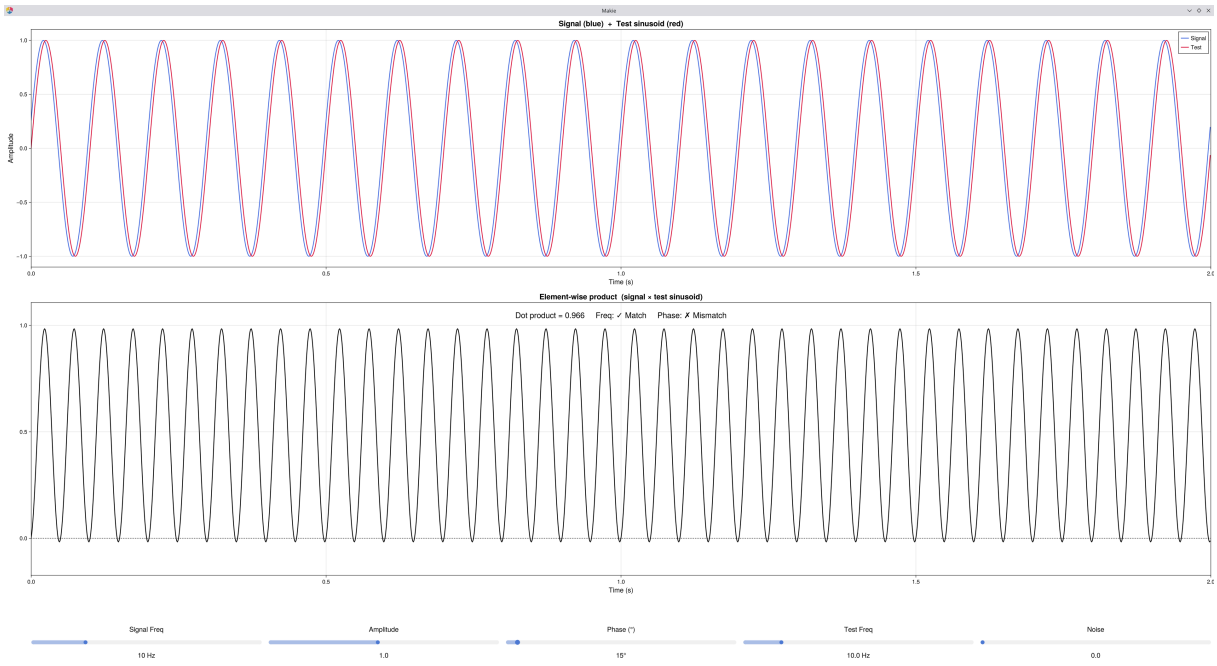

Figure 30.3: Signal Example Dot Product

What it shows

Row	Plot	Description
1	Signal + Test sinusoid(s)	The signal (blue) and the sine test sinusoid (red) overlaid. When **Complex** is on, the cosine test (green) also appears.
2	Sine product	Signal × sine test sinusoid, with the dot product value and Freq/Phase match status shown.
3	Cosine product	Signal × cosine test sinusoid (visible only when **Complex** is on).

Key Concept

The **dot product** measures how similar two signals are — multiply them point-by-point and sum the results:

$$\text{dot product} = \sum_t a[t] \cdot b[t]$$

The DFT uses exactly this idea to answer "how much of frequency f is in this signal?" by setting one of the signals to a test sinusoid:

$$\text{dot product} = \sum_t x[t] \cdot \sin(2\pi f t)$$

Freq match, Phase match ⟶ product is all-positive ⟶ large sum ⟶ large dot product.

Freq mismatch ⟶ product alternates +/- ⟶ cancels to near zero.

Phase mismatch (90°) ⟶ product alternates even at matching frequency ⟶ dot product ≈ 0.

Why "complex"?

The sine-only dot product is **phase-sensitive** — it fails when the signal and test sinusoid are 90° out of phase. The DFT solves this by computing *two* dot products — one with a sine and one with a cosine — and combining them into a single phase-independent measure:

$$\text{magnitude} = \sqrt{\text{sine_ dp}^2 + \text{cosine_ dp}^2}$$

The sine captures the "how much matches my phase" part. The cosine captures the "how much is 90° away" part. Together, they capture *all* the energy at that frequency, no matter the phase. Toggle **Complex** on and drag the Phase slider to see this in action.

Controls

Control	Range	Description
Signal Freq	1–40 Hz	Frequency of the signal
Amplitude	0.1–2	Signal amplitude — dot product scales proportionally
Phase (°)	0–355°	Signal phase offset — try 90° at matching frequency
Test Freq	0.5–60 Hz	The DFT "probe" frequency
Noise	0–1	Additive Gaussian noise
Complex	toggle	Add cosine probing for phase-independent magnitude

Things to Try

1. **Frequency match:** set Signal Freq = Test Freq = 10 Hz ⟶ product all-positive, dot product ≈ amplitude.
2. **Frequency mismatch:** move Test Freq to 15 Hz ⟶ product alternates, dot product ≈ 0.

3. **Phase sensitivity:** set both frequencies to 10 Hz, then drag Phase to 90° ⟶ dot product drops to ≈ 0 even though frequencies match.
4. **Complex to the rescue:** with Phase still at 90°, toggle **Complex** on ⟶ the magnitude (purple) recovers to ≈ amplitude. Drag Phase anywhere — the magnitude stays constant.
5. **Amplitude scaling:** raise Amplitude to 2 ⟶ dot product doubles. The test signal is always ±1; the result reflects only the signal amplitude.
6. **Noise:** add noise ⟶ individual products become noisy but the sum (dot product) is still large at the matching frequency (averaging suppresses noise).

See Also

- Signal Example — Composition — building signals from sine waves
- Signal Example — Convolution — sliding the dot product across time
- Cohen, M. X. (2014). *Analyzing Neural Time Series Data*. MIT Press. — Chapter 11

Code

```
using EegFun
EegFun.signal_example_dotproduct()
```

30.4 Convolution

Signal Example — Convolution

Interactive demo showing discrete convolution by sliding a kernel across a signal, with three kernel types including a Morlet wavelet to bridge filtering and time-frequency analysis.

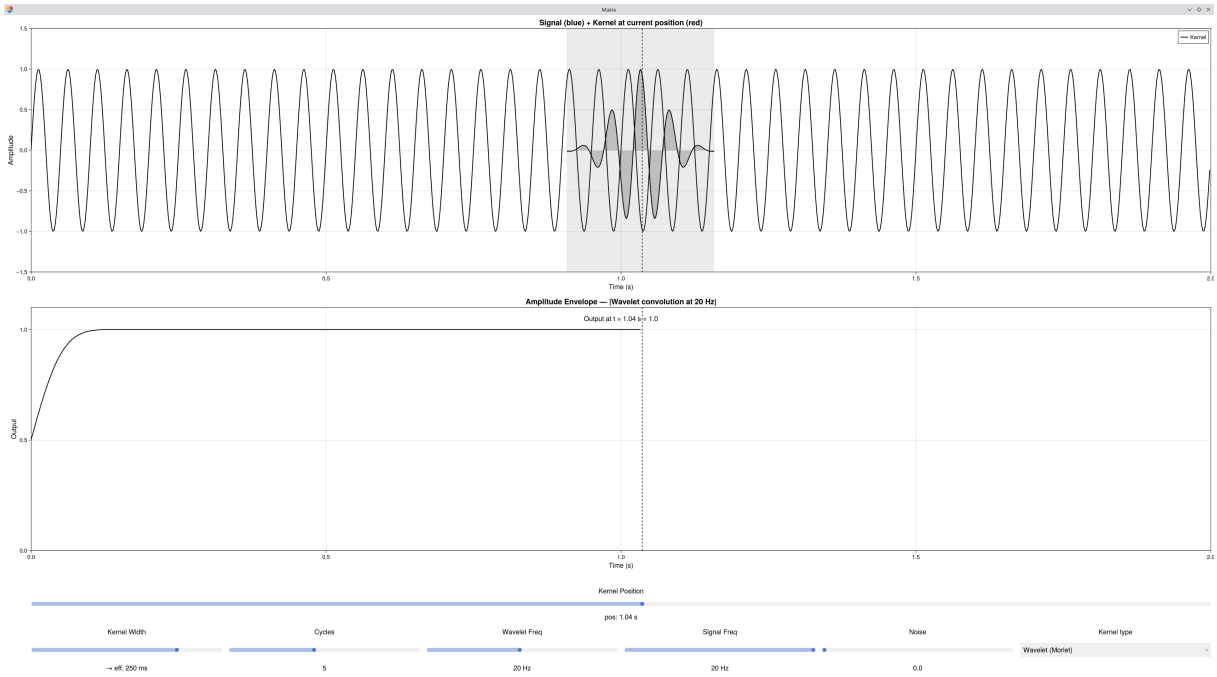

Figure 30.4: Signal Example Convolution

What it shows

Row	Plot	Description
1	Signal + Kernel	The input signal with the kernel overlaid at the current position — the shaded band marks the overlap region.
2	Output	The convolution result accumulated up to the current kernel position. For the Wavelet kernel, this is the amplitude envelope.

Key Concept

Convolution computes, at each time point *t*, the weighted sum of the signal under the kernel:

$$(x*h)[t] = \sum_{\tau} x[\tau] \cdot h[t - \tau]$$

Gaussian: smooth low-pass filter — noise suppressed, slow waves preserved.

Boxcar: running average — same idea but with sharper edges in frequency.

Wavelet (Morlet): `Gaussian × cos(2πfτ)` — the kernel is frequency-selective. The output is the amplitude envelope: how much of *Wavelet Freq* is present at each time point. This is one row of a time-frequency spectrogram.

Controls

Control	Applies to	Description
Kernel Position	All	Scrub the kernel across the signal — output builds up as you drag right
Kernel Width	Gaussian / Boxcar	Total span of the kernel (label updates to "⟶ eff. X ms" in Wavelet mode)
Cycles	Wavelet	Number of oscillations in the kernel — more cycles = better frequency resolution, worse time resolution
Wavelet Freq	Wavelet	The target frequency to detect
Signal Freq	All	Frequency of the input signal
Noise	All	Additive Gaussian noise

Kernel type	All	Switch between Gaussian, Boxcar, Wavelet (Morlet)

Things to Try

1. **Low-pass filtering:** Gaussian kernel, drag position to end — output is a smoother version of the noisy input.
2. **Kernel width effect:** Gaussian, widen the kernel → noise disappears, but the sine is also attenuated as the kernel begins to span whole cycles.
3. **Switch to Wavelet Freq = Signal Freq** → amplitude envelope is near 1.0 (frequency detected).
4. **Mismatch frequencies** (e.g. Wavelet = 8 Hz, Signal = 4 Hz) → envelope near 0.
5. **Cycles tradeoff:** with two nearby signal frequencies, fewer cycles blurs them together; more cycles separates them.
6. **Link to Gaussian:** set Kernel Width = value shown in "→ eff. X ms" label, then switch kernel type — you will see the same Gaussian envelope.

See Also

- Signal Example — Composition — building signals from sine waves
- Signal Example — Dot Product — the core dot product mechanism
- Signal Example — Time-Frequency — running Morlet convolution at every frequency
- Cohen, M. X. (2014). *Analyzing Neural Time Series Data.* MIT Press. — Chapter 11/12

Code

```
using EegFun
EegFun.signal_example_convolution()
```

30.5 Time-Frequency

Signal Example — Time-Frequency Analysis

Interactive demo comparing three time-frequency analysis methods on a synthetic multi-component signal, with controls for baseline normalization and frequency-axis scaling.

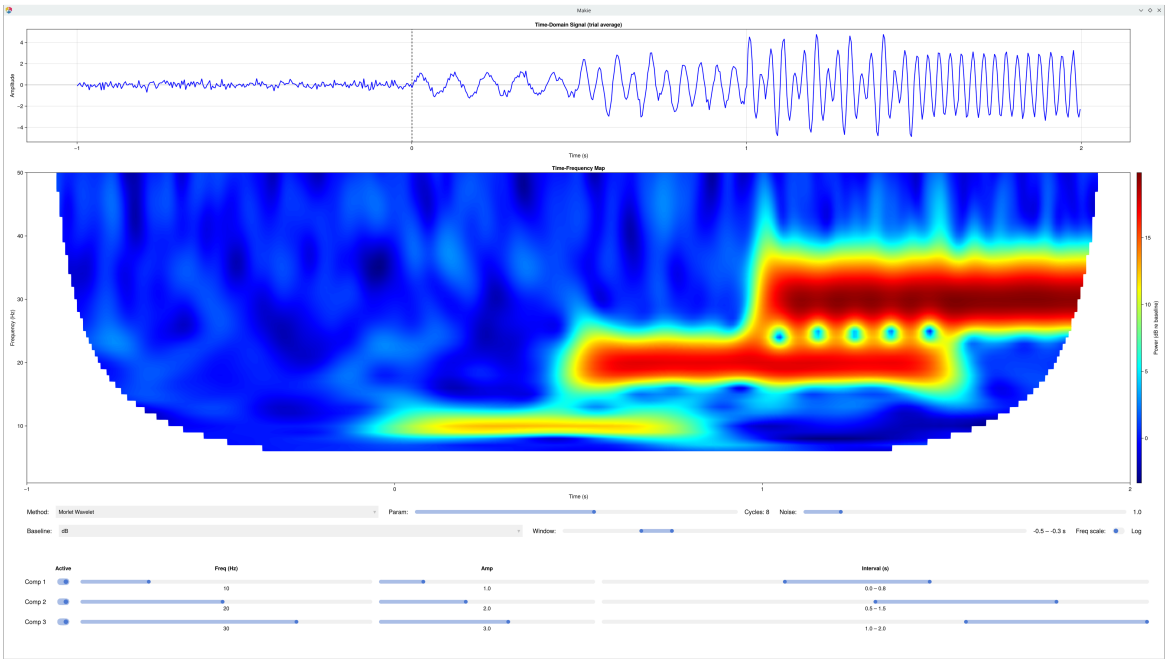

Figure 30.5: Signal Example 3

What it shows

Row	Plot	Description
1	Time Domain	Synthetic signal averaged across 20 trials. Dashed line = t = 0.
2	Time-Frequency Map	Power heatmap from the selected method, optionally baseline-corrected.

Up to three independent frequency components can be active simultaneously, each with its own onset–offset interval. This lets you directly compare how each method handles multiple overlapping signals with different frequencies and timings.

The Core Idea: Heisenberg's Uncertainty Principle

You cannot have perfect time *and* frequency resolution simultaneously — narrowing one always blurs the other. Every TF method is a choice about where to make that trade-off.

Method	Window	Resolution trade-off
Morlet Wavelet	Gaussian, adaptive	At 10 Hz: wide window → sharp frequency, poor time. At 40 Hz: narrow window → sharp time, blurry frequency.

		The window scales automatically with frequency.
STFT (Hanning)	Hanning, fixed	Same window length for all frequencies → uniform resolution. 233 ms window gives 4 Hz frequency resolution at every band.
Multitaper (DPSS)	Slepian tapers, adaptive	Like Morlet but averages across multiple orthogonal tapers → lower variance estimates at the cost of slight frequency smoothing.

> **Cohen (2014), Ch. 12–13** — Morlet Wavelets and Wavelet Convolution **Cohen (2014), Ch. 15** — Short-time FFT **Cohen (2014), Ch. 16** — Multitaper

Baseline Normalization

Raw TF power cannot be compared across methods, frequencies, or participants because absolute power varies with electrode impedance, distance from the source, and method-specific scaling. Baseline normalization expresses power *relative to a reference period* (typically pre-stimulus).

Method	Formula	Range	When to use
None	raw power	unbounded	Comparing absolute power (rarely appropriate)
dB	$10 \cdot \log_{10}(P / BL)$	unbounded, 0 at baseline	Most robust for EEG — log compression handles large dynamic range
% Change	$100 \cdot (P - BL) / BL$	unbounded, 0 at baseline	Intuitively interpretable scale
Z-score	$(P - BL_{mean}) / BL_{std}$	z-units	Group-level comparison; requires stable baseline std
Relative	P / BL_mean	ratio, 1 at baseline	Clean background; can't compare across frequencies
Rel. Change	(P − BL) / BL	ratio, 0 at baseline	Like % Change without the ×100

Norm. Change	(P − BL) / (P + BL)	−1 to +1	Sensitive to small fluctuations in low-SNR regions
Absolute	P − BL	same units as power	Rarely used; frequency-dependent bias

The baseline window defaults to −1.0 − 0.0 s (pre-stimulus). Move it with the IntervalSlider — it can be placed anywhere in the epoch.

> **Cohen (2014), Ch. 18** — Time-Frequency Power and Baseline Normalizations

Things to Try

1. The resolution trade-off (Morlet)

Enable Comp 1 (10 Hz, 0.0 − 0.8 s), baseline = dB. Sweep the Param slider from 3 to 12 cycles: watch the blob grow narrow in frequency but wide in time at high cycles, and vice versa. This is Heisenberg in action.

2. Morlet vs. STFT on the same signal

Add Comp 3 (40 Hz, 1.0 − 2.0 s). Switch between Morlet and STFT with Param = 7. Notice:

- STFT: all three component blobs are the same rectangular shape regardless of frequency.
- Morlet: the 10 Hz blob is wider in time, the 40 Hz blob is narrower — because the window adapts.

3. STFT onset artifact

Set Param to 3 (short window = 100 ms), enable only Comp 1, set baseline = None. Switch to STFT. The "flame" pattern at onset is real physics: as the fixed Hanning window slides past the signal's hard edge at t = 0, it captures successive phases of the 10 Hz oscillation, producing structured amplitude modulation across the transition. Switch to Morlet — the Gaussian taper handles the onset far more gracefully.

4. Multitaper variance reduction

Enable all three components with a moderate noise (Noise = 2.0). Toggle rapidly between Morlet and Multitaper: the Multitaper map is smoother because it averages independent spectral estimates from multiple orthogonal Slepian tapers, reducing random variance without introducing the bias that additional smoothing would.

5. Baseline normalization sensitivity

Set three components active and baseline = dB. Sweep the baseline window across the stimulus period: you can clearly see the baseline reference contaminating the normalization when signal is included. Move it back to pre-stimulus only to see clean dB increases.

6. Log frequency axis

Enable the Log toggle. Low frequencies are spread out — revealing fine structure in the 1–10 Hz range — while high frequencies compress. This matches the 1/f power distribution of real EEG.

Controls

Control	Range	Description
Method	Morlet / STFT / Multitaper	TF decomposition method
Param	3–12	Cycles (Morlet/Multitaper) or window ≈ N/30 Hz in ms (STFT)
Noise	0.5–5	Amplitude of additive white noise per trial
Baseline	None / dB / % Change / …	Baseline correction method (all EegFun `tf_baseline` types)
Window	−1.0–2.0 s	Baseline time window (IntervalSlider)
Freq scale	Linear / Log	Toggle $\log_{10}$ y-axis on the TF map
☐ Comp 1/2/3	on/off	Enable/disable each component
Freq (×3)	1–40 Hz	Component frequency
Amp (×3)	0–5	Component amplitude
Interval (×3)	−1.0–2.0 s	Component onset–offset window (IntervalSlider)

See Also

- Cohen, M. X. (2014). *Analyzing Neural Time Series Data.* MIT Press.
- Section III: Frequency and Time-Frequency Domains Analyses
- Signal Example — Sampling — Nyquist theorem and sampling
- Signal Example — Composition — signal composition and filtering
- Signal Example — Convolution — sliding kernels including Morlet wavelets

Code

```
using EegFun
EegFun.signal_example_tf()
```

Chapter 31

ICA

31.1 CLT

Signal Example — ICA 1: Central Limit Theorem (CLT)

Before jumping into the heavy geometry and optimization algorithms of ICA, we need to answer a fundamental question: **How does an algorithm mathematically know when it has successfully unmixed a signal?**

> [!TIP] This demo bridges the gap between the concept of mixtures and the objective function the ICA algorithm uses to untangle them.

The Central Limit Theorem in EEG

The Central Limit Theorem (CLT) states that if you take multiple independent variables and add them together, the distribution of their *sum* will always be **more Gaussian** (more like a normal bell-curve) than any of the original variables.

Brain sources (like a blink artifact, a heart beat, or a rigid alpha burst) are usually highly structured, repeating events. Statistically, they are very "pointy" (Super-Gaussian) or very "flat" (Sub-Gaussian). They do not look like perfect bell curves.

However, because a scalp electrode records a huge volume-conducted mixture of hundreds of these independent brain, muscle, and eye sources, it effectively sums them all up. Because of the CLT, the resulting EEG signal on your screen looks incredibly Gaussian!

The Core Logic of ICA

Because of the Central Limit Theorem, ICA algorithms operate on one beautifully simple reverse rule: **Mixtures are Gaussian. True sources are not.**

If an algorithm wants to find a pure brain source, all it has to do is computationally rotate the data backwards until the resulting distribution stops looking like a bell curve! This is why you will hear the phrase "ICA maximizes Non-Gaussianity" or "ICA maximizes Kurtosis".

The Interactive Sandbox

This demonstration is the purest possible physical proof of the Central Limit Theorem.

- It initializes a single flat distribution (Sub-Gaussian, Kurtosis = −1.20).
- The slider lets you physically add more independent flat distributions together.
- Adding just 2 shapes forms a triangle.
- Adding 5+ shapes instantly forces the histogram perfectly into a mathematically smooth Gaussian Bell Curve (kurtosis dropping to 0.0).

This conclusively proves the statistical behavior that makes Blind Source Separation possible.

Code

```
using EegFun
EegFun.signal_example_ica_clt()
```

31.2 Mixture

Signal Example — ICA 2: What Is a Mixture?

Often in EEG analysis, we fall into the trap of thinking about a specific channel (like `Fz` or `Oz`) as measuring a specific brain area. But due to physics, the electrical potentials generated by the brain smear and conduct across the scalp.

What you record at an electrode isn't a single "source" — it is a **mixture** of many sources added together.

> [!TIP] This is Part 2 of the ICA pedagogical sequence. It formally proves why we need an algorithm to separate our data.

The Forward Problem

This demonstration creates two distinct underlying components:
1. **Source 1**: A healthy brain oscillation (alpha/spindle bursts).
2. **Source 2**: A massive blink artifact.

This minimalist demo serves as a sandbox to prove that **an electrode is not a brain signal; it is a mixture**.

You can adjust the **Mix Balance** slider to smoothly sweep from seeing 100% pure brain signal, to a 50/50 mix, all the way to 100% blink artifact. This proves exactly how a single scalp electrode physically crossfades and scales electrical potentials based on distances and conductivities. This creates the exact "forward-problem" that ICA aims to solve backward!

Code

```
using EegFun
EegFun.signal_example_ica_mixture()
```

See Also

- Part 1: Central Limit Theorem — Why mixing creates Gaussian distributions
- Part 3: Mixing & Unmixing — Two electrodes, mixing matrices, and the ICA algorithm
- Part 4: 3 Sources & Scatter Geometry — Advanced: rotation geometry, scatter plots, kurtosis
- Part 5: Sphering (Whitening) — Why ICA optimization is just finding an angle
- Part 6: Inside the Black Box — How the Gradient Ascent math actually loops

31.3 Separation

Signal Example — ICA 3: Mixing & Unmixing

Interactive demonstration of **signal mixing and ICA unmixing**.

The Cocktail Party Problem

Imagine you're at a party with two people talking. You have two microphones, but each one picks up *both* voices — just at different volumes depending on how close each person is to each mic.

That's exactly what happens with EEG:

- Your **brain** generates an electrical signal
- Your **eyes** produce large signal changes every time you blink
- Each **electrode** on your scalp picks up a *mixture* of both

The result? Every electrode's recording is a jumbled combination of brain activity and blink artifacts. You never see the clean sources directly.

ICA (Independent Component Analysis) is the algorithm that untangles the mixture back into the original sources — like being able to isolate each person's voice from the party recording.

What This Demo Shows

Column	What you see
Left — "What's really happening"	The two true source signals: a brain oscillation and eye blink artifacts
Middle — "What electrodes record"	The mixed signals, with the mixing equation shown in the title (e.g. `E1 = 1.00 × Brain + 0.50 × Blink`)
Right — "What ICA recovers"	The unmixed signals recovered by ICA

Below the plots, three panels show the **Matrix Equations**:

1. **Mixing M**: How sources `S` become electrodes `E` (`M × S = E`).
2. **Exact Inverse M^{-1}**: The mathematically perfect "reverse recipe".
3. **Your Weights / ICA Found**: The current unmixing recipe being applied.

Things to Try

Step 1: See the Problem

1. Set **Mix Amount** to 0.5. Look at the middle column — the blink spikes are now mixed into the brain electrode (E1), and brain oscillations contaminate the blink electrode (E2).
2. Look at the **Mixing M** panel. It's a colored 2×2 grid showing exactly how much each source bleeds into each electrode.

Step 2: Try to Fix It Yourself

1. Use the **"Subtract E2 from E1"** slider. You're trying to remove the blink contamination from electrode 1 by subtracting a scaled version of electrode 2.
2. Watch the **Shape Match** — can you get to 100%? It's surprisingly hard!
3. Notice that even if you get 100% Shape, the **Amplitude Match** might be low. This is because a simple subtraction doesn't correctly invert the whole mixing matrix.

Step 3: Let ICA Do It

1. Click **Unmix!** — ICA finds the optimal 2×2 matrix automatically.
2. Watch the manual sliders *jump* to the values ICA found — now you can see exactly what weights it used.
3. Compare the **ICA Found Matrix** with the **Exact Inverse M^{-1}** — ICA essentially "reverse engineered" the mixing recipe without ever seeing the original sources!

Controls

Control	Range	Description
Mix Amount	0–1	How much the sources bleed across electrodes
Brain Freq	4–20 Hz	Frequency of the oscillatory brain signal
Blink Size	0.5–5	Amplitude of eye blink artifacts
Noise	0–1	Additive sensor noise
Subtract E2⟶E1	0–1.5	Manual unmixing: remove E2 contribution from E1
Subtract E1⟶E2	0–1.5	Manual unmixing: remove E1 contribution from E2
Unmix!	—	Run ICA to find optimal unmixing weights
Reset	—	Clear all unmixing

Why Does ICA Work?

ICA relies on one key insight: **brain signals and eye blinks are statistically independent** — knowing the state of one tells you nothing about the state of the other.

The Central Limit Theorem says that mixing independent, non-Gaussian sources always produces something *more* Gaussian (more random-looking) than either source alone. ICA exploits this in reverse: it searches for the unmixing matrix that makes the outputs *as non-Gaussian as possible* — which turns out to be equivalent to making them statistically independent.

> [!TIP] This demo shows that unmixing is just **Matrix Inversion** — see also Part 4: ICA with Scatter Geometry.

See Also

- Part 1: Central Limit Theorem — Why mixing creates Gaussian distributions
- Part 2: What Is a Mixture? — Start here if you've never seen ICA
- Part 4: 3 Sources & Scatter Geometry — Advanced: rotation geometry, scatter plots, kurtosis
- Part 5: Sphering (Whitening) — Why ICA optimization is just finding an angle
- Part 6: Inside the Black Box — How the algorithm actually finds the sources
- ICA Demo — Full ICA workflow on real EEG data
- Signal Example — Composition — Building complex waveforms from sine waves
- EEGLAB: Running ICA — Practical guide to ICA in EEGLAB
- ICA for Dummies — Arnaud Delorme's accessible introduction to ICA

Code

```
using EegFun
EegFun.signal_example_ica_separation()
```

31.4 Geometry

Signal Example — ICA 4: Blind Source Separation

Interactive demonstration of **Independent Component Analysis (ICA)** — 3 sources, rotation geometry, and scatter plots.

> [!TIP] This is Part 4 of the ICA series. Start with Part 2: What Is a Mixture? or Part 3: Mixing & Unmixing first.

What it shows

Feature	Description
True Sources (Row 1)	Three independent signals: S1 (oscillatory bursts, Gabor-shaped), S2 (blink artifacts), S3 (muscle burst)
EEG Recordings (Row 2)	The mixed signals as they would appear at elec trodes — each is a weighted combination of all sources
Recovered Components (Row 3)	The unmixed signals, either via manual rotation sliders or the Auto-ICA (Infomax) algorithm
Scatter Plots	Three pairwise joint distributions (S1 vs S2, S1 vs S3, S2 vs S3) — see below

Excess Kurtosis	Displayed for sources and recovered components

The Scatter Plots

The scatter plots are the most informative panel:

- **Independent, non-Gaussian signals** → distinctive **cross or T-shape** (the signals are statistically independent)
- **Mixed signals** → correlated **blob** (everything jumbled together)
- **Correctly unmixed** → cross shape restored

Things to Try

1. Set all mixing angles > 0° and watch the scatter plots collapse from crosses into blobs.
2. Drag the unmixing sliders manually to match the mixing angles — the sources reappear.
3. Reset unmixing angles to 0°, then click **Infomax** — ICA automatically recovers the sources.
4. Increase the **Noise** slider — Infomax still works because the blink and muscle sources are highly non-Gaussian.
5. Compare manual rotation with Infomax — Infomax finds a general unmixing matrix, not just a rotation.
6. Try **Extended** Infomax, which handles both sub-Gaussian and super-Gaussian sources.

Controls

Control	Range	Description
Mix α (S1-S2)	0–90°	Rotation angle blending S1 and S2 into the recordings
Mix β (S1-S3)	0–90°	Rotation angle blending S1 and S3 into the recordings
Mix γ (S2-S3)	0–90°	Rotation angle blending S2 and S3 into the recordings
Unmix φ (S1-S2)	0–90°	Inverse rotation to recover S1/S2 (manual mode)
Unmix ψ (S1-S3)	0–90°	Inverse rotation to recover S1/S3 (manual mode)
Unmix χ (S2-S3)	0–90°	Inverse rotation to recover S2/S3 (manual mode)
Burst Freq	1–20 Hz	Frequency of the oscillatory source (S1)
Noise	0–1	Additive Gaussian noise on all sources

Infomax	—	Run standard Infomax ICA (Bell & Sejnowski, 1995)
Extended	—	Run Extended Infomax (Lee et al., 1999)

The Mixing Model

With N sources and N sensors, mixing is an N×N matrix multiplication. For 3 sources, we compose three pairwise 2D rotations (α, β, γ):

Mixing: $R_{23}(\gamma) \cdot R_{13}(\beta) \cdot R_{12}(\alpha) \cdot S$ **Unmixing**: $R_{12}^{\intercal}(\varphi) \cdot R_{13}^{\intercal}(\psi) \cdot R_{23}^{\intercal}(\chi) \cdot M$

When $\varphi=\alpha$, $\psi=\beta$, $\chi=\gamma \longrightarrow$ perfect recovery.

Why Non-Gaussianity?

ICA exploits the **Central Limit Theorem in reverse**: mixtures of independent signals are *more Gaussian* than the original signals. So finding the unmixing that maximises non-Gaussianity (measured by kurtosis or negentropy) recovers the independent sources.

- **Excess kurtosis = 0**: Gaussian distribution
- **Excess kurtosis > 0**: Super-Gaussian (sharp peaks, heavy tails — like blink spikes)
- **Excess kurtosis < 0**: Sub-Gaussian (flat-topped distributions)

The Infomax algorithm (Bell & Sejnowski, 1995) used in EegFun maximises the total non-Gaussianity of the output components.

- Part 1: Central Limit Theorem — Deep Dive: Why mixing creates Gaussian blobs
- Part 2: What Is a Mixture? — Simplest introduction to the problem
- Part 3: Mixing & Unmixing — 2-source introduction to ICA
- Part 5: Sphering (Whitening) — Why ICA optimization is just finding an angle
- Part 6: Inside the Black Box — The optimization landscape and gradient ascent
- ICA Demo — Full ICA workflow on real EEG data
- EEGLAB: Running ICA — Practical guide to ICA in EEGLAB
- ICA for Dummies — Arnaud Delorme's accessible introduction to ICA
- Bell, A. J., & Sejnowski, T. J. (1995). An information-maximization approach to blind separation. *Neural Computation*, 7(6), 1129–1159.
- Hyvärinen, A., & Oja, E. (2000). Independent component analysis: algorithms and applications. *Neural Networks*, 13(4-5), 411–430.

Code

```
using EegFun
EegFun.signal_example_ica_geometry()
```

31.5 Sphering

Signal Example — ICA 5: Sphering (Whitening)

This interactive visualization reveals the hidden mathematical shortcut that makes ICA possible: **Sphering**.

> [!TIP] This is Part 5 of the ICA series. It bridges the gap between the correlated mixtures of Part 4 and the optimization search space of Part 6.

The Problem

If you look at raw EEG mixtures, they form skewed, stretched, off-center blobs. If an algorithm blindly tried to untangle that, it would have to calculate huge mathematical matrices involving stretching, skewing, shifting, and rotating all simultaneously.

That is too difficult for a fast optimization algorithm.

The Solution: Sphering

Instead of doing everything at once, all modern ICA algorithms apply a standard "Preprocessing" pipeline first called Sphering (or Whitening).

This demo lets you physically drag the data through these three exact mathematical steps:

1. **Centering**: Subtract the mean value so the data block is physically anchored exactly precisely on `(0,0)`.
2. **PCA Rotation**: Run Principal Component Analysis to find the longest arms of the blob, and physically rotate the blob so it aligns securely with the X and Y axes. This removes any cross-correlation.
3. **Variance Scaling (Sphering)**: Shrink or stretch the X and Y axes depending on their variance until the entire blob is a perfect mathematical sphere.

Why Do We Do This?

Because once the data is transformed into a perfect sphere, the algorithm's job is over functionally: the only property left to search is **Rotation**.

By dragging the "Morph" slider all the way to 3.0 (Sphered), you are setting up the perfect circular search space. This directly leads into Part 6: Inside the Black Box, where we will simply rotate a line through this sphered data to locate the independent components!

Code

```
using EegFun
EegFun.signal_example_ica_sphering()
```

See Also

- Part 4: 3 Sources & Geometry — Scatter plots and correlation blobs
- ICA: Central Limit Theorem — Why mixed blobs become Gaussian
- Part 6: Inside the Black Box — Gradient ascent rotation

31.6 Optimization

Signal Example — ICA 6: Inside the Black Box

This interactive visualization demystifies the ICA algorithm by showing how it physically searches for independent sources by treating it as an optimization ("hill climbing") problem.

> [!TIP] This is Part 6 of the ICA series. Before attempting this, ensure you understand why the search space is purely rotational by looking at **Part 5: Sphering**.

What is the Black Box doing?

Earlier demos showed that ICA "magically" finds an unmixing matrix to recover the sources. Mathematically, it does this by making the projected data as **non-Gaussian** as possible.

The "Central Limit Theorem in reverse" tells us:

- Mixed signals are more Gaussian (like a standard bell curve).
- The original independent sources are non-Gaussian (often "peaky" or super-Gaussian, having high Kurtosis).

So to find the sources, ICA just rotates its unmixing weights, checks the "peakiness" (Kurtosis or Entropy), and keeps turning in the direction where it gets more peaky.

The Search Space (The Landscape)

This demo removes time entirely and just looks at the distribution of the mixed values:

1. **The Search Space (Sphered Data)**: Instead of plotting $x_1(t)$ against t, we plot $x_1(t)$ against $x_2(t)$. You'll notice the data is "Sphered" (or whitened). Sphering is a preprocessing step that forces the mixture blob to be perfectly round (variance of 1 in all directions). By perfectly rounding out the data first, ICA's job is massively simplified: it only has to **rotate** the line to find the sources, rather than stretching and squishing it as well. The red line $\mathbf{w}$ is our rotatable unmixing vector.
2. **The Resulting 1D Data (Top Right)**: If we collapse the 2D blob down onto the red line, what is the shape of the histogram? When the line cuts through an "arm", the histogram becomes very peaky.
3. **The Objective Landscape (Bottom)**: We can calculate the "Peakiness" (Kurtosis) for *every possible angle* of $\mathbf{w}$ from 0° to 180°. This creates a hill-landscape curve. The red dot represents our current angle.

Things to Try

1. **Manual Search**: Drag the **Angle slider** manually. Watch the red dot move across the landscape. Notice how the histogram peaks strongly precisely when the red dot hits the top of a hill.
2. **Single Step**: Set the angle manually to 15°. Now click **Step Algorithm**. Watch the red line visibly rotate and "climb" up the slope of the hill. This is directly executing the gradient ascent learning rule!
3. **Convergence**: Click **Run to Convergence** to watch the algorithm automatically race to the peak (the isolated source) and stop when the slope is zero.

Code

```
using EegFun
EegFun.signal_example_ica_optimization()
```

Chapter 32

Machine Learning

32.1 Decoding

Simulate Decoding — MVPA Playground

Interactive demo showing time-resolved multivariate pattern classification (MVPA/decoding) on synthetic EEG data.

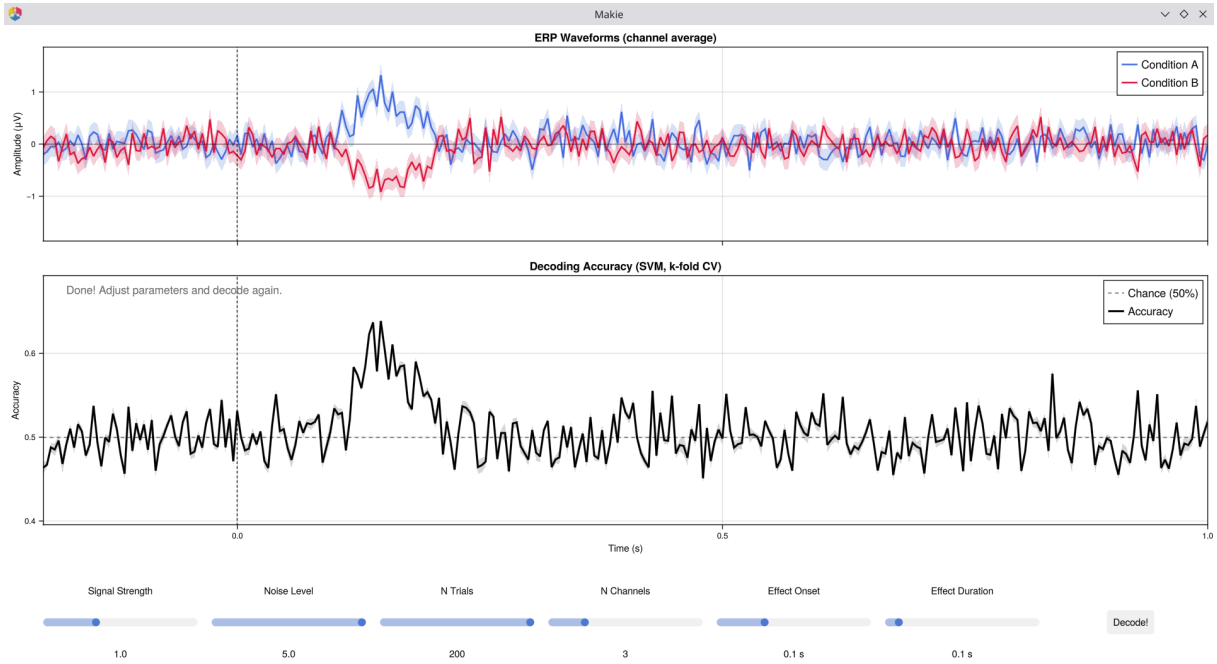

Figure 32.1: Simulate Decoding

What it shows

Element	Description
Blue/Red lines	Mean ERP waveforms for Condition A and B (channel average), with ±1 SE shading
Black line	Time-resolved SVM classification accuracy
Grey dashed line	50% chance level — accuracy here means no decodable information

The core insight: at each time point, a Support Vector Machine (SVM) classifier tries to distinguish the two conditions using the pattern of activity across channels. Accuracy above chance means the brain's response to the two conditions is reliably different at that moment.

Key Concepts

Multivariate vs. univariate: A t-test compares one channel at a time. MVPA considers the *joint pattern* across all channels simultaneously — exploiting spatial correlations that univariate methods miss. This is why more channels can improve decoding even when individual channels show weak effects.

Cross-validation: The data is split into training and test sets. The SVM learns the pattern from training data and is tested on data it has never seen. This prevents **overfitting** — being fooled by noise patterns that don't generalise.

Effect size and SNR: Signal Strength controls how different the two conditions are. With zero signal, accuracy bounces around 50% (chance). As signal increases, accuracy rises. But noise fights back — high noise can bury even a strong signal if you don't have enough trials.

Things to Try

1. **Default settings:** press Decode! — you should see accuracy rising above chance in the effect window.
2. **Kill the signal:** set Signal Strength = 0, decode ⟶ accuracy fluctuates around 50%. This is the null distribution.
3. **Crank the signal:** set Signal Strength = 3 ⟶ accuracy approaches 100%.
4. **Multivariate advantage:** set N Channels = 1, decode. Then set N Channels = 10, decode again — more channels = higher accuracy.
5. **Trial count matters:** set N Trials = 10 ⟶ noisy, unreliable curve. Set N Trials = 200 ⟶ smooth, stable curve.
6. **Timing information:** shift Effect Onset to 0.5 s ⟶ accuracy peak shifts to match. Change Duration to see the peak widen or narrow.

Controls

Control	Range	Description
Signal Strength	0–3	Amplitude of the condition difference
Noise Level	0.1–5	Background noise amplitude
N Trials	10–200	Trials per condition
N Channels	1–10	Number of EEG channels (SVM features)
Effect Onset	−0.2–0.8 s	When the condition difference begins
Effect Duration	0.05–0.8 s	How long the condition difference lasts
Decode!	button	Runs the SVM classification

See Also

- Simulate ERP — how trial averaging extracts signals from noise
- Signal Example — Composition — signal composition and filtering
- Signal Example — Time-Frequency — time-frequency analysis methods
- Grootswagers, T., Wardle, S. G., & Carlson, T. A. (2017). Decoding dynamic brain patterns from evoked responses: a tutorial on multivariate pattern analysis applied to time series neuroimaging data. *Journal of Cognitive Neuroscience*, 29(4), 677–697.

Code

```
using EegFun
EegFun.signal_example_decoding()
```

Chapter 33

ERP

33.1 Simulate ERP

Simulate ERP — Signal Averaging Demo

Interactive ERP simulator for teaching how trial averaging extracts signals from noise.

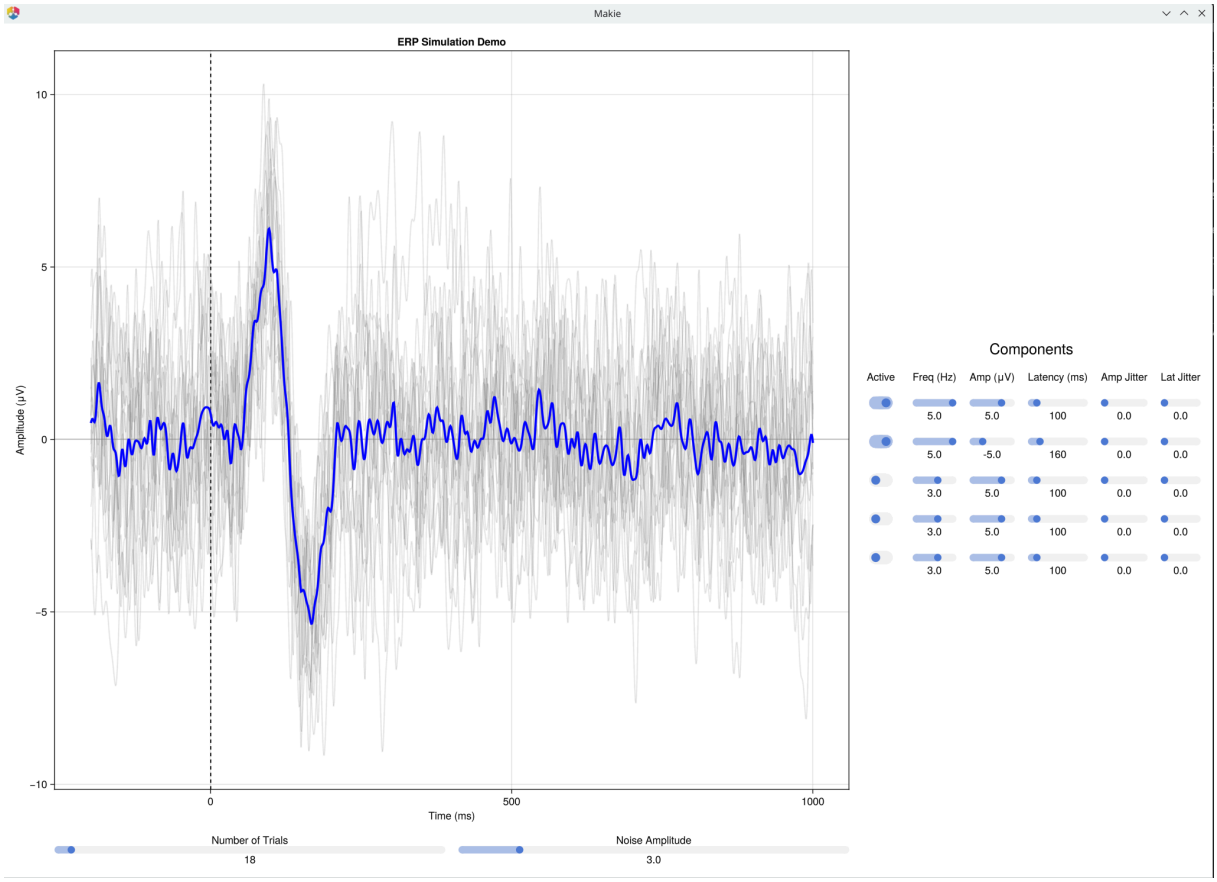

Figure 33.1: Simulate ERP

What it shows

Element	Description
Grey lines	Individual simulated EEG trials with realistic 1/f background noise
Blue line	Trial-average ERP — clarifies as more trials are added

Components	Up to 5 independent ERP components, each shaped as a single cosine lobe (the central peak of a cosine wave, masked to ±π/2) to produce a smooth, bell-like waveform

The core insight: The signal-to-noise ratio of the ERP average scales with √N (where N is the number of trials). Doubling the number of trials improves SNR by 41%; to *halve* the noise, you need 4× as many trials.

The background noise uses a realistic human EEG power spectrum (1/f structure), not white noise — so individual trials look like real EEG epochs rather than random static.

This demo builds directly on Signal Example 2: the ERP waveform you see in the average is a real-world instance of multiple frequency components summing together, now embedded in realistic noise.

Things to Try

- Start with 1 trial and a single active component — the ERP equals the single trial.
- Add noise and increase trials to watch the ERP "emerge" from the background activity.
- Activate a second component at a different latency to show how components superpose — and try setting one to a **negative amplitude** to model typical ERP polarities (N1 negative, P3 positive).
- Introduce **latency jitter** to show how trial-to-trial variability smears and attenuates the average — a key confound in real ERP research.
- Introduce **amplitude jitter** to show how variability in peak amplitude scales the average downward.

Controls

Each component has its own row of controls in the right panel.

Control	Range	Description
Number of Trials	1–500	Trials to average
Noise Amplitude	0–20	Background 1/f EEG noise level
Active toggle (×5)	on/off	Enable or disable each component
Freq (×5)	0.1–5.0 Hz	Component shape (cosine frequency)
Amp (×5)	−10 to 10 μV	Peak amplitude (negative = typical N-wave polarity)
Latency (×5)	0–1000 ms (steps: 10 ms)	Peak latency
Amp Jitter (×5)	0–20 (Gaussian SD)	Trial-to-trial amplitude variability

Lat Jitter (×5)	0–50 ms (Gaussian SD)	Trial-to-trial latency variability

See Also

- Luck, S. J. (2014). *An Introduction to the Event-Related Potential Technique* (2nd ed.). MIT Press. — Chapter 8 (averaging and signal-to-noise ratio)
- Signal Example — Sampling — Nyquist theorem and signal reconstruction
- Signal Example — Composition — signal composition and filtering
- Simulate Decoding — multivariate pattern classification on synthetic EEG

Code

```
using EegFun
EegFun.simulate_erp()
```

Chapter 34

Psychophysiology (EOG / EMG / ECG)

34.1 EOG Dipole & Derivation

EOG — Corneo-Retinal Dipole & Bipolar Derivation Demo

Interactive Electrooculography (hEOG) simulator illustrating how horizontal eye rotations re-orient the corneo-retinal standing dipole and generate deflections in a bipolar difference amplifier.

What it shows

- **Corneo-retinal standing potential**: The cornea is positive (+) relative to the retina (–), forming a 0.4–1.0 mV dipole in each eye.
- **Bipolar derivation**: Right outer canthus ($V_{(+)}$) minus left outer canthus ($V_{(-)}$) cancels common-mode background EEG while doubling opposite-polarity eye-movement potentials.
- **Saccade playback**: Real-time saccade animation (0° ⟶ target gaze angle ⟶ 0°) with synchronized waveform playhead and optional dipole electric field lines.

Controls

Control	Description
Gaze Angle	Rotation angle from −45° (left) to +45° (right)
Duration	Saccade fixation hold duration (100–1200 ms)
Noise	Physiological baseline noise (0–100 μV)
Play Saccade	Animates a real-time saccade sweep
Field Lines	Toggles bilateral dipole electric field contours
Monopolar (CMR)	Overlays unipolar right/left temple traces to illustrate Common-Mode Rejection

Code

```
using EegFun
using GLMakie

EegFun.signal_example_heog()
```

34.2 Surface EMG Cascade

Surface EMG — Muscle Activation & Processing Cascade Demo

Interactive Surface Electromyography (sEMG) simulator illustrating motor-unit superposition and the canonical 4-stage processing pipeline: **Raw → Rectify → Filter (RMS) → Integrate**.

What it shows

1. **Raw Surface EMG [µV]**: Zero-mean stochastic interference pattern formed by asynchronous motor-unit action potential trains.
2. **Full-Wave Rectification [µV]**: Absolute value $|V(t)|$ preserving signal energy.
3. **Linear / RMS Envelope [µV]**: Sliding-window smoothing demonstrating the trade-off between temporal resolution and variance.
4. **Integrated EMG [µV·s]**: Cumulative muscle activation across the contraction burst.

Code

```
using EegFun
using GLMakie

EegFun.signal_example_emg()
```

34.3 ECG Einthoven's Triangle

ECG — Einthoven's Triangle & Vectorcardiogram Demo

Interactive Electrocardiography (ECG) simulator illustrating cardiac dipole propagation (P, Q, R, S, T waves), Einthoven's triangle, and bipolar limb leads (Lead I, Lead II, Lead III).

What it shows

- **Cardiac dipole projection**: How a 2D/3D depolarization vector projects onto Lead I (0°), Lead II (+60°), and Lead III (+120°).
- **Mean electrical axis rotation**: Real-time waveform morphology changes across leads as the heart axis shifts from −45° to +120°.
- **Vectorcardiogram loop**: Complete P, QRS, and T loops synchronized with the ECG strip.

Code

```
using EegFun
using GLMakie

EegFun.signal_example_ecg_einthoven()
```

34.4 ECG Heart Rate Variability

ECG — Heart Rate Variability (HRV) & Respiratory Sinus Arrhythmia Demo

Interactive Heart Rate Variability (HRV) simulator demonstrating how respiration modulates beat-to-beat R–R intervals under relaxed (high HRV) versus stressed (low HRV) autonomic states.

What it shows

- **Respiratory Sinus Arrhythmia (RSA)**: Heart rate accelerates during inhalation and decelerates during exhalation.
- **Autonomic flexibility**: Comparing large, flexible R–R oscillations at rest against rigid, low-variability rhythms under sympathetic stress.

Code

```
using EegFun
using GLMakie

EegFun.signal_example_ecg_hrv()
```

Part VI

Cheatsheet

Chapter 35

Cheatsheet

Quick reference for common EEG processing tasks in EegFun.jl.

35.1 Data Import

I want to...	Function
Load a BioSemi .bdf file	`EegFun.read_raw_data("file.bdf")`
Load a BrainVision file	`EegFun.read_raw_data("file.vhdr")`
Load an EDF/EDF+ file	`EegFun.read_raw_data("file.edf")`
Load a Neuromag/MNE .fif file	`EegFun.read_raw_data("file.fif")`
Load an XDF file	`EegFun.read_xdf("file.xdf")`
Load an EEGLAB file	`EegFun.read_eeglab("file.set")`
Load a FieldTrip .mat file	`EegFun.read_fieldtrip("file.mat")`
Load a BIDS run	`EegFun.read_bids("bids_root", subject="01", task="posner")`
Load a channel layout	`EegFun.read_layout("layout.csv")`
Create EegFun data structure	`EegFun.create_eegfun_data(raw, layout)`

35.2 Data Persistence (JLD2)

I want to...	Function
Save data to JLD2	`jldsave("file.jld2"; data = dat)`
Load data from JLD2	`load("file.jld2", "data")`
Smart-load EegFun data	`EegFun.read_data("file.jld2")`
Batch load all matching files	`EegFun.read_all_data("dir/pattern")`
Group loaded data by condition	`EegFun.group_by_condition(data)`

35.3 Preprocessing

I want to...	Function

Rereference to average	`EegFun.rereference!(dat, :avg)`
Rereference to specific channels	`EegFun.rereference!(dat, [:M1, :M2])`
High-pass filter	`EegFun.highpass_filter!(dat, 0.1)`
Low-pass filter	`EegFun.lowpass_filter!(dat, 30.0)`
Bandpass filter (sequential)	`EegFun.highpass_filter!(dat, 0.1); EegFun.lowpass_filter!(dat, 30.0)`
Resample data	`EegFun.resample!(dat, 256)`
Apply baseline correction	`EegFun.baseline!(dat, (-0.2, 0.0))`
Create channel difference	`EegFun.channel_difference!(dat, channel_selection1=EegFun.channels([:C3]), channel_selection2=EegFun.channels([:C4]), channel_out=:laterality)`
Average channels into ROI	`EegFun.channel_average!(dat, channel_selections=[EegFun.channels([:Fz, :Cz, :Pz])], output_labels=[:midline])`
Delete channels	`EegFun.channel_delete!(dat, EegFun.channels([:M1, :M2]))`
Mark extreme values	`EegFun.is_extreme_value!(dat, 100)`
View trigger counts	`EegFun.trigger_count(dat)`
Search trigger sequences	`EegFun.search_sequence(dat.data.trigger, [1, 2])`

35.4 ICA

I want to...	Function
Run ICA decomposition	`EegFun.run_ica(dat)`
Identify EOG components	`EegFun.identify_eog_components(dat, ica)`
Identify ECG components	`EegFun.identify_ecg_components(dat, ica)`
Identify line noise components	`EegFun.identify_line_noise_components(dat, ica)`
Remove components	`EegFun.subtract_ica_components!(dat, ica, component_selection=EegFun.components([1, 3]))`

Restore components	`EegFun.add_ica_components!(dat, ica, component_selection=EegFun.components([1]))`

35.5 Epoching

I want to…	Function
Define epoch conditions	`cfg = [EegFun.EpochCondition(name="Cond1", trigger_sequences=[[1]])]`
Dry-run epoch count	`EegFun.epochs_count(dat, cfg, (-0.2, 1.0))`
Extract epochs	`EegFun.extract_epochs(dat, cfg, (-0.2, 1.0))`
Print epoch count table	`EegFun.epochs_table(epochs)`
Baseline correct epochs	`EegFun.baseline!(epochs, (-0.2, 0.0))`
Detect bad epochs automatically	`EegFun.detect_bad_epochs_automatic(epochs)`
Detect bad epochs interactively	`EegFun.detect_bad_epochs_interactive(epochs)`
Reject marked epochs	`EegFun.reject_epochs(epochs)`
Log before/after rejection table	`EegFun.log_epochs_table(before, after)`
Repair artifacts (interpolation)	`EegFun.repair_artifacts(epochs)`

35.6 ERP Operations

I want to…	Function
Average epochs into ERPs	`EegFun.average_epochs(epochs)`
Combine conditions	`EegFun.condition_combine(erps, [[1, 2]])`
Subtract conditions	`EegFun.condition_difference(erps, [(1, 2)])`
Average conditions	`EegFun.condition_average(erps, [[1, 2]])`
Compute grand average	`EegFun.grand_average(all_erps)`
Jackknife average	`EegFun.jackknife_average(all_erps)`

Compute GFP	`EegFun.gfp(erps)`
Realign epochs to response trigger	`EegFun.realign!(epochs, [201])`
Compute LRP	`EegFun.lrp(erps, channel_selection1=EegFun.channels([:C3]), channel_selection2=EegFun.channels([:C4]))`

35.7 ERP Measurements & Export

I want to…	Function
Extract mean amplitude (in-memory)	`EegFun.erp_measurements(erps, "mean_amplitude", analysis_interval=(0.3, 0.5))`
Extract mean amplitude (from dir)	`EegFun.erp_measurements("erps", "mean_amplitude", analysis_interval=(0.3, 0.5))`
Extract peak amplitude	`EegFun.erp_measurements(erps, "max_peak_amplitude", analysis_interval=(0.3, 0.5))`
Extract peak latency	`EegFun.erp_measurements(erps, "max_peak_latency", analysis_interval=(0.3, 0.5))`
Explore measurements in GUI	`EegFun.plot_erp_measurement_gui(erps)`

35.8 Time-Frequency Analysis

I want to…	Function
Morlet wavelet decomposition	`EegFun.tf_morlet(epochs, frequencies=4:1:30, cycles=3)`
Multitaper decomposition	`EegFun.tf_multitaper(epochs, frequencies=4:1:30)`
STFT decomposition	`EegFun.tf_stft(epochs, frequencies=4:1:30)`
Baseline correct TF data	`EegFun.tf_baseline!(tf, (-0.2, 0.0); method=:db)`
TF channel average	`EegFun.channel_average!(tf, channel_selections=[EegFun.channels([:Fz, :Cz])], output_labels=[:frontal])`
TF channel difference	`EegFun.channel_difference!(tf, channel_selection1=EegFun.channels([:C3]), channel_selection2=EegFun.channels([:C4]))`
TF condition difference	`EegFun.condition_difference(tf_data, [(1, 2)])`

TF condition average	`EegFun.condition_average(tf_data, [[1, 2]])`
TF grand average	`EegFun.grand_average(all_tf)`

35.9 Statistics

I want to…	Function
Prepare group data for stats	`EegFun.prepare_stats("erps", :paired, condition_selection=EegFun.conditions([1, 2]))`
Run analytic t-test	`EegFun.analytic_test(stat_data)`
Run permutation test	`EegFun.permutation_test(stat_data, n_permutations=1000, alpha=0.05, cluster_alpha=0.05)`
Test decoding against chance	`EegFun.test_against_chance(decoded_list)`
Cluster permutation (decoding)	`EegFun.test_against_chance_cluster(decoded_list)`

35.10 Plotting

I want to…	Function
Browse continuous data	`EegFun.plot_databrowser(dat)`
Browse with ICA	`EegFun.plot_databrowser(dat, ica)`
Plot ERPs	`EegFun.plot_erp(erps, channel_selection=EegFun.channels(:Pz))`
Plot topography	`EegFun.plot_topography(erps, interval_selection=EegFun.times(0.3, 0.5))`
Plot ERP statistics	`EegFun.plot_erp_stats(result, channel_selection=EegFun.channels(:Pz))`
Plot TF power	`EegFun.plot_tf(tf, channel_selection=EegFun.channels(:Cz))`
Plot TF statistics	`EegFun.plot_tf_stats(result, channel_selection=EegFun.channels(:Cz))`
Plot TF topography stats	`EegFun.plot_topography_stats(result, freq_range=(8.0, 12.0))`
Plot ICA components	`EegFun.plot_topography(ica)`

Plot epochs grid	`EegFun.plot_epochs(epochs)`
Plot ERP image	`EegFun.plot_erp_image(epochs, channel_selection=EegFun.channels(:Pz))`
Plot GFP	`EegFun.plot_gfp(erps)`
Plot channel layout	`EegFun.plot_layout_2d(layout)`
Plot artifact detection	`EegFun.plot_artifact_detection(epochs)`
Plot frequency spectrum	`EegFun.plot_channel_spectrum(dat)`
Plot filter response	`EegFun.plot_filter_response(dat)`
Plot decoding results	`EegFun.plot_decoding(decoded)`
Plot RSA results	`EegFun.plot_rdm_heatmap(rsa_result)`

35.11 Selection Helpers

I want to select…	Helper
Specific channels	`EegFun.channels([:Fz, :Cz, :Pz])`
All channels except	`EegFun.channels_not([:M1, :M2])`
Channel range	`EegFun.channels(1:32)`
Specific conditions	`EegFun.conditions([1, 2])`
All conditions except	`EegFun.conditions_not([3])`
Time interval	`EegFun.times(0.3, 0.5)`
Specific participants	`EegFun.participants([1, 2, 3])`
All participants except	`EegFun.participants_not([10])`
Specific samples	`EegFun.samples(:is_extreme_value_100)`
All samples except	`EegFun.samples_not(:is_extreme_value_100)`
Specific components	`EegFun.components([1, 3, 5])`

Part VII

Reference

Chapter 36

Overview

36.1 API Reference

Complete documentation for all public EegFun.jl functions and types.

Section	Contents
Analysis	Data processing, analysis, and I/O functions
Plotting	All `plot_*` visualisation functions
Types	Data structures and type definitions

Chapter 37

Analysis

37.1 Analysis Functions

All public data access, I/O, preprocessing, ERP, RIDE, time-frequency, statistical, decoding, RSA, and pipeline functions in `EegFun.jl` .

Index

- EegFun.add_ica_components
- EegFun.add_ica_components!
- EegFun.add_noise_ceiling
- EegFun.add_noise_ceiling!
- EegFun.all_data
- EegFun.all_data
- EegFun.all_labels
- EegFun.amica_ica
- EegFun.analytic_test
- EegFun.apply_analysis_settings
- EegFun.apply_analysis_settings!
- EegFun.autoreject
- EegFun.autoreject!
- EegFun.average_epochs
- EegFun.average_number_of_neighbors
- EegFun.baseline
- EegFun.baseline!
- EegFun.batch_process
- EegFun.calculate_eog_channels
- EegFun.calculate_eog_channels!
- EegFun.calculate_trigger_interval
- EegFun.calculate_trigger_interval!
- EegFun.channel_average
- EegFun.channel_average!
- EegFun.channel_data
- EegFun.channel_delete
- EegFun.channel_delete!
- EegFun.channel_difference

- EegFun.channel_difference!
- EegFun.channel_joint_probability
- EegFun.channel_labels
- EegFun.channel_labels
- EegFun.channel_repairable
- EegFun.channel_repairable!
- EegFun.channel_repairable!
- EegFun.channel_summary
- EegFun.channels
- EegFun.channels_not
- EegFun.check_channel_neighbors
- EegFun.check_raw_data
- EegFun.cleanline
- EegFun.cleanline!
- EegFun.clear_neighbors!
- EegFun.close_global_logging
- EegFun.close_logging
- EegFun.combine_artifact_components
- EegFun.common_channels
- EegFun.compare_models
- EegFun.compare_models!
- EegFun.compare_rejections
- EegFun.components
- EegFun.components_not
- EegFun.compute_csd
- EegFun.compute_csd!
- EegFun.compute_noise_ceiling
- EegFun.condition_average
- EegFun.condition_combine
- EegFun.condition_difference
- EegFun.condition_name
- EegFun.condition_number
- EegFun.condition_parse_epoch
- EegFun.conditions
- EegFun.conditions_not
- EegFun.correlation_matrix
- EegFun.correlation_matrix_dual_selection
- EegFun.correlation_matrix_eog

- EegFun.create_continuous_repair_info
- EegFun.create_custom_layout
- EegFun.create_eegfun_data
- EegFun.create_epoch_repair_info
- EegFun.create_highpass_filter
- EegFun.create_layout
- EegFun.create_lowpass_filter
- EegFun.create_model_rdms
- EegFun.create_rdm
- EegFun.create_temporal_model_rdms
- EegFun.decode_libsvm
- EegFun.detect_bad_epochs_automatic
- EegFun.detect_bad_epochs_interactive
- EegFun.detect_eog_onsets
- EegFun.detect_eog_onsets!
- EegFun.detect_eog_onsets!
- EegFun.detrend
- EegFun.download_eegfun_datasets
- EegFun.duration
- EegFun.epoch_to_continuous
- EegFun.epochs
- EegFun.epochs_count
- EegFun.epochs_not
- EegFun.epochs_table
- EegFun.erp_measurements
- EegFun.erp_measurements!
- EegFun.example_path
- EegFun.export_bids
- EegFun.extra_data
- EegFun.extra_labels
- EegFun.extract_epochs
- EegFun.extract_ride
- EegFun.file_name
- EegFun.find_bridged_channels
- EegFun.find_file
- EegFun.find_nearest_peak
- EegFun.find_times
- EegFun.freq_spectrum

- EegFun.frequency_vector
- EegFun.generate_config_template
- EegFun.generate_permutation_matrix
- EegFun.generate_pipeline_template
- EegFun.generate_signal
- EegFun.get_all_ica_components
- EegFun.get_eog_channels
- EegFun.get_files
- EegFun.get_filter_characteristics
- EegFun.get_neighbors_xy!
- EegFun.get_neighbors_xyz!
- EegFun.get_rejected_epochs
- EegFun.get_selected_channels
- EegFun.get_selected_channels
- EegFun.get_selected_components
- EegFun.get_selected_conditions
- EegFun.get_selected_epochs
- EegFun.get_selected_samples
- EegFun.gfp
- EegFun.gfp_and_dissimilarity
- EegFun.global_dissimilarity
- EegFun.grand_average
- EegFun.group_by_condition
- EegFun.has_2d_coords
- EegFun.has_3d_coords
- EegFun.has_channels
- EegFun.has_neighbors
- EegFun.has_valid_coordinates
- EegFun.head
- EegFun.highpass_filter
- EegFun.highpass_filter!
- EegFun.identify_bad_channels
- EegFun.identify_components
- EegFun.identify_ecg_components
- EegFun.identify_eog_components
- EegFun.identify_line_noise_components
- EegFun.identify_spatial_kurtosis_components
- EegFun.infomax_extended_ica

- EegFun.infomax_ica
- EegFun.is_extreme_value
- EegFun.is_extreme_value!
- EegFun.is_flatline
- EegFun.is_flatline!
- EegFun.is_peak_to_peak
- EegFun.is_peak_to_peak!
- EegFun.is_step_value
- EegFun.is_step_value!
- EegFun.jackknife_average
- EegFun.libsvm_classifier
- EegFun.list_bids_runs
- EegFun.log_epochs_table
- EegFun.lowpass_filter
- EegFun.lowpass_filter!
- EegFun.lrp
- EegFun.mark_epoch_intervals!
- EegFun.meta_data
- EegFun.meta_labels
- EegFun.mirror
- EegFun.mirror!
- EegFun.n_channels
- EegFun.n_epochs
- EegFun.n_extreme_value
- EegFun.n_flatline
- EegFun.n_layout
- EegFun.n_peak_to_peak
- EegFun.n_samples
- EegFun.n_step_value
- EegFun.n_values
- EegFun.neighbor_criterion
- EegFun.neighbors
- EegFun.normalize_rdm
- EegFun.participants
- EegFun.participants_not
- EegFun.partition_channels_by_eog_correlation
- EegFun.permutation_test
- EegFun.picard_ica

- EegFun.polar_to_cartesian_xy!
- EegFun.polar_to_cartesian_xyz!
- EegFun.positions_2D
- EegFun.positions_3D
- EegFun.positions_polar
- EegFun.prepare_decoding
- EegFun.prepare_stats
- EegFun.prepare_stats
- EegFun.prepare_stats
- EegFun.preprocess
- EegFun.print_config
- EegFun.print_filter_characteristics
- EegFun.print_layout_neighbors
- EegFun.read_all_data
- EegFun.read_bids
- EegFun.read_config
- EegFun.read_data
- EegFun.read_eeglab
- EegFun.read_fieldtrip
- EegFun.read_layout
- EegFun.read_raw_data
- EegFun.read_xdf
- EegFun.realign
- EegFun.realign!
- EegFun.reference
- EegFun.reject_epochs
- EegFun.reject_epochs!
- EegFun.rename_channel
- EegFun.rename_channel
- EegFun.rename_channel!
- EegFun.rename_channel!
- EegFun.repair_artifacts
- EegFun.repair_artifacts!
- EegFun.repair_channels
- EegFun.repair_channels!
- EegFun.repair_channels!
- EegFun.rereference
- EegFun.rereference!

- EegFun.resample
- EegFun.resample!
- EegFun.resample_temporal_data
- EegFun.ride_decompose
- EegFun.rsa
- EegFun.rsa_crossvalidated
- EegFun.run_ica
- EegFun.sample_rate
- EegFun.samples
- EegFun.samples_and
- EegFun.samples_and_not
- EegFun.samples_not
- EegFun.samples_or
- EegFun.samples_or_not
- EegFun.search_sequence
- EegFun.section
- EegFun.setup_global_logging
- EegFun.setup_logging
- EegFun.shift_matrix
- EegFun.shift_matrix!
- EegFun.shift_signal
- EegFun.shift_signal!
- EegFun.show_parameter_info
- EegFun.subset
- EegFun.subset
- EegFun.subset_bad_data
- EegFun.subset_layout
- EegFun.subset_layout!
- EegFun.subtract_ica_components
- EegFun.subtract_ica_components!
- EegFun.summarize_electrode_repairs
- EegFun.summarize_ica_components
- EegFun.tail
- EegFun.test_against_chance
- EegFun.test_against_chance_cluster
- EegFun.tf_baseline
- EegFun.tf_baseline!
- EegFun.tf_morlet

- EegFun.tf_morlet
- EegFun.tf_multitaper
- EegFun.tf_stft
- EegFun.time_vector
- EegFun.times
- EegFun.to_data_frame
- EegFun.trigger_count
- EegFun.trigger_info
- EegFun.unique_rejections
- EegFun.unmirror
- EegFun.unmirror!
- EegFun.validate_config
- EegFun.validate_layout
- EegFun.version_info
- EegFun.viewer
- EegFun.woody_filter

Functions

EegFun.add_ica_components! – Method.

```
add_ica_components!(dat, ica::InfoIca; component_selection = nothing)
```

Restore previously subtracted ICA components in-place using activations stored in ica.removed_activations (restores all removed components when component_selection === nothing).

source

EegFun.add_ica_components – Method.

```
add_ica_components(dat, ica::InfoIca; component_selection = nothing) ->
(restored_data, ica_updated)
```

Non-mutating restoration of previously removed ICA components; returns (restored_data, ica_updated) .

source

EegFun.add_noise_ceiling! – Method.

```
add_noise_ceiling!(rsa_data::RsaData,
rsa_data_list::AbstractVector{<:RsaData}; correlation_method::Symbol
= :spearman) -> RsaData
```

Compute and attach a NoiseCeiling to rsa_data in-place.

source

EegFun.add_noise_ceiling – Method.

```
add_noise_ceiling(rsa_data::RsaData,
rsa_data_list::AbstractVector{<:RsaData}; correlation_method::Symbol
= :spearman) -> RsaData
```

Return a copy of `rsa_data` with the computed `NoiseCeiling` attached.

source

`EegFun.all_data` – Method.

Flatten all rejection data (abs + z-score) into a single `DataFrame` with reason tags.

source

`EegFun.all_data` – Method.

```
all_data(dat) -> DataFrame
```

Return the underlying tabular data as a `DataFrame` .

source

`EegFun.all_labels` – Method.

```
all_labels(dat) -> Vector{Symbol}
```

Return all column names of `dat` .

source

`EegFun.amica_ica` – Method.

```
amica_ica(dat_ica, layout, filename; n_components, params = IcaPrms(), rng =
Random.default_rng()) -> InfoIca
```

Run Adaptive Mixture Independent Component Analysis (AMICA), modeling source densities with generalized Gaussian mixtures, second-order Newton updates, and log-likelihood rollback protection.

source

`EegFun.analytic_test` – Method.

```
analytic_test(prepared::StatisticalData; alpha = 0.05, tail = :both,
correction_method = :none)
analytic_test(prepared::TFStatisticalData; alpha = 0.05, tail = :both,
correction_method = :none)
```

Perform an analytic (parametric) mass-univariate t-test on prepared ERP (`StatisticalData`) or time-frequency (`TFStatisticalData`) data.

Keyword Arguments

- `alpha::Real = 0.05` : Significance threshold
- `tail::Symbol = :both` : Test tail (`:both` , `:left` , or `:right`)
- `correction_method::Symbol = :none` : Multiple comparison correction (`:none` / `:no` or `:bonferroni`)

source

`EegFun.apply_analysis_settings!` – Method.

```
apply_analysis_settings!(dat::EegData, settings::Union{AnalysisSettings,
Observable{AnalysisSettings}}) -> EegData
apply_analysis_settings!(dat::EegData, ica::InfoIca, settings) -> EegData
```

Apply interactive `AnalysisSettings` (filtering, rereferencing, channel repair, region selection, and optional ICA component subtraction) to `dat` in place.

source

`EegFun.apply_analysis_settings` – Method.

```
apply_analysis_settings(dat::EegData, args...; kwargs...) -> EegData
```

Non-mutating version of `apply_analysis_settings!` .

source

`EegFun.autoreject!` – Method.

```
autoreject!(dat::AbstractVector{<:EpochData}; kwargs...) -> AutoRejectResult
autoreject!(dat::EpochData; kwargs...) -> AutoRejectResult
```

Adaptive cross-validated artifact rejection and channel repair (Jas et al., 2017), mutating `dat` in-place.

Keyword Arguments

- `mode::Symbol = :local` : `:local` (per-channel CV thresholds + repair/reject) or `:global` (single global threshold, reject-only).
- `channel_selection = channels()` : Channel selector.
- `interval_selection::Interval = times()` : Time window for peak-to-peak evaluation.
- `n_folds::Int = 10` , `n_candidates::Int = 30` , `candidate_quantiles = (0.40, 1.0)` : CV search parameters.
- `consensus_perc::Real = 0.2` , `max_repair_perc::Real = 0.1` : Consensus rejection (κ) and max repair (ρ) channel fractions.
- `repair_method::Symbol = :spherical_spline` : `:spherical_spline` (`m = 4` , `lambda = 1e-5`) or `:neighbor_interpolation` .
- `seed::Union{Integer,Nothing} = 42` , `reject::Bool = true` , `verbose::Bool = true` .

source

`EegFun.autoreject` – Method.

```
autoreject(dat::AbstractVector{<:EpochData}; kwargs...) ->
Tuple{Vector{EpochData}, AutoRejectResult}
autoreject(dat::EpochData; kwargs...) -> Tuple{EpochData, AutoRejectResult}
```

Non-mutating version of `autoreject!` . Returns `(cleaned_data, result)` without modifying `dat` .

source

`EegFun.average_epochs` – Method.

```
average_epochs(dat::EpochData) -> ErpData
average_epochs(dat::AbstractVector{<:EpochData}) -> Vector{ErpData}
average_epochs(file_pattern::String; input_dir = pwd(), participant_selection
= participants(),
               condition_selection = conditions(), output_dir = nothing)
```

Average epoched EEG data across trials to produce `ErpData` (in-memory or batch JLD2 processing).

source

`EegFun.average_number_of_neighbors` – Method.

```
average_number_of_neighbors(neighbors_dict::Dict{Symbol,Neighbors}) ->
Float64
```

Calculate the mean number of neighbors per electrode.

source

`EegFun.baseline!` – Function.

```
baseline!(dat::EegData, baseline_interval::Interval = times();
channel_selection = channels()) -> EegData
baseline!(dat::AbstractVector{<:EegData}, baseline_interval::Interval =
times(); channel_selection = channels())
```

Subtract the mean amplitude over `baseline_interval` from each selected channel in-place. Omitting `baseline_interval` uses the full time range.

source

`EegFun.baseline` – Method.

```
baseline(dat, args...; kwargs...)
baseline(file_pattern::String, baseline_interval; input_dir = pwd(),
         participant_selection = participants(), condition_selection =
         conditions(), output_dir = nothing)
```

Non-mutating or batch JLD2 version of `baseline!` .

source

`EegFun.batch_process` – Method.

```
batch_process(process_fn::Function, file_pattern::String, input_dir::String,
output_dir::String,
              participant_selection, operation_name::String;
              filename_modifier::Function = identity, parallel::Bool = false)
```

Apply `process_fn(input_path, output_path)` across all `.jld2` files in `input_dir` matching `file_pattern` and `participant_selection` .

source

`EegFun.calculate_eog_channels!` – Method.

```
calculate_eog_channels!(dat::EegData, eog_cfg::Union{EogConfig, Dict}) ->
EegData
```

Compute vertical and horizontal bipolar EOG channels in place according to `eog_cfg` .

source

`EegFun.calculate_eog_channels` – Method.

```
calculate_eog_channels(dat, eog_cfg)
```

Non-mutating version of `calculate_eog_channels!` .

source

`EegFun.calculate_trigger_interval!` – Method.

```
calculate_trigger_interval!(dat::EpochData, start_triggers, end_triggers;
column_name = :interval) -> EpochData
calculate_trigger_interval!(dat_vec::AbstractVector{<:EpochData},
start_triggers, end_triggers; column_name = :interval) ->
AbstractVector{<:EpochData}
```

Calculate the time interval (`end - start` , in seconds) between the first occurrence of `start_triggers` and the subsequent first occurrence of `end_triggers` within each epoch, storing the result as a constant column `column_name` (`NaN` if either trigger is missing). Accepts integer or vector trigger specs.

source

`EegFun.calculate_trigger_interval` – Method.

```
calculate_trigger_interval(dat::EpochData, start_triggers, end_triggers;
column_name = :interval) -> EpochData
calculate_trigger_interval(dat_vec::AbstractVector{<:EpochData},
start_triggers, end_triggers; column_name = :interval) -> Vector{EpochData}
calculate_trigger_interval(file_pattern::String, start_triggers, end_triggers;
input_dir = pwd(), participant_selection = participants(), output_dir =
nothing, column_name = :interval)
```

Non-mutating and batch file versions of `calculate_trigger_interval!` .

source

`EegFun.channel_average!` – Method.

```
channel_average!(dat::EegData; channel_selections = [channels()],
output_labels = nothing,
                 include_extra::Bool = false, reduce::Bool = false) -> EegData
```

Average each group of selected channels in `dat` (linearly for amplitude/power, circularly for TF phase). Output labels default to `:avg` when all channels are selected, or underscore-joined channel names otherwise. When `reduce = true` , keeps only metadata and the averaged channels in both data and layout.

source

`EegFun.channel_average` – Method.

```
channel_average(dat, args...; kwargs...)
channel_average(file_pattern::String, channel_selections::AbstractVector;
                output_labels = nothing, input_dir::String = pwd(),
                participant_selection = participants(), condition_selection =
                conditions(),
                output_dir = nothing, reduce::Bool = false)
```

Non-mutating version of `channel_average!` , or batch-process matching JLD2 files into `output_dir` .

source

`EegFun.channel_data` – Method.

```
channel_data(dat::EegData) -> DataFrame
channel_data(dat::EegData, channel::Symbol) -> Vector{Float64}
```

Return EEG channel columns of `dat` as a `DataFrame` , or extract a single `channel` signal vector.

source

`EegFun.channel_delete!` – Method.

```
channel_delete!(dat::EegData, channels_to_delete) -> EegData
```

Delete specified channel(s) (`Symbol` , `String` , or vector thereof) from `dat` in place, updating both data columns and layout rows (clearing cached layout neighbors if layout channels change).

source

`EegFun.channel_delete` – Method.

```
channel_delete(dat, channels_to_delete)
```

Non-mutating version of `channel_delete!` .

source

`EegFun.channel_difference!` – Method.

```
channel_difference!(dat::EegData; channel_selection1 = channels(),
                    channel_selection2 = channels(), channel_out = :diff) ->
                    EegData
```

Compute `mean(channel_selection1) - mean(channel_selection2)` and store it as `channel_out` in `dat` .

source

`EegFun.channel_difference` – Method.

```
channel_difference(dat, args...; kwargs...)
```

Non-mutating version of `channel_difference!` .

source

`EegFun.channel_joint_probability` – Method.

```
channel_joint_probability(dat::SingleDataFrameEeg; sample_selection =
samples(),
                          interval_selection::Interval = times(),
                          channel_selection = channels(),
                          include_extra::Bool = false, threshold::Real = 5.0,
                          normalize::Int = 2, discret::Int = 1000) ->
                          DataFrame
```

Compute per-channel joint probability (negative log-likelihood) and threshold-based rejection flags.

source

`EegFun.channel_labels` – Method.

```
channel_labels(dat) -> Vector{Symbol}
```

Return electrode channel column names of `dat` .

source

`EegFun.channel_labels` – Method.

```
channel_labels(layout::Layout) -> Vector{Symbol}
```

Return electrode labels from `layout` .

source

`EegFun.channel_repairable!` – Method.

```
channel_repairable!(repair_info::ContinuousRepairInfo,
bad_channels::Vector{Symbol}, layout::Layout) -> ContinuousRepairInfo
```

Partition `bad_channels` into `repair_info.repaired` and `repair_info.skipped` based on valid neighbor availability in `layout` .

source

`EegFun.channel_repairable!` – Method.

```
channel_repairable!(artifacts::EpochRejectionInfo, layout::Layout)
```

Determine which rejected channels per epoch can be repaired from `layout` neighbor information, populating `artifacts.repaired` and `artifacts.skipped` in-place.

source

`EegFun.channel_repairable` – Method.

```
channel_repairable(dat, args...; kwargs...)
```

Non-mutating copy variant of `channel_repairable!` .

source

`EegFun.channel_summary` – Method.

```
channel_summary(dat::SingleDataFrameEeg; sample_selection = samples(),
interval_selection::Interval = times(),
                channel_selection = channels(), include_meta::Bool = false,
                include_extra::Bool = false) -> DataFrame
channel_summary(dat::MultiDataFrameEeg; sample_selection = samples(),
interval_selection::Interval = times(),
                channel_selection = channels(), include_meta::Bool = false,
                include_extra::Bool = false) -> DataFrame
channel_summary(file_pattern::String; input_dir = pwd(), participant_selection
= participants(),
                condition_selection = conditions(), sample_selection =
                samples(), interval_selection = times(),
                channel_selection = channels(), include_extra::Bool = false,
                output_dir = nothing,
                output_file::String = "channel_summary")
```

Compute summary statistics (`min` , `max` , `std` , `range` , `var` , `zvar`) per channel (and per epoch for `MultiDataFrameEeg` , or saved to CSV across JLD2 files in batch mode).

source

`EegFun.channels` – Method.

```
channels()
channels(names::Symbol...)
channels(names::AbstractVector{Symbol})
channels(indices)
channels(predicate::Function)
```

Create a channel selection predicate for `channel_selection` arguments.

source

`EegFun.channels_not` – Method.

```
channels_not(args...)
```

Create a channel selection predicate excluding the specified channels (inverse of `channels`).

source

`EegFun.check_channel_neighbors` – Method.

```
check_channel_neighbors(bad_channels::Vector{Symbol}, layout::Layout) ->
Vector{Symbol}
```

Return channels in `bad_channels` that have at least 2 spatial neighbors in `layout` and none of their neighbors in `bad_channels` .

source

`EegFun.check_raw_data` – Method.

```
check_raw_data(directory::String; raw_data_files::String = "\\.(bdf|edf|
vhdr|eeg|fif|xdf)$", recursive::Bool = false) -> DataFrame
```

Inspect raw EEG files in `directory`, summarize channel counts, sampling rates, and durations, and warn on metadata discrepancies across files.

source

`EegFun.cleanline!` – Method.

```
cleanline!(dat::ContinuousData; line_frequencies = [50.0, 100.0, 150.0,
200.0],
           bandwidth::Real = 2.0, sliding_win_length::Real = 4.0,
           sliding_win_step::Real = 2.0,
           time_bandwidth::Real = 3.0, k_tapers::Int = 5, p_value::Real =
           0.05,
           pad::Int = 2, channel_selection = channels())
```

Remove sinusoidal line noise in-place via adaptive sliding-window multi-taper F-test regression (Chronux / EEGLAB CleanLine algorithm). Harmonics `>= sample_rate / 2` are automatically skipped.

source

`EegFun.cleanline` – Method.

```
cleanline(dat::ContinuousData, args...; kwargs...) -> ContinuousData
cleanline(dat::AbstractVector{<:ContinuousData}, args...; kwargs...) ->
Vector{ContinuousData}
```

Non-mutating version of `cleanline!`.

source

`EegFun.clear_neighbors!` – Method.

```
clear_neighbors!(layout::Layout) -> Layout
```

Clear cached neighbor data and distance criterion from `layout`.

source

`EegFun.close_global_logging` – Method.

```
close_global_logging()
```

Close the active global session log file and restore the previous logger.

source

`EegFun.close_logging` – Method.

```
close_logging(; is_global::Bool = false)
```

Close the active local (or global) log file and restore the previous logger.

source

`EegFun.combine_artifact_components` – Method.

```
combine_artifact_components(eog_comps, ecg_comps, line_noise_comps,
channel_noise_comps) -> ArtifactComponents
```

Combine per-category ICA artifact indices into an `ArtifactComponents` struct.

source

`EegFun.common_channels` – Method.

```
common_channels(dat1::EegData, dat2::EegData) -> Vector{Symbol}
```

Return channels shared by `dat1` and `dat2` .

source

`EegFun.compare_models!` – Method.

```
compare_models!(
    rsa_data::RsaData,
    model_rdms::AbstractVector;
    model_names::Union{Vector{String},Nothing} = nothing,
    correlation_type::Symbol = :spearman,
    n_permutations::Int = 1000,
    tail::Symbol = :right,
    rng::AbstractRNG = Random.default_rng(),
) -> RsaData
```

Compare neural RDMs in `rsa_data` to `model_rdms` in-place (without mutating `model_rdms`). See `compare_models` for full parameter details.

source

`EegFun.compare_models` – Method.

```
compare_models(
    rsa_data::RsaData,
    model_rdms::AbstractVector;
    model_names::Union{Vector{String},Nothing} = nothing,
    correlation_type::Symbol = :spearman,
    n_permutations::Int = 1000,
    tail::Symbol = :right,
    rng::AbstractRNG = Random.default_rng(),
) -> RsaData
```

Compare neural RDMs to static (`Matrix{Float64}`) or temporal (`Array{Float64,3}`) model RDMs, returning a new `RsaData` copy with Mantel condition-permutation `p_values` (exact `N!` for `3 <= N <= 4` , Monte Carlo for `N >= 5`).

source

`EegFun.compare_rejections` – Method.

```
compare_rejections(rejection1::Vector{EpochRejectionInfo},
rejection2::Vector{EpochRejectionInfo}) -> DataFrame
```

Compare epoch rejection counts before (`rejection1`) and after (`rejection2`) channel repair per condition.

source

EegFun.components – Method.

```
components()
components(indices...)
components(predicate::Function)
```

Create an ICA component selection predicate.

source

EegFun.components_not – Method.

```
components_not(args...)
```

Create an ICA component selection predicate excluding the specified components (inverse of components).

source

EegFun.compute_csd! – Method.

```
compute_csd!(data; m::Int=4, lambda::Real=1e-5, n_legendre_terms::Int=50)
```

Compute the Current Source Density (CSD / surface Laplacian) of ContinuousData , EpochData , or ErpData in-place using spherical splines (Perrin et al., 1989; 1990 Corrigenda). Sets data.analysis_info.reference = :csd .

source

EegFun.compute_csd – Method.

```
compute_csd(dat, args...; kwargs...)
```

Non-mutating version of compute_csd! .

source

EegFun.compute_noise_ceiling – Method.

```
compute_noise_ceiling(
    rsa_data_list::AbstractVector{<:RsaData};
    correlation_method::Symbol = :spearman,
) -> NoiseCeiling
```

Compute leave-one-out lower and upper RSA noise ceiling bounds across >= 2 participants (Nili et al., 2014), rank-transforming (:spearman) and z-scoring each participant's upper-triangular RDM vector before group averaging so lower_bound <= upper_bound holds strictly.

source

EegFun.condition_average – Method.

```
condition_average(data::AbstractVector{<:Union{ErpData, TimeFreqData}},
condition_groups) -> Vector
condition_average(file_pattern::String, condition_groups; input_dir = pwd(),
```

```
                    participant_selection = participants(), output_dir =
                    nothing)
```

Average ERP waveforms or time-frequency representations (linear power, circular phase) across each group of condition indices in `condition_groups` , or batch-process matching JLD2 files.

source

`EegFun.condition_combine` – Method.

```
condition_combine(data::AbstractVector{<:EpochData},
condition_groups)::Vector{EpochData}
condition_combine(file_pattern::String, condition_groups; input_dir = pwd(),
                  participant_selection = participants(), output_dir =
                  nothing)
```

Combine `EpochData` conditions by concatenating independent copies of epochs within each group in `condition_groups` (matching `.condition` numbers with positional fallback), or batch-process JLD2 files.

source

`EegFun.condition_difference` – Method.

```
condition_difference(data::AbstractVector{<:Union{ErpData, TimeFreqData}},
condition_pairs) -> Vector
condition_difference(file_pattern::String, condition_pairs; input_dir = pwd(),
                     participant_selection = participants(), output_dir =
                     nothing)
```

Create condition difference waves (`a - b` for each `(a, b)` in `condition_pairs` , wrapping TF phase differences onto `[-π, π]`), or batch-process matching JLD2 files.

source

`EegFun.condition_name` – Method.

```
condition_name(dat::EegData) -> String
```

Return the condition name (`"Raw Data"` for `ContinuousData`).

source

`EegFun.condition_number` – Method.

```
condition_number(dat::EegData) -> Union{Int,String}
```

Return the condition number (`"Raw Data"` for `ContinuousData`).

source

`EegFun.condition_parse_epoch` – Method.

```
condition_parse_epoch(config::Dict) -> Vector{EpochCondition}
condition_parse_epoch(toml_file::String) -> Vector{EpochCondition}
```

Parse epoch condition configurations from a TOML file path or a pre-parsed dictionary.

source

EegFun.conditions – Method.

```
conditions()
conditions(indices_or_names...)
conditions(predicate::Function)
```

Create a condition selection predicate by index, name, or custom function.

source

EegFun.conditions_not – Method.

```
conditions_not(args...)
```

Create a condition selection predicate excluding the specified conditions (inverse of `conditions`).

source

EegFun.correlation_matrix – Method.

```
correlation_matrix(dat::SingleDataFrameEeg; sample_selection = samples(),
                   interval_selection::Interval = times(), channel_selection =
                   channels(),
                   include_extra::Bool = false) -> DataFrame
```

Compute the Pearson correlation matrix across selected EEG channels, returned as a `DataFrame` with a leading `:row` column of channel symbols.

source

EegFun.correlation_matrix_dual_selection – Method.

```
correlation_matrix_dual_selection(dat::SingleDataFrameEeg; sample_selection
= samples(),
                                  interval_selection::Interval = times(),
                                  channel_selection1 = channels(),
                                  channel_selection2 = channels(),
                                  include_extra_selection1::Bool = false,
                                  include_extra_selection2::Bool = true) ->
                                  DataFrame
```

Compute the cross-correlation matrix between two channel sets (`channel_selection1` as rows, `channel_selection2` as columns) over a shared sample selection.

source

EegFun.correlation_matrix_eog – Method.

```
correlation_matrix_eog(dat::SingleDataFrameEeg, eog_cfg::EogConfig;
sample_selection = samples(),
                       interval_selection::Interval = times(),
                       channel_selection = channels(),
                       include_extra::Bool = false) -> DataFrame
```

Compute the correlation matrix between EEG channels (`channel_selection`) and the EOG channels configured in `eog_cfg` .

source

`EegFun.create_continuous_repair_info` – Method.

```
create_continuous_repair_info(method::Symbol; name::String =
"continuous_repair") -> ContinuousRepairInfo
```

Create an empty `ContinuousRepairInfo` initialized for `method` .

source

`EegFun.create_custom_layout` – Method.

```
create_custom_layout(positions::Vector{Tuple{Float64, Float64}},
channels::Vector{Symbol})
```

Create a custom layout with specific `(x, y)` positions for each channel.

source

`EegFun.create_eegfun_data` – Method.

```
create_eegfun_data(dat, [layout::Layout])
```

Convert raw imported EEG data (`BiosemiData` , `EdfData` , `BrainVisionData` , `FifData` , `FifEpochs` , or `XdfData`) into EegFun `ContinuousData` , `EpochData` , or `ErpData` (scaling V/mV channels to µV).

source

`EegFun.create_epoch_repair_info` – Function.

```
create_epoch_repair_info(dat::EpochData, repaired, skipped, method::Symbol,
neighbors_dict = nothing) -> EpochRepairInfo
```

Construct an `EpochRepairInfo` sorted by epoch index from per-epoch `repaired` and `skipped` channel maps.

source

`EegFun.create_highpass_filter` – Method.

```
create_highpass_filter(cutoff_freq::Real, sample_rate::Real;
filter_method::Symbol = :iir,
                       order::Integer = 2, transition_width = nothing,
                       transition_band_hz = nothing, min_taps::Int = 101) ->
                       FilterInfo
```

Create a highpass digital filter specification (`:iir` Butterworth or `:fir` Hamming-windowed). When applied with zero-phase `filtfilt` (default), effective filter order is doubled.

source

`EegFun.create_layout` – Method.

```
create_layout(layout_spec, channels, eeg_layout; kwargs...)
```

Create a `PlotLayout` object based on `layout_spec` (`:single` , `:grid` , `:topo` , or an existing `PlotLayout`).

source

`EegFun.create_lowpass_filter` – Method.

```
create_lowpass_filter(cutoff_freq::Real, sample_rate::Real;
filter_method::Symbol = :iir,
                      order::Integer = 2, transition_width = nothing,
                      transition_band_hz = nothing, min_taps::Int = 101) ->
                      FilterInfo
```

Create a lowpass digital filter specification (`:iir` Butterworth or `:fir` Hamming-windowed). When applied with zero-phase `filtfilt` (default), effective filter order is doubled.

source

`EegFun.create_model_rdms` – Method.

```
create_model_rdms(model_data::AbstractDict{<:AbstractString,<:Any}) ->
(Vector{Matrix{Float64}}, Vector{String})
```

Batch-create static model RDMs from a dictionary mapping model names to model inputs accepted by `create_rdm` .

source

`EegFun.create_rdm` – Method.

```
create_rdm(vectors::AbstractVector{<:AbstractVector{<:Real}};
dissimilarity_measure::Symbol = :correlation)
create_rdm(data_matrix::AbstractMatrix{<:Real}; dissimilarity_measure::Symbol
= :correlation, is_similarity::Bool = false)
create_rdm(distances::AbstractVector{<:Real}, n_conditions::Integer)
create_rdm(values::AbstractVector{<:AbstractFloat}; method::Symbol
= :difference)
create_rdm(categories::AbstractVector{<:Union{Integer,Symbol,AbstractString}};
method::Symbol = :categorical)
create_rdm(temporal_data::AbstractArray{<:Real,3},
times::AbstractVector{<:Real};
           dissimilarity_measure::Symbol = :correlation, align_to = nothing,
           interpolation_method::Symbol = :linear)
create_rdm(model_data::AbstractDict{<:AbstractString,<:Any})
```

Construct a Representational Dissimilarity Matrix (`Matrix{Float64}`) or temporal RDM series (`Array{Float64,3} [time × condition × condition]`) from behavioral, categorical, feature, similarity, or time-resolved model data.

source

`EegFun.create_temporal_model_rdms` – Method.

```
create_temporal_model_rdms(
    temporal_model_data::AbstractDict{<:AbstractString,<:Any},
```

```
    times::AbstractVector{<:Real};
    dissimilarity_measure::Symbol = :correlation,
    align_to = nothing,
    interpolation_method::Symbol = :linear,
) -> (Vector{Array{Float64,3}}, Vector{String})
```

Batch-create temporal model RDMs from a dictionary mapping model names to `Array{<:Real,3}` , `(Array{<:Real,3}, times)` tuples, or nested temporal vectors.

source

`EegFun.decode_libsvm` – Method.

```
decode_libsvm(epochs::AbstractVector{EpochData}; channel_selection =
channels(),
              interval_selection = times(), n_iterations::Int = 100,
              n_folds::Int = 3,
              equalize_trials::Bool = true, cost::Real = 1.0,
              save_raw_predictions::Bool = true, show_progress::Bool = true,
              rng::AbstractRNG = Random.default_rng())
decode_libsvm(epochs::AbstractVector{TimeFreqEpochData}; kwargs...)

decode_libsvm(participant_epochs::AbstractVector{<:AbstractVector{<:MultiDataFr
kwargs...)
```

Perform time-point-by-time-point cross-validated linear SVM decoding (`LIBLINEAR`) across conditions on `EpochData` (channel features) or `TimeFreqEpochData` (channel × frequency features).

Keyword Arguments

- `channel_selection` : Channel selection predicate (default: `channels()`)
- `interval_selection::Interval` : Time interval filter (default: `times()`)
- `n_iterations::Int` : Random fold/subsample iterations (default: `100`)
- `n_folds::Int` : Cross-validation folds (default: `3`)
- `equalize_trials::Bool` : Subsample conditions to minimum trial count per iteration (default: `true`)
- `cost::Real` : SVM regularization parameter `C > 0` (default: `1.0`)
- `save_raw_predictions::Bool` : Store 4D prediction array in `DecodedData` (default: `true`)
- `show_progress::Bool` : Show progress bar (default: `true`)
- `rng::AbstractRNG` : Random number generator for reproducible splits

Returns

- `DecodedData` (or `Vector{DecodedData}` for multi-participant batch input)

source

`EegFun.detect_bad_epochs_automatic` – Method.

```
detect_bad_epochs_automatic(dat::EpochData; z_criterion::Real=3,
abs_criterion::Real=100,
```

```
                                channel_selection=channels(),
                                interval_selection::Interval=times(),
                                z_measures::AbstractVector{Symbol}
                                =[:variance, :max, :min, :abs, :range, :kurtosis],
                                name::String="rejection_info") ->
                                EpochRejectionInfo
```

Detect bad channel-epoch pairs using across-epoch z-score thresholds (`z_criterion` , set `0` to disable) and/or an absolute voltage threshold in µV (`abs_criterion` , set `0` to disable).

At least one of `z_criterion` or `abs_criterion` must be positive. Supported `z_measures` are `:variance` , `:max` , `:min` , `:abs` , `:range` , and `:kurtosis` .

source

`EegFun.detect_bad_epochs_interactive` – Method.

```
detect_bad_epochs_interactive(dat::EpochData;
                              channel_selection::Function = channels(),
                              artifact_info::Union{Nothing,EpochRejectionInfo}
                              = nothing,
                              kwargs...)::EpochRejectionState
```

Create an interactive interface for visually rejecting epochs.

Displays epochs in a grid layout with toggles for marking each epoch as bad (checked) or good (unchecked) and navigation buttons for scrolling through pages of epochs.

Arguments

- `dat::EpochData` : Epoched EEG data to review
- `channel_selection::Function` : Channels to display (default: all channels)
- `artifact_info::Union{Nothing,EpochRejectionInfo}` : Optional artifact detection info for bad channel filtering
- `kwargs` : Additional keyword arguments

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Whether to add origin lines at x=0 and y=0
- `bad_epoch_color::Symbol = :red` : Color for bad epoch spines
- `colormap::Nothing = nothing` : Colormap for channel traces
- `dims::Tuple{Int64, Int64} = (4, 6)` : Grid dimensions as (rows, cols) for epoch display
- `display_plot::Bool = true` : Whether to display the plot
- `global_ylim::Bool = false` : Whether to use global Y-axis limits across all epochs
- `good_epoch_color::Symbol = :green` : Color for good epoch spines
- `linewidth::Nothing = nothing` : Line width for epoch traces
- `spine_width::Int64 = 2` : Width of axis spines

- `theme_fontsize::Nothing = nothing` : Font size for theme
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple, or nothing for automatic
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple, or nothing for automatic
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Returns

- `EpochRejectionState` : State object containing rejection decisions

Examples

```
state = detect_bad_epochs_interactive(epochs)
rejected_indices = get_rejected_epochs(state)
cleaned = reject_epochs(epochs, state)
```

source

`EegFun.detect_eog_onsets!` – Method.

```
detect_eog_onsets!(dat::ContinuousData, criterion::Real, channel_in::Symbol,
channel_out::Symbol; step_size::Int=20)
detect_eog_onsets!(dat::ContinuousData, eog_cfg::Dict)
```

Detect electrooculogram (EOG) step onsets in `channel_in` exceeding `criterion` (μV) and record boolean flags in `channel_out` . When passed `eog_cfg::Dict` , detects both vertical (`vEOG`) and horizontal (`hEOG`) onsets in-place.

source

`EegFun.detect_eog_onsets!` – Method.

```
detect_eog_onsets!(dat::EegData, eog_cfg::EogConfig) -> EegData
```

Detect vertical and horizontal EOG step onsets in place using the criteria and channel names in `eog_cfg` .

source

`EegFun.detect_eog_onsets` – Method.

```
detect_eog_onsets(dat, args...; kwargs...)
```

Non-mutating version of `detect_eog_onsets!` .

source

`EegFun.detrend` – Method.

```
detrend(x::AbstractVector, y::AbstractVector) -> Vector{Float64}
```

Remove the least-squares linear trend of `y` with respect to `x`.

source

`EegFun.download_eegfun_datasets` – Method.

```
download_eegfun_datasets() -> String
```

Download and cache built-in tutorial datasets via `DataDeps.jl`, returning the local directory path.

source

`EegFun.duration` – Method.

```
duration(dat) -> Float64
```

Return the time span in seconds (`last(time) - first(time)`).

source

`EegFun.epoch_to_continuous` – Method.

```
epoch_to_continuous(dat::MultiDataFrameEeg, epoch_idx::Int) ->
ContinuousData
```

Extract epoch `epoch_idx` from `dat` as an independent `ContinuousData` object.

source

`EegFun.epochs` – Method.

```
epochs()
epochs(indices...)
epochs(predicate::Function)
```

Create an epoch selection predicate for `epoch_selection` arguments.

source

`EegFun.epochs_count` – Method.

```
epochs_count(dat::ContinuousData, toml_file::String; print_table::Bool =
true) -> DataFrame
epochs_count(dat::ContinuousData, conditions::Vector{EpochCondition};
print_table::Bool = true) -> DataFrame
```

Dry-run epoch counting for each condition without extracting waveform slices.

source

`EegFun.epochs_not` – Method.

```
epochs_not(args...)
```

Create an epoch selection predicate excluding the specified epochs (inverse of `epochs`).

source

`EegFun.epochs_table` – Method.

```
epochs_table(epochs::AbstractVector{<:EpochData}; print_table::Bool = true,
kwargs...) -> DataFrame
epochs_table(epochs_raw::AbstractVector{<:EpochData},
epochs_corrected::AbstractVector{<:EpochData};
             print_table::Bool = true, kwargs...) -> DataFrame
```

Format and optionally print a summary (or before/after rejection comparison) table of epoch counts per condition.

source

`EegFun.erp_measurements!` – Method.

```
erp_measurements!(dat::ErpData, analysis_type; analysis_interval = times(),
baseline_interval = nothing, channel_selection = channels(), participant =
0, kwargs...)
erp_measurements!(dat::EpochData, analysis_type; kwargs...)
erp_measurements!(data::AbstractVector{<:Union{ErpData,EpochData}},
analysis_type; kwargs...)

erp_measurements(data::Union{ErpData,EpochData,AbstractVector{<:Union{ErpData,E
analysis_type; kwargs...) -> Union{ErpMeasurementsResult,Nothing}
erp_measurements(file_pattern::String, analysis_type; analysis_interval =
times(), baseline_interval = nothing,
                 participant_selection = participants(), condition_selection =
                 conditions(), channel_selection = channels(),
                 input_dir = pwd(), output_dir = nothing, output_file =
                 "erp_measurements", kwargs...)
```

Extract ERP amplitude, latency, or area measurements from EEG data.

- `erp_measurements` on in-memory `ErpData` , `EpochData` , or `Vector` returns an `ErpMeasurementsResult` without mutating input data.
- `erp_measurements!` mutates input data in-place when applying `baseline_interval` / condition columns and returns raw `NamedTuple` row(s).
- `erp_measurements(file_pattern, ...)` batch-processes JLD2 files and saves a CSV.

`analysis_type` Options

- `"mean_amplitude"` — mean amplitude in `analysis_interval`
- `"max_peak_amplitude"` / `"min_peak_amplitude"` — robust local peak amplitude (with simple extremum fallback)
- `"max_peak_latency"` / `"min_peak_latency"` — peak latency in seconds
- `"peak_to_peak_amplitude"` / `"peak_to_peak_latency"` — difference between max and min peaks
- `"integral"` / `"rectified_area"` / `"positive_area"` / `"negative_area"` — trapezoidal area in µV·s
- `"fractional_area_latency"` / `"fractional_peak_latency"` — interpolated onset/offset latencies

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `area_boundary::Symbol = :rectified` : Area boundary mode for fractional area latency: :rectified (default), :positive, :negative, or :signed
- `fractional_area_fraction::Float64 = 0.5` : Fraction for fractional area latency (0.0-1.0). Finds latency where this fraction of area is to the left.
- `fractional_peak_direction::Symbol = :onset` : Direction for fractional peak latency: :onset (before peak) or :offset (after peak)
- `fractional_peak_fraction::Float64 = 0.5` : Fraction for fractional peak latency (0.0-1.0). Finds latency where amplitude is this fraction of peak.
- `local_interval::Int64 = 3` : Number of samples on each side of peak (total interval = 2*local_interval + 1). Peak must be larger than neighbors and local averages within this window.
- `peak_polarity::Symbol = :abs` : Peak polarity for fractional peak latency: :abs (default, largest absolute peak), :positive, or :negative

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.erp_measurements` – Method.

```
erp_measurements(data::Union{ErpData,EpochData}, analysis_type; kwargs...) -
> Union{ErpMeasurementsResult,Nothing}
erp_measurements(data::AbstractVector{<:Union{ErpData,EpochData}},
analysis_type; condition_selection = conditions(), kwargs...) ->
Union{ErpMeasurementsResult,Nothing}
```

Non-mutating in-memory ERP measurement extraction returning an `ErpMeasurementsResult` .

source

`EegFun.example_path` – Method.

```
example_path(relative_path::String) -> String
```

Return the absolute path to a bundled resource file under `pkgdir(EegFun)/resources/` .

source

`EegFun.export_bids` – Method.

```
export_bids(; task, raw_dir = ".", raw_pattern = "\\.bdf", derivatives_dir =
"",
              output_dir = "./bids_dataset", layout_file = "", participant_map
              = nothing,
```

```
                dataset_name = "", dataset_authors = String[], dataset_license =
                "CC0",
                power_line_frequency = 50, overwrite = false, ...)
```

Export an EegFun project (raw data and optional preprocessed derivatives) to a BIDS-compliant directory structure.

Keyword Arguments

- `task::String` : **Required.** BIDS task label (e.g., `"posner"`).
- `raw_dir::String` : Directory containing raw data files (default: `"."`).
- `raw_pattern::String` : Regex pattern for raw files (default: `"\\.bdf"`).
- `derivatives_dir::String` : Pipeline output directory containing JLD2 files (default: `""`).
- `output_dir::String` : Output BIDS directory (default: `"./bids_dataset"`).
- `layout_file::String` : Electrode layout CSV for 3D electrode positions (default: `""`).
- `participant_map::Union{Nothing,Dict{String,String}}` : Optional mapping from raw basenames (without extension) to BIDS subject IDs (e.g., `"sub-01"`).
- `dataset_name::String` : Name for `dataset_description.json` (default: `""`).
- `dataset_authors::Vector{String}` : Authors for `dataset_description.json` (default: `String[]`).
- `dataset_license::String` : Dataset license (default: `"CC0"`).
- `power_line_frequency::Int` : Power line frequency in Hz (default: `50`).
- `overwrite::Bool` : Overwrite existing output directory (default: `false`).
- Recommended sidecar metadata keywords: `task_description` , `instructions` , `manufacturer` , `manufacturer_model` , `cap_manufacturer` , `cap_model` , `institution_name` , `institution_address` , `institution_department` , `eeg_ground` , `eeg_placement_scheme` .

source

`EegFun.extra_data` – Method.

```
extra_data(dat::EegData) -> DataFrame
```

Return derived/extra columns (EOG, artifact flags, etc.) of `dat` .

source

`EegFun.extra_labels` – Method.

```
extra_labels(dat) -> Vector{Symbol}
```

Return derived/extra column names (columns following the last layout channel).

source

`EegFun.extract_epochs` – Method.

```
extract_epochs(dat::ContinuousData, condition::Integer,
epoch_condition::EpochCondition, epoch_interval) -> EpochData
extract_epochs(dat::ContinuousData,
epoch_conditions::AbstractVector{<:EpochCondition}, epoch_interval) ->
Vector{EpochData}
```

```
extract_epochs(dat::ContinuousData, epoch_condition::EpochCondition,
epoch_interval) -> Vector{EpochData}
```

Extract time-locked epochs from `dat` matching `epoch_condition` over `epoch_interval` (`start_time, end_time`) in seconds.

source

`EegFun.extract_ride` – Method.

```
extract_ride(epochs::EpochData, comps::Vector{RideComponent}; kwargs...) ->
Dict{Symbol, ErpData}
```

Extract RIDE components from `EpochData` , returning a `Dict{Symbol, ErpData}` mapping each component name to its decomposed template (with independent copies of `layout` and `analysis_info`).

source

`EegFun.file_name` – Method.

```
file_name(dat::EegFunData) -> String
```

Return the source filename of `dat` .

source

`EegFun.find_bridged_channels` – Function.

```
find_bridged_channels(dat::SingleDataFrameEeg,
correlation_threshold::Real=0.99; channel_selection=channels())
find_bridged_channels(dat::MultiDataFrameEeg,
correlation_threshold::Real=0.99; channel_selection=channels())
```

Find channels that are highly correlated with each other, indicative of an electrolyte gel bridge. Returns a `DataFrame` of the bridged pairs and their correlation coefficients.

source

`EegFun.find_file` – Method.

```
find_file(filename::String, search_dir::String;
          recursive::Bool = true, extensions::Vector{String} = String[]) ->
          Union{String, Nothing}
```

Search `search_dir` (and subdirectories when `recursive = true`) for `filename` (or `filename .* extensions`), returning the first matching full path or `nothing` .

source

`EegFun.find_nearest_peak` – Method.

```
find_nearest_peak(signal::AbstractVector, center::Int)
```

Find the strict local maximum in `signal` closest to index `center` , returning `(peak_index, found_peak)` .

source

`EegFun.find_times` – Method.

```
find_times(time::AbstractVector, requested_times::AbstractVector) ->
(indices::Vector{Int}, times::Vector{Float64})
```

Find the nearest deduplicated sample indices and time values in sorted `time` for each value in `requested_times` falling within `[first(time), last(time)]`.

source

`EegFun.freq_spectrum` – Method.

```
freq_spectrum(dat::EegData;
              channel_selection=channels(),
              window_size::Union{Int,Nothing}=nothing,
              overlap::Real=0.5,
              window_function::Function=DSP.hanning,
              max_freq::Union{Nothing,Real}=nothing)
```

Compute the Welch power spectral density (`SpectrumData` , in μV²/Hz) for `ContinuousData` , `ErpData` , or `EpochData` (computed per epoch and averaged across epochs).

Keyword Arguments

- `channel_selection=channels()` : Channel selection predicate.
- `window_size::Union{Int,Nothing}=nothing` : FFT window size in samples (defaults to `min(2048, n_samples(dat))`).
- `overlap::Real=0.5` : Window overlap fraction in `[0, 1)` .
- `window_function::Function=DSP.hanning` : Spectral window function.
- `max_freq::Union{Nothing,Real}=nothing` : Optional upper frequency cutoff in Hz.

Examples

```
spectrum = freq_spectrum(epochs)
spectrum = freq_spectrum(epochs; channel_selection=channels(:Cz),
window_size=2048, max_freq=100.0)
```

source

`EegFun.frequency_vector` – Method.

```
frequency_vector(dat) -> Vector{Float64}
```

Return the sorted unique frequency vector in Hz from a time-frequency, spectrum, or TF statistical object.

source

`EegFun.generate_config_template` – Method.

```
generate_config_template(; filename::String = "config_template.toml")
```

Generate and save a commented TOML configuration template with all parameters and defaults.

source

`EegFun.generate_permutation_matrix` – Method.

```
generate_permutation_matrix(df, block_col; sync_col = nothing, n_perms =
1000, type = :within, rng = Random.default_rng())
```

Generate an `N × n_perms` permutation matrix of integer row indices for Monte Carlo / permutation tests.

Arguments

- `df` : A `DataFrame` containing the experimental design / trial metadata.
- `block_col` : Column identifier (`Symbol` or `String`) for the grouping block (e.g. `:Subject` or `:Item`).

Keyword Arguments

- `sync_col = nothing` : Column identifier to synchronize across for balanced crossed designs.
- `n_perms = 1000` : Number of permutations to generate.
- `type = :within` : `:within` (shuffle within blocks), `:synchronized` (synchronized shuffle across `sync_col`), or `:global` .
- `rng = Random.default_rng()` : Random number generator for reproducibility.

Returns

- `Matrix{Int}` : An `N × n_perms` matrix where each column is a valid permutation of `1:N` .

source

`EegFun.generate_pipeline_template` – Function.

```
generate_pipeline_template(output_file = "custom_pipeline.jl", function_name
= "custom_preprocess"; options = PipelineTemplateOptions()) -> String
```

Generate a ready-to-customize EEG preprocessing pipeline script with setup, per-file logging, error handling, and summary boilerplate.

source

`EegFun.generate_signal` – Method.

```
generate_signal(n_trials, time_window, sample_rate, signal_freqs,
signal_amps, signal_times, noise)
```

Generate synthetic trials containing windowed sinusoids plus Gaussian noise. Returns `(times, signal)` where `signal` is `(n_samples × n_trials)` .

source

`EegFun.get_all_ica_components` – Method.

```
get_all_ica_components(artifacts::ArtifactComponents) -> Vector{Int}
```

Return all unique flagged ICA component indices in `artifacts` as a sorted vector.

source

`EegFun.get_eog_channels` – Method.

```
get_eog_channels(eog_cfg::EogConfig) -> Vector{Symbol}
```

Return `[vEOG_output_symbol, hEOG_output_symbol]` from `eog_cfg` .

source

EegFun.get_files – Method.

```
get_files(directory::String, files::String; recursive::Bool = false)
get_files(directory::String, files::Vector{String}; recursive::Bool = false)
```

Return naturally sorted full paths in `directory` matching regex `files` (or explicit filenames `files`), optionally searching subdirectories when `recursive = true`.

source

EegFun.get_filter_characteristics – Method.

```
get_filter_characteristics(filter_info::FilterInfo; npoints::Int = 1000,
filter_func = :filtfilt)
```

Compute key frequency-response characteristics (`cutoff_freq_3db`, `cutoff_freq_6db`, `passband_ripple`, `stopband_atten`, `phase_delay`, `group_delay`, `transition_band`, `filter_type`) for `filter_info`. When `filter_func = :filtfilt` (default), dB attenuation reflects two-pass zero-phase filtering.

source

EegFun.get_neighbors_xy! – Method.

```
get_neighbors_xy!(layout::Layout, distance_criterion::Real) -> Layout
```

Compute and cache 2D electrode neighbors within `distance_criterion` normalized head-radius units.

source

EegFun.get_neighbors_xyz! – Method.

```
get_neighbors_xyz!(layout::Layout, distance_criterion::Real) -> Layout
```

Compute and cache 3D electrode neighbors within `distance_criterion` normalized coordinate units.

source

EegFun.get_rejected_epochs – Method.

```
get_rejected_epochs(state::EpochRejectionState)::Vector{Int}
```

Get indices of rejected epochs from the rejection state.

Examples

```
state = detect_bad_epochs_interactive(epochs)
rejected_indices = get_rejected_epochs(state)
```

source

EegFun.get_selected_channels – Method.

```
get_selected_channels(dat, channel_selection; include_meta = true,
include_extra = true) -> Vector{Symbol}
```

Resolve `channel_selection` on `dat` into ordered column symbols, optionally prepending metadata columns and including extra columns.

source

`EegFun.get_selected_channels` – Method.

```
get_selected_channels(layout::Layout, channel_selection) -> Vector{Symbol}
```

Return channel labels in `layout` matching `channel_selection` .

source

`EegFun.get_selected_components` – Method.

```
get_selected_components(ica_result::InfoIca, component_selection) ->
Vector{Int}
```

Resolve `component_selection` into 1-based ICA component indices.

source

`EegFun.get_selected_conditions` – Method.

```
get_selected_conditions(datasets::AbstractVector{<:EegFunData},
condition_selection) -> Vector{Int}
```

Resolve `condition_selection` into 1-based condition dataset indices.

source

`EegFun.get_selected_epochs` – Method.

```
get_selected_epochs(dat::MultiDataFrameEeg, epoch_selection) -> Vector{Int}
```

Resolve `epoch_selection` into 1-based epoch indices.

source

`EegFun.get_selected_samples` – Method.

```
get_selected_samples(dat, sample_selection) -> Vector{Int}
```

Resolve `sample_selection` into 1-based row indices.

source

`EegFun.gfp` – Method.

```
gfp(dat::ErpData; channel_selection = channels(), normalize::Bool = false) -
> DataFrame
gfp(dat::AbstractVector{<:ErpData}; condition_selection = conditions(),
channel_selection = channels(), normalize::Bool = false) -> Vector{DataFrame}
```

Calculate Global Field Power (GFP) as the population standard deviation across selected channels at each time point (Lehmann & Skrandies, 1980).

Arguments

- `dat` : `ErpData` or `Vector{ErpData}`
- `channel_selection` : Channel predicate or symbols (default: `channels()`)

- `normalize::Bool` : Normalize GFP to `0–100%` range (default: `false`)

Returns

- `DataFrame` (or `Vector{DataFrame}`) with `:time` , metadata columns, and `:gfp` .

source

`EegFun.gfp_and_dissimilarity` – Method.

```
gfp_and_dissimilarity(dat::ErpData; channel_selection = channels(),
normalize::Bool = false) -> DataFrame
gfp_and_dissimilarity(dat::AbstractVector{<:ErpData}; condition_selection =
conditions(), channel_selection = channels(), normalize::Bool = false) ->
Vector{DataFrame}
```

Calculate both Global Field Power (`:gfp`) and Global Dissimilarity (`:dissimilarity`) in a single pass.

source

`EegFun.global_dissimilarity` – Method.

```
global_dissimilarity(dat::ErpData; channel_selection = channels(),
normalize::Bool = false) -> DataFrame
global_dissimilarity(dat::AbstractVector{<:ErpData}; condition_selection =
conditions(), channel_selection = channels(), normalize::Bool = false) ->
Vector{DataFrame}
```

Calculate reference-independent Global Dissimilarity (GD) across adjacent time points after subtracting the spatial channel mean and normalizing by GFP (Lehmann & Skrandies, 1980).

Arguments

- `dat` : `ErpData` or `Vector{ErpData}`
- `channel_selection` : Channel predicate or symbols (default: `channels()`)
- `normalize::Bool` : Normalize GD to `0–100%` range (default: `false`)

Returns

- `DataFrame` (or `Vector{DataFrame}`) with `:time` , metadata columns, and `:dissimilarity` .

source

`EegFun.grand_average` – Method.

```
grand_average(data::AbstractVector{<:Union{ErpData, TimeFreqData}};
condition_selection = conditions()) -> Vector
grand_average(rsa_data_list::AbstractVector{RsaData};
compute_noise_ceiling::Bool = true) -> RsaData
grand_average(decoded_list::AbstractVector{DecodedData}) -> DecodedData
grand_average(file_pattern::String; input_dir = pwd(), participant_selection =
participants(),
              condition_selection = conditions(), output_dir = nothing)
```

Average ERP, time-frequency, RSA, or decoding results across participants (per condition where applicable), or batch-process matching JLD2 files into `output_dir` .

source

`EegFun.group_by_condition` – Method.

```
group_by_condition(items::AbstractVector{T}) -> OrderedDict{Int, Vector{T}}
```

Group items by their `.condition` field into a condition-sorted `OrderedDict` .

source

`EegFun.has_2d_coords` – Method.

```
has_2d_coords(layout::Layout) -> Bool
```

Return `true` if `layout` has `:x2` and `:y2` columns.

source

`EegFun.has_3d_coords` – Method.

```
has_3d_coords(layout::Layout) -> Bool
```

Return `true` if `layout` has `:x3` , `:y3` , and `:z3` columns.

source

`EegFun.has_channels` – Method.

```
has_channels(dat::EegData, chans::Vector{Symbol}) -> Bool
```

Return `true` if `dat` contains every channel in `chans` .

source

`EegFun.has_neighbors` – Method.

```
has_neighbors(layout::Layout) -> Bool
```

Return `true` if neighbor relationships have been calculated on `layout` .

source

`EegFun.has_valid_coordinates` – Method.

```
has_valid_coordinates(layout::Layout) -> Bool
```

Return `true` if `layout` has non-zero `:inc` or `:azi` coordinates suitable for topographic plotting.

source

`EegFun.head` – Method.

```
head(dat::EegData; n = 5)
```

Display and return a view of the first `n` rows of `all_data(dat)` .

source

`EegFun.highpass_filter!` – Method.

```
highpass_filter!(dat::EegData, cutoff_freq::Real; order::Int = 1,
transition_width = nothing,
```

```
                filter_method::Symbol = :iir, channel_selection = channels(),
                filter_func::Symbol = :filtfilt)
highpass_filter!(dat::EegData, filter_cfg::FilterConfig; section::Symbol
= :highpass)
```

Apply a digital highpass filter to `dat` (or a `Vector` of `EegData`) in-place.

source

`EegFun.highpass_filter` – Method.

```
highpass_filter(dat, cutoff_freq::Real; kwargs...)
highpass_filter(file_pattern::String, cutoff_freq::Real; input_dir = pwd(),
                participant_selection = participants(), condition_selection =
                conditions(), output_dir = nothing)
```

Non-mutating or batch JLD2 version of `highpass_filter!`.

source

`EegFun.identify_bad_channels` – Method.

```
identify_bad_channels(summary_df::DataFrame, joint_prob_df::DataFrame;
zvar_criterion::Real = 3.0) -> Vector{Symbol}
```

Return the union of channels flagged by `abs(zvar) > zvar_criterion` in `summary_df` or `rejection == true` in `joint_prob_df`.

source

`EegFun.identify_components` – Method.

```
identify_components(dat::ContinuousData, ica::InfoIca; method
= :correlation,
                    sample_selection = samples(), interval_selection =
                    times(),
                    eog = (;), ecg = (;), line_noise = (;), channel_noise =
                    (;),
                    kwargs...) -> (ArtifactComponents, Dict{Symbol,
                    DataFrame})
```

Identify EOG, ECG, line-noise, and focal channel-noise ICA components in a single pass, sharing precomputed ICA activations across detectors.

source

`EegFun.identify_ecg_components` – Method.

```
identify_ecg_components(dat::ContinuousData, ica::InfoIca;
                        min_bpm=40, max_bpm=120, min_prominence_std=4,
                        min_peaks=100, max_ibi_std_s=0.2, min_peak_ratio=0.5,
                        sample_selection=samples(),
                        interval_selection=times())
```

Identify cardiac (ECG) ICA components via dual-polarity QRS peak detection and inter-beat interval regularity. Returns `(ecg_indices, metrics_df)`.

source

EegFun.identify_eog_components – Method.

```
identify_eog_components(dat::ContinuousData, ica::InfoIca;
                        vEOG_channel=:vEOG, hEOG_channel=:hEOG,
                        z_threshold=3.0, min_correlation=0.5, two_step=true,
                        sample_selection=samples(),
                        interval_selection=times())
```

Identify vertical and horizontal EOG ICA components via MAD-scaled robust z-scored correlation (plus optional `two_step` spatial topography and lagged temporal cross-correlation matching). Returns `(Dict(:vEOG => ..., :hEOG => ...), metrics_df)`.

source

EegFun.identify_line_noise_components – Method.

```
identify_line_noise_components(dat::ContinuousData, ica::InfoIca;
                               line_freq=50.0, freq_bandwidth=1.0,
                               z_threshold=3.0, min_harmonic_power=1.5,
                               sample_selection=samples(),
                               interval_selection=times())
```

Identify line-noise ICA components via Welch PSD peak-to-flank ratios at `line_freq` (and its 2nd harmonic when below Nyquist). Returns `(line_noise_indices, metrics_df)`.

source

EegFun.identify_spatial_kurtosis_components – Method.

```
identify_spatial_kurtosis_components(ica::InfoIca; z_threshold = 3.0)
```

Identify focal channel-noise ICA components via MAD-scaled robust z-scores of mixing-weight kurtosis.

source

EegFun.infomax_extended_ica – Method.

```
infomax_extended_ica(dat_ica, layout, filename; n_components, params =
IcaPrms(), rng = Random.default_rng()) -> InfoIca
```

Extended Infomax ICA decomposition separating mixed super-Gaussian and sub-Gaussian sources via kurtosis-based sign switching (Lee, Girolami, & Sejnowski, 1999).

source

EegFun.infomax_ica – Method.

```
infomax_ica(dat_ica, layout, filename; n_components, params = IcaPrms(), rng
= Random.default_rng()) -> InfoIca
```

Standard natural-gradient Infomax ICA decomposition (super-Gaussian sources).

source

EegFun.is_extreme_value! – Method.

```
is_extreme_value!(dat::SingleDataFrameEeg, threshold::Real;
channel_selection = channels(),
                  sample_selection = samples(), interval_selection = times(),
                  mode::Symbol = :combined, channel_out = nothing)
is_extreme_value!(dat::MultiDataFrameEeg, threshold::Real; channel_selection =
channels(),
                  sample_selection = samples(), interval_selection = times(),
                  epoch_selection = epochs(),
                  mode::Symbol = :combined, channel_out = nothing)
is_extreme_value!(dat::AbstractVector{<:EpochData}, threshold::Real;
kwargs...)
```

Detect extreme values (`|x| > threshold`) and store boolean flag columns in-place (`:is_extreme_value_<threshold>` in `:combined` mode, or `:is_extreme_value_<ch>_<threshold>` in `:separate` mode).

source

`EegFun.is_extreme_value` – Method.

```
is_extreme_value(dat, threshold::Real; channel_selection = channels(),
sample_selection = samples(),
                 interval_selection = times(), mode::Symbol = :combined)
```

Detect extreme values (`|x| > threshold`) without modifying `dat` . Returns a `Vector{Bool}` (or per-epoch vector) in `:combined` mode, or a `DataFrame` (or per-epoch vector of `DataFrame` s) in `:separate` mode.

source

`EegFun.is_flatline!` – Method.

```
is_flatline!(dat::SingleDataFrameEeg, threshold::Real, window_size::Real;
             channel_selection = channels(), sample_selection = samples(),
             interval_selection = times(), mode::Symbol = :combined,
             channel_out = nothing)
is_flatline!(dat::MultiDataFrameEeg, threshold::Real, window_size::Real;
kwargs...)
is_flatline!(dat::AbstractVector{<:EpochData}, threshold::Real,
window_size::Real; kwargs...)
```

Detect flatline artifacts in sliding windows of `window_size` seconds where `std < threshold` , storing boolean flag columns in-place (`:is_flatline` or `:is_flatline_<ch>` by default).

source

`EegFun.is_flatline` – Method.

```
is_flatline(dat, threshold::Real, window_size::Real; channel_selection =
channels(),
            sample_selection = samples(), interval_selection = times(),
            mode::Symbol = :combined)
```

Detect flatline artifacts without modifying `dat` . Returns a `Vector{Bool}` (or per-epoch vector) in `:combined` mode, or a `DataFrame` (or per-epoch vector) in `:separate` mode.

source

`EegFun.is_peak_to_peak!` – Method.

```
is_peak_to_peak!(dat::SingleDataFrameEeg, threshold::Real,
window_size::Real;
                 channel_selection = channels(), sample_selection = samples(),
                 interval_selection = times(), mode::Symbol = :combined,
                 channel_out = nothing)
is_peak_to_peak!(dat::MultiDataFrameEeg, threshold::Real, window_size::Real;
kwargs...)
is_peak_to_peak!(dat::AbstractVector{<:EpochData}, threshold::Real,
window_size::Real; kwargs...)
```

Detect moving-window peak-to-peak artifacts (`max - min > threshold` within `window_size` seconds), storing boolean flag columns in-place (`:is_peak_to_peak` or `:is_peak_to_peak_<ch>` by default).

source

`EegFun.is_peak_to_peak` – Method.

```
is_peak_to_peak(dat, threshold::Real, window_size::Real; channel_selection =
channels(),
                sample_selection = samples(), interval_selection = times(),
                mode::Symbol = :combined)
```

Detect moving-window peak-to-peak artifacts without modifying `dat` . Returns a `Vector{Bool}` (or per-epoch vector) in `:combined` mode, or a `DataFrame` (or per-epoch vector) in `:separate` mode.

source

`EegFun.is_step_value!` – Method.

```
is_step_value!(dat::SingleDataFrameEeg, threshold::Real; channel_selection =
channels(),
               sample_selection = samples(), interval_selection = times(),
               mode::Symbol = :combined, channel_out = nothing)
is_step_value!(dat::MultiDataFrameEeg, threshold::Real; channel_selection =
channels(),
               sample_selection = samples(), interval_selection = times(),
               epoch_selection = epochs(),
               mode::Symbol = :combined, channel_out = nothing)
is_step_value!(dat::AbstractVector{<:EpochData}, threshold::Real; kwargs...)
```

Detect step artifacts (`|x[i] - x[i-1]| > threshold`) and store boolean flag columns in-place (`:is_step_value_<threshold>` in `:combined` mode, or `:is_step_value_<ch>_<threshold>` in `:separate` mode).

source

`EegFun.is_step_value` – Method.

```
is_step_value(dat, threshold::Real; channel_selection = channels(),
sample_selection = samples(),
              interval_selection = times(), mode::Symbol = :combined)
```

Detect step artifacts (`|x[i] - x[i-1]| > threshold`) without modifying `dat` . Returns a `Vector{Bool}` (or per-epoch vector) in `:combined` mode, or a `DataFrame` (or per-epoch vector) in `:separate` mode.

source

`EegFun.jackknife_average` – Method.

```
jackknife_average(erps::AbstractVector{<:ErpData}; condition_selection =
conditions(), pseudovalues::Bool = false)::Vector{ErpData}
jackknife_average(measurements::ErpMeasurementsResult; pseudovalues::Bool =
true)::ErpMeasurementsResult
jackknife_average(file_pattern::String; input_dir = pwd(),
participant_selection = participants(),
                  condition_selection = conditions(), output_dir = nothing,
                  pseudovalues::Bool = false)
```

Create leave-one-out jackknife subaverages (or Smulders, 2010 pseudovalues) from ERP/LRP data.

For each participant `i` , computes the leave-one-out average `X̄_{-i}` of all other `N - 1` participants (excluding `i`). This technique is standard for nonlinear ERP/LRP latency measures (e.g., fractional peak or fractional area onset latency) where single-participant waveforms have low signal-to-noise ratio.

Statistical Testing & Variance Correction

Because leave-one-out subaverages `X̄_{-i}` share `N - 2` participants, their sample variance is compressed by `(N - 1)^2` relative to single-participant variance:

- **Ulrich & Miller (2001) correction (`pseudovalues = false`)**: Divide `t` -statistics by `N - 1` and `F` -statistics by `(N - 1)^2` .
- **Smulders (2010) pseudovalues (`pseudovalues = true`)**: Converts leave-one-out values `X̄_{-i}` into single-participant pseudovalues $J_i = N\bar{X} - (N-1)\bar{X}_{-i}$, allowing standard `t` -tests, ANOVAs, correlations, and mixed-effects models without manual rescaling. For nonlinear measures (e.g., onset latency), extract measurements on `pseudovalues = false` subaverages first, then convert via `jackknife_average(meas; pseudovalues = true)` .

References

- Miller, J., Patterson, T., & Ulrich, R. (1998). Jackknife-based method for measuring LRP onset latency differences. *Psychophysiology*, 35(1), 99-115.
- Ulrich, R., & Miller, J. (2001). Using the jackknife-based scoring method for measuring LRP onset effects in factorial designs. *Psychophysiology*, 38(5), 816-827.
- Smulders, F. T. Y. (2010). Simplifying jackknifing of ERPs and getting more out of it. *Psychophysiology*, 47(2), 387-392.

source

`EegFun.libsvm_classifier` – Method.

```
libsvm_classifier(
```

```
    X_train::AbstractMatrix{Float64},
    y_train::AbstractVector{Int},
    X_test::AbstractMatrix{Float64};
    cost::Real = 1.0,
) -> Vector{Int}
```

Train a linear L2-regularized L2-loss SVM (`LIBLINEAR` , `bias = 1.0`) on `X_train` `[n_samples × n_features]` and `y_train` , returning predicted integer class labels for `X_test` .

source

`EegFun.list_bids_runs` – Method.

```
list_bids_runs(dataset_dir::String; datatype::String = "eeg") -> DataFrame
```

Discover all BIDS recordings in `dataset_dir` for `datatype` and return a `DataFrame` with columns `subject` , `session` , `task` , `run` , `datatype` , and `file` .

source

`EegFun.log_epochs_table` – Method.

```
log_epochs_table(epochs...; print_table::Bool = false, kwargs...) ->
DataFrame
```

Log an `epochs_table` via `@info` and return the summary `DataFrame` .

source

`EegFun.lowpass_filter!` – Method.

```
lowpass_filter!(dat::EegData, cutoff_freq::Real; order::Int = 3,
transition_width = nothing,
                filter_method::Symbol = :iir, channel_selection = channels(),
                filter_func::Symbol = :filtfilt)
lowpass_filter!(dat::EegData, filter_cfg::FilterConfig; section::Symbol
= :lowpass)
```

Apply a digital lowpass filter to `dat` (or a `Vector` of `EegData`) in-place.

source

`EegFun.lowpass_filter` – Method.

```
lowpass_filter(dat, cutoff_freq::Real; kwargs...)
lowpass_filter(file_pattern::String, cutoff_freq::Real; input_dir = pwd(),
               participant_selection = participants(), condition_selection =
               conditions(), output_dir = nothing)
```

Non-mutating or batch JLD2 version of `lowpass_filter!` .

source

`EegFun.lrp` – Method.

```
lrp(erp_left::ErpData, erp_right::ErpData; channel_selection = channels()) -
> ErpData
```

```
lrp(erps::AbstractVector{<:ErpData}, condition_pairs; condition_selection =
conditions(), channel_selection = channels()) -> Vector{ErpData}
lrp(file_pattern::String, condition_pairs; input_dir = pwd(),
channel_selection = channels(), participant_selection = participants(),
output_dir = nothing)
```

Calculate the lateralized readiness potential (LRP) from ERP data using the double-subtraction formula:

- `LRP_odd  = 0.5 * ((odd_right - even_right) + (even_left - odd_left))`
- `LRP_even = -LRP_odd`

Arguments

- `erp_left` , `erp_right` : Paired left- and right-response `ErpData`
- `channel_selection` : Odd/left hemisphere channel selector; default `channels()` auto-detects all odd/even pairs (e.g., `C3` / `C4` , `CP3` / `CP4`)
- `condition_pairs` : Vector of `(left_cond, right_cond)` integer tuples

References

- Coles, M. G. H. (1989). Modern mind-brain reading. *Psychophysiology*, 26(3), 251-269.
- de Jong, R. et al. (1988). Use of partial stimulus information. *JEP:HPP*, 14(4), 682-692.
- Oostenveld, R. et al. (2003). Brain symmetry and topographic analysis. *Clin. Neurophysiology*, 114(7), 1194-1202.

source

`EegFun.mark_epoch_intervals!` – Method.

```
mark_epoch_intervals!(dat::ContinuousData,
triggers_of_interest::AbstractVector{<:Integer}, time_window; channel_out
= :epoch_interval)
mark_epoch_intervals!(dat::ContinuousData, channel_in::Symbol, time_window;
channel_out = Symbol(string(channel_in) * "_window"))
mark_epoch_intervals!(dat::ContinuousData,
channels_in::AbstractVector{Symbol}, time_window; channel_out = :event_window)
mark_epoch_intervals!(dat::ContinuousData, epoch_conditions, time_window;
channel_out = :epoch_interval)
```

Mark samples within `time_window` around trigger events, boolean flag columns, or `EpochCondition` matches in-place.

source

`EegFun.meta_data` – Method.

```
meta_data(dat::EegData) -> DataFrame
```

Return a `DataFrame` containing only the metadata columns of `dat` .

source

`EegFun.meta_labels` – Method.

```
meta_labels(dat::EegData) -> Vector{Symbol}
```

Return metadata column names (columns preceding the first layout channel).

source

`EegFun.mirror` – Function.

```
mirror(dat::EegData, side::Symbol = :both)
mirror(data_vec::Vector{<:EegData}, side::Symbol = :both)
```

Non-mutating version of `mirror!` .

source

`EegFun.mirror!` – Function.

```
mirror!(dat::EegData, side::Symbol = :both)
mirror!(data_vec::Vector{<:EegData}, side::Symbol = :both)
```

Mirror data in-place by reflecting time-reversed copies at boundaries (`:pre` , `:post` , or `:both`) to suppress edge artifacts during filtering. Call `unmirror!` with the same `side` afterward.

source

`EegFun.n_channels` – Method.

```
n_channels(dat) -> Int
```

Return the number of EEG channels in `dat` .

source

`EegFun.n_epochs` – Method.

```
n_epochs(dat::EegData) -> Int
```

Return the number of epochs (`1` for continuous/spectrum/TF-average, `dat.n_epochs` for `ErpData`).

source

`EegFun.n_extreme_value` – Method.

```
n_extreme_value(dat, threshold::Real; channel_selection = channels(),
sample_selection = samples(),
                interval_selection = times(), mode::Symbol = :combined)
```

Count extreme values (`|x| > threshold`) across selected channels, returning a total `Int` (`:combined`) or a per-channel `DataFrame` (`:separate`).

source

`EegFun.n_flatline` – Method.

```
n_flatline(dat, threshold::Real, window_size::Real; channel_selection =
channels(),
           sample_selection = samples(), interval_selection = times(),
           mode::Symbol = :combined)
```

Count flatline samples across selected channels, returning a total `Int` (`:combined`) or a per-channel `DataFrame` (`:separate`).

source

`EegFun.n_layout` – Method.

```
n_layout(layout::Layout) -> Int
```

Return the number of electrodes in `layout` .

source

`EegFun.n_peak_to_peak` – Method.

```
n_peak_to_peak(dat, threshold::Real, window_size::Real; channel_selection =
channels(),
               sample_selection = samples(), interval_selection = times(),
               mode::Symbol = :combined)
```

Count peak-to-peak artifact samples across selected channels, returning a total `Int` (`:combined`) or a per-channel `DataFrame` (`:separate`).

source

`EegFun.n_samples` – Method.

```
n_samples(dat) -> Int
```

Return the number of time/frequency samples per epoch or DataFrame.

source

`EegFun.n_step_value` – Method.

```
n_step_value(dat, threshold::Real; channel_selection = channels(),
sample_selection = samples(),
             interval_selection = times(), mode::Symbol = :combined)
```

Count step artifacts (`|x[i] - x[i-1]| > threshold`) across selected channels, returning a total `Int` (`:combined`) or a per-channel `DataFrame` (`:separate`).

source

`EegFun.n_values` – Method.

```
n_values(df::DataFrame, column::Symbol)
n_values(dat::SingleDataFrameEeg, column::Symbol)
n_values(dat::MultiDataFrameEeg, column::Symbol)
n_values(dat::AbstractVector{<:EegData}, column::Symbol)
```

Count the number of `true` values in boolean `column` (returning `0` if `column` is absent).

source

`EegFun.neighbor_criterion` – Method.

```
neighbor_criterion(layout::Layout) -> Union{Nothing,Float64}
```

Return the distance criterion used for neighbor calculation on `layout`, or `nothing`.

source

`EegFun.neighbors` – Method.

```
neighbors(layout::Layout) -> Union{Nothing,Dict{Symbol,Neighbors}}
```

Return the dictionary of electrode `Neighbors` on `layout`, or `nothing`.

source

`EegFun.normalize_rdm` – Method.

```
normalize_rdm(rdm::AbstractMatrix{Float64}; method::Symbol = :none) ->
Matrix{Float64}
```

Normalize upper/lower off-diagonal entries of a symmetric RDM (preserving zero diagonal) using `:none`, `:zscore`, `:rank` (average tied ranks), or `:minmax` (`[0, 1]`).

source

`EegFun.participants` – Method.

```
participants()
participants(ids...)
participants(predicate::Function)
```

Create a participant ID selection predicate for batch operations.

source

`EegFun.participants_not` – Method.

```
participants_not(args...)
```

Create a participant ID selection predicate excluding the specified IDs (inverse of `participants`).

source

`EegFun.partition_channels_by_eog_correlation` – Method.

```
partition_channels_by_eog_correlation(bad_channels::Vector{Symbol},
eog_correlation_df::DataFrame;
                                      eog_channels::Vector{Symbol} =
                                      [:hEOG, :vEOG],
                                      threshold::Real = 0.3, use_z::Bool =
                                      false) ->
                                      Tuple{Vector{Symbol},Vector{Symbol}}
```

Split `bad_channels` into `(non_eog_bad, eog_related)` according to whether any correlation in `eog_correlation_df` exceeds `threshold` in magnitude.

source

`EegFun.permutation_test` – Method.

```
permutation_test(prepared::StatisticalData; kwargs...)
permutation_test(prepared::TFStatisticalData; kwargs...)
```

Perform a cluster-based permutation test (maxsum) on prepared ERP (`StatisticalData`) or time-frequency (`TFStatisticalData`) data.

Keyword Arguments

- `n_permutations::Int = 1000` : Number of Monte Carlo permutations
- `alpha::Real = 0.05` : Cluster-level family-wise error rate significance threshold
- `cluster_alpha::Real = 0.05` : Point-wise cluster-forming p-value threshold
- `threshold_method::Symbol = :parametric` : `:parametric` , `:nonparametric_common` , or `:nonparametric_individual`
- `cluster_type::Symbol` : `:spatial` , `:temporal` , or `:spatiotemporal` for ERP (default `:spatiotemporal`); additionally `:spectral` , `:spectrotemporal` , or `:full` for TF (default `:full`)
- `min_num_neighbors::Int = 2` : Minimum neighboring significant channels required to retain a point
- `tail::Symbol = :both` : `:both` , `:left` , or `:right`
- `show_progress::Bool = true` : Show progress bar
- `use_gpu::Bool = false` : (ERP only) Compute permutation t-matrices on an active GPU backend
- `rng::AbstractRNG = Random.default_rng()` : Random number generator (thread-count independent)

source

`EegFun.picard_ica` – Method.

```
picard_ica(dat_ica, layout, filename; n_components, extended = false,
           params = IcaPrms(), rng = Random.default_rng()) -> InfoIca
```

Run Preconditioned ICA for Real Data (Picard; Ablin, Cardoso, & Gramfort, 2018), using an exact 2×2 block Hessian preconditioner with L-BFGS (`extended = true` separates mixed sub-/super-Gaussian sources).

source

`EegFun.polar_to_cartesian_xy!` – Method.

```
polar_to_cartesian_xy!(layout::Layout; normalization_radius::Real = 1.0,
preserve_radial_distance::Bool = true)
```

Convert polar (`:inc` , `:azi`) coordinates into 2D Cartesian (`:x2` , `:y2`) coordinates in place and clear cached neighbors. When `preserve_radial_distance = true` , `90°` maps to radius `1.0` without shifting the vertex `(0, 0)` .

source

`EegFun.polar_to_cartesian_xyz!` – Method.

```
polar_to_cartesian_xyz!(layout::Layout)
```

Convert polar (`:inc` , `:azi`) coordinates into centered, normalized 3D Cartesian (`:x3` , `:y3` , `:z3`) coordinates in place and clear cached neighbors.

source

`EegFun.positions_2D` – Method.

```
positions_2D(layout::Layout) -> DataFrame
```

Return 2D Cartesian coordinate columns (`:x2` , `:y2`) from `layout` .

source

`EegFun.positions_3D` – Method.

```
positions_3D(layout::Layout) -> DataFrame
```

Return 3D Cartesian coordinate columns (`:x3` , `:y3` , `:z3`) from `layout` .

source

`EegFun.positions_polar` – Method.

```
positions_polar(layout::Layout) -> DataFrame
```

Return polar coordinate columns (`:inc` , `:azi`) from `layout` .

source

`EegFun.prepare_decoding` – Method.

```
prepare_decoding(epochs::AbstractVector{T}; condition_selection =
conditions(),
                 channel_selection = channels(), interval_selection = times())
                 where {T<:MultiDataFrameEeg}
prepare_decoding(file_pattern::String; input_dir = pwd(),
participant_selection = participants(),
                 condition_selection = conditions(), channel_selection =
                 channels(),
                 interval_selection = times())
```

Organize multi-participant epoch data (`EpochData` or `TimeFreqEpochData`) into per-participant condition vectors `Vector{Vector{T}}` ready for `decode_libsvm` .

Validates consistent participants, channels, sample rates, and time vectors across selected conditions.

source

`EegFun.prepare_stats` – Method.

```
prepare_stats(erps::AbstractVector{<:ErpData}; design::Symbol = :paired,
condition_selection = conditions(), channel_selection = channels(),
interval_selection::Interval = times(), baseline_interval::Interval =
nothing, analysis_interval::Interval = times())
```

Prepare `ErpData` for comparing two conditions in statistical tests (permutation and analytic tests).

Organizes `ErpData` into `participant × electrode × time` arrays, computes grand averages and standard errors over the display interval, and optionally narrows the statistical testing window via `analysis_interval` .

Returns

- `StatisticalData` : Prepared data structure ready for statistical testing

source

`EegFun.prepare_stats` – Method.

```
prepare_stats(tfs::AbstractVector{<:TimeFreqData}; design = :paired,
condition_selection = conditions(), channel_selection = channels(),
frequency_selection = nothing, interval_selection = nothing,
baseline_interval = nothing, baseline_method = :db, analysis_interval =
nothing)
```

Prepare `TimeFreqData` for comparing two conditions in statistical tests.

Organizes `TimeFreqData` power values into `participant × electrode × frequency × time` arrays, computes grand averages over the display interval, and optionally applies baseline normalization and an `analysis_interval` .

Returns

- `TFStatisticalData` : Prepared data structure ready for statistical testing

source

`EegFun.prepare_stats` – Method.

Load ERP/TF files by pattern and call the typed `prepare_stats` method.

source

`EegFun.preprocess` – Method.

```
preprocess(config::String; base_dir = nothing, log_level = :info,
skip_existing = false, dry_run = false)
```

Run the EEG preprocessing pipeline defined by TOML file `config` .

Keyword Arguments

- `base_dir::Union{String,Nothing} = nothing` : Base directory for relative paths in `config` (defaults to `dirname(abspath(config))`).
- `log_level::Symbol = :info` : Logging verbosity (`:debug` , `:info` , `:warn` , `:error`).
- `skip_existing::Bool = false` : Skip files whose final `_epochs.jld2` already exists.
- `dry_run::Bool = false` : Validate configuration, paths, and input files without writing outputs or processing data.

source

`EegFun.print_config` – Function.

```
print_config(config, [io::IO = stdout])
print_config(config, filename::String)
```

Write `config` in TOML format (with fresh `metadata` prepended) to `io` or `filename` .

source

`EegFun.print_filter_characteristics` – Method.

```
print_filter_characteristics(filter_info::FilterInfo; npoints::Int = 1000,
filter_func = :filtfilt)
```

Log a formatted summary of filter cutoff, transition band, stopband attenuation, and delay characteristics.

source

`EegFun.print_layout_neighbors` – Method.

```
print_layout_neighbors(layout_or_dict[, filename::String])
```

Write electrode neighbors, distances, and inverse-distance interpolation weights as TOML to `filename` (or `stdout` if `filename` is omitted).

source

`EegFun.read_all_data` – Method.

```
read_all_data(path_pattern::String, participant_selection = participants())
read_all_data(::Type{T}, path_pattern::String, participant_selection =
participants())
```

Find and load EEG data from `.jld2` files in `dirname(path_pattern)` whose names contain `basename(path_pattern)` , optionally filtered by `participant_selection` .

source

`EegFun.read_bids` – Method.

```
read_bids(dataset_dir::String; subject::String, task::String,
session::String = "", run::Int = 0, datatype::String = "eeg") ->
ContinuousData
```

Read an EEG recording from a BIDS dataset, integrating `electrodes.tsv` 3D coordinates and `events.tsv` annotations when present.

Keyword Arguments

- `subject::String` : Participant ID (e.g., `"01"` or `"sub-01"`).
- `task::String` : Task label (e.g., `"posner"`).
- `session::String` : Optional session label.
- `run::Int` : Optional run number (`0` omits the run entity).
- `datatype::String` : BIDS data type folder (default: `"eeg"`).

source

`EegFun.read_config` – Method.

```
read_config(config_file::String) -> Dict{String,Any}
```

Load, merge with defaults, and validate a TOML configuration file.

source

`EegFun.read_data` – Method.

```
read_data(filepath::String)
```

Read `EegFunData` (or a vector of `EegFunData`) from a JLD2 file, returning `nothing` if the file is missing, empty, or contains no `EegFunData` objects.

source

`EegFun.read_eeglab` – Method.

```
read_eeglab(filepath::String; preserve_radial_distance::Bool = true)
```

Load an EEGLAB `.set` file and return `ContinuousData`, `EpochData`, or `ErpData` (or `(eeg_data, ica_data::InfoIca)` when ICA decomposition weights are present).

Arguments

- `filepath::String`: Path to `.set` file
- `preserve_radial_distance::Bool`: Preserve radial distances for electrodes with `inc > 90°` (default: `true`)

source

`EegFun.read_fieldtrip` – Method.

```
read_fieldtrip(filepath::String, layout::Layout)
```

Load a FieldTrip `.mat` file and convert it to `ContinuousData`, `EpochData`, or `ErpData`.

Arguments

- `filepath::String`: Path to FieldTrip `.mat` file
- `layout::Layout`: Channel layout

source

`EegFun.read_layout` – Method.

```
read_layout(file::String) -> Layout
```

Read an electrode layout CSV file and return a `Layout` object.

source

`EegFun.read_raw_data` – Method.

```
read_raw_data(filepath::String; header_only::Bool = false, kwargs...)
```

Read raw EEG data from `filepath` (dispatching on `.bdf`, `.vhdr` / `.eeg` / `.vmrk`, `.edf`, `.fif` / `.fiff`, or `.xdf` / `.xdfz` / `.xdf.gz`).

source

`EegFun.read_xdf` – Method.

```
read_xdf(filepath::String; select_streams = nothing, sync = true,
dejitter_timestamps = true, handle_clock_resets = true) ->
ExtensibleDataFormat.XdfData
```

Read an Extensible Data Format (`.xdf`, `.xdfz`, or `.xdf.gz`) Lab Streaming Layer container file. Pass the returned `XdfData` to `create_eegfun_data` to convert it into `ContinuousData`.

Keyword Arguments

- `select_streams::Union{Nothing,Vector{Int}} = nothing` : Stream IDs to load (`nothing` loads all streams)
- `sync::Bool = true` : Enable cross-stream clock synchronization
- `dejitter_timestamps::Bool = true` : Remove timestamp jitter from regularly sampled streams
- `handle_clock_resets::Bool = true` : Detect and correct clock resets within the recording

source

`EegFun.realign!` – Method.

```
realign!(dat::EpochData, realignment_triggers) -> EpochData
realign!(dat_vec::AbstractVector{<:EpochData}, realignment_triggers;
global_interval = nothing, global_n_samples = nothing) ->
AbstractVector{<:EpochData}
```

Realign epoched data in-place so the first matching trigger in `realignment_triggers` becomes `t = 0` , dropping epochs without a matching trigger and cropping all remaining epochs to a shared common time window. Accepts an integer trigger or vector of triggers.

source

`EegFun.realign` – Method.

```
realign(dat::EpochData, realignment_triggers) -> EpochData
realign(dat_vec::AbstractVector{<:EpochData}, realignment_triggers;
global_interval = nothing, global_n_samples = nothing) -> Vector{EpochData}
realign(file_pattern::String, realignment_triggers; input_dir = pwd(),
participant_selection = participants(), output_dir = nothing)
```

Non-mutating and batch file versions of `realign!` .

source

`EegFun.reference` – Method.

```
reference(dat) -> Symbol
```

Return the applied reference scheme.

source

`EegFun.reject_epochs!` – Method.

```
reject_epochs!(dat::EpochData, info::EpochRejectionInfo) -> EpochData
reject_epochs!(dat::AbstractVector{<:EpochData},
info::AbstractVector{<:EpochRejectionInfo})
reject_epochs!(dat::EpochData,
bad_columns::Union{Symbol,AbstractVector{Symbol}}) -> EpochData
reject_epochs!(dat::AbstractVector{<:EpochData},
bad_columns::Union{Symbol,AbstractVector{Symbol}})
```

Remove bad epochs from `dat` in-place using `EpochRejectionInfo` or one or more boolean flag columns.

source

EegFun.reject_epochs – Method.

```
reject_epochs(dat::EpochData, info::EpochRejectionInfo) -> EpochData
reject_epochs(dat::AbstractVector{<:EpochData},
info::AbstractVector{<:EpochRejectionInfo}) -> Vector{EpochData}
reject_epochs(dat::EpochData,
bad_columns::Union{Symbol,AbstractVector{Symbol}}) -> EpochData
reject_epochs(dat::AbstractVector{<:EpochData},
bad_columns::Union{Symbol,AbstractVector{Symbol}}) -> Vector{EpochData}
```

Non-mutating version of `reject_epochs!` .

source

EegFun.rename_channel! – Method.

```
rename_channel!(dat::EegData, rename_dict::Dict{Symbol,Symbol})
```

Rename channels in `dat` and its `Layout` in place according to `rename_dict` (`old => new`), supporting simultaneous swaps (e.g., `Dict(:A => :B, :B => :A)`).

source

EegFun.rename_channel! – Method.

```
rename_channel!(layout::Layout, rename_dict::Dict{Symbol,Symbol}) -> Layout
```

Rename channels in `layout` in place according to `rename_dict` (`old => new`), clearing cached neighbors.

source

EegFun.rename_channel – Method.

```
rename_channel(dat::EegData, rename_dict::Dict{Symbol,Symbol})
```

Return a copy of `dat` with channels renamed according to `rename_dict` .

source

EegFun.rename_channel – Method.

```
rename_channel(layout::Layout, rename_dict::Dict{Symbol,Symbol}) -> Layout
```

Return a copy of `layout` with channels renamed according to `rename_dict` .

source

EegFun.repair_artifacts! – Method.

```
repair_artifacts!(dat::EpochData, artifacts::EpochRejectionInfo;
method::Symbol=:neighbor_interpolation, kwargs...)
```

Repair detected epoch artifacts in-place using `:neighbor_interpolation` (weighted neighbor interpolation from `dat.layout.neighbors`) or `:spherical_spline` (accepts `m::Int=4` , `lambda::Real=1e-5`). Updates `artifacts.repaired` and `artifacts.skipped` .

source

EegFun.repair_artifacts – Method.

```
repair_artifacts(dat::EpochData, artifacts::EpochRejectionInfo;
method::Symbol=:neighbor_interpolation, kwargs...)
```

Non-mutating version of `repair_artifacts!` leaves both `dat` and `artifacts` unmodified.

source

`EegFun.repair_channels!` – Method.

```
repair_channels!(data, channels_to_repair;
method::Symbol=:neighbor_interpolation, kwargs...)
```

Repair bad EEG channels in `ContinuousData` , `ErpData` , `EpochData` , or an `AbstractMatrix` in-place.

Keyword Arguments

- `method::Symbol` : `:neighbor_interpolation` (default; uses `neighbors_dict` or `data.layout.neighbors`) or `:spherical_spline` (accepts `m::Int=4` , `lambda::Real=1e-5`).
- `epoch_selection` : Epoch selector for `EpochData` (default: `epochs()`).

source

`EegFun.repair_channels!` – Method.

```
repair_channels!(data::ContinuousData, repair_info::ContinuousRepairInfo;
method::Symbol = :neighbor_interpolation, kwargs...) -> ContinuousData
```

Repair channels listed in `repair_info.repaired` (populated via `channel_repairable!`) in place using `:neighbor_interpolation` or `:spherical_spline` .

source

`EegFun.repair_channels` – Method.

```
repair_channels(dat, args...; kwargs...)
```

Non-mutating version of `repair_channels!` .

source

`EegFun.rereference!` – Method.

```
rereference!(dat::EegData, reference_selection; channel_selection =
channels())
rereference!(dat::AbstractVector{<:EegData}, reference_selection;
channel_selection = channels())
```

Rereference EEG data in-place. `reference_selection` can be `:avg` , `:mastoid` , `:none` , or a channel name / vector of channel names.

source

`EegFun.rereference` – Method.

```
rereference(dat, reference_selection; channel_selection = channels())
rereference(file_pattern::String; input_dir = pwd(), reference_selection
= :avg,
```

```
            participant_selection = participants(), condition_selection =
            conditions(), output_dir = nothing)
```

Non-mutating or batch JLD2 version of `rereference!`.

source

`EegFun.resample!` – Method.

```
resample!(dat::Union{ContinuousData, ErpData, EpochData}, target_rate::Real)
resample!(data_vec::AbstractVector{<:_ResampleData}, target_rate::Real)
```

Resample EEG data in-place to integer `target_rate` (Hz) using polyphase anti-alias filtering.

source

`EegFun.resample` – Method.

```
resample(dat::Union{ContinuousData,ErpData,EpochData}, target_rate::Real)
resample(data_vec::AbstractVector{<:Union{ContinuousData,ErpData,EpochData}},
target_rate::Real)
resample(file_pattern::String, target_rate::Real; input_dir = pwd(),
participant_selection = participants(), output_dir = nothing)
```

Non-mutating or batch JLD2 version of `resample!`.

source

`EegFun.resample_temporal_data` – Method.

```
resample_temporal_data(
    temporal_data::AbstractArray{<:Real,3},
    original_times::AbstractVector{<:Real},
    target_times::AbstractVector{<:Real};
    method::Symbol = :linear,
) -> (Array{Float64,3}, Vector{Float64})
```

Interpolate temporal model data `[conditions × features × time]` from sorted `original_times` onto `target_times` using `:linear` or `:nearest` interpolation (clamped outside overlap; returns zeros with a warning when time ranges do not overlap).

source

`EegFun.ride_decompose` – Method.

```
ride_decompose(data, comps; outer_iters=4, inner_iters=50, inner_tol=1e-3,
               method=:median, taper_alpha=0.2, times=nothing,
               sample_rate=nothing,
               baseline_range=nothing, rng=Random.default_rng()) -> RideResult
```

Run Residue Iteration DEcomposition (RIDE) on a 3D tensor `(samples, channels, trials)`.

source

`EegFun.rsa` – Method.

```
rsa(
```

```
    epochs::AbstractVector{<:EpochData};
    channel_selection = channels(),
    sample_selection = samples(),
    interval_selection::Interval = times(),
    dissimilarity_measure::Symbol = :correlation,
    average_trials::Bool = true,
    normalize_method::Symbol = :none,
) -> RsaData
rsa(all_participant_epochs::Vector{Vector{EpochData}}; kwargs...) ->
Vector{RsaData}
```

Compute time-resolved Representational Dissimilarity Matrices (RDMs) across conditions for a participant (or vector of participants).

Keyword Arguments

- `channel_selection` : Channel selection predicate (default: `channels()`)
- `sample_selection` : Sample selection predicate (default: `samples()`)
- `interval_selection::Interval` : Time interval filter (default: `times()`)
- `dissimilarity_measure::Symbol` : `:correlation` (`:pearson`), `:spearman` , `:euclidean` , or `:mahalanobis`
- `average_trials::Bool` : Average trials per condition before computing RDMs (default: `true`)
- `normalize_method::Symbol` : `:none` (default), `:zscore` , `:rank` , or `:minmax`

source

`EegFun.rsa_crossvalidated` – Method.

```
rsa_crossvalidated(
    epochs::AbstractVector{<:EpochData};
    channel_selection = channels(),
    sample_selection = samples(),
    interval_selection::Interval = times(),
    dissimilarity_measure::Symbol = :correlation,
    cv_method::Symbol = :splithalf,
    n_folds::Int = 5,
    n_iterations::Int = 100,
    normalize_method::Symbol = :none,
    rng::AbstractRNG = Random.default_rng(),
) -> RsaData
```

Compute unbiased cross-validated RDMs across disjoint trial splits (Walther et al., 2016), supporting `:splithalf` , `:leaveoneout` , and `:kfold` cross-validation and `:correlation` (`:pearson`), `:spearman` , `:euclidean` , and `:mahalanobis` (`:crossnobis`) measures.

source

`EegFun.run_ica` – Method.

```
run_ica(dat::ContinuousData; n_components = nothing, sample_selection =
samples(),
        interval_selection = times(), channel_selection = channels(),
        include_extra = false,
        percentage_of_data = 100.0, algorithm = :infomax, use_gpu = false,
        params = IcaPrms(), rng = Random.default_rng()) -> InfoIca
run_ica(epoched_data::AbstractVector{<:EpochData}; remove_duplicates = true,
kwargs...) -> InfoIca
run_ica(epoched_data::EpochData; kwargs...) -> InfoIca
```

Run Independent Component Analysis (ICA) on continuous or epoched EEG data (`:infomax` , `:infomax_extended` , `:picard` , `:picard_extended` , or `:amica`), defaulting `n_components` to the effective numerical rank of the data.

Examples

```
ica_result = run_ica(dat)
ica_result = run_ica(dat; sample_selection =
samples_not(:is_extreme_value_100), percentage_of_data = 25.0)
ica_result = run_ica(epochs_vec; algorithm = :picard_extended)
```

source

`EegFun.sample_rate` – Method.

```
sample_rate(dat) -> Int
```

Return the sample rate in Hz.

source

`EegFun.samples` – Method.

```
samples()
samples(column::Symbol)
samples(predicate::Function)
```

Create a row/sample selection predicate for `sample_selection` arguments.

source

`EegFun.samples_and` – Method.

```
samples_and(columns::Vector{Symbol})
```

Select rows where all boolean columns in `columns` are `true` .

source

`EegFun.samples_and_not` – Method.

```
samples_and_not(columns::Vector{Symbol})
```

Select rows where not all boolean `columns` are `true` .

source

`EegFun.samples_not` – Method.

```
samples_not(column::Symbol)
```

Select rows where boolean `column` is `false` .

source

`EegFun.samples_or` – Method.

```
samples_or(columns::Vector{Symbol})
```

Select rows where any boolean column in `columns` is `true` .

source

`EegFun.samples_or_not` – Method.

```
samples_or_not(columns::Vector{Symbol})
```

Select rows where none of the boolean `columns` are `true` .

source

`EegFun.search_sequence` – Method.

```
search_sequence(array, sequences::Vector{<:Vector}; ignore_values = [0],
sort_indices = true) -> Vector{Vector{Int}}
search_sequence(array, sequence::Vector; ignore_values = [0], sort_indices =
true) -> Vector{Vector{Int}}
search_sequence(array, value::Integer; ignore_values = [0]) -> Vector{Int}
search_sequence(array, range::UnitRange; ignore_values = [0], sort_indices =
true) -> Vector{Int}
search_sequence(array, ranges::Vector{<:UnitRange}; ignore_values = [0],
sort_indices = true) -> Vector{Int}
```

Search `array` for trigger onsets, ranges, or multi-element trigger sequences (supporting `:any` wildcards, `UnitRange` s, and set-match `Vector{Int}` elements).

source

`EegFun.section` – Method.

```
section(title::String; width::Int = 80) -> String
```

Format a triple-line section header with `title` centered between dashes.

source

`EegFun.setup_global_logging` – Method.

```
setup_global_logging(log_file::String; kwargs...)
setup_global_logging(f::Function, log_file::String; kwargs...)
```

Configure session-wide file logging (equivalent to `setup_logging(...; is_global = true)`).

source

`EegFun.setup_logging` – Method.

```
setup_logging(log_file::String; log_level::Symbol = :info, is_global::Bool =
false, include_kwargs::Bool = !is_global)
setup_logging(f::Function, log_file::String; is_global::Bool = false,
kwargs...)
```

Configure logging to both `stdout` and `log_file`. Local file logs take precedence over global logs.

source

`EegFun.shift_matrix!` – Method.

```
shift_matrix!(dest::AbstractMatrix{T}, src::AbstractMatrix{T}, shift::Int)
where {T}
```

Shift each column of `src` (`samples × channels`) by `shift` samples into `dest`.

source

`EegFun.shift_matrix` – Method.

```
shift_matrix(src::AbstractMatrix{T}, shift::Int) where {T}
```

Out-of-place version of `shift_matrix!`.

source

`EegFun.shift_signal!` – Method.

```
shift_signal!(dest::AbstractVector{T}, src::AbstractVector{T}, shift::Int)
where {T}
```

Shift `src` by `shift` samples into `dest`, zero-padding vacated elements (`shift > 0` shifts right, `shift < 0` shifts left).

source

`EegFun.shift_signal` – Method.

```
shift_signal(src::AbstractVector{T}, shift::Int) where {T}
```

Out-of-place version of `shift_signal!`.

source

`EegFun.show_parameter_info` – Method.

```
show_parameter_info(; parameter_name::String = "")
```

Display documentation for all configuration parameters, a section prefix, or a single parameter.

source

`EegFun.subset` – Method.

```
subset(file_pattern::String; input_dir = pwd(), participant_selection =
participants(), output_dir = nothing, kwargs...)
```

Batch-subset JLD2 files matching `file_pattern` using any `subset` keyword arguments.

source

`EegFun.subset` – Method.

```
subset(dat::EegData; channel_selection = channels(), sample_selection =
samples(), interval_selection = times(), epoch_selection = epochs(),
include_extra = false)
subset(datasets::AbstractVector{<:EegData}; condition_selection =
conditions(), kwargs...)
```

Return a subsetted copy of `dat` (or vector of condition datasets) filtered by channel, sample/time interval, epoch, and/or condition predicates.

source

`EegFun.subset_bad_data` – Method.

```
subset_bad_data(data_path::String, threshold::Real;
subset_directory::String="excluded")
```

Move files for participants whose epoch retention falls below `threshold` % in any condition (from `epoch_summary.jld2` in `data_path`) into `subset_directory` , and write filtered `epoch_summary_subset.jld2` and `file_summary_subset.jld2` files.

source

`EegFun.subset_layout!` – Method.

```
subset_layout!(layout::Layout; channel_selection = channels()) -> Layout
```

Subset `layout` in place to channels matching `channel_selection` , updating cached neighbors.

source

`EegFun.subset_layout` – Method.

```
subset_layout(layout::Layout; channel_selection = channels()) -> Layout
```

Return a subsetted copy of `layout` containing channels matching `channel_selection` .

source

`EegFun.subtract_ica_components!` – Method.

```
subtract_ica_components!(dat, ica::InfoIca; component_selection = nothing)
```

Remove selected ICA components in-place from a `DataFrame` , `Vector{DataFrame}` , `ContinuousData` , `EpochData` , or `AbstractVector{<:EpochData}` , storing removed activations in `ica.removed_activations` for exact restoration via `add_ica_components!` .

An explicit `component_selection` (e.g. `components([1, 3])` or `components()`) is required.

source

`EegFun.subtract_ica_components` – Method.

```
subtract_ica_components(dat, ica::InfoIca; component_selection = nothing) ->
(cleaned_data, ica_updated)
```

Non-mutating ICA component removal; copies both `dat` and `ica` and returns `(cleaned_data, ica_updated)` .

source

EegFun.summarize_electrode_repairs – Method.

```
summarize_electrode_repairs(continuous_repairs::Vector{ContinuousRepairInfo})
-> DataFrame
summarize_electrode_repairs(file_pattern::String; input_dir::String = pwd()) -
> Tuple{DataFrame,DataFrame}
```

Summarize continuous-level electrode repairs across participants (either directly from a `Vector{ContinuousRepairInfo}` or loaded from matching `*_artifact_info.jld2` files in `input_dir`).

source

EegFun.summarize_ica_components – Method.

```
summarize_ica_components(ica_components::Vector{ArtifactComponents}) ->
Tuple{DataFrame,DataFrame}
summarize_ica_components(file_pattern::String; input_dir::String = pwd()) ->
Tuple{DataFrame,DataFrame}
```

Summarize removed ICA artifact components per participant and on average across participants.

source

EegFun.tail – Method.

```
tail(dat::EegData; n = 5)
```

Display and return a view of the last `n` rows of `all_data(dat)`.

source

EegFun.test_against_chance – Method.

```
test_against_chance(decoded_list::Vector{DecodedData}; alpha::Real = 0.05,
correction_method::Symbol = :none)
```

Perform a right-tailed one-sample t-test against chance level across participants at each time point.

Keyword Arguments

- `alpha::Real` : Significance threshold in `(0, 1)` (default: `0.05`)
- `correction_method::Symbol` : `:none` (default) or `:bonferroni`

Returns

- `DecodingStatisticsResult`

source

EegFun.test_against_chance_cluster – Method.

```
test_against_chance_cluster(
    decoded_list::Vector{DecodedData};
    alpha::Real = 0.05,
```

```
    n_permutations::Int = 1000,
    cluster_statistic::Symbol = :sum,
    show_progress::Bool = true,
    rng::AbstractRNG = Random.default_rng(),
)
```

Perform a right-tailed 1D temporal cluster-based sign-flip permutation test against chance level across participants.

Keyword Arguments

- `alpha::Real` : Cluster-forming and cluster-level significance threshold (default: `0.05`)
- `n_permutations::Int` : Number of random sign-flip permutations (default: `1000`)
- `cluster_statistic::Symbol` : `:sum` (default) or `:max`
- `show_progress::Bool` : Show progress bar (default: `true`)
- `rng::AbstractRNG` : Random number generator for reproducible sign flips

Returns

- `DecodingStatisticsResult`

source

`EegFun.tf_baseline!` – Method.

```
tf_baseline!(tf_data::TimeFreqData, baseline_interval; method=:db)
tf_baseline!(tf_data::TimeFreqEpochData, baseline_interval; method=:db)
```

Apply baseline correction to `TimeFreqData` or single-trial `TimeFreqEpochData` in-place.

Supported `method` values: `:db` (`10 * log10(P / B)`), `:absolute` (`P - B`), `:relative` (`P / B`), `:relchange` (`(P - B) / B`), `:percent` (`100 * (P - B) / B`), `:normchange` (`(P - B) / (P + B)`), and `:zscore` (`(P - B) / SD_B`).

Example

```
tf_baseline!(tf_data, (-0.3, 0.0); method=:db)
```

source

`EegFun.tf_baseline` – Method.

```
tf_baseline(file_pattern::String, baseline_interval;
            method=:db, input_dir=pwd(), output_dir=nothing,
            participant_selection=participants(),
            condition_selection=conditions())
```

Apply baseline correction to `TimeFreqData` or `TimeFreqEpochData` `.jld2` files in batch.

Example

```
tf_baseline("tf_epochs_wavelet", (-0.3, 0.0); method=:db)
```

source

`EegFun.tf_morlet` – Method.

```
tf_morlet(dat::EpochData;
          channel_selection=channels(),
          interval_selection::Interval=times(),
          frequencies::AbstractVector{<:Real}=range(1, 40, length=40),
          cycles::Union{Real,Tuple{Real,Real}}=7,
          pad::Union{Nothing,Symbol}=nothing,
          return_trials::Bool=false,
          return_phase::Bool=false,
          filter_edges::Bool=true,
          use_gpu::Bool=false)
```

Morlet wavelet time-frequency decomposition using MNE-style discrete L^2 wavelet normalization ($\| W \|_2 = \sqrt{2}$).

Returns `TimeFreqData` when `return_trials=false` or `TimeFreqEpochData` when `return_trials=true`.

Examples

```
tf_data = tf_morlet(epochs; frequencies=range(2, 40, length=30), cycles=7)
tf_trials = tf_morlet(epochs; frequencies=logrange(2, 40, length=20),
cycles=(3, 10), return_trials=true, return_phase=true)
```

source

`EegFun.tf_morlet` – Method.

```
tf_morlet(file_pattern::String;
          input_dir::String=pwd(),
          output_dir::Union{String,Nothing}=nothing,
          participant_selection=participants(),
          condition_selection=conditions(),
          kwargs...)
```

Batch Morlet wavelet time-frequency analysis on `EpochData` `.jld2` files.

source

`EegFun.tf_multitaper` – Method.

```
tf_multitaper(dat::EpochData;
              channel_selection=channels(),
              interval_selection::Interval=times(),
              frequencies::AbstractVector{<:Real}=range(1, 40, length=40),
              time_steps::Real=0.05,
              cycles::Real,
              frequency_smoothing::Union{Nothing,Real}=nothing,
              pad::Union{Nothing,Symbol}=nothing,
              return_trials::Bool=false,
              return_phase::Bool=false,
              filter_edges::Bool=true,
```

```
                use_gpu::Bool=false)
```

Multitaper time-frequency decomposition using orthogonal DPSS (Slepian) tapers (`mtmconvol`). Uses adaptive window lengths (`cycles / frequency`) and `frequency_smoothing * frequency` bandwidth (defaulting to `0.4 * frequency` when `frequency_smoothing = nothing`).

Returns `TimeFreqData` when `return_trials=false` or `TimeFreqEpochData` when `return_trials=true` .

Examples

```
tf_data = tf_multitaper(epochs; cycles=5)
tf_log = tf_multitaper(epochs; frequencies=logrange(2, 80, length=30),
cycles=5, frequency_smoothing=0.4)
```

source

`EegFun.tf_stft` – Method.

```
tf_stft(dat::EpochData;
        channel_selection=channels(),
        interval_selection::Interval=times(),
        frequencies::AbstractVector{<:Real}=range(1, 40, length=40),
        time_steps::Real=0.05,
        window_length::Union{Nothing,Real}=nothing,
        cycles::Union{Nothing,Real}=nothing,
        pad::Union{Nothing,Symbol}=nothing,
        return_trials::Bool=false,
        return_phase::Bool=false,
        filter_edges::Bool=true,
        use_gpu::Bool=false)
```

Short-Time Fourier Transform (STFT) time-frequency decomposition using a sliding Hanning window. Specify **exactly one** of `window_length` (fixed window in seconds) or `cycles` (adaptive window `cycles / freq`).

Returns `TimeFreqData` when `return_trials=false` or `TimeFreqEpochData` when `return_trials=true` .

Examples

```
tf_data = tf_stft(epochs; window_length=0.3)
tf_adaptive = tf_stft(epochs; frequencies=2:1:30, cycles=7,
interval_selection=times(-0.5, 1.5), time_steps=0.05)
```

source

`EegFun.time_vector` – Method.

```
time_vector(dat) -> Vector{Float64}
```

Return the `:time` vector in seconds from `dat` (or its first epoch for `MultiDataFrameEeg`).

source

`EegFun.times` – Method.

```
times()
times(t::Real)
times(start::Real, stop::Real)
times(interval::Tuple{Real,Real})
```

Construct a time-interval specification (`AllSelection` or `TimeSelection`) for `interval_selection` .

source

`EegFun.to_data_frame` – Method.

```
to_data_frame(dat) -> DataFrame
```

Concatenate all epoch `DataFrame` s in a `MultiDataFrameEeg` (or vector of them) into a single `DataFrame` .

source

`EegFun.trigger_count` – Method.

```
trigger_count(dat::ContinuousData) -> TriggerInfo
trigger_count(df::DataFrame) -> TriggerInfo
trigger_count(dat::BiosemiDataFormat.BiosemiData) -> TriggerInfo
trigger_count(dat::EuropeanDataFormat.EdfData) -> TriggerInfo
trigger_count(dat::BrainVisionDataFormat.BrainVisionData) -> TriggerInfo
trigger_count(dat::ExtensibleDataFormat.XdfData) -> TriggerInfo
```

Count occurrences of each non-zero trigger value in continuous or raw EEG data, returning a formatted `TriggerInfo` summary table.

source

`EegFun.trigger_info` – Method.

```
trigger_info(dat::EpochData) ->
Vector{NamedTuple{(:sequence, :t0),Tuple{Vector{Int},Int}}}
trigger_info(dat_vec::AbstractVector{<:EpochData}) -> Dict{String,Vector{...}}
trigger_info(file_pattern::String; input_dir = pwd(), participant_selection =
participants())
```

Return the unique non-zero trigger sequences and `t == 0` trigger values observed per condition in epoched data.

source

`EegFun.unique_rejections` – Method.

```
unique_rejections(rejections)
```

De-duplicate `Rejection` entries by `(channel, epoch)` , or extract unique rejections from `EpochRejectionInfo` .

source

`EegFun.unmirror` – Function.

```
unmirror(dat::EegData, side::Symbol = :both)
unmirror(data_vec::Vector{<:EegData}, side::Symbol = :both)
```

Non-mutating version of `unmirror!` .

source

`EegFun.unmirror!` – Function.

```
unmirror!(dat::EegData, side::Symbol = :both)
unmirror!(data_vec::Vector{<:EegData}, side::Symbol = :both)
```

Remove mirrored boundary segments in-place, restoring original length. `side` must match `mirror!` .

source

`EegFun.validate_config` – Method.

```
validate_config(config::String; base_dir::Union{String,Nothing} = nothing)
```

Validate a preprocessing pipeline configuration file (`dry_run = true`) without processing data or writing output files.

source

`EegFun.validate_layout` – Method.

```
validate_layout(layout::Layout) -> Layout
```

Validate that `layout` contains required `:label` , `:inc` , and `:azi` columns.

source

`EegFun.version_info` – Method.

```
version_info() -> Dict{String,Any}
```

Return a dictionary containing `"julia_version"` , `"EegFun_version"` , and `"timestamp"` .

source

`EegFun.viewer` – Method.

```
viewer(dat)
```

Display `dat` (or `all_data(dat)` when defined) using `Main.vscodedisplay` if available, falling back to `display` .

source

`EegFun.woody_filter` – Method.

```
woody_filter(data, window_radius; template=nothing,
initial_latencies=nothing,
             max_lag=nothing, rng=Random.default_rng()) -> Vector{Int}
```

Estimate single-trial sample shifts via multi-channel cross-covariance against `template` (or leave-one-out trial averages when `template === nothing`), centered around the median shift.

source

Chapter 38

Plotting

38.1 Plotting & Interactive Demos

All public visualization functions, themes, topography helpers, and interactive teaching demos in `EegFun.jl`.

Index

- EegFun.add_topo_rois!
- EegFun.plot_artifact_components
- EegFun.plot_artifact_detection
- EegFun.plot_artifact_rejection
- EegFun.plot_artifact_repair
- EegFun.plot_channel_spectrum
- EegFun.plot_channel_spectrum
- EegFun.plot_channel_summary
- EegFun.plot_channel_summary!
- EegFun.plot_confusion_matrix
- EegFun.plot_correlation_heatmap
- EegFun.plot_correlation_heatmap!
- EegFun.plot_databrowser
- EegFun.plot_decoding
- EegFun.plot_ecg_component_features
- EegFun.plot_eog_component_features
- EegFun.plot_epochs
- EegFun.plot_erp
- EegFun.plot_erp!
- EegFun.plot_erp_errorbar
- EegFun.plot_erp_errorbar!
- EegFun.plot_erp_filter_gui
- EegFun.plot_erp_image
- EegFun.plot_erp_measurement_gui
- EegFun.plot_erp_measurements
- EegFun.plot_erp_stats
- EegFun.plot_filter_response
- EegFun.plot_gfp

- EegFun.plot_gui
- EegFun.plot_ica_component_activation
- EegFun.plot_ica_component_spectrum
- EegFun.plot_joint_probability
- EegFun.plot_joint_probability!
- EegFun.plot_layout_2d
- EegFun.plot_layout_2d!
- EegFun.plot_layout_3d
- EegFun.plot_layout_3d!
- EegFun.plot_line_noise_components
- EegFun.plot_model_correlations
- EegFun.plot_rdm_heatmap
- EegFun.plot_rdm_timecourse
- EegFun.plot_rsa
- EegFun.plot_spatial_kurtosis_components
- EegFun.plot_stat_heatmap
- EegFun.plot_stat_heatmap!
- EegFun.plot_tf
- EegFun.plot_tf_stats
- EegFun.plot_topography
- EegFun.plot_topography
- EegFun.plot_topography
- EegFun.plot_topography!
- EegFun.plot_topography_3d
- EegFun.plot_topography_stats
- EegFun.plot_topography_stats
- EegFun.plot_trigger_overview
- EegFun.plot_trigger_timing
- EegFun.signal_example_composition
- EegFun.signal_example_convolution
- EegFun.signal_example_decoding
- EegFun.signal_example_dotproduct
- EegFun.signal_example_ecg_einthoven
- EegFun.signal_example_ecg_hrv
- EegFun.signal_example_emg
- EegFun.signal_example_heog
- EegFun.signal_example_ica_clt
- EegFun.signal_example_ica_geometry

- EegFun.signal_example_ica_mixture
- EegFun.signal_example_ica_optimization
- EegFun.signal_example_ica_separation
- EegFun.signal_example_ica_sphering
- EegFun.signal_example_sampling
- EegFun.signal_example_tf
- EegFun.simulate_erp
- EegFun.theme_eegfun

Plotting Functions

`EegFun.plot_artifact_components` – Method.

```
plot_artifact_components(ica::InfoIca, artifacts::ArtifactComponents;
kwargs...) -> NamedTuple
```

Plot scalp topographies for all identified ICA artifact components grouped by category (`vEOG` , `hEOG` , `ECG` , `Line Noise` , `Channel Noise`).

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = ""` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the topography
- `colorrange::Nothing = nothing` : Color scale limits as (min, max) tuple (nothing for auto)
- `component_selection::EegFun.var"#components##0#components##1" = EegFun.var"#components##0#components##1"()` : Function that returns boolean vector for component filtering
- `dims::Nothing = nothing` : Grid dimensions (rows, cols). If nothing, calculates best square-ish grid
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `gridscale::Int64 = 75` : Grid resolution for interpolation

- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Float64 = 1.0` : Radius of the head shape in mm.
- `highlight_channels::Nothing = nothing` : Highlight channel groups as a NamedTuple or Vector of NamedTuples. Each group: (channels=[:Cz, :Pz], color=:white, size=8, marker=:circle)
- `interactive::Bool = true` : Whether to enable interactive features
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `method::Symbol = :thin_plate` : Interpolation method: :multiquadratic, :inverse*multiquadratic, :gaussian, :inverse*quadratic, :thin*plate, :polyharmonic, :shepard,* See https://eljungsk.github.io/ScatteredInterpolation.jl/dev/methods/ for details on methods.
- `num_levels::Int64 = 20` : Number of contour levels (for ICA plots).
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for display labels (:s or :ms). Only affects title strings — all intervals and selections remain in seconds.
- `use_global_scale::Bool = false` : Do multiple ICA topoplots share the same color scale based on min/max across all components?
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_artifact_detection` – Method.

```
plot_artifact_detection(epochs::EpochData, artifacts::EpochRejectionInfo;
channel_selection=channels(), kwargs...)
plot_artifact_detection(epochs::AbstractVector{<:EpochData},
artifacts::AbstractVector{<:EpochRejectionInfo}; kwargs...)
```

Interactive plot of epoch artifact detection results with epoch and artifact navigation controls.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Whether to add origin lines at x=0 and y=0
- `alpha_normal::Float64 = 0.2` : Transparency for normal channels.
- `alpha_rejected::Float64 = 1.0` : Transparency for rejected channels.
- `colormap::Nothing = nothing` : Colormap (e.g., :Set1_9, :tab20).
- `display_plot::Bool = true` : Display plot.
- `linewidth_normal::Int64 = 1` : Line width for normal channels.
- `linewidth_rejected::Int64 = 2` : Line width for rejected channels.
- `plot_title::Nothing = nothing` : Custom plot title override.
- `selection_threshold::Int64 = 50` : Distance threshold for channel selection.
- `show_title::Bool = true` : Whether to show the plot title.
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple or nothing for auto-scaling
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple or nothing for auto-scaling
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

Legend Options

- `legend::Bool = true` : Whether to display channel legend.

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_artifact_rejection` – Method.

```
plot_artifact_rejection(epochs_raw::EpochData, epochs_rejected::EpochData,
artifacts::EpochRejectionInfo; channel_selection=channels(), kwargs...)
```

Interactive before/after comparison between original and post-rejection epochs aligned by epoch number. Rejected epochs display a blank lower panel with red axis spines.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Whether to add origin lines at x=0 and y=0
- `alpha_normal::Float64 = 0.2` : Transparency for normal channels.
- `alpha_rejected::Float64 = 1.0` : Transparency for rejected channels.
- `colormap::Nothing = nothing` : Colormap (e.g., :Set1_9, :tab20).
- `display_plot::Bool = true` : Display plot.
- `linewidth_normal::Int64 = 1` : Line width for normal channels.
- `linewidth_rejected::Int64 = 2` : Line width for rejected channels.
- `plot_title::Nothing = nothing` : Custom plot title override.
- `selection_threshold::Int64 = 50` : Distance threshold for channel selection.
- `show_title::Bool = true` : Whether to show the plot title.
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple or nothing for auto-scaling
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple or nothing for auto-scaling
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

Legend Options

- `legend::Bool = true` : Whether to display channel legend.

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_artifact_repair` – Method.

```
plot_artifact_repair(epochs_raw::EpochData, epochs_repaired::EpochData,
artifacts::EpochRejectionInfo; channel_selection=channels(), kwargs...)
```

Interactive before/after comparison of original and repaired epochs with navigation controls.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Whether to add origin lines at x=0 and y=0
- `alpha_normal::Float64 = 0.2` : Transparency for normal channels.

- `alpha_rejected::Float64 = 1.0` : Transparency for rejected channels.
- `colormap::Nothing = nothing` : Colormap (e.g., :Set1_9, :tab20).
- `display_plot::Bool = true` : Display plot.
- `linewidth_normal::Int64 = 1` : Line width for normal channels.
- `linewidth_rejected::Int64 = 2` : Line width for rejected channels.
- `plot_title::Nothing = nothing` : Custom plot title override.
- `selection_threshold::Int64 = 50` : Distance threshold for channel selection.
- `show_title::Bool = true` : Whether to show the plot title.
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple or nothing for auto-scaling
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple or nothing for auto-scaling
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

Legend Options

- `legend::Bool = true` : Whether to display channel legend.

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_channel_spectrum` – Method.

```
plot_channel_spectrum(dat; kwargs...)
```

Plot power spectrum for specified channels from EEG data with interactive controls.

Arguments

- `dat::SingleDataFrameEeg` : The EEG data object

Keyword Arguments

- `sample_selection::Function` : Function that returns boolean vector for sample filtering (default: samples() - all samples)
- `channel_selection::Function` : Function that returns boolean vector for channel filtering (default: channels() - all channels)
- `window_size::Int` : Size of the FFT window for spectral estimation (default: 1024)
- `overlap::Real` : Overlap between windows for Welch's method (default: 0.5)
- `max_freq::Real` : Maximum frequency to display in Hz (default: 200.0)
- `xscale::Symbol` : Initial scale for x-axis, one of :linear or :log10 (default: :linear)
- `yscale::Symbol` : Initial scale for y-axis, one of :linear or :log10 (default: :linear)
- `unit::Symbol` : Power unit, one of :linear (μV²/Hz) or :dB (decibels) (default: :linear)

- `window_function::Function` : Window function to use for spectral estimation (default: DSP.hanning)
- `show_freq_bands::Bool` : Whether to show frequency band indicators (default: true)
- `legend::Bool` : Whether to show the legend (default: true)
- `display_plot::Bool` : Whether to display the plot (default: true)

Features

- Interactive checkboxes to toggle between linear and log scales for both axes
- Automatic multi-column legend for many channels
- Frequency band indicators (δ, θ, α, β, γ) below the main plot
- Responsive grid layout with controls in the top-right corner

Returns

- `fig::Figure` : The Makie Figure containing the power spectrum plot
- `ax::Axis` : The axis containing the plot

Examples

```
# Basic usage with interactive controls
fig, ax = plot_channel_spectrum(dat)

# With specific channels
fig, ax = plot_channel_spectrum(dat,
    channel_selection = channels([:Fp1, :Fp2]))

# With log scales
fig, ax = plot_channel_spectrum(dat,
    xscale = :log10,
    yscale = :log10)

# With dB units (EEGLAB-style)
fig, ax = plot_channel_spectrum(dat, unit = :dB)
```

source

`EegFun.plot_channel_spectrum` – Method.

```
plot_channel_spectrum(dat::SpectrumData; kwargs...)
```

Plot a pre-computed power spectrum with interactive controls.

source

`EegFun.plot_channel_summary!` – Method.

```
plot_channel_summary!(fig::Figure, ax::Axis, dat::DataFrame, col;
channel_selection=channels(), kwargs...) -> Axis
plot_channel_summary(dat::DataFrame, col; channel_selection=channels(),
kwargs...) -> NamedTuple
```

Plot per-channel summary bar charts (with optional `average_over` 95% CI error bars and multi-column grids) from a summary `DataFrame` .

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `average_over::Nothing = nothing` : Column to average over (e.g., :epoch). If specified, will compute mean ± 95% CI.
- `bar_alpha::Float64 = 0.7` : Bar alpha.
- `bar_color::Symbol = :steelblue` : Bar color.
- `bar_width::Float64 = 0.8` : Bar width.
- `dims::Nothing = nothing` : Tuple (rows, cols) for grid layout. default = *best*rect(n_columns).
- `display_plot::Bool = true` : Display plot.
- `error_color::Symbol = :black` : Error bar color.
- `error_linewidth::Int64 = 2` : Error bar linewidth.
- `label_fontsize::Int64 = 14` : Font size for axis labels.
- `plot_title::Nothing = nothing` : Plot title.
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for title.
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `show_title::Bool = true` : Whether to show the plot title.
- `sort_values::Bool = false` : Sort the bars by the values in the `col` column in descending order.
- `tick_fontsize::Int64 = 12` : Font size for tick labels.
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlabel::String = "Electrode"` : x-axis label.
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `xtick_rotation::Float64 = 0.7853981633974483` : Rotation angle for x-axis tick labels.
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylabel::String = ""` : y-axis label
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_channel_summary` – Method.

```
plot_channel_summary(dat::DataFrame, col; channel_selection=channels(),
kwargs...) -> NamedTuple
```

Create a figure and plot per-channel summary bar charts (single or multi-column). See `plot_channel_summary!` for keyword options.

source

`EegFun.plot_confusion_matrix` – Method.

```
plot_confusion_matrix(decoded::DecodedData; time_point::Union{Float64, Int,
Nothing} = nothing, kwargs...)
```

Plot confusion matrix for decoding results, where rows (`i`) correspond to True class and columns (`j`) correspond to Predicted class.

Arguments

- `decoded::DecodedData` : DecodedData object containing confusion matrices
- `time_point::Union{Float64, Int, Nothing}` : Time point to plot (in seconds or index). If nothing, plots average across all time points
- `kwargs` : Additional keyword arguments

Examples

```
# Plot confusion matrix at a specific time point
plot_confusion_matrix(decoded, time_point=0.3)

# Plot average confusion matrix
plot_confusion_matrix(decoded)
```

source

`EegFun.plot_correlation_heatmap!` – Method.

```
plot_correlation_heatmap!(fig::Figure, ax::Axis, corr_df::DataFrame;
kwargs...) -> Axis
plot_correlation_heatmap(corr_df::DataFrame; kwargs...) -> NamedTuple
```

Plot a correlation matrix `DataFrame` (first column row labels, remaining columns correlation values) as a heatmap.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = "Correlation"` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to display the colorbar
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)

- `colormap::Nothing = nothing` : Colormap for the heatmap
- `colorrange::Tuple{Int64, Int64} = (-1, 1)` : Color range for the heatmap and colorbar (should be −1 to 1 for correlations)
- `display_plot::Bool = true` : Whether to display the plot
- `label_fontsize::Int64 = 14` : Font size for axis labels
- `mask_range::Nothing = nothing` : Optional tuple (min*val*, *max*val) to mask correlations within this range
- `nan_color::Symbol = :transparent` : Color for NaN values
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `show_title::Bool = true` : Whether to show the plot title
- `tick_fontsize::Int64 = 12` : Font size for tick labels
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlabel::String = ""` : Label for x-axis
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `xtick_rotation::Float64 = 0.7853981633974483` : Rotation angle for x-axis tick labels
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylabel::String = ""` : Label for y-axis
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid
- `ytick_rotation::Int64 = 0` : Rotation angle for y-axis tick labels

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_correlation_heatmap` – Method.

```
plot_correlation_heatmap(corr_df::DataFrame; kwargs...) -> NamedTuple
```

Create a figure and plot a correlation matrix `DataFrame` as a heatmap. See `plot_correlation_heatmap!` for keyword options.

source

`EegFun.plot_databrowser` – Function.

```
plot_databrowser(dat::EegData, ica = nothing; screen = nothing, kwargs...)
plot_databrowser(; launcher::Bool = false, screen = nothing, kwargs...)
```

```
plot_databrowser(filename::String, ica = nothing; input_dir = pwd(),
participant_selection = participants(), screen = nothing, kwargs...)
plot_databrowser(data::Vector{<:EegData}, ica = nothing; screen = nothing,
kwargs...)
```

Launch the interactive EEG data browser for continuous, epoched, or ERP datasets, with optional dynamic ICA component subtraction, filtering, re-referencing, channel repair, and region selection. Called with no arguments, opens the bundled `example1.bdf` dataset (`launcher = true` opens the drag-and-drop file launcher).

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `channel_line_alpha::Int64 = 1` : Transparency for channel lines
- `channel_line_width::Int64 = 1` : Line width for channel lines
- `channel_offset_margin::Float64 = 0.9` : Margin factor for channel offset range
- `channel_offset_scale::Int64 = 1500` : Scale factor for channel vertical offset
- `cleaned_channel_color::Symbol = :green` : Color for cleaned/corrected channels (Data - ICA)
- `default_hp_freq::Float64 = 0.1` : Default high-pass filter frequency in Hz
- `default_lp_freq::Float64 = 40.0` : Default low-pass filter frequency in Hz
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (50, 30, 50, 50)` : Figure padding as (left, right, bottom, top)
- `figure_size::Tuple{Int64, Int64} = (1300, 800)` : Figure size as (width, height)
- `scale_indicator_position::Tuple{Float64, Float64} = (0.92, 0.96)` : Scale indicator position as (x, y) in axis coordinates (0-1)
- `scale_indicator_value::Float64 = 100.0` : Scale indicator value in µV
- `scroll_step::Int64 = 200` : Number of samples to scroll per arrow key press
- `selected_channel_color::Symbol = :black` : Color for selected channels
- `selection_alpha::Float64 = 0.1` : Transparency for selection rectangle
- `selection_color::Symbol = :blue` : Color for selection rectangle
- `show_cleaned_ica::Bool = false` : Whether to show cleaned/corrected channels (Data - ICA) by default
- `show_scale_indicator::Bool = true` : Show scale indicator bar
- `theme_fontsize::Nothing = nothing` : Font size for theme and UI elements
- `unselected_channel_color::Symbol = :darkgrey` : Color for unselected channels
- `xlabel::String = "Time (s)"` : X-axis label
- `ylabel::String = "Amplitude (µV)"` : Y-axis label
- `zoom_step::Int64 = 100` : Y-axis zoom increment per arrow key press

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_decoding` – Method.

```
plot_decoding(decoded::DecodedData; kwargs...)
```

Plot decoding accuracy over time.

Creates a line plot showing classification accuracy at each time point, with optional error bars and chance level reference line.

Arguments

- `decoded::DecodedData` : DecodedData object containing decoding results
- `kwargs` : Additional keyword arguments (see PLOT*DECODING*KWARGS)

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Add origin lines at x=0 and y=chance (true/false)
- `chance_color::Symbol = :gray` : Color for chance level line
- `chance_linestyle::Symbol = :dash` : Line style for chance level
- `chance_linewidth::Int64 = 1` : Line width for chance level
- `color::Nothing = nothing` : Color for decoding curve
- `colorbar_plot::Bool = true` : Show colorbar on confusion matrix (true/false)
- `colormap::Nothing = nothing` : Colormap for confusion matrix
- `colorrange::Nothing = nothing` : Color scale limits for confusion matrix (default: (0, 1))
- `display_plot::Bool = true` : Display the plot (true/false)
- `error_alpha::Float64 = 0.3` : Transparency for error shading
- `error_color::Symbol = :blue` : Color for error bars
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `interactive::Bool = true` : Enable interactive features (true/false)
- `linestyle::Nothing = nothing` : Line style
- `linewidth::Int64 = 2` : Line width for decoding curve
- `plot_title::Nothing = nothing` : Plot title

- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `show_error::Bool = true` : Show standard error bars (true/false)
- `show_significance::Bool = true` : Show significance markers when statistics are provided (true/false)
- `show_title::Bool = true` : Show plot title (true/false)
- `sig_alpha::Float64 = 0.5` : Transparency for significance markers
- `sig_color::Symbol = :black` : Color for significance markers
- `significance_position::Symbol = :bottom` : Vertical position of significance bars (:bottom, :top, or numeric y value)
- `xgrid::Bool = true` : Show x-axis grid (true/false)
- `xlabel::String = "Time (s)"` : Label for x-axis
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple. If nothing, automatically determined
- `ygrid::Bool = true` : Show y-axis grid (true/false)
- `ylabel::String = "Classification Accuracy"` : Label for y-axis
- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple. If nothing, automatically determined

Legend Options

- `legend::Bool = false` : Show legend (true/false)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Examples

```
# Basic plot
plot_decoding(decoded)

# Custom styling
plot_decoding(decoded, color=:red, linewidth=3, show_error=false)

# With custom title
plot_decoding(decoded, plot_title="Face vs. Object Decoding")
```

```
plot_decoding(filepath::String; input_dir=pwd(),
participant_selection=participants(), kwargs...)
```

Load decoded data and plot. Accepts either a `.jld2` filepath or a pattern to discover and plot all matching files (one plot per file).

Examples

```
plot_decoding("decoded.jld2")
plot_decoding("decoded")
```

source

`EegFun.plot_ecg_component_features` – Method.

```
plot_ecg_component_features(identified_comps::AbstractVector{<:Integer},
metrics_df::DataFrame; kwargs...) -> NamedTuple
```

Plot heart-rate vs peak-ratio and inter-beat-interval regularity metrics for ECG ICA component identification.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `correlation_color::Symbol = :steelblue` : Color for correlation plots
- `display_plot::Bool = true` : Whether to display the plot
- `identified_color::Symbol = :green` : Color for identified components
- `label_fontsize::Int64 = 14` : Font size for axis labels
- `line_alpha::Float64 = 0.8` : Transparency for plot lines
- `linewidth::Int64 = 2` : Line width for plot lines
- `marker_alpha::Float64 = 0.7` : Transparency of markers
- `marker_size::Int64 = 8` : Size of markers
- `max_ibi_std_s::Float64 = 0.2` : Maximum inter-beat interval standard deviation for ECG detection
- `min_harmonic_power::Float64 = 1.5` : Minimum harmonic power threshold for line noise detection
- `min_peak_ratio::Float64 = 0.7` : Minimum peak ratio for ECG detection
- `plot_title::Nothing = nothing` : Custom plot title override
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `rejected_color::Symbol = :red` : Color for rejected components
- `show_title::Bool = true` : Whether to show the plot title
- `threshold_color::Symbol = :red` : Color for threshold lines
- `threshold_style::Symbol = :dash` : Line style for threshold lines
- `tick_fontsize::Int64 = 12` : Font size for tick labels
- `z_threshold::Float64 = 3.0` : Z-score threshold for identifying components
- `zscore_threshold_color::Symbol = :red` : Color for z-score threshold lines
- `zscore_threshold_style::Symbol = :dash` : Line style for z-score threshold lines

Legend Options

- `legend::Bool = true` : Whether to display the legend

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_eog_component_features` – Method.

```
plot_eog_component_features(identified_comps::Dict, metrics_df::DataFrame;
kwargs...) -> NamedTuple
```

Plot vEOG and hEOG correlation z-scores across ICA components and highlight identified ocular components.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `correlation_color::Symbol = :steelblue` : Color for correlation plots
- `display_plot::Bool = true` : Whether to display the plot
- `identified_color::Symbol = :green` : Color for identified components
- `label_fontsize::Int64 = 14` : Font size for axis labels
- `line_alpha::Float64 = 0.8` : Transparency for plot lines
- `linewidth::Int64 = 2` : Line width for plot lines
- `marker_alpha::Float64 = 0.7` : Transparency of markers
- `marker_size::Int64 = 8` : Size of markers
- `max_ibi_std_s::Float64 = 0.2` : Maximum inter-beat interval standard deviation for ECG detection
- `min_harmonic_power::Float64 = 1.5` : Minimum harmonic power threshold for line noise detection
- `min_peak_ratio::Float64 = 0.7` : Minimum peak ratio for ECG detection
- `plot_title::Nothing = nothing` : Custom plot title override
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `rejected_color::Symbol = :red` : Color for rejected components
- `show_title::Bool = true` : Whether to show the plot title
- `threshold_color::Symbol = :red` : Color for threshold lines
- `threshold_style::Symbol = :dash` : Line style for threshold lines
- `tick_fontsize::Int64 = 12` : Font size for tick labels
- `z_threshold::Float64 = 3.0` : Z-score threshold for identifying components
- `zscore_threshold_color::Symbol = :red` : Color for z-score threshold lines

- `zscore_threshold_style::Symbol = :dash` : Line style for z-score threshold lines

Legend Options

- `legend::Bool = true` : Whether to display the legend

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_epochs` – Method.

```
plot_epochs(filename::String;
            input_dir::String = pwd(),
            participant_selection::Function = participants(),
            channel_selection::Function = channels(),
            sample_selection::Function = samples(),
            interval_selection::Interval = times(),
            epoch_selection::Function = epochs(),
            include_extra::Bool = false,
            layout = :single,
            kwargs...)
```

Load epoch data and create plots. Accepts either a direct `.jld2` filepath or a pattern string to discover and plot all matching files (one plot per file).

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Whether to add origin lines at x=0 and y=0
- `average_channels::Bool = false` : Whether to average across channels
- `avg_linewidth_multiplier::Float64 = 2.0` : Multiplier for average line width.
- `axis_type::Symbol = :standard` : Type of axis to draw (:standard or :origin)
- `baseline_interval::Nothing = nothing` : Baseline interval as (start, stop) tuple or range in seconds
- `color::Nothing = nothing` : Color for epoch traces (can be a single color or a vector of colors, one per condition)
- `colormap::Nothing = nothing` : Colormap for multi-condition plots
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `interactive::Bool = true` : Whether to enable interactive features
- `layout::Symbol = :single` : Layout type: :single, :grid, or :topo

- `linewidth::Nothing = nothing` : Line width for epoch traces
- `plot_avg_trials::Bool = true` : Whether to draw ERP average overlay
- `plot_title::Nothing = nothing` : Plot title. If nothing, automatically determined
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `scale_x_value::Nothing = nothing` : X-axis scale value/step size (e.g. 0.1 for 100 ms)
- `scale_y_value::Nothing = nothing` : Y-axis scale value/step size (e.g. 50.0 for 50 μV)
- `selection_alpha::Float64 = 0.3` : Alpha (transparency) for interactive selection rectangles
- `selection_color::Symbol = :blue` : Color for interactive selection rectangles
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `trial_alpha::Float64 = 0.25` : Alpha (transparency) for individual trial traces
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlabel::String = "Time (s)"` : Label for x-axis
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple. If nothing, automatically determined
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylabel::String = "μV"` : Label for y-axis
- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple. If nothing, automatically determined
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid
- `yreversed::Bool = false` : Whether to reverse the y-axis
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

Layout Options

- `layout_kwargs::@NamedTuple{} = NamedTuple()` : Additional parameters to configure grid or topographical layouts (e.g., grid*dims, topo*plot*width, grid*rowgap)

Legend Options

- `legend::Bool = true` : Whether to show the legend
- `legend_channel::Vector{Any} = Any[]` : If plotting multiple plots, within channel to put the legend on.
- `legend_label::String = ""` : Title for the legend
- `legend_nbanks::Nothing = nothing` : Number of columns for the legend. If nothing, automatically determined.
- `legend_position::Symbol = :lt` : Position of the legend for axislegend() (symbol like :lt, :rt, :lb, :rb, or tuple like (:left, :top), or (0.5, 0.5))

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Examples

```
plot_epochs("epochs_raw.jld2")
plot_epochs("epochs_raw")
plot_epochs("epochs_raw", participant_selection = participants(1))
```

source

`EegFun.plot_erp!` – Method.

```
plot_erp!(fig::Figure, ax::Axis, dat::ErpData; kwargs...)
plot_erp!(fig::Figure, ax::Axis, datasets::AbstractVector{<:ErpData};
kwargs...)
```

Plot ERP data on an existing axis, mutating the figure and axis. Returns `ax` .

source

`EegFun.plot_erp` – Method.

```
plot_erp(filepath::String;
         input_dir::String = pwd(),
         participant_selection::Function = participants(),
         layout::Union{Symbol, PlotLayout} = :single,
         condition_selection::Function = conditions(),
         channel_selection::Function = channels(),
         channel_plot_order::Union{Nothing, Vector{Symbol}} = nothing,
         sample_selection::Function = samples(),
         interval_selection::Interval = times(),
         baseline_interval::Interval = nothing,
         kwargs...)
```

Load ERP data and create plots. Accepts either a direct `.jld2` filepath or a filename pattern to discover and plot all matching files.

Arguments

- `filepath::String` : Either a `.jld2` file path, or a pattern string (e.g. `"erps_final"`) to match against files in `input_dir`
- `input_dir::String` : Directory to search for pattern-matched files (default: `pwd()`)
- `participant_selection::Function` : Participant filter for pattern mode (default: `participants()`)
- `layout` : Layout specification (see main plot_erp documentation)
- `channel_selection::Function` : Function that returns boolean vector for channel filtering
- `sample_selection::Function` : Function that returns boolean vector for sample filtering

- `kwargs` : Additional keyword arguments

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Add origin lines at x=0 and y=0 (true/false)
- `average_channels::Bool = false` : Average across channels (true/false)
- `axis_type::Symbol = :standard` : Type of axis to draw (:standard or :origin)
- `color::Nothing = nothing` : Color for ERPs (single color or a vector of colors, one per dataset)
- `colormap::Nothing = nothing` : Colormap for multi-channel plots
- `display_plot::Bool = true` : Display the plot (true/false)
- `error_bars::Symbol = :none` : Error bars type to plot: :none, :sem, :within_sem, :ci95
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `highlight_regions::Nothing = nothing` : Highlight regions as a NamedTuple or Vector of NamedTuples. Each region: (x1, x2, y1=-Inf, y2=Inf, color=:gray, alpha=0.3)
- `interactive::Bool = true` : Enable interactive features (true/false)
- `linestyle::Nothing = nothing` : Line style for ERPs (single style or a vector of styles, one per dataset)
- `linewidth::Nothing = nothing` : Line width for ERPs
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `scale_x_value::Nothing = nothing` : X-axis scale value/step size (e.g. 0.1 for 100 ms)
- `scale_y_value::Nothing = nothing` : Y-axis scale value/step size (e.g. 5.0 for 5 μV)
- `selection_alpha::Float64 = 0.3` : Alpha (transparency) for interactive selection rectangles
- `selection_color::Symbol = :blue` : Color for interactive selection rectangles
- `show_title::Bool = true` : Show plot title (true/false)
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for x-axis display (:s or :ms). Only affects axis labels and tick formatting — all intervals remain in seconds.
- `window_title::String = ""` : Title for the OS plot window. If empty, it's generated automatically.
- `xgrid::Bool = false` : Show x-axis grid (true/false)

- `xlabel::String = "Time (s)"` : Label for x-axis
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple. If nothing, automatically determined
- `xminorgrid::Bool = false` : Show x-axis minor grid (true/false)
- `xticks::Nothing = nothing` : X-axis tick positions (e.g., −0.1:0.1:0.8 or [0, 0.2, 0.4]). If nothing, automatically determined
- `ygrid::Bool = false` : Show y-axis grid (true/false)
- `ylabel::String = "μV"` : Label for y-axis
- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple. If nothing, automatically determined
- `yminorgrid::Bool = false` : Show y-axis minor grid (true/false)
- `yreversed::Bool = false` : Whether to reverse the y-axis
- `yticks::Nothing = nothing` : Y-axis tick positions (e.g., −4:2:4 or [−2, 0, 2]). If nothing, automatically determined
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

Layout Options

- `layout_kwargs::@NamedTuple{} = NamedTuple()` : Additional parameters to configure grid or topographical layouts (e.g., grid*dims, topo*plot*width, grid*rowgap)

Legend Options

- `legend::Bool = true` : Show the legend (true/false)
- `legend_channel::Vector{Any} = Any[]` : Which channel to put the legend on.
- `legend_label::String = ""` : Title for the legend
- `legend_nbanks::Nothing = nothing` : Number of legend columns.
- `legend_position::Symbol = :lt` : Legend position (:lt, :rt, :lb, :rb, or tuple like (:left, :top), or (0.5, 0.5))

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Examples

```
plot_erp("grand_average_erps_final.jld2")
plot_erp("erps_final")
plot_erp("erps_final", participant_selection = participants(1))
plot_erp("erps_final", channel_selection = channels([:PO7, :PO8]), layout
= :grid)
```

source

`EegFun.plot_erp_errorbar!` – Method.

```
plot_erp_errorbar!(fig::Figure, ax::Axis, dat::ErpData; kwargs...)
```

```
plot_erp_errorbar!(fig::Figure, ax::Axis, datasets::AbstractVector{<:ErpData};
kwargs...)
```

Plot grand-averaged ERP data with error ribbons on an existing axis, mutating the figure and axis. Returns `ax` .

source

`EegFun.plot_erp_errorbar` – Method.

```
plot_erp_errorbar(filepath::String; kwargs...)
plot_erp_errorbar(dataset::ErpData; kwargs...)
plot_erp_errorbar(datasets::AbstractVector{<:ErpData}; kwargs...)
```

Plot grand-averaged ERP waveforms with across-participant error ribbons (`:sem` , `:within_sem` , or `:ci95`).

source

`EegFun.plot_erp_filter_gui` – Method.

```
plot_erp_filter_gui(; channel = :Cz, kwargs...)
plot_erp_filter_gui(filepath::String; kwargs...)
plot_erp_filter_gui(erp::Union{ErpData,ContinuousData}; channel = nothing,
kwargs...)
plot_erp_filter_gui(erps::AbstractVector{<:Union{ErpData,ContinuousData}};
channel = nothing, display_plot = true)
```

Launch an interactive GUI for exploring IIR/FIR low-pass and high-pass filter effects on ERP or continuous waveforms.

Arguments

- `filepath::String` : Path to a `.jld2` file containing ERP data
- `erp` / `erps` : Single dataset or vector of `ErpData` / `ContinuousData`

Keyword Arguments

- `channel::Union{Symbol,Nothing}` : Initial channel to display (default: `:Cz` for demo, first channel otherwise)
- `display_plot::Bool` : Display the interactive figure (default: `true`)

Examples

```
plot_erp_filter_gui()
plot_erp_filter_gui(channel = :Cz)
plot_erp_filter_gui("participant_1_erps.jld2", channel = :Pz)
```

source

`EegFun.plot_erp_image` – Method.

```
plot_erp_image(filepath::String; input_dir = pwd(), participant_selection =
participants(), kwargs...)
plot_erp_image(dat::EpochData;
               layout::Union{Symbol, PlotLayout} = :single,
```

```
                channel_selection = channels(),
                channel_plot_order = nothing,
                sample_selection = samples(),
                interval_selection::Interval = times(),
                kwargs...)
plot_erp_image(data::AbstractVector{<:EpochData}; kwargs...)
```

Plot single-trial ERP image heatmaps across epochs for specified channels and samples.

Arguments

- `dat::EpochData` : Epoch data structure (or `.jld2` filepath / filename pattern)
- `layout` : `:single` (default), `:grid` , `:topo` , or `PlotLayout`
- `channel_selection` : Channel selection predicate or symbol/vector (default: `channels()`)
- `sample_selection` : Sample selection predicate (default: `samples()`)
- `interval_selection` : Time interval filter (default: `times()`)

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Add origin lines at x=0 and y=0 (true/false)
- `boxcar_average::Int64 = 1` : Boxcar average window size for smoothing the ERP image (1 = no smoothing)
- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = "μV"` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the image
- `colorrange::Nothing = nothing` : Color range for the image. If nothing, automatically determined
- `display_plot::Bool = true` : Display the plot (true/false)
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `interactive::Bool = true` : Enable interactive features (true/false)
- `interpolate::Bool = false` : Whether to interpolate the heatmap pixels for smoother appearance
- `plot_erp::Bool = true` : Whether to plot ERP average overlay

- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `selection_alpha::Float64 = 0.3` : Alpha (transparency) for interactive selection rectangles
- `selection_color::Symbol = :blue` : Color for interactive selection rectangles
- `show_title::Bool = true` : Show plot title (true/false)
- `xgrid::Bool = false` : Show x-axis grid (true/false)
- `xlabel::String = "Time (s)"` : Label for x-axis
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple. If nothing, automatically determined
- `xminorgrid::Bool = false` : Show x-axis minor grid (true/false)
- `ygrid::Bool = false` : Show y-axis grid (true/false)
- `ylabel::String = "Epoch"` : Label for y-axis
- `ylim::Nothing = nothing` : Y-axis limits for ERP waveform as (min, max) tuple. If nothing, automatically determined
- `yminorgrid::Bool = false` : Show y-axis minor grid (true/false)
- `yreversed::Bool = false` : Whether to reverse the y-axis
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

Layout Options

- `layout_kwargs::@NamedTuple{} = NamedTuple()` : Additional parameters to configure grid or topographical layouts (e.g., grid*dims, topo*plot*width, grid*rowgap)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Returns

- `NamedTuple (fig = fig, axes = axes)` (or vector of `NamedTuple` s for multi-condition/pattern inputs)

source

`EegFun.plot_erp_measurement_gui` – Method.

```
plot_erp_measurement_gui(; filter_lp = 30.0, channel = :Cz, analysis_type =
"mean_amplitude",
                           analysis_interval = (0.3, 0.5), baseline_interval =
                           (-0.2, 0.0), kwargs...)
plot_erp_measurement_gui(filepath::String; kwargs...)
```

```
plot_erp_measurement_gui(erp::ErpData; kwargs...)
plot_erp_measurement_gui(erps::AbstractVector{<:ErpData}; kwargs...)
```

Launch an interactive GUI for exploring ERP measurements with live visual feedback.

Arguments

- `filepath::String` : Path to a `.jld2` file containing ERP data
- `erp::ErpData` or `erps::AbstractVector{<:ErpData}` : Single ERP or multiple conditions to overlay

Keyword Arguments

- `filter_lp::Union{Real,Nothing}` : Low-pass filter cutoff in Hz applied when loading default demo data (default: `30.0` ; `nothing` disables)
- `channel::Union{Symbol,Vector{Symbol},Nothing}` : Initial channel(s) to display (default: `:Cz` for demo, first channel otherwise)
- `analysis_type::String` : Initial measurement type (default: `"mean_amplitude"`)
- `analysis_interval::Union{Tuple{Real,Real},Nothing}` : Initial measurement interval
- `baseline_interval::Union{Tuple{Real,Real},Nothing}` : Initial baseline interval
- `display_plot::Bool` : Display the figure window (default: `true`)

Examples

```
plot_erp_measurement_gui()
plot_erp_measurement_gui("participant_1_erps_final.jld2")
plot_erp_measurement_gui(erp, analysis_type = "max_peak_latency",
analysis_interval = (0.3, 0.5))
```

source

`EegFun.plot_erp_measurements` – Method.

```
plot_erp_measurements(erp_data, analysis_type;
                      analysis_interval = nothing,
                      baseline_interval = nothing,
                      layout = :single,
                      channel_selection = channels(),
                      condition_selection = conditions(),
                      average_channels = false,
                      kwargs...)
```

Plot ERP data with measurement overlays computed inline.

Computes a single measurement type across all selected channels and conditions, then overlays the results on the ERP waveforms.

Arguments

- `erp_data::Union{String, ErpData, AbstractVector{<:ErpData}}` : ERP data (filepath, pattern, or data object)
- `analysis_type::String` : Measurement type (e.g. `"mean_amplitude"` , `"max_peak_latency"`)

Keyword Arguments

- `analysis_interval::Interval` : Time interval for measurement (default: full range)
- `baseline_interval::Interval` : Baseline interval for correction (default: `nothing`)
- `layout::Union{Symbol, PlotLayout}` : Plot layout — `:single` , `:grid` , or `:topo` (default: `:single`)
- `channel_selection` : Channel selection predicate (default: `channels()`)
- `condition_selection` : Condition selection predicate (default: `conditions()`)
- `average_channels::Bool` : Evaluate measurement across spatially pooled ROI waveform (default: `false`)
- `kwargs...` : Additional arguments passed to `plot_erp` and `erp_measurements`

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `area_boundary::Symbol = :rectified` : Area boundary mode for fractional area latency: :rectified (default), :positive, :negative, or :signed
- `fractional_area_fraction::Float64 = 0.5` : Fraction for fractional area latency (0.0-1.0). Finds latency where this fraction of area is to the left.
- `fractional_peak_direction::Symbol = :onset` : Direction for fractional peak latency: :onset (before peak) or :offset (after peak)
- `fractional_peak_fraction::Float64 = 0.5` : Fraction for fractional peak latency (0.0-1.0). Finds latency where amplitude is this fraction of peak.
- `local_interval::Int64 = 3` : Number of samples on each side of peak (total interval = 2*local_interval + 1). Peak must be larger than neighbors and local averages within this window.
- `peak_polarity::Symbol = :abs` : Peak polarity for fractional peak latency: :abs (default, largest absolute peak), :positive, or :negative

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Examples

```
plot_erp_measurements(dat, "mean_amplitude", analysis_interval = (0.3, 0.5),
baseline_interval = (-0.2, 0.0))
plot_erp_measurements("erps_final.jld2", "max_peak_amplitude",
analysis_interval = (0.1, 0.3))
```

source

`EegFun.plot_erp_stats` – Method.

```
plot_erp_stats(result::StatsResult;
               layout::Union{Symbol,PlotLayout} = :single,
```

```
                condition_selection = conditions(),
                channel_selection = channels(),
                channel_plot_order = nothing,
                plot_erp::Bool = true,
                plot_difference::Bool = false,
                plot_tvalues::Bool = false,
                plot_significance::Bool = false,
                plot_critical_t::Bool = false,
                plot_se::Bool = false,
                difference_offset::Real = 0.0,
                significance_position::Union{Symbol,Real} = :auto,
                significance_color = (:gray, 0.6),
                kwargs...)
```

Plot ERP waveforms and statistical results for `AnalyticResult` or `PermutationResult` across `:single` , `:grid` , `:topo` , or custom `PlotLayout` layouts.

Arguments

- `result::StatsResult` : Result from `analytic_test` or `permutation_test` .
- `layout` : `:single` (default), `:grid` , `:topo` , or a `PlotLayout` .
- `condition_selection` : Predicate to select conditions when `plot_erp = true` .
- `channel_selection` : Predicate, symbol(s), or name(s) selecting channels (default: `channels()`).
- `channel_plot_order` : Optional vector overriding channel plot order.
- `plot_erp` : Plot condition average waveforms (default: `true`).
- `plot_difference` : Plot difference wave `A - B` (default: `false`).
- `plot_tvalues` : Plot t-statistics (default: `false` ; auto-enabled when `plot_critical_t = true`).
- `plot_significance` : Highlight significant time intervals (default: `false`).
- `plot_critical_t` : Plot critical t thresholds (default: `false`).
- `plot_se` : Plot ±1 SEM bands around condition and difference waveforms (default: `false`).
- `difference_offset` : Vertical offset for the difference wave (default: `0.0`).
- `significance_position` : `:auto` (default), `:zero` , `:bottom` , or custom `Real` y-coordinate.
- `significance_color` : Color for significance bars (default: `(:gray, 0.6)`).

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_xy_origin::Bool = true` : Add origin lines at x=0 and y=0 (true/false)
- `average_channels::Bool = false` : Average across channels (true/false)
- `axis_type::Symbol = :standard` : Type of axis to draw (:standard or :origin)

- `color::Nothing = nothing` : Color for ERPs (single color or a vector of colors, one per dataset)
- `colormap::Nothing = nothing` : Colormap for multi-channel plots
- `display_plot::Bool = true` : Display the plot (true/false)
- `error_bars::Symbol = :none` : Error bars type to plot: :none, :sem, :within_sem, :ci95
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `highlight_regions::Nothing = nothing` : Highlight regions as a NamedTuple or Vector of NamedTuples. Each region: (x1, x2, y1=-Inf, y2=Inf, color=:gray, alpha=0.3)
- `interactive::Bool = true` : Enable interactive features (true/false)
- `linestyle::Nothing = nothing` : Line style for ERPs (single style or a vector of styles, one per dataset)
- `linewidth::Nothing = nothing` : Line width for ERPs
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `scale_x_value::Nothing = nothing` : X-axis scale value/step size (e.g. 0.1 for 100 ms)
- `scale_y_value::Nothing = nothing` : Y-axis scale value/step size (e.g. 5.0 for 5 μV)
- `selection_alpha::Float64 = 0.3` : Alpha (transparency) for interactive selection rectangles
- `selection_color::Symbol = :blue` : Color for interactive selection rectangles
- `show_title::Bool = true` : Show plot title (true/false)
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for x-axis display (:s or :ms). Only affects axis labels and tick formatting — all intervals remain in seconds.
- `window_title::String = ""` : Title for the OS plot window. If empty, it's generated automatically.
- `xgrid::Bool = false` : Show x-axis grid (true/false)
- `xlabel::String = "Time (s)"` : Label for x-axis
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple. If nothing, automatically determined
- `xminorgrid::Bool = false` : Show x-axis minor grid (true/false)
- `xticks::Nothing = nothing` : X-axis tick positions (e.g., −0.1:0.1:0.8 or [0, 0.2, 0.4]). If nothing, automatically determined
- `ygrid::Bool = false` : Show y-axis grid (true/false)
- `ylabel::String = "μV"` : Label for y-axis

- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple. If nothing, automatically determined
- `yminorgrid::Bool = false` : Show y-axis minor grid (true/false)
- `yreversed::Bool = false` : Whether to reverse the y-axis
- `yticks::Nothing = nothing` : Y-axis tick positions (e.g., −4:2:4 or [−2, 0, 2]). If nothing, automatically determined
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

Layout Options

- `layout_kwargs::@NamedTuple{} = NamedTuple()` : Additional parameters to configure grid or topographical layouts (e.g., grid*dims, topo*plot*width, grid*rowgap)

Legend Options

- `legend::Bool = true` : Show the legend (true/false)
- `legend_channel::Vector{Any} = Any[]` : Which channel to put the legend on.
- `legend_label::String = ""` : Title for the legend
- `legend_nbanks::Nothing = nothing` : Number of legend columns.
- `legend_position::Symbol = :lt` : Legend position (:lt, :rt, :lb, :rb, or tuple like (:left, :top), or (0.5, 0.5))

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Returns

- Named tuple `(fig, axes)`

source

`EegFun.plot_filter_response` – Method.

```
plot_filter_response(filter_info::FilterInfo; filter_func::Function =
filtfilt, kwargs...) -> NamedTuple
```

Plot the frequency response (dB) and impulse response of a digital `FilterInfo` .

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `color::Nothing = nothing` : Color for response curves
- `display_plot::Bool = true` : Whether to display the plot
- `label_fontsize::Int64 = 22` : Font size for axis labels
- `linewidth::Int64 = 4` : Line width for response curves
- `n_points::Int64 = 2000` : Number of frequency points for response calculation

- `plot_title::String = "Filter Frequency Response"` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 24` : Font size for title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `reference_color::Symbol = :gray` : Color for reference lines
- `reference_lines::Vector{Int64} = [-3, -6]` : Reference lines in dB to display
- `reference_linestyle::Symbol = :dash` : Line style for reference lines
- `show_title::Bool = true` : Whether to show the plot title
- `tick_fontsize::Int64 = 20` : Font size for tick labels
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlabel::String = "Frequency (Hz)"` : X-axis label
- `xlim::Nothing = nothing` : X-axis limits in Hz as (min, max) tuple. If nothing, automatically determined
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `xscale::Makie.Symlog10{Makie.ReversibleScale{Makie.var"#forward#1244"{Int64, Float64, Float64}, Makie.var"#inverse#1245"{Int64, Float64, Float64}, IntervalSets.OpenInterval{Float32}}} = Symlog10(-10.0, 10.0; linscale = 1.0)` : X-axis scale type
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylabel::String = "Magnitude (dB)"` : Y-axis label
- `ylim::Tuple{Int64, Int64} = (-100, 5)` : Y-axis limits in dB as (min, max) tuple
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_gfp` – Method.

```
plot_gfp(dat::ErpData; channel_selection = channels(), normalize = true,
kwargs...)
plot_gfp(datasets::AbstractVector{<:ErpData}; condition_selection =
conditions(), channel_selection = channels(), normalize = true, kwargs...)
plot_gfp(gfp_data::Union{DataFrame,AbstractVector{<:DataFrame}}; normalize =
nothing, kwargs...)
plot_gfp(filepath::String; input_dir = pwd(), participant_selection =
participants(), kwargs...)
```

Plot Global Field Power (GFP) and optionally ERP channel traces (`show_erp_traces = true`) and Global Dissimilarity (`show_dissimilarity = true`).

Arguments

- `dat` / `datasets` / `gfp_data` / `filepath` : `ErpData` , vector of `ErpData` , precomputed GFP `DataFrame` (s), or `.jld2` file/pattern.
- `channel_selection` : Channel predicate for selecting channels (default: `channels()`).
- `condition_selection` : Condition filter for `Vector{ErpData}` (default: `conditions()`).
- `normalize` : Normalize GFP to 0–100% (`true` by default for `ErpData` ; inferred from metadata when `nothing` for `DataFrame` s).

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `add_x_origin::Bool = true` : Whether to add vertical line at x=0
- `color::Nothing = nothing` : Color for traces
- `display_plot::Bool = true` : Whether to display the plot
- `linestyle::Nothing = nothing` : Line style for traces
- `linewidth::Int64 = 2` : Line width for GFP/dissimilarity traces
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `show_dissimilarity::Bool = false` : Whether to include Global Dissimilarity panel
- `show_erp_traces::Bool = false` : Whether to show individual ERP channel traces in top panel
- `show_title::Bool = true` : Whether to show the plot title
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xlabel::String = "Time (s)"` : Label for x-axis
- `xlim::Nothing = nothing` : X-axis limits as (min, max) tuple. If nothing, automatically determined
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `ygrid::Bool = false` : Whether to show y-axis grid
- `ylabel::Nothing = nothing` : Label for y-axis (auto-determined based on normalize flag)
- `ylim::Nothing = nothing` : Y-axis limits as (min, max) tuple. If nothing, automatically determined
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

Legend Options

- `legend::Bool = true` : Whether to show the legend when plotting multiple conditions

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_gui` – Method.

```
plot_gui(; display_plot::Bool = true)
```

Launch the interactive GUI for quick EEG data exploration and visualization.

Provides file/directory browsing, plot-type selection (Data Browser, Epochs, ERP, Topography, ERP Image, Time-Frequency, ICA, ERP Analysis, and Diagnostic plots), participant/condition/epoch/channel filtering, and axis/baseline controls.

Examples

```
EegFun.plot_gui()
```

source

`EegFun.plot_ica_component_activation` – Method.

```
plot_ica_component_activation(dat::ContinuousData, ica::InfoIca; artifacts =
nothing, kwargs...) -> NamedTuple
```

Interactive viewer for ICA component time-course activations and scalp topographies.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = ""` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the topography
- `colorrange::Nothing = nothing` : Color scale limits as (min, max) tuple (nothing for auto)
- `component_selection::EegFun.var"#components##0#components##1" = EegFun.var"#components##0#components##1"()` : Function that returns boolean vector for component filtering
- `dims::Nothing = nothing` : Grid dimensions (rows, cols). If nothing, calculates best square-ish grid

- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `gridscale::Int64 = 75` : Grid resolution for interpolation
- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Float64 = 1.0` : Radius of the head shape in mm.
- `highlight_channels::Nothing = nothing` : Highlight channel groups as a NamedTuple or Vector of NamedTuples. Each group: (channels=[:Cz, :Pz], color=:white, size=8, marker=:circle)
- `interactive::Bool = true` : Whether to enable interactive features
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `method::Symbol = :thin_plate` : Interpolation method: :multiquadratic, :inverse*multiquadratic, :gaussian, :inverse*quadratic, :thin*plate, :polyharmonic, :shepard,* See https://eljungsk.github.io/ScatteredInterpolation.jl/dev/methods/ for details on methods.
- `num_levels::Int64 = 20` : Number of contour levels (for ICA plots).
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for display labels (:s or :ms). Only affects title strings — all intervals and selections remain in seconds.
- `use_global_scale::Bool = false` : Do multiple ICA topoplots share the same color scale based on min/max across all components?
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_ica_component_spectrum` – Method.

```
plot_ica_component_spectrum(dat::ContinuousData, ica_result::InfoIca;
component_selection::Function = components(), kwargs...)
```

Plot power spectrum of ICA component(s) with interactive controls for axis scaling.

Arguments

- `dat::ContinuousData` : The continuous data.
- `ica_result::InfoIca` : The ICA result object.
- `component_selection::Function` : Function that returns boolean vector for component filtering (default: all components).

Keyword Arguments

- `sample_selection::Function` : Function that returns boolean vector for sample filtering (default: samples() - all samples)
- `window_size::Int` : Size of the FFT window for spectral estimation (default: 1024).
- `overlap::Real` : Overlap between windows for Welch's method (default: 0.5).
- `max_freq::Real` : Maximum frequency to display in Hz (default: 100.0).
- `xscale::Symbol` : Initial scale for x-axis, one of :linear or :log10 (default: :linear).
- `yscale::Symbol` : Initial scale for y-axis, one of :linear or :log10 (default: :linear).
- `unit::Symbol` : Power unit, one of :linear (μV²/Hz) or :dB (decibels) (default: :linear).
- `window_function::Function` : Window function to use for spectral estimation (default: DSP.hanning).
- `show_freq_bands::Bool` : Whether to show frequency band indicators (default: true).
- `legend::Bool` : Whether to show the legend with component variance percentages (default: true).
- `display_plot::Bool` : Whether to display the plot (default: true).

Returns

- Named tuple `(fig, axes)` containing the Makie `Figure` and `[ax]` .

Examples

```
plot_ica_component_spectrum(dat, ica_result)
plot_ica_component_spectrum(dat, ica_result, component_selection =
components([1, 3, 5]))
plot_ica_component_spectrum(dat, ica_result, component_selection =
components(1), unit = :dB)
```

source

EegFun.plot_joint_probability! – Method.

```
plot_joint_probability!(fig::Figure, ax::Axis, dat::DataFrame; kwargs...) ->
Axis
plot_joint_probability(dat::DataFrame; kwargs...) -> NamedTuple
```

Plot per-channel joint probability (:jp) bar charts from dat .

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- bar_alpha::Float64 = 0.7 : Transparency of bars
- bar_color::Symbol = :steelblue : Color of the bars
- bar_width::Float64 = 0.8 : Width of bars
- display_plot::Bool = true : Whether to display the plot
- label_fontsize::Int64 = 14 : Font size for axis labels
- plot_title::Nothing = nothing : Plot title
- plot_title_align::Tuple{Symbol, Symbol} = (:center, :top) : Alignment of the inner plot title
- plot_title_fontsize::Int64 = 16 : Font size for title
- plot_title_position::Nothing = nothing : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- show_title::Bool = true : Whether to show the plot title
- sort_values::Bool = false : If true, sort the bars by joint probability values in descending order
- tick_fontsize::Int64 = 12 : Font size for tick labels
- xgrid::Bool = true : Whether to show x-axis grid
- xlabel::String = "Electrode" : Label for x-axis
- xminorgrid::Bool = false : Whether to show x-axis minor grid
- xtick_rotation::Float64 = 0.7853981633974483 : Rotation angle for x-axis tick labels
- ygrid::Bool = true : Whether to show y-axis grid
- ylabel::String = "Joint Probability" : Label for y-axis
- yminorgrid::Bool = false : Whether to show y-axis minor grid

> **Further Legend Options** EegFun.jl passes any argument prefixed with legend_ directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., legend_bgcolor , legend_patchsize). See the Makie.jl documentation for a full list of discoverable arguments.

source

EegFun.plot_joint_probability – Method.

```
plot_joint_probability(dat::DataFrame; kwargs...) -> NamedTuple
```

Create a figure and plot per-channel joint probability (`:jp`) bar charts. See `plot_joint_probability!` for keyword options.

source

`EegFun.plot_layout_2d!` – Method.

```
plot_layout_2d!(fig::Figure, ax::Axis, layout::Layout;
neighbors::Bool=false, correlation_matrix::Union{DataFrame,Nothing}=nothing,
kwargs...) -> Axis
plot_layout_2d(layout::Layout; neighbors::Bool=false,
correlation_matrix::Union{DataFrame,Nothing}=nothing, display_plot::Bool=true,
kwargs...) -> NamedTuple
```

Plot a 2D EEG electrode layout with customizable head outline, electrode markers, labels, and optional hover neighbor or correlation overlays.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Int64 = 1` : Radius of the head shape (normalized units).

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `label_align::Tuple{Symbol, Symbol} = (:left, :bottom)` : Text alignment for electrode labels (e.g., (:center, :center)).
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `label_zoffset::Int64 = 0` : Z-axis offset for electrode labels (3D only).

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_layout_2d` – Method.

```
plot_layout_2d(layout::Layout; neighbors::Bool=false,
correlation_matrix=nothing, display_plot::Bool=true, kwargs...) ->
NamedTuple
```

Create a figure and plot a 2D EEG electrode layout. See `plot_layout_2d!` for keyword options.

source

`EegFun.plot_layout_3d!` – Method.

```
plot_layout_3d!(fig::Figure, ax::Axis3, layout::Layout;
neighbors::Bool=false, kwargs...) -> Axis3
plot_layout_3d(layout::Layout; neighbors::Bool=false, display_plot::Bool=true,
kwargs...) -> NamedTuple
```

Plot a 3D EEG electrode layout with electrode markers, labels, and optional hover neighbor connections.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Int64 = 1` : Radius of the head shape (normalized units).

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `label_align::Tuple{Symbol, Symbol} = (:left, :bottom)` : Text alignment for electrode labels (e.g., (:center, :center)).
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `label_zoffset::Int64 = 0` : Z-axis offset for electrode labels (3D only).

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_layout_3d` – Method.

```
plot_layout_3d(layout::Layout; neighbors::Bool=false,
display_plot::Bool=true, kwargs...) -> NamedTuple
```

Create a figure and plot a 3D EEG electrode layout. See `plot_layout_3d!` for keyword options.

source

`EegFun.plot_line_noise_components` – Method.

```
plot_line_noise_components(line_noise_comps::AbstractVector{<:Integer},
metrics_df::DataFrame; kwargs...) -> NamedTuple
```

Plot line-frequency power ratio z-scores and harmonic power ratios across ICA components.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `correlation_color::Symbol = :steelblue` : Color for correlation plots
- `display_plot::Bool = true` : Whether to display the plot
- `identified_color::Symbol = :green` : Color for identified components
- `label_fontsize::Int64 = 14` : Font size for axis labels
- `line_alpha::Float64 = 0.8` : Transparency for plot lines
- `linewidth::Int64 = 2` : Line width for plot lines
- `marker_alpha::Float64 = 0.7` : Transparency of markers
- `marker_size::Int64 = 8` : Size of markers
- `max_ibi_std_s::Float64 = 0.2` : Maximum inter-beat interval standard deviation for ECG detection
- `min_harmonic_power::Float64 = 1.5` : Minimum harmonic power threshold for line noise detection
- `min_peak_ratio::Float64 = 0.7` : Minimum peak ratio for ECG detection
- `plot_title::Nothing = nothing` : Custom plot title override
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `rejected_color::Symbol = :red` : Color for rejected components
- `show_title::Bool = true` : Whether to show the plot title
- `threshold_color::Symbol = :red` : Color for threshold lines
- `threshold_style::Symbol = :dash` : Line style for threshold lines
- `tick_fontsize::Int64 = 12` : Font size for tick labels
- `z_threshold::Float64 = 3.0` : Z-score threshold for identifying components
- `zscore_threshold_color::Symbol = :red` : Color for z-score threshold lines
- `zscore_threshold_style::Symbol = :dash` : Line style for z-score threshold lines

Legend Options

- `legend::Bool = true` : Whether to display the legend

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can

> pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_model_correlations` – Method.

```
plot_model_correlations(rsa_data::RsaData; kwargs...)
```

Plot model correlations over time.

Arguments

- `rsa_data::RsaData` : RSA results with model comparisons
- `kwargs` : Additional keyword arguments

Examples

```
# Plot correlations with models
plot_model_correlations(rsa_result)

# Custom styling
plot_model_correlations(rsa_result, colors=[:red, :blue], plot_title="Model Comparison")
```

source

`EegFun.plot_rdm_heatmap` – Method.

```
plot_rdm_heatmap(rsa_data::RsaData; time_point::Union{Float64, Int, Nothing} = nothing, kwargs...)
```

Plot Representational Dissimilarity Matrix (RDM) as a heatmap.

Shows a matrix visualization of pairwise dissimilarities between conditions at a specific time point or averaged across time.

Arguments

- `rsa_data::RsaData` : RSA results
- `time_point::Union{Float64, Int, Nothing}` : Time point to plot (Float64 = time in seconds, Int = index, nothing = average across time)
- `kwargs` : Additional keyword arguments

Examples

```
# Plot RDM at a specific time point
plot_rdm_heatmap(rsa_result, time_point=0.3)

# Plot average RDM across all time points
plot_rdm_heatmap(rsa_result)

# Plot RDM at time index 50
plot_rdm_heatmap(rsa_result, time_point=50)
```

source

`EegFun.plot_rdm_timecourse` – Method.

```
plot_rdm_timecourse(
    rsa_data::RsaData;
    condition_pairs::Union{Vector{Tuple{Int,Int}}, Symbol, Nothing} = :all,
    kwargs...
)
```

Plot dissimilarity timecourse for condition pairs.

Shows how dissimilarity between conditions evolves over time as lines. This helps visualize when conditions become distinguishable.

Arguments

- `rsa_data::RsaData` : RSA results
- `condition_pairs` : Which condition pairs to plot
  - `:all` - All pairwise comparisons (default)
  - `:upper` - Upper triangle only (same as :all, no duplicates)
  - `Vector{Tuple{Int,Int}}` - Specific pairs, e.g., `[(1,2), (1,3)]`
- `kwargs` : Additional keyword arguments

Examples

```
# Plot all condition pairs
plot_rdm_timecourse(rsa_result)

# Plot only specific pairs
plot_rdm_timecourse(rsa_result, condition_pairs=[(1,2), (1,3)])

# Custom styling
plot_rdm_timecourse(rsa_result, plot_title="Dissimilarity Over Time")
```

source

`EegFun.plot_rsa` – Method.

```
plot_rsa(rsa_data::RsaData; kwargs...)
```

Main plotting function for RSA results.

If model correlations are available, plots them. Otherwise, plots RDM heatmap at average time.

Arguments

- `rsa_data::RsaData` : RSA results
- `kwargs` : Additional keyword arguments (passed to plot*model*correlations or plot*rdm*heatmap)

Examples

```
# Plot RSA results (automatically chooses best visualization)
plot_rsa(rsa_result)
```

```
# Force RDM heatmap plot
plot_rsa(rsa_result, plot_type=:rdm, time_point=0.3)
```

```
plot_rsa(filepath::String; input_dir=pwd(),
participant_selection=participants(), kwargs...)
```

Load RSA data and plot. Accepts either a `.jld2` filepath or a pattern to discover and plot all matching files (one plot per file).

Examples

```
plot_rsa("rsa_results.jld2")
plot_rsa("rsa_results")
```

source

`EegFun.plot_spatial_kurtosis_components` – Method.

```
plot_spatial_kurtosis_components(kurtosis_comps::AbstractVector{<:Integer},
metrics_df::DataFrame; kwargs...) -> NamedTuple
```

Plot spatial kurtosis z-scores across ICA components and highlight focal channel-noise components.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `correlation_color::Symbol = :steelblue` : Color for correlation plots
- `display_plot::Bool = true` : Whether to display the plot
- `identified_color::Symbol = :green` : Color for identified components
- `label_fontsize::Int64 = 14` : Font size for axis labels
- `line_alpha::Float64 = 0.8` : Transparency for plot lines
- `linewidth::Int64 = 2` : Line width for plot lines
- `marker_alpha::Float64 = 0.7` : Transparency of markers
- `marker_size::Int64 = 8` : Size of markers
- `max_ibi_std_s::Float64 = 0.2` : Maximum inter-beat interval standard deviation for ECG detection
- `min_harmonic_power::Float64 = 1.5` : Minimum harmonic power threshold for line noise detection
- `min_peak_ratio::Float64 = 0.7` : Minimum peak ratio for ECG detection
- `plot_title::Nothing = nothing` : Custom plot title override
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `rejected_color::Symbol = :red` : Color for rejected components
- `show_title::Bool = true` : Whether to show the plot title
- `threshold_color::Symbol = :red` : Color for threshold lines

- `threshold_style::Symbol = :dash` : Line style for threshold lines
- `tick_fontsize::Int64 = 12` : Font size for tick labels
- `z_threshold::Float64 = 3.0` : Z-score threshold for identifying components
- `zscore_threshold_color::Symbol = :red` : Color for z-score threshold lines
- `zscore_threshold_style::Symbol = :dash` : Line style for z-score threshold lines

Legend Options

- `legend::Bool = true` : Whether to display the legend

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_stat_heatmap!` – Method.

```
plot_stat_heatmap!(fig, ax, result::StatsResult; kwargs...)
```

Mutating variant of `plot_stat_heatmap` to draw a 2D statistical heatmap into an existing `Axis` .

source

`EegFun.plot_stat_heatmap` – Method.

```
plot_stat_heatmap(result::StatsResult; kwargs...)
```

Plot a 2D heatmap of t-statistics (`Channels × Time`) from an analytic or permutation test.

source

`EegFun.plot_tf` – Method.

```
plot_tf(filepath::String; kwargs...)
plot_tf(tf::TimeFreqData; channel_selection, baseline_interval, kwargs...)
plot_tf(tfs::Vector{TimeFreqData}; channel_selection, baseline_interval,
kwargs...)
```

Plot time-frequency data as a heatmap (time × frequency). Supports single condition, multi-condition grid, and topographic layouts.

Arguments

- `filepath::String` : Path to a `.jld2` file, or a `TimeFreqData` / `Vector{TimeFreqData}` object
- `channel_selection::Function` : Channel filter (default: all channels)
- `baseline_interval` : Baseline window as `(start, stop)` in seconds (default: `nothing`)

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `baseline_method::Symbol = :db` : Baseline correction
  method: :db, :absolute, :relative, :relchange, :percent, :zscore
- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::Nothing = nothing` : Custom colorbar label. If nothing, automatically determined
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the heatmap
- `colorrange::Nothing = nothing` : Color range as (min, max) tuple. If nothing, automatically determined from data
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `interpolate::Bool = false` : Whether to interpolate the heatmap for a smoother appearance
- `layout::Symbol = :single` : Layout type: :single, :grid, or :topo
- `plot_title::Nothing = nothing` : Plot title. If nothing, automatically determined from condition name and channel
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for plot titles
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for x-axis display (:s or :ms). Only affects axis labels and tick formatting — all intervals remain in seconds.
- `xlim::Nothing = nothing` : X-axis time limits as (min, max) tuple in seconds. If nothing, shows all time points
- `xscale::Symbol = :linear` : X-axis scale (:linear or :log10)
- `xticks::Nothing = nothing` : Custom x-axis ticks (e.g., −0.2:0.2:1.0)
- `yscale::Symbol = :linear` : Frequency (y) axis scale (:linear or :log10)
- `yticks::Nothing = nothing` : Custom y-axis ticks (e.g., [2, 10, 20, 40, 80])

Layout Options

- `layout_kwargs::@NamedTuple{} = NamedTuple()` : Additional parameters to configure grid or topographical layouts (e.g., grid*dims, topo*plot*width, grid*rowgap)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can

> pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Examples

```
plot_tf("tf_morlet_result.jld2")
plot_tf("tf_morlet_result.jld2"; layout=:grid,
channel_selection=channels([:Cz, :Pz]))
plot_tf(tf; colormap=:RdBu, yscale=:log10, baseline_interval=(-0.3, 0.0))
```

source

`EegFun.plot_tf_stats` – Method.

```
plot_tf_stats(result::TFStatsResult;
              channel_selection = channels(),
              channel_plot_order::Union{Nothing, AbstractVector} = nothing,
              content::Symbol = :tvalues,
              mask_style::Symbol = :alpha,
              colormap = nothing,
              colorrange = nothing,
              xscale::Symbol = :linear,
              yscale::Symbol = :linear,
              colorbar_plot::Bool = true,
              plot_title::Union{Nothing, AbstractString} = nothing,
              show_title::Bool = true,
              significance_color = :black,
              significance_linewidth::Real = 2.0,
              figure_size = nothing,
              display_plot::Bool = true)
```

Plot TF statistical results as heatmaps with significance overlays.

Works with both `TFClusterPermutationResult` (from `permutation_test`) and `TFAnalyticResult` (from `analytic_test`).

Arguments

- `result::TFStatsResult` : Statistical result to plot
- `channel_selection` : Channel selection predicate (default: all channels)
- `content::Symbol` : What to plot as the heatmap:
  - `:tvalues` (default) - t-statistic values
  - `:difference` - power difference (condition A - B)
  - `:power_a` - grand average power for condition A
  - `:power_b` - grand average power for condition B
- `mask_style::Symbol` : How to visualize significant regions:
  - `:alpha` (default) - dim non-significant regions
  - `:contour` - black contour lines around significant regions

  - `:hide` - hide non-significant regions
  - `:none` - no significance overlay
- `colormap` : Colormap (default: `:RdBu`)
- `colorrange` : Color range tuple or `nothing` for auto (auto-symmetric for t-values)
- `xscale::Symbol` : Time (x) axis scale (`:linear` or `:log10` , default: `:linear`)
- `yscale::Symbol` : Frequency (y) axis scale (`:linear` or `:log10` , default: `:linear`)
- `colorbar_plot::Bool` : Show colorbar (default: `true`)
- `plot_title` : Optional header title override (default: `nothing`)
- `show_title::Bool` : Show header and channel titles (default: `true`)
- `significance_color` : Color for contour lines (default: `:black`)
- `significance_linewidth::Real` : Width of contour lines (default: 2.0)
- `figure_size` : Figure size tuple or `nothing` for auto
- `display_plot::Bool` : Display the plot (default: true)

Returns

Named tuple `(fig, axes)` with Makie Figure and vector of Axes.

Examples

```
# Basic t-value heatmap with contour significance
result = permutation_test(prepared; n_permutations=1000,
cluster_type=:temporal)
plot_tf_stats(result, channel_selection=channels(:Cz))

# Power difference with hide mask
plot_tf_stats(result, content=:difference, mask_style=:hide)

# Multiple channels in grid
plot_tf_stats(result, channel_selection=channels([:Cz, :Fz, :Pz, :Oz]))

# With analytic test results
result_analytic = analytic_test(prepared)
plot_tf_stats(result_analytic, mask_style=:alpha, colormap=:viridis)
```

source

`EegFun.plot_topography!` – Method.

```
plot_topography!(fig, ax, dat::SingleDataFrameEeg; kwargs...)
plot_topography!(fig, ax, dat::MultiDataFrameEeg, epoch::Int; kwargs...)
```

In-place topographic plot: subset `dat` and render onto an existing `Figure` and `Axis` .

source

`EegFun.plot_topography` – Method.

```
plot_topography(ica::InfoIca; component_selection = components(), kwargs...)
-> NamedTuple
```

Plot ICA component scalp topographies in a grid layout.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = ""` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the topography
- `colorrange::Nothing = nothing` : Color scale limits as (min, max) tuple (nothing for auto)
- `component_selection::EegFun.var"#components##0#components##1" = EegFun.var"#components##0#components##1"()` : Function that returns boolean vector for component filtering
- `dims::Nothing = nothing` : Grid dimensions (rows, cols). If nothing, calculates best square-ish grid
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `gridscale::Int64 = 75` : Grid resolution for interpolation
- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Float64 = 1.0` : Radius of the head shape in mm.
- `highlight_channels::Nothing = nothing` : Highlight channel groups as a NamedTuple or Vector of NamedTuples. Each group: (channels=[:Cz, :Pz], color=:white, size=8, marker=:circle)
- `interactive::Bool = true` : Whether to enable interactive features
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `method::Symbol = :thin_plate` : Interpolation method: :multiquadratic, :inverse*multiquadratic, :gaussian, :inverse*quadratic, :thin*plate, :polyharmonic, :shepard,* See https://eljungsk.github.io/ScatteredInterpolation.jl/dev/methods/ for details on methods.

- `num_levels::Int64 = 20` : Number of contour levels (for ICA plots).
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for display labels (:s or :ms). Only affects title strings — all intervals and selections remain in seconds.
- `use_global_scale::Bool = false` : Do multiple ICA topoplots share the same color scale based on min/max across all components?
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_topography` – Method.

```
plot_topography(tf::TimeFreqData;
                freq_range::Tuple{Real, Real},
                interval_selection::Interval = times(),
                baseline_interval = nothing,
                baseline_method::Symbol = :db,
                kwargs...)
```

Plot a topographic map of average power within a frequency band and time window.

Arguments

- `tf::TimeFreqData` : Time-frequency data
- `freq_range::Tuple{Real, Real}` : Frequency range to average over (e.g., `(8.0, 12.0)` for alpha)
- `interval_selection::Interval` : Time window (default: all time points)
- `baseline_interval` : Baseline window as `(start, stop)` in seconds (e.g., `(-0.3, 0.0)`). Default: `nothing`

- `baseline_method::Symbol` : Baseline method (default: `:db`).
  Options: `:db` , `:absolute` , `:relative` , `:relchange` , `:percent` , `:zscore`

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = ""` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the topography
- `colorrange::Nothing = nothing` : Color scale limits as (min, max) tuple (nothing for auto)
- `component_selection::EegFun.var"#components##0#components##1" = EegFun.var"#components##0#components##1"()` : Function that returns boolean vector for component filtering
- `dims::Nothing = nothing` : Grid dimensions (rows, cols). If nothing, calculates best square-ish grid
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `gridscale::Int64 = 75` : Grid resolution for interpolation
- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Float64 = 1.0` : Radius of the head shape in mm.
- `highlight_channels::Nothing = nothing` : Highlight channel groups as a NamedTuple or Vector of NamedTuples. Each group: (channels=[:Cz, :Pz], color=:white, size=8, marker=:circle)
- `interactive::Bool = true` : Whether to enable interactive features
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.

- `method::Symbol = :thin_plate` : Interpolation method: :multiquadratic, :inverse*multiquadratic, :gaussian, :inverse*quadratic, :thin*plate, :polyharmonic, :shepard,* See https://eljungsk.github.io/ScatteredInterpolation.jl/dev/methods/ for details on methods.
- `num_levels::Int64 = 20` : Number of contour levels (for ICA plots).
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for display labels (:s or :ms). Only affects title strings — all intervals and selections remain in seconds.
- `use_global_scale::Bool = false` : Do multiple ICA topoplots share the same color scale based on min/max across all components?
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Returns

- Named tuple `(fig, axes, ax)`

Examples

```
# Alpha-band power (8-12 Hz) in 300-500ms window with dB baseline
plot_topography(tf, freq_range=(8.0, 12.0), interval_selection=times(0.3,
0.5),
                baseline_interval=(-0.3, 0.0))

# Theta-band power across full time window
plot_topography(tf, freq_range=(4.0, 7.0))
```

source

`EegFun.plot_topography` – Method.

```
plot_topography(filepath::String; input_dir = pwd(), participant_selection =
participants(), kwargs...)
plot_topography(dat::SingleDataFrameEeg; channel_selection = channels(),
sample_selection = samples(), interval_selection = times(), baseline_interval
= nothing, n_topo = nothing, scrubber = false, display_plot = true,
interactive = true, kwargs...)
plot_topography(dat::AbstractVector{<:SingleDataFrameEeg}; kwargs...)
plot_topography(dat::AbstractVector{<:EpochData}; kwargs...)
plot_topography(dat::MultiDataFrameEeg, epoch::Int; kwargs...)
```

Plot 2D scalp topographies for ERP, continuous, or epoched EEG data (or `.jld2` files/patterns).

Supports multi-bin time expansion via `n_topo` , interactive playback via `scrubber = true` , and Shift+drag spatial channel selection with right-click ERP plotting.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = ""` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the topography
- `colorrange::Nothing = nothing` : Color scale limits as (min, max) tuple (nothing for auto)
- `component_selection::EegFun.var"#components##0#components##1" = EegFun.var"#components##0#components##1"()` : Function that returns boolean vector for component filtering
- `dims::Nothing = nothing` : Grid dimensions (rows, cols). If nothing, calculates best square-ish grid
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `gridscale::Int64 = 75` : Grid resolution for interpolation
- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Float64 = 1.0` : Radius of the head shape in mm.

- `highlight_channels::Nothing = nothing` : Highlight channel groups as a NamedTuple or Vector of NamedTuples. Each group: (channels=[:Cz, :Pz], color=:white, size=8, marker=:circle)
- `interactive::Bool = true` : Whether to enable interactive features
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `method::Symbol = :thin_plate` : Interpolation method: :multiquadratic, :inverse*multiquadratic, :gaussian, :inverse*quadratic, :thin*plate, :polyharmonic, :shepard,* See https://eljungsk.github.io/ScatteredInterpolation.jl/dev/methods/ for details on methods.
- `num_levels::Int64 = 20` : Number of contour levels (for ICA plots).
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for display labels (:s or :ms). Only affects title strings — all intervals and selections remain in seconds.
- `use_global_scale::Bool = false` : Do multiple ICA topoplots share the same color scale based on min/max across all components?
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_topography_3d` – Method.

```
plot_topography_3d(dat::SingleDataFrameEeg; scrubber::Bool = false,
kwargs...)
```

```
plot_topography_3d(dat::MultiDataFrameEeg, epoch::Int; scrubber::Bool = false,
kwargs...)
```

Plot a smooth 3D interpolation of EEG data mapped onto a realistically proportioned head mesh. When `scrubber = true`, opens an interactive time scrubber with slider, wall-clock playback, and amplitude scaling controls.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `camera_azimuth::Nothing = nothing` : Initial camera azimuth angle (degrees) for 3D plots. (nothing for auto)
- `camera_elevation::Nothing = nothing` : Initial camera elevation angle (degrees) for 3D plots. (nothing for auto)
- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = ""` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the topography
- `colorrange::Nothing = nothing` : Color scale limits as (min, max) tuple (nothing for auto)
- `component_selection::EegFun.var"#components##0#components##1" = EegFun.var"#components##0#components##1"()` : Function that returns boolean vector for component filtering
- `dims::Nothing = nothing` : Grid dimensions (rows, cols). If nothing, calculates best square-ish grid
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `gridscale::Int64 = 75` : Grid resolution for interpolation
- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Float64 = 1.0` : Radius of the head shape in mm.
- `highlight_channels::Nothing = nothing` : Highlight channel groups as a NamedTuple or Vector of NamedTuples. Each group: (channels=[:Cz, :Pz], color=:white, size=8, marker=:circle)
- `interactive::Bool = true` : Whether to enable interactive features

- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `method::Symbol = :thin_plate` : Interpolation method: :multiquadratic, :inverse*multiquadratic, :gaussian, :inverse*quadratic, :thin*plate, :polyharmonic, :shepard,* See https://eljungsk.github.io/ScatteredInterpolation.jl/dev/methods/ for details on methods.
- `num_levels::Int64 = 20` : Number of contour levels (for ICA plots).
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for display labels (:s or :ms). Only affects title strings — all intervals and selections remain in seconds.
- `use_global_scale::Bool = false` : Do multiple ICA topoplots share the same color scale based on min/max across all components?
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_topography_stats` – Method.

```
plot_topography_stats(result::StatsResult;
                      n_topos::Int = 10,
                      interval_selection::Interval = times(),
                      topo_data::Symbol = :tvalues,
                      highlight_significant::Bool = true,
                      highlight_color = :white,
                      highlight_marker::Symbol = :circle,
```

```
                        highlight_size::Real = 8,
                        highlight_threshold::Real = 0.5,
                        kwargs...)
```

Plot a grid of topographic maps showing statistical results across time windows.

Each panel displays t-statistics (`:tvalues`) or difference amplitudes (`:difference`) averaged within a time bin, with significant channels highlighted as markers.

Arguments

- `result::StatsResult` : Result from `analytic_test` or `permutation_test` .
- `n_topos::Int` : Number of topographic panels (default: `10`).
- `interval_selection::Interval` : Time window to display (default: `times()`).
- `topo_data::Symbol` : `:tvalues` (default) or `:difference` .
- `highlight_significant::Bool` : Overlay markers on significant channels (default: `true`).
- `highlight_color` : Marker color for significant channels (default: `:white`).
- `highlight_marker::Symbol` : Marker shape for significant channels (default: `:circle`).
- `highlight_size::Real` : Marker size for significant channels (default: `8`).
- `highlight_threshold::Real` : Minimum significant time-point fraction in a bin to highlight a channel (default: `0.5`).

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = ""` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar
- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the topography
- `colorrange::Nothing = nothing` : Color scale limits as (min, max) tuple (nothing for auto)
- `component_selection::EegFun.var"#components##0#components##1" = EegFun.var"#components##0#components##1"()` : Function that returns boolean vector for component filtering
- `dims::Nothing = nothing` : Grid dimensions (rows, cols). If nothing, calculates best square-ish grid
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas

- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `gridscale::Int64 = 75` : Grid resolution for interpolation
- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Float64 = 1.0` : Radius of the head shape in mm.
- `highlight_channels::Nothing = nothing` : Highlight channel groups as a NamedTuple or Vector of NamedTuples. Each group: (channels=[:Cz, :Pz], color=:white, size=8, marker=:circle)
- `interactive::Bool = true` : Whether to enable interactive features
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `method::Symbol = :thin_plate` : Interpolation method: :multiquadratic, :inverse*multiquadratic, :gaussian, :inverse*quadratic, :thin*plate, :polyharmonic, :shepard,* See https://eljungsk.github.io/ScatteredInterpolation.jl/dev/methods/ for details on methods.
- `num_levels::Int64 = 20` : Number of contour levels (for ICA plots).
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.
- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for display labels (:s or :ms). Only affects title strings — all intervals and selections remain in seconds.
- `use_global_scale::Bool = false` : Do multiple ICA topoplots share the same color scale based on min/max across all components?
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Returns

- Named tuple `(fig, axes, colorbar)`

source

`EegFun.plot_topography_stats` – Method.

```
plot_topography_stats(result::TFStatsResult;
                  freq_range::Tuple{Real, Real},
                  n_topos::Int = 10,
                  interval_selection::Interval = times(),
                  topo_data::Symbol = :tvalues,
                  highlight_significant::Bool = true,
                  highlight_color = :white,
                  highlight_marker::Symbol = :circle,
                  highlight_size::Real = 8,
                  highlight_threshold::Real = 0.5,
                  kwargs...)
```

Plot a grid of topographic maps from TF statistical results across time windows for a selected frequency band.

Each panel shows the spatial distribution of t-statistics (or power difference) averaged within a time window and frequency band, with significant channels highlighted.

A channel is marked significant when the proportion of significant time×frequency bins within the panel's window meets or exceeds `highlight_threshold` .

Arguments

- `result::TFStatsResult` : TF statistical result (from `permutation_test` or `analytic_test`)
- `freq_range::Tuple{Real, Real}` : Frequency range to average over (e.g., `(8.0, 12.0)`)
- `n_topos::Int` : Number of topographic panels (default: 10)
- `interval_selection::Interval` : Time window to display (default: full range). Use `times(start, stop)` to specify
- `topo_data::Symbol` : `:tvalues` (default) or `:difference`
- `highlight_significant::Bool` : Overlay markers on significant channels (default: true)
- `highlight_threshold::Real` : Proportion of time×freq bins that must be significant to mark a channel (default: 0.5). `0.0` = any bin, `1.0` = all bins
- `highlight_color` , `highlight_marker` , `highlight_size` : Marker styling

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `colorbar_kwargs::@NamedTuple{} = NamedTuple()` : Additional kwargs passed directly to the Makie Colorbar block (see Makie's Colorbar documentation for available attributes).
- `colorbar_label::String = ""` : Label for the colorbar
- `colorbar_plot::Bool = true` : Whether to show the colorbar

- `colorbar_plot_numbers::Vector{Any} = Any[]` : Plot indices for which to show colorbars. Empty list shows colorbars for all plots.
- `colorbar_position::Symbol = :right` : Position of the colorbar (:right, :left, :top, :bottom, or tuple)
- `colormap::Nothing = nothing` : Colormap for the topography
- `colorrange::Nothing = nothing` : Color scale limits as (min, max) tuple (nothing for auto)
- `component_selection::EegFun.var"#components##0#components##1" = EegFun.var"#components##0#components##1"()` : Function that returns boolean vector for component filtering
- `dims::Nothing = nothing` : Grid dimensions (rows, cols). If nothing, calculates best square-ish grid
- `display_plot::Bool = true` : Whether to display the plot
- `figure_padding::NTuple{4, Int64} = (10, 30, 10, 10)` : Padding around entire figure as (left, right, bottom, top) tuple (in pixels)
- `figure_title::String = ""` : Title drawn at the top of the entire figure canvas
- `figure_title_fontsize::Int64 = 24` : Font size for figure title
- `gridscale::Int64 = 75` : Grid resolution for interpolation
- `head_color::Nothing = nothing` : Color of the head shape outline.
- `head_linewidth::Int64 = 2` : Line width of the head shape outline.
- `head_radius::Float64 = 1.0` : Radius of the head shape in mm.
- `highlight_channels::Nothing = nothing` : Highlight channel groups as a NamedTuple or Vector of NamedTuples. Each group: (channels=[:Cz, :Pz], color=:white, size=8, marker=:circle)
- `interactive::Bool = true` : Whether to enable interactive features
- `label_color::Nothing = nothing` : Color of electrode labels.
- `label_fontsize::Int64 = 20` : Font size for electrode labels.
- `label_plot::Bool = true` : Whether to plot electrode labels.
- `label_xoffset::Int64 = 0` : X-axis offset for electrode labels.
- `label_yoffset::Int64 = 0` : Y-axis offset for electrode labels.
- `method::Symbol = :thin_plate` : Interpolation method: :multiquadratic, :inverse*multiquadratic, :gaussian, :inverse*quadratic, :thin*plate, :polyharmonic, :shepard,* See https://eljungsk.github.io/ScatteredInterpolation.jl/dev/methods/ for details on methods.
- `num_levels::Int64 = 20` : Number of contour levels (for ICA plots).
- `plot_title::Nothing = nothing` : Plot title
- `plot_title_align::Tuple{Symbol, Symbol} = (:center, :top)` : Alignment of the inner plot title
- `plot_title_fontsize::Int64 = 16` : Font size for the title
- `plot_title_position::Nothing = nothing` : Relative (x, y) coordinates for the plot title (e.g., (0.5, 0.95)). If provided, the title is drawn inside the axis.
- `point_color::Nothing = nothing` : Color of electrode points.
- `point_marker::Symbol = :circle` : Marker style for electrode points.

- `point_markersize::Int64 = 12` : Size of electrode point markers.
- `point_plot::Bool = true` : Whether to plot electrode points.
- `show_title::Bool = true` : Whether to show the plot title
- `theme_fontsize::Nothing = nothing` : Font size for theme
- `time_unit::Symbol = :s` : Time unit for display labels (:s or :ms). Only affects title strings — all intervals and selections remain in seconds.
- `use_global_scale::Bool = false` : Do multiple ICA topoplots share the same color scale based on min/max across all components?
- `zoom_step::Float64 = 0.2` : Fractional zoom step for arrow keys (e.g. 0.2 means 20% zoom in/out)

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

Returns

- Named tuple `(fig, axes, colorbar)`

Examples

```
# Alpha-band statistics
plot_topography_stats(result, freq_range=(8.0, 12.0))

# Theta-band with more panels and lower threshold
plot_topography_stats(result, freq_range=(4.0, 7.0), n_topos=15,
highlight_threshold=0.3)

# Specific time window
plot_topography_stats(result, freq_range=(8.0, 12.0),
interval_selection=times(0.1, 0.5))
```

source

`EegFun.plot_trigger_overview` – Method.

```
plot_trigger_overview(trigger_times, trigger_values, trigger_count;
kwargs...) -> NamedTuple
plot_trigger_overview(dat::ContinuousData; kwargs...) -> NamedTuple
```

Plot trigger onset occurrences across time grouped by trigger value with a summary count legend.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `display_plot::Bool = true` : Whether to display the plot.
- `event_line_height::Float64 = 0.05` : Height of event lines.

- `event_line_width::Int64 = 1` : Line width for individual event lines.
- `font_size::Int64 = 24` : Font size for text elements.
- `ignore_triggers::Vector{Int64} = Int64[]` : Vector of trigger codes to ignore.
- `initial_position::Float64 = -2.0` : Initial position of the time interval in seconds.
- `label_fontsize::Int64 = 20` : Font size for labels.
- `line_offset::Float64 = 0.1` : Offset for trigger vertical lines.
- `linewidth_trigger::Int64 = 1` : Line width for trigger vertical lines.
- `marker_size::Int64 = 15` : Size of markers in the trigger overview plot.
- `max_window_size::Float64 = 100.0` : Maximum size of the time interval in seconds.
- `min_window_size::Float64 = 0.1` : Minimum size of the time interval in seconds.
- `plot_title::Nothing = nothing` : Custom plot title override.
- `position_step::Float64 = 0.5` : Step size for position slider.
- `show_title::Bool = true` : Whether to show the plot title.
- `time_label_offset::Float64 = -0.001` : Vertical offset for time labels.
- `timeline_width::Int64 = 2` : Line width for the main timeline.
- `window_size::Float64 = 10.0` : Size of the time window to display in seconds.
- `window_size_step::Float64 = 1.0` : Step size for window size slider.
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `y_max_limit::Float64 = 0.15` : Maximum y-axis limit.
- `y_min_limit::Float64 = -0.1` : Minimum y-axis limit.
- `ygrid::Bool = false` : Whether to show y-axis grid
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

Legend Options

- `legend::Bool = true` : Whether to show the trigger count legend.

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.plot_trigger_timing` – Method.

```
plot_trigger_timing(dat::ContinuousData; kwargs...) -> NamedTuple
```

Plot interactive trigger onset timing and inter-trigger intervals with position and window-size sliders.

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `display_plot::Bool = true` : Whether to display the plot.
- `event_line_height::Float64 = 0.05` : Height of event lines.
- `event_line_width::Int64 = 1` : Line width for individual event lines.
- `font_size::Int64 = 24` : Font size for text elements.
- `ignore_triggers::Vector{Int64} = Int64[]` : Vector of trigger codes to ignore.
- `initial_position::Float64 = -2.0` : Initial position of the time interval in seconds.
- `label_fontsize::Int64 = 20` : Font size for labels.
- `line_offset::Float64 = 0.1` : Offset for trigger vertical lines.
- `linewidth_trigger::Int64 = 1` : Line width for trigger vertical lines.
- `marker_size::Int64 = 15` : Size of markers in the trigger overview plot.
- `max_window_size::Float64 = 100.0` : Maximum size of the time interval in seconds.
- `min_window_size::Float64 = 0.1` : Minimum size of the time interval in seconds.
- `plot_title::Nothing = nothing` : Custom plot title override.
- `position_step::Float64 = 0.5` : Step size for position slider.
- `show_title::Bool = true` : Whether to show the plot title.
- `time_label_offset::Float64 = -0.001` : Vertical offset for time labels.
- `timeline_width::Int64 = 2` : Line width for the main timeline.
- `window_size::Float64 = 10.0` : Size of the time window to display in seconds.
- `window_size_step::Float64 = 1.0` : Step size for window size slider.
- `xgrid::Bool = false` : Whether to show x-axis grid
- `xminorgrid::Bool = false` : Whether to show x-axis minor grid
- `y_max_limit::Float64 = 0.15` : Maximum y-axis limit.
- `y_min_limit::Float64 = -0.1` : Minimum y-axis limit.
- `ygrid::Bool = false` : Whether to show y-axis grid
- `yminorgrid::Bool = false` : Whether to show y-axis minor grid

Legend Options

- `legend::Bool = true` : Whether to show the trigger count legend.

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

Themes & Topography Overlays

`EegFun.add_topo_rois!` – Method.

```
add_topo_rois!(ax::Axis, layout::Layout, rois::Vector{<:Vector{Symbol}};
kwargs...)
```

Draw padded convex-hull region-of-interest (ROI) outlines or fills around groups of electrodes on `ax` .

Keyword Arguments

All keyword arguments below have sensible defaults. You can override any by passing the corresponding keyword argument.

General Plot Settings

- `roi_border_size::Int64 = 10` : Size of the border around ROI points.
- `roi_fill::Bool = false` : Whether to fill the ROI area.
- `roi_fillalpha::Float64 = 0.2` : Transparency of ROI fill.
- `roi_fillcolor::Symbol = :gray` : Color of ROI fill.
- `roi_linecolor::Nothing = nothing` : Color of ROI outline.
- `roi_linewidth::Int64 = 2` : Line width of ROI outline.

> **Further Legend Options** `EegFun.jl` passes any argument prefixed with `legend_` directly to Makie's legend system. While only the most common options are listed above, you can pass **any** standard Makie legend attribute (e.g., `legend_bgcolor` , `legend_patchsize`). See the Makie.jl documentation for a full list of discoverable arguments.

source

`EegFun.theme_eegfun` – Method.

```
theme_eegfun()
```

Returns a `Makie.Theme` optimized for EEG data visualization.

Features

- Base `fontsize` = 16, `linewidth` = 2
- Default `colormap` = `:coolwarm`
- Categorical `palette` = Makie's colorblind-friendly `wong_colors`
- Clean `Axis` without gridlines

source

Interactive Teaching Demos

`EegFun.signal_example_composition` – Method.

```
signal_example_composition()
```

Interactive multi-signal composer with filtering and frequency-domain analysis.

Creates a GUI with three sine-wave generators, additive Gaussian noise, optional Butterworth low-pass and high-pass filters, waveform presets (square, sawtooth, phase scramble), and a live power spectrum.

Examples

```
using GLMakie, EegFun
EegFun.signal_example_composition()
```

Returns

- `fig::Figure` : The Makie figure object containing the interactive GUI
- `axes::Vector{Axis}` : The six axis objects (rows 1–6)

source

`EegFun.signal_example_convolution` – Method.

```
signal_example_convolution()
```

Interactive discrete convolution demo showing Gaussian, Boxcar, and Morlet wavelet kernels sliding across a noisy sinusoid and accumulating the output or amplitude envelope.

Examples

```
using GLMakie, EegFun
EegFun.signal_example_convolution()
```

Returns

- `fig::Figure` : The Makie figure object
- `axes::Tuple` : `(ax_sig, ax_out)` — the two axis objects

source

`EegFun.signal_example_decoding` – Method.

```
signal_example_decoding()
```

Interactive MVPA decoding playground demonstrating time-resolved k-fold cross-validated SVM classification of two synthetic multi-channel EEG conditions.

Examples

```
using GLMakie, EegFun
EegFun.signal_example_decoding()
```

Returns

- `fig::Figure` : The Makie figure object
- `axes::Tuple` : `(ax_erp, ax_acc)` — the two axis objects

source

`EegFun.signal_example_dotproduct` – Method.

```
signal_example_dotproduct()
```

Interactive dot-product demo illustrating how the Discrete Fourier Transform (DFT) detects frequency power via sine and complex (sine + cosine) probe sinusoids.

Examples

```
using GLMakie, EegFun
EegFun.signal_example_dotproduct()
```

Returns

- `fig::Figure` : The Makie figure object
- `axes::Tuple` : `(ax_sig, ax_prod_sin, ax_prod_cos)` — the three axis objects

source

`EegFun.signal_example_ecg_einthoven` – Method.

```
signal_example_ecg_einthoven()
```

Interactive Electrocardiography (ECG) simulator illustrating cardiac depolarization/repolarization dipoles (P, Q, R, S, T), Einthoven's triangle, Vectorcardiogram loops, and bipolar limb-lead projections (Lead I, Lead II, Lead III).

```
using GLMakie, EegFun
signal_example_ecg_einthoven()
```

Returns

- `fig::Figure` : The Makie figure object containing the interactive GUI
- `ax_torso::Axis` : Anatomical torso, Einthoven's triangle, cardiac dipole & Vectorcardiogram axis
- `ax_ecg::Axis` : Calibrated ECG strip axis

source

`EegFun.signal_example_ecg_hrv` – Method.

```
signal_example_ecg_hrv()
```

Interactive Heart Rate Variability (HRV) simulator illustrating respiratory sinus arrhythmia (breathing-coupled R–R interval modulation) across relaxed (high vagal tone) and stressed states.

```
using GLMakie, EegFun
EegFun.signal_example_ecg_hrv()
```

Returns

- `fig::Figure` : Makie figure object
- `ax_ecg::Axis` : ECG strip with beat-to-beat intervals
- `ax_hr::Axis` : Heart rate and breathing synchrony plot

source

`EegFun.signal_example_emg` – Method.

```
signal_example_emg()
```

Interactive Surface Electromyography (sEMG) simulator illustrating motor unit recruitment (Henneman's size principle), bipolar interference recording, and the 4-stage processing cascade: **Raw → Rectify → Filter (RMS Envelope) → Integrate (iEMG)**.

```
using GLMakie, EegFun
signal_example_emg()
```

Returns

- `fig::Figure` : The Makie figure object containing the interactive GUI
- `ax_anatomy::Axis` : The anatomical muscle, motor units & bipolar amplifier schematic axis
- `ax_raw::Axis` : The raw surface EMG interference pattern axis
- `ax_rect::Axis` : The full-wave rectified EMG axis
- `ax_filt::Axis` : The linear envelope / filtered EMG axis
- `ax_integ::Axis` : The cumulative integrated EMG (iEMG) axis

source

`EegFun.signal_example_heog` – Method.

```
signal_example_heog(; kwargs...)
```

Interactive Horizontal Electrooculography (hEOG) simulator illustrating corneo-retinal standing dipoles, volume-conducted electric fields, common-mode rejection (CMR), and bipolar outer-canthi derivation ($V_{\text{out}} = V_{(+)} - V_{(-)}$).

Example

```
using GLMakie, EegFun
signal_example_heog()
```

Returns

- `fig::Figure` : The Makie figure object containing the interactive GUI
- `ax_eyes::Axis` : The anatomical dipole & circuit schematic axis
- `ax_eog::Axis` : The potential vs. time signal recording axis

source

`EegFun.signal_example_ica_clt` – Method.

```
signal_example_ica_clt()
```

Interactive Central Limit Theorem (CLT) demonstration for Independent Component Analysis (ICA).

Mixes up to 50 independent super-Gaussian (Laplace) sources to show how their sum converges toward a Gaussian distribution (excess kurtosis $\to 0$), illustrating why ICA recovers sources by maximizing non-Gaussianity.

Examples

```
using GLMakie, EegFun
EegFun.signal_example_ica_clt()
```

source

`EegFun.signal_example_ica_geometry` – Method.

```
signal_example_ica_geometry()
```

Interactive 3-source / 3-sensor ICA blind source separation demo illustrating pairwise 3D rotations, joint-distribution scatter geometry, excess kurtosis, and Infomax / Extended Infomax unmixing.

See Also

- `signal_example_ica_mixture` , `signal_example_ica_separation` , `signal_example_ica_clt` , `signal_example_ica_sphering` , `signal_example_ica_optimization`

Examples

```
using GLMakie, EegFun
EegFun.signal_example_ica_geometry()
```

Returns

- `fig::Figure` : The Makie figure object

source

`EegFun.signal_example_ica_mixture` – Method.

```
signal_example_ica_mixture()
```

Interactive demonstration of linear signal mixing in EEG, showing how a single scalp electrode records a weighted sum of an underlying brain oscillation and a blink artifact.

See Also

- `signal_example_ica_separation` , `signal_example_ica_geometry` , `signal_example_ica_clt` , `signal_example_ica_sphering` , `signal_example_ica_optimization`

Examples

```
using GLMakie, EegFun
EegFun.signal_example_ica_mixture()
```

source

`EegFun.signal_example_ica_optimization` – Method.

```
signal_example_ica_optimization()
```

Interactive ICA optimization demo visualizing the sphered 2D search space, projected 1D density, kurtosis objective landscape, and single-step / convergent gradient ascent.

See Also

- `signal_example_ica_mixture` , `signal_example_ica_separation` , `signal_example_ica_geometry` , `signal_example_ica_sphering`

Examples

```
using GLMakie, EegFun
EegFun.signal_example_ica_optimization()
```

Returns

- `fig::Figure` : The Makie figure object

source

`EegFun.signal_example_ica_separation` – Method.

```
signal_example_ica_separation()
```

Interactive 2-source / 2-electrode ICA separation demo illustrating forward mixing matrices, manual subtraction weights, and automatic recovery via Infomax ICA.

See Also

- `signal_example_ica_mixture` , `signal_example_ica_geometry` , `signal_example_ica_clt` , `signal_example_ica_sphering` , `signal_example_ica_optimization`

Examples

```
using GLMakie, EegFun
EegFun.signal_example_ica_separation()
```

Returns

- `fig::Figure` : The Makie figure object

source

`EegFun.signal_example_ica_sphering` – Method.

```
signal_example_ica_sphering()
```

Interactive ICA preprocessing demo showing how centering, PCA rotation, and variance scaling (whitening/sphering) transform a correlated mixture into a rotationally symmetric search space.

See Also

- `signal_example_ica_geometry` , `signal_example_ica_optimization`

Examples

```
using GLMakie, EegFun
EegFun.signal_example_ica_sphering()
```

source

`EegFun.signal_example_sampling` – Method.

```
signal_example_sampling()
```

Interactive Nyquist–Shannon sampling theorem demonstration.

Displays a continuous sine wave alongside optional linear (join-the-dots) and ideal Whittaker–Shannon `sinc` reconstructions as duration, frequency, phase, and sample rate vary.

Example

```
using GLMakie, EegFun
EegFun.signal_example_sampling()
```

Returns

- `fig::Figure` : The Makie figure object containing the interactive GUI
- `ax::Axis` : The axis object containing the plots

source

`EegFun.signal_example_tf` – Method.

```
signal_example_tf()
```

Interactive time-frequency analysis demo comparing Morlet wavelets, STFT (Hanning), and Multitaper (DPSS) decomposition across up to three synthetic oscillatory components, with live resolution, noise, frequency-scale, and baseline-correction controls.

Examples

```
using GLMakie, EegFun
EegFun.signal_example_tf()
```

Returns

- `fig::Figure` : The Makie figure object
- `axes::NamedTuple` : Named tuple with `:time` and `:tf` axes

source

`EegFun.simulate_erp` – Method.

```
simulate_erp()
```

Interactive ERP simulator for teaching signal averaging and component superposition.

Generates synthetic EEG trials from up to 5 cosine-shaped ERP components (with controllable frequency, amplitude, latency, and trial-to-trial jitter) plus 1/f background noise, displaying individual trials (or a 95% confidence interval) alongside the trial-averaged ERP.

Examples

```
using GLMakie, EegFun
EegFun.simulate_erp()
```

Returns

- `fig::Figure` : The Makie figure object
- `ax::Axis` : The main plot axis

source

Chapter 39

Types

39.1 Core Types

Data structures and type definitions in `EegFun.jl` .

Abstract & Utility Types

`EegFun.EegFunData` – Type.

```
EegFunData
```

Abstract supertype for all primary EegFun data and result structures.

source

`EegFun.EegData` – Type.

```
EegData <: EegFunData
```

Abstract supertype for EEG data containers (continuous, epoched, ERP, spectrum, and time-frequency).

source

`EegFun.SingleDataFrameEeg` – Type.

```
SingleDataFrameEeg <: EegData
```

Abstract supertype for EEG data stored in a single `DataFrame` (e.g., `ContinuousData` , `ErpData` , `SpectrumData` , `TimeFreqData`).

source

`EegFun.MultiDataFrameEeg` – Type.

```
MultiDataFrameEeg <: EegData
```

Abstract supertype for trial-level EEG data stored across a `Vector{DataFrame}` (e.g., `EpochData` , `TimeFreqEpochData`).

source

`EegFun.EegFunError` – Type.

```
EegFunError <: Exception
```

User-facing exception raised by `@minimal_error` for invalid inputs, files, selections, or configurations.

source

`EegFun.Interval` – Type.

```
Interval
```

Type alias for time/sample interval specifications: `TimeSelection` (via `times(start, stop)`), `AllSelection`, `(start, stop)` tuples, numeric vectors, sample predicate `Function`s, or `nothing`.

source

Data Container & Metadata Types

`EegFun.AnalysisInfo` – Type.

```
AnalysisInfo
```

Preprocessing and analysis metadata attached to EEG data containers.

Fields

- `reference::Symbol = :none` : Reference scheme (`:avg`, `:mastoid`, `:none`, or channel label)
- `hp_filter::Float64 = 0.0` : High-pass cutoff in Hz (`0.0` if unfiltered)
- `lp_filter::Float64 = 0.0` : Low-pass cutoff in Hz (`0.0` if unfiltered)
- `sample_rate::Float64 = 0.0` : Sample rate in Hz
- `n_ica_components_removed::Int = 0` : Number of ICA components removed
- `n_channels_repaired::Int = 0` : Number of channels repaired
- `repaired_channels::Vector{Symbol} = Symbol[]` : Labels of repaired channels

source

`EegFun.BaselineInfo` – Type.

```
BaselineInfo
```

Baseline correction parameters applied to time-frequency data (`method` and `(start, stop)` `window` in seconds).

source

`EegFun.FilterInfo` – Type.

```
FilterInfo
```

Metadata and DSP filter object for a configured digital EEG filter.

Fields

- `filter_type::Symbol` : `:hp` (highpass) or `:lp` (lowpass)
- `filter_object::Union{ZeroPoleGain,Vector{Float64}}` : `ZeroPoleGain` for IIR or tap vector for FIR
- `filter_method::Symbol` : `:iir` or `:fir`
- `cutoff_freq::Float64` : Cutoff frequency in Hz
- `sample_rate::Float64` : Sampling rate in Hz
- `order::Int` : Filter order (for IIR filters)
- `n_taps::Union{Nothing,Int}` : Tap count for FIR filters (`nothing` for IIR)
- `transition_band::Float64` : Transition band width in Hz

source

`EegFun.FilterSection` – Type.

```
FilterSection
```

Settings for a single high-pass or low-pass filter stage (`apply` , `type` , `freq` , `func` , `method` , `order`).

source

`EegFun.TriggerInfo` – Type.

```
TriggerInfo
```

Wrapper around a trigger-count `DataFrame` for formatted display.

source

`EegFun.ContinuousData` – Type.

```
ContinuousData <: SingleDataFrameEeg
```

Continuous EEG recording with layout and preprocessing metadata.

Fields

- `file::String` : Source filename
- `data::DataFrame` : Continuous samples (`:time` , metadata, channel columns, and optional extra columns)
- `layout::Layout` : Electrode layout
- `sample_rate::Int64` : Sample rate in Hz
- `analysis_info::AnalysisInfo` : Preprocessing metadata

source

`EegFun.ErpData` – Type.

```
ErpData <: SingleDataFrameEeg
```

Condition-averaged event-related potential (ERP) data.

Fields

- `file::String` : Source filename
- `condition::Int64` : Condition number
- `condition_name::String` : Condition label
- `data::DataFrame` : Averaged ERP samples
- `layout::Layout` : Electrode layout
- `sample_rate::Int64` : Sample rate in Hz
- `analysis_info::AnalysisInfo` : Preprocessing metadata
- `n_epochs::Int64` : Number of epochs averaged

source

`EegFun.EpochData` – Type.

```
EpochData <: MultiDataFrameEeg
```

Single-trial epoched EEG data for one experimental condition.

Fields

- `file::String` : Source filename
- `condition::Int64` : Condition number
- `condition_name::String` : Condition label
- `data::Vector{DataFrame}` : Per-epoch `DataFrame` s
- `layout::Layout` : Electrode layout
- `sample_rate::Int64` : Sample rate in Hz
- `analysis_info::AnalysisInfo` : Preprocessing metadata

source

`EegFun.TimeFreqData` – Type.

```
TimeFreqData <: SingleDataFrameEeg
```

Condition-averaged time-frequency power and phase data. Accessing `.data` aliases `.data_power` .

Fields

- `file::String` : Source filename
- `condition::Int64` : Condition number
- `condition_name::String` : Condition label
- `data_power::DataFrame` : Power values (`:time` , `:freq` , and channel columns)
- `data_phase::DataFrame` : Phase values in radians (`:time` , `:freq` , and channel columns)
- `layout::Layout` : Electrode layout
- `sample_rate::Int64` : Original sample rate in Hz
- `method::Symbol` : Spectral estimation method
- `baseline::Union{BaselineInfo,Nothing}` : Applied baseline correction, if any
- `analysis_info::AnalysisInfo` : Preprocessing metadata

source

`EegFun.TimeFreqEpochData` – Type.

```
TimeFreqEpochData <: MultiDataFrameEeg
```

Single-trial time-frequency power and phase data. Accessing `.data` aliases `.data_power` .

Fields

- `file::String` : Source filename
- `condition::Int64` : Condition number
- `condition_name::String` : Condition label
- `data_power::Vector{DataFrame}` : Per-trial power `DataFrame` s
- `data_phase::Vector{DataFrame}` : Per-trial phase `DataFrame` s (radians)
- `layout::Layout` : Electrode layout

- `sample_rate::Int64` : Original sample rate in Hz
- `method::Symbol` : Spectral estimation method
- `baseline::Union{BaselineInfo,Nothing}` : Applied baseline correction, if any
- `analysis_info::AnalysisInfo` : Preprocessing metadata

source

`EegFun.SpectrumData` – Type.

```
SpectrumData <: SingleDataFrameEeg
```

Frequency-domain power spectral density data (`:freq` and electrode channel columns).

Fields

- `file::String` : Source filename
- `condition::Int64` : Condition number
- `condition_name::String` : Condition label
- `data::DataFrame` : Power spectrum values (μV²/Hz)
- `layout::Layout` : Electrode layout
- `sample_rate::Int64` : Original sample rate in Hz
- `method::Symbol` : Spectral estimation method (`:welch` , `:multitaper` , etc.)
- `analysis_info::AnalysisInfo` : Preprocessing metadata

source

Layout & Plotting Types

`EegFun.Layout` – Type.

```
Layout
```

Electrode positions (polar, 2D, and/or 3D Cartesian) and spatial neighbor graph.

Fields

- `data::DataFrame` : Electrode coordinate table (`:label` , `:inc` , `:azi` , and optional Cartesian columns)
- `neighbors::Union{Nothing,Dict{Symbol,Neighbors}}` : Neighbor graph keyed by electrode label
- `criterion::Union{Nothing,Float64}` : Distance criterion used to compute neighbors
- `criterion_type::Union{Nothing,Symbol}` : Coordinate space used for neighbors (`:xy` or `:xyz`)

source

`EegFun.Neighbors` – Type.

```
Neighbors
```

Spatial neighbor channels, distances, and interpolation weights for a single electrode.

Fields

- `channels::Vector{Symbol}` : Neighboring channel labels
- `distances::Vector{Float64}` : Distances to each neighbor in normalized coordinate units
- `weights::Vector{Float64}` : Spatial interpolation weights

source

`EegFun.PlotLayout` – Type.

```
PlotLayout
```

A struct that defines how to arrange plots in a figure (`:single` , `:grid` , `:topo` , or `:custom`).

source

Epoch & RIDE Types

`EegFun.EpochCondition` – Type.

```
EpochCondition
```

Trigger-matching and timing rules defining an experimental condition for epoch extraction.

Fields

- `name::String` : Condition name
- `trigger_sequences::Vector{Vector{Any}}` : Trigger sequence(s) to match (integers, `:any` , ranges, or sets)
- `reference_index::Int = 1` : 1-based position within the sequence aligned to `t = 0`
- `timing_pairs::Union{Nothing,Vector{Tuple{Int,Int}}} = nothing` : Sequence index pairs constrained by `min_interval` / `max_interval`
- `min_interval::Union{Nothing,Float64} = nothing` : Minimum inter-trigger interval in seconds
- `max_interval::Union{Nothing,Float64} = nothing` : Maximum inter-trigger interval in seconds
- `mask_before_trigger::Union{Nothing,Int} = nothing` : Only match sequences before this trigger code
- `mask_after_trigger::Union{Nothing,Int} = nothing` : Only match sequences after this trigger code
- `mask_triggers::Union{Nothing,Vector{Int}} = nothing` : Trigger codes defining search boundaries
- `mask_between_triggers::Union{Nothing,Vector{Tuple{Int,Int}}} = nothing` : Trigger code pairs bounding valid search regions

source

`EegFun.RideComponent` – Type.

```
RideComponent
```

Specification for a single RIDE component (`:S` , `:C` , `:R` , etc.).

Fields

- `name::Symbol` : Component name (e.g. `:S` , `:C` , `:R`).
- `latencies::Union{Vector{Int}, Nothing}` : Known trial sample latencies (centered around their median by `ride_decompose`), or `nothing` when latencies are estimated via `woody_filter` .
- `window_radius::Int` : Woody-filter search radius in samples when `latencies === nothing` .
- `time_window::Union{Nothing, Tuple{Float64, Float64}}` : Optional active component window `(t_start, t_end)` .

source

`EegFun.RideResult` – Type.

```
RideResult{T<:Real}
```

Decomposed RIDE templates (`samples × channels × components`), centered single-trial `latencies` , `latency_offsets` , residual tensor `residue` , convergence flag, and outer iteration count.

source

ICA Types

`EegFun.IcaPrms` – Type.

```
IcaPrms
```

Algorithm hyperparameters for Infomax, Picard, and AMICA ICA decomposition.

source

`EegFun.InfoIca` – Type.

```
InfoIca <: EegFunData
```

ICA decomposition result storing unmixing/mixing/sphering matrices, component variances, and layout.

Fields

- `filename::String` : Source filename
- `unmixing::Matrix{Float64}` : Unmixing matrix (`sources = unmixing * data`)
- `mixing::Matrix{Float64}` : Mixing matrix (`data = mixing * sources`)
- `sphere::Matrix{Float64}` : Sphering matrix
- `variance::Vector{Float64}` : Proportion of variance explained per component
- `scale::Float64` : Data scaling factor
- `mean::Vector{Float64}` : Per-channel mean vector
- `ica_label::Vector{Symbol}` : Component labels (`:IC1` , `:IC2` , …)
- `removed_activations::Dict{Int,Matrix{Float64}}` : Removed component activations keyed by epoch
- `layout::Layout` : Electrode layout
- `is_sub_gaussian::Vector{Bool}` : Sub-Gaussian component flags

source

`EegFun.ArtifactComponents` – Type.

```
ArtifactComponents
```

Container of identified ICA artifact component indices (`eog` , `ecg` , `line_noise` , `channel_noise` , `manual`).

source

Statistics Types

`EegFun.StatisticalData` – Type.

```
StatisticalData
```

Prepared ERP data for permutation or analytic statistical testing (`prepare_stats`).

Fields

- `data::Vector{ErpData}` : Grand-average ERPs for conditions 1 and 2
- `analysis::AnalysisData` : Design, 3D participant arrays `[participants × electrodes × time]` , and analysis time points
- `se_cond1::Array{Float64,2}` : Condition 1 standard error `[electrodes × time]`
- `se_cond2::Array{Float64,2}` : Condition 2 standard error `[electrodes × time]`
- `se_diff::Array{Float64,2}` : Mean-difference standard error `[electrodes × time]`

source

`EegFun.StatsResult` – Type.

```
StatsResult <: EegFunData
```

Abstract supertype for ERP and time-frequency statistical test results.

source

`EegFun.PermutationResult` – Type.

```
PermutationResult <: StatsResult
```

Cluster-based permutation test results on ERP data (`permutation_test`).

Fields

- `test_info::TestInfo` : Test design, degrees of freedom, alpha, tail, and `ClusterInfo`
- `data::Vector{ErpData}` : Grand-average ERPs for conditions 1 and 2
- `stat_matrix::StatMatrix` : T-statistic matrix `[electrodes × time]`
- `masks::Masks` : Significant positive and negative masks `[electrodes × time]`
- `clusters::Clusters` : Detected positive and negative clusters
- `permutation_distribution::PermutationDistribution` : Null distribution of maximum cluster statistics
- `electrodes::Vector{Symbol}` : Electrode labels

- `time_points::Vector{Float64}` : Analysis time points in seconds
- `critical_t::Union{Array{Float64,2},Tuple{Float64,Float64},Tuple{Array{Float64,2},Array{Float6` Cluster-forming critical t-values
- `se::Array{Float64,2}` : Mean-difference standard error over the analysis interval
- `se_cond1::Array{Float64,2}` : Condition 1 standard error over the display interval
- `se_cond2::Array{Float64,2}` : Condition 2 standard error over the display interval
- `se_diff::Array{Float64,2}` : Mean-difference standard error over the display interval

source

`EegFun.AnalyticResult` – Type.

```
AnalyticResult <: StatsResult
```

Parametric point-wise t-test results on ERP data (`analytic_test`).

Fields

- `test_info::TestInfo` : Test design, degrees of freedom, alpha, tail, and multiple-comparison correction
- `data::Vector{ErpData}` : Grand-average ERPs for conditions 1 and 2
- `stat_matrix::StatMatrix` : T-statistic and p-value matrices `[electrodes × time]`
- `masks::Masks` : Significant positive and negative masks `[electrodes × time]`
- `electrodes::Vector{Symbol}` : Electrode labels
- `time_points::Vector{Float64}` : Analysis time points in seconds
- `critical_t::Float64` : Critical t-threshold
- `se::Array{Float64,2}` : Mean-difference standard error over the analysis interval
- `se_cond1::Array{Float64,2}` : Condition 1 standard error over the display interval
- `se_cond2::Array{Float64,2}` : Condition 2 standard error over the display interval
- `se_diff::Array{Float64,2}` : Mean-difference standard error over the display interval

source

`EegFun.TFStatisticalData` – Type.

```
TFStatisticalData
```

Prepared time-frequency data for permutation or analytic statistical testing (`prepare_stats`).

Fields

- `data::Vector{TimeFreqData}` : Grand-average TF data for conditions 1 and 2
- `analysis::TFAnalysisData` : Design, 4D participant arrays `[participants × electrodes × frequencies × time]` , frequencies, and time points

source

`EegFun.TFStatsResult` – Type.

```
TFStatsResult <: StatsResult
```

Abstract supertype for 3D time-frequency statistical test results (`[electrodes × frequencies × time]`).

source

`EegFun.TFClusterPermutationResult` – Type.

```
TFClusterPermutationResult <: TFStatsResult
```

Cluster-based permutation test results on 3D time-frequency data (`permutation_test`).

Fields

- `test_info::TestInfo` : Test design, degrees of freedom, alpha, tail, and `ClusterInfo`
- `data::Vector{TimeFreqData}` : Grand-average TF data for conditions 1 and 2
- `stat_matrix::TFStatMatrix` : T-statistic matrix `[electrodes × frequencies × time]`
- `masks::TFMasks` : Significant positive and negative masks `[electrodes × frequencies × time]`
- `clusters::TFClusters` : Detected positive and negative 3D TF clusters
- `permutation_distribution::PermutationDistribution` : Null distribution of maximum cluster statistics
- `electrodes::Vector{Symbol}` : Electrode labels
- `frequencies::Vector{Float64}` : Frequency bins in Hz
- `time_points::Vector{Float64}` : Time points in seconds

source

`EegFun.TFAnalyticResult` – Type.

```
TFAnalyticResult <: TFStatsResult
```

Parametric point-wise t-test results on 3D time-frequency data (`analytic_test`).

Fields

- `test_info::TestInfo` : Test design, degrees of freedom, alpha, tail, and multiple-comparison correction
- `data::Vector{TimeFreqData}` : Grand-average TF data for conditions 1 and 2
- `stat_matrix::TFStatMatrix` : T-statistic and p-value matrices `[electrodes × frequencies × time]`
- `masks::TFMasks` : Significant positive and negative masks `[electrodes × frequencies × time]`
- `electrodes::Vector{Symbol}` : Electrode labels
- `frequencies::Vector{Float64}` : Frequency bins in Hz
- `time_points::Vector{Float64}` : Time points in seconds
- `critical_t::Float64` : Critical t-threshold

source

Decoding & RSA Types

`EegFun.DecodingParameters` – Type.

```
DecodingParameters
```

Cross-validation and multi-class coding parameters for MVPA/decoding.

Fields

- `chance_level::Float64` : Theoretical chance accuracy (e.g., `0.5` for binary, `1 / n_classes` for multi-class)
- `n_iterations::Int64` : Number of cross-validation iterations
- `n_folds::Int64` : Number of cross-validation folds
- `class_coding::Symbol` : Multi-class coding scheme (`:one_vs_one` , `:one_vs_all` , `:binary`)
- `n_classes::Int64` : Number of decoded classes

source

`EegFun.DecodedData` – Type.

```
DecodedData <: EegFunData
```

Time-resolved MVPA/decoding results for a single participant.

Fields

- `file::String` : Source filename
- `condition_names::Vector{String}` : Decoded condition names
- `times::Vector{Float64}` : Decoded time points in seconds
- `average_score::Vector{Float64}` : Mean classification accuracy at each time point
- `channels::Vector{Symbol}` : Channels included as features
- `parameters::DecodingParameters` : Cross-validation parameters
- `stderror::Union{Vector{Float64},Nothing}` : Standard error of accuracy across iterations/folds
- `confusion_matrix::Union{Array{Float64,3},Nothing}` : Confusion matrices `[time × true_class × predicted_class]`
- `raw_predictions::Union{Array{Float64,4},Nothing}` : Raw predictions `[iteration × fold × time × class]`

source

`EegFun.DecodingStatisticsResult` – Type.

```
DecodingStatisticsResult
```

Stores statistical test results on multi-participant decoding accuracy (`times` , `t_statistics` , `p_values` , `significant_mask` , `df` , `alpha` , `correction_method` , `clusters`).

source

`EegFun.NoiseCeiling` – Type.

```
NoiseCeiling <: EegFunData
```

Leave-one-participant-out RSA noise ceiling estimates across time (Nili et al., 2014).

Fields

- `lower_bound::Vector{Float64}` : Conservative lower bound at each time point
- `upper_bound::Vector{Float64}` : Liberal upper bound at each time point
- `n_participants::Int` : Number of participants included

source

`EegFun.RsaData` – Type.

```
RsaData <: EegFunData
```

Time-resolved Representational Similarity Analysis (RSA) results.

Fields

- `file::String` : Source filename
- `condition_names::Vector{String}` : Analyzed condition labels
- `times::Vector{Float64}` : Time points in seconds
- `rdm::Array{Float64,3}` : Representational dissimilarity matrices `[time × condition × condition]`
- `dissimilarity_measure::Symbol` : Dissimilarity metric (`:correlation` , `:euclidean` , `:mahalanobis` , `:spearman`)
- `channels::Vector{Symbol}` : Included electrode labels
- `layout::Layout` : Electrode layout
- `sample_rate::Int64` : Sample rate in Hz
- `model_correlations::Union{Array{Float64,2},Nothing}` : Model RDM correlations `[time × model]`
- `model_names::Union{Vector{String},Nothing}` : Model names
- `p_values::Union{Array{Float64,2},Nothing}` : Model correlation p-values `[time × model]`
- `noise_ceiling::Union{NoiseCeiling,Nothing}` : Group noise ceiling estimates
- `analysis_info::AnalysisInfo` : Preprocessing metadata

source

ERP Measurement Types

`EegFun.ErpMeasurementsResult` – Type.

```
ErpMeasurementsResult <: EegFunData
```

Tabular ERP amplitude/latency measurements bundled with their `analysis_type` and time intervals.

source

`EegFun.ValidationResult` – Type.

```
ValidationResult
```

Result of configuration parameter validation (`success` , `error` , `key_path`).

source

Artifact, Rejection & Repair Types

`EegFun.ArtifactInfo` – Type.

```
ArtifactInfo <: EegFunData
```

Container for preprocessing artifact records (`continuous_repairs` , `epoch_rejections` , `ica_components`).

source

`EegFun.Rejection` – Type.

```
Rejection(channel::Symbol, epoch::Int)
```

Rejected channel-epoch pair from artifact detection.

source

`EegFun.EpochRejectionInfo` – Type.

```
EpochRejectionInfo
```

Stores epoch rejection criteria, per-channel/epoch rejections, and optional channel repair tracking (`repaired` , `skipped`).

source

`EegFun.EpochRejectionState` – Type.

```
EpochRejectionState
```

Stores the state of the epoch rejection interface.

source

`EegFun.ZScoreRejectionInfo` – Type.

```
ZScoreRejectionInfo
```

Per-measure z-score rejection lists (`:variance` , `:max` , `:min` , `:abs` , `:range` , `:kurtosis`).

source

`EegFun.ContinuousRepairInfo` – Type.

```
ContinuousRepairInfo
```

Tracks channels repaired and skipped at the continuous data level (`name` , `method` , `repaired` , `skipped`).

source

`EegFun.EpochRepairInfo` – Type.

```
EpochRepairInfo
```

Tracks channels repaired and skipped per epoch for a condition (`condition` , `condition_name` , `repaired` , `method` , `neighbors` , `skipped`).

source

`EegFun.ChannelRepairInfo` – Type.

```
ChannelRepairInfo
```

Combined container for `continuous` (`Union{ContinuousRepairInfo,Nothing}`) and `epochs` (`Vector{EpochRepairInfo}`) channel repairs.

source

`EegFun.AutoRejectConfig` – Type.

```
AutoRejectConfig
```

Adaptive cross-validated Autoreject artifact rejection and channel repair settings.

source

`EegFun.AutoRejectResult` – Type.

```
AutoRejectResult
```

Result of adaptive artifact rejection and channel repair (`autoreject` / `autoreject!`).

Fields

- `thresholds::Dict{Symbol,Float64}` : Learned peak-to-peak amplitude threshold per channel (μV).
- `mode::Symbol` : Rejection mode (`:local` or `:global`).
- `consensus_perc::Float64` : Consensus rejection fraction κ (`bad > κ * N` rejects epoch).
- `max_repair_perc::Float64` : Maximum repair fraction ρ (`bad <= ρ * N` interpolates all; middle band repairs worst `ρ * N`).
- `channels::Vector{Symbol}` : Analyzed channels.
- `ptp::Matrix{Float64}` : Peak-to-peak amplitude matrix (`n_epochs × n_channels`).
- `bad_matrix::BitMatrix` : Channel × epoch threshold violations.
- `epochs_clean::Vector{Int}` , `epochs_repaired::Vector{Int}` , `epochs_rejected::Vector{Int}` : Global epoch indices.
- `repaired_channels::OrderedDict{Int,Vector{Symbol}}` , `skipped_channels::OrderedDict{Int,Vector{Symbol}}` : Per-epoch channel tracking.
- `rejection_info::Vector{EpochRejectionInfo}` : Per-condition `EpochRejectionInfo` .
- `summary_table::DataFrame` , `channel_table::DataFrame` : Summary and per-channel tables.
- `n_folds::Int` , `n_candidates::Int` , `seed::Union{Int,Nothing}` , `repair_method::Symbol` : Run configuration.

source

Batch, Configuration & Pipeline Types

`EegFun.AnalysisSettings` – Type.

```
AnalysisSettings
```

Interactive preprocessing settings recorded from the databrowser.

Fields

- `hp_filter::Float64 = 0.0` : High-pass filter cutoff in Hz
- `lp_filter::Float64 = 0.0` : Low-pass filter cutoff in Hz
- `reference::Symbol = :none` : Reference scheme
- `repaired_channels::Vector{Symbol} = Symbol[]` : Repaired channel labels
- `repair_method::Symbol = :none` : Channel repair method
- `selected_regions::Vector{Tuple{Float64,Float64}} = Tuple{Float64,Float64}[]` : Selected time intervals in seconds
- `removed_ica_components::Vector{Int} = Int[]` : Removed ICA component indices

source

`EegFun.BatchResult` – Type.

```
BatchResult
```

Result of processing a single file in a batch operation (`success` , `filename` , `message`).

source

`EegFun.CleanLineConfig` – Type.

```
CleanLineConfig
```

Multi-taper CleanLine sinusoidal line-noise removal settings.

source

`EegFun.EegConfig` – Type.

```
EegConfig
```

EEG rereferencing and amplitude/z-score artifact rejection thresholds (μV).

source

`EegFun.EogConfig` – Type.

```
EogConfig
```

Vertical and horizontal EOG bipolar channel definitions and artifact thresholds (μV).

source

`EegFun.FilesConfig` – Type.

```
FilesConfig
```

Input and output file configuration for preprocessing pipelines.

source

`EegFun.FilterConfig` – Type.

```
FilterConfig
```

High-pass and low-pass filter settings for main preprocessing and ICA preparation.

source

`EegFun.IcaConfig` – Type.

```
IcaConfig
```

ICA decomposition and artifact component identification settings for preprocessing pipelines.

source

`EegFun.InputFilesConfig` – Type.

```
InputFilesConfig
```

Pipeline input directory, file pattern, layout file, and epoch condition file settings.

source

`EegFun.OutputFilesConfig` – Type.

```
OutputFilesConfig
```

Pipeline output directory and intermediate/final stage saving flags.

source

`EegFun.PipelineConfig` – Type.

```
PipelineConfig
```

Top-level pipeline configuration bundling `files::FilesConfig` and `preprocess::PreprocessConfig`.

source

`EegFun.PipelineTemplateOptions` – Type.

```
PipelineTemplateOptions
```

Options for `generate_pipeline_template` (`num_steps`, `subsections_per_step`, `include_usage_example`, `include_setup`, `include_summary`).

source

`EegFun.PreprocessConfig` – Type.

```
PreprocessConfig
```

Complete EEG preprocessing configuration combining filtering, CleanLine, resampling, EOG, ICA, Autoreject, and repair settings.

source

`EegFun.ResampleConfig` – Type.

```
ResampleConfig
```

Resampling settings (`apply` and `target_rate` in Hz).

source

Constants

`EegFun.VALID_MEASUREMENT_TYPES` – Constant.

Valid ERP measurement type strings.

source

`EegFun.MEASUREMENT_TYPE_LABELS` – Constant.

Human-readable labels paired with measurement type strings, for use in GUI menus.

source

See Also

- Data structures